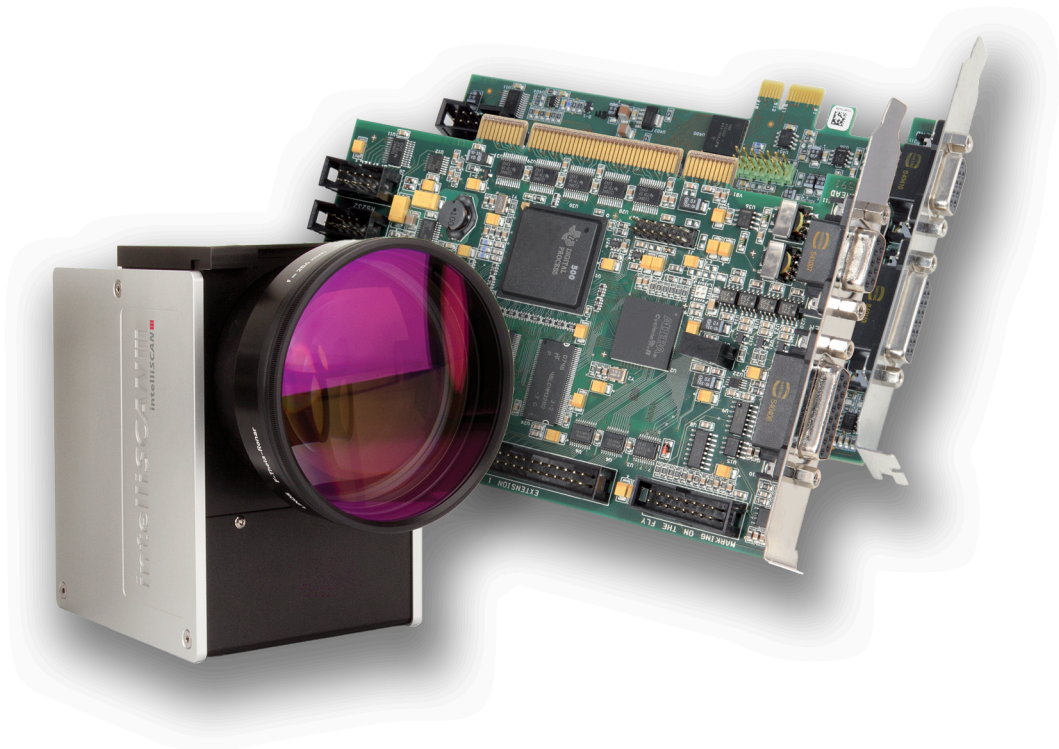




Installation and Operation

RTC5 PCI Board RTC5 PCIe Board

Real Time Control of Scan Systems and Lasers



SCANLAB GmbH
Siemensstr. 2a
82178 Puchheim
Germany

Tel. +49 (89) 800 746-0
Fax: +49 (89) 800 746-199

info@scanlab.de
www.scanlab.de

© SCANLAB GmbH

SCANLAB reserves the right to change the information in this document without notice.

No part of this manual may be processed, reproduced or distributed in any form (photocopy, print, microfilm or by any other means), electronic or mechanical, for any purpose without the written permission of SCANLAB GmbH.

All mentioned trademarks are hereby acknowledged as properties of their respective owners.



Contents

1	About this Manual	21
1.1	Manufacturer	21
1.2	Related Documents	22
1.3	Glossary and Abbreviations	23
2	Product Overview	27
2.1	Labeling	27
2.2	Unpacking Instructions and Typical Scope of Delivery	27
2.3	Delivered RTC5 Software Package	27
	Folder Correction Files	27
	Folder CorrectionFileConverter	27
	Folder DemoFiles	28
	Folder HPGL	28
	Folder iSCANConfig	28
	Folder RTC5 Driver	28
	Folder RTC5 Files	28
	Folder RTC5 Tools	29
2.4	Intended Use	30
2.5	System Requirements	32
2.5.1	Hardware	32
2.5.2	Software	32
2.6	Options	34
2.7	Jumper Settings and Type Identification	35
2.7.1	Jumper JP1 – Configuring the Output Signal Level at the EXTENSION 1 Socket Connector	36
2.7.2	Jumper JP2...JP8 – Configuring Pin15 and Pin17 at the EXTENSION 2 Socket Connector ...	36
2.8	Accessories for the RTC5 PCI Board	37
2.8.1	XY2-100 Converter	37
2.8.2	Laser Adapter	37
2.8.3	Data Cables	37
2.8.4	Slot Cover with 9-pin D-SUB Connector for 2. SCANHEAD Socket Connector	38
2.8.5	Slot Cover with 15-pin D-SUB Connector for MARKING ON THE FLY Socket Connector	38
2.8.6	Slot Cover with 15-pin D-SUB Connector and 9-pin D-SUB Connector	39
2.8.7	ADC Add-On Board	39
2.8.8	Extension Board “RTC5/6 varioSCAN FLEX Extension”	39
2.9	Supplementary Software	40
2.10	Notes for RTC4 Users	40
2.10.1	Hardware Changes	40
	Controlling Scan Systems	40
	Controlling the Laser	41
	EXTENSION 1 Socket Connector	41
	EXTENSION 2 Socket Connector	41
	MARKING ON THE FLY Socket Connector	41
	Other Hardware Interfaces	41
2.10.2	Porting RTC4 Source Code to the RTC5 PCI Board	42
	Changed Initialization	42
	Command Changes	42
	Increased Parameter Resolution	43
	Changed Timing Behavior	44
2.10.3	New and Changed Functionality	45
	Interface to the PC	45

Controlling Scan Systems	45
Controlling the Laser	46
Interfaces for Peripheral Equipment	46
General Programming	47
Laser Marking	48
Special Functions	48
3 Safety During Installation and Operation	50
3.1 Steps for Safe Operation	50
3.2 Laser Safety	50
4 RTC5 PCI Board – Layout and Interfaces	51
4.1 Layout – Upper Side	51
4.2 Layout – Lower Side	52
4.3 Interface to PC	52
4.4 Master Socket Connector, Slave Socket Connector	53
4.5 Interfaces to Scan System	54
4.5.1 Scan Head Connectors and Transfer Protocol	54
SCANHEAD Connector	54
2. SCANHEAD Socket Connector	55
4.5.2 XY2-100 Converter (Accessory)	56
4.5.3 Data Cables (Accessories)	58
4.6 Interfaces for the Laser and Peripheral Equipment	60
4.6.1 LASER Connector	60
Laser Control Signals	60
External Control Signals	61
BUSY List Execution Status	61
2-Bit Digital Input Port	61
2-Bit Digital Output Port	61
12-Bit Analog Output Port 1 and 2	61
Input/Output Wiring	61
Laser Adapter (Accessory)	63
4.6.2 EXTENSION 1 Socket Connector	65
Configuring the Output Signal Level	65
16-Bit Digital Input Port and 16-Bit Digital Output Port	65
Synchronization of Data Transmission	66
BUSY List Execution Status	66
4.6.3 EXTENSION 2 Socket Connector	67
Configuration by Solder Jumpers	67
Laser Control Signals	68
8-Bit Digital Output Port	68
4.6.4 MARKING ON THE FLY Socket Connector	69
Encoder Input Ports	69
External Control Signals	69
Analog Output Port	69
BUSY list execution status Status	69
MARKING ON THE FLY Slot Cover (Accessory)	70
4.6.5 RS232 Socket Connector	70
4.6.6 SPI / I2C Socket Connector	71
McBSP Interface	71
I ² C Interface	73
SPI Interface	73
4.6.7 STEPPER MOTOR Socket Connector	75

4.6.8	Analog Inputs (Accessory)	75
5	Installation and Start-Up	77
5.1	Checking Jumper Settings	77
5.2	Installing the RTC5 PCI Board	77
5.3	Installing the RTC5 Board Driver	78
5.3.1	RTC5 Software Package Upgrades	79
5.3.2	Exchange of RTC Boards and Update of RTC Board Driver	79
5.4	Installing the RTC5 Software	80
5.5	Safe Start-up and Shutdown Sequences	80
5.6	Functionality Test	81
5.7	User Programs and Demo Software	81
5.8	Exchanging RTC5 Boards	82
6	Developing RTC5 User Programs	83
6.1	RTC5 Software Concept Basics	83
6.1.1	Controlling Scan Systems and Lasers – An Introductory Example	83
6.1.2	Control Commands and List Commands	83
6.2	Initialization and Program Start-Up	84
6.2.1	DLL Calling Convention	84
6.2.2	Importing Commands	84
	Pascal	84
	C	85
	C++	85
	C#	85
6.2.3	Initializing the RTC5 DLL and Board Management	86
6.2.4	Start of RTC5 PCI Board Operation	87
	Initializing the Board	87
	Configuring the Board	87
	Initializing the Scan System Control	88
	Initializing the Laser Control	88
	Loading and Executing Lists	88
6.2.5	Example Code (C)	89
6.3	RTC5 List Memory	91
6.3.1	Lists and the Protected List Memory Area	91
	“List 1” and “List 2”	91
	“List 3”– Protected List Memory Area	91
6.3.2	Configuring the RTC5 List Memory	92
6.4	List Handling	94
6.4.1	Loading Lists	94
	“Unconditional” Loading	94
	Loading with Protection	95
	Terminating Lists	95
6.4.2	List Status	96
6.4.3	List Execution Status	97
6.4.4	Starting and Stopping Lists	98
6.4.5	Interrupting Lists for Synchronization of Processing	99
6.4.6	Automatic List Changing	99
	One-Time List Change	99
	Alternating List Changes	100
6.5	Structured Programming	101
6.5.1	Subroutines	101
	Non-Indexed Subroutines	101

Indexed Subroutines	102
General Information on Calling Subroutines	103
Index Management and Defragmentation	104
Subsequent Protection and Conversion of Non-Indexed Subroutines	105
Unprotecting Subroutines	106
6.5.2 Character Sets and Text Strings	107
Defining Indexed Character Sets	107
Calling Indexed Characters	108
Defining Indexed Text Strings for Times, Dates and Serial Numbers	108
Calling Indexed Text Strings	109
Managing Indexed Characters and Text Strings	109
6.5.3 Jumps	109
6.5.4 RTC4 Circular Queue Mode	110
6.5.5 Loops	111
6.6 Using Several RTC5 PCI Boards in One PC	112
6.6.1 Multi-Board Programming	112
Examples (Pascal)	112
6.6.2 Single-Board Programming	113
6.6.3 Master/Slave Operation	113
Initialization	114
Matching of Short-Command Counts	114
Synchronous Starts and Synchronous Stops	115
Example Code (Delphi)	116
6.7 Usage of RTC5 PCI Boards by Several User Programs	117
6.7.1 Board Acquisition by a User Program	117
6.8 Error Handling	119
6.8.1 Download Verification	120
6.8.2 Checking for Overruns	120
6.8.3 Example Code (C)	121
6.9 Miscellaneous	123
6.9.1 Free Variables	123
7 Basic Functions for Scan Head and Laser Control	124
7.1 Marking Dots, Lines and Arcs	124
7.1.1 Marking with Vector Commands and "Arc" Commands	124
Jump Commands	125
Mark Commands	125
Arc Commands	125
Polylines	125
Ellipse Commands	126
[*]Para[*] Commands	127
7.1.2 Microstepping	128
7.1.3 Marking Single Dots	129
7.1.4 Example Code (C)	130
7.2 Delay Settings – Coordinating Scan Head Control and Laser Control	133
7.2.1 Laser Delays	133
LaserOn Delay	134
LaserOff Delay	134
7.2.2 Scanner Delays	135
Jump Delay	135
Variable Jump Delays	136
Mark Delay	137
Polygon Delay	138

Variable Polygon Delays	139
User-defined "Variable Polygon Delays"	141
7.2.3 Notes on Optimizing the Delays	143
Recommended Optimization Sequence	143
Automatic Delay Adjustments	143
Potential Errors when Optimizing the Delays	146
7.2.4 Sky Writing	149
Sky Writing Mode 1	149
Sky Writing Mode 2	150
Sky Writing Mode 3	151
Synchronization	151
7.3 Scan Head Control	156
7.3.1 Reference System	156
7.3.2 Image Field Size and Image Field Calibration	156
Typical Image Field	157
Compatibility Modes	157
7.3.3 Virtual Image Field	158
Coordinate Transformations in the Virtual Image Field	158
7.3.4 3D Image Field	159
Carrying Out Adjustment	159
Checking the z Axis Calibration	159
Testing 3-Axis Scan Systems with F-Theta Objective	161
7.3.5 Image Field Correction and Correction Tables	162
Field Distortion	162
Image Field Correction Algorithm	163
Activating Image Field Correction	163
2D Correction Files and 3D Correction Files	163
Loading Correction Tables	164
Assigning Loaded Correction Tables	164
1to1 Correction Tables	166
Inverse Tables	166
ct5 Correction File Header	167
Converting Correction Files	169
7.3.6 Output Values to the Scan System	170
Calculation	170
Value Ranges	171
Precalculation and Diagnosis	171
Clock Overruns	171
7.3.7 Status Monitoring and Diagnostics	172
Status Information Returned from the Scan System	172
7.4 Laser Control	173
7.4.1 Enabling, Activating and Switching Laser Control Signals	173
Signals for "Laser Active" Operation	175
Signals for "Laser Standby" Operation	175
Automatic Suppression of Laser Control Signals	176
7.4.2 Configuring the LASER Connector	176
7.4.3 CO ₂ Mode	177
7.4.4 YAG Mode 1, YAG Mode 2, YAG Mode 3, YAG Mode 5	179
Q-Switch Signal	179
FirstPulseKiller Signal	179
Differences Between the YAG Modes	180
Lamp Current (Laser Power)	180

7.4.5	Laser Mode 4	182
7.4.6	Laser Mode 6	183
7.4.7	Softstart Mode	184
7.4.8	Pulse Picking Laser Mode	186
7.4.9	“Automatic Laser Control”	187
	Position-Dependent Laser Control	189
	Speed-Dependent Laser Control	191
	Encoder-Speed-Dependent Laser Control	192
	Loading and Determining the Nonlinearity Curve	192
	Vector-Defined Laser Control	194
7.4.10	Output Synchronization	195
7.5	Marking Dates, Times and Serial Numbers	196
7.5.1	Marking the Date and Time	196
7.5.2	Marking Serial Numbers	196
8	Advanced Functions for Scan Head Control and Laser Control	198
8.1	iDRIVE Functions	198
8.1.1	Transfer Protocol	198
8.1.2	Configuring the Data Signal Transmission Behavior of the Scan System	199
	Setting Data Types	199
	Reading Out Data	199
8.1.3	Monitoring the Positioning	200
8.1.4	Configuring Tuning (Dynamics Settings)	202
8.1.5	Jump Mode	202
	Functional Principle	202
	Requirements and Activation	203
	Jump-Length-Dependent Jump Delays	204
	Experimental Determination of Jump Delay Values	204
	Notes on Loading Determined Jump Delay Values	205
	Automatic Determination of the Jump Delay Table	206
8.1.6	Configuring the PosAck Limit Value	207
8.1.7	Configuring the Effective Calibration	207
8.1.8	Configuring the Start Behavior	208
8.1.9	Fault Diagnosis and Functional Test	208
8.2	Coordinate Transformations	209
8.3	Online Positioning	213
8.3.1	“Local Online Positioning”	213
	Configuring “Local Online Positioning”	214
8.3.2	“Global Online Positioning”	216
	Configuring “Global Online Positioning”	216
8.4	Wobbel Mode	217
	Example Code (C++)	218
8.4.1	Wobbel Shapes – Important Notes on Choosing Appropriate Parameter Values	219
	“Classic” Wobbel Shapes	219
	“Freely Definable Wobbel Shapes”	220
8.5	Controlling 2D Scan Systems and 3D Scan Systems	221
8.5.1	2D Scan Systems	221
8.5.2	3D Scan Systems	222
	Intended Use	222
	Connection and Initialization	222
	3D Commands	223
	Enhanced 3D Correction	225
8.5.3	Using Several Correction Tables	226

8.6	Processing-on-the-fly	227
8.6.1	Intended Use and Initialization	227
	Overview	227
8.6.2	Compensation of Linear Motions	228
	Correction via Encoder Counters	228
	Correction via McBSP Interface	230
	Correction via McBSP Interface with Additional McBSP Input	231
	Correction via McBSP Interface with Enhanced McBSP Input	231
8.6.3	Compensating Rotary Motions	232
	Correction via Encoder Counter	232
	Correction via McBSP Interface	233
	Correction via McBSP Interface with Additional McBSP Input	233
8.6.4	Compensating 2D Motions	234
	2D Encoder Compensation for xy Positioning Stages	234
8.6.5	Deactivating Processing-on-the-fly Corrections	236
8.6.6	Virtual Image Field with Processing-on-the-fly	237
8.6.7	Synchronizing Processing-on-the-fly Applications	237
8.6.8	Encoder Resets	239
8.6.9	Monitoring Processing-on-the-fly Corrections	240
	Customer-Defined Monitoring Area	241
8.6.10	Tracking Error Compensation of Encoder Values for Processing-on-the-fly Applications	242
8.6.11	Processing-on-the-fly Correction for the z Axis	242
8.7	Pixel Output Mode	244
8.7.1	Principle of Operation	244
8.7.2	RTC5 Commands	244
8.7.3	Laser Control	246
8.7.4	Synchronization	247
8.8	micro_vector[*] Commands	249
8.9	Timed Commands	250
8.10	Automatic Self-Calibration	251
	8.10.1 Using for Drift Compensation	251
	8.10.2 How it Works	251
	8.10.3 Determining Reference Values	253
	8.10.4 Calibration During the Application	253
	Automatic Self-Calibration	253
	Customer-Specific Calibration	254
	Supplemental Information about Calibration	254
8.11	Camming	255
8.12	Time Measurements	257
	8.12.1 RTC5 Timer	257
	8.12.2 Timestamps	257
9	Programming Peripheral Interfaces	258
9.1	Signal Output	258
	9.1.1 16-Bit Digital Output Port	258
	9.1.2 8-Bit Digital Output Port	259
	9.1.3 2 Bit Digital Output Port	259
	9.1.4 12-Bit Analog Output Port 1 and 2	259
	9.1.5 Controlling Stepper Motors	260
	Output Signals	260
	Reference Motions and Position Initialization	261
	Set-Position Motions	261

Querying Signals and Status Values	262
Terminating Infinite Motions	262
WaitTime Parameter	262
9.1.6 RS-232 interface	263
9.1.7 McBSP Interface	263
9.2 Signal Input	264
9.2.1 16-Bit Digital Input Port	264
9.2.2 2-Bit Digital Input Port	264
9.2.3 RS-232 Interface	264
9.2.4 McBSP Interface	264
9.3 Control by External Signals	265
9.3.1 Starting and Stopping Lists	
by External Control Signals and Master/Slave Synchronization	265
External Stop	265
External Start	266
External Start with Track Delay	268
Regular (Periodic) External Starts	269
9.3.2 Conditional Command Execution	271
Example Code (Pascal)	272
9.3.3 Synchronization by Encoder Signals	274
Intended Use	274
Input Ports for External Encoder Signals	275
Encoder Simulation	275
9.3.4 Synchronization and Online Positioning by McBSP/SPI Signals	276
9.4 Periodical I/O Signals	277
10 RTC5 Commands	278
10.1 Overview	278
10.1.1 Designations	278
Multi-Board Commands	278
Normal, Short, Variable and Multiple List Commands	278
Undelayed and Delayed Short List Commands	279
Multiple List Commands	280
10.1.2 Compatibility	280
10.1.3 Version Information	281
10.1.4 Optional Functions	281
10.1.5 Control Commands	282
10.1.6 List Commands	285
10.1.7 Data Types	289
Pointer to Locations in the PC Memory	289
10.2 RTC5 Command Set	290
acquire_rtc	291
activate_fly_2d	292
activate_fly_2d_encoder	293
activate_fly_xy	294
activate_fly_xy_encoder	294
apply_mcbbsp	295
apply_mcbbsp_list	296
arc_abs	297
arc_abs_3d	298
arc_rel	299
arc_rel_3d	300
auto_cal	301

auto_change	304
auto_change_pos	305
bounce_supp	306
camming	307
clear_fly_overflow	309
clear_fly_overflow_ctrl	309
clear_io_cond_list	310
config_laser_signals	311
config_laser_signals_list	312
config_list	313
control_command	315
copy_dst_src	317
disable_laser	318
enable_laser	319
execute_at_pointer	320
execute_list	320
execute_list_1	321
execute_list_2	321
execute_list_pos	321
fly_return	323
fly_return_z	324
free_rtc5_dll	325
get_auto_cal	326
get_char_pointer	327
get_config_list	328
get_counts	328
get_dll_version	329
get_encoder	329
get_error	330
get_fly_2d_offset	334
get_free_variable	334
get_galvo_controls	335
get_head_para	337
get_head_status	338
get_hex_version	340
get_hi_data	340
get_hi_pos	341
get_input_pointer	342
get_io_status	342
get_jump_table	343
get_lap_time	343
get_laser_pin_in	344
get_last_error	344
get_list_pointer	345
get_list_serial	346
get_list_space	346
get_marking_info	347
get_master_slave	350
get_mcbasp	351
get_mcbasp_list	351
get_out_pointer	351
get_overrun	352

get_rtc_mode	352
get_rtc_version	353
get_serial	354
get_serial_number	354
get_standby	355
get_startstop_info	356
get_status	358
get_stepper_status	360
get_sub_pointer	361
get_sync_status	362
get_table_para	363
get_text_table_pointer	364
get_time	364
get_transform	365
get_value	368
get_values	370
get_wait_status	371
get_waveform	372
get_z_distance	373
goto_xy	374
goto_xyz	375
home_position	376
home_position_xyz	377
if_cond	378
if_fly_x_overflow	378
if_fly_y_overflow	379
if_fly_z_overflow	379
if_not_activated	380
if_not_cond	381
if_not_fly_x_overflow	382
if_not_fly_y_overflow	382
if_not_fly_z_overflow	383
if_not_pin_cond	383
if_pin_cond	384
init_fly_2d	385
init_rtc5_dll	386
jump_abs	388
jump_abs_3d	389
jump_abs_drill	389
jump_abs_drill_2	389
jump_rel	390
jump_rel_3d	391
jump_rel_drill	391
jump_rel_drill_2	391
laser_on_list	392
laser_on_pulses_list	393
laser_signal_off	395
laser_signal_off_list	395
laser_signal_on	396
laser_signal_on_list	396
list_call	397
list_call_abs	399

list_call_abs_cond	400
list_call_abs_repeat	400
list_call_cond	401
list_call_repeat	402
list_continue	403
list_jump_cond	404
list_jump_pos	405
list_jump_pos_cond	406
list_jump_rel	407
list_jump_rel_cond	408
list_next	409
list_nop	409
list_repeat	410
list_return	411
list_until	412
load_auto_laser_control	413
load_char	415
load_correction_file	416
load_disk	420
load_fly_2d_table	422
load_jump_table	423
load_jump_table_offset	424
load_list	426
load_position_control	428
load_program_file	430
load_stretch_table	433
load_sub	435
load_text_table	436
load_varpolydelay	437
load_z_table	439
load_zoom_correction_file	440
long_delay	441
mark_abs	442
mark_abs_3d	443
mark_char	444
mark_char_abs	445
mark_date	446
mark_date_abs	448
mark_ellipse_abs	449
mark_ellipse_rel	450
mark_rel	451
mark_rel_3d	452
mark_serial	453
mark_serial_abs	455
mark_text	456
mark_text_abs	457
mark_time	458
mark_time_abs	460
mcbasp_init	461
mcbasp_init_spi	462
measurement_status	463
micro_vector_abs	464

micro_vector_abs_3d	465
micro_vector_rel	466
micro_vector_rel_3d	467
move_to	467
number_of_correction_tables	468
para_jump_abs	469
para_jump_abs_3d	470
para_jump_rel	471
para_jump_rel_3d	472
para_laser_on_pulses_list	473
para_mark_abs	475
para_mark_abs_3d	476
para_mark_rel	477
para_mark_rel_3d	478
park_position	479
park_return	481
pause_list	483
periodic_toggle	484
periodic_toggle_list	485
quit_loop	486
range_checking	487
read_abc_from_file	489
read_analog_in	490
read_encoder	491
read_io_port	492
read_io_port_buffer	493
read_io_port_list	494
read_mcbasp	495
read_multi_mcbasp	496
read_status	497
read_user_data	499
release_rtc	500
release_wait	501
reset_error	502
restart_list	503
rs232_config	503
rs232_read_data	504
rs232_write_data	505
rs232_write_text	505
rs232_write_text_list	506
rtc5_count_cards	507
save_and_restart_timer	508
save_disk	509
select_char_set	511
select_cor_table	512
select_cor_table_list	515
select_rtc	516
select_serial_set	518
select_serial_set_list	518
send_user_data	519
set_angle	520
set_angle_list	521

set_auto_laser_control	522
set_auto_laser_params	525
set_auto_laser_params_list	525
set_char_pointer	526
set_char_table	527
set_control_mode	528
set_control_mode_list	530
set_default_pixel	530
set_default_pixel_list	530
set_defocus	531
set_defocus_list	532
set_defocus_offset	532
set_defocus_offset_list	533
set_delay_mode	534
set_delay_mode_list	535
set_dsp_mode	536
set_ellipse	537
set_encoder_speed	538
set_encoder_speed_ctrl	539
set_end_of_list	540
set_extstartpos	541
set_extstartpos_list	541
set_ext_start_delay	542
set_ext_start_delay_list	543
set_firstpulse_killer	544
set_firstpulse_killer_list	544
set_fly_2d	545
set_fly_limits	546
set_fly_limits_z	547
set_fly_rot	548
set_fly_rot_pos	549
set_fly_tracking_error	550
set_fly_x	551
set_fly_x_pos	552
set_fly_y	553
set_fly_y_pos	554
set_fly_z	555
set_free_variable	556
set_free_variable_list	556
set_hi	557
set_input_pointer	558
set_io_cond_list	559
set_jump_mode	560
set_jump_mode_list	563
set_jump_speed	564
set_jump_speed_ctrl	564
set_jump_table	565
set_laser_control	566
set_laser_delays	569
set_laser_mode	570
set_laser_off_default	570
set_laser_pin_out	571

set_laser_pin_out_list	571
set_laser_pulses	572
set_laser_pulses_ctrl	573
set_laser_timing	574
set_list_jump	575
set_mark_speed	576
set_mark_speed_ctrl	576
set_matrix	577
set_matrix_list	579
set_max_counts	580
set_mcbbsp_freq	580
set_mcbbsp_global_matrix	581
set_mcbbsp_global_matrix_list	581
set_mcbbsp_global_rot	582
set_mcbbsp_global_rot_list	582
set_mcbbsp_global_x	583
set_mcbbsp_global_x_list	583
set_mcbbsp_global_y	584
set_mcbbsp_global_y_list	584
set_mcbbsp_in	585
set_mcbbsp_in_list	586
set_mcbbsp_matrix	587
set_mcbbsp_matrix_list	588
set_mcbbsp_out	588
set_mcbbsp_out_ptr	589
set_mcbbsp_out_ptr_list	590
set_mcbbsp_rot	591
set_mcbbsp_rot_list	591
set_mcbbsp_x	592
set_mcbbsp_x_list	592
set_mcbbsp_y	593
set_mcbbsp_y_list	593
set_multi_mcbbsp_in	594
set_multi_mcbbsp_in_list	596
set_n_pixel	597
set_offset	598
set_offset_list	598
set_offset_xyz	599
set_offset_xyz_list	600
set_pause_list_cond	601
set_pause_list_not_cond	602
set_pixel	603
set_pixel_line	604
set_pixel_line_3d	606
set_port_default	607
set_port_default_list	608
set_pulse_picking	609
set_pulse_picking_length	609
set_pulse_picking_list	610
set_qswitch_delay	610
set_qswitch_delay_list	610
set_rot_center	611

set_rot_center_list	611
set_rtc4_mode	612
set_rtc5_mode	613
set_scale	614
set_scale_list	614
set_scanner_delays	615
set_serial	615
set_serial_step	616
set_serial_step_list	616
set_sky_writing	617
set_sky_writing_limit	618
set_sky_writing_limit_list	618
set_sky_writing_list	618
set_sky_writing_mode	619
set_sky_writing_mode_list	620
set_sky_writing_para	621
set_sky_writing_para_list	623
set_softstart_level	624
set_softstart_level_list	625
set_softstart_mode	626
set_softstart_mode_list	627
set_standby	628
set_standby_list	629
set_start_list	630
set_start_list_1	630
set_start_list_2	630
set_start_list_pos	631
set_sub_pointer	632
set_text_table_pointer	633
set_trigger	634
set_trigger4	642
set_vector_control	643
set_verify	645
set_wait	646
set_wobbel	647
set_wobbel_control	649
set_wobbel_direction	650
set_wobbel_mode	651
set_wobbel_offset	653
set_wobbel_vector	654
set_zoom	656
set_zoom_list	656
simulate_encoder	657
simulate_ext_start	658
simulate_ext_start_ctrl	659
simulate_ext_stop	660
start_loop	661
stepper_abs	662
stepper_abs_list	663
stepper_abs_no	663
stepper_abs_no_list	664
stepper_control	665

stepper_control_list	665
stepper_disable_switch	666
stepper_enable	667
stepper_enable_list	667
stepper_init	668
stepper_rel	670
stepper_rel_list	670
stepper_rel_no	671
stepper_rel_no_list	671
stepper_wait	672
stop_execution	673
stop_list	673
stop_trigger	674
store_encoder	674
sub_call	675
sub_call_abs	676
sub_call_abs_cond	676
sub_call_abs_repeat	677
sub_call_cond	677
sub_call_repeat	678
switch_ioport	679
sync_slaves	680
time_fix	682
time_fix_f	682
time_fix_f_off	683
time_update	684
timed_arc_abs	685
timed_arc_rel	686
timed_jump_abs	687
timed_jump_abs_3d	688
timed_jump_rel	689
timed_jump_rel_3d	690
timed_mark_abs	691
timed_mark_abs_3d	692
timed_mark_rel	693
timed_mark_rel_3d	694
timed_para_jump_abs	695
timed_para_jump_abs_3d	696
timed_para_jump_rel	697
timed_para_jump_rel_3d	698
timed_para_mark_abs	699
timed_para_mark_abs_3d	700
timed_para_mark_rel	701
timed_para_mark_rel_3d	702
transform	703
upload_transform	706
verify_checksum	708
wait_for_encoder	709
wait_for_encoder_in_range	710
wait_for_encoder_mode	711
wait_for_mcbasp	713
write_8bit_port	714

write_8bit_port_list	714
write_abc_to_file	715
write_da_1	716
write_da_1_list	716
write_da_2	717
write_da_2_list	717
write_da_x	718
write_da_x_list	719
write_hi_pos	720
write_io_port	721
write_io_port_list	721
write_io_port_mask	722
write_io_port_mask_list	722
10.3 Unsupported RTC2/RTC3/RTC4 Commands	723
11 Demo Programs	728
12 Troubleshooting	729
13 Customer Service	731
13.1 Servicing and Repairs	731
13.2 Warranty	731
13.3 Contacting SCANLAB	731
13.4 Product Disposal	731
14 Legal	732
14.1 EU Declaration of Conformity – RTC5 PCI Board	732
14.2 Compliance with FCC Rules	733
15 Technical Specifications – RTC5 PCI Board	734
16 Appendix A: RTC5 PCIe Board	738
16.1 Product Overview	738
16.1.1 Intended Use – Comparison to the RTC5 PCI Board	738
16.1.2 System Requirements	738
Hardware	738
Software	738
16.1.3 Optional Functionality	738
16.1.4 Labeling	739
16.1.5 Type Identification	739
16.1.6 Unpacking Instructions and Typical Scope of Delivery	739
Delivered Software	739
16.1.7 Accessories	739
16.1.8 Supplementary Software	739
16.2 RTC5 PCIe Board – Layout and Interfaces	740
16.2.1 Layout – Upper Side	740
16.2.2 Layout – Lower Side	741
16.2.3 Interface to PC	741
16.2.4 SPI/I2C Socket Connector	742
McBSP Interface	742
Analog Input Ports	742
16.3 Installation and Start-Up	743
16.4 Legal	744
16.4.1 EU Declaration of Conformity – RTC5 PCIe Board	744
16.4.2 Compliance with FCC Rules	745
16.5 Technical Specifications – RTC5 PCIe Board	746



17 Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards	747
18 Appendix C: Change Index	790

1 About this Manual

This manual describes the available SCANLAB RTC5 Boards and their usage for synchronous control of scan systems, lasers and peripheral equipment.

The main chapters of this manual use the RTC5 PCI Board to exemplify. For a product overview see [Chapter 2 "Product Overview", Page 27](#).

Other RTC5 boards variants are described in the Appendix:

- RTC5 PCIe Board, [page 738](#)

The manual is a part of the product. Read these instructions carefully before you proceed with installing and operating the RTC5 Board.

In particular, observe all safety guidelines in this manual. If there are any questions regarding the contents of this manual, contact SCANLAB, see [Chapter 1.1 "Manufacturer", Page 21](#).

Keep the manual available for servicing, repairs and product disposal. This manual should accompany the product if ownership changes hands.

This manual refers to:

- [RTC5_Software_2024-09-27](#)⁽¹⁾⁽²⁾⁽³⁾

DLL file for 32 bit user programs ^(a)	RTC5DLL.dll	Version 551 (DLL 551) ^(b)
DLL file for 64 bit user programs ^(a)	RTC5DLLx64.dll	
Program file for the DSP	RTC5OUT.out	Version 553 (OUT 553) ^(b)
Firmware file for the FPGA	RTC5RBF.rbf	Version 534 (RBF 534) ^(b)
Auxiliary file	RTC5DAT.dat	Version 500 (DAT 500) ^(b)

(a) Software for laser-scan processes, which controls RTC5 boards based on this [RTC5 DLL](#) file is consistently denoted as "user program" in this manual.

(b) Abbreviated version in this manual.

1.1 Manufacturer

SCANLAB GmbH
 Siemensstr. 2a
 82178 Puchheim
 Germany
 Tel. +49 (89) 800 746-0
 Fax: +49 (89) 800 746-199
info@scanlab.de
www.scanlab.de

- (1) Software changes prior to [RTC5_Software_Release_2015_07_15](#) are This manual no longer described in this manual.
- (2) The version numbers of the supplied [RTC5 DLL](#) and [RTC5 files](#) are indicated in the names of the corresponding zip files, see [Section "Folder RTC5 Files", Page 28](#).
- (3) To identify the version numbers of your files after installation, use [get_dll_version](#), [get_hex_version](#) and [get_rtc_version](#).



1.2 Related Documents

- "Calibrating a 3-Axis Laser Scan System" Manual
- "excelliSCAN Scan Heads – Functional Principle of SCANahead Servo Control and Operation by RTC6 Boards" Manual

1.3 Glossary and Abbreviations

[*]mark[*] Command	All commands with “mark” as part of their names. See Section “Mark Commands”, Page 125 .
[*]para[*] Command	All commands with “para” as part of their names. See Section “[*]Para[*] Commands”, Page 127 .
3D image field	Synonym: working volume (process volume). See Chapter 7.3.4 “3D Image Field”, Page 159 .
“Arc” command	Umbrella term for Arc Commands and Ellipse Commands .
BCD	Binary Coded Decimal.
DSP	Digital signal processor on the RTC5 board.
Dynamic focusing unit	This includes, for example, the following SCANLAB products: varioSCAN, varioSCAN _{de} , varioSCAN FC and varioSCAN FLEX, excelliSHIFT.
EEPROM	Non-volatile memory of the RTC5 board.
FPGA	Field programmable gate array on the RTC5 board.
Hardware reset	New start after powering the RTC5 board. Synonym: “power up”, “power cycle”.
Hard jump	Direct output to a specified position. Decomposition into Microsteps into a single 10 μ s clock cycle.
High-Bandwidth Return Channel Multiplexing	Functionality not yet released.
iDRIVE scan systems	In this manual, the term subsumes, for example, the following SCANLAB products: intelliSCAN, intelliSCAN _{de} , intelliSCAN _{se} , intelliDRILL, intellicube, intelliWELD, varioSCAN _{de} , powerSCAN II 50i, excelliSCAN.
Image field	Synonym: Working field .
intelliSCAN	In this manual, the term subsumes, for example, the following SCANLAB products: intelliSCAN, intelliSCAN _{de} , intelliSCAN _{se} , intelliDRILL, intellicube, intelliWELD, powerSCAN II 50i.
Jump command	Serves to move the scan system axes to a new position while the laser is <i>off</i> . See also Section “Jump Commands”, Page 125 . See 2D Jump Commands, Page 285 and 3D Jump Commands, Page 285 .

Laser Control Signals	LASER1, LASER2, LASERON. See, for example, Chapter 7.4.1 "Enabling, Activating and Switching Laser Control Signals", Page 173.
LSB	Least Significant Bit.
Mark command	Serves to perform marking motions while the laser is switched on. Examples: mark, arc and ellipse. See 2D Mark Commands, Page 285 and 3D Mark Commands⁽¹⁾, Page 285.
McBSP	Multi channel Buffered Serial Port.
Microstep	See Chapter 7.1.2 "Microstepping", Page 128.
Microvector	The term refers to <code>micro_vector[*]</code> commands, see Chapter 8.8 "micro_vector[*] Commands", Page 249.
<code>micro_vector[*]</code> command	All commands with "micro_vector" as part of their names. See Chapter 8.8 "micro_vector[*] Commands", Page 249.
MSB	Most Significant Bit.
NULL	Means on the one hand the number 0, on the other hand a pointer with the value 0. The spelling for this is different in the different programming languages.
null-terminated	"Terminated with a null character as end identifier" (a character with the internal value zero, often called "NUL", not the same as the glyph zero).
PCB	Printed Circuit Board.
Pixel mode	Brief for " Pixel Output Mode ". See Chapter 8.7 "Pixel Output Mode", Page 244.
Polyline	A direct sequence of <code>[*]mark[*]</code> Commands or " Arc " commands. The marking is continuous. Synonym: polygonal chain, polygonal line, polygonal traversal.
PosAck	"Position Acknowledge", see PosAck signal.

PosAck signal	• As of scan system firmware ≥ 2001. • PosAck signal-capable SCANLAB scan systems, for example, intelliSCAN • Refers to the meaning of: – Bit #12, Bit #4, Bit #11, Bit #3 of the XY2-100 status word transferred with 16-bit protocol – Bit #16, Bit #8, Bit #15, Bit #7 of the XY2-100 status word transferred with 20-bit protocol • These bits are set, if the position errors of x axis and y axis are in the allowed range: – See also “excelliSCAN Scan Heads – Functional Principle of SCANahead Servo Control and Operation by RTC6 Boards” Manual, Chapter 2.1.3 “TrAck Signal”, Page 19. – For more information, refer to the corresponding scan head manuals. • Compare to TrAck signal.
Processing-on-the-fly session	A section of a list that uses external inputs (encoder pulses, McBSP transmissions) to correct moving workpiece positions.
Reset of the RTC5 board	Synonym: Software reset .
RTC5 files	See RTC5 files , page 28.
SCANahead Servo Control	Equipment feature of certain SCANLAB scan systems (= “ SCANahead Systems ”), essentially based on an ISB1 as a servo board for the 2 galvanometer scanner with corresponding firmware and parameterization. Refer to “ excelliSCAN Scan Heads – Functional Principle of SCANahead Servo Control and Operation by RTC6 Boards ” Manual.
SCANahead System	SCANLAB scan system with SCANahead Servo Control , for example, excelliSCAN series. Compare to Tracking error System .
Software reset	Restart after load_program_file . This does not reset everything, for example, loaded correction tables are retained. Synonym: Reset of the RTC5 board .
SPI	Serial Peripheral Interface.
TrAck	“Trajectory Acknowledge”, see TrAck signal .

TrAck signal	- As of scan system firmware $\geq 5102 + \geq 5112$. TrAck signal-capable SCANLAB scan systems, for example, SCANahead Systems - Refers to the meaning of: - Bit #12, Bit #4, Bit #11, Bit #3 of the XY2-100 status word transferred with 16-bit protocol - Bit #16, Bit #8, Bit #15, Bit #7 of the XY2-100 status word transferred with 20-bit protocol - These bits are set, if the Trajectory errors of x axis and y axis are in the allowed range: - See also "excelliSCAN Scan Heads – Functional Principle of SCANahead Servo Control and Operation by RTC6 Boards" Manual, Chapter 2.1.3 "TrAck Signal", Page 19. - For more information, refer to the corresponding scan head manuals. - Compare to PosAck signal.
Tracking error	Time difference between the planned and actual reaching of a certain mirror position at constant speed.
Tracking error Servo Control	The "conventional" servo control of Tracking error Systems .
Tracking error System	SCANLAB scan system with conventional control (1st generation) = with Tracking error and without preprocessing. For example, intelliSCAN III. Compare to SCANahead System.
Trajectory	In this manual: curve with $10 \mu s$ parameterization.
Trajectory error	Deviation of the actual Trajectory from the set Trajectory .
Vector command	Umbrella term for Jump Commands and Mark Commands .
Working field	Synonym: Image field .

2 Product Overview

2.1 Labeling

The serial number of the RTC5 PCI Board is printed on a label attached to the board (format: "RTC SN...").

The ID number and configuration of the board are described in the packaging list, see [Chapter 2.6 "Options", Page 34](#) and [Chapter 2.7 "Jumper Settings and Type Identification", Page 35](#).

2.2 Unpacking Instructions and Typical Scope of Delivery

- (1) Carefully remove the RTC5 Board from the package.
- (2) Keep the packaging, including the antistatic bag the RTC5 Board is delivered in, so that in case of repair the board can be properly repackaged and returned to SCANLAB.
- (3) Also remove all other articles from the package. Check that all parts have been delivered. Refer to the corresponding packaging list.
The scope of delivery typically includes an RTC5 PCI Board and a data CD (with the RTC5 software package, see below, and this manual). Possibly additional components are also contained, see [Chapter 2.8 "Accessories for the RTC5 PCI Board", Page 37](#).

2.3 Delivered RTC5 Software Package

The delivered RTC5 software package contains the RTC5 board driver for the 32-bit and 64-bit versions of the operating systems Microsoft Windows 10, 8, 7.

The data CD contains all files as unzipped versions. The complete RTC5 software package is also delivered zipped for easy identification and management of different software versions:

- RTC5_Software_<Date>.zip

The content of the RTC5 software package is as follows:

- Readme.txt
Description in English
- Liesmich.txt
Description in German

Folder Correction Files

- Cor_1to1.ct5
1to1 correction file⁽¹⁾⁽²⁾

Folder CorrectionFileConverter

- CorrectionFileConverter.exe
This Win32-based program converts RTC4 correction files *.ctb to RTC5 correction files *.ct5 and vice versa. The corresponding manual is supplied in English and German.

(1) Additional correction files (D2_XXX.CT5, D3_YYY.CT5) and *_ReadMe.txt file(s) are not part of the RTC5 software package.

(2) See also [Section "1to1 Correction Tables", Page 166](#).

Folder DemoFiles

- Source codes in C of Demo1...Demo7 (Demo1.cpp...Demo7.cpp)
- Demo1 compiled to executable files for 32-bit as well as 64-bit
- Source code in C# of Demo3 (Class1.cs)
- Project generating file for CMake (CMakeLists.txt) and the corresponding include file (RTC_Variables.cmake)⁽¹⁾

Folder HPGL

- Hpgl.exe is a Win32-based HPGL-to-RTC5 converter. See also [Chapter 5.6 "Functionality Test", Page 81](#).
- The folder contains some *.plt files (Hewlett Packard HPGL format (vector graphic plotter files) for test purposes.
- Hpgl.exe needs **RTC5 files** and the **RTC5DLL.dll** in the same folder.

Folder iSCANConfig

- iSCANcfg.exe is a Win32-based diagnosis and configuration program for iDRIVE scan systems⁽²⁾. RTC4, RTC5 and RTC6 (PCI/PCIe as well as Ethernet variants) are supported.
- Needs **RTC5 files** and the **RTC5DLL.dll** in the same folder and in addition RtcHalDLL.dll. The corresponding manual is supplied in English (Manual_iSCANcfg_v1-7.pdf) and German (Handbuch_iSCANcfg_v1-7.pdf).

(1) These are CMake (cross-platform make) files to easily generate executable demo programs. CMake (<https://cmake.org>) is a free and open-source cross-platform programming tool for the development and creating software. Using Cmake, make files and projects for many integrated development environments and compilers can be generated by script files (CMakesList.txt).

(2) See Glossary entry on [page 23](#).

Folder RTC5 Driver

(RTC5 board driver for Windows)

- RTC5DRV.sys, RTC5DRVx64.sys, RTC5DRVx86.sys
RTC5 driver files
- RTC5DRV.inf
Installation file (setup information)
- RTC5DRV.cat, rtc5drv64.cat, rtc5drv86.cat
Security catalog files
- amd64/WdfCoInstaller01009.dll, x86/WdfCoInstaller01009.dll
Installation assistant help files
- AfterInstallation/ScanlabClassChecker.cmd,
Security installation script with description in ReadMe_ScanlabClassChecker.pdf

Folder RTC5 Files

- RTC5 DLL
 - RTC5DLL.dll
Win32-based RTC5 dynamic link library
 - RTC5DLLx64.dll
Win64-based RTC5 dynamic link library
- RTC5 files
 - RTC5OUT.out
Program file for the **DSP**
 - RTC5RBF.rbf
Firmware file for the **FPGA**
 - RTC5DAT.dat
Binary auxiliary file
- Utility Files for C, C++ and C#
 - RTC5expl.c
C functions for **RTC5 DLL** handling for explicit linking
 - RTC5expl.h
C function prototypes of the RTC5 for explicit linking of the **RTC5 DLL**
 - RTC5DLL.lib
Visual C++ import libraries for implicit linking of the **RTC5 DLL** for Win32-based user programs
 - RTC5DLLx64.lib
Visual C++ import libraries for implicit linking of the **RTC5 DLL** for Win64-based user programs
- RTC5_Software_RevisionHistory_<Date>.pdf
Description of RTC5 software package changes in English
- RTC5_Software_Aenderungshistorie_<Date>.pdf
Description of RTC5 software package changes in German

- RTC5impl.h
C function prototypes of the RTC5 for implicit linking of the **RTC5 DLL**
- RTC5impl.hpp
C++ function prototypes of the RTC5 for implicit linking of the **RTC5 DLL**
- RTC5Wrap.cs
Import declarations of the wrapper class for implicit linking in C#
- Utility File for Delphi
 - RTC5Import.pas
Import declarations for Delphi

Differing versions of **RTC5 files** and **RTC5 DLL** cannot be arbitrarily combined with another. Therefore, for easy identification of the versions, the following zip files are provided (each includes a text file with version and compatibility information):

- RTC5DAT_<current DAT version number>.zip
- RTC5DLL_<current DLL version number>.zip
(includes DLLs and related utility files)
- RTC5OUT_<current OUT version number>.zip
- RTC5RBF_<current RBF version number>.zip

Folder RTC5 Tools

- SleepMode.cmd
Script to deactivate *all* Windows sleep and hibernate modes.
- ReadMe_SleepMode.pdf
Description of the use of **SleepMode.cmd**

2.4 Intended Use

The SCANLAB RTC5 PCI Board and its associated RTC5 software package is intended for synchronous real-time control of scan systems, lasers and peripheral equipment by a Windows PC with a PCI bus interface.

RTC5 boards are available in different hardware variants, [Chapter 1 "About this Manual", Page 21](#).

The delivered **RTC5 DLL** provides an extensive command set for control. This allows a quick and flexible software development for laser-scan processes.

The RTC5 PCI Board is equipped with a fast digital signal processor (**DSP**). During execution of control commands it also handles more complex signal processing, such as simultaneous control of two scan systems or coordinate transformations.

Moreover, you can store controlling commands (= list commands) on the RTC5 PCI Board and start their execution at a later time. Command execution by the RTC5 PCI Board can then take place independently of the host PC. This makes it possible to meet the stringent demands of real-time control for scan systems, lasers and peripheral equipment, even if the PC must simultaneously respond to other tasks such as machine control and network communication.

The interface to the scan system, together with the associated software commands, allows bidirectional communication with the scan system, thereby providing both control and monitoring capabilities for the scan system.

With the RTC5 PCI Board, commonly used laser types can be controlled. To control lasers, the supplies interfaces that output laser control signals and are software-configurable for each application's requirements.

Users can choose among different laser modes and set the signal parameters (for example, the signal level – active-HIGH or active-LOW) or the output frequency to a suitable value.

For controlling peripheral equipment and incorporating external control signals, the RTC5 PCI Board provides a range of interfaces (for example, a 16-bit digital input port, a 16-bit digital output port, two 12-bit analog output ports and an RS-232 interface, see [Chapter 4 "RTC5 PCI Board – Layout and Interfaces", Page 51](#)) and associated software commands.

As many RTC5 PCI Boards can be operated in one PC at the same time as the PC provides PCI slots.

Moreover, the **RTC5 DLL** allows multi-threading as well as multi-processing. Therefore, several user programs can even be used simultaneously.

No board can be simultaneously used by multiple applications. Multiple threads of *one* user program can use the same board, but can not send commands to it at the same time. The **RTC5 DLL** serializes these accesses automatically.

RTC5 PCI Boards are available in various configurations, see [Chapter 2.6 "Options", Page 34](#) and [Chapter 2.7 "Jumper Settings and Type Identification", Page 35](#).

The RTC5 PCI Board interfaces are described on [Chapter 4 "RTC5 PCI Board – Layout and Interfaces", Page 51](#), installation and start-up on [Chapter 5 "Installation and Start-Up", Page 77](#), and programming on [Chapter 6 "Developing RTC5 User Programs", Page 83](#). Individual command descriptions are listed beginning with [Chapter 10 "RTC5 Commands", Page 278](#).

The technical specifications of the RTC5 PCI Board are summarized in [Chapter 15 "Technical Specifications – RTC5 PCI Board", Page 734](#).



Caution!

- Do not operate the RTC5 PCI Board outside of the PC.
- The RTC5 PCI Board is intended only for industrial usage. It is designed to be incorporated in machines (normally laser systems). It does *not* meet all criteria of ready-to-use products. Do not operate the RTC5 PCI Board unless it is incorporated in a machine which itself complies with the regulations of all applicable standards and directives (of the country concerned). It is *not* suitable to be used as toy, in household or under unfavorable environment conditions (for example, in the open). Appropriate precautions to avoid such unforeseeable misapplications must be taken by users.
- Installation and operation must only be performed by trained specialists, among other things knowledgeable in the safe and proper use of electrical devices. Only carry out installation and maintenance work when supply voltages and lasers have been switched off.
- The RTC5 PCI Board is a class A product. In a domestic environment this product may cause radio interferences in which case the user may be required to take adequate measures.



2.5 System Requirements

2.5.1 Hardware

The RTC5 PCI Board requires a Windows PC with a PCI bus interface and at least one free PCI slot.

RTC5 PCI Boards intended for synchronized master/slave operation should (recommended) be installed in adjacent PCI slots.

2.5.2 Software

To operate the RTC5 PCI Board, RTC5 board driver and **RTC5 DLL** files for Microsoft Windows operating systems must be used. These are included in the scope of delivery (RTC5 software package). For the supported Windows versions, see [Chapter 2.3 "Delivered RTC5 Software Package"](#), Page 27.

The RTC5 board driver supports the plug and play capability of the RTC5 PCI Board as well as the simultaneous operation of any number of RTC5 PCI Boards. The **RTC5 DLL** files contain the command set to control laser scan systems.

Notice!

- The RTC5 PCI Board does not support power-saving modes, that switch off power to the PCI bus. Accordingly, you must disable standby or sleep modes of the operating system. See also [Section "Notes"](#), Page 33.

Notice!

- If on your PC an WDM technology-based RTC3/4/5 board driver
 - is yet installed or
 - was installed and has been removed or
 - you are not sure in this regard:
 - (1) Install the RTC5 board driver.
 - (2) Run `ScanlabClassChecker.cmd` as Administrator (part of the delivered RTC5 software package; see there also the background information in [ReadMe_ScanlabClassChecker.pdf](#)).
Step 2 can be skipped on brand new PCs on which an RTC board driver never has been installed.

Notes

- RTC5 software package version 2013_02_21 and later contain (newer) WDF⁽¹⁾ drivers (version 6.1.7600.16385).
Earlier RTC5 software packages still contain a WDM⁽²⁾ driver (outdated today; version 1.0.4.0 or version 2.0.6.0).
- DLL $\leq$ 533 are *not* compatible with the WDF drivers.
- DLL $\geq$ 535 are even compatible with the WDM drivers.
- In comparison to the outdated WDM driver, the WDF driver offers the following new functionality: If `init_rtc5_dll` is called, then the WDF driver prevents automatic activation of standby or sleep modes (continuously until the next system restart). This enables RTC5 PCI Boards to continue processing lists autonomously even when the initiating program has already terminated.
However, standby or sleep modes cannot be prevented if triggered manually or by discharged batteries. Afterward, loaded lists and other settings of the RTC5 PCI Board are lost. After a wake-up, the RTC5 PCI Board might no longer be correctly addressable. Before calling `init_rtc5_dll` for the first time, make sure that standby or sleep modes of the operating system are deactivated. The script `SleepMode.cmd` which is contained in the RTC5 software package deactivates *all* sleep and hibernation modes, see [ReadMe_SleepMode.pdf](#).
- RTC5 software packages versions *newer* than 2022-03-09 (that is, with $\geq$ DLL 551)
 - RTC5 board driver and **RTC5 DLL** files are designed for 32-bit as well as 64-bit versions of Microsoft Windows 10, 8, 7.
 - RTC5 DLL** files from these packages *do not* support Microsoft Vista, XP SP2, XP SP3 and earlier any more.
- RTC5 software packages version 2013_02_21 through 2022-03-09 (with $\geq$ DLL 535...DLL 550)
 - RTC5 board driver and **RTC5 DLL** files were designed for 32-bit as well as 64-bit versions of Microsoft Windows 10, 8, 7, Vista, XP SP2, XP SP3.
 - RTC5 DLL** files from these packages *did not* support Microsoft Windows 2000 and XP $\leq$ SP1 any more.
- RTC5 software packages version 2011_09_29 through 2012_09_05 (with DLL 528...DLL 535)
 - RTC5 board driver and **RTC5 DLL** files were designed for Microsoft 32-bit as well as 64-bit versions of Windows 7, Vista, XP SP2, XP SP3.
 - RTC5 DLL** files from these packages *did not* support Microsoft Windows 2000 and XP $\leq$ SP1 any more.
- RTC5 software packages of version 2011_07_27 and earlier (with $\leq$ DLL 527)
 - RTC5 board driver and **RTC5 DLL** files were designed for 32-bit versions of Microsoft Windows 7, Vista, XP and 2000.

(1) Windows Driver Framework. Current technology.

(2) Windows Driver Model. Legacy technology.

2.6 Options

RTC5 PCI Boards can be equipped with different options (features). For new cards, the options ordered are enabled/installed at the SCANLAB factory on delivery. For information on retrofitting already delivered boards, see [Section "Notes", Page 34](#).

To query which options are actually enabled on a certain board, `get_rtc_version` is used.

The following options are available for RTC5 PCI Boards:

- **Option Processing-on-the-fly**
Allows that a Processing-on-the-fly correction can be activated, see [Chapter 8.6 "Processing-on-the-fly", Page 227](#).
- **Option "3D"**
 - **Controlling a 3-Axis Scan System**
Allows that an RTC5 PCI Board can synchronously control a third axis (z axis, for example, a varioSCAN as a dynamic focus unit) along with the scan system's x axis and y axis (usually two galvanometer scanners) by its two scan head connectors, see [Chapter 8.5.2 "3D Scan Systems", Page 222](#).
- **Option "Second Scan Head Control"**
 - **Controlling Two Scan Systems Simultaneously**
Allows that an RTC5 PCI Board can simultaneously control two xy-scan systems via its two scan head connectors, see also [Chapter 4.5.1 "Scan Head Connectors and Transfer Protocol", Page 54](#) and [Chapter 8.5 "Controlling 2D Scan Systems and 3D Scan Systems", Page 221](#).
- **Option "DC/DC Converter"**
These boards are equipped with an extra DC/DC converter (optoelectronic coupler) ex works. Therefore, the laser control signals LASERON, LASER1 and LASER2 at the LASER connector and EXTENSION 2 socket connector are galvanically decoupled from the PC ground, see [Section "Laser Control Signals", Page 60](#), and [Section "Laser Control Signals", Page 68](#).

Notes

- Each RTC5 PCI Board article number indicates which of the options above are enabled and/or installed. These are stated in the packing list. The naming there is as follows:
 - "Fly" for [Option Processing-on-the-fly](#)
 - "3D" for [Option "3D"](#)
 - "SSHC" for [Option "Second Scan Head Control"](#)
 - "DCDC" for [Option "DC/DC Converter"](#)
- To query which options are actually enabled on a board, `get_rtc_version` is used.
- If you need one or more options which are not activated out of the factory on your RTC5 PCI Board: you can activate them by special upgrade software available from SCANLAB. When requesting the upgrade, you will need to supply the serial number of your board.
- For retrofitting your RTC5 PCI Board with the extra DC/DC converter (optoelectronic coupler) you need to send it to SCANLAB.
- For optical data transfer between the RTC5 PCI Board and the scan system, solely a suitable data cable is required, see [Chapter 4.5.3 "Data Cables \(Accessories\)", Page 58](#). Neither the RTC5 PCI Board side nor the scan system side requires a special optical data interface for that.

2.7 Jumper Settings and Type Identification

SCANLAB ships RTC5 PCI Boards in various jumper configurations. Jumpers are connections which are either open or closed. The *factory* solder jumper configuration of an RTC5 PCI Board can be identified by its article number.

In addition, a three-digit type code scheme is used, for example,

- “RTC5 PCI TYPE 000”
 - No signals (that is, all solder jumpers are open)
- “RTC5 PCI TYPE 124”
 - 5 V output signal level at the EXTENSION 1 socket connector
 - DATA7 at pin 15 of the EXTENSION 2 socket connector
 - LATCH at pin 17 of the EXTENSION 2 socket connector

Trained users can reconfigure solder jumpers by using a soldering iron. The assignment of a desired signal is done by closing or opening (applying or removing solder or zero-ohm resistor) of corresponding solder jumpers as described in the following⁽¹⁾⁽²⁾⁽³⁾.

Notice!

- Only configure allowed jumper settings. Otherwise, the board gets damaged!

Digit 1	Refers to the output signal level at the EXTENSION 1 socket connector ⁽¹⁾ .
	=0: No signals. =1: 5 V. =2: 3,3 V.
Digit 2	Refers to pin 15 of the EXTENSION 2 socket connector ⁽³⁾ .
	=0: No signals. =1: +5 V. =2: DATA7. =3: GROUND.
Digit 3	Refers to pin 17 of the EXTENSION 2 socket connector ⁽²⁾ .
	=0: No signals. =1: +5 V. =2: DATA7. =3: GROUND. =4: LATCH.

(1) For the solder jumper setting, see [Chapter 2.7.1 “Jumper JP1 – Configuring the Output Signal Level at the EXTENSION 1 Socket Connector”](#), Page 36.

(2) For the solder jumper setting, see [Chapter 2.7.2 “Jumper JP2...JP8 – Configuring Pin15 and Pin17 at the EXTENSION 2 Socket Connector”](#), Page 36.

(3) For the solder jumper setting, see [Chapter 2.7.2 “Jumper JP2...JP8 – Configuring Pin15 and Pin17 at the EXTENSION 2 Socket Connector”](#), Page 36.

2.7.1 Jumper JP1 – Configuring the Output Signal Level at the EXTENSION 1 Socket Connector

By Jumper JP1 on the lower side of the RTC5, see [Figure 6](#), the level of all output signals at the EXTENSION 1 socket connector can be configured for 5 V or 3.3 V.

See also [Section “Configuring the Output Signal Level”, Page 65](#).

Jumper JP1	Signal Level
Position 1-2* 	5 V
Position 2-3* 	3.3 V
open 	no output signals
* Caution: make sure that <i>only one</i> position is closed in this solder jumper field. Other combinations are not allowed and cause damage to the board!	

2.7.2 Jumper JP2...JP8 – Configuring Pin15 and Pin17 at the EXTENSION 2 Socket Connector

By the jumpers JP2...JP8 on the lower side of the RTC5, see [Figure 6](#), pins (15) and (17) of the EXTENSION 2 socket connector can be configured. See also [Section “Configuration by Solder Jumpers”, Page 67](#).

The most significant bit (DATA7) of the 8-bit output value can be assigned to pin (15) or to pin (17). Alternatively, each of the two pins can be set permanently to +5 V (HIGH) or to GND (LOW level). Alternatively, pin (17) can be configured for the LATCH signal by closing the correspondingly labeled jumper.

[Figure 1](#) shows an example jumper configuration assigning DATA7 to pin (15) and the LATCH signal to pin (17).

	Pin17	Pin15
(JP6) Data7		 Data7(JP3)
(JP5) +5V		 +5V (JP2)
(JP7) GND		 GND (JP4)
(JP8) Latch		

Example configuration for jumpers JP2...JP8: Pin (15): DATA7, Pin (17): LATCH signal.

Notice!

- Make sure that not more than **one** of the three jumpers for pin (15) is closed. Also make sure that not more than **one** of the 4 jumpers for pin (17) is closed. Other combinations are not allowed and cause damage to the board!

2.8 Accessories for the RTC5 PCI Board

Only hardware extensions from SCANLAB should be used in combination with the RTC5 PCI Board. In addition to the RTC5 PCI Board and its software package, the following accessories can be obtained:

- [XY2-100 Converter](#), page 37
- [Laser Adapter](#), page 37
- [Data Cables](#), page 37
- [Slot Cover with 9-pin D-SUB Connector for 2. SCANHEAD Socket Connector](#), page 38
- [Slot Cover with 15-pin D-SUB Connector for MARKING ON THE FLY Socket Connector](#), page 38
- [Slot Cover with 15-pin D-SUB Connector and 9-pin D-SUB Connector](#), page 39
- [ADC Add-On Board](#), page 39
- [Extension Board "RTC5/6 varioSCAN FLEX Extension"](#), page 39

2.8.1 XY2-100 Converter

The [XY2-100 Converter \(Accessory\)](#) allows the RTC5 PCI Board to control scan systems which are equipped with a conventional XY2-100 interface.

2.8.2 Laser Adapter

The SCANLAB laser adapter is plugged into the 15-pin LASER connector of the RTC5 PCI Board. Then a 9-pin female D-SUB connector of this adapter provides the same signals and pin-out as the 9-pin laser connector of the RTC4. See also [Section "Laser Adapter \(Accessory\)"](#), Page 63.

2.8.3 Data Cables

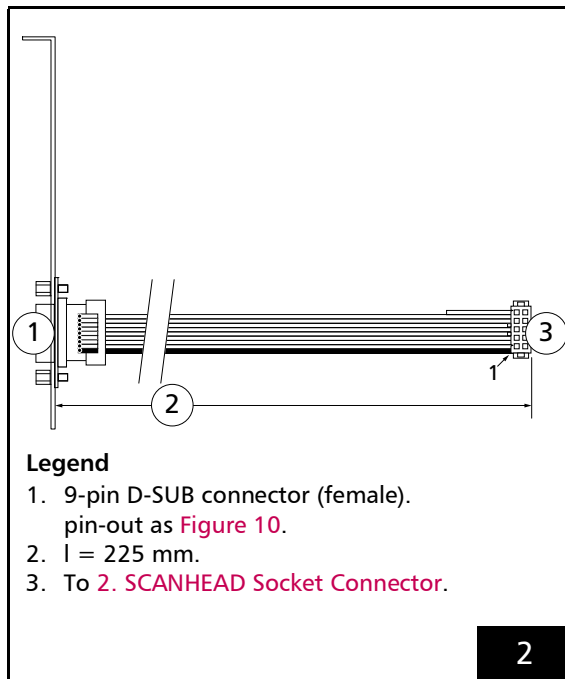
To connect the RTC5 PCI Board to scan systems, SCANLAB offers appropriate cables in a variety of lengths – either conventional cables for electrical data transfer or polymer optical fiber cables or optical data transfer. See also [Chapter 4.5.3 "Data Cables \(Accessories\)"](#), Page 58).

2.8.4 Slot Cover with 9-pin D-SUB Connector for 2. SCANHEAD Socket Connector

For connecting a second scan head or a z axis to the **2. SCANHEAD Socket Connector** a slot cover is available, see **Figure 2**:

- #0115132

Its 9-pin D-SUB connector has the same pin-out as the **SCANHEAD Connector**, see **Figure 10**.
See also **Section "Second Scan Head Slot Cover (Accessory)", Page 55**.



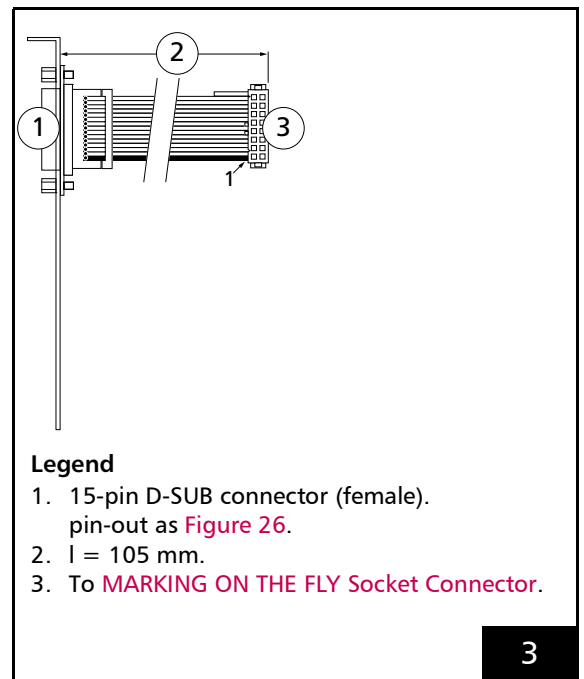
Second Scan Head Slot Cover (Accessory) #0115132.

2.8.5 Slot Cover with 15-pin D-SUB Connector for MARKING ON THE FLY Socket Connector

For using the inputs and signals of the **MARKING ON THE FLY Socket Connector**, a slot cover is available, see **Figure 3**:

- 0109272

The pin-out of its 15-pin D-SUB connector (female) is shown in **Figure 26**.
See also **Section "MARKING ON THE FLY Slot Cover (Accessory)", Page 70**.

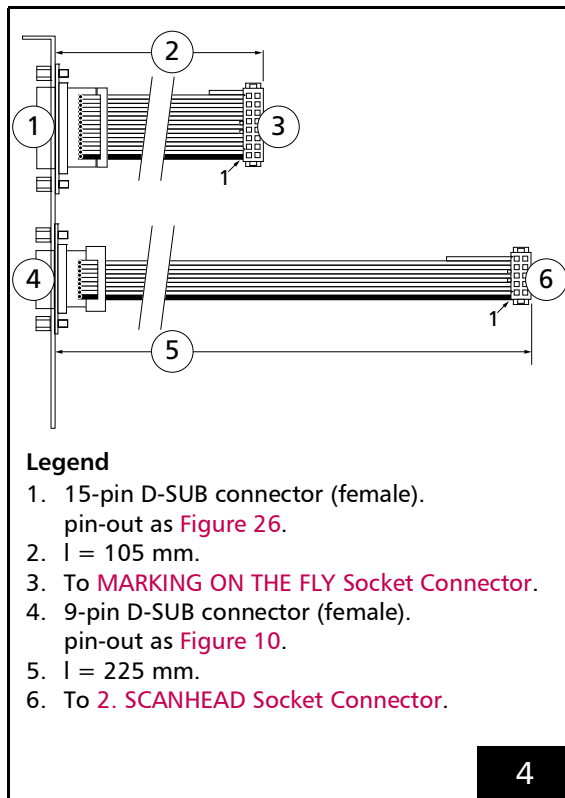


MARKING ON THE FLY Slot Cover (Accessory) 0109272.

2.8.6 Slot Cover with 15-pin D-SUB Connector and 9-pin D-SUB Connector

The connectors of #0115132 and 0109272 together on a single bracket (for the same purpose as these) features bracket, see Figure 4:

- #0130209



Slot cover #0130209.

2.8.7 ADC Add-On Board

Only for the RTC5 PCI Board⁽¹⁾, the SCANLAB ADC Add-On Board (#0121126) is available, see Figure 33.

With this add-on board installed, the RTC5 PCI Board provides three 10 V analog inputs, see Chapter 4.6.8 "Analog Inputs (Accessory)", Page 75.

The RTC5 PCIe Board provides two 10 V analog inputs even without add-on board, see Chapter 16.2.4 "SPI/I2C Socket Connector", Page 742.

2.8.8 Extension Board "RTC5/6 varioSCAN FLEX Extension"

For RTC5/6 Boards, SCANLAB offers the "RTC5/6 varioSCAN 40 FLEX Extension" extension board (#0128683)⁽²⁾.

It has been specially developed to control analog and digital varioSCAN 40 FLEX devices by the SCANLAB laserDESK software. A position change of the varioSCAN 40 FLEX focusing optics is caused by the step motor which changes the working distance of the 3D scan system in the end.

For further information, refer to the pertaining manual "Installation and Operation RTC5/6 varioSCAN FLEX Extension for RTC5 and RTC6 Control Boards".

Notes

- "RTC5/6 varioSCAN 40 FLEX Extension" extension board (#0128683) does *not* support **move_to**⁽³⁾.

(1) Not for use with the RTC5 PCIe Board.

(2) In contrast to extension board "RTC4 STEP MOTOR EXTENSION" (#0112097) – which can also be used with RTC5 Boards – it has the advantage that the EXTENSION 1 socket connector remains unoccupied.

(3) **move_to** has been introduced for extension board "RTC4 STEP MOTOR EXTENSION" (#0112097).

2.9 Supplementary Software

To facilitate customizing RTC correction files basing on data of your own test measurements, SCANLAB offers the correXion pro software with accompanying manual, see also [Section "Image Field Correction Algorithm", Page 163](#).

By using the SCANLAB laserDESK software, own laser marking and material-processing programs ("laser jobs") can be created and executed without software development. Many of the RTC5 functions are supported.⁽¹⁾

For further information on laserDESK, refer to the SCANLAB homepage.

2.10 Notes for RTC4 Users

This chapter provides an overview of key changes introduced by the RTC5 PCI Board in comparison to the RTC4 PCI Board.

For example, [Chapter 2.10.2 "Porting RTC4 Source Code to the RTC5 PCI Board", Page 42](#) discusses a possible approach to porting RTC4 programs to run on the RTC5.

The individual command descriptions in particular note changes.

2.10.1 Hardware Changes

When migrating from the RTC4 to the RTC5, you need to consider the following hardware changes for correct cabling of the system components.

Controlling Scan Systems

- To control a scan system with the XY2-100 or XY2-100 Enhanced interface, you also need the [XY2-100 Converter \(Accessory\)](#).
- The RTC5 cannot control scan systems with XY2-100-O interfaces (for optical data transfer).
- To control a scan system with the SL2-100 interface, you need a different data cable (available from SCANLAB), see [Chapter 4.5.3 "Data Cables \(Accessories\)", Page 58](#).
- To use the second scan head connector, you need a different adapter cable (including slot cover available from SCANLAB), see [Section "Second Scan Head Slot Cover \(Accessory\)", Page 55](#).
- For controlling a 3-axis scan system, both scan head connectors must be used, see [Chapter 8.5.2 "3D Scan Systems", Page 222](#).
- Simultaneous control of two 3-axis scan systems requires two RTC5 Boards, see [Chapter 8.5.2 "3D Scan Systems", Page 222](#).

(1) See also [Notice!, Page 60](#).

Controlling the Laser

- The female D-SUB laser connector at the RTC5 slot cover has 15 pins (the RTC4 laser connector, on the other hand, has 9 pins). The pin-outs are not jumper-configurable.
 - The RTC5 does not require jumpers X6 and X7 of the RTC4 because all signals are available at the RTC5 laser connector.
 - The voltage range of the analog outputs is always 0...10 V (0 V...2.50 V is no longer supported; the RTC4's jumper X3 does not exist on the RTC5).
- If you want to connect a laser to the RTC5 by the same (9-pin) cable that you previously used with the RTC4, then you need an adapter with a 9-pin female D-SUB connector (available from SCANLAB).
 - For use of the laser adapter from SCANLAB (see [Section "Laser Adapter \(Accessory\)", Page 63](#)), two jumpers are provided for configuring the pin-outs (JP1 corresponds to jumper X7 of the RTC4, JP2 corresponds to jumper X6 of the RTC4).
- The signal levels of the laser control signals are no longer determined by configuring jumpers. Instead, they can/must be software-configured (see [set_laser_control](#)).
 - Jumper X10 of the RTC4 does not exist on the RTC5.

EXTENSION 1 Socket Connector

- The RTC5 EXTENSION 1 socket connector is – except for the additionally provided signals at pins 33-35 – identical to the EXTENSION 1 socket connector of the RTC4 (see [Figure 22](#)).
- With the RTC5, the level of all output signals at the EXTENSION 1 socket connector can be configured for 5 V or 3.3 V by a jumper (see [Section "Configuring the Output Signal Level", Page 65](#)).

EXTENSION 2 Socket Connector

- The RTC4 EXTENSION 2 socket connector does not exist on the RTC5. Accordingly, no I/O extension board can be attached to the RTC5.
- The RTC5 EXTENSION 2 socket connector is – except for the optional LATCH signal at pin 17 – identical to the LASER EXTENSION socket connector of the RTC4 (see [Figure 24](#)).
 - The RTC5 provides jumpers JP2...JP8 for configuring pin-outs (these jumpers are equivalent to jumpers X8 and X9 of the RTC4) (see also [page 67](#)).

MARKING ON THE FLY Socket Connector

- The RTC5 MARKING ON THE FLY socket connector is identical to the RTC4 MARKING ON THE FLY socket connector, see [Figure 25](#). Observe the safety notice on encoder counter direction, [page 49](#).

Other Hardware Interfaces

- PCI bus requirements are identical to those of the RTC4.
- PCI-Express version only available for the RTC5.
- The following interfaces only exist on the RTC5:
 - MASTER and SLAVE (see [Chapter 4.4 "Master Socket Connector, Slave Socket Connector", Page 53](#))
 - RS 232 (see [Chapter 4.6.5 "RS232 Socket Connector", Page 70](#))
 - SPI / I²C / McBSP (see [Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71](#))
 - STEPPER MOTOR (see [Chapter 4.6.7 "STEPPER MOTOR Socket Connector", Page 75](#))

2.10.2 Porting RTC4 Source Code to the RTC5 PCI Board

User programs written for the RTC4 can only run on the RTC5 after suitable code revision. This applies even when the actual program flow shall remain unchanged.

Changed Initialization

The program's initialization section should be revised at least as follows:

- At the beginning of the user program, a **init_rtc5_dll** command must be inserted for initializing the RTC5 DLL and RTC5 board management (see [Chapter 6.2.3 "Initializing the RTC5 DLL and Board Management"](#), Page 86).
- The files for initializing the board by **load_program_file** are different than with the RTC4 (see command description).
- Scan system initialization by **load_correction_file** and **select_cor_table** utilizes different correction files (with file extension *.ct5) (see [Chapter 7.3.5 "Image Field Correction and Correction Tables"](#), Page 162).
- For laser control initialization, the **set_laser_control** command must be additionally inserted (see [Chapter 7.4 "Laser Control"](#), Page 173).

Command Changes

All unsupported RTC4 commands must be removed or replaced, see also [Chapter 10.3 "Unsupported RTC2/RTC3/RTC4 Commands"](#), Page 723.

Changed or enhanced RTC4 commands need to be handled differently in the program (for example, by modifying supplied parameter values or evaluating returned values differently). Changes to supported commands are listed in the individual command descriptions (in [Chapter 10.2 "RTC5 Command Set"](#), Page 290) under the heading "RTC4→RTC5". RTC4 commands that need to be replaced or checked are:

- **aut_change** not supported
- **auto_cal** changed
- **auto_change_pos** changed

- **control_command** changed
- **dsp_start** not supported
- **get_head_status** changed
- **get_hi_data** changed
- **get_list_space** changed
- **get_marking_info** changed
- **get_rtc_version** changed
- **get_startstop_info** changed
- **get_status** changed
- **get_value** changed
- **get_waveform** changed
- **get_xy_pos** not supported
- **get_xyz_pos** not supported
- **goto_xy** changed
- **goto_xyz** changed
- **list_jump_cond** changed
- **list_nop** changed
- **load_cor** not supported
- **load_correction_file** changed
- **load_pro** not supported
- **load_program_file** changed
- **read_pixel_ad** not supported
- **read_status** changed
- **rtc3_count_cards** not supported
- **rtc4_count_cards** not supported
- **select_cor_table** changed
- **select_list** not supported
- **select_rtc** changed
- **set_control_mode** enhanced
- **set_control_mode_list** enhanced
- **set_laser_mode** enhanced
- **set_laser_timing** changed
- **set_list_mode** not supported
- **set_matrix** changed
- **set_matrix_list** changed
- **set_offset** changed
- **set_offset_list** changed
- **set_piso_control** not supported
- **set_pixel** changed
- **set_pixel_line** changed
- **set_softstart_mode** changed
- **set_trigger** enhanced
- **set_wobbel** changed
- **set_wobbel_xy** not supported

Increased Parameter Resolution

When switching from the RTC4 to the RTC5 – even for some commands not mentioned above – take note that the resolution has been increased for several parameters. Examples:

- For commands such as **mark_abs** or **jump_rel**, the real-image-field x coordinate values and y coordinate values are specified with 20-bit resolution for the RTC5 (whereas with 16-bit resolution for the RTC4), see also [Chapter 7.3.2 "Image Field Size and Image Field Calibration"](#), [Page 156](#).
An extended, virtual 24-bit value range is available with the RTC5 for Processing-on-the-fly applications, see also [Chapter 7.3.3 "Virtual Image Field"](#), [Page 158](#).
- For commands such as **write_da_x**, analog output values are specified with 12-bit resolution for the RTC5 (whereas with 10-bit resolution for the RTC4).
- For **set_laser_timing**, output period and pulse length are specified with $1/64 \mu\text{s}$ resolution for the RTC5 (whereas with $1/8 \mu\text{s}$ or $1 \mu\text{s}$ resolution for the RTC4).
- For **set_laser_delays**, the **Laser Delays** are specified with $0.5 \mu\text{s}$ resolution for the RTC5 (whereas with $1 \mu\text{s}$ resolution for the RTC4).

Here, though, it generally suffices to set the RTC5 DLL to **RTC4 Compatibility Mode** by **set_rtc4_mode**. Then the **RTC5 DLL** automatically converts parameter values so that many RTC4 command sequences can run unchanged on the RTC5.

RTC4 Compatibility Mode affects the following RTC4 commands (descriptions of the respective commands include relevant information under the heading "RTC4→RTC5"):

- **arc_abs**
- **arc_rel**
- **fly_return**
- **get_z_distance**
- **goto_xy** (changed)
- **goto_xyz** (changed)
- **home_position**
- **jump_abs**
- **jump_abs_3d**
- **jump_rel**
- **jump_rel_3d**
- **mark_abs**
- **mark_abs_3d**
- **mark_rel**
- **mark_rel_3d**
- **set_delay_mode**
- **set_ext_start_delay**
- **set_ext_start_delay_list**
- **set_firstpulse_killer**
- **set_firstpulse_killer_list**
- **set_fly_x**
- **set_fly_y**
- **set_jump_speed**
- **set_laser_delays**
- **set_mark_speed**
- **set_pixel** (changed)
- **set_pixel_line** (changed)
- **set_rot_center**
- **set_softstart_level**
- **set_standby**
- **set_standby_list**
- **simulate_ext_start**
- **timed_jump_abs** (changed)
- **timed_jump_rel** (changed)
- **timed_mark_abs** (changed)
- **timed_mark_rel** (changed)

- **write_da_1**
- **write_da_1_list**
- **write_da_2**
- **write_da_2_list**
- **write_da_x**
- **write_da_x_list**

The previously mentioned revision of initialization and checking of unsupported or changed RTC4 commands needs to be carried out regardless of whether the program is to execute in **RTC5 Standard Mode** or **RTC4 Compatibility Mode**.

Changed Timing Behavior

The following RTC4 list commands are processed by the RTC5 as “short list commands”. This can result in a changed timing behavior during execution (see [Section “Normal, Short, Variable and Multiple List Commands”, Page 278](#)).

- **clear_io_cond_list**
- **list_call**
- **list_call_cond**
- **list_jump_cond** (changed)
- **list_return**
- **save_and_restart_timer**
- **set_extstartpos_list**
- **set_firstpulse_killer_list**
- **set_io_cond_list**
- **set_jump_speed**
- **set_laser_delays**
- **set_laser_timing** (changed)
- **set_list_jump**
- **set_mark_speed**
- **set_scanner_delays**
- **set_standby_list**
- **set_trigger** (enhanced)
- **set_wobbel** (changed)
- **write_8bit_port_list**
- **write_da_1_list**
- **write_da_2_list**
- **write_da_x_list**
- **write_io_port_list**

Likewise, automatic delay adjustments can produce a changed timing behavior (see [Section “Automatic Delay Adjustments”, Page 143](#)).

2.10.3 New and Changed Functionality

Interface to the PC

- The RTC5 board driver supports simultaneous control of any number of RTC5 PCI Boards in a single PC, see [Chapter 6.6 "Using Several RTC5 PCI Boards in One PC"](#), Page 112.
- Connectors and commands for master/slave synchronization of several RTC5 PCI Boards, see [Chapter 4.4 "Master Socket Connector, Slave Socket Connector"](#), Page 53.

Controlling Scan Systems

- New interface to the scan system, see [Chapter 4.5.1 "Scan Head Connectors and Transfer Protocol"](#), Page 54:
 - 9-pin female D-SUB connector at the RTC5 PCI Board slot cover and 10-pin socket connector
 - SL2-100 transfer protocol
 - 2 data channels each for both scan head connectors
 - Galvanically isolated signals
 - 20-bit positioning resolution, see [Chapter 7.3.2 "Image Field Size and Image Field Calibration"](#), Page 156
 - Enhanced status return from the scan system, see [Chapter 7.3.7 "Status Monitoring and Diagnostics"](#), Page 172
 - Control and status channels for enhanced data transfer with iDRIVE scan systems⁽¹⁾
- An **XY2-100 Converter (Accessory)** is available for data transfer according to the XY2-100 protocol.
 - 25-pin female D-SUB connector
 - 16-bit positioning resolution
 - Status return according XY2-100 or XY2-100 Enhanced protocol
 - Transfer synchronization is configurable for long data cables by solder jumpers in the **XY2-100 Converter (Accessory)**
 The RTC5 PCI Board provides power for the **XY2-100 Converter (Accessory)**.
- For optical data transfer between the RTC5 PCI Board and scan systems, *no* special variant of the RTC5 PCI Board (with XY2-100-O interface) is required. Optical data transfer can be realized by a SCANLAB data cable with electrical-to-optical conversion in its D-SUB connector housing, see [Chapter 4.5.3 "Data Cables \(Accessories\)"](#), Page 58.
- For controlling a 3-axis scan system, see [Chapter 8.5.2 "3D Scan Systems"](#), Page 222:
 - Both scan head connectors must be used
 - Simultaneous control of two 3-axis scan systems requires 2 RTC5 PCI Boards
- Image field correction
 - New correction files are needed, see [Chapter 7.3.5 "Image Field Correction and Correction Tables"](#), Page 162:
 - File name extension ".ct5"
 - Correction with higher resolution
 - Queryable information in the correction file header
 - For the RTC5 PCI Board, RTC4 correction files (.ctb) need to be newly calculated or converted by [CorrectionFileConverter.exe](#) which is part of the RTC5 software package.
 - Up to 4 correction files can be loaded to the RTC5 PCI Board
 - Enhanced **3D image field** correction by stretch correction tables, see [Section "Enhanced 3D Correction"](#), Page 225
- Coordinate transformations (see [Chapter 8.2 "Coordinate Transformations"](#), Page 209):
 - The correction file is no longer transformed (rotation, shift, extension) at download
 - Matrix transformations are only applied after **Microstepping** – this may cause the mark speed to change
 - **"Local Online Positioning"**, see [Chapter 8.3.1 "Local Online Positioning"](#), Page 213

(1) See Glossary entry on [page 23](#).

- Coordinate transformations in the virtual **Image field** (incl. **"Global Online Positioning"**), see **Chapter 7.3.3 "Virtual Image Field"**, Page 158
 - 24-bit position coordinates (virtual **Image field**): objects larger than the real **Image field** are possible
- Position monitoring of iDRIVE scan systems⁽¹⁾ by backward transformation of actual position values, see **Chapter 8.1.3 "Monitoring the Positioning"**, Page 200
- Automatic self-calibration, see **Chapter 8.10 "Automatic Self-Calibration"**, Page 251:
 - Optimization of previous functions
 - ASC hardware check
- **Jump Mode**, see **Chapter 8.1.5 "Jump Mode"**, Page 202
- Output synchronization, see **Chapter 7.4.10 "Output Synchronization"**, Page 195
- **Pulse Picking Laser Mode**, see **Chapter 7.4.8 "Pulse Picking Laser Mode"**, Page 186
- Laser pulse period, pulse length or analog output are also programmable within a **Polyline** between two vectors – where the laser remains on⁽²⁾, see **"short list commands"** in **Section "Normal, Short, Variable and Multiple List Commands"**, Page 278
- Commands for position-dependent, speed-dependent, vector-defined and encoder-speed-dependent laser control, see **Chapter 7.4.9 "Automatic Laser Control"**, Page 187

Controlling the Laser

- The signal levels of the laser control signals are no longer determined by a jumper configuration. Instead, they are software-configured, see **set_laser_control**
- 15-pin female D-SUB LASER connector with all laser signals at the RTC5 PCI Board slot cover, see **Chapter 4.6.1 "LASER Connector"**, Page 60, 9-pin female D-SUB connector only by the SCANLAB laser adapter, see **Section "Laser Adapter (Accessory)"**, Page 63
- 15-pin female D-SUB LASER connector configurable by software command, see **Chapter 7.4.2 "Configuring the LASER Connector"**, Page 176
- Laser control signals with 15 ns resolution and 20 mA output current
- Standby signals in YAG modes, see **Chapter 7.4.4 "YAG Mode 1, YAG Mode 2, YAG Mode 3, YAG Mode 5"**, Page 179
- YAG Mode 5: Time between **FirstPulseKiller** signal and first laser pulse in YAG mode is freely programmable, see **Chapter 7.4.4 "YAG Mode 1, YAG Mode 2, YAG Mode 3, YAG Mode 5"**, Page 179
- **Laser Mode 6**: LASERON signal synchronized with a continuously-running LASER1 signal, see **Chapter 7.4.6 "Laser Mode 6"**, Page 183

Interfaces for Peripheral Equipment

- 16-bit digital output, see **Section "16-Bit Digital Input Port and 16-Bit Digital Output Port"**, Page 65, and **Chapter 9.1.1 "16-Bit Digital Output Port"**, Page 258:
 - Level of output signals selectable by a jumper (3.3 V or 5 V)
 - LATCH signal for synchronization of data transmission
- 8-bit digital output port, see **Section "8-Bit Digital Output Port"**, Page 68 and **Chapter 9.1.2 "8-Bit Digital Output Port"**, Page 259:
 - Provided at the EXTENSION 2 socket connector (on the RTC4, this socket connector is named "LASER EXTENSION")
 - LATCH signal for synchronization of data transmission
 - Adjustable "stop output value"
- Analog output ports, see **Section "12-Bit Analog Output Port 1 and 2"**, Page 61, and **Chapter 9.1.4 "12-Bit Analog Output Port 1 and 2"**, Page 259:
 - 12 bit resolution
 - 0...10 V (0 V...2.50 V no longer available)
 - Adjustable "stop output value"
- 16-bit digital input port, see **Section "16-Bit Digital Input Port and 16-Bit Digital Output Port"**, Page 65, and **Chapter 9.2.1 "16-Bit Digital Input Port"**, Page 264.
 - SYNC signal for synchronization of data transmission

(1) See Glossary entry on **page 23**.

(2) Is switched off with the RTC4.

- Programmable debouncing of external start signals, see **bounce_supp** and **Section "External Start"**, **Page 266**
- Regular (periodic) **External Starts**, see **Section "Regular (Periodic) External Starts"**, **Page 269**
- New interfaces:
 - 2-bit digital input and 2-bit digital output at the D-SUB LASER connector, see **Section "2-Bit Digital Input Port"**, **Page 61** and **Section "2-Bit Digital Output Port"**, **Page 61**
 - RS-232 interface by an 10-pin socket connector, see **Chapter 4.6.5 "RS232 Socket Connector"**, **Page 70**
 - Stepper motor signals for 2 motors by an on-board 10-pin socket connector, see **Chapter 4.6.7 "STEPPER MOTOR Socket Connector"**, **Page 75**
 - **McBSP**, **I²C** and **SPI** by a 10-pin socket connector, see **Chapter 4.6.6 "SPI / I2C Socket Connector"**, **Page 71**
- For the RTC5 PCI Board there is no IO extension board. Therefore, it does not have a socket connector for installing such a board (on the RTC4, this socket connector is named "EXTENSION 2"; the RTC5 EXTENSION 2 socket connector corresponds to the RTC4 "LASER EXTENSION" socket connector).

General Programming

- Utility files for C, C++ , C# and Delphi, see **Chapter 6.2.2 "Importing Commands"**, **Page 84**, but no longer utility files for Basic
- Commands for changing access rights to RTC5 PCI Boards, see **Chapter 6.7 "Usage of RTC5 PCI Boards by Several User Programs"**, **Page 117**
- Improved and extended list handling, see **Chapter 6.4 "List Handling"**, **Page 94**
 - List memory with 1,048,576 storage positions
 - List memory free configurable (in 2 unprotected list memory areas and 1 protected list memory area)
 - Defining protected subroutines
 - Loading lists with protection
 - Loops in lists and subroutines
 - RTC4-Circular queue mode is not available (but can be coded alternatively)
 - **List Status**
 - **List Execution Status**
- "Short" list commands (for example, to change speed, analog output, I/O port, etc.) can be executed without time losses (multiple "short" list commands can be executed within one 10 μ s clock cycle, see **Section "Normal, Short, Variable and Multiple List Commands"**, **Page 278**
- Functions for error handling and download verification, see **Chapter 6.8 "Error Handling"**, **Page 119**

Laser Marking

- Vectors and arcs
 - Commands for marking ellipses, see [Section "Ellipse Commands", Page 126](#)
 - Commands for marking helices, see [Chapter "3D Commands", Page 223](#)
 - Timed arc commands, timed 3D vector commands, see [Chapter 8.9 "Timed Commands", Page 250](#)
 - Para-Mark commands and Para-Jump commands for "vector-controlled laser control", see [Section "Vector-Defined Laser Control", Page 194](#)
- Characters and texts
 - Defining character sets and text strings, see [Section "Defining Indexed Character Sets", Page 107](#)
 - Commands for marking individual characters and for marking texts (with selectable character set), see [Section "Calling Indexed Characters", Page 108](#)
 - Commands for marking dates, times and serial numbers (with selectable character set and selectable serial-number-set), see [Chapter 7.5 "Marking Dates, Times and Serial Numbers", Page 196](#)
- `micro_vector[*]` commands (direct position output without Microstepping), see [Chapter 8.8 "micro_vector\[*\] Commands", Page 249](#).

Special Functions

- Synchronization of scan system and laser control
 - Sky Writing, see [Chapter 7.2.4 "Sky Writing", Page 149](#)
- Pixel Output Mode (marking of bitmaps), see [Chapter 8.7 "Pixel Output Mode", Page 244](#):
 - Pixel output frequencies up to 300 kHz, irrespective of the 10 μ s clock cycle
 - 15 ns resolution
 - 0...100% laser pulse length
 - Pixel-Mode 0 not supported
 - Reading of analog voltages is not supported
 - Pixel marking on sloped surfaces
 - 3D pixel lines with `set_pixel_line_3d`
- Commands for conditional execution of any list command, see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#)
- Possible wobble motion shapes include not only circles, but also ellipses, horizontal figure-of-8s, vertical figure-of-8s, and "freely definable wobble shapes". Options for the orientation of the wobble shapes are: stationary in space, continuously and automatically adjusted to the current direction of motion, or any other freely assigned motion direction. With "Freely definable wobble shapes", also the laser power can be varied, see [Chapter 8.4 "Wobble Mode", Page 217](#)
- Camming functionality, see [Chapter 8.11 "Camming", Page 255](#)
- Enhanced signal recording, see `set_trigger` and `set_trigger4`

- Processing-on-the-fly, see [Chapter 8.6 "Processing-on-the-fly", Page 227](#)
 - 2 encoder input ports (RS-422) with 32-bit counter for Processing-on-the-fly correction with encoder signals on 2 axes, see [Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274](#); alternatively: Processing-on-the-fly correction with **McBSP** signals, see [Chapter 9.3.4 "Synchronization and Online Positioning by McBSP/SPI Signals", Page 276](#)
 - 24-bit coordinates (virtual **Image field**): objects larger than the real **Image field** are possible, see [Chapter 7.3.3 "Virtual Image Field", Page 158](#), and [Chapter 8.6.6 "Virtual Image Field with Processing-on-the-fly", Page 237](#)
 - Up to 8 objects within the Processing-on-the-fly track delay (between trigger and marking position), see [Section "External Start", Page 266](#)
 - Accurate **"External Start"**:
If accordingly configured by **set_control_mode**, the encoder counter can be reset by external start signals for synchronizing a Processing-on-the-fly process. The reset occurs fully simultaneously (without 10 μ s jitter) with the external start signal.
 - Compensation of 2D motions (XY table)
 - Encoder-based Processing-on-the-fly correction for the z axis (FlyZ correction), see [Chapter 8.6.11 "Processing-on-the-fly Correction for the z Axis", Page 242](#)

Notice!

- The encoder counter direction of RTC4 boards (includes RTC4 Ethernet Board and RTC SCANalone Board) is opposite to that of RTC5 boards and RTC6 boards.
Therefore, when changing RTC-control board you need to either adapt the cabling or your user program accordingly.

3 Safety During Installation and Operation

Read these operating instructions completely before you proceed with installing and operating the RTC5 PCI Board.

If there are any questions regarding the contents of this manual, please contact SCANLAB.

The following conventions apply to safety notices in this manual:



- Safety notices which draw attention to severely injuries or even death are identified by the hazard symbol and the signal word "Warning!".



- Safety notices which draw attention to a health hazard are identified by the hazard symbol and the signal word "Caution!".
- Safety notices that recommend proper use of the device or warn against possible damage to property are (without hazard sign) only identified by the signal word "Notice!".

3.1 Steps for Safe Operation

Notice!

- Carefully check your user program before running it. Programming errors can cause a break down of the system. In this case neither the laser nor the scan system can be controlled.
- Protect the board from humidity, dust, corrosive vapors and mechanical stress.

Notice!

- For storage and operation of the RTC5 PCI Board, avoid electromagnetic fields and static electricity. These can damage the electronics on the board. For storage, always use the antistatic bag the board is delivered in.
- The allowed operating temperature range is 15 °C to 60 °C.
- The storage temperature should be between –20 °C and +60 °C.

3.2 Laser Safety

The RTC5 is intended for controlling scan systems and lasers. Therefore all relevant laser safety directives must be known and applied before installation and operation. The customer is solely responsible for ensuring the laser safety of the entire system.

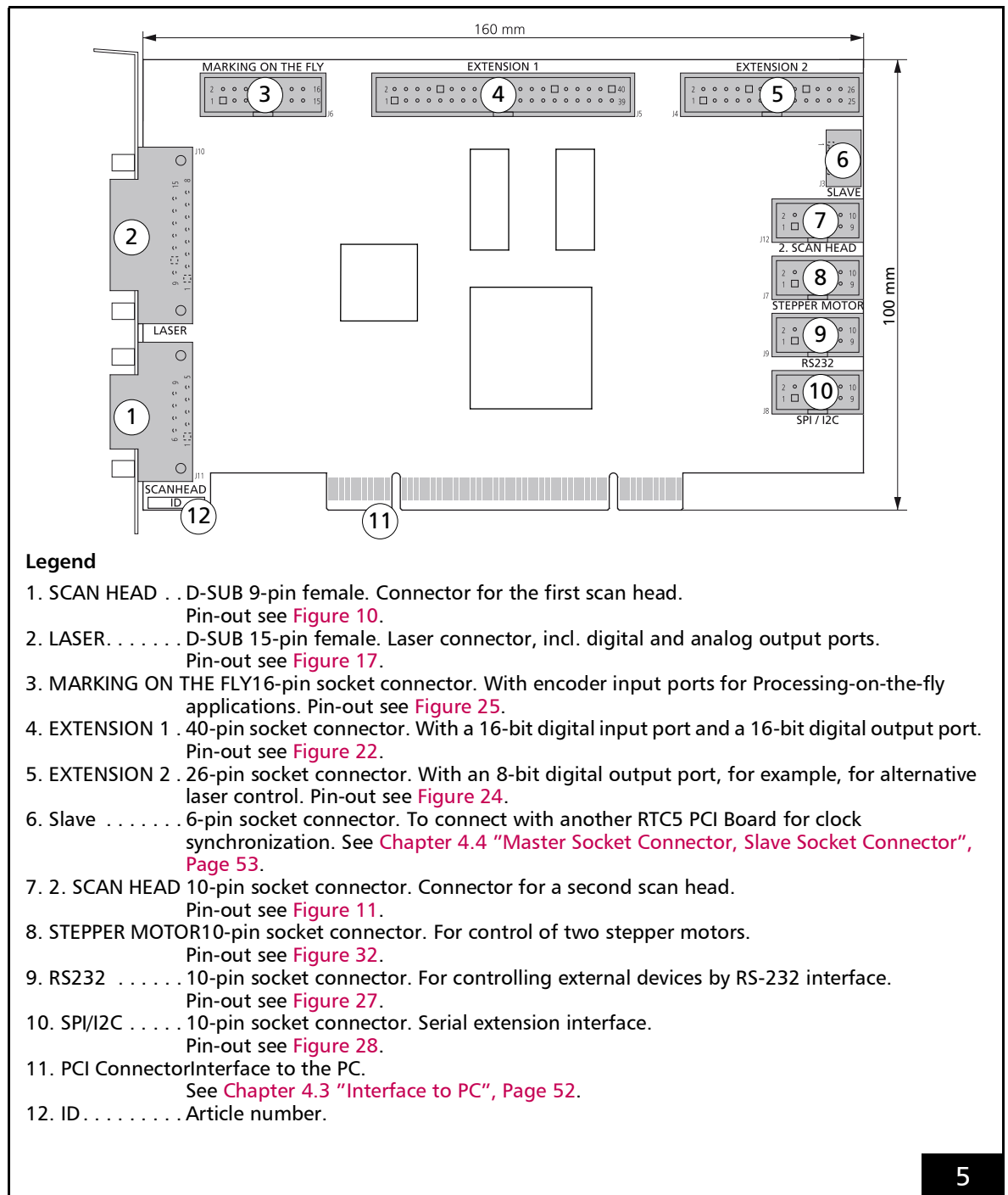


Caution!

- All applicable laser safety directives must be adhered to. Safety regulations may differ from country to country. It is the responsibility of the customer to comply with all local regulations.
- Observe all laser safety instructions as described in your scan system manual, chapter "Safety during Installation and Operation".
- *Always turn on the PC and the power supply for the scan head first before turning on the laser. Otherwise, there is the danger of uncontrolled deflection of the laser beam.*
SCANLAB recommends the use of a shutter to prevent uncontrolled emission of laser radiation.

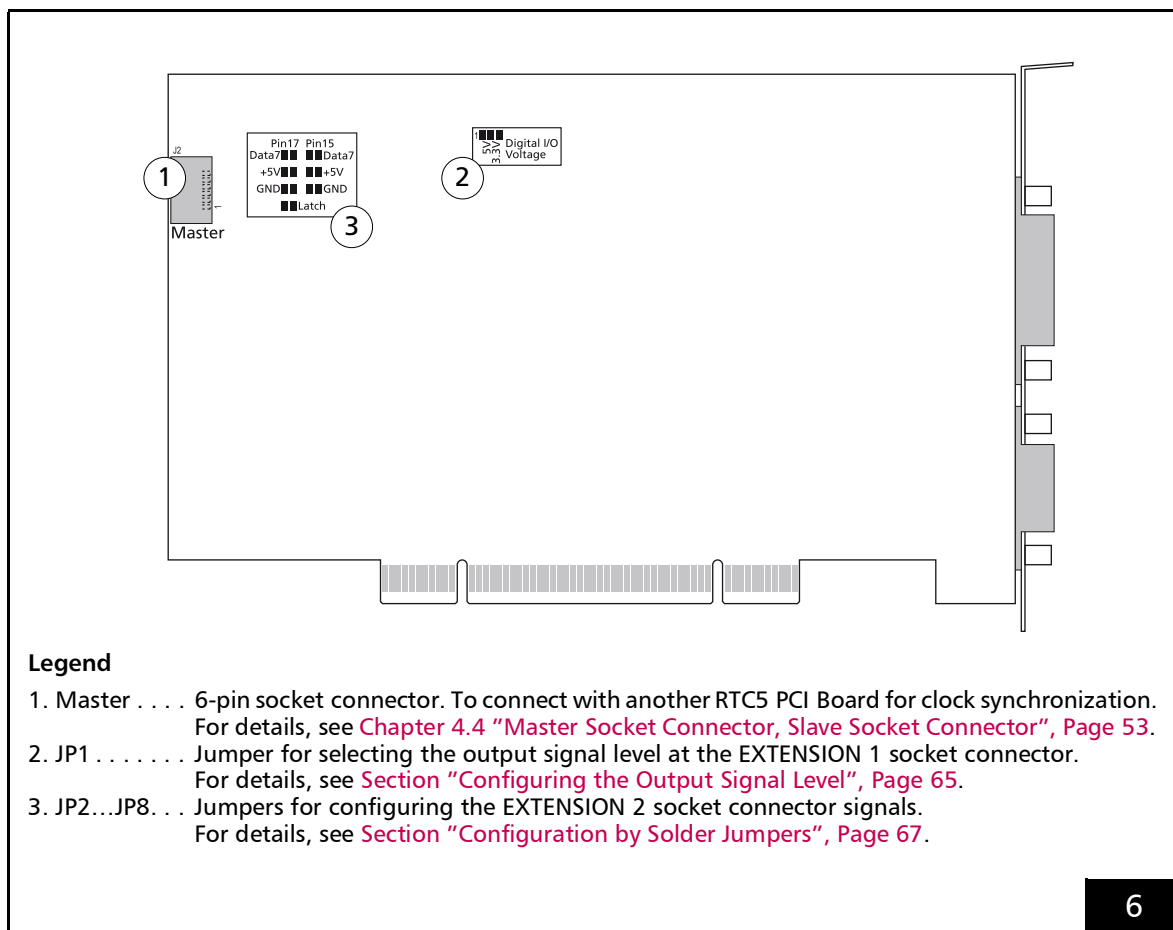
4 RTC5 PCI Board – Layout and Interfaces

4.1 Layout – Upper Side



RTC5 PCI Board: upper side.

4.2 Layout – Lower Side



RTC5 PCI Board: lower side.

4.3 Interface to PC

The RTC5 PCI Board PCI connector is the interface to the PC.

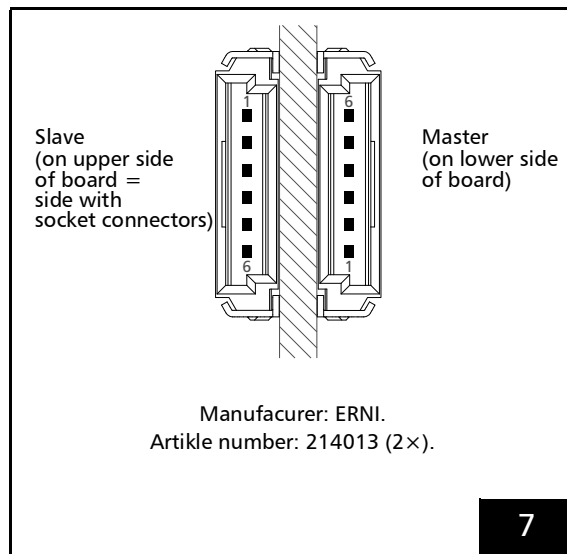
The RTC5 PCI Board can be installed into any Windows PC with a PCI bus interface and at least one free PCI slot.

Notice!

- The RTC5 PCI Board does not support power-saving modes that switch off power to the PCI bus. Accordingly, you must disable standby or sleep modes of the operating system. See also [Section "Notes", Page 33.](#)

4.4 Master Socket Connector, Slave Socket Connector

The Slave socket connector as well as the Master socket connector have 6 pins, see [Figure 7](#). The Slave socket connector is located on the upper side of the RTC5 PCI Board, see [Figure 5](#). The Master socket connector is located on the lower side, see [Figure 6](#).



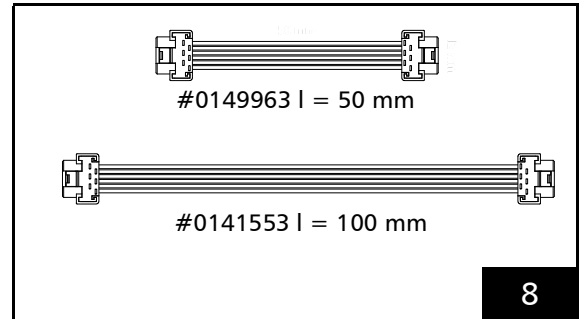
Slave socket connector and Master socket connector.
The pitch of the pins is 1.27 mm.

Purpose of the both socket connectors is to make a clock cycle synchronization of several RTC5 PCI Boards possible. Then they must be connected pairwise with each other by the Master and Slave socket connectors. Always connect a Master connector of a board to the Slave connector of another board by a suitable cable (available from SCANLAB, see [Figure 8](#)). The necessary information for assembling your own cables is shown in [Figure 9](#).

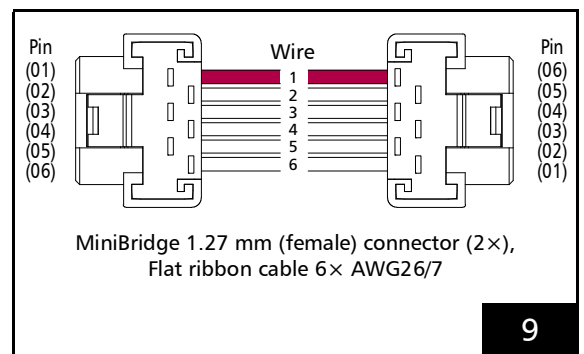
Interconnected RTC5 PCI Boards should (recommended) be plugged into adjacent PCI slots.

Important: In order to use the master/slave functionality, see prerequisites in [Chapter 6.6.3 "Master/Slave Operation"](#), Page 113.

See also [Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization"](#), Page 265.



SCANLAB Master/Slave connecting cable.



Cable to connect the Master socket connector and Slave socket connector: requirements. Keep length as short as possible!

4.5 Interfaces to Scan System

4.5.1 Scan Head Connectors and Transfer Protocol

The first scan head connector SCAN HEAD and the optionally activated second scan head connector "2. SCAN HEAD" are available for digitally controlling scan systems (see Figure 5). At those connectors, scan-system control values are transmitted and scan-system status signals received. Each scan head connector can transmit data for up to two axes. Consult your scan system's operating manual to determine which status signals are generated by your scan system and how they can be applied for monitoring purposes.

Data transfer between the RTC5 and the scan system is in accordance with the SL2-100 protocol. The **XY2-100 Converter (Accessory)** is available for converting the signals (for data transmission according to the XY2-100 protocol).

If neither the **Option "Second Scan Head Control"** nor the **Option "3D"** is enabled, only the first scan head connector outputs signals for an xy scan system.

If the **Option "Second Scan Head Control"** is enabled, two xy scan systems can be simultaneously controlled by one RTC5 PCI Board.

If the **Option "3D"** is enabled, then a 3-axis scan system can be controlled by the two scan head connectors (if the first scan head connector has been assigned a 3D correction table). Signals can then be outputted by the first scan head connector to an xy scan head – and by both channels of the second scan head connector to the third axis (z axis).

If both options (**Option "Second Scan Head Control"** and **Option "3D"**) are enabled, the assignment of the correction tables determines which signals (xy or z) are to be outputted by which connector, see also **Section "2D Correction Files and 3D Correction Files"**, Page 163.

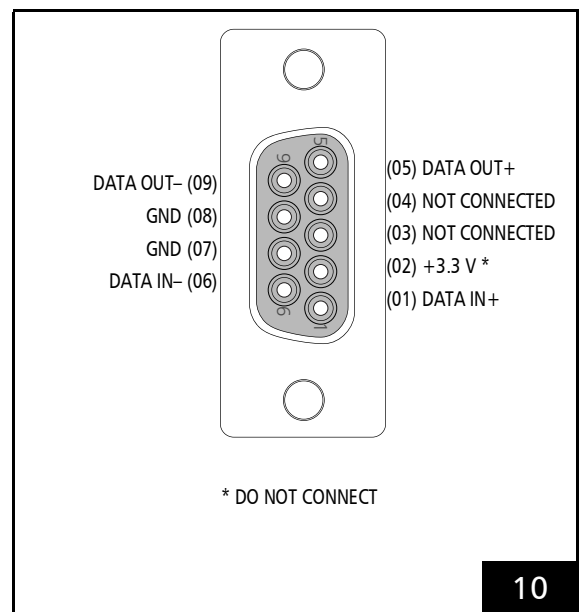
If several RTC5 PCI Boards with enabled **Option "3D"** are installed in a PC, then that many 3-axis systems can be simultaneously controlled.

For master/slave functionality, multiple RTC5 PCI Boards can be run – synchronously clocked – in one PC if **load_program_file** has been executed on all boards (see **Chapter 4.4 "Master Socket Connector, Slave Socket Connector"**, Page 53 and **Section "Initializing the Board"**, Page 87).

SCANHEAD Connector

(connector for the first scan head)

The pin-out of the first scan head connector SCANHEAD (D-SUB 9-pin female) is shown in Figure 10.



SCANHEAD connector (9-pin female D-SUB connector): pin-out.

The differential DATA OUT output port transmits control values to the scan system.

The differential DATA IN input port receives the status signals returned by the scan system.

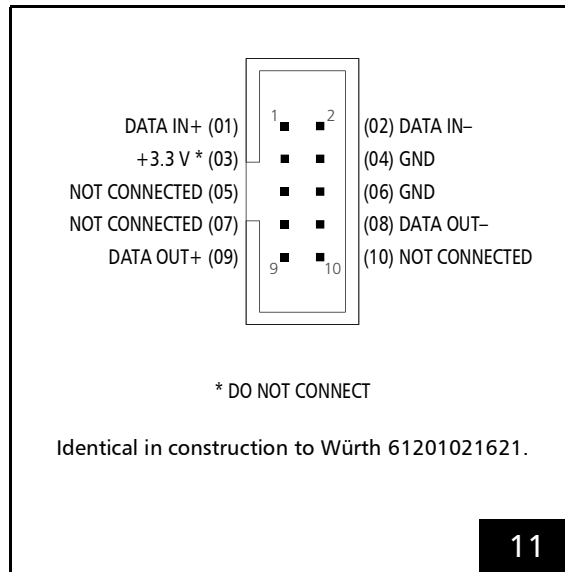
Pin 2 supplies 3.3 V power for the **XY2-100 Converter (Accessory)** (or a Polymer Optical Fiber converter for optical data transmission). This voltage should not be used for other purposes.

2. SCANHEAD Socket Connector

(connector for the second scan head)

The pin-out of the second scan head connector

2. SCANHEAD (10-pin socket connector) is shown in **Figure 11**.



2. SCANHEAD socket connector: pin-out. The pitch of the pins is 2.54 mm. Signals for second scan head are only outputted with enabled **Option "Second Scan Head Control"**.

Second Scan Head Slot Cover (Accessory)

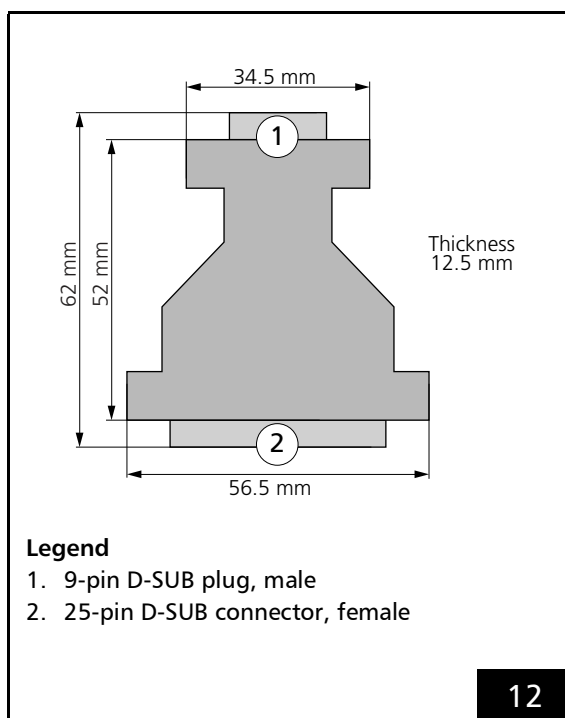
SCANLAB recommends using an additional slot cover for connecting a second scan head or a z axis to the **2. SCANHEAD Socket Connector**, see **Chapter 2.8.4 "Slot Cover with 9-pin D-SUB Connector for 2. SCANHEAD Socket Connector"**, Page 38.

4.5.2 XY2-100 Converter (Accessory)

The SCANLAB **XY2-100 Converter (Accessory)** (#0125377) converts the RTC5's SL2-100 control signals (20 bit) into XY2-100-compliant signals (16 bit), and converts scan system XY2-100 status signals into SL2-100-compliant signals (see [page 172](#)).

The **XY2-100 Converter (Accessory)** introduces a 10 μ s runtime latency to scan-system control. This runtime latency can be compensated by increasing the **LaserOn Delay** and **LaserOff Delay** by 10 μ s each (see [set_laser_delays](#)).

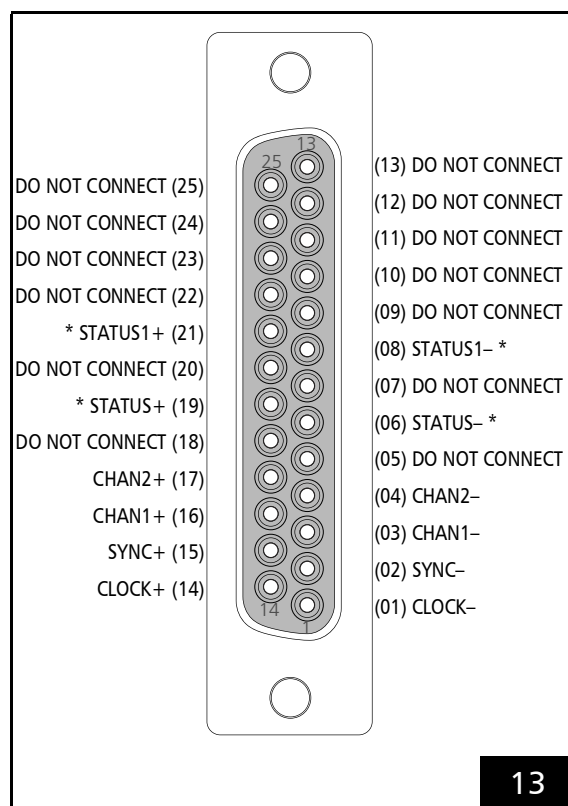
The dimensions are shown in [Figure 12](#).



XY2-100 Converter (Accessory): dimensions.

The **XY2-100 Converter (Accessory)**'s 9-pin D-SUB connector should be directly plugged into the RTC5's first scan head connector or (by a short-as-possible 1:1 cable) connected to the corresponding pins of the RTC5's second scan head connector. For the pin-out, see [Figure 10](#) and [Figure 11](#).

The 25-pin D-SUB connector of the **XY2-100 Converter (Accessory)** is compatible with scan heads that provide an XY2-100 standard interface. The pin-out is shown in [Figure 13](#).



XY2-100 Converter (Accessory): pin-out of the 25-pin D-SUB connector (female).

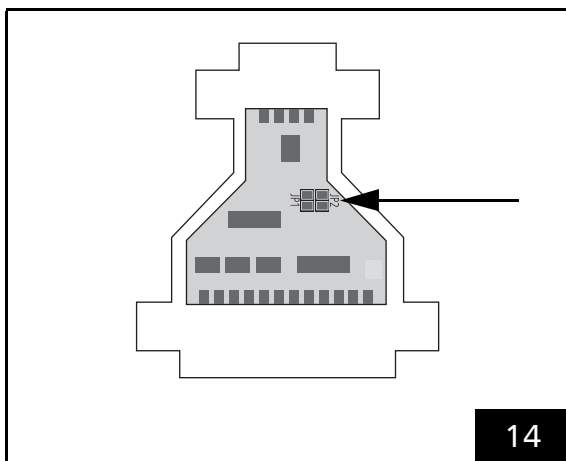
* For iDRIVE scan systems (intelliSCAN, intelliSCAN_{de}, intelliDRILL, intellcube, intelliWELD, varioSCAN_{de}), the STATUS $\pm$ channel is the status channel of axis 2 (x axis; this channel is then also called STATUS2 $\pm$) and the STATUS1 $\pm$ channel is the status channel of axis 1 (y axis). For other scan systems, the STATUS1 $\pm$ channel can not be used: "DO NOT CONNECT".

The data channels CHAN1 and CHAN2 transmit control values to the scan head. The SYNC and CLOCK channels transmit synchronization and clock signals to the scan system. The STATUS channel (and, if appropriate, the STATUS1 channel) receives XY2-100 compliant status signals returned by the scan system.

If cables longer than 20 m (not recommended) are used for data transmission between the **XY2-100 Converter (Accessory)** and the scan system, then the synchronization of bidirectional communication between the scan system and the RTC5 should be configured.

This can be accomplished by two solder jumpers in the **XY2-100 Converter (Accessory)**.

To do so, carefully open the housing of the **XY2-100 Converter (Accessory)** by its 4 clip latches⁽¹⁾. The solder jumpers **JP1** and **JP2** are on the PCB, see arrow in **Figure 14**.



XY2-100 Converter (Accessory): positions of solder jumpers **JP1** and **JP2** on the PCB.

The following table shows the possible jumper settings and the corresponding cable lengths. Other jumper settings are not allowed. Cable lengths above 20 m are not recommended.

Jumper setting	Cable length*
JP1 closed JP2 open (default configuration) 	0 m to 20 m
JP1 open JP2 closed 	20 m to 40 m
* The cable length range mentioned here for the respective jumper configuration depends on the used cable type and may differ from the values mentioned above. Cable lengths over 20 m are generally not recommended (and require extensive checks by users prior to productive use).	

Notes

- Observe the note on **get_head_status**, page 199.

(1) For example, using a slotted screwdriver.

4.5.3 Data Cables (Accessories)

For transmission of the data signals between the RTC5 (or the **XY2-100 Converter (Accessory)**) and the scan system, appropriate cables are obtainable from SCANLAB. Data cables are generally not included in the scope of delivery.

For SCANLAB scan systems equipped with an SL2-100 interface (9-pin female D-SUB connector), data cables are available for electrical transmission, for optical transmission also optical fiber cables⁽¹⁾ (only upon request).

For SCANLAB scan systems equipped with a conventional XY2-100 interface (25-pin female D-SUB connector) solely electrical cables are obtainable.

Scan systems equipped with an optical interface (XY2-100-O) cannot be controlled by the RTC5.

With self-constructions, SCANLAB recommends the following design for electrical data transmission:

- For SL2-100-compliant data transmission, see **Figure 15**, the cable should be fitted with 9-pin male D-SUB connectors at both ends. The two channels DATA IN $\pm$ and DATA OUT $\pm$ must consist of twisted cable pairs and be *cross-connected* at both D-SUB connectors (for example, so that the RTC5's DATA OUT signal flows to the scan system's DATA IN input). The cable length should not exceed 25 m. SCANLAB recommends a cable impedance of 110 Ω , independent from the cable length.

- For XY2-100-compliant data transmission, see **Figure 16**, the cable must have identical 25-pin (male) D-SUB connectors at both ends. The five (or six) channels SYNC $\pm$, CHAN1 $\pm$, CHAN2 $\pm$, STATUS $\pm$ (and STATUS1 $\pm$) and CLOCK $\pm$ must consist of twisted cable pairs. Together with an **XY2-100 Converter (Accessory)** with standard jumper configuration, the data cable should not be longer than 20 m. If a longer data cable is needed, then the solder jumper setting of the **XY2-100 Converter (Accessory)** need to be reconfigured.
- For XY2-100-compliant data transmission, the controller end of the data cable must be fitted with a ferrite ring (for example, Würth WE 74271132).

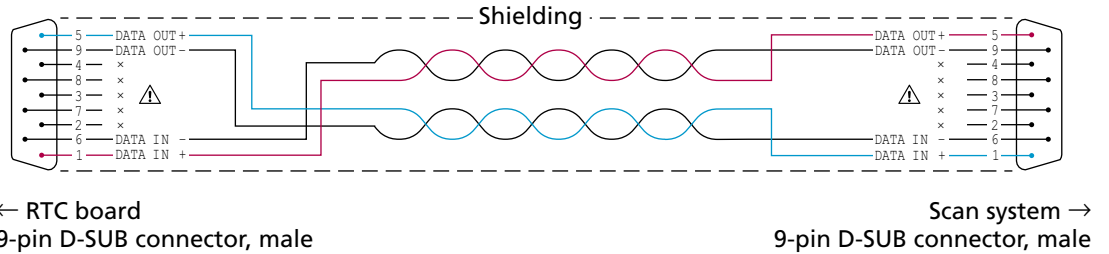
Independently from the data transmission protocol, the following applies in addition:

- The data cable must have coaxial copper braided shielding.
- The D-SUB connectors must have fully shielded metal housings.
- The electrical connection of the cable's braided shielding to the D-SUB housing should *not* be implemented as a wire. Instead, the cable's braided shielding should be *coaxially* connected to the D-SUB housing by shielded clamps.

Some scan heads have a single connector which provides both the power supply and the data signals. For these scan heads, SCANLAB recommends using a Y cable (voltage and data are to be transferred in separate cables; on the condition see above).

(1) The optical fiber cables, too, are attached by 9-pin D-SUB connectors. Optical conversion (Polymer Optical Fiber conversion) for optical data transmission takes place inside the D-SUB connectors. The operating voltage for Polymer Optical Fiber conversion is supplied at the RTC5 scan head connectors and the digital interface of the scan system.

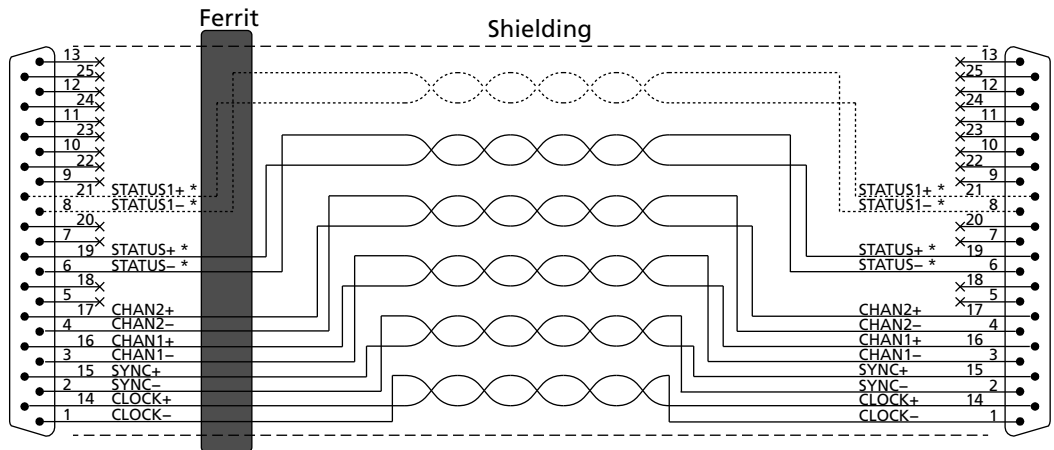
Cable for SL2-100-compliant data transmission



15

Cable for data transmission compliant to SL2-100 protocol: requirements and pin assignments:

Cable for XY2-100-compliant data transmission



- * For iDRIVE scan systems (see Glossary entry on [page 23](#)), the following applies:
The STATUS± channel is the status channel for axis 2 (x axis; this channel is also called STATUS2± there).
The STATUS1± channel is the status channel for axis 1 (y axis).
For other scan systems, the STATUS1± channel is not needed.

16

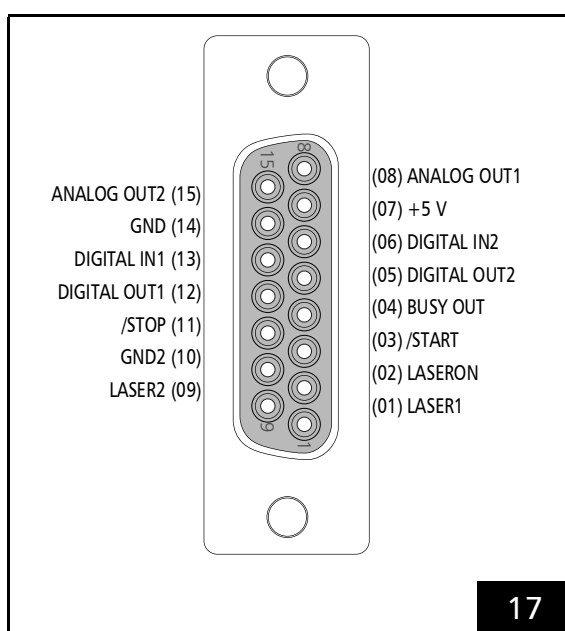
Cable for data transmission compliant to XY2-100 protocol: requirements and pin assignments.

4.6 Interfaces for the Laser and Peripheral Equipment

4.6.1 LASER Connector

The LASER connector is a 15-pin D-SUB connector (female). It is located on the slot cover of the RTC5 PCI Board, see [Figure 5](#).

The pin-out is shown in [Figure 17](#).



LASER connector (15-pin D-SUB connector, female): pin-out. On RTC5 PCI Boards without [Option "DC/DC Converter"](#), GND2 are GND identical.

Notice!

- If you want to use the RTC5 PCI Board in conjunction with laserDESK, observe the following:
 - laserDESK uses additional pins on the [EXTENSION 1 Socket Connector](#), for example, to control the laser.
 - Refer to the extended documentation for the LASER connector which can be found in the laserDESK online help (alternatively available from SCANLAB or in laserDESK-zip, which can be downloaded from the SCANLAB website).

Laser Control Signals

The laser control signals LASER1 and LASER2 at pin (1) and pin (9) depend on the selected laser control mode, see the corresponding timing diagrams in [Chapter 7.4 "Laser Control", Page 173](#):

	LASER1 pin (01)	LASER2 pin (09)
CO ₂ Mode	Modulation pulse 1, standby signal	Modulation pulse 2, standby signal
YAG Mode 1, YAG Mode 2, YAG Mode 3, YAG Mode 5	Q-Switch signal	FirstPulseKiller signal
Laser Mode 4	Standby signal	FirstPulseKiller signal
Laser Mode 6	Standby signal	–

All laser control signals (LASERON, LASER1 and LASER2) are permanently generated by the RTC5 Board. They are digital TTL level signals and are referenced to GND2.

On RTC5 boards with the [Option "DC/DC Converter"](#) the GND2 and the laser output signal are galvanically decoupled from GND⁽¹⁾. On RTC5 boards without [Option "DC/DC Converter"](#), GND2 are GND identical, see [Chapter 2.6 "Options", Page 34](#).

For the maximum current load see [Chapter 15 "Technical Specifications – RTC5 PCI Board", Page 734](#).

All laser signals can be set to either active-LOW or active-HIGH logic by [set_laser_control](#). active-LOW means that a logical 1 ("Laser On", for instance) is represented by a LOW level (0 V, TTL). active-HIGH means a logical 1 is represented by a HIGH level (+5 V, TTL). Set the TTL laser signal level according to the specifications of your laser control. Observe the documentation of your laser.

[config_laser_signals](#) and [config_laser_signals_list](#) can be used to configure pins (1), (2) and (9) of the laser connector, see [Chapter 7.4.2 "Configuring the LASER Connector", Page 176](#).

(1) With RTC5 boards, GND is the PC ground.

External Control Signals

The external control signals are /START and /STOP (TTL active-LOW). Both input signals are connected internally to +3.3 V by pull-up resistors (4,7 k Ω), see [Figure 18](#). The signals are referenced to GND⁽¹⁾. See also [Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization"](#), Page 265.

BUSY List Execution Status

The **BUSY list execution status** is available as the BUSY OUT signal at pin (04). When the **BUSY list execution status** is set, the BUSY OUT signal is HIGH. See also [Chapter 6.4.3 "List Execution Status"](#), Page 97. The signal is referenced to GND⁽¹⁾.

2-Bit Digital Input Port

The RTC5 PCI Board provides a 2-bit digital input port (DIGITAL IN1 and DIGITAL IN2).

For technical data see [Chapter 15 "Technical Specifications – RTC5 PCI Board"](#), Page 734.

For programming the input port see [Chapter 9.2.2 "2-Bit Digital Input Port"](#), Page 264.

2-Bit Digital Output Port

The RTC5 PCI Board provides a buffered 2-bit digital output port (DIGITAL OUT1 and DIGITAL OUT2).

For technical data see [Chapter 15 "Technical Specifications – RTC5 PCI Board"](#), Page 734.

For programming the output port see [Chapter 9.1.3 "2 Bit Digital Output Port"](#), Page 259.

12-Bit Analog Output Port 1 and 2

The RTC5 PCI Board provides two 12-bit analog output ports:

- ANALOG OUT1
- ANALOG OUT2⁽²⁾

For technical data see [Chapter 15 "Technical Specifications – RTC5 PCI Board"](#), Page 734.

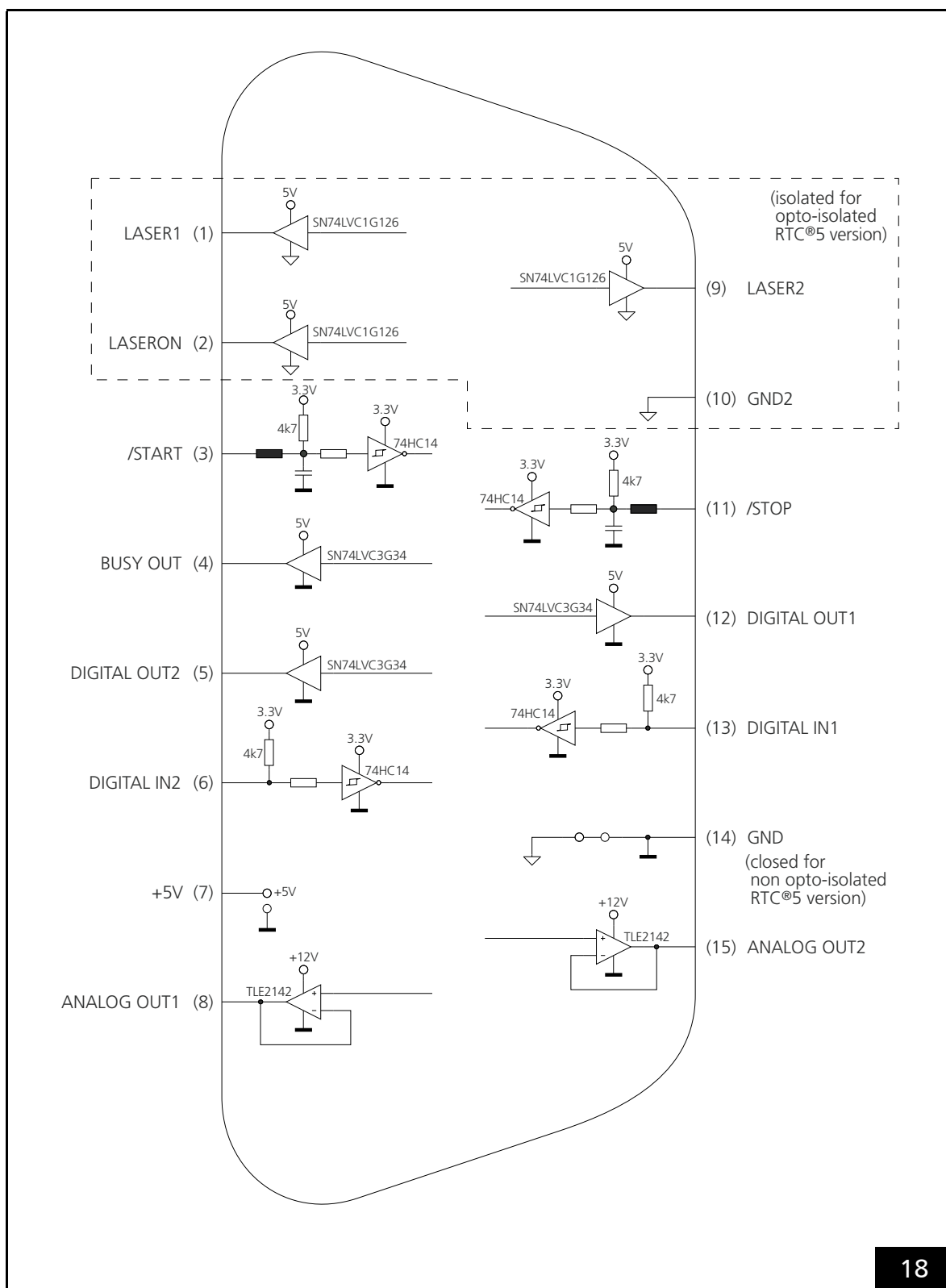
For programming the outputs see [Chapter 9.1.4 "12-Bit Analog Output Port 1 and 2"](#), Page 259.

Input/Output Wiring

The input/output wiring of the LASER connector is shown in [Figure 18](#).

(1) See footnote on [page 60](#).

(2) The signal of **ANALOG OUT2** is also available by the MARKING ON THE FLY socket connector, see [Section "Analog Output Port"](#), Page 69.



LASER connector, see also [Figure 17: input/output wiring](#).
See also [Section "Option "DC/DC Converter""](#), [Page 34](#).

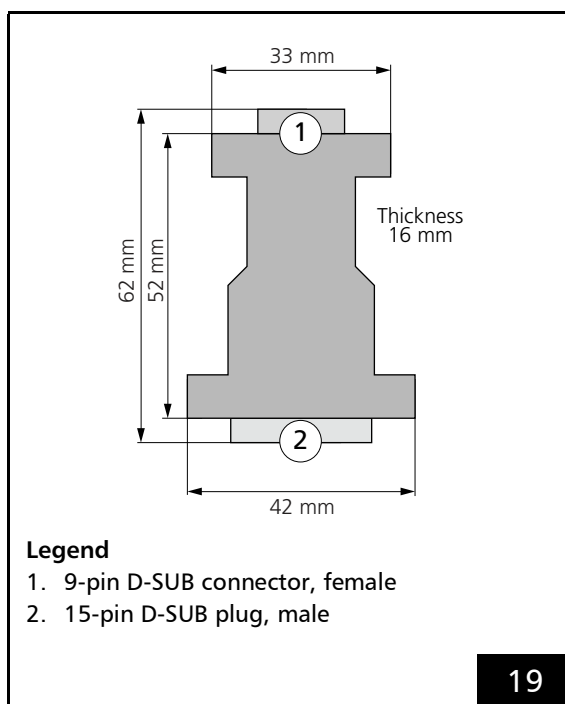
Laser Adapter (Accessory)

The SCANLAB laser adapter (accessory; #0114774) is plugged into the 15-pin LASER connector of the RTC5 PCI Board. Then its 9-pin D-SUB connector (female) provides the same signals and pin-out as the RTC4 9-pin LASER connector.

Notes

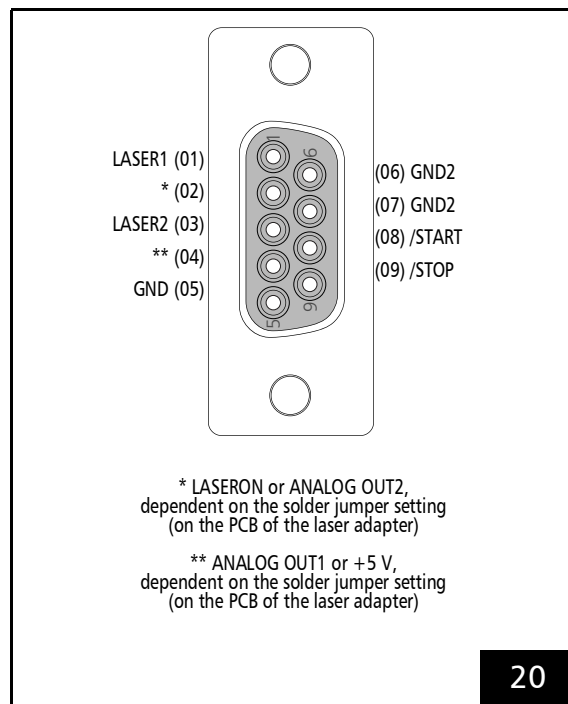
- The laser adapter can *not* be directly plugged into the RTC5 PCI Board, if an **XY2-100 Converter (Accessory)** is already plugged in.
- The BUSY OUT signal, 2-bit digital input and 2-bit digital output are *not* available at the laser adapter's 9-pin D-SUB connector.

The dimensions of the laser adapter is shown in **Figure 19**.



Laser adapter (accessory): dimensions.

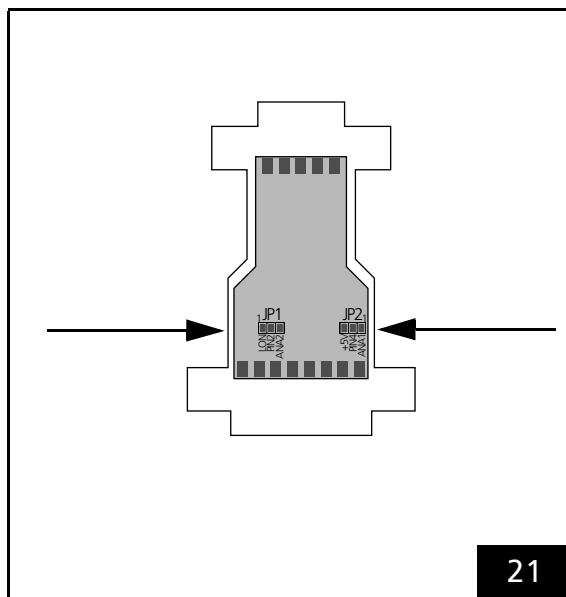
The pin-out of the 9-pin D-SUB connector is shown in **Figure 20**.



Laser adapter (accessory): pin-out of the 9-pin D-SUB connector, female.

The signals at pins (02) and (04) of the 9-pin D-SUB connector can be selected by two solder jumpers in the laser adapter. To do so, carefully open the laser adapter's housing by its 4 clip latches (for example, using a screwdriver). The solder jumpers **JP1** and **JP2** are on the PCB of the laser adapter between the two D-SUB connectors.

The positions of the solder jumpers on the PCB is shown in **Figure 21**.



Laser adapter (accessory): position of solder jumper JP1 and JP2 on the printed circuit board.

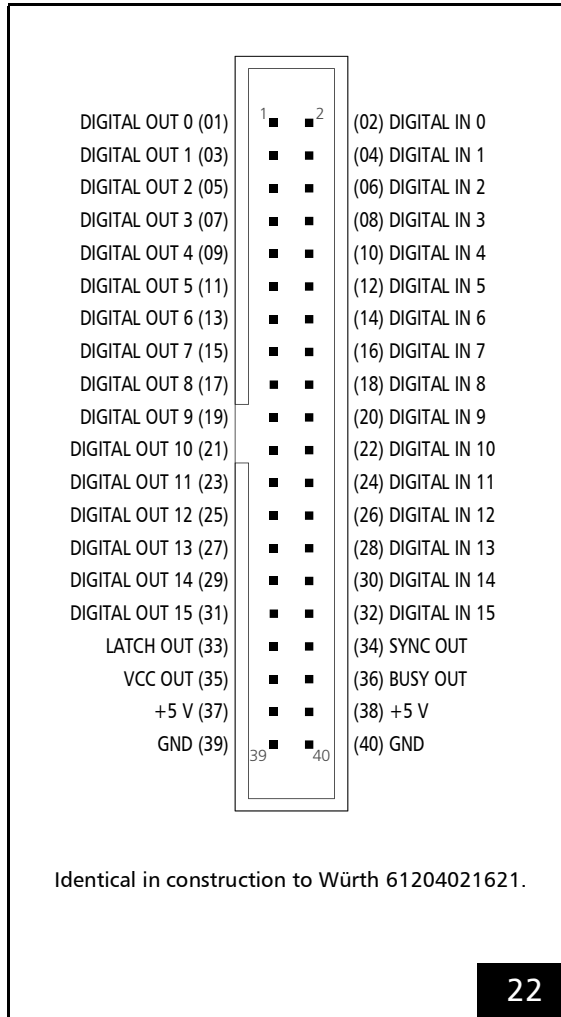
The following table shows the possible jumper settings. Other jumper settings are not allowed.

Solder jumper JP1: signal at pin (02)	Position 1-2  1 02 04 LON PIN2 ANA2 LASERON (default configuration)	Position 2-3  1 02 05 LON PIN2 ANA2 ANALOG OUT2
Solder jumper JP2: signal at pin (04)	Position 1-2  1 04 05 +5V PIN4 ANA1 ANALOG OUT1 (default configuration)	Position 2-3  1 04 06 +5V PIN4 ANA1 +5 V

4.6.2 EXTENSION 1 Socket Connector

The EXTENSION 1 socket connector has 40 pins. It is located on the upper side of the RTC5 PCI Board, see [Figure 5](#).

The pin-out is shown in [Figure 22](#).



Configuring the Output Signal Level

By Jumper JP1 on the lower side of the RTC5 PCI Board, see [Figure 6](#), the level of all output signals at the EXTENSION 1 socket connector (DIGITAL OUT0-15, LATCH_OUT, SYNC_OUT, BUSY_OUT, VCC_OUT) can be configured for 5 V or 3.3 V, see [Chapter 2.7.1 "Jumper JP1 – Configuring the Output Signal Level at the EXTENSION 1 Socket Connector"](#), [Page 36](#).

For user monitoring purposes, the selected signal level is also continuously outputted at pin (35): signal VCC_OUT. VCC_OUT is referenced to PC ground (GND). The maximum current load is 100 mA.

16-Bit Digital Input Port and 16-Bit Digital Output Port

The 40-pin EXTENSION 1 socket connector provides a 16-bit digital TTL input and a buffered 16-bit digital TTL output, see [Figure 22](#). This requires the output signals level to be configured by the jumper setting, see [Section "Configuring the Output Signal Level"](#), [Page 65](#).

For programming see:

- [Chapter 9.1.1 "16-Bit Digital Output Port"](#), [Page 258](#)
- [Chapter 9.2.1 "16-Bit Digital Input Port"](#), [Page 264](#)
- [Chapter 9.3.2 "Conditional Command Execution"](#), [Page 271](#)

The input and output signals are referenced to PC ground (GND). The maximum current load of the output signals is 8 mA.

EXTENSION 1 socket connector: pin-out. The pitch of the pins is 2.54 mm.

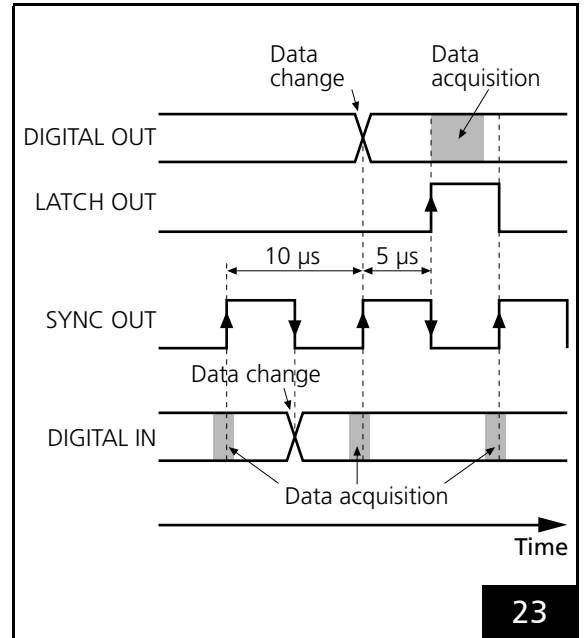
Synchronization of Data Transmission

If several bits are simultaneously transferred as a data words (and not independently from each other) by the 16-bit digital output port or the 16-bit digital input port, then the LATCH or SYNC signal (respectively) should be used for synchronization of data transmission.

The LATCH signal (a 5 μ s pulse, active-HIGH) is outputted at pin (33) as a trigger signal for acquiring the output values of the 16-bit digital output port. The RTC5 automatically generates the LATCH signal when the output value at the 16-bit digital output port is outputted. The output value should be read-out with the rising edge of the LATCH signal, which is generated 5 μ s after the value has been outputted at the 16-bit digital output port (see [Figure 23](#)).

To synchronize data transmission at the 16-bit digital input port, pin (34) continuously provides a SYNC signal (a square wave with 5 μ s pulse length and 10 μ s period). Value changes at the 16-bit digital input port should be made with a falling edge of the SYNC signal. The RTC5 **DSP** always accepts the currently provided value with the rising edge of the SYNC signal (see [Figure 23](#)).

The synchronization signals are referenced to PC ground (GND). The maximum current load is 10 mA.



Synchronization of data acquisition by LATCH OUT signal or SYNC OUT signal.

BUSY List Execution Status

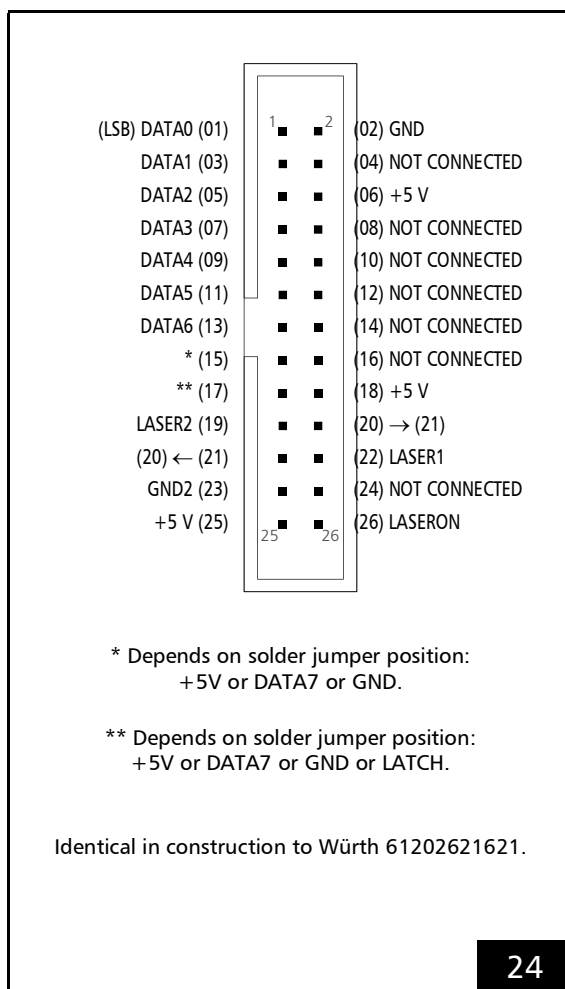
The BUSY OUT signal at pin (36) is identical to the BUSY OUT signal at the LASER connector (see [Section "BUSY List Execution Status", Page 61](#)). The signal is referenced to GND.

4.6.3 EXTENSION 2 Socket Connector

The EXTENSION 2 socket connector⁽¹⁾ has 26 pins. It is located on the upper side of the RTC5 PCI Board, see [Figure 5](#).

It provides a buffered 8-bit digital output port: DATA0...DATA7.

The pin-out is shown in [Figure 24](#).



EXTENSION 2 socket connector: pin-out. The pitch of the pins is 2.54 mm.

Notes

- Pin (15) and pin (17) are configurable by solder jumpers.

Configuration by Solder Jumpers

Pins (15) and (17) of the EXTENSION 2 socket connector have to be configured by the jumpers JP2...JP8 on the lower side of the RTC5 (see [Figure 6](#)). The most significant bit (DATA7) of the 8-bit output value can be assigned to pin (15) or to pin (17). Alternatively, each of the two pins can be set permanently to +5 V (HIGH) or to GND (LOW level). Alternatively, pin (17) can be configured for the LATCH signal (see below) by closing the correspondingly labeled jumper (see [Chapter 2.7.2 "Jumper JP2...JP8 – Configuring Pin15 and Pin17 at the EXTENSION 2 Socket Connector", Page 36](#)).

Notes

- If the DATA7 bit is assigned to pin (15), the full 8-bit output value is available at the output port (odd numbered pins (1) to (15) of the EXTENSION 2 socket connector).
- Setting pin (15) permanently to HIGH results in an offset of 128 for the output value and restricts the output value range to (128...255).
- Setting pin (15) to LOW restricts the output value range to 0...127.
- The DATA7 bit can be used for other purposes by assigning it to pin (17).

(1) On the RTC4, the functional corresponding socket connector is labeled LASER EXTENSION.

Laser Control Signals

Like the laser signals of the LASER connector, the output signals LASER1 and LASER2 depend on the selected laser control mode (see [Section "Laser Control Signals", Page 68](#)) and are referenced to GND2. On RTC5 Boards with the [Option "DC/DC Converter"](#) the GND2 and the laser output signal are galvanically decoupled from the PC ground (GND). Otherwise, GND2 are GND identical (see [Chapter 2.6 "Options", Page 34](#)).

8-Bit Digital Output Port

The buffered 8-bit digital output port (TTL level) is intended for YAG lasers with a digital lamp current control. However, it can be used for any other purpose as well. The output is in high-impedance mode until an initial value is assigned to it.

For programming the output see [Chapter 9.1.2 "8-Bit Digital Output Port", Page 259](#).

The most significant bit ([MSB](#)) (DATA7) of the output value can be used for other purposes. To do this, the [MSB](#) can be assigned to an extra pin on the EXTENSION 2 socket connector (see above).

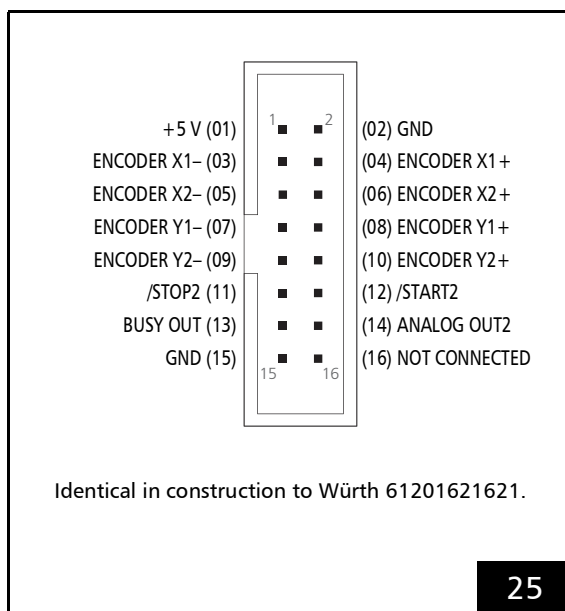
By an appropriate jumper setting (see above), Pin (17) can output a LATCH signal. The RTC5 automatically generates the LATCH signal (a 5 μ s pulse, active-HIGH) when the value at the 8-bit digital output port has been outputted. If several bits are simultaneously transferred as a data word (that is, if the bits are not transferred independently from each other) by the 8-bit digital output port, then the LATCH signal should be used for synchronization of data transmission (for example, as a trigger signal for the laser to assume a different lamp current). The value at the 8-bit digital output port should be read-out with the rising edge of the LATCH signal, which is generated 5 μ s after the value at the 8-bit digital output port has been outputted (see also [Figure 23](#)). The LATCH signal is referenced to PC ground (GND). The maximum current load is 10 mA.

4.6.4 MARKING ON THE FLY Socket Connector

The MARKING ON THE FLY socket connector provides, among other things, encoder input ports for incorporating encoder signals.

Together with the optional Processing-on-the-fly-functionality, this enables laser material processing of moving workpieces (for example, on a moving conveyor belt or rotating disk, see [Chapter 8.6 "Processing-on-the-fly", Page 227](#)).

The pin-out is shown in [Figure 25](#).



Encoder Input Ports

The RTC5 PCI Board provides 2 encoder input ports:

- ENCODER X
- ENCODER Y

ENCODER X and ENCODER Y are each designed for a pair of standardized differential input signals (RS-422; HIGH level ≥ 2.0 V, LOW level ≤ 0.8 V). See also [Section "Input Ports for External Encoder Signals", Page 275](#).

External Control Signals

The external control signals /START2 and /STOP2 (TTL active-LOW) are referenced to PC ground (GND). Both input signals are connected internally to +3.3 V by pull-up resistors. See also [Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization", Page 265](#).

Analog Output Port

The 12-bit analog output port ANALOG OUT2 at pin (14) is identical to the ANALOG OUT2 output port at the LASER connector, see also [Section "12-Bit Analog Output Port 1 and 2", Page 61](#).

The signal is referenced to GND.

BUSY list execution status Status

The BUSY OUT signal at pin (13) is identical to the BUSY OUT signal at the LASER connector, see [Section "BUSY List Execution Status", Page 61](#).

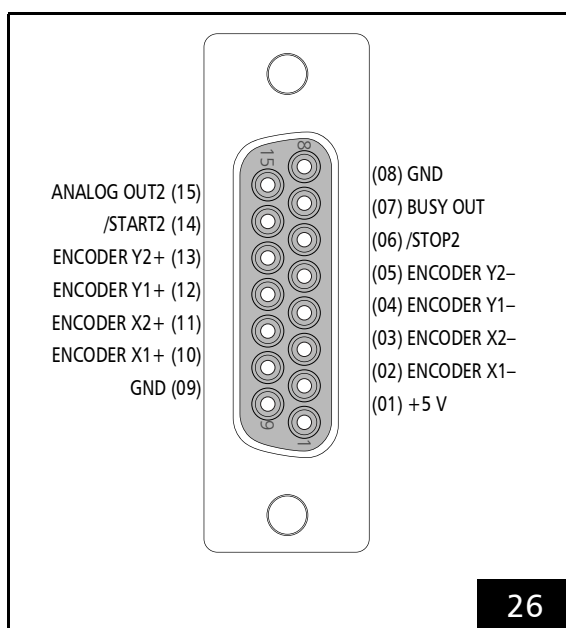
The signal is referenced to GND.

MARKING ON THE FLY socket connector: pin-out. The pitch of the pins is 2.54 mm.

MARKING ON THE FLY Slot Cover (Accessory)

SCANLAB recommends using an additional slot cover for using the inputs and signals of the **MARKING ON THE FLY Socket Connector**, see Chapter 2.8.5 "Slot Cover with 15-pin D-SUB Connector for MARKING ON THE FLY Socket Connector", Page 38.

The pin-out is shown in Figure 26.



MARKING ON THE FLY Slot Cover (Accessory), 0109272: pin-out of the 15-pin female D-SUB connector.

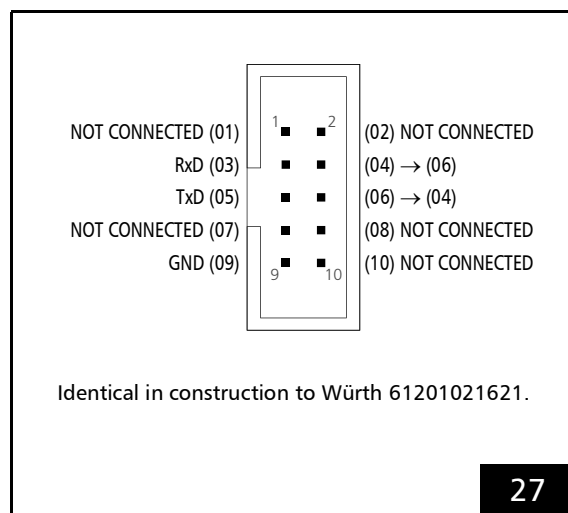
Alternatively, the **Slot Cover with 15-pin D-SUB Connector and 9-pin D-SUB Connector**, Page 39 is also available.

4.6.5 RS232 Socket Connector

The RS232 socket connector has 10 pins and is located on the upper side of the RTC5 PCI Board, see Figure 5.

The RS232 socket connector is a hardware interface designed for bidirectional data exchange.

The pin-out is shown in Figure 27.



RS232 socket connector: pin-out. The pitch of the pins is 2.54 mm.

Max. input voltage range: -25 V...+25 V

Max. output voltage range: -13 V...+13 V

All signals are referenced to GND.

rs232_config can be used to set the baud rate (default setting: 9600 baud). Other parameters cannot be altered (data bits: 8, start bits: 1, stop bits: 1, parity: none).

For outputting data by the RS-232 interface, see Chapter 9.1.6 "RS-232 interface", Page 263.

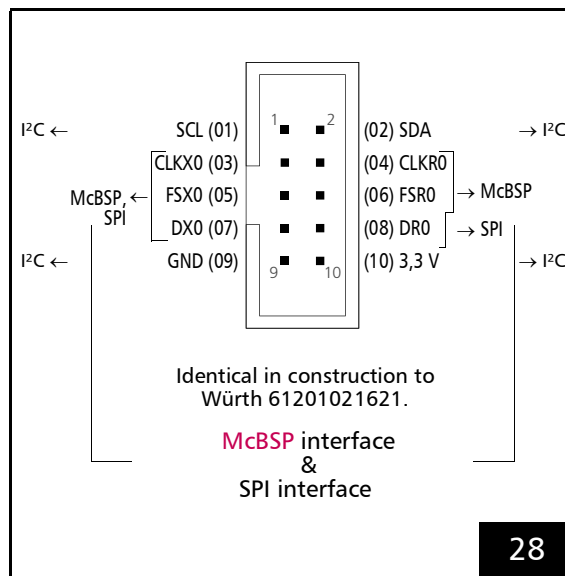
For reading-in data by the RS-232 interface, see Chapter 9.2.3 "RS-232 Interface", Page 264.

4.6.6 SPI / I²C Socket Connector

The SPI/I²C socket connector is a hardware interface for bidirectional data exchange:

- **McBSP**
- **SPI**
- **I²C** (Inter-Integrated Circuit)

The pin-out is shown in **Figure 28**.



SPI/I²C socket connector: pin-out. The pitch of the pins is 2.54 mm.

McBSP Interface

The **McBSP interface**, see **Figure 28**, is initialized by **load_program_file** with:

- $XDelay = RDelay = 1$
- 8 MHz clock frequency

Intended for **McBSP** are pin (03), (05), (07) (outgoing signals) and pin (04), (06), (08) (incoming signals).

The following signals are continuously outputted after **load_program_file**:

- the clock signal at pin (03)
- every 10 μ s a frame synchronization signal FSX at pin (05)
- a data signal DX at pin (07) (Default: SampleY|SampleX), see **set_mcbbsp_out**

The **McBSP interface** can be integrated into applications for various purposes:

- As an alternative to encoder signals, the position of a moving workpiece can be directly integrated, see also **Chapter "Correction via McBSP Interface"**, Page 230. Prerequisite: **Option Processing-on-the-fly**, see **Chapter 2.6 "Options"**, Page 34.
- As an alternative to matrix and offset commands, coordinate transformations can be transmitted and integrated to align a workpiece with controllable timing (**Online Positioning**), see also **Chapter 9.3.4 "Synchronization and Online Positioning by McBSP/SPI Signals"**, Page 276, and **"Global Online Positioning"**.
- With **set_multi_mcbbsp_in** you can transfer not only a 3D fly correction with position information but also laser power and other parameters, see also **Section "Correction via McBSP Interface with Additional McBSP Input"**, Page 231. Prerequisite: **Option Processing-on-the-fly**, see **Chapter 2.6 "Options"**, Page 34.
- Permanent data output in 10 μ s cycles. With **set_mcbbsp_out** and **set_mcbbsp_out_ptr** it can be selected which data signals are outputted.

For data output using the **McBSP interface**, see **Chapter 9.1.7 "McBSP Interface"**, Page 263.

For data input using the **McBSP interface**, see **Chapter 9.2.4 "McBSP Interface"**, Page 264.

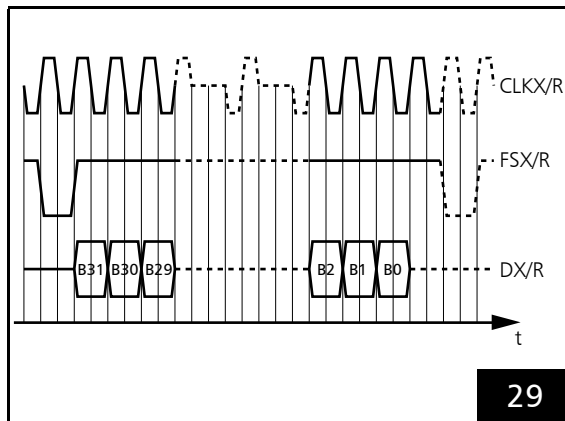
mcbbsp_init allows setting a data delay for the **McBSP** data transmission independently for the transmitter and receiver. Possible values range from 0 to 2 (default: 1).

All signals are referenced to GND.

RTC5 as Transmitter

The following specifications apply to CLKX0, FSX0 and DX0:

- Signal level 3.3 V TTL
- McBSP mode:
 - Single phase frame
 - Single element per frame
 - 32 bits per element
 - Data delay X_{Delay} bits
- The timing diagram of the McBSP signals is shown in **Figure 29**. A frame synchronization signal (active-LOW) is generated upon the rising edge of the clock and held for one clock cycle (1 data bit). The 32 data bits are transmitted after X_{Delay} clock cycles at the rising edges of the clock and in the sequence Bit #31...Bit #0. The transmission frequency is by default 8 MHz (4 μ s per data word). Alternatively, a value between 4 MHz and 16 MHz can be set by **set_mcbbsp_freq**.



Timing diagram of **McBSP** signals at 1 bit data delay (default: $X_{Delay} = R_{Delay} = 1$).

RTC5 as Receiver

The following specifications apply to CLKR0, FSR0, DR0:

- Signal level 3.3 V or 5 V TTL.
- McBSP mode:
 - Single phase frame
 - Single element per frame
 - 32 bits per element
 - Data delay R_{Delay} bits
- The timing diagram of the McBSP signals is shown in **Figure 29**. The frame synchronization signal (active-LOW) generated upon a rising edge (of an external clock signal) is detected upon the clock's next falling edge (trailing edge of the external clock pulse). The duration of the frame synchronization signal is irrelevant. After R_{Delay} clock cycles the 32 data bits are detected with each additional falling edge of the clock signals in the sequence Bit #31...Bit #0, provided they are transmitted with rising edges.

Take account of the following information:

- The **McBSP interface** always ignores the first frame synchronization signal after a **load_program_file** or **mcbbsp_init**. Therefore, the data provided is not transmitted. If necessary, a dummy value should be initially sent to the interface. You can use **read_mcbbsp** to check for successful transmission.
- The bit frequency (receiving frequency) is exclusively determined by the incoming clock pulses and has a maximum limit of 16 MHz.
- The last data bit (Bit #0) must be followed by transmission of at least one additional external clock cycle to ensure that the interface's **DSP** side acquires and buffers the data word. Simultaneously with this clock cycle, you can already initiate another new transfer by a frame synchronization signal.

- After a **load_program_file**, the McBSP data is internally permanently acquired and buffered as soon as (new) data is transmitted. Newer data words overwrite older ones.
 - After a reset by **load_program_file** and/or after activation of Processing-on-the-fly correction by **set_fly_x_pos**, **set_fly_y_pos** or **set_fly_rot_pos**, the input values are stored to internal memory location 0.
 - After activation of an **Online Positioning** the input values are stored to internal memory locations 1 or 2
 - For a “Local Online Positioning” by **set_mcbasp_x**, **set_mcbasp_y** and/or **set_mcbasp_rot** or **set_mcbasp_matrix**, see Section “Configuring “Local Online Positioning””, Page 214
 - For a “Global Online Positioning” by **set_mcbasp_global_x**, **set_mcbasp_global_y** and/or **set_mcbasp_global_rot** or **set_mcbasp_global_matrix**, see Section “Configuring “Global Online Positioning””, Page 216
 - The most recent fully transferred values can be queried from a corresponding memory location by **read_mcbasp**.
- After activation of Processing-on-the-fly correction by **set_mcbasp_in** or **set_mcbasp_in_list**, the input values are transferred to internal memory locations 0 and 3.
- After activation of Processing-on-the-fly correction by **set_multi_mcbasp_in** or **set_multi_mcbasp_in_list**, the input values are transferred to internal memory locations 0 through 3.

I²C Interface

The SPI/I²C socket connector is used to transmit the digital data to the **DSP**, when the **ADC Add-On Board** is operated, see also Chapter 4.6.8 “Analog Inputs (Accessory)”, Page 75.

At present, the I²C clock cycle signals (SCL at pin (1) and I²C data signals (SDA at pin (2) are not discretionary for other purposes.

SPI Interface

The SPI/I²C socket connector can be set-up for a serial synchronous data transmission, if required. Then **SPI** functionality is used instead of the McBSP functionality (default). For this purpose, **mcbasp_init_spi** can be called at any time. A data transmission which is in progress is canceled by that. The previous state (McBSP functionality) can be reestablished with **mcbasp_init**.

After initialization only one data word can be transmitted every 10 μ s, for example, see **set_mcbasp_in**.

The default transmission frequency is 8 MHz (4 μ s per data word). Alternatively, a value between 4 MHz and 16 MHz can be set. For this purpose, **set_mcbasp_freq** is called (see also **mcbasp_init** or **mcbasp_init_spi**).

After **mcbasp_init_spi** has been called the RTC5 acts as **SPI** master device in terms of the **SPI** standard. The RTC5 is a master device for a single **SPI** slave only.

Necessary for operation are:

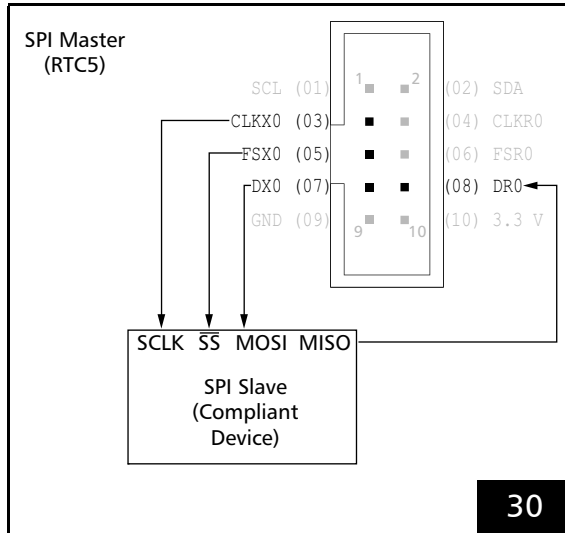
- 1 control line for the clock signal (CLKX0 at pin (3))
- 1 control line for the active-LOW slave select signal (FSX0 at pin (5))
- 1 data line – from a master point of view – outgoing (DX0 at pin (7))
- 1 data line – from a master point of view – incoming (DR0 at pin (8))

Pin (4) CLKR0 and pin (6) FSR0 are not connected.

The specification 3.3 V TTL or 5 V TTL applies for all signal levels.

The signal sequence is the same as with the McBSP mode, though **XDelay** and **RDelay** are *always* = 1.

The electrical connection of the RTC5 and an **SPI** slave device is shown in Figure 30.



RTC5 as **SPI** master with **SPI** slave – signals and electrical connection.

SCLK = Serial Clock, MOSI = Master Output - Slave Input, MISO = Master Input - Slave Output, SS = Slave Select. The relevant abbreviations and designations are inconsistent in technical references. That is, they are arbitrarily chosen in this document and a variety of synonyms does exist.

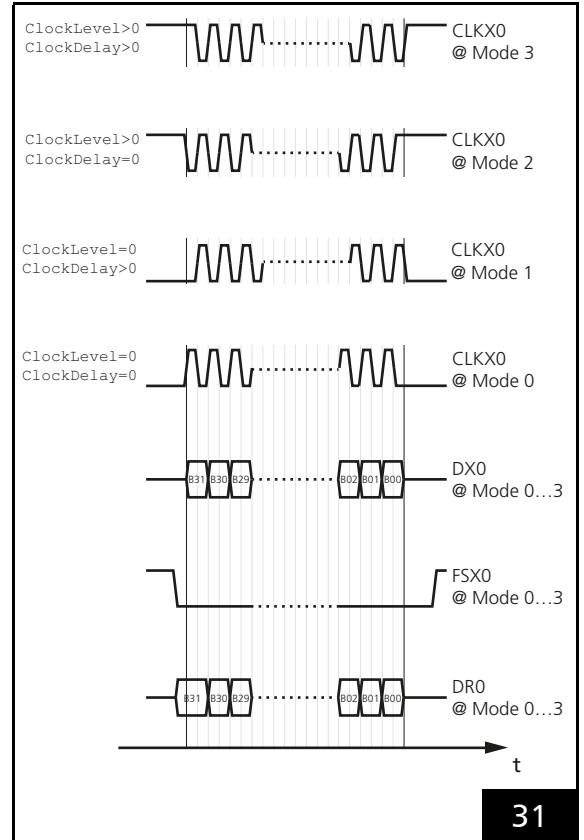
With **SPI** (in contrast to McBSP data transmission), data input and data output take place synchronously together with a clock signal (the clock signal is generated by the **SPI** master). The **SPI** timing diagram is shown in **Figure 31**.

The clock signal properties 'polarity' (ClockLevel) and 'phase' (ClockDelay) can be set by **mcbsp_init_spi**. The resulting data transmission formats are designated Mode 0...Mode 3 according to the **SPI** standard, see the following table.

SPI mode	Clock signal polarity	Clock signal phase
0	clock idle LOW – ClockLevel = 0	is simultaneous with data bits – ClockPhase = 0
1	clock idle LOW – ClockLevel = 0	is delayed a half clock signal – ClockPhase > 0
2	clock idle HIGH – ClockLevel > 0	is simultaneous with data bits – ClockPhase = 0
3	clock idle HIGH – ClockLevel > 0	is delayed a half clock signal – ClockPhase > 0

All signals are referenced to PC ground GND.

In contrast to the McBSP configuration, immediately after initialization all signals at the corresponding pins are static and the RTC5 does not generate a clock signal permanently.

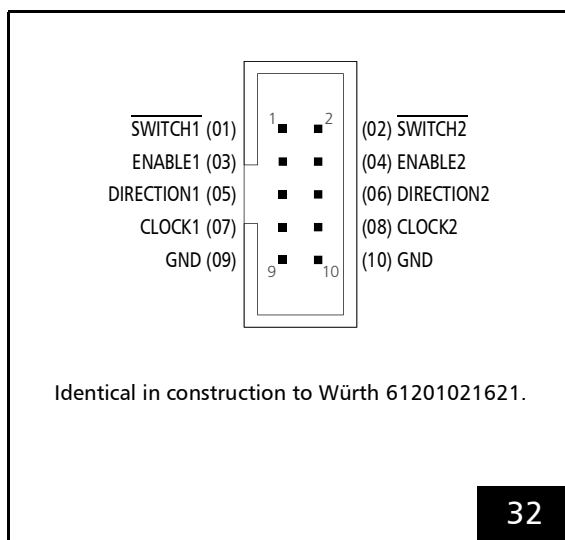


Timing-Diagram of the **SPI** signals. The clock signal differs in regards to polarity and phase for mode 0, 1, 2 and 3. Grid spacing is a *half* clock signal.

4.6.7 STEPPER MOTOR Socket Connector

The STEPPER MOTOR socket connector can supply signals for controlling two stepper motors⁽¹⁾.

The pin-out is shown in [Figure 32](#).



STEPPER MOTOR socket connector: pin-out. The pitch of the pins is 2.54 mm.

All signals are referenced to PC ground GND.

The output signals (ENABLE, DIRECTION and CLOCK) are TTL-level signals (5 V). The RTC5 generates a periodic clock signal of active-HIGH 5 μ s pulses (the CLOCK signal is permanently LOW between these pulses). With the ENABLE signals a user program can, for example, switch the motor current on and off. The DIRECTION signals can set the direction and each CLOCK pulse can be used to execute a single step.

The SWITCH inputs (active-LOW) are connected internally to +3.3 V by 10 k Ω pull-up resistors to facilitate integration of a final switch signal (3.3 V or 5 V TTL signal or input referenced to ground).

For programming the stepper motor signals, see [Chapter 9.1.5 "Controlling Stepper Motors"](#), [Page 260](#).

(1) Stepper motor signals are available only for RTC5 Boards with [DSP](#) version numbers 2 and higher (see [get_rtc_version Bit #16...Bit #23](#)). For older RTC5 Boards, stepper motor commands do not execute.

4.6.8 Analog Inputs (Accessory)

When the RTC5 PCI Board is equipped with the [ADC Add-On Board](#), then three 10 V analog inputs are available

- ANALOG IN0
- ANALOG IN1
- [ANALOG IN2](#), see [Section "Notes", Page 76](#)

After the first [read_analog_in](#) call, voltages that are applied at the 2 (RTC5 PCIe Board) or 3 ([ADC Add-On Board](#)) analog input ports are

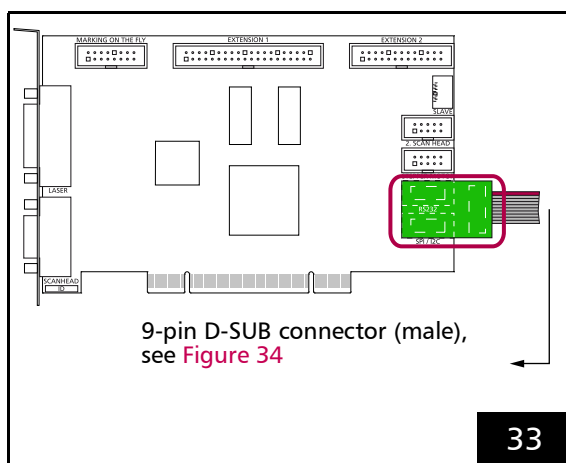
- automatically converted endlessly (duration approx. 0.3 ms)
- transferred to the [DSP](#) as 12-bit digital values

The current input values can be read by the control command [read_analog_in](#) at any time.

As of version DLL 543, OUT 543, RBF 524: The analog input values ([ANALOG IN0](#) and [ANALOG IN1](#), see [read_analog_in](#)) can be recorded with signal 54 (see [set_trigger/set_trigger4](#)). However, old bits are not recorded. The format of the data is (([ANALOG IN1](#) << 16) + [ANALOG IN0](#)).

Installation and Pin-out

The [ADC Add-On Board](#) must be plugged into the SPI/I²C and RS232 socket connectors of the RTC5 PCI Board. When installing, ensure that the add-on board is correctly oriented: the soldered-on flat ribbon cable must point outward, see [Figure 33](#).

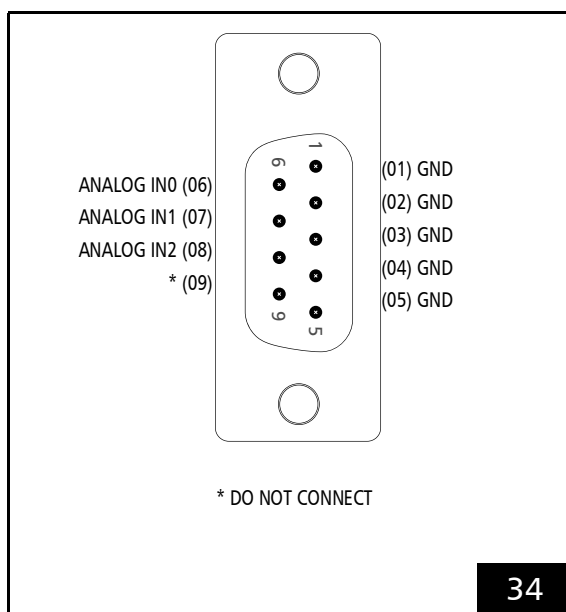


ADC Add-On Board (#0121126) on RTC5 PCI Board.

Internally, the **ADC Add-On Board** is only electrically connected to pins 01, 02, 09 and 10 of the SPI/I²C socket connector.

Attachment to the RS232 socket connector is only for mechanical stability.

The analog inputs are then available by a 9-pin D-SUB connector (male) at the soldered-on flat ribbon cable. Its pin-out is shown in **Figure 34**.



ADC Add-On Board (#0121126): pin-out of the 9-pin D-SUB connector, male.

Specifications

- Input voltage range: 0 V...10 V
- Input impedance: 4 k Ω
- ADC resolution: 12 bit

The input signals are referenced to ground GND.

Notes

- The **ADC Add-On Board** even provides a *third* 10 V analog input with identical specifications, see **Figure 34**:
– ANALOG IN2
However, the input values of **ANALOG IN2** can neither be queried by **read_analog_in** nor recorded by **set_trigger** or **set_trigger4**.

5 Installation and Start-Up

Installation of the RTC5 PCI Board consists of the following steps:

- (1) Checking the jumpers, see [Chapter 5.1 "Checking Jumper Settings"](#), Page 77.
- (2) Hardware installation: Installing the RTC5 Board, see [Chapter 5.2 "Installing the RTC5 PCI Board"](#), Page 77.
- (3) Installing the RTC5 board driver, see [Chapter 5.3 "Installing the RTC5 Board Driver"](#), Page 78.
- (4) Installing the RTC5 software, see [Chapter 5.4 "Installing the RTC5 Software"](#), Page 80.
- (5) When turning on the laser system, observe the safe start-up sequence, see [Chapter 5.5 "Safe Start-up and Shutdown Sequences"](#), Page 80.
- (6) The included HPGL converter program can be used for conducting a post-installation functionality test, see [Chapter 5.6 "Functionality Test"](#), Page 81.

5.1 Checking Jumper Settings

SCANLAB ships RTC5 PCI Boards in various jumper configurations.

- (1) Make sure that the to-be-installed RTC5 PCI Board has the required jumper configuration. If you need to change the factory settings, proceed as described in [Chapter 2.7 "Jumper Settings and Type Identification"](#), Page 35.
- (2) Proceed with [Chapter 5.2 "Installing the RTC5 PCI Board"](#), Page 77.

5.2 Installing the RTC5 PCI Board

Notice!

- Store the board in an electrostatically neutral environment using the supplied anti-static bag.
- Carry out the installation in an area that complies with Electro Static Discharge (ESD) directives. When installing the board, observe the instructions given in the instruction for use of the PC.
- Do not touch the contacts of the board.
- Protect the board from humidity, dust, corrosive vapors and mechanical stress.

Perform the following steps to install the RTC5 in a PC:

- (1) Remove all diskettes and CDs from your PC.
- (2) Shut down the operating system and switch off the PC. Disconnect the PC from the mains.
- (3) Disconnect all peripherals (for example, monitor, mouse, keyboard, printer etc.) from the PC.
- (4) Place the PC on a work table that complies with ESD directives.
- (5) Remove the housing of the PC. Locate a free PCI slot and remove the slot cover. A height from the slot connector's top of at least 105 mm is required.
- (6) Remove the RTC5 from the antistatic bag. Do not touch the contacts of the board.

- (7) If you want to use the signals on the RTC5's socket connectors, then attach the appropriate cables. SCANLAB recommends installing additional slot covers with suitable connectors, so that the signals are accessible even when the PC's housing is closed. All RTC5 connectors and signals are described in [Chapter 4 "RTC5 PCI Board – Layout and Interfaces", Page 51](#).
- (8) Install the RTC5 into the PCI slot. Observe the instructions in the manual of your PC⁽¹⁾.
- (9) Close the housing of the PC.
- (10) Place the PC at its operational location and connect all peripheral equipment.
- (11) Connect the 9-pin D-SUB connector on the RTC5 – using the [XY2-100 Converter \(Accessory\)](#), if necessary – to the scan system by a data cable (see [Chapter 4.5 "Interfaces to Scan System", Page 54](#)).
- (12) Connect the 15-pin D-SUB connector on the RTC5 to the laser by a suitable interface (see [Chapter 4.6 "Interfaces for the Laser and Peripheral Equipment", Page 60](#)).
- (13) If necessary, connect one or more of the RTC5's socket connectors (using additional slot covers, see above) to the laser, a handling system or other supplementary equipment of the overall system.
- (14) Check all connections and turn on the PC.

5.3 Installing the RTC5 Board Driver

Notice!

- If on your PC an WDM technology-based RTC3/4/5 board driver
 - is yet installed or
 - was installed and has been removed or
 - you are not sure in this regard:
 - (1) Install the RTC5 board driver.
 - (2) Run `ScanlabClassChecker.cmd` as Administrator (part of the delivered RTC5 software package; see there also the background information in [ReadMe_ScanlabClassChecker.pdf](#)). Step 2 can be skipped on brand new PCs on which an RTC board driver never has been installed.

To install the RTC5 board driver for Windows 10, 8, 7, proceed as follows:

- After the RTC5 Board has been installed, start the computer.

For Windows 7:

- (1) If the "Add Hardware Wizard" does not come up automatically, call it from the Control Panel.
- (2) In the "Add Hardware Wizard" specify the folder that includes the RTC5 board driver. During installation, the operating system automatically selects the appropriate driver file (32- or 64-bit).

- (1) If multiple master/slave-synchronized RTC5 Boards are to be used in a PC, then the RTC5 Boards must first be connected pairwise with each other by the MASTER and SLAVE connectors and then installed in adjacent PCI slots. (see also [Chapter 4.4 "Master Socket Connector, Slave Socket Connector", Page 53](#)).

For Windows 10, 8:

- (1) Open the Device Manager to display the device tree. Look for the "PCI device" entry and update its driver by specifying the folder that includes the RTC5 board driver.
- (2) Run `ScanlabClassChecker.cmd` as Administrator (part of the delivered RTC5 software package; see there also the background information in [ReadMe_ScanlabClassChecker.pdf](#)).

Notice!

- The RTC5 PCI Board does not support power-saving modes that switch off power to the PCI bus. Accordingly, you must disable standby or sleep modes of the operating system. Particularly disable the Windows sleep mode option (some Windows operating systems enable this option by default). You can use the `SleepMode.cmd` script included in the RTC5 software package (under `RTC5 Tools`) to do this. See also [Section "Notes", Page 33](#).

5.3.1 RTC5 Software Package Upgrades

If you currently use an RTC5 software package version 2012_09_05 or earlier (still contains WDM driver) and you want to upgrade to an RTC5 software package version 2013_02_21 or later (contains already WDF driver), then proceed as follows:

- (1) First install only the RTC5 software files, see [Chapter 5.4 "Installing the RTC5 Software", Page 80](#).
- (2) Test the proper functioning of your user program.
- (3) Only afterwards install the new driver (this is a WDF driver). This is because quickly changing back to older RTC5 software files is then only possible if you also change the driver (see also note on [page 33](#)).
- (4) Right-click on the delivered script `ScanlabClassChecker.cmd` and choose 'Run as Administrator' from the appearing list. For further information, refer to the file [ReadMe_ScanlabClassChecker.pdf](#).

5.3.2 Exchange of RTC Boards and Update of RTC Board Driver

After the exchange of any RTC board of type RTC3, RTC4, RTC SCANAlone, RTC5 for an RTC5 (also: RTC5 PCIe Board) or, to update an (outdated) WDM driver to a (newer) WDF driver proceed as follows:

- (1) Install the latest driver for the new board (this is a WDF driver; downloadable from the SCANLAB homepage).
- (2) Right-click on the delivered script `ScanlabClassChecker.cmd` and choose 'Run as Administrator' from the appearing list. For further information, refer to the file [ReadMe_ScanlabClassChecker.pdf](#).

5.4 Installing the RTC5 Software

Additionally to installing the drivers, the RTC5 software must be copied and unzipped onto the hard drive of the PC. The following files are required:

- **RTC5 DLL**
 - **RTC5DLL.dll**
Win32-based RTC5 dynamic link library
or
 - **RTC5DLLx64.dll**
Win64-based RTC5 dynamic link library
- **RTC5 files**
 - **RTC5OUT.out**,
Program file for the **DSP**
 - **RTC5RBF.rbf**
Firmware file for the **FPGA**
 - **RTC5DAT.dat**
Binary auxiliary file

The files are included in the RTC5 software package. For easy identifying and archiving of different software versions, some of the files are also delivered zipped (the zip file names RTC5<...>_<Version>.zip include the version numbers).

To install the RTC5 software, follow these steps:

- Copy or unzip the files **RTC5DLL.dll** (or **RTC5DLLx64.dll**), **RTC5OUT.out**, **RTC5RBF.rbf** and **RTC5DAT.dat** (of desired version) to the hard drive of your PC.
When replacing individual software files, note that differing program versions cannot be arbitrarily combined with another (each zip file includes a text file with corresponding version information).

Notes

Also provide the necessary correction file(s) (existing *.ct5 correction files can still be used; do not overwrite customized correction files!).

5.5 Safe Start-up and Shutdown Sequences

To assure safety during start-up, turn on the components of your laser system exactly in the following order:

- (1) Turn on the PC containing the RTC5 Board and start up the control software (initialization of the board).
- (2) Turn on the desired peripheral equipment.
- (3) Turn on the power supply for the scan system.
- (4) Turn on the laser.

When shutting down the laser system, turn off the components exactly in reverse order:

- (1) Turn off the laser.
- (2) Turn off the scan system's power supply.
- (3) Turn off the peripheral equipment.
- (4) Close the control software and turn off the PC containing the RTC5 Board.



Caution!

- While the PC is turned on and off, short-term level variations at the output ports of the RTC5 PCI Board, for instance short-term arbitrary variations of the laser control signals can occur. Therefore always observe the above listed start-up and shutdown sequences. Otherwise, there is the risk that the laser unexpectedly turns on for a short term.
- Always turn on the scan system after the PC and control software and turn it off prior to the PC and control software. Otherwise, arbitrary scan motions of the scan system could occur. Always turn on the laser as the last component and turn off the laser as the first component. Otherwise, there is the risk that the laser beam might be deflected in an uncontrolled direction.

5.6 Functionality Test



Caution!

- *Always turn on the PC and the power supply for the scan head first before turning on the laser. Otherwise, there is the danger of uncontrolled deflection of the laser beam.*
SCANLAB recommends the use of a shutter to prevent uncontrolled emission of laser radiation.

The HPGL conversion program `HPGL.EXE` is supplied for testing control of the scan head, see also [Section "Folder HPGL", Page 28](#). This program lets you load graphics files in Hewlett Packard HPGL format (vector graphic plotter files `*.PLT`) for transfer to the RTC5.

- (1) Copy the HPGL converter and the supplied demo file into the same folder as the **RTC5 DLL**, the RTC5 **DSP** program file(s) and correction file(s).
- (2) Start the HPGL converter.
- (3) Choose **Options > Correction**, and then select a correction file.
- (4) Choose **File > Load HPGL-File**, and then select a `*.plt`.
- (5) To start output, choose **Mark > Start Marking**.

5.7 User Programs and Demo Software

The DLLs for RTC5 user programs (**RTC5DLL.dll**, **RTC5DLLx64.dll**) support the RTC5 under 32-bit and 64-bit Microsoft operating systems Windows 10, 8, 7. The DLLs provide all required functions for operating the RTC5.

Programming of user programs is described in detail in [Chapter 6 "Developing RTC5 User Programs", Page 83](#). [Chapter 6.2 "Initialization and Program Start-Up", Page 84](#) shows how to import the functions of the **RTC5 DLL** into user programs, if they are written in Pascal, C, C++ or C#.

On a 64-bit operating system, the 64-bit variant of the RTC5 board driver supports function calls from Win64-based applications as well as from Win32-based applications.

Therefore, existing Win32-based applications for the RTC5 are able to execute even on 64-bit systems, if the Win32-based **RTC5DLL.dll** from the RTC5 software package is used. For Win64-based applications, the Win64-based **RTC5DLLx64.dll** is included in the RTC5 software package. In case a user program utilizes implicit linking to the **RTC5DLLx64.dll**, it must be linked with the Win64-based import library **RTC5DLLx64.lib**.

To help programmers get started, some example codes are listed on [page 89](#), [page 121](#) and [page 130](#). In addition the RTC5 software package includes a variety of demo programs (see [page 728](#)).

Notice!

- Carefully check your user program before running it. Programming errors can cause a system breakdown. In this case, neither the laser nor the scan head can be controlled.



5.8 Exchanging RTC5 Boards

If an already installed RTC5 board of an older type is replaced by an RTC5 board of a newer type, then it is recommended to carry-out a software update.

This is particularly important when RTC5 Boards with DSP version 0 get replaced by RTC5 Boards with DSP version 2 (**get_rtc_version Bit #16...Bit #23**). The suitable RTC5 software package can be found on the delivered CD. You can also download the latest files From the SCANLAB website you can also download the latest RTC5 software package.

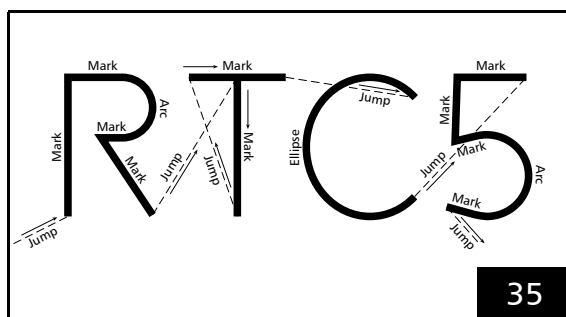
See also **Chapter 5.3.2 "Exchange of RTC Boards and Update of RTC Board Driver", Page 79.**

6 Developing RTC5 User Programs

6.1 RTC5 Software Concept Basics

6.1.1 Controlling Scan Systems and Lasers – An Introductory Example

The SCANLAB RTC5 PCI Board and its related software are designed for controlling scan systems and lasers. To illustrate the principle of operation, [Figure 35](#) shows a simple laser marking sample⁽¹⁾.



A laser marking sample.

The lettering is made up of straight line segments (vectors) and arc segments. The [RTC5 DLL](#) provides a set of jump, mark, arc and ellipse commands for laser processing along such segments. Each of these commands describes one vector or arc. Additional software commands are available for controlling the laser during the marking process. The RTC5 processes the commands it receives and precisely transmits the required marking signals to the scan head (using a predefined 10 μ s time raster) and to the laser. The scan head's galvanometer scanners accurately position their deflection mirrors in synchronization with the incoming control signals.

The current status of a scan head can also be queried via the RTC5 PCI Board using appropriate commands.

For laser control, the RTC5 provides various analog and digital signal outputs freely available for tailoring laser control to customer-specific requirements.

6.1.2 Control Commands and List Commands

The RTC5 command set consists of:

- Control commands
- List commands

Control commands are executed immediately. They are used, for instance, for initializing, controlling execution of lists, setting some general parameters, or for directly controlling the laser and scan head.

Before *list commands* can be sent to the RTC5, a control command must define the input pointer to which subsequent list commands are transferred (this corresponds to opening a list, see [Chapter 6.4.1 "Loading Lists", Page 94](#)). List commands sent to the RTC5 afterwards are not executed immediately, but stored in a list memory area. The RTC5 begins reading the commands from the list memory area and processing them in real time only after the list is started.

Some list commands, which are called short list commands, do not require the full 10 μ s clock cycle for command execution - instead, they execute along with the next list command, one directly after the other within a single 10 μ s clock cycle, during which control commands cannot execute. The total list processing time is thereby reduced.

List commands include [Jump commands](#), [\[*\]mark\[*\] Commands](#) and ["Arc" commands](#), as well as commands for setting various scanning parameters such as laser power, jump speed and mark speed.

Short list commands include [list_jump_pos](#), [list_call](#) etc. With a [Polyline](#), for example, the laser power can be set individually for each vector by the short list command [write_da_x_list](#) without interrupting it (the laser remains on).

(1) In this manual, laser marking is mentioned only as an example of the many possible laser materials processing applications.

Some commands exist in two versions: as a list command and as a control command. Among these dual-version commands are the I/O commands.

All RTC5 commands are described in detail in [Chapter 10 "RTC5 Commands", Page 278](#). An overview is provided in [Chapter 10.1 "Overview", Page 278](#).

6.2 Initialization and Program Start-Up

To use the RTC5's commands and functions in a user program:

- The RTC5 must be fully installed – this includes the RTC5 hardware, driver and software (see [Chapter 5 "Installation and Start-Up", Page 77](#))
- The desired **RTC5 DLL** (Win32- or Win64-based) must be linked to the user program (see [Chapter 6.2.1 "DLL Calling Convention", Page 84](#))
- The DLL functions must be imported to the user program (see [Chapter 6.2.2 "Importing Commands", Page 84](#))

At the beginning of each user program, commands must be called:

- that initialize the **RTC5 DLL** for the calling user program and assigns access rights for the installed RTC5 Boards (see [Chapter 6.2.3 "Initializing the RTC5 DLL and Board Management", Page 86](#))
- that initializes the installed RTC5 Boards themselves (see [Chapter 6.2.4 "Start of RTC5 PCI Board Operation", Page 87](#)).

Only afterward can the user program load its command lists into the RTC5's list memory and start processing.

These steps are described individually in the following sections and summarized by a simple example program in the last section of this subchapter (see [page 89](#)).

6.2.1 DLL Calling Convention

Link the user program to the **RTC5DLL.dll**/**RTC5DLLx64.dll**⁽¹⁾.

The **RTC5 DLL** calling convention is `stdcall` for Win32 and `fastcall` for Win64.

The structure alignment is 4 byte for Win32 and 8 byte for Win64.

6.2.2 Importing Commands

To facilitate importing the commands of the **RTC5 DLL** into a C, C++, C# or Pascal user program, the RTC5 software package contains corresponding utility files.

Notice!

- Some RTC5 commands can only be *completely* executed with appropriate options, see also [Chapter 2.6 "Options", Page 34](#).
- **get_rtc_version** provides information about the options installed on your RTC5 PCI Board.

Pascal

Use the file **RTC5Import.pas** as a unit and call the RTC5 commands you need, like

```
goto_xy(1000, 2500);
```

for performing a jump to location 1000, 2500.

(1) See [Chapter 5.4 "Installing the RTC5 Software", Page 80](#).

C

In C, you can choose either implicit linking – also known as static load or load-time dynamic linking – or explicit linking – also known as dynamic load or run-time dynamic linking.

Implicit Linking

To accomplishing implicit linking, include the header file `RTC5impl.h` and link with the (C) import library `RTC5DLL.lib` (for Win32-based applications or with `RTC5DLLx64.lib` for Win64-based applications) for building the executable.

Call the RTC5 commands you need like

```
goto_xy(1000, 2500);
```

for causing a jump to location 1000, 2500.

Explicit Linking

To accomplishing explicit linking, include the header file `RTC5expl.h`. Before calling any RTC5 function, initialize the **RTC5 DLL** by calling the function `RTC5open` (which is defined in the file `RTC5expl.c`). When you are finished using the RTC5, close the **RTC5 DLL** by calling the function `RTC5close` (also defined in the file `RTC5expl.c`).

For building the executable, link with the file `RTC5EXPL.OBJ`, which you can generate from the source code `RTC5expl.c`.

Call the RTC5 commands you need, like

```
goto_xy(1000, 2500);
```

for causing a jump to location 1000, 2500.

Pros and Cons of Implicit and Explicit Linking

	Implicit Linking	Explicit Linking
Necessary Files	<code>RTC5impl.h</code> or <code>RTC5impl.hpp</code> , <code>RTC5DLL.lib</code> or <code>RTC5DLLx64.lib</code>	<code>RTC5expl.h</code> , <code>RTC5expl.c</code>
Advantage	Easiest linking method	Eliminates the need to link the user program with an import library
Negative Aspect	Need to link the user program with a compiler-specific import library	Need to initialize (<code>RTC5open</code>) and close (<code>RTC5close</code>) the DLL

C++

If you want to use references instead of pointers for returning values to function parameters in C++ (for instance `UINT&Pos` instead of `UINT*Pos`), use the files `RTC5impl.hpp` instead of `RTC5impl.h`. Otherwise, the command import is realized as in C (see above).

C#

For implicit linking of the **RTC5 DLL** in C# the wrapper class is provided. It also supports the 'Any CPU' option for simultaneous usage with 32-bit and 64-bit programs.

6.2.3 Initializing the **RTC5 DLL** and Board Management

As many RTC5 PCI Boards can be operated in one PC at the same time as the PC provides PCI slots.

The **RTC5 DLL** allows multi-processing⁽¹⁾ as well as multi-threading.

However, no board can be simultaneously used by multiple applications. Access rights (even if temporary) to existing boards are assigned on an exclusive basis by the DLL. Multiple threads of one user program can use the same board, but can not send commands to it at the same time (the **RTC5 DLL** automatically serializes the command calls).

The **RTC5 DLL**-internal RTC5 board management coordinates usage of different boards by different applications. RTC5 board management is initiated by **init_rtc5_dll**.

init_rtc5_dll is a prerequisite for RTC5 access and must be called at the beginning of every user program – even if only one RTC5 Board is to be used by only one user program.

init_rtc5_dll:

- searches for all existing RTC5 Boards
- establishes corresponding board management
- automatically assigns the user program access rights to the found boards (as long as access rights are not already assigned to another user program)
- assigns **RTC5 DLL**-internal numbers for all found RTC5 Boards (important for multi-board commands)
- sets one board as the active board, which is the target for non-multi-board commands

Further commands enable subsequent changes to access rights and changing the active board. Usage of multiple boards is described in [Chapter 6.6 "Using Several RTC5 PCI Boards in One PC", Page 112](#); usage by multiple applications is described in [Chapter 6.7 "Usage of RTC5 PCI Boards by Several User Programs", Page 117](#).

After **RTC5 DLL** initialization by **init_rtc5_dll**, users can additionally select one of two operation modes (see **set_rtc4_mode** and **set_rtc5_mode**). The default is **RTC5 Standard Mode**. An **RTC4 Compatibility Mode** is also provided so that applications written for the RTC4 can be processed by the RTC5 (to a large extent) without modification. However, a prerequisite here is that the program can only contain RTC4 commands that also exist with unchanged functionality as RTC5 commands. In the command reference of this manual ([Chapter 10.2 "RTC5 Command Set", Page 290](#)), when applicable, notes such changes in the "RTC4→RTC5" section (see also [Section "Increased Parameter Resolution", Page 43](#)).

(1) Several user programs can run simultaneously.

6.2.4 Start of RTC5 PCI Board Operation

At the beginning of every RTC5 user program – after initialization of the **RTC5 DLL**, see [Chapter 6.2.3 “Initializing the RTC5 DLL and Board Management”, Page 86](#) – it is recommended to carry out the following sequence of steps in order to start RTC5 PCI Board operation.

- (1) [Initializing the Board, Page 87](#)
- (2) [Configuring the Board, Page 87](#)
- (3) [Initializing the Scan System Control, Page 88](#)
- (4) [Initializing the Laser Control, Page 88](#)
- (5) [Loading and Executing Lists, Page 88](#)

Initializing the Board

- Call **load_program_file**. For the individual actions, see [Function, Page 430](#).
After execution of **load_program_file**, the position output is automatically set to the null position (0|0)⁽¹⁾ and laser control is *deactivated*⁽²⁾⁽³⁾.
In order to be able to combine **RTC5 files** (**RTC5DLL.dll/RTC5DLLx64.dll**, **RTC5OUT.out**, **RTC5RBF.rbf**, **RTC5DAT.dat**), they must have certain file versions, see also [Chapter 5.4 “Installing the RTC5 Software”, Page 80](#).
load_program_file performs a version compatibility check. If a version error exists (error code **7** and **get_last_error** return code **RTC5_VERSION_MISMATCH**), the board is released by **release_rtc**. Thus, it is not available for further RTC5 commands other than those not requiring granted access rights.

RTC5 PCI Boards can only be operated if all **RTC5 files** are available with a compatible combination of versions, see [Chapter 5.4 “Installing the RTC5 Software”, Page 80](#).

If the version compatibility check detects an error and the board's access rights are not assigned to another user program, the multi-board command **n_load_program_file** can be used to load a correct program version, followed by **select_rtc** or **acquire_rtc** to access the board.

If multiple RTC5 Boards are connected as master and slave, then **load_program_file** must have been called on all boards and a clock phase synchronization should be performed (see [Chapter 6.6.3 “Master/Slave Operation”, Page 113](#)).

Configuring the Board

- If necessary, configure the RTC5 list memory by **config_list**. By default, the list memory is preconfigured such that the memory areas “List 1” and “List 2” can each store 4,000 list commands after **load_program_file**. The protected list memory area “List 3” comprises the remaining 1,040,576 of the 2²⁰ storage positions.

- (1) Center of the **Image field**.
- (2) There are no laser control signals at the corresponding pins (LASERON, LASER1, LASER2). They are in high-impedance state.
- (3) On the state of the various output ports, see [page 431](#).

Initializing the Scan System Control

- (1) Use **load_correction_file** to download the necessary correction file(s) to the RTC5 (you can load a correction table before or after **load_program_file**⁽¹⁾ but you should load it *before* **select_cor_table**; you should at least load a 1to1 correction table).

See **Chapter 8.5 "Controlling 2D Scan Systems and 3D Scan Systems", Page 221**, for information about using several different correction tables.

- (2) Assign the previously loaded correction table(s) to the scan head control port(s) by **select_cor_table**. This causes the intended **Image field** correction to also be applied to the default scanner position (0|0) previously set by **load_program_file** (in some circumstances, **Image field** correction can even shift the null position).
After **load_program_file**, the default assignment is:

- The first scan head control port uses correction table #1.
 - The second scan head control port is off.
- The desired **Image field** correction becomes active only after a subsequent **select_cor_table**, **select_cor_table_list**, **Jump command** or **Mark command**.

- (3) Define the scanner delay mode (for instance "Variable **Polygon Delay**" or constant **Polygon Delay**; command **set_delay_mode**).
- (4) If necessary, load a table for the "Variable **Polygon Delay**" (by **load_varpolydelay**).

The remaining settings (**Scanner Delays**, jump speed and mark speed) are set by further control commands or list commands.

Initializing the Laser Control

- (1) Set the laser mode by **set_laser_mode**.
- (2) Set the polarity of the laser control signals appropriate to your laser system by **set_laser_control**.
- (3) Set the **FirstPulseKiller** signal (only for YAG modes) by **set_firstpulse_killer**.
- (4) Set the delay of the Q-Switch signal (only for YAG modes, in particular for YAG Mode 5) by **set_qswitch_delay**.
- (5) Set the stand-by pulses by **set_standby** (in particular for **Laser Mode 4** and **Laser Mode 6**).
- (6) Enable the laser control signals (see **enable_laser**), if they have been suppressed by **set_laser_control**.

The remaining settings (laser timing or **Laser Delays**) are set by further control commands or list commands, see example in **Chapter 7.1.4 "Example Code (C)", Page 130** or **Section "Signals for "Laser Active" Operation", Page 175**.

Loading and Executing Lists

- (1) Load the list(s).
- (2) If necessary, enable the external start input (by **set_control_mode**).

Notice!

- Carefully check your user program before running it. Programming errors can cause a break down of the system. In this case, neither the laser nor the scan head can be controlled.

(1) **load_program_file** deletes correction tables number 3 and 4.



6.2.5 Example Code (C)

The following C source code for a console user program (environment: Win32) illustrates the programming fundamentals of **RTC5 DLL** and **RTC5 PCI Board** initialization (for complete demo programs, see **Chapter 11 "Demo Programs"**, **Page 728**).

Necessary sources: **RTC5impl.h**, **RTC5DLL.lib** (for implicit linking) or **RTC5expl.h**, **RTC5expl.c** (for explicit linking) to link the **RTC5 DLL** to the program (see **Chapter 6.2.1 "DLL Calling Convention"**, **Page 84**). If the operating system does not find the **RTC5DLL.dll** on user program startup, it produces a corresponding error message and terminates the program.

```
// System header files
#include <windows.h>
#include <stdio.h>
#include <conio.h>

// RTC5 header file for implicitly linking to the RTC5DLL.dll (for building the executable, also link
// with the (Visual C++) import library RTC5DLL.lib):
#include "RTC5impl.h"
// Alternatively: RTC5 header file for explicitly linking to the RTC5DLL.dll (for building the executable,
// link with the file RTC5EXPL.OBJ, which you can generate from the source code RTC5expl.c):
// #include "RTC5expl.h"

void __cdecl main( void*, void* )
{

    // only for explicitly linking:
    // if ( RTC5open() ) // error detected, RTC5open returns 0 for no error
    // {
    //     printf( "Error: RTC5DLL.dll not found\n" );
    //     terminateDLL();
    //     return;
    // }

    printf( "Initializing the DLL\n\n" );

    UINT  ErrorCode;

    // Initializing the RTC5 DLL (the following command must be called as the first RTC5 command)
    ErrorCode = init_rtc5_dll();

    // Following init_rtc5_dll you should include a program code to catch an error during
    // initialization, for example, for the case the desired Board is not detected, access is denied or
    // for another error (for example, a version mismatch).
    // See Chapter 6.8.3 "Example Code (C)", Page 121.
```



```
// Initializing the RTC5 PCI Board:
// - Selecting the RTC5 Board number 1 as the active Board for this user program
// - If desired: Selecting the RTC4 Compatibility Mode as operation mode
// (default: RTC5 Standard Mode).
// - Stopping any list running on RTC5 PCI Board number 1
// (if this board has been used previously by another user program,
// a list might still be executed. This would prevent load_program_file and load_correction_file
// from being executed).
// - Calling load_program_file for resetting the board, loading the program file, etc. (here also a
// program code should be included to catch possible errors - for example, file or
// system errors - during initialization, see Chapter 6.8.3 "Example Code (C)", Page 121).
// - Clearing all previous error codes (stop_execution might have created an RTC5_TIMEOUT or
// RTC5_BUSY error).
// - Configuring the RTC5 list memory, default: 4000 storage positions for list 1,
// 4000 for list 2).
(void) select_rtc( 1 );
set_rtc4_mode();
stop_execution();
ErrorCode = load_program_file( 0 ); // Path = 0: path of current working directory
if ( ErrorCode )
{
    printf( "Program file loading error: %d\n", ErrorCode );
    free_rtc5_dll();
    return;
}
reset_error( -1 );
config_list( 4000, 4000 );

// Following the above initialization code you can include the program code defining
// the laser scan process. An example code is in Chapter 7.1.4 "Example Code (C)", Page 130.

// End of main program
terminateDLL();
return;
}

void terminateDLL()
{
    printf( "- Press any key to shut down \n" );
    while( !kbhit() );
    (void) getch();
    printf( "\n" );
    // Close the RTC5 DLL
    free_rtc5_dll();
    // only for explicitly linking:
    // RTC5close();
}
```

6.3 RTC5 List Memory

The RTC5 list memory serves as intermediate storage for list commands.

Before list commands can be transferred to the RTC5 PCI Board, a control command must define the input pointer to which subsequent list commands are transferred. This corresponds to opening a list, see [Chapter 6.4.1 "Loading Lists", Page 94](#).

By additional control commands, the processing of the transferred list commands can be started.

6.3.1 Lists and the Protected List Memory Area

The RTC5 list memory offers $1\text{M} = 1,048,576 = 2^{20}$ storage positions in total.

In general, it can be split into three areas. These sizes are freely configurable.

- Two list memory areas, "List 1" and "List 2". In this manual, these are also simply designated as "lists".
- User-definable is in addition:
a third protected list memory area "List 3".
This is protected against unintended overwriting (by loading normal command lists).

"List 1" and "List 2"

In principal, both list memory areas "List 1" and "List 2" can be used in a manner identical to the two list memory areas of the RTC4, RTC3 or RTC2 Boards, for example, for continuous loading and processing of command lists.

However, the RTC5 PCI Board has a bigger total list memory and furthermore, the size of each memory area can be freely configured, see [Chapter 6.3.2 "Configuring the RTC5 List Memory", Page 92](#).

For list handling, see [Chapter 6.4 "List Handling", Page 94](#).

"List 3" – Protected List Memory Area

"List 3" is intended for a protected storage of frequently needed list command sequences (as subroutines or character set definitions). It is protected against unintended overwriting (during loading of normal command lists).

There are principally two alternative ways to utilize this protection feature:

- (1) Subroutines can be written to the upper positions of the list area. These subroutines can be subsequently assigned to the protected list memory area "List 3". Such subroutines are called – both initially in the list memory area as well as subsequently in the protected list memory area "List 3" – by **list_call**, specifying an absolute memory position.
- (2) Special commands allow subroutines and character set definitions to be loaded directly in the protected list memory area "List 3" as indexed subroutines or definitions. They can then be called by providing the corresponding index. Indexed character set definitions can, for example, be used in conjunction with **mark_text** for directly marking text.

SCANLAB strongly recommends *not* intermixing usage of these two methods. Otherwise, unintended data loss by overwriting can occur even in the protected list memory area "List 3".

The RTC5 command set includes appropriate conversion commands so that users are not forced to continuously use only one of the two methods. The defining of subroutines and character sets, as well as their management and subsequent conversion options are detailed in [Chapter 6.5 "Structured Programming", Page 101](#).

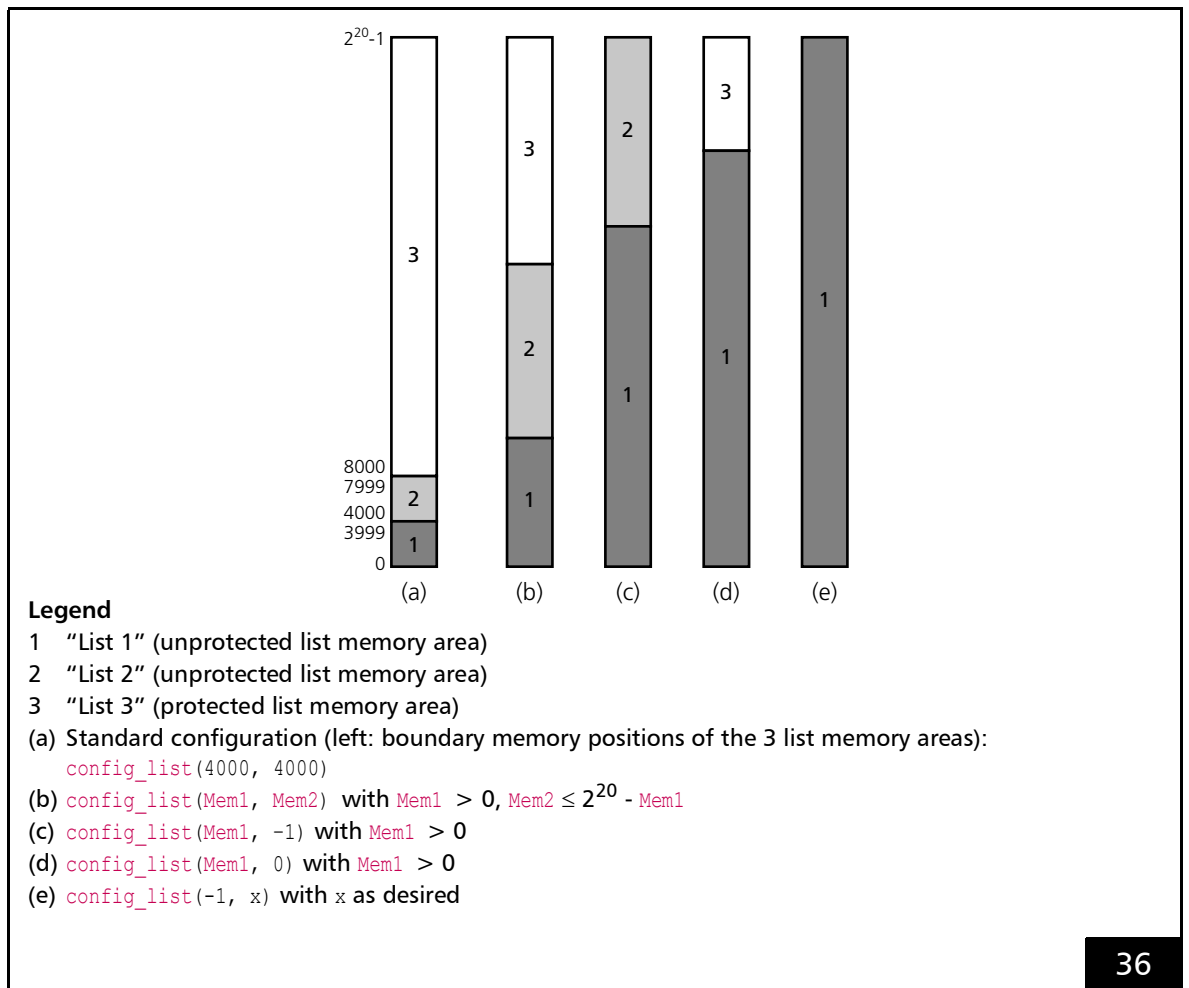
6.3.2 Configuring the RTC5 List Memory

For compatibility, `load_program_file` configures the RTC5 list memory area such that "List 1" and "List 2" can each accept 4,000 list commands by default.

The protected "List 3" then owns the remaining 1040576 of the 2^{20} storage positions (see Configuration (a) in Figure 36).

By `config_list`, the list memory areas can be reconfigured.

For example, if a user program only needs one list, then the RTC5's entire memory can be treated as a single list with a total capacity of 2^{20} positions (see Configuration (e) in Figure 36). In this case, the single list ("List 1") can be loaded with up to 2^{20} commands.



Examples of allowed list memory configurations.

Generally, during configuration are assigned:

- lower storage position numbers to "List 1"
- medium storage positions numbers to "List 2"
- upper storage position numbers to "List 3"

When configuring the list memory areas, the following must be observed:

- "List 1" must contain at least *one* storage position
- the total sum of storage positions for "List 1" and "List 2" must not exceed 2^{20} .

Other than that, the sizes of "List 1" and "List 2" can be set as desired. For example, "List 2" can be configured even with 0 storage positions, see configuration (d) or (e) in [Figure 36](#).

With every configuration, remaining storage positions (not assigned to "List 1" or "List 2") are automatically assigned the protected list memory area "List 3".

The memory content is not altered by the configuration process. Therefore, repeating the call with differing parameters is nondestructive.

When altering the configuration, you should observe also the following:

- List boundaries should not be moved to within an eventual subroutine.
- The protection of a "List 3" range is removed, if this range is assigned to list memory area "List 1" or "List 2".
- Valid jump addresses specified in [Jump commands](#) might become invalid if the configuration is altered, see [Chapter 6.5.3 "Jumps"](#), Page 109.
- After the protected list memory area "List 3" has been made larger, defragmentation might be needed to make the newly assigned memory area usable, see [Section "Index Management and Defragmentation"](#), Page 104.

6.4 List Handling

The two list memory areas “List 1” and “List 2” serve as intermediate storage for the continuous loading and processing of list commands.

6.4.1 Loading Lists

“List 1” and “List 2” are enabled to be filled with list commands by **set_start_list_pos**, **load_list** or other control commands (see below). An input pointer is thereby defined for the selected list. This input pointer specifies the memory position to which the subsequent list commands are transferred.

Lists are self-contained memory blocks for list commands. When in the process of list loading the list end is reached without setting the input pointer to another list, then the input pointer is automatically reset to the start of the current list, where loading continues.

An automatic change of the input pointer to another list never occurs, particularly not to the protected list memory area “List 3”.

In general, when list commands are loaded into storage positions, any list commands previously stored there are overwritten. This occurs even if they have not yet been processed or are currently being executed. User should make sure not to overwrite commands still needed by the user program (see below).

PCI transfer of the list commands into list memory is buffered to increase the speed for continuous downloads. The buffer is 16 commands in size.

Whenever the buffer is full or when the commands **set_end_of_list**, **list_return**, **set_input_pointer** (and related commands), **execute_list_pos** (and related commands), **auto_change**, **auto_change_pos**, **start_loop** or **release_rtc** are called, this automatically results in a flush. Thereby, the still buffered list commands are transferred to the list memory.

A flush can be initiated at any time by **set_input_pointer(get_input_pointer())**, even if the buffer is yet “incomplete”. This is only necessary in some circumstances when list commands should be processed and list input is not yet finished (for example, with an **External Start**).

“Unconditional” Loading

The input pointer is set:

- to the beginning of the selected list by
 - **set_start_list**
 - **set_start_list_1**
 - **set_start_list_2**
- to the specified address of the selected list by
 - **set_start_list_pos**
 - **set_input_pointer**

If needed, the current positions of the input pointer and output pointer can be queried by **get_input_pointer** or **get_list_pointer** and **get_status** or **get_out_pointer** – for example, to ensure that not-yet-processed list commands are not overwritten.

Loading with Protection

The loading process is initialized by **load_list**, which sets the input pointer to the specified address in the selected list (just like **set_start_list_pos**). However, this occurs only, if the selected list is not currently in use.

Alternatively, you can simply let the input pointer be set to a currently non-active or already processed list by **load_list** (the RTC5 PCI Board automatically determines the corresponding appropriate list).

The return value of **load_list** reveals if, and in which list, the loading procedure has been successfully initialized.

Otherwise, the input pointer is set to an invalid position. Then, no further list commands can be input until the input pointer is correctly set back to a valid position again (for example, by repeating **load_list** with a positive result or **set_start_list_pos**).

This automatically prevents unintentional overwriting of commands that are still to be executed.

load_list is useful in scenarios such as alternating list changes, where you want to wait specifically for a list to be processed, see [Section "Alternating List Changes", Page 100](#).

Terminating Lists

A command list can be, but need not necessarily be, terminated by a **set_end_of_list**.

However, if an unterminated command list is executed and the output pointer thereby encounters the last possible position in the list, the output pointer automatically resets to the start of the list and processing continues there.

Automatic list changing after a list is processed can only occur, if the list has been terminated by **set_end_of_list**, see [Chapter 6.4.6 "Automatic List Changing", Page 99](#).

The loading of a **set_end_of_list** does *not* stop the loading procedure itself. Therefore, list commands immediately following a **set_end_of_list** are still loaded into the same list.

6.4.2 List Status

Dependent on the command input and output statuses, lists receive particular list status values (compare to [Chapter 6.4.3 "List Execution Status"](#), [Page 97](#)).

By control command **read_status**, the current list status values can be queried – separately for both lists.

- **LOAD list status**
The **LOAD list status** **LOAD1** or **LOAD2** indicates that the input pointer is currently in this list. In any case, the **LOAD list status** of the other list is *not* set.
- **READY list status**
The **READY list status** **READY1** or **READY2** is set when a **set_end_of_list** is written into the list during the loading procedure. The **READY list status** is reset when the **LOAD list status** of the list is newly set.
- **BUSY list status**
The **BUSY list status** **BUSY1** or **BUSY2** indicates that the output pointer is currently in this list after list execution (of "List 1" or "List 2") has been started. In any case, the **BUSY list status** of the other list are then *not* set. The **BUSY list status** of a list is reset when **set_end_of_list** is executed (or is alternating set, if automatic list changing has been previously activated). If a list is opened for loading while still being processed, its **BUSY list status** still remains set.
- **USED list status**
The **USED list status** **USED1** or **USED2** is set when a **set_end_of_list** is reached during processing. The **USED list status** is reset when the **LOAD list status** of the list is set.

Notes

- If the list status is queried during processing of a subroutine in the protected list memory area "List 3", then the status is returned of the list "List 1" or "List 2" in which the output pointer most recently resided (typically from where the subroutine has been originally called).
- If list execution is interrupted (by **pause_list**, **stop_list** or **set_wait**), then the above-mentioned status values remain unchanged.
- If list execution is aborted (by **stop_execution** or an **External Stop**), the **USED list status** is set for both lists (as by initialization). See also `load_list( ListNo = 3 )`.
- If you want to explicitly set the **USED list status** for a list `ListNo` (for example, after abortion by **stop_execution** or an **External Stop**), then load a **set_end_of_list** to a free position `Pos` of this list by `set_start_list_pos( ListNo, Pos )` and execute by `execute_list_pos( ListNo, Pos )`. If no other list has been active at this moment, then list `ListNo` has the **USED list status** afterward.
- When interpreting the status values read back by **read_status**, always take into account the programmed loading or execution processes of the lists (see command description).

6.4.3 List Execution Status

In addition to the list status values, see [Chapter 6.4.2 "List Status", Page 96](#), list execution status values are provided.

These can be queried by the control command **get_status**.

- **BUSY list execution status**
The **BUSY list execution status** is set when:
 - the RTC5 PCI Board currently processes one of the two lists
 - a list has been paused by the control commands **pause_list** or **stop_list**
 The **BUSY list execution status** is *not* set when a list has been paused by the list command **set_wait** (is set again by a subsequent **release_wait**).
- **PAUSED list execution status**
The **PAUSED list execution status** is set when processing of the list has been paused by **pause_list**, **stop_list** or **set_wait**. It is reset by a subsequent **restart_list** or **release_wait**, see also [Chapter 6.4.5 "Interrupting Lists for Synchronization of Processing", Page 99](#).
- **INTERNAL-BUSY list execution status**
The **INTERNAL-BUSY list execution status** is set when the RTC5 PCI Board is busy with executing a control command, which needs more than 10 μ s for executing a scan motion (for example, **goto_xy** or possibly **set_offset**) or while a home jump or home return is executed (with **set_wait**, **set_end_of_list** or **release_wait**, if the home jump mode has been previously activated by **home_position** or **home_position_xyz**).

Notes

- The **INTERNAL-BUSY list execution status** and the **BUSY list execution status** cannot be set at the same time.
- With the RTC5 PCI Board, the **BUSY list execution status** is also available as BUSY OUT signal at:
 - LASER connector, see [Figure 17](#)
 - EXTENSION 1 socket connector, see [Figure 22](#)
 - MARKING ON THE FLY socket connector, see [Figure 25](#)
- Some control commands are ignored (not executed), when the **BUSY list execution status** and/or **INTERNAL-BUSY list execution status** are set (for example, **auto_cal**, **goto_xy**, **load_correction_file**) or – with set **INTERNAL-BUSY list execution status** – are only executed with delay after the **INTERNAL-BUSY list execution status** has been reset again (for example, **execute_list_pos**, **set_offset**).

6.4.4 Starting and Stopping Lists

List processing ("List 1" or "List 2") can be started by:

- The control command **execute_list**
- An external start signal, see Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization", Page 265

execute_at_pointer can be used to start output of a list at a specified address. If an external start signal is used, see Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization", Page 265, then **set_extstartpos_list** allows definition of a start address for the **External Start**.

The RTC5 PCI Board starts the execution immediately. Even during 10 μ s-clocked execution of the list commands, you can still send control commands to the RTC5 PCI Board. These are immediately executed without hindering execution of the list.

This is useful, for example, for loading a second list while the first list is being processed (the PC and scan head then work in parallel). However, the second list can only be started after processing of the first list has been finished. During list processing, **execute_list** or an external start signal is ignored.

Execution of a list can also be stopped at any time, for example, for implementing an emergency shutdown. As soon **stop_execution** is called or an external stop signal is transferred to the RTC5 PCI Board, the currently executed list is aborted and the **Signals for "Laser Active" Operation** are turned off (but not deactivated).

With **range_checking**, the processing of a list can also be terminated automatically (like with **stop_execution**).

If, during list processing, the list end is reached without encountering a **set_end_of_list**, then processing continues at the beginning of the current list. This is repeated until either **stop_execution** is called or an external stop signal is transferred to the RTC5 PCI Board.

If during list processing a **set_end_of_list** is reached, then list execution stops – unless **auto_change**, **auto_change_pos** or **start_loop** has been previously called, a list change takes place, see Chapter 6.4.6 "Automatic List Changing", Page 99. This list change occurs only upon reaching a **set_end_of_list**.

Notes

- Lists are not automatically started. Regardless of how many commands are loaded, a list must be started as described in order to be processed.
- To also enable starting and stopping of list execution by external signals, the RTC5 PCI Board provides corresponding control inputs, see Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization", Page 265.
- **set_pause_list_cond** or **set_pause_list_not_cond** can be used to set a condition for the 16-bit input port so that a **pause_list** is executed instead of **stop_execution** when an **External Stop** is present. In The list execution can then be continued by **restart_list**.

6.4.5 Interrupting Lists for Synchronization of Processing

The list command **set_wait** makes it possible to insert numbered break points (“wait markers”) into a list. Each break point is associated with a number greater than zero. When the RTC5 PCI Board reaches a break point during list execution, see [Chapter 6.5 “Structured Programming”, Page 101](#), output of the list is temporarily interrupted and the laser is switched off.

get_wait_status checks whether list processing is currently interrupted at a break point. If processing is interrupted, **get_wait_status** returns the number (wait_word) of the break point (otherwise the value zero).

Break points are provided for synchronization purposes. The user program should perform a handling routine for each break point. When that handling routine is finished, list processing can be resumed (at the list command that follows) by the control command **release_wait**.

By **set_wait** the **PAUSED list execution status** (queryable by **get_status**) is set and the **BUSY list execution status** is reset. The opposite occurs after a subsequent **release_wait**.

List execution can be interrupted at any desired point in time by the control command **pause_list** (or by the synonym **stop_list**) and resumed by **restart_list**. By **pause_list**, **Signals for “Laser Active” Operation** are suppressed and the scan system remains in the most recently defined state – even if in the middle of **Microstepping**. After a subsequent **restart_list**, the scan system resumes the planned motions (of the current command) and the laser control signals are released again (in general, an interrupted marking cannot be continued without a disruption in the marking result).

By **pause_list** the **PAUSED list execution status** (queryable by **get_status**) is set and is reset by **restart_list**. The **BUSY list execution status** is left unchanged by both commands.

6.4.6 Automatic List Changing

If the RTC5 list memory is configured for two list memory areas “List 1” and “List 2”, see [Chapter 6.3.2 “Configuring the RTC5 List Memory”, Page 92](#), then a second list can be loaded while the first list is still being processed.

It typically takes substantially longer to process a list than to write it into the memory. Continuous processing of arbitrarily long lists is therefore also possible, if they are divided into command blocks.

Continuous command output, which requires switching between two lists, can be achieved by automatic list changing as described in the following sections.

The commands for automatic list changing only take effect when the next following **set_end_of_list** is executed. That is, automatic list changing after processing a list can only occur if that list has been finished with a **set_end_of_list**. Otherwise, processing resumes at the beginning of the same list.

If the RTC5 list memory is configured for a list memory area (“List 2”) to size 0, then all automatic list change commands lead to “List 1” to the specified position.

One-Time List Change

auto_change and **auto_change_pos** activate an automatic, one-time list change between “List 1” and “List 2”. After processing of the current list (when **set_end_of_list** is reached), processing of the next list is thereby automatically started.

When using **auto_change** the next list is started at position 0; when using **auto_change_pos** the next list is started the specified start position (list memory address as an offset to the beginning of the list).

Alternating List Changes

Another way to achieve continuous command output is by alternatingly repeating output of the two lists.

To do so, **start_loop** must be called. This causes a continuous, automatic and alternatingly repeating processing of both lists, provided both lists are finished each with **set_end_of_list**.

The alternating processing repeats until **quit_loop** is called. **quit_loop** terminates continuous processing as soon as the current list is finished.

The currently non-active list can be newly reloaded even as the other list is processed. This allows continuous alternating output of two lists with not only fixed content, but also constantly new content.

Notes

- The commands for starting a one-time automatic list change and **start_loop** to start an alternating list change can be called at any point in time. However, they do not take effect until the next **set_end_of_list** is reached.
- When loading a list while another is being processed, make sure no still-needed commands are thereby overwritten. Useful here is **load_list**, which only starts loading a list if it is currently not in use or already has been processed, see [Chapter 6.4.1 "Loading Lists", Page 94](#).
- Moreover, the currently new list should have made a certain amount of loading progress before the list change occurs. The input pointer should always be adequately ahead of the output pointer (because the PCI transfer of the list commands is buffered, see [Chapter 6.4.1 "Loading Lists", Page 94](#), and so-called short list commands can be used, see [Chapter 6.1.2 "Control Commands and List Commands", Page 83](#)). Otherwise, "old" commands might be unintentionally executed.
- The RTC5 PCI Board does not support the RTC4 circular queue mode, see [Chapter 6.5.4 "RTC4 Circular Queue Mode", Page 110](#). However, this operating mode can also be effectively replaced using an alternating list change and **load_list** described above.

6.5 Structured Programming

The RTC5 command set supports structured programming and output of list commands by numerous ways to define subroutines and character sets, as well as list commands for controlling program flow.

6.5.1 Subroutines

- As list-command sequences, subroutines can principally be located in any part of the list memory.
- Preferably, subroutines should be written to a upper portion of the list memory, see [Section ""List 3"– Protected List Memory Area", Page 91](#).
- A list boundary should not run through a subroutine.
- A subroutines must be terminated with a **list_return**.
- It can be defined:
 - [Non-Indexed Subroutines](#)
 - [Indexed Subroutines](#)

Non-Indexed Subroutines

As with "normal" list-command sequences, non-indexed subroutines are loaded into a list memory area ("List 1" or "List 2") by list-loading commands (see [Chapter 6.4.1 "Loading Lists", Page 94](#)). Each subroutine must be terminated with a **list_return**. It is called by **list_call** with a parameter specifying the absolute memory address.

After the subroutine (including the terminating **list_return** command) has been processed, it is continued with the command that follows the calling position.

Notes

- Non-indexed subroutines cannot be written directly to the protected list memory area "List 3". However, they can be subsequently protected, see also [Section "Subsequent Protection and Conversion of Non-Indexed Subroutines", Page 105](#). For the subroutine to be indexed for this purpose with **set_sub_pointer**, however, its start address must be known. Prior to loading a non-indexed subroutine into the list memory area "List 1" or "List 2", you should therefore always read out the start address by **get_input_pointer**.
- Make sure that there is no **list_return** in a normal list flow or in a body of a subroutine, for which there has not been a corresponding subroutine call. Otherwise, with nested subroutine calls the integrity of the nesting is destroyed. If there is no still active subroutine call, list processing is continued at the absolute position 0.
Valid as of version OUT 540: If a subroutine begins directly after a **list_call**, **list_call_abs**, **list_call_repeat**, or **list_call_abs_repeat**, then the return address is automatically set from $\text{Pos}(\text{list_call}) + 1$ to $\text{Pos}(\text{list_return}) + 1$. That is, the next processed command is the one which follows after **list_return** but not the command which follows after **list_call** (which would process the subroutine once again having an uncorrelated **list_return**).

Indexed Subroutines

load_sub assigns a desired index to a subroutine (which is defined by subsequent list commands), and loads it into the protected list memory area “List 3”.

An indexed subroutine must be terminated by a **list_return** (otherwise it is not stored). It is called by **list_call** (with a parameter specifying the index).

A maximum of 1024 subroutines can be stored. The management of the indexed subroutines occurs on the RTC5 PCI Board automatically.

For more information on index management, see [Section “Index Management and Defragmentation”, Page 104](#).

The memory address of an indexed subroutine can be queried by **get_sub_pointer**. With it, the indexed subroutine can be called by specifying the absolute memory address (just as with non-indexed subroutines).

An indexed subroutine is only stored by **load_sub** under the following circumstances:

- If prior to loading, configuration of “List 1” and “List 2” resulted in a protected list memory area “List 3” of sufficient size. For example, if all memory is assigned to one or both lists, then no indexed subroutines can be stored. **get_list_space**, if called after a **load_sub** (but before the terminating **list_return**), can be used for querying the amount of still-available memory in the protected list memory area “List 3”
- If the indexed subroutine is terminated by a **list_return**
- If **list_return** is preceded by no other command for positioning the input pointer (for example, another **load_sub**, **set_input_pointer**, or **set_start_list_pos**)
- If the index is within the valid range (0...1023)

After a **list_return**, the input pointer becomes invalid. Any subsequent list commands are no longer stored.

Indexed subroutines are written to “List 3” by **load_sub** commands in order of entry. The starting address is automatically set after the end of the last subprogram.

If an indexed subroutine is stored using an already-existing index, then the prior subroutine with that same index is not overwritten. It remains in the protected list memory area “List 3”, though it is no longer indexed. Therefore, it can no longer be called through its index by **sub_call** (whereas it can be still called through its absolute memory address by **list_call**).

Use **get_sub_pointer** to query whether a subroutine is referenced by a particular index. If no subroutine is referenced, **get_sub_pointer** returns the value “-1” (that is, $2^{32}-1$).

To load an indexed subroutine into the protected list memory area “List 3” that is already fully loaded with indexed subroutines, you must first appropriately expand the size of the protected list memory area “List 3” by **config_list** and then defragment it by **save_disk/load_disk**. Note that expanding the size alone is not sufficient, see [Section “Index Management and Defragmentation”, Page 104](#).

Notes on Not-Indexed Calls

- Index management or defragmentation, see [Section “Index Management and Defragmentation”, Page 104](#), can result in a change of the indexed subroutine absolute memory address. SCANLAB therefore advises against calling an indexed subroutine by **list_call**.

Rules for Programming

Observe the following guidelines when programming indexed subroutines:

- In an indexed subroutine, **set_end_of_list** is replaced by a **list_nop**.
- Absolute jumps within or out from the protected list memory area "List 3" are ignored during processing, see [Chapter 6.5.3 "Jumps", Page 109](#). Therefore, absolute jumps cannot be used in indexed subroutines.
- When the subroutine is processed, also ignored are:
 - Relative jumps that exceed the boundaries of an indexed subroutine
 - **Jump commands** which initiate a jump to themselves

General Information on Calling Subroutines

Nested calls up to a maximum depth of 63 are possible.

When calling with **sub_call** or **list_call**, only relative **Jump commands** and **Mark command** may be used, if the subroutine execution is to be repeated at different **Image field** places.

To be able to use the absolute **Jump commands** and **Mark commands**, which are often easier to handle, the so-called "**AbsCalls**" are provided.

"AbsCalls"

If the subroutines contain only relative **Vector commands** and "**Arc**" commands, see [Chapter 7.1.1 "Marking with Vector Commands and "Arc" Commands", Page 124](#), the corresponding processes can be repeated at different places in the **Image field**.

With "**AbsCalls**" the current position is taken over as offset. This offset is then taken into account for all subsequent **Vector commands** and "**Arc**" commands in the subroutine.

Nested calls are taken into account when the offset is determined. This can be used, for instance, to define character sets by absolute vectors.

"**AbsCalls**" from subroutines are made with **list_call_abs** and **sub_call_abs**.

Conditional Calls

To enable calling of subroutines ("normal" calls and "**AbsCalls**") dependent on external control signals, additional commands are available for conditional branching during program execution (see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#)).

Repeatedly Executed Calls

sub_call_repeat, **sub_call_abs_repeat**, **list_call_repeat** and **list_call_abs_repeat** can be used to automatically execute the body of a subroutine several times.

Index Management and Defragmentation

Index Management

To duplicate, renumber or convert indexed subroutines, **copy_dst_src** is provided.

By **copy_dst_src**, an indexed subroutine is indexed one more time. **copy_dst_src** only alters the corresponding entries in the internal management table and does not modify the list memory content.

No longer needed indices (unneeded entries in the internal management table) can be deleted by **load_sub** directly followed by a **list_return**. Here, too, deletion occurs only in the internal management table, while the list commands of the previously indexed subroutine continue to reside in the list memory.

get_sub_pointer can be used to query whether a subroutine for a particular index exists. If no subroutine exists, **get_sub_pointer** returns a “-1” value (that is, $2^{32}-1$).

A true duplicate of an indexed subroutine in the protected list memory area “List 3” can be created (after **copy_dst_src**) with **save_disk/load_disk**. Subroutines with multiple indices are thereby written several times to the list memory. Keep this in mind in order to prevent unintended memory overflow of the protected list memory area “List 3”.

load_sub always enters a new indexed subroutine after the indexed subroutine with the highest memory address. Therefore, subroutines that are no longer required and are located in the protected list memory area “List 3” may block memory positions for further indexed subroutines.

For this reason, simply increasing the size of the protected list memory area “List 3” by **config_list** fails to produce further usable memory for storing additional indexed subroutines (the protected list memory area “List 3” can only be expanded downward, not upward).

Defragmentation

This situation can be resolved by defragmenting with **save_disk/load_disk**.

Thereby, all indexed subroutines and (by **set_sub_pointer**) subsequently indexed subroutines are rewritten to the protected list memory area “List 3” (in index sequence, starting at the lowest memory position of the protected list memory area “List 3”).

The now-available upper memory positions can then be used for storing additional indexed subroutines.

Notes

- Before calling **load_disk**, be sure the protected list memory area “List 3” is of sufficient size after configuration of “List 1” and “List 2”. Indexed subroutines without sufficient space there are *not* stored by **load_disk**. **save_disk** returns the number of stored list commands. No-longer-needed subroutines should previously be deleted from the index management by a **load_sub** which is directly followed by a **list_return**.
- In some circumstances, index management or defragmentation can alter the absolute memory address of an indexed subroutine. It is therefore not advisable to call an indexed subroutine by **list_call**.
- **save_disk** stores subroutines starting from the start address to the first-encountered **list_return**. Relative jumps are not evaluated. So do not use branches to several **list_return**. Instead, reclose eventual branches prior to one single **list_return**.

Subsequent Protection and Conversion of Non-Indexed Subroutines

Non-indexed subroutines can be (directly) written only to a list memory area ("List 1" or "List 2"), but not to the protected list memory area "List 3".

There are basically two methods to protect non-indexed subroutines subsequently:

(1) Changing the configuration

The part of the list memory area in which the non-indexed subroutine has been written is assigned to the protected list memory area "List 3" by **config_list**. The subroutine subsequently protected in this manner remains non-indexed (with unaltered memory address) and can, as before, be called by **list_call**.

(2) Converting to indexed subroutines

set_sub_pointer is used to index a non-indexed subroutine and thus include it in the memory management of the indexed subroutines.

By **save_disk/load_disk**, it can subsequently be copied as an indexed subroutine to the protected list memory area "List 3", see [Section "Defragmentation", Page 104](#).

With a subsequent call by **sub_call** via the index, the subroutine in the protected list memory area "List 3" is then started.

If you want to subsequently protect a non-indexed subroutine – either by method 1 or method 2 – then be aware that absolute jumps within and out from the protected list memory area "List 3" are not allowed, see [Chapter 6.5.3 "Jumps", Page 109](#).

With converting to indexed subroutines (method 2), also all other programming rules for indexed subroutines must be observed, see [Section "Rules for Programming", Page 103](#).

Always try to use only one of the two methods. This avoids unintended data loss in the protected list memory area "List 3" by overwriting.

If you begin working with method 1 but later want to also use indexed subroutines: then you should convert all non-indexed subroutines residing in the protected list memory area "List 3" to indexed subroutines using method 2, before you define the first indexed subroutine by **load_sub**.

When doing so, observe the following Notes.

Notes

- If method 1 is used and you remove overwrite-protection for a part of the protected list memory area "List 3", then you risk overwriting indexed subroutines or previously protected subroutines.
- Non-indexed subroutines subsequently protected with method 1 can under some circumstances be overwritten by a later **load_sub** or **load_disk**.
- **set_sub_pointer** links the supplied index with the specified start address, even if an indexed subroutine had already been previously defined for this index. The original indexed subroutine with this index is then no longer indexed and no longer callable by the index.

- If using method 2, you should use it fully. If the **set_sub_pointer** alone is executed, then the subroutine is already callable by **sub_call** and its index, but the subroutine remains unprotected against overwriting. Protection is obtained only after the subroutine is subsequently copied as an indexed subroutine by **save_disk/load_disk** into the protected list memory area "List 3".
- **save_disk** ignores all non-indexed subroutines, even those subsequently protected in the protected list memory area "List 3" by method 1. Be aware that they can be overwritten there by **load_disk**.
- **save_disk/load_disk** automatically replaces unallowed commands (for example,, **set_end_of_list**) with **list_nop** commands.
- Indexed subroutines repeatedly indexed with **copy_dst_src** are duplicated in the list memory at a subsequent **save_disk/load_disk**. This can result in a memory overflow in the protected list memory area "List 3".
- Before executing **load_disk**, be sure the protected list memory area "List 3" is of sufficient size after configuration of "List 1" and "List 2" (**save_disk** returns the number of stored list commands). An indexed subroutine is *not* stored by **load_disk** if there is not sufficient memory.
- Conversion of a subroutine by method 2 changes the absolute memory address of the subroutine.

Unprotecting Subroutines

The protection of a subroutine stored in the protected list memory area "List 3" is removed, if it is assigned by **config_list** to one of the list memory area "List 1" or "List 2".

The subroutine can then still be called using the same parameters (index or absolute memory address). But it no longer has protection against unintentional overwriting.

6.5.2 Character Sets and Text Strings

For marking tasks, it is convenient to use the RTC5 list memory for storing command lists as separate subroutines that define how the scan system should mark the needed characters and/or text strings.

To simplify management of characters and text strings, the RTC5 PCI Board provides the possibility of storing indexed character definitions and text string definitions in its protected list memory area "List 3" and calling them by simple commands.

Indexed character definitions and text string definitions are essentially indexed subroutines, but definable and callable by their own commands, and managed by a dedicated internal RTC5 PCI Board management table – separately from indexed subroutines.

The individual character and text string definitions must specify the shape and orientation (for example, parallel to the x or y axis) of the characters or text strings. Both relative and absolute **Vector commands** can be used for this. The end position of a character or a text string should be chosen to serve as the start position of a subsequent character. Each character definition or text string definition must be terminated with **list_return**.

Defining Indexed Character Sets

A sequence of character-defining list commands can be directly stored in the protected list memory area "List 3" by **load_char** (the resultant automatically-assigned memory address can be queried by **get_char_pointer**). Alternatively, a non-indexed subroutine can be subsequently indexed with **set_char_pointer** and then copied by **save_disk/load_disk** as an indexed character in the protected list memory area "List 3".

The RTC5 PCI Board manages up to 4 character sets, each with 256 indexed characters.

Other than that, the same rules as for indexed subroutines are applicable, see [Section "Indexed Subroutines", Page 102](#) and [Section "Subsequent Protection and Conversion of Non-Indexed Subroutines", Page 105](#).

Notes

- \0 (NUL) is a markable character, too.
 \0 also serves as a text-output delimiter (for text strings), in which case it is not marked.
- Indexed character set definitions cannot use **mark_text**, **mark_time**, **mark_date** and **mark_serial**. Otherwise, improper marking might occur during execution of the indexed character.

Calling Indexed Characters

Marking of an individual character is started by calling **mark_char** (or the "AbsCall" command **mark_char_abs**) along with the index of the corresponding indexed character definition.

To label serial numbers, indexed characters (digits) can also be called up with **mark_serial**, see [Chapter 7.5 "Marking Dates, Times and Serial Numbers", Page 196](#).

The marking of entire text passages can be started by **mark_text** (or the "AbsCall" command **mark_text_abs**). The desired character set can be selected in advance by **select_char_set**.

When a **mark_text** is loaded, the to-be-marked text (if more than 12 characters in length) is split into blocks of 12 characters, with each block receiving its own **mark_text** in the list memory. Make sure that no unwanted memory overflow of the respective memory area occurs.

Defining Indexed Text Strings for Times, Dates and Serial Numbers

For the marking of times, dates and serial numbers, it can be useful to define text strings such as months ("January" ... "December", "Jan." ... "Dec.", "/01/" ... "/12/" etc.) and days of the week ("Sunday" ... "Saturday" or "Sun." ... "Sat." etc.).

Here, you can likewise use previously-defined character sets with the **mark_char** and **mark_text**.

With **load_text_table**, a sequence of list commands defining a text string can be loaded directly into the protected list memory area "List 3" as an indexed text string (the resultant automatically-assigned memory address can be queried by **get_text_table_pointer**).

Alternatively, a non-indexed subroutine can be subsequently indexed with **set_text_table_pointer** and then copied by **save_disk/load_disk** as an indexed text string in the protected list memory area "List 3".

The RTC5 PCI Board manages up to 42 indexed text strings.

Other than that, the same rules as for indexed subroutines are applicable, see [Section "Indexed Subroutines", Page 102](#) and [Section "Subsequent Protection and Conversion of Non-Indexed Subroutines", Page 105](#).

Notes

- **set_char_table** is synonymous with **set_text_table_pointer**.

Calling Indexed Text Strings

Indexed text strings can be called for marking times, dates and serial numbers by **mark_time**, **mark_date** and **mark_serial** (or the "AbsCall" commands **mark_time_abs**, **mark_date_abs** and **mark_serial_abs**), see Chapter 7.5 "Marking Dates, Times and Serial Numbers", Page 196.

Managing Indexed Characters and Text Strings

The index management of indexed characters and indexed text strings occurs separately from the index management of indexed subroutines.

Index management by users (renumbering, duplicating, ...) resembles index management of indexed subroutines, see Section "Index Management and Defragmentation", Page 104, using **copy_dst_src**, **load_char**, **load_text_table**, **get_char_pointer**, **get_text_table_pointer** and **save_disk/load_disk**. Defragmentation of the protected list memory area "List 3" also includes indexed characters and text strings.

6.5.3 Jumps

list_jump_pos (synonymous with **set_list_jump**) and **list_jump_rel** allow the definition of jumps to a specified address which are carried out by the RTC5 PCI Board at runtime.

With **list_jump_pos**, an absolute memory address within the configured list memory area ("List 1" and "List 2") can be specified. Jumps into and out of the protected list memory area "List 3" are not allowed with **list_jump_pos**. A **list_jump_pos** having such an unallowed jump address is ignored during execution.

With **list_jump_rel**, jump distances (that is, relative memory addresses) can be specified. **list_jump_rel** can be used in all list memory areas, even the protected list memory area "List 3". Nevertheless, when specifying jump addresses, you should be sure the jump does not exceed the boundary of the corresponding memory area. Otherwise, **list_jump_rel** is ignored by the RTC5 PCI Board during processing.

If **list_jump_rel** is used in an indexed subroutine, you must further ensure the jump does not exceed the boundaries of the subroutine. During processing of indexed subroutines, relative jumps that exceed the boundaries of a subroutine are ignored by the RTC5 PCI Board.

Notes

- Reconfiguration of the list memory or conversion of a subroutine can result in an originally-valid jump address becoming invalid due to new list boundaries or an altered subroutine position in the memory. In this case, the RTC5 PCI Board ignores the corresponding **Jump command** – hence, the user program does probably no longer function as intended. Therefore, exercise care when programming **Jump commands**.
- When conditional **Jump commands** are used, execution of a jump is dependent on an external control signal, see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#).
- **Jump commands** initiating a jump to themselves as `list_jump_rel( 0 )` are ignored at runtime to prevent an infinite loop that excludes further activities. On the other hand, conditional **Jump commands** as `list_jump_rel_cond( Mask1, Mask0, 0 )` are allowed, for example, to wait for confirmation of a signal.

6.5.4 RTC4 Circular Queue Mode

With the RTC5 PCI Board, the RTC4 circular queue mode does not exist.

Nevertheless, users can actually replace this operational mode with the RTC5 PCI Board by using an alternating list change and **load_list**.

`load_list ( 3, 0 )` ensures that new commands are loaded only into an already processed list (that is not **BUSY list execution status**), without needing to explicitly specifying the number of the list, see also [Section "Alternating List Changes", Page 100](#) and [Section "Loading with Protection", Page 95](#).

6.5.5 Loops

Although list jumps, see [Chapter 6.5.3 "Jumps", Page 109](#), and conditional jumps, see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#), let you repeat any number of list commands limitlessly or under external control, precisely specifying the number of executions is not always reliably possible.

But this can be achieved by the command pair **list_repeat** and **list_until**. The command sequence between these two short list commands execute exactly as often as specified with the **list_until** command's parameter, but at least once. Here, nesting up to 8 loops deep is allowed.

list_repeat and **list_until** must always be used in pairs. Unpaired or supernumerous commands (**list_until** without an associated **list_repeat**, as well as **list_repeat** commands leading to a nesting depth greater than 8) are ignored. Empty loops (for example, **list_repeat** directly followed by **list_until**) terminate immediately and are not repeated.

The command pairs can be located both within lists and within subroutines.

Within subroutines, **list_until** performs a **list_jump_rel** to the address directly after the associated **list_repeat**. Loops do not function beyond the boundaries of a subroutine, because list jumps into or out of subroutines are not allowed, see [Chapter 6.5.3 "Jumps", Page 109](#).

Within lists, however, **list_until** executes a **list_jump_pos** (to the address directly after the associated **list_repeat**). Thus, **list_repeat** and **list_until** can even reside in two different lists, provided that list changing is ensured (by either an explicit list jump or an automatic list change).

If, on the other hand, a list actually has been terminated (as may be the case when using **auto_change_pos**), then the **list_repeat** stack gets automatically deleted and the started loop can no longer be ended because the next **list_until** no longer finds an associated **list_repeat**.

set_end_of_list deletes the entire loop management, if no automatic list change is pending, but **list_return** does not.

Explicit list jumps into or out of the body of a **list_repeat/list_until** loop are allowed because they cannot be monitored. Careless use could therefore compromise loop management integrity so severely that started loops do not execute as expected (but subroutine calls from inside a loop are always reliably possible as long as the subroutine itself contains no unpaired **list_repeat/list_until** commands).

If a **list_repeat/list_until** loop is to be executed with an initially unknown number of repetitions, a high value (for example, greater than the highest expected number) can be specified for the **Number** parameter of **list_until**. Within the loop, a conditional branch (for example, which is dependent on an external signal) can jump to a position outside the loop and leave the loop this way. At this point there should be a **list_until**(**Number** = 0) to end the just left loop properly.

6.6 Using Several RTC5 PCI Boards in One PC

As many RTC5 PCI Boards can be operated in one PC at the same time as the PC provides PCI slots.

The RTC5 PCI Boards work completely independently of each other. The command lists of all boards can be loaded and executed at any time.

There are two different methods for writing user programs using several RTC5 PCI Boards:

- “Multi-Board Programming”, Page 112
- “Single-Board Programming”, Page 113

6.6.1 Multi-Board Programming

In this programming method, the multi-board versions (command names with prefix “n_”) of the RTC5 commands are used.

Compared to single-board commands (command names without prefix “n_”), the board number⁽¹⁾ (32-bit unsigned value) to which the command is to be transmitted must be specified before the parameters. All other parameters are identical.

The installed RTC5 PCI Boards are numbered in the order found during initialization (starting with 1).

The multi-board command `n_get_serial_number` can be used to determine which RTC5 PCI Boards have been assigned numbers. See also example (3) below.

`rtc5_count_cards` returns the number of RTC5 PCI Boards in the RTC5 board management.

Notes

- Multi-board commands are sent to the active board (default board), if the specified number is > `rtc5_count_cards` or 0 (real boards begin at 1).
- If no real card is entered in the RTC5 board management under the specified number, the Multi-board command is rejected.

All multi-board commands are listed in [Chapter 10.1 “Overview”, Page 278](#). for nearly all single-board commands a corresponding multi-board command is available. Exceptions are explicitly noted in the command descriptions (in [Chapter 10.2 “RTC5 Command Set”, Page 290](#)).

Examples (Pascal)

- (1) Write a **Jump command** to the point (500, 500) into the current list of RTC5 PCI Board #1:

```
n_jump_abs(1, 500, 500)
```

- (2) Process list with number `list_no` (1 or 2) on the RTC5 PCI Board with the number specified by the variable `RTC5_no`:

```
n_execute_list(RTC5_no, list_no)
```

- (3) Return the serial number of RTC5 board #1:

```
sn_1 := n_get_serial_number(1)
```

(1) As an unsigned 32-bit value.

6.6.2 Single-Board Programming

During single board programming, one of the inserted RTC5 PCI Boards is defined with **select_rtc** as default card. All single board commands following **select_rtc** are sent to the defined board until **select_rtc** is called once again.

multi-board commands are not influenced by **select_rtc** (if the card number is valid, see above).

Care must be taken if a process uses multiple boards by multiple threads, because **select_rtc** is not thread-specific but board-specific. It immediately redirects the output of *all* currently running threads of a process to the specified RTC5 PCI Board.

Notice!

- **select_rtc** defines the active RTC5 PCI Board for all threads of one process (user program) that are currently running.
In multi-threaded user programs, this can result in programming errors.

6.6.3 Master/Slave Operation

If several RTC5 PCI Boards are to be operated clock-synchronized, then they must be connected pairwise with each other by the Master and Slave connectors and installed in preferably (recommended) adjacent PCI slots. Connect the Master connector of one board to the Slave connector of another board. Suitable connection cables (see [Figure 8](#)) are available from SCANLAB.

An RTC5 PCI Board automatically gets the master board of a master/slave chain, if a further RTC5 PCI Board is connected to its Master connector but no further RTC5 PCI Board is connected to its Slave connector. All other RTC5 PCI Boards are slave boards. See also [Chapter 4.4 "Master Socket Connector, Slave Socket Connector"](#), [Page 53](#).

get_master_slave can be used to query separately for each RTC5 PCI Board the master/slave status, that is, whether it is operated as a master, slave or single board.

For a source code example on how to check which RTC5 PCI Board is the master and which one is slave, see [Section "Example Code \(Delphi\)"](#), [Page 116](#).

Initialization

On all RTC5 PCI Boards of a master/slave chain must have been executed:

- **load_program_file**
- **load_correction_file**

The synchronous timing with stable phase position of a master/slave chain is severed by the first not-initialized board. If an RTC5 PCI Board is initialized by **load_program_file** but connected as slave to a board which has not been initialized by **load_program_file**, then it is subject to its own clock with a random phase position.

Clock Phase Synchronization

If the RTC5 PCI Boards of a master/slave chain are to be synchronously clocked with a defined relative clock phase, then the boards must be correspondingly synchronized by **sync_slaves**.

For this, it is necessary to send **sync_slaves** one-time to the master board only. SCANLAB recommends performing the synchronization immediately after all boards have been initialized (by **load_program_file** and **load_correction_file**). It is sufficient to call **load_correction_file(0,1,2)** or to temporarily detach all *scan heads*.

After synchronization, the clock phase of each slave board is (reproduceably) delayed by approx. $0.16 \mu\text{s}$ in relation to the clock phase of the preceding (upstream) board.

Without synchronization, delays of up to $10 \mu\text{s}$ can occur. You can use **get_sync_status** to check if a slave board is synchronized to the master board (or to the preceding board in the master/slave chain).

Notes

- The master board does not pass encoder signals to the slave board(s). They must always be individually supplied to the slave board(s). Here, you need to take into account the 160 ns clock phase shift.
- The correction file and lists must be loaded separately onto all RTC5 PCI Boards.

Matching of Short-Command Counts

The maximum allowed number of directly consecutive short list commands within a $10 \mu\text{s}$ clock cycle depends on the **DSP** version. To ensure that short list commands execute synchronously even when using multiple RTC5 Boards with differing **DSP** versions in a master/slave chain, the **sync_slaves** command (if sent to the master board) reduces the short list count on all faster boards in the master/slave chain to equal that of the slowest board (that is, the board with the lowest **DSP** version number).

Notes

- The CPU clock frequency is not altered, only the count of short list commands.
- You can also use the **set_dsp_mode** command to make appropriate individual adjustments for each RTC5 Board.
- But note that some commands (for example, **mark_ellipse_abs**) are only available on boards with higher-numbered **DSP** versions. Adjusting the short-command count does not change this fact. To ensure that the master/slave chain remains synchronized, only use commands that are available even for the board with the lowest **DSP** version number.

Synchronous Starts and Synchronous Stops

Within a master/slave chain, **External Starts** (if enabled with **set_control_mode**) and **External Stops** are passed on:

- from one board to all downstream slave boards

Therefore, it can be triggered:

- A synchronous start of all boards (with presettable track delays) by an external start signal, a **simulate_ext_start** or a **simulate_ext_start_ctrl** at the master board
- A synchronous stop of all boards by an external stop signal or a **simulate_ext_stop** at the master board

In contrast, *not* passed on are:

- Starts by **execute_list**
- Starts by **execute_at_pointer**
- Stops by **stop_execution**

Therefore, these must be separately executed even at master/slave-synchronized boards.

Notes

- See also [Chapter 9.3.1 "Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization"](#), Page 265.

Example Code (Delphi)

The following example Delphi source code shows how to check which RTC5 PCI Board is the master and which one is slave.

The code must be included in a user program, see [Chapter 6.2.5 "Example Code \(C\)", Page 89](#).

```
if init_rtc5_dll() <> 0 then halt; // Initialize the RTC5 DLL
if rtc5_count_cards() <> 2 then halt; // Are 2 RTC5 in the PC?

for CardNo := 1 to 2 do // Load program and 3D correction files onto both boards
begin
    n_stop_execution(CardNo); // Stop RTC if a task is currently still running
    if n_load_program_file(CardNo, nil) <> 0 then halt;
    if n_load_correction_file(CardNo, 'D3_nnn.ct5', 1, 3) <> 0 then halt;
end;

// Detect master board
// Is board 1 the master and board 2 a single slave?
if (n_get_master_slave(1) = 2) and (n_get_master_slave(2) = 1) then
begin
    Master := 1;
    Slave := 2;
end else
// Is board 2 the master and board 1 a single slave?
if (n_get_master_slave(1) = 1) and (n_get_master_slave(2) = 2) then
begin
    Master := 2;
    Slave := 1;
end
else
halt; // Something wrong with master-slave configuration

n_select_cor_table(Master, 1, 0);
n_select_cor_table(Slave, 1, 0);

n_set_control_mode(Master, 1 + 8); // Master slave, activate Start Stop control
n_set_control_mode(Slave, 1 + 8); // For external control, you must use the master
// board's control inputs
n_sync_slaves(Master); // Synchronize master and slave boards
// check synchronization status at any time
// provide an External Start or a simulated External Start before checking
n_simulate_ext_start_ctrl(Master);
Result = n_get_sync_status(Master); // must be 640
Result = n_get_sync_status(Slave); // must be < 11
```


6.7 Usage of RTC5 PCI Boards by Several User Programs

Usage of RTC5 PCI Boards by several user programs is coordinated by the (RTC5 DLL-internal) RTC5 board management. By `init_rtc5_dll`, it is initialized.

By `acquire_rtc`, the `init_rtc5_dll` command automatically assigns access rights to the found boards, as long as access rights are not already assigned to another user program (any number of RTC5 Boards or applications can be used simultaneously, but no board can be simultaneously used by multiple applications). Access rights (even if temporary) to existing boards are assigned on an exclusive basis by the DLL. Multiple threads of one user program can use the same board, but can not send commands to it at the same time (the RTC5 DLL automatically serializes the command calls).

Without access rights, a board can only be accessed by a user program through purely RTC5 DLL-internal functions that do not require access rights – for example, `get_error`, `get_last_error` or `select_rtc`.

If a user program has gained access rights for a board, then this board can be accessed by another user program only after the original user program explicitly releases its access rights by `release_rtc` or `free_rtc5_dll`. Acquisition of a released board can then be achieved by `acquire_rtc` (or `init_rtc5_dll` or `select_rtc`).

When a board is acquired by `acquire_rtc` (or `init_rtc5_dll` or `select_rtc`), a version compatibility check is performed on the RTC5 DLL and the program files `RTC5OUT.out` and `RTC5RBF.rbf`. If no program files have been loaded, then a version check cannot be explicitly performed but the check is still regarded as successful and does thus not hinder the board's acquisition. If program files have been loaded and the version check detects an error, then access is denied (`get_last_error` return code

`RTC5_ACCESS_DENIED`|`RTC5_VERSION_MISMATCH`).

6.7.1 Board Acquisition by a User Program

`init_rtc5_dll`, `acquire_rtc`, `free_rtc5_dll`, `release_rtc` and `select_rtc` affect the access rights of the installed RTC5 PCI Boards. They do not initialize RTC5 PCI Boards.

A user program acquiring a board by `acquire_rtc` (or `init_rtc5_dll` or `select_rtc`) inherits its unadjusted memory contents and operational state. The user program therefore can use the stored data and settings of the board and could intervene in the flow of any list program started (by the previous user program).

If a user program releases a board by `release_rtc` and subsequently reacquires it – without it having been acquired in the meantime by another user program – then the RTC5 Board can be further used without changes, because in this situation all RTC5 DLL configuration data remain unaltered. However, the above does not apply if the acquired board has been released by `free_rtc5_dll` and reacquired with `init_rtc5_dll`.

When an RTC5 Board is acquired by another user program, some of the previous user program's key data managed only in the RTC5 DLL is *not* (automatically) inherited. The acquiring user program thereby lacks information related to memory configuration, protected-area management, or the operational status.

If board acquisition is followed by a board reset by `load_program_file`, then all settings are newly defined anyway and such missing information would not be relevant.

On the other hand, if the acquiring user program intends to further use the RTC5 Board's inherited state, then it must explicitly query the missing **RTC5 DLL** information, receive it from the previous user program and explicitly re-establish it so that the board's **RTC5 DLL** remains consistent. In this regard, observe the following:

- For a correct behavior of the input pointer at the list borders, the memory configuration currently set in the **RTC5 DLL** for the acquiring user program must be consistent to the current memory configuration of the acquired board. **get_config_list** obtains the current memory configuration of the board and sets it in the **RTC5 DLL** correspondingly.
- The management table of protected functions (indexed subroutines, character sets and text strings) is saved on the board and therefore inherited through board acquisition. All protected functions stored on the board therefore remain callable. On the other hand, information on where the next protected function should be loaded is lost. This information can only be restored by **save_disk/load_disk** (see [page 104](#)). An alternative restoration method is not possible. The intermixed loading of protected functions by differing applications should therefore be avoided.
- If, during loading a protected function, **release_rtc** is called before a subsequent **list_return** command is transferred, then the function is not stored on the RTC5 Board.
- The input pointer is generally not inherited (the input pointer location currently saved in the **RTC5 DLL** for the acquiring user program is used, maybe corrected by **get_config_list**). On the other hand, output pointers of lists can be queried after an acquisition by **get_status**.
- After acquisition and until the next **load_...** command, the list status (with reference to **LOAD list status** and **READY list status**) might be incorrect. But for further execution this is not important, and the status is newly set after the next **load_...** command.
- Other settings such as **start_loop** or laser settings are not relevant to the DLL. Though settings used by the previous user program can not generally be queried, new settings can of course be established as desired.
- Error handling is performed separately for each board and each user program. When access rights are exchanged, this data is not included.

6.8 Error Handling

So that RTC5 errors can be caught at program runtime by suitable programming, the RTC5 performs general error handling. In addition, some commands allow for specific error handling.

General errors occur, for example, if a user program has no access rights for the board (**RTC5_ACCESS_DENIED**), or if the board fails to respond to a control command (**RTC5_TIMEOUT**), or if PCI communication problems occur during loading (**RTC5_SEND_ERROR**).

Examples of specific errors are: calling a command with an unallowed (uncorrectable) parameter (**RTC5_PARAM_ERROR**, for example, see **get_value** or **write_da_x**), or rejected loading of a list command (**RTC5_REJECTED**, for example, due to an illegal input pointer), or transmitting a control command at an improper time (**RTC5_BUSY**, for example, **goto_xy**, when a list is still being processed).

In such cases, the control commands are not executed; list commands are typically replaced with **list_nop** commands (for example, for **RTC5_PARAM_ERROR** or **RTC5_IGNORED**, see **set_end_of_list** as an example).

The bits assigned to these errors are set or accumulated in the **RTC5 DLL** with each command in the board-specific error variables:

- **LastError**
 - Error code
 - Is automatically reset at the beginning of every command
 - Therefore, is a listing of occurred errors from the most recently executed command
 - Can be queried by **get_last_error**⁽¹⁾
- **AccError**
 - Cumulative error code
 - Is reset when initializing the **RTC5 DLL**
 - Can be reset by the user program itself by **reset_error**
 - Are all accumulated error bits since the last error bit reset by **reset_error**
 - Can be queried by **get_error**⁽¹⁾

Error handling takes place separately for each board and each user program. A **reset_error** does not delete the error code of another user program with current access rights to the specified board. If access rights are exchanged, this data is not also exchanged (neither is any other board data exchanged).

Error handling only takes place during initialization and when loading onto the RTC5, but not during execution of a list program.

An example program of how to incorporate board-specific error variables is provided in the description of **get_error**.

Some control commands (for example, **init_rtc5_dll**, **load_correction_file** or **load_program_file**) additionally return a special error code that is not buffered and must therefore be immediately evaluated or discarded by the user program.

(1) The described mechanism only applies for commands that establish communication with the RTC5. Commands that *do not* establish communication with the RTC5 (for example, **rtc5_count_cards**, **set_rtc4_mode** or **get_serial_number**) neither generate nor alter **LastError** or **AccError** (see also comments in the corresponding command descriptions).

6.8.1 Download Verification

Verification of RTC5 communication is vital particularly in medical applications. For this purpose, you can activate download verification separately for each board by **set_verify**.

However, this automatically results in extended download times.

If download verification is activated and an error is found, then the error code **RTC5_VERIFY_ERROR** is set, which can be queried by **get_last_error** or **get_error**. Certain operations might immediately be aborted and the board would then no longer be functional (for example, if **load_program_file** has been aborted).

With download verification activated, the following checks are performed (also note the comments in the command description of **set_verify**):

(1) Loading of list commands

For list-command downloads, each download is read back and compared (for equality) against the sent command. Here, only transfer to the RTC5 PCI Board itself is checked; automatic parameter adjustments (for example, clipping) are not taken into account.

(2) Loading of control commands

For control commands, the corresponding parameters are read back and compared for equality against the sent parameters. Automatic parameter adjustments are not taken into account.

(3) **load_program_file**

For sending of **load_program_file**, the following is checked:

- **RTC5DAT.dat** is tested by a checksum for file correctness and PCI-transfer correctness.
- **RTC5RBF.rbf** is only checked by a bitwise transfer handshake. No other checking is possible.
- Each loaded section of **RTC5OUT.out** is immediately read back for checking. If an error is detected, then the loading process aborts.

(4) Loading of correction files

For loading by **load_correction_file**, the integrity of the to-be-loaded correction file is checked (by the checksum) and the transfer itself checked for correctness by an immediate read back of the correction table. For this function, the correction file must contain a checksum (see command description of **set_verify** and **verify_checksum**).

(5) Loading of tables

For loading other tables (for example, by **load_varpolydelay**), the transfer is checked for correctness by an immediate read back of the table. In addition to the **get_last_error** return code **RTC5_VERIFY_ERROR**, the corresponding error return value of the loading command is also get set.

6.8.2 Checking for Overruns

By **get_overrun**, it can be checked whether overruns of the 10 μ s clock cycle have occurred. See also [Section "Clock Overruns", Page 171](#).

6.8.3 Example Code (C)

The following example C source code shows how to catch an error during initialization. It ensures the program terminates with an error message, if:

- An error occurs during initialization with `init_rtc5_dll` (for example, if no RTC5 PCI Board has been detected)
- The desired RTC5 PCI Board (here: the board with serial number 12345) is not detected
- Access is denied to the desired RTC5 PCI Board
- An error occurs during `load_program_file` (for example, a version mismatch, file or system error)

The code must be included in a user program, see [Chapter 6.2.5 "Example Code \(C\)", Page 89](#).

```
UINT ErrorCode;

ErrorCode = init_rtc5_dll();
if ( ErrorCode )
{
    // Reading the number of RTC5 Boards detected during initialization with init_rtc5_dll
    const UINT RTC5CountCards = rtc5_count_cards();
    if ( RTC5CountCards )
    {
        // Detailed error analysis for all detected boards
        UINT AccError( 0 );
        for ( UINT i = 1; i <= RTC5CountCards; i++ )
        {
            // Errors which occurred during execution of init_rtc5_dll
            const UINT Error = n_get_last_error( i );
            if ( Error != 0 )
            {
                AccError |= Error;
                const UINT SerialNumber = n_get_serial_number ( i );
                printf( "RTC5 Board number %d (serial number %d): Error %d detected\n",
                    i, SerialNumber, Error );
                n_reset_error( i, Error );
            }
        }
        if ( AccError )
        {
            free_rtc5_dll();
            return;
        }
    }
}
```



```
else
{
    printf( "Initializing the DLL: Error %d detected\n", ErrorCode );
    free_rtc5_dll();
    return;
}
}
else
{
    // Reading the internal board number for the desired RTC5 Board
    const UINT SerialNumberOfDesiredBoard ( 12345 );
    const UINT RTC5CountCards = rtc5_count_cards();
    UINT InternalNumberOfDesiredBoard ( 0 );
    for ( UINT i = 1; i <= RTC5CountCards; i++ )
    {
        if ( n_get_serial_number( i ) == SerialNumberOfDesiredBoard )
        {
            InternalNumberOfDesiredBoard = i;
        }
    }
    if ( InternalNumberOfDesiredBoard == 0 )
    {
        printf( "RTC5 Board with serial number %d not detected.\n", SerialNumberOfDesiredBoard);
        free_rtc5_dll();
        return;
    }
    // Selecting the desired RTC5 Board as the active RTC5 Board for this user program
    if ( InternalNumberOfDesiredBoard != select_rtc( InternalNumberOfDesiredBoard ) )
    {
        // Errors which occurred during execution of select_rtc
        ErrorCode = n_get_last_error( InternalNumberOfDesiredBoard );
        if ( ErrorCode & 256 )    // RTC5_VERSION_MISMATCH
        {
            if ( ErrorCode = n_load_program_file( InternalNumberOfDesiredBoard, 0 ) )
            {
                printf( "n_load_program_file returned error code %d\n", ErrorCode );
            }
        }
        else
        {
            printf( "No access to RTC5 Board with serial number %d\n", SerialNumberOfDesiredBoard );
            free_rtc5_dll();
            return;
        }
        if ( ErrorCode )
        {
            printf( "No access to RTC5 Board with serial number %d\n", SerialNumberOfDesiredBoard );
            free_rtc5_dll();
            return;
        }
        else
        {
            // if n_load_program_file has been successful, select the desired board
            (void) select_rtc( InternalNumberOfDesiredBoard );
        }
    }
}
```

6.9 Miscellaneous

6.9.1 Free Variables

8 so-called “free” variables are available.

Users can freely assign data to them by the control command **set_free_variable** and the short list command **set_free_variable_list**.

These variable values can be:

- outputted at the **McBSP interface**, see also [Chapter 9.1.7 “McBSP Interface”, Page 263](#)
- read back by **get_free_variable** and **get_value** (see command descriptions)
- recorded by **set_trigger/set_trigger4** (see command descriptions)

The free variables let you, for example, transmit control commands over the **McBSP interface** to user hardware or document the operational states of the board (for example, with branches).

Notes

- You can use **set_free_variable** and **set_free_variable_list** to define any unsigned 32-bit values as variable values. However, the **McBSP interface** only outputs 24-bit values by **set_mcbbsp_out_ptr** and 16-bit values by **set_mcbbsp_out** (but **get_free_variable**, **get_value** and **set_trigger/set_trigger4** return full 32-bit values).

7 Basic Functions for Scan Head and Laser Control

7.1 Marking Dots, Lines and Arcs

7.1.1 Marking with Vector Commands and "Arc" Commands

As explained in [Chapter 6.1 "RTC5 Software Concept Basics", Page 83](#), positioning of the scan system axes (and thus of the laser beam) under RTC5 PCI Board control is achieved by calling:

- [Jump Commands](#)
- [Mark Commands](#)
- [Arc Commands](#)
- [Ellipse Commands](#)

Each of these commands describes one vector or arc.⁽¹⁾ By using [micro_vector\[*\] Commands](#), arbitrarily shaped trajectories⁽²⁾ can be implemented.

Even numeric and alphabetic characters ultimately consist of the constituent lines, dots and arcs that define them, see [Chapter 7.5 "Marking Dates, Times and Serial Numbers", Page 196](#).

[Vector commands](#) ([Jump Commands](#), [Mark Commands](#)) require as parameters the coordinates of the *end* point of the corresponding vector⁽³⁾. Each vector starts at the *current output position*, which is the end point of the preceding vector or arc.

["Arc" commands](#) ([Arc Commands](#) and [Ellipse Commands](#)) require parameters for the coordinates of the arc center and the arc angle(s). Circular arcs start at the current output position. The elliptical arc start at the position specified by the command parameters. A direct connection to the last output position must be explicitly ensured by the user program itself with suitable parameters. Otherwise, there is a ["Hard jump"](#) there.

The output position after a RTC5 PCI Board [Hardware reset](#) is the center of the [Image field](#), that is, the point (0|0). Refer to [Chapter 7.3 "Scan Head Control", Page 156](#) for a description of the [Image field](#) coordinate system.

At run-time, each vector or arc to be traced by the scan system gets divided by the RTC5 PCI Board into [Microsteps](#), see [Chapter 7.1.2 "Microstepping", Page 128](#)⁽⁴⁾.

[Jump commands](#) serve to move the scan system axes to a new position while the laser is switched *off*.

In contrast, [Mark commands](#) initiate a marking motion while the laser is switched *on* (see also the following description).⁽⁵⁾

To mark a point, the [Signals for "Laser Active" Operation](#) must be switched on for the desired time period after a [Jump command](#) or [\[*\]mark\[*\] Command](#), see [Chapter 7.1.3 "Marking Single Dots", Page 129](#).

For line and arc marking, the RTC5 PCI Board automatically switches the [Signals for "Laser Active" Operation](#) on at the beginning of a [Mark command](#) and later switches it back off (for example, at the beginning of a subsequent [Jump command](#)).

The synchronization of scan head control and laser control can be adjusted by the user to the respective application by setting delays, see [Chapter 7.2 "Delay Settings – Coordinating Scan Head Control and Laser Control", Page 133](#).

Adjustment of laser parameters is described in [Chapter 7.4 "Laser Control", Page 173](#).

A thoroughly-commented sample code for a basic marking task is shown in [Chapter 7.1.4 "Example Code \(C\)", Page 130](#).

(1) Here, wider line widths can be specified by [set_wobble_mode](#).

(2) See Glossary entry on [page 26](#).

(3) The coordinates must be specified as digital control values (without units). To avoid confusion with coordinates in [mm], SCANLAB uses the expression "coordinate values [in bits]".

(4) Only *iDRIVE* scan systems (see Glossary entry on [page 23](#)) which are equipped with an appropriate tuning can execute jumps also in [Jump Mode](#), see [Chapter 8.1.5 "Jump Mode", Page 202](#).

(5) Outside a list, repositioning can be achieved by [goto_xy](#) or [goto_xyz](#) (even while the laser control signals are on).

Jump Commands

A **Jump command** (**jump_abs** or **jump_rel**⁽¹⁾) causes the mirrors to move from the start point to the end point of a vector. The **Signals for "Laser Active" Operation** are automatically switched off at the beginning of the vector and remain switched off during the jump, see also **Chapter 7.2.1 "Laser Delays", Page 133** and **Chapter 7.2.2 "Scanner Delays", Page 135**. The jump speed is defined by **set_jump_speed** and **set_jump_speed_ctrl**.

If the laser system does not allow fast switching, the jump speed must be set high enough to prevent a visible marking effect on the workpiece. See also the commands **home_position** and **home_position_xyz**.

Mark Commands

Upon execution of a **[*]mark[*] Command** (**mark_abs** or **mark_rel**⁽¹⁾), the laser focus is moved linearly from the start point to the end point of the vector. The RTC5 PCIe Board automatically turns on the **Signals for "Laser Active" Operation** at the beginning of a **[*]mark[*] Command**, see also **Section "Polylines", Page 125**.

The mark speed is defined by **set_mark_speed** and **set_mark_speed_ctrl**. It can be changed anywhere in a list by **set_mark_speed** or by **set_mark_speed_ctrl**, if no list is currently being processed.

Arc Commands

The **Arc Commands** **arc_abs** and **arc_rel** can be used for marking circular arcs⁽²⁾. The parameters to be specified are coordinates of the arc center and the arc angle. The circular arc starts at the current output position, with angles counted positively and clockwise (contrary to the mathematical definition).

When an arc command is executed, the laser focus is guided along the specified arc at the specified speed. The **Signals for "Laser Active" Operation** are automatically switched on at the beginning of the execution of an arc command, see also **Section "Polylines", Page 125**.

Polylines

If another **[*]mark[*] Command** (or "Arc" command) follows immediately afterward ("Polyline"), the **Signals for "Laser Active" Operation** remain on.

Therefore, a continuous marking is possible by a direct line-up of **[*]mark[*] Commands** (and "Arc" commands).

The **Signals for "Laser Active" Operation** are switched off at the beginning of the (normal) command that follows the last **[*]mark[*] Command** of a **Polyline**.

See also **Section "EdgeLevel", Page 140**.

(1) For using abs and rel commands, see **Section ""AbsCalls"", Page 103**. Additionally are available: **timed vector commands**, see **Chapter 8.9 "Timed Commands", Page 250**, para vector commands, see **Section "Vector-Defined Laser Control", Page 194** and 3D vector commands, see **Chapter "3D Commands", Page 223**.

(2) For using abs and rel commands, see **Section ""AbsCalls"", Page 103**. Additionally, timed arc commands are available, see **Chapter 8.9 "Timed Commands", Page 250**.

Ellipse Commands

The RTC5 command set also provides commands for marking elliptical arcs.

Here (unlike marking of vectors or circular arcs), you generally need to call two commands: **set_ellipse** as well as **mark_ellipse_abs** or **mark_ellipse_rel**⁽¹⁾.

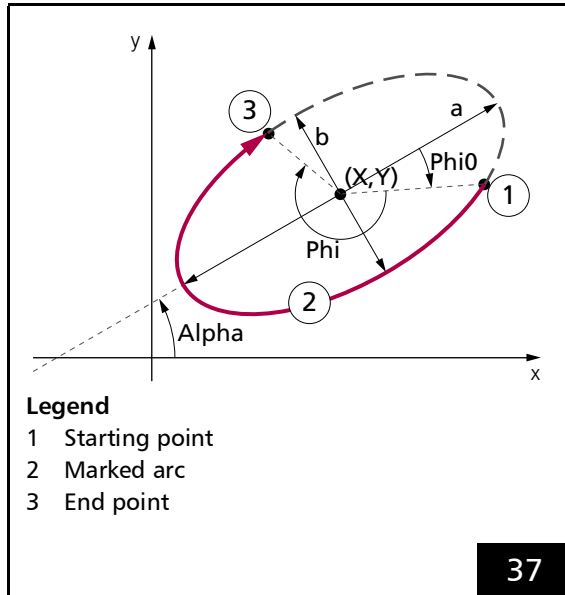
By **set_ellipse** the arc shape is specified, see **Figure 37**:

- Lengths a and b of the ellipse half-axes
- The beginning phase angle Φ_{i0} (and thereby the arc starting point position relative to the end point of half-axis a)
- The arc angle Φ_i (and thereby the length of the to-be-marked ellipse section)

By **mark_ellipse_abs** or **mark_ellipse_rel**, position and orientation of the to-be-executed arc is specified, see **Figure 37**:

- The coordinates (X, Y) of the ellipse midpoint

The angle α between the ellipse half-axis a and the x axis.



Marking ellipse-shaped arcs.

Notes

- By a , you can specify either the short or long half-axis (then use b for the other axis). Φ_{i0} , Φ_i and α are always relative to axis a .
- Φ_{i0} and Φ_i are counted positively clockwise (in contrast to mathematical convention). In contrast, α is counterclockwise (in accordance with mathematical convention).
- As with **Mark Commands** and **Arc Commands**, the laser focus moves with the specified mark speed along the specified arc when the Ellipse command is executed. The **Signals for "Laser Active" Operation** are automatically switched on at the beginning of an **"Arc" command**, see also **Section "Polylines"**, Page 125.
- **set_ellipse** is a short list command, see also **Section "Normal, Short, Variable and Multiple List Commands"**, Page 278. Therefore, it can be called between a **Mark command** and an Ellipse command without thereby interrupting the **Polyline** (the laser remains on).
- **Ellipse Commands** always begin marking at the starting point determined by the above-mentioned parameters (in contrast to **Mark Commands** and **Arc Commands** which automatically begin marking at the current output position). If the starting point and current position do not match, then a **Hard jump** to the starting point is executed at the beginning of marking (without a **Jump Delay** becoming effective).

(1) For using abs and rel commands, see **Section "AbsCalls"**, Page 103.

- Elliptical arcs can also be marked by circular **Arc Commands** (for example, **arc_abs**) if an appropriate coordinate transformation (for example, scaling that differs in the x direction/y direction) has been specified with **set_matrix**. Here, though, the effective mark speed varies along the arc, see also the note on [page 212](#).

This contrasts with **mark_ellipse_abs** and **mark_ellipse_rel**, where in 10 μ s intervals the step length gets adjusted for the ellipse's shape at the current position such that the arc is marked with a (largely) constant mark speed.

For very large eccentricities and also at high mark speeds, however, such stepwise ellipse approximation by a 10 μ s clock can produce numerical inaccuracies in the end point regions of the large half-axis. Consequently, the effective mark speed there might not be precisely constant (for example, an eccentricity of $a/b = 2$ and 100 **Microsteps** per circumference would produce a speed deviation of approx. 3.7%). However, the outputted point always lies exactly on the ellipse.

Moreover, as closed equations do not exist for calculating an ellipse arc length, the step length of the finally-marked **Microstep** is generally shorter and the mark speed correspondingly lower than specified. Nevertheless, the end position is always exact.

Likewise, **Sky Writing** might produce run-in/run-out irregularities at the large half-axis. Users themselves must ensure that the parameter values used are consistent with the required precision.

[*]Para[*] Commands

- **para_jump_abs**
- **para_jump_abs_3d**
- **para_jump_rel**
- **para_jump_rel_3d**
- **para_laser_on_pulses_list** (special case)
- **para_mark_abs**
- **para_mark_abs_3d**
- **para_mark_rel**
- **para_mark_rel_3d**
- **timed_para_jump_abs**
- **timed_para_jump_abs_3d**
- **timed_para_jump_rel**
- **timed_para_jump_rel_3d**
- **timed_para_mark_abs**
- **timed_para_mark_abs_3d**
- **timed_para_mark_rel**
- **timed_para_mark_rel_3d**

If the "vector-controlled laser control" is activated, these commands simultaneously vary a signal parameter linearly along the mark or jump vector, see [Section "Vector-Defined Laser Control", Page 194](#).

[*]**para_mark**[*] commands generally do not take **Sky Writing** into account.

7.1.2 Microstepping

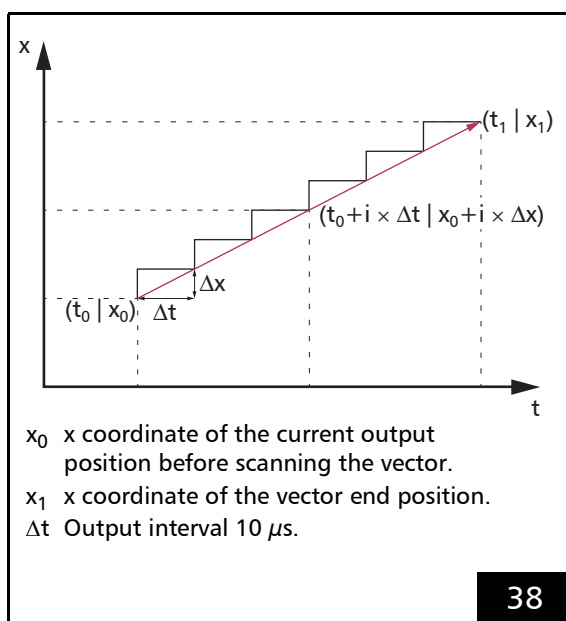
The RTC5 PCI Board splits up each

- **Jump command**,
- **[*]mark[*] Command** and
- **"Arc" command**

into so-called

- Microsteps
(not: Microvectors, see Chapter 8.8
"micro_vector[*] Commands", Page 249).

The split-up of the x component of a vector into **Microsteps** is shown in Figure 38.



The x component of a vector is split-up into **Microsteps**.
The y component is split-up in the same way.

All **Microsteps** are transferred to the scan head with a constant output interval (Δt) of 10 μs and *cannot* be changed.

The following applies:

$$\text{Length } \Delta s \text{ of a Microstep} = v \times \Delta t^{(1)}$$

(v = current jump speed or mark speed)

Notes

- Custom curves can be implemented by using **micro_vector[*] commands**, see Chapter 8.8 "micro_vector[*] Commands", Page 249.
- Only iDRIVE scan systems⁽²⁾ which are equipped with an appropriate tuning can execute jumps also in **Jump Mode**, see Chapter 8.1.5 "Jump Mode", Page 202.

(1) Alternatively see Chapter 8.9 "Timed Commands", Page 250.

(2) See Glossary entry on page 23.



7.1.3 Marking Single Dots

To mark a single point, the **Signals for "Laser Active" Operation** must be switched on for the desired time period, see **laser_on_list**, **laser_on_pulses_list**, **para_laser_on_pulses_list** and Chapter 7.4 "Laser Control", Page 173.

Alternatively, a single dot can also be marked by a timed **Mark command** of length zero, see Chapter 8.9 "Timed Commands", Page 250.

7.1.4 Example Code (C)

The following example C source code shows the commands of a simple laser scan application.

A point, a square and a circle are marked in **CO₂ Mode**. Here it is assumed that the **RTC4 Compatibility Mode** is activated.

The code must be included in a user program, see **Chapter 6.2.5 "Example Code (C)", Page 89**.

```
// Scan system initialization
// Loading and assigning a correction file
ErrorCode = load_correction_file( 0, // initialize like "D2_1to1.ct5",
                                1, // table (#1 is used by default)
                                2 ); // use 2D only

if ( ErrorCode )
{
    printf( "Correction file loading error: %d\n", ErrorCode );
    free_rtc5_dll();
    return;
}
select_cor_table( 1, 0 ); // assigning table #1 to the first scan head connector (default)

// Laser control initialization, see Chapter 7.4 "Laser Control", Page 173
// Setting the CO2 Mode
set_laser_mode( 0 );

// Setting and enabling the Signals for "Laser Active" Operation
set_laser_control( 0x18 ); // All laser signals LOW active (Bit #3 and #4)
// This command must be called at least once to activate laser signals. Later on
// enable_laser/disable_laser would be sufficient.

// Opens List 1
set_start_list( 1 );

// Setting the standby pulses
set_standby_list( 800, 8 );
// In RTC4 Compatibility Mode the standby parameters are specified in units of 1/8 µs.
// The RTC5 multiplies the specified values by 8 to convert to integer-multiple of 1/64 µs.
// Half of the standby output period = 100 µs.
// Pulse length of the standby pulses = 1 µs.

// Timing, delay and speed preset
// Setting the Scanner Delays (see Chapter 7.2.2 "Scanner Delays", Page 135):
set_scanner_delays( 25, 10, 5 );
// Jump Delay = 250 µs (specified in [10 µs])
// Mark Delay = 100 µs (specified in [10 µs])
// Polygon Delay = 50 µs (specified in [10 µs])
```



```
// Setting the jump speed and mark speed:
set_jump_speed( 1000.0 );
set_mark_speed( 250.0 );
    // In RTC4 Compatibility Mode the RTC5 multiplies the speed values by 16.
    // Jump speed = 1000.0 bits/ms.
    // Mark speed = 250.0 bits/ms.

// Setting the laser timing, see Chapter 7.4 "Laser Control", Page 173.
set_laser_pulses( 800, 400 );
    // In RTC4 Compatibility Mode the timing parameters are specified in units of 1/8 µs.
    // The RTC5 multiplies the specified values by 8 to convert in integer-multiple of 1/64 µs.
    // Laser HalfPeriod = 100 µs.
    // Laser pulse length = 50 µs.

// Setting the Laser Delays, Chapter 7.2.1 "Laser Delays", Page 133.
set_laser_delays( 100, 100 );
    // In RTC4 Compatibility Mode the Laser Delays are specified in units of 1 µs.
    // The RTC5 multiplies the specified values by 2 to convert in integer-multiple of 0.5 µs.
    // LaserOn Delay = 100 µs.
    // LaserOff Delay = 100 µs.

// Defining the end of the list and the end of command transfer to the RTC5 Board
set_end_of_list();

// Execute the list commands for initialization
execute_list( 1 );

// Marking procedure
// Waiting for list 1 to be not busy. (load_list( 1, 0 ) returns 1 if successful, otherwise 0);
// if list 1 is not (no longer) busy:
// opening the list memory for writing of list commands and setting the input pointer
// to the start of list 1
while ( !load_list( 1, 0 ) );

// In the following the list commands for marking point, square and circle are defined and transferred
// to the RTC5 board.

// Marking the center point of the Image field:
jump_abs( 0, 0 ); // Jump to center point
    // A Jump Delay is automatically inserted after the jump.
laser_on_list( 5 ); // Turning on the laser control signals for 50 µs.
```



```
// Marking a square around the center point:
jump_abs( -20000, -20000 ); // Jump to the bottom left corner of the square
// A Jump Delay is automatically inserted after the jump.
mark_abs( -20000, 20000 ); // Marking the left edge of the square
mark_abs( 20000, 20000 ); // Marking the top edge of the square
mark_abs( 20000, -20000 ); // Marking the right edge of the square
mark_abs( -20000, -20000 ); // Marking the bottom edge of the square
// The laser control signals are automatically switched on
// with the first [*]mark[*] Command after a LaserOn Delay and remain on for
// all 4 [*]mark[*] Commands.
// A Polygon Delay is automatically inserted after the first three [*]mark[*] Commands, each.
// Initiated by the following non-marking command (Jump command, see below), a Mark Delay
// is automatically inserted after the last [*]mark[*] Command and the laser control signals
// are automatically switched off after a LaserOff Delay, because a Jump command follows.

// Marking a circle around the center point:
// The laser control signals are automatically switched on with the arc command
// after a LaserOn Delay.
// Initiated by the following non-marking command (set_end_of_list, see below),
// a Mark Delay is automatically inserted after the arc command and the laser control signals
// are automatically switched off after a LaserOff Delay, because a set_end_of_list follows.

// Defining the end of the list and the end of command transfer to the RTC5 Board
set_end_of_list();

// Starting the transferred list
execute_list( 1 );
```


7.2 Delay Settings – Coordinating Scan Head Control and Laser Control

Scan head control and laser control should suit the dynamic behavior of the system components, that is, the

- response behavior of the laser
- response behavior of the galvanometer scanners (**Tracking error**)
- the type of interaction between laser radiation and material

The following delays are available for this purpose:

- **Laser Delays**
 - **LaserOn Delay**
 - **LaserOff Delay**
- Scanner delays
 - **Jump Delay** (optionally: variable)
 - **Mark Delay**
 - **Polygon Delay** (optionally: variable)

7.2.1 Laser Delays

There are two different **Laser Delays**:

- **LaserOn Delay**
- **LaserOff Delay**

The **Laser Delays** determine when the **Signals for “Laser Active” Operation** are switched on and off.

As a rule, **Laser Delays** have *no* influence on the total marking time.

Exceptions:

- A negative **LaserOnDelay** value, see **Section “LaserOn Delay”, Page 134**
- Artificially inserted delays, see **Section “Automatic Delay Adjustments”, Page 143**

The **LaserOn Delay** and the **LaserOff Delay** are set by the undelayed short list command **set_laser_delays**. Their unit is $0.5 \mu s$ each.

In order to avoid burn-in effects at start and end points of a marking, the laser focus should be moved at a speed which is as constant as possible. Therefore, the laser delay durations must be adjusted to the tracking delay of the scan head and the set mark speed⁽¹⁾, see also **Chapter 7.2.3 “Notes on Optimizing the Delays”, Page 143**.

(1) In addition, automatic readjustment of the laser power during marking can be applied for optimization, see **Chapter 7.4.9 “Automatic Laser Control”, Page 187**.

LaserOn Delay

The **LaserOn Delay** is automatically inserted at the start of a single **Mark command** and at the start of a series of **Mark command** ("Polyline") and delays the switching on of the laser.

It can be used for several purposes:

- At the beginning of a marking, the mirrors must be accelerated to the specified mark speed (if necessary, from a halt), see **Figure 41**. This phase can be suppressed with a sufficiently high positive **LaserOn Delay** value until the mirrors have already reached a certain angular speed before the laser is switched on. On the other hand, the **LaserOn Delay** must not be too long, otherwise the first part of the marking is cut off.
- Some materials/applications (for example, welding) take some time until they react as desired to the exposure to laser radiation. In this case, it can be useful to "preheat" the starting point of the marking. This can be achieved by setting a *negative* **LaserOn Delay** value. However, this extends the total marking time because the **LaserOn Delay** is inserted prior to the current **Mark command** as a scanner delay. This scanner delay is automatically extended, if a preceding **LaserOff Delay** has not yet been expired, see **Section "Automatic Delay Adjustments"**, Page 143.

LaserOff Delay

The end of a marking is not defined by the marking itself, but by the first "normal" list command that is not a **Mark command**, for example, a **Jump command**.

The acceleration phase at the beginning of a motion leads to a time difference between the respective set position and the actual position of the mirrors, see **Figure 41**.

The laser is to be switched off until the *actual value* of the end position is reached (but not already at the *set position*). Therefore, a **LaserOff Delay** is inserted automatically after the end of each marking by the first "normal" list command (no "short" list command), see also **Section "Notes"**, Page 137. This can be used to compensate for the **Tracking error** of the scan head.

The following applies with short marking vectors: if a preceding **LaserOn Delay** has not yet been expired, the **LaserOff Delay** is temporarily automatically extended accordingly, see **Section "Automatic Delay Adjustments"**, Page 143.

7.2.2 Scanner Delays

There are three different types of **Scanner Delays**:

- **Jump Delays** (optionally: variable)
- **Mark Delays**
- **Polygon Delays** (optionally: variable)

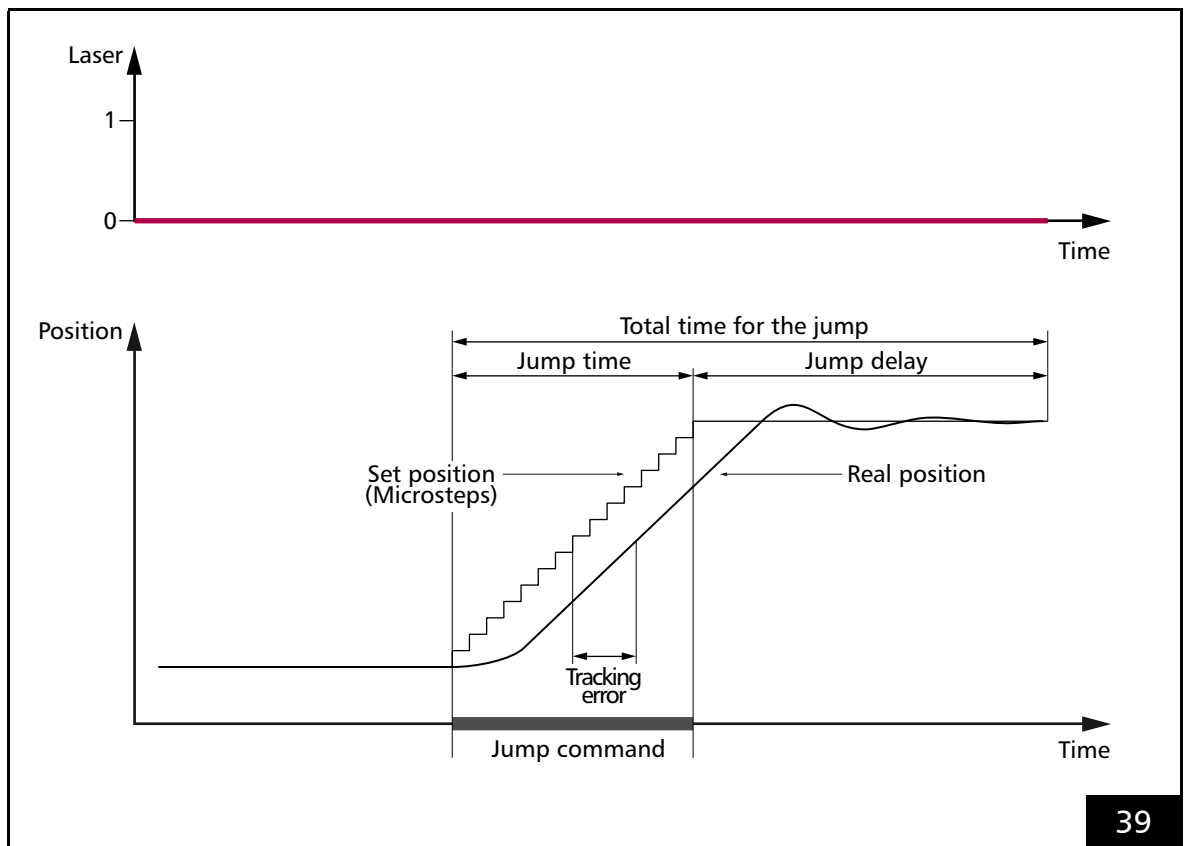
After each **Jump command** and **Mark command**, the RTC5 PCI Board inserts one of these **Scanner Delays** before the next command is executed (unless otherwise specified).

These **Scanner Delays** are defined by **set_scanner_delays**. Their unit is $10 \mu s$ each.

Jump Delay

The **Jump Delay** is specified by **set_scanner_delays** with the **Jump** parameter.

A typical course for a single **Jump command** and the belonging **Jump Delay** is shown in Figure 39.



Scan head control during a **Jump command** with a constant **Jump Delay**.
The laser remains off.

Variable Jump Delays

With short jumps, the scan head often does not reach the full jump speed. Then *shorter* **Jump Delays** are sufficient for settling of the mirrors:

- “Variable Jump Delays”

For this, there is:

- “Variable Jump Delays” mode

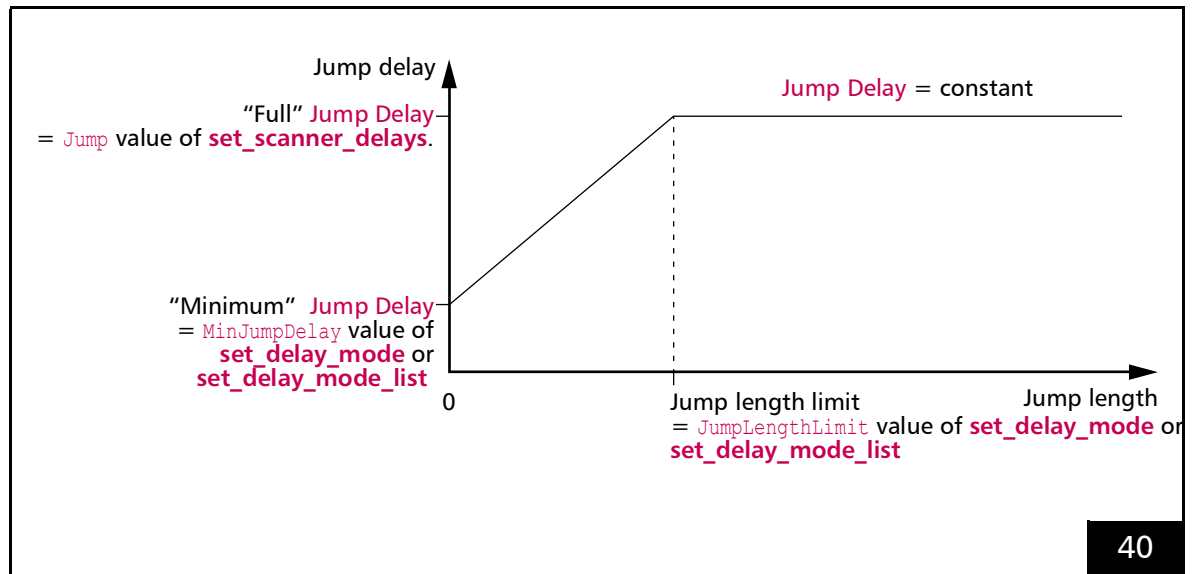
“Variable Jump Delays” mode is switched on by `JumpLengthLimit > 0` (at `set_delay_mode` and `set_delay_mode_list`).

Then the RTC5 PCI Board inserts a linearly interpolated **Jump Delay** between `MinJumpDelay` (from `set_delay_mode` or `set_delay_mode_list`) and `Jump` (`Jump Delay` from `set_scanner_delays`) for jump lengths between 0 and `Jump length limit`, see Figure 40.

With jump lengths larger than `JumpLengthLimit` a “full” **Jump Delay** is always inserted.

Notes

- With the “Variable Jump Delays” mode, total marking time is reduced, especially when there are many short jumps.
- The “Variable Jump Delays” mode is switched off by `JumpLengthLimit = 0` (at `set_delay_mode` and `set_delay_mode_list`).
- After jump vectors of length 0, the duration of “Variable Jump Delays” is 0.
- The “minimum” **Jump Delay** should not be larger than the “normal” **Jump Delay**. Otherwise, “Variable Jump Delays” can become very large.



“Variable Jump Delays” mode: **Jump Delay** value depending on the jump length.

Mark Delay

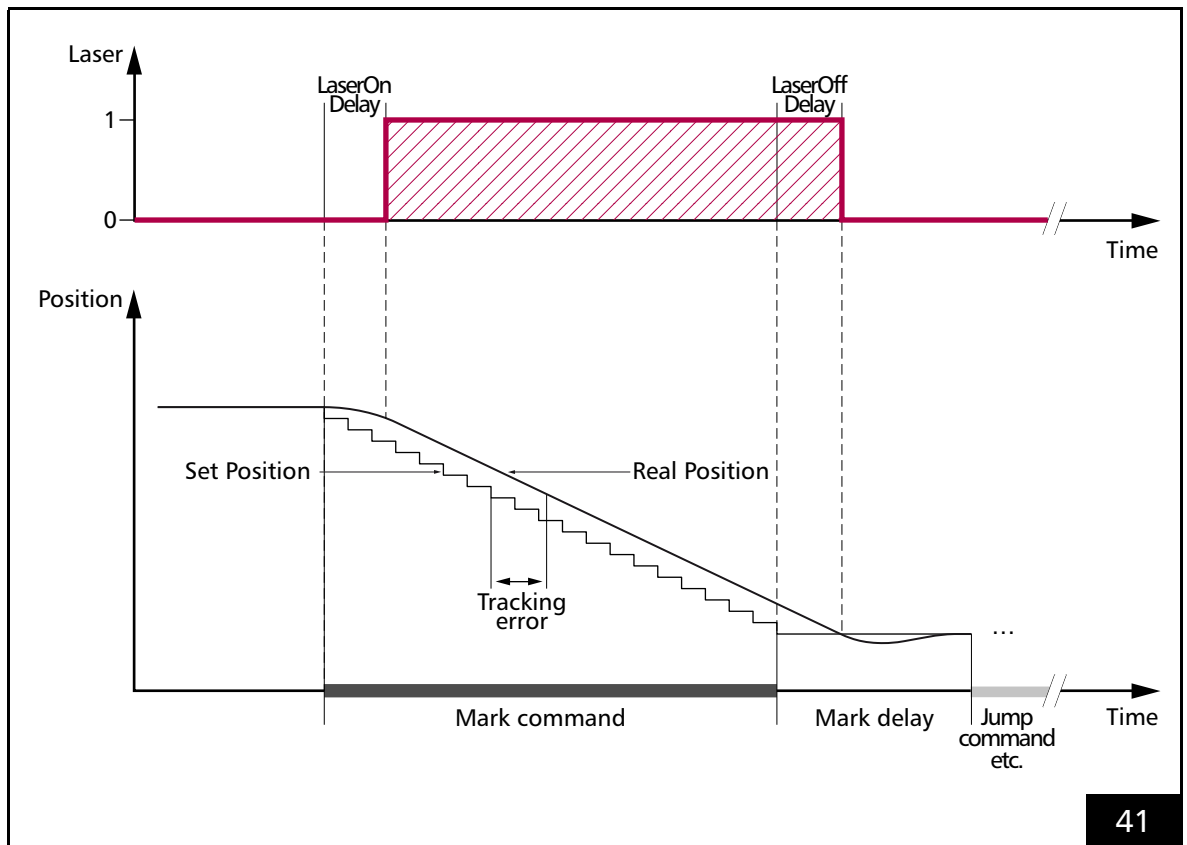
The **Mark Delay** is specified by **set_scanner_delays** with the **Mark** parameter.

A typical course for a single **Mark command** with the and the belonging **Laser Delays** is shown in **Figure 41**.

The duration of the **Mark Delay** should suit the dynamic properties of the scan head (**Tracking error**, tuning) and the mark speed set.

Notes

- If no further **Mark command** follows a **Mark command**, a **Mark Delay** is inserted automatically and the laser is switched off after a **LaserOff Delay**.
- If a further **Mark command** follows a **Mark command** (= "Polyline"), a (variable) **Polygon Delay** is inserted and the laser remains switched on, see **Section "Variable Polygon Delays"**, **Page 139**.
- Short control commands do not lead to insertion of a **Mark Delay** or **Polygon Delay** and not to a laser switch off.
- Note that a **Mark command** of length 0 corresponds to a **list_continue**, if it is not timed, see **Chapter 8.9 "Timed Commands"**, **Page 250**.



Scan head control and laser control timing during a **Mark command** or "**Arc**" command with a **Mark Delay**. The laser is on in the hachured period.

Polygon Delay

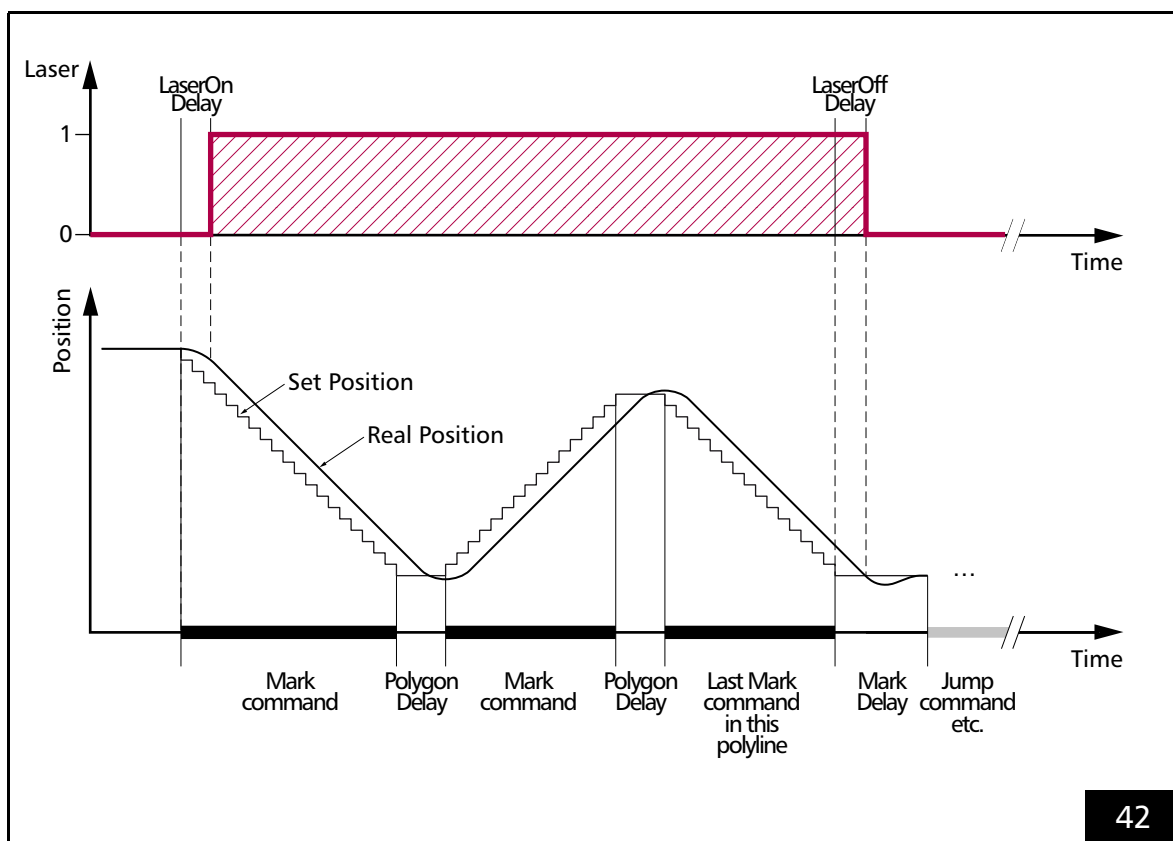
The **Polygon Delay** is specified by **set_scanner_delays** with the **Polygon** parameter.

A typical course for a series of **Mark commands** ("Polyline") where **Polygon Delays** (instead of **Mark Delays**) are inserted between the individual **Mark command** is shown in **Figure 42**.

Short list commands do not interrupt a **Polyline**.

If the (usually smaller) angles between the markings vary only slightly usually a **Polygon Delay** value can be specified that is smaller than the **Mark Delay** value.

If, on the other hand, the angles vary significantly, then a "Variable **Polygon Delay**" is recommended, see **Section "Variable Polygon Delays", Page 139**.



Scan head control and laser control timing during a **Polyline** with a constant **Polygon Delay**.

Variable Polygon Delays

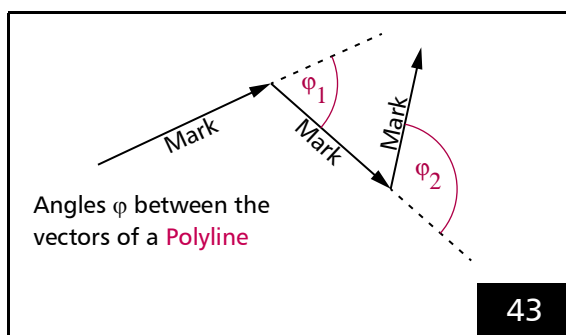
By setting the `VarPoly` value > 0 (at `set_delay_mode` or `set_delay_mode_list`) it is enabled:

- “Variable Polygon Delays” mode

Then, the RTC5 PCI Board calculates the “Variable Polygon Delay” $v_delay(\varphi)$ for every **Polyline** corner according to:

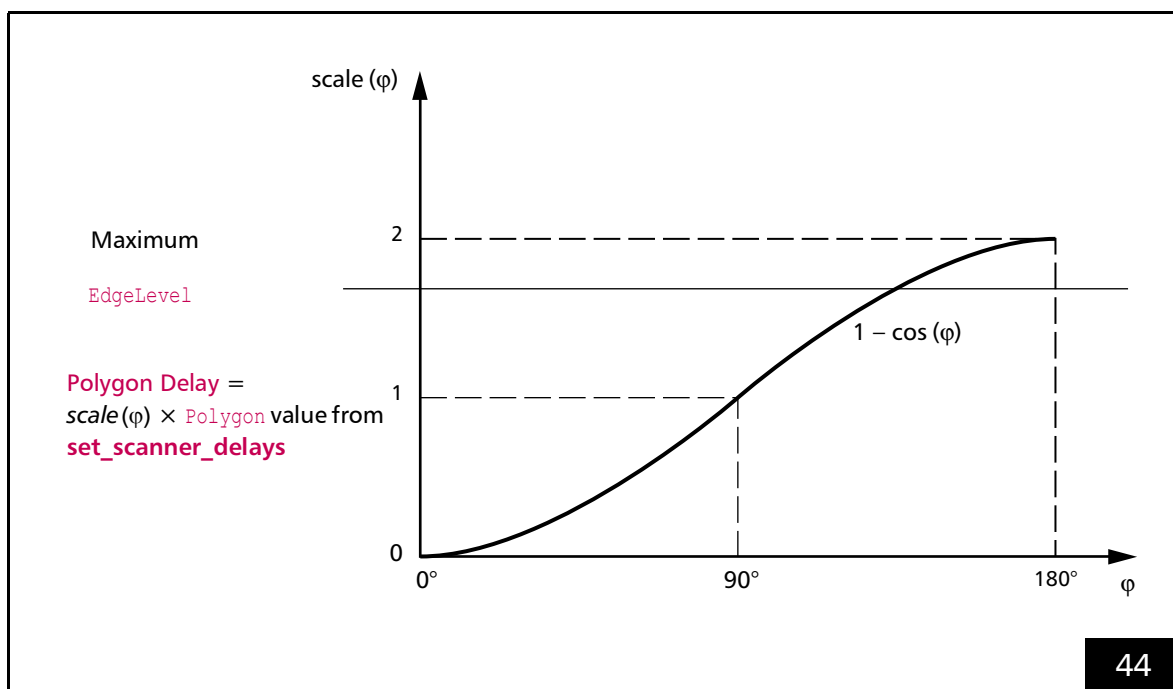
$$v_delay(\varphi) = scale(\varphi) \times polygon_delay$$

For the definition of the angle φ using the example of mark vectors, see **Figure 43**.



Variable Polygon Delays. Definition of the angle φ .

For circular arcs and ellipses, analogously the tangents at the connection point are relevant. $scale(\varphi)$ is a scaling function for the **Polygon** value from `set_scanner_delays` (constraint $0 \leq scale(\varphi) \leq 2$), see **Figure 44**. The default scaling function after `load_program_file` is $scale(\varphi) = 1 - \cos(\varphi)$. It can be replaced by a user-defined scaling function, see **Section “User-defined “Variable Polygon Delays””, Page 141**.



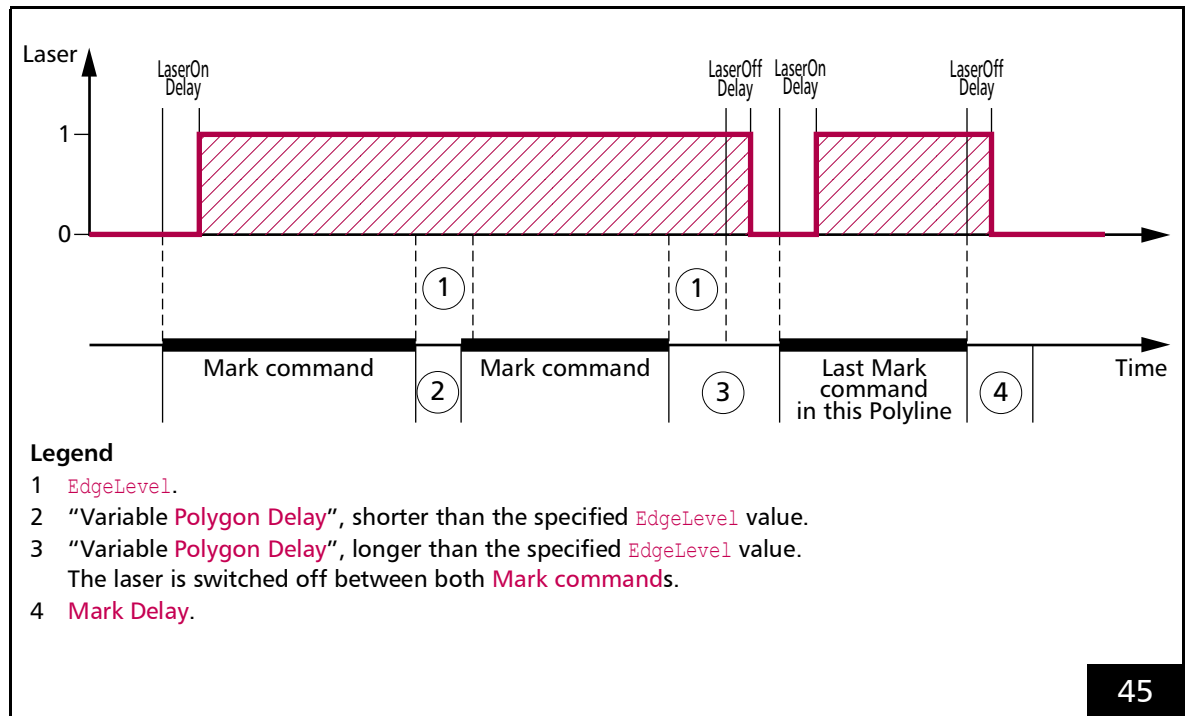
Variable Polygon Delays. Variation of the **Polygon Delay** (default scaling function after `load_program_file`).

EdgeLevel

The “Variable **Polygon Delay**” becomes quite long with angles φ close to 180° , see **Figure 44**. This might lead to burn-in effects in sharp corners of a **Polyline**.

If an **EdgeLevel** value > 0 is specified (at **set_delay_mode**), the RTC5 PCI Board switches off the laser with a **LaserOff Delay** after **EdgeLevel** delay clock cycles at the latest and thus terminates the current **Polyline**, see **Figure 45**.

The next **Mark command** starts as usual with the current “Variable **Polygon Delay**”.



Laser control timing during a **Polyline** with “Variable **Polygon Delay**”.

An **EdgeLevel** value has been defined with **set_delay_mode**.

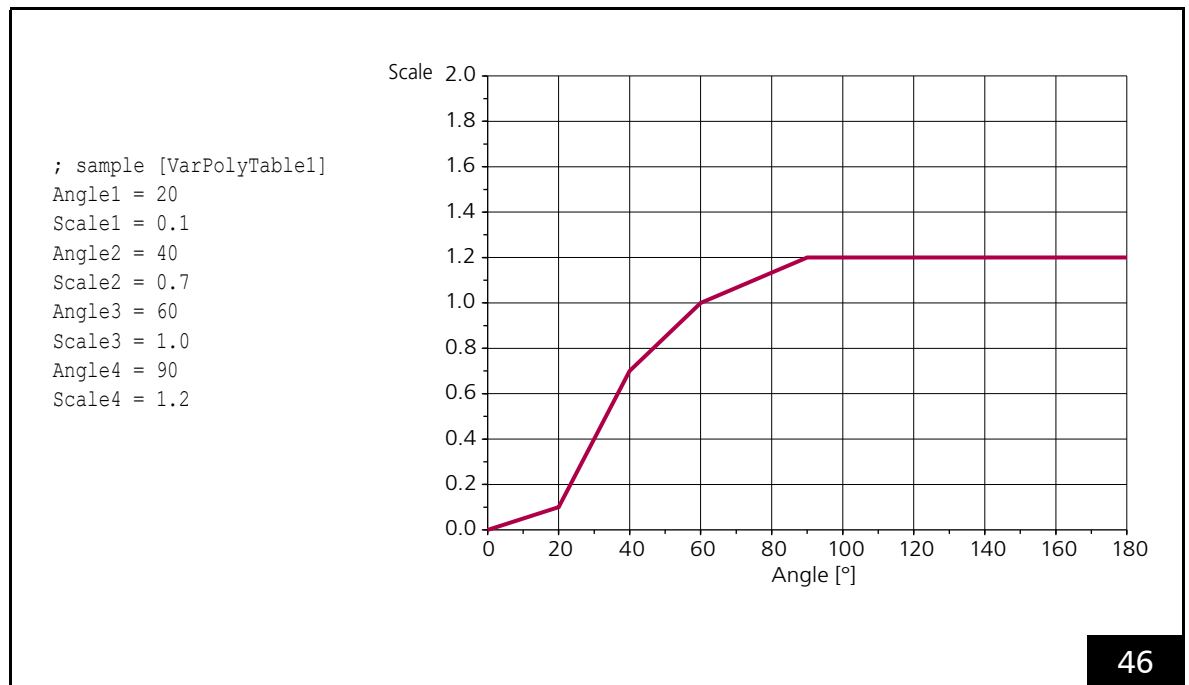
User-defined “Variable Polygon Delays”

- For the $scale(\varphi)$ scaling function, **load_varpolydelay** loads a table from an ASCII text file. See also Section “Variable Polygon Delays”, Page 139.
- The ASCII text file can contain one or more tables⁽¹⁾.
- Each table can contain up to 50 data points (φ | $scale(\varphi)$).
- The $scale(\varphi)$ function is linearly interpolated from the data points.
- As an example, Figure 46 shows a table with 4 data points and the corresponding scaling function.

Table 1: Possible table types in the ASCII text file. Each type can occur more than once.

[VarPolyTable<No>]	User-defined “Variable Polygon Delays”, see page 141
[PositionCtrlTable<No>]	Scaling function , see page 189
[AutoLaserCtrlTable<No>]	Nonlinearity curve, see page 192
[JumpTable<No>]	Jump Delay values, see page 205
[StretchTable<No>]	2D stretch correction table, see page 225
[Fly2DTable<No>]	2D compensation table, see page 235

(1) Even of another type, see table 1, page 141.



Example: Table for User-defined “Variable Polygon Delays” with 4 data points (left) and corresponding scaling function $scale(\varphi)$ (right).

For the tables, the following rules apply:

- Each table must begin with the line:
`[VarPolyTable<No>]`
`<No>` represents the table number.
- If the table contains multiple `[VarPolyTable<No>]` entries with the same `<No>`, then only the lines after the first entry are used. Only lines up to the next '[' character (that is not preceded by a semicolon) are used.
- Each data point (φ | $scale(\varphi)$) is defined as follows:
`Angle<n> = <Value>`
`Scale<n> = <Value>`
 where `<n>` must be replaced by a number ($1 \leq <n> \leq 50$) which denotes the number of the data point. The values `<Value>` for the angle φ (in degrees) and the scaling factor can be specified as (unsigned) floating point numbers. Decimal separator: period (.).
- If the table contains multiple data points with the same Index `<n>`, then the most recently read one is used and the previous ones are ignored.
- If the table contains multiple data points with the same angle φ , then the data point with the largest Index `<n>` is used and the others ignored. Equality is checked to within $\pm 0.01^\circ$.
- For `<Value>`, the following ranges apply:
 $0.0^\circ \leq \varphi \leq 180.0^\circ$ and $0.0 \leq scale(\varphi) \leq 2.0$.
- Each instruction must be in a separate line.
- Spaces and tabs in a line (for example, between '=' and `<Value>`) are ignored.
- Empty lines are ignored.
- Data points with invalid values are ignored.
- The data point of a particular index `<n>` is ignored if the corresponding `Angle<n>` and/or `Scale<n>` definition is missing.
- The semicolon ';' can be used for comments. All characters in a line following a semicolon are ignored.
- The instructions for data points in the table can be ordered as desired.
- Indices for data point pairs in the table can be selected as desired within the range [1...50] (the table is then automatically sorted by ascending angles).
- If the table contains *no* valid data point, then `load_varpolydelay` has no effect (return value 1 or 13).
- The angle $\varphi = 0^\circ$ means that two successive vectors are parallel and are marked in the same direction.
 If the table contains no explicit data for $\varphi = 0^\circ$ (equality is checked to within $\pm 0.01^\circ$), then a data point for $\varphi = 0^\circ$ with the scaling factor $scale(0^\circ) = 0$ is added.
- The angle $\varphi = 180^\circ$ means that two successive vectors are marked in opposite directions.
 If the table contains no explicit data for $\varphi = 180^\circ$ (equality is checked to within $\pm 0.01^\circ$), then a data point for $\varphi = 180^\circ$ with the largest scaling factor found in the table for $scale(180^\circ)$ is added.

After initialization by `load_program_file`, the RTC5 PCI Board uses the internal (default) table for the "Variable Polygon Delay" ($1 - \cos(\varphi)$, see Figure 44). Alternatively, this can also be achieved with `Name = NULL` in `load_varpolydelay`.

7.2.3 Notes on Optimizing the Delays

The delays are set by **set_scanner_delays** and **set_laser_delays**.

They should be appropriate for:

- the set jump speed
- the set mark speed
- the **Tracking error** of the used scan head
- the **ControlPreview** time of the used "Preprocessing System"

Non-optimized delays may lead to marking results of insufficient quality and excessive scan times, see also **Section "Potential Errors when Optimizing the Delays"**, Page 146.

Notes

- The **Laser Delays** are specified in units of $0.5 \mu\text{s}$.
- In contrast, the **Scanner Delays** (**Jump Delay**, **Mark Delay** and **Polygon Delay**) are specified in units of $10 \mu\text{s}$.

Recommended Optimization Sequence

(1) Laser Delays.

It is recommended to set **Jump Delays** and **Mark Delays** to a high value. The laser delay lengths have no influence on the total scan time as long as positive values are specified (see also following section).

(2) Scanner Delays.

Automatic Delay Adjustments

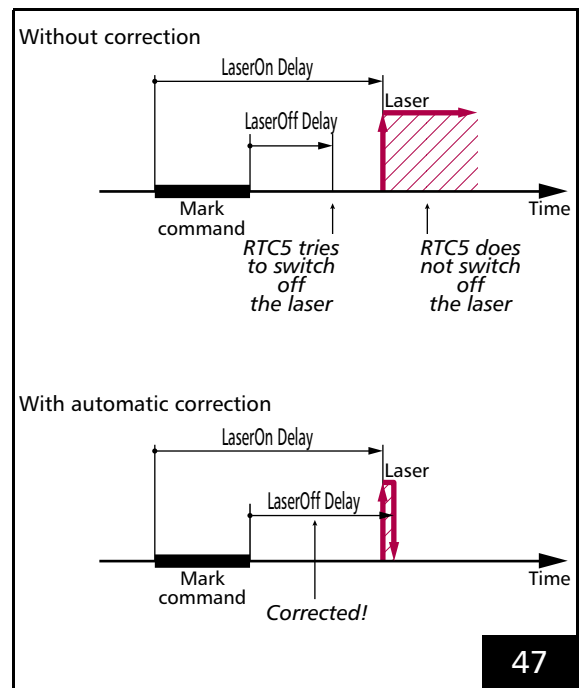
In principle, all delays can be set for any value within the corresponding allowed range. In some situations the laser control could be disturbed, for example, when Laser-On and Laser-Off overlap. Therefore, for each command that switches the laser, the RTC5 PCI Board checks the set **Laser Delays**, corrects them and, if necessary, inserts a scanner delay "artificially". These situations, which are listed below, should actually already be avoided in the user program, because the "corrected" marking does not necessarily meet the requirements.

Laser-On and Laser-Off Overlap

The RTC5 PCI Board corrects the faulty laser delay so that:

- the Laser-On occurs $0.5 \mu\text{s}$ after the currently effective **LaserOff Delay**
- or the Laser-Off occurs $0.5 \mu\text{s}$ after the still running **LaserOn Delay**

In both cases, the chronological next mark is cut off somewhat, see also **Figure 47**.



Automatic adjustment of **LaserOff Delay**.

Laser-On and Laser-On Overlap

If the **LaserOn Delay** is switched from a long delay to a short delay between two markings, the short **LaserOn Delay** could switch on the laser before the long **LaserOn Delay** of the previous marking has expired, if the laser has been switched off between the markers (otherwise the laser would stay on and the new **LaserOn Delay** would not be effective at this point).

To avoid this overlap, the RTC5 PCI Board in this case inserts a scanner delay "artificially". This increases the processing time and changes the dynamics of the galvanometer scanner motion.

Laser-Off and Laser-Off Overlap

If the **LaserOff Delay** is switched from a long delay to a short delay, the short **LaserOff Delay** could switch off the laser before the long **LaserOff Delay** of the previous marking has expired, if a marking had switched on the laser in the meantime (otherwise the laser would stay off and the new **LaserOff Delay** would not be effective at this point)

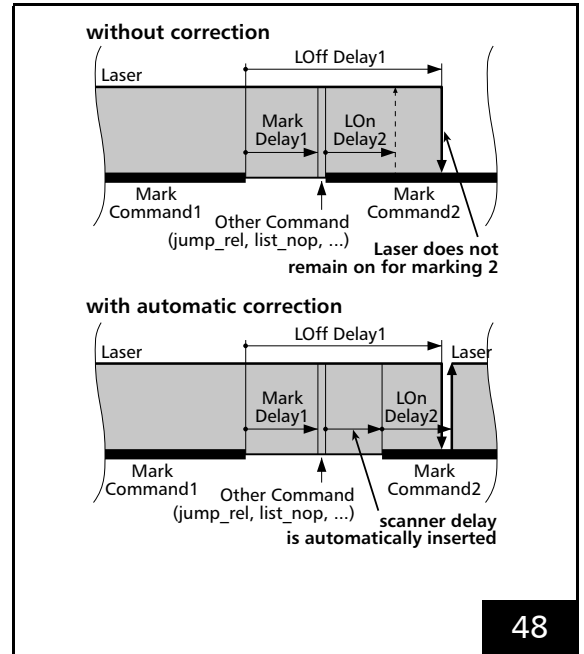
To avoid this overlap, the RTC5 PCI Board in this case extends the short **LaserOff Delay** so long that it only expires after the previous one (and of course only after the intermediate **LaserOn Delay**).

Several Simultaneously Expiring Laser Delays

This can happen if a laser delay is only effective beyond the length of the next command. The RTC5 PCI Board can only process 1 **LaserOn Delay** and 1 **LaserOff Delay** at the same time. The RTC5 PCI Board ensures this by adjusting the **Scanner Delays**.

Adjustment of the Scanner Delays

Under some circumstances with the RTC4, a short delay could result in laser control errors if a non-mark command (for example, `jump_rel( 0, 0 )` or `list_nop`) comes between two **Mark Commands**. Here, the laser would be switched off during the second mark command, but unexpectedly not switched on again if the **LaserOff Delay** is longer than the sum of the **Mark Delay** (and any **Jump Delay**) and the **LaserOn Delay** (see Figure 48).

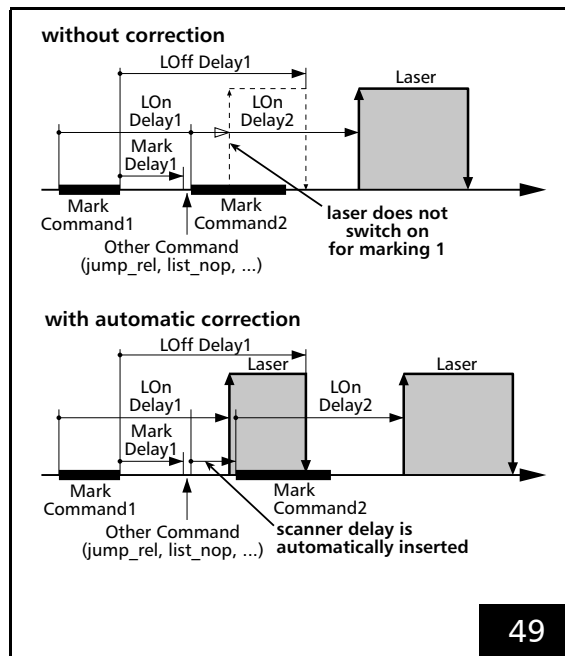


Automatic adjustment of the **Scanner Delays** for small **Mark Delays**.

One way of avoiding this problem is to generally select a **Mark Delay** that is larger than the difference between the **LaserOff Delay** and the **LaserOn Delay** ($\text{Mark Delay} > \text{LaserOff Delay} - \text{LaserOn Delay}$, as with the predecessor boards). But here, a minimum **Mark Delay** size is required that otherwise would not be necessary where **Mark Commands** directly follow another; this thus unnecessarily increases the total marking time. On the other hand, the RTC5 now checks each mark command's **Laser Delays** and automatically extends the scanner delay so that a preceding **LaserOff Delay** first finishes before another **Laser-On** follows. Thus, marking time is only increased when necessary. The **Scanner Delays** can now be optimized independently of the **Laser Delays**.

The same applies for the `edgelevel` of the "Variable **Polygon Delay**" as well as for a too-short **Polygon Delay** if a polygon chain is continued by a list change but no `auto_change` or `start_loop` is used and the second list is started immediately after the first list finishes processing. For predecessor boards in the latter case, the laser might remain switched-off for the rest of the **Polyline**.

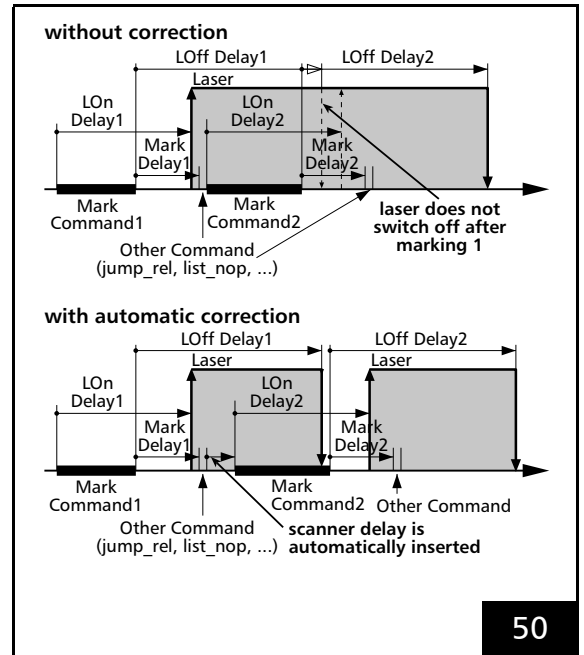
For very large **LaserOn Delays** extending across a mark command as well as a subsequent non-mark command and up to the next mark command, appropriate **Scanner Delays** are inserted to prevent overlap of a not-yet-expired **LaserOn Delay** with the start of the second marking command. Otherwise, the currently executing **LaserOn Delay** would have gotten overwritten and the laser would not have even switched on for the first marking command, but instead only for the second marking command and only after expiration of the **LaserOff Delay** (see [Figure 49](#)).



Automatic adjustment of the **Scanner Delays** for large **LaserOn Delays**.

Similarly, overlaps would occur for very large **LaserOff Delays** set for the beginning of a non-mark command and extending across the next mark command up to yet another non-mark command. Here, the laser would not switch off between the two mark commands and instead only switch off after the second non-mark command and only after expiration of the **LaserOn Delay** (see [Figure 50](#)). To prevent this situation, a check is performed at the start of a mark command to determine if the laser would have already switched off prior to the end of the command. If not, then the mark command gets postponed by an appropriate scanner delay.

On the other hand, laser switch-off during a **Polyline** with sharp corners (edgelevel parameter) only occurs if the laser has been already on during this **Polyline** (that is, when all previously initiated **LaserOn Delays** have already expired and no **LaserOff Delays** remains active).



Automatic adjustment of the **Scanner Delays** for large **LaserOff Delays**

Notes

- Thanks to these previously described automatic adjustments, the RTC5 always fulfills the scanner delay conditions
 $(\text{Mark Delay} > \text{LaserOff Delay} - \text{LaserOn Delay}, \text{EdgeLevel} > \text{LaserOff Delay} - \text{LaserOn Delay}, \text{Polygon Delay} + \text{LaserOn Delay} > \text{LaserOff Delay})$
at run-time that had to be explicitly observed with the RTC4.
- Automatic adjustment of the **Scanner Delays** might result in a total marking time longer than that expected from just adding-up the predefined vector lengths and delays.

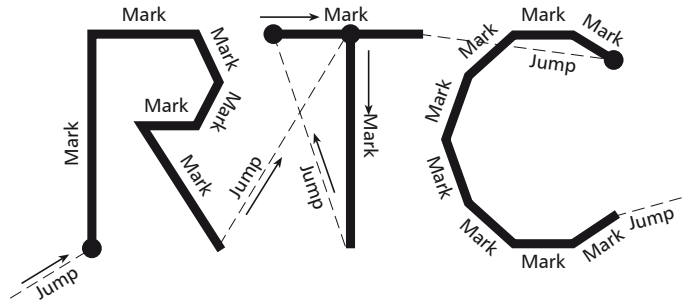
Potential Errors when Optimizing the Delays

The following figures show the various effects of unsuitable delays on the marking result, in this example on the lettering "RTC".

LaserOn Delay too short

At the beginning of a mark vector the laser is switched on, even though the mirrors have not yet reached the necessary angular velocity.

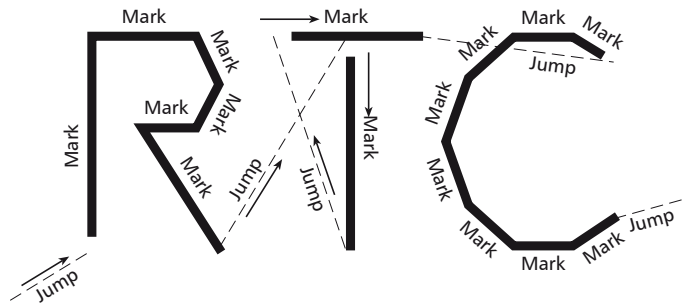
Burn-in effects at the start points of the respective vectors result.



LaserOn Delay too long

The laser is switched on too late at the beginning of a mark vector.

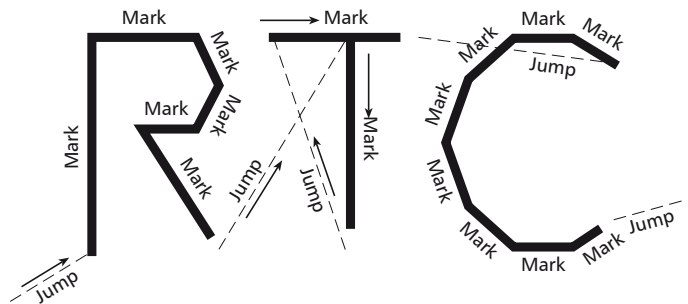
The first part of the vector is not marked.



LaserOff Delay too short

The laser is switched off after a mark command before the mirrors have reached the vector end position.

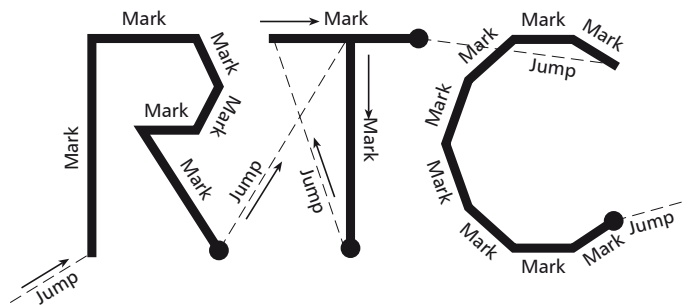
The respective vectors are not marked completely.



LaserOff Delay too long

The laser is switched off too late after a Mark command. The laser is still on, although the mirrors are already slowed down considerably or have already stopped.

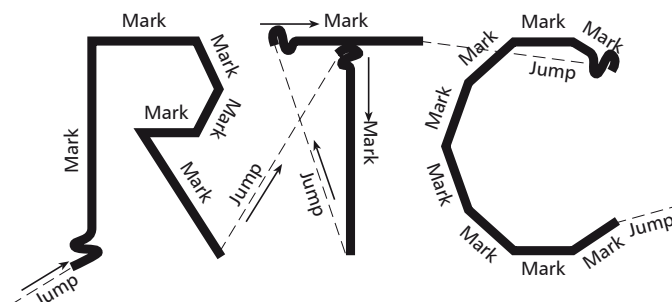
The results are burn-in effects at the end points of the respective vectors.



Jump Delay too short

The mark vector that follows a **Jump command** has already started although the scanners have not yet settled.

A running-in oscillation or overshoot is visible.



Jump Delay too long

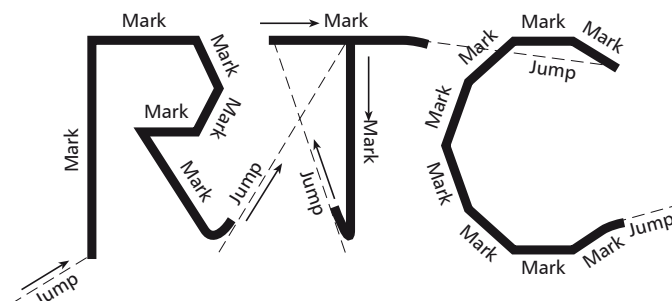
There are no visible effects if the **Jump Delay** is too long. However, the scanning time is extended.



Mark Delay too short

The traversing of a vector is started before the mirrors have reached the end position of the preceding mark vector.

The end of the mark vector is turned towards the direction of the jump vector.



Mark Delay too long

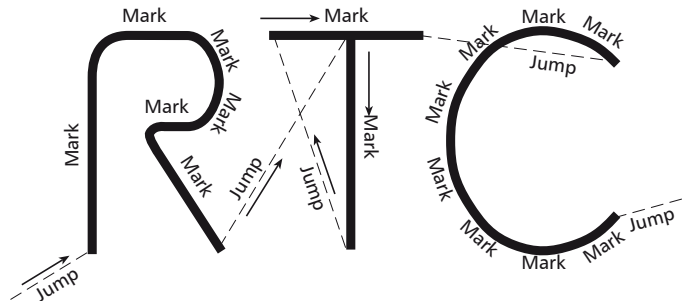
There are no visible effects if the **Mark Delay** is too long, but the scanning time is increased.



Polygon Delay too short

The subsequent mark command in a **Polyline** is already executing, although the mirrors have not yet reached the end position of the preceding mark vector.

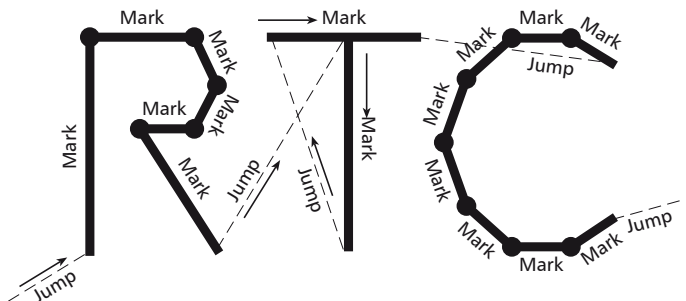
The corners of the **Polyline** are rounded off.



Polygon Delay too long

If the **Polygon Delay** is too long, the mirrors are moving too slowly or are even stopping between subsequent mark commands.

Since the laser is not turned off between these vectors, burn-in effects occur.



In

"Variable Polygon Delays" mode

, a maximum length

("EdgeLevel") can be defined,

see **Section "EdgeLevel"**,

Page 140 for details.

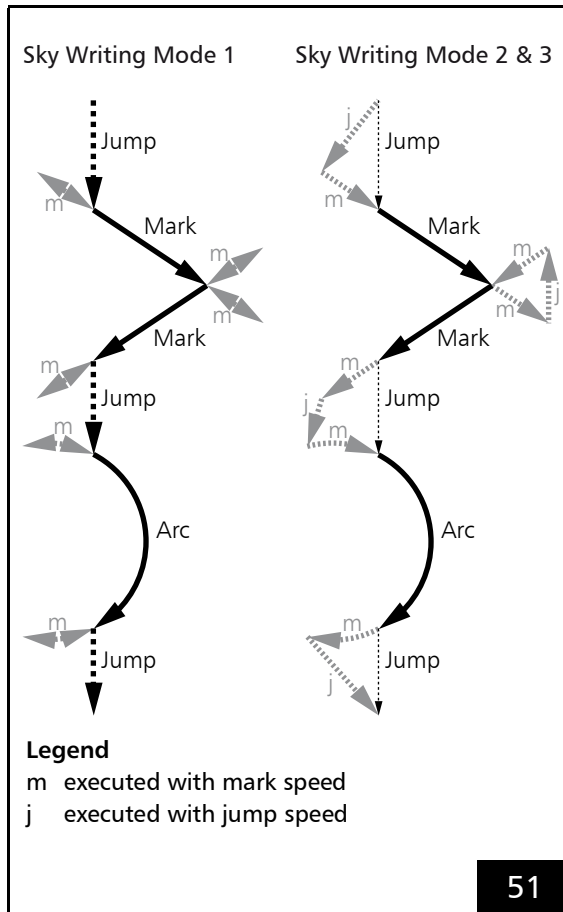
7.2.4 Sky Writing

For applications with elevated accuracy requirements the so-called **Sky Writing** can be switched on. Then, every mark vector is precisely executed at a constant mark speed over the entire vector length.

For **Mark commands** with **Sky Writing** enabled, the RTC5 PCI Board automatically performs non-marking **Sky Writing** scan motions before and after the to-be-executed vectors and arcs, see **Figure 51**.

The following are always executed *without* **Sky Writing**:

- **micro_vector[*] Commands**
- **[*]para[*] Commands**



Sky Writing motions (grey).
For **Sky Writing Mode 3** see also **Figure 52**.

By the **Sky Writing** command parameters you can specify:

- the duration of these run-in motions
- the duration of these run-out motions
- the synchronization between scan motion and laser control

Sky Writing is available in the following modes:

- **Sky Writing Mode 1**, Page 149
- **Sky Writing Mode 2**, Page 150
- **Sky Writing Mode 3**, Page 151

Sky Writing Mode 1

In **Sky Writing Mode 1**, the RTC5 PCI Board performs the following **Sky Writing** motions for each **Mark command** – regardless of previous or subsequent commands:

- In the run-in phase, the vector/arc is preceded by a “forerun” motion performed by the galvanometer scanners at mark speed, see **Figure 51**: the galvanometer scanners are driven a short distance parallel to the vector (or along the arc extension), initially from the startpoint in the opposing direction, then back to the startpoint.
- After the vector/arc has been processed at mark speed, it gets a short deceleration and retrace motion of the galvanometer scanners (at mark speed) appended in the run-out phase.

Sky Writing Mode 1 can be switched on/off by:

- **set_sky_writing**
- **set_sky_writing_list**
- **set_sky_writing_para**
- **set_sky_writing_para_list**

Sky Writing Mode 2

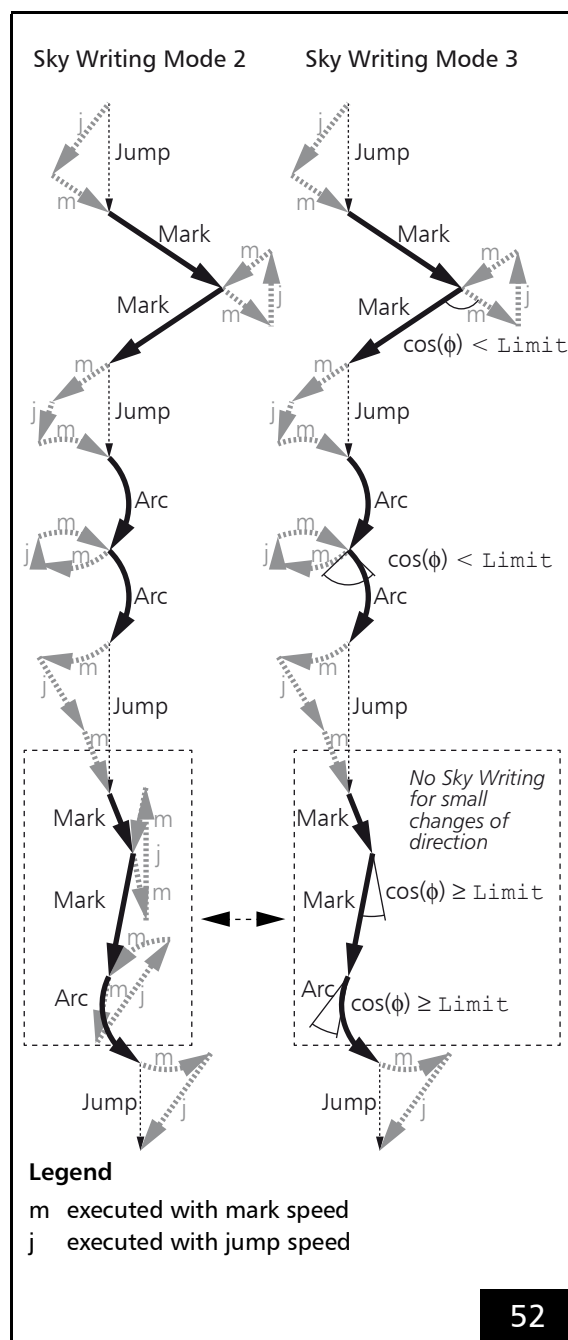
In **Sky Writing Mode 2**, the RTC5 PCI Board calculates **time-shortened Sky Writing** motions.

Here, too, each to-be-executed vector/arc gets preceded and appended with a run-in motion and a run-out motion in extension of the vector/arc at mark speed. Within a **Sky Writing Mode 2** marking sequence, however, neither forerun motions (in the run-in phase) nor retrace motions (in the run-out phase) of the galvanometer scanners occur. Instead, the RTC5 PCI Board executes **Sky Writing** jumps (at the currently specified jump speed) from jump vector startpoints to run-in startpoints, from run-out endpoints to run-in startpoints, and from run-out endpoints to jump vector endpoints, see **Figure 51**.

set_sky_writing and **set_sky_writing_para** always switch on **Sky Writing Mode 1**. Therefore, it is only possible to *change the mode afterwards* to **Sky Writing Mode 2** by **set_sky_writing_mode** or **set_sky_writing_mode_list**.

Sky Writing Mode 2 activation affects the same commands as **Sky Writing Mode 1** (except ellipse commands, see below). **Time-shortened Sky Writing**, however, can only occur within a sequence of non-parameterized **[*]mark[*]** Commands, arc commands and **Jump** commands (may also contain timed or 3D commands). If such a sequence gets interrupted by some other "non-**Sky Writing Mode 2**-capable" list command (for example, a **[*]para[*]** Command, ellipse command or any short list command), then the RTC5 PCI Board suspends **Sky Writing Mode 2** and complete the preceding command in **Sky Writing Mode 1** (with a retrace motion of the galvanometer scanners). This suspension of **Sky Writing Mode 2** does not deactivate it: subsequent "**Sky Writing Mode 2**-capable" commands are started at in **Sky Writing Mode 1** (with a forerun motion of the scan head) and then executed again in **Sky Writing Mode 2**.

Even with **Sky Writing Mode 2** switched on, ellipse commands are always executed in **Sky Writing Mode 1**.



Sky Writing motions (grey) for **Sky Writing Mode 2** and **Sky Writing Mode 3**.

Sky Writing Mode 3

The time cost of **Sky Writing** motions for vectors and arcs having only small directional changes within a **Polyline** is probably disproportionately high for the gained accuracy.

Therefore, **set_sky_writing_mode** or **set_sky_writing_mode_list** can be used to switch to **Sky Writing Mode 3**. A switching limit angle can be defined for this with **set_sky_writing_limit** or **set_sky_writing_limit_list**.

Then only for larger angle deviations ($\cos(\varphi) < \text{Limit}$) between successive **Mark commands** of a **Polyline** a **Sky Writing** motions is executed as in **Sky Writing Mode 2**.

In contrast, smaller angular changes ($\cos(\varphi) \geq \text{Limit}$) result in a (variable) polygonal delay, see **Section "Variable Polygon Delays"**, Page 139. See also **Figure 52**.

Notes

- In case a **Polyline** ends with a short mark vector (shorter than mark speed $\times$ Timelag) - and a **Polygon Delay** has been executed but not a **Sky Writing** motion - then the length of the short vector (caused by the **Tracking error**) may possibly not achieve the precision expected with **Sky Writing**.
- At the beginning and end of a **Polyline**, as well as when interrupted with "non-**Sky Writing Mode 2-capable**" list commands, a **Sky Writing Mode 1** motion always occurs in **Sky Writing Mode 3**, see **Section "Sky Writing Mode 2"**, Page 150.

Synchronization

The timing diagram for scan-head and laser control in **Sky Writing Mode 1** is shown in **Figure 53**. The timing diagram for **Sky Writing Mode 2** and **Sky Writing Mode 3** is similar, but here the galvanometer scanners perform no reverse motion in the run-in and run-out phases.

The **Timelag**, **Nprev**, **Npost** and **LaserOnShift** parameters are specifiable for **set_sky_writing_para** and **set_sky_writing** and the corresponding list commands:

- The **Nprev** parameter is a whole number in units of $10 \mu\text{s}$ and defines the duration of the run-in:

Mode	Duration
1	$20 \times \text{Nprev} [\mu\text{s}]$
2, 3	$10 \times \text{Nprev} [\mu\text{s}]$

- The **Npost** parameter is a whole number in units of $10 \mu\text{s}$ and defines the duration of the run-out:

Mode	Duration
1	$20 \times \text{Npost} [\mu\text{s}]$
2, 3	$10 \times \text{Npost} [\mu\text{s}]$

- The parameters **Timelag** (64-bit IEEE floating point value rounded to integer multiple of $1 \mu\text{s}$) and **LaserOnShift** (whole number in units of $0,5 \mu\text{s}$) define the delay of the **Signals for "Laser Active" Operation** switch-on and switch-off point in times relative to the set starting position and set ending position:
 - Delay of switch-on point in time relative to the set starting position / μs
 $= \text{Timelag} + 0,5 \mu\text{s} \times \text{LaserOnShift}$
 - Delay of switch-off point in time relative to the set ending position / μs
 $= \text{Timelag}$

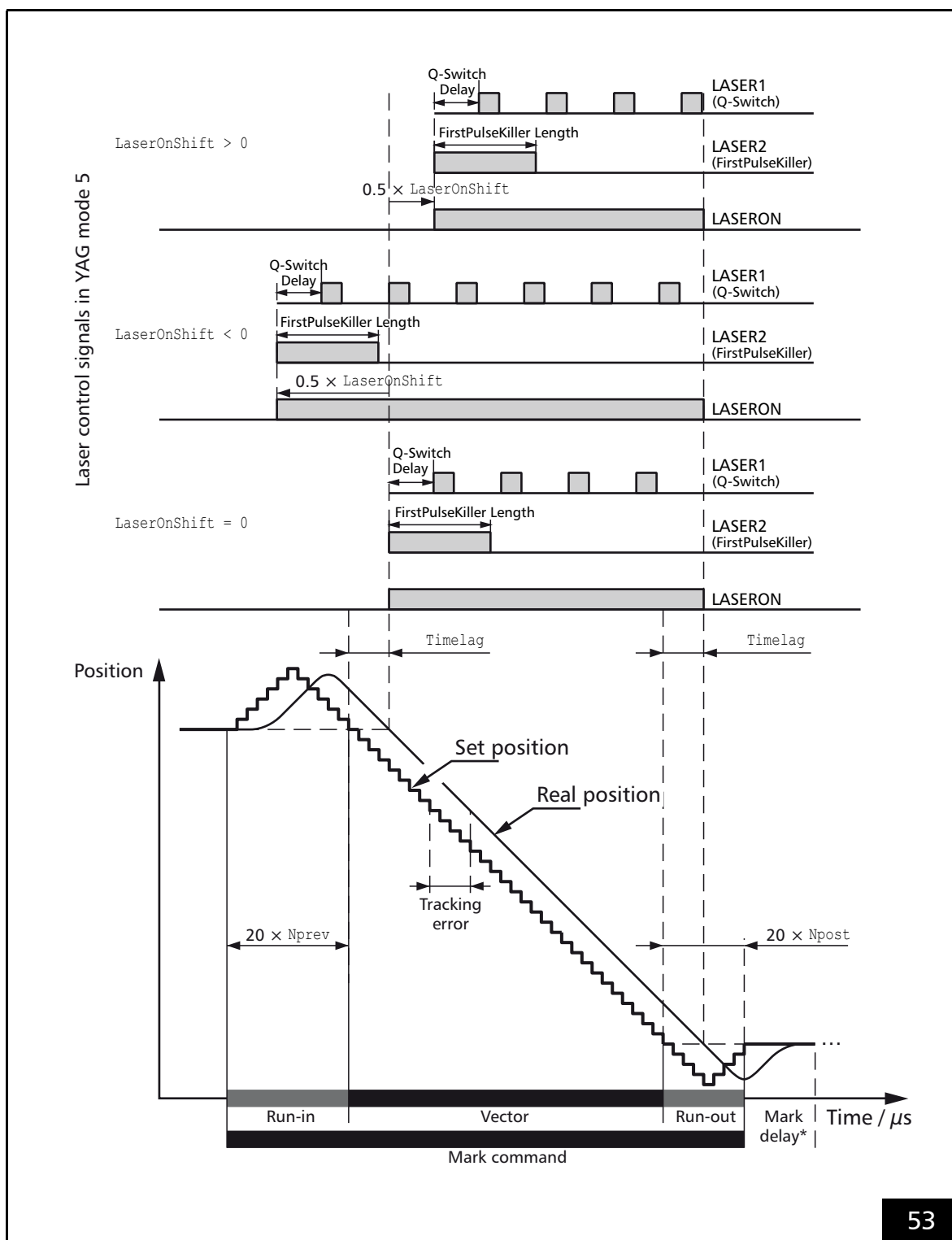
For `LaserOnShift = 0`, the **Signals for “Laser Active” Operation** are switched on (off) with a delay of `Timelag` relative to the set starting position (set ending position).

By setting the parameter `Timelag` to the actual **Tracking error** (and the parameters `Nprev` and `Npost` to sufficiently large values), it is ensured that:

- the **Signals for “Laser Active” Operation** switch on/off precisely at the endpoints of the desired vector
- the **Tracking error** becomes constant even before the vector’s startpoint is reached
- the vector is executed right up to its endpoint with constant mark speed

The **Laser Delays** from `set_laser_delays` are not effective with **Sky Writing**.

The switch-on point in time of the **Signals for “Laser Active” Operation** can be adjusted with the parameter `LaserOnShift` (for example, to the `HalfPeriod` of `set_laser_pulses` or to the reaction times of the laser itself), so that the first laser pulse occurs exactly with the set starting position. Positive `LaserOnShift` values delay the switching on of the laser control signals. Negative `LaserOnShift` values advance the switching on of the laser control signals (at the earliest, however, until the beginning of the forward run). Negative values are also necessary to “make room” for a Q-Switch delay or a **FirstPulseKiller** signal in one of the YAG modes.



Scan-head control and laser control in **Sky Writing Mode 1** (with parameters **Timelag**, **Nprev**, **Npost** and **LaserOnShift**) (the laser control signals LASER1 and LASER2 in YAG Mode 5 are exemplarily shown)
 (* This **Mark Delay** is only effective with **Sky Writing Mode 3**).

Notes

- By **set_sky_writing** and **set_sky_writing_mode_list**, you can switch back and forth between **Sky Writing Mode 1** and **Sky Writing Mode 2** or **Sky Writing Mode 3**. Temporarily switching off is also possible, as long as `Timelag > 0`.

- When searching for appropriate **Sky Writing** parameters, initially **set_sky_writing** and **Sky Writing Mode 1** should be used: as start value for the `Timelag` parameter, the tracking delay of the galvanometer scanners. should be set. Then `Timelag` (with `LaserOnShift = 0`) can be fine-tuned such that marking vector *ends* are exactly marked and only afterwards the parameter `LaserOnShift` should be adjusted (keeping constant the parameter `Timelag`), so that the marking vectors' *beginnings* are exactly marked, too.

In a second step **set_sky_writing_para** can be used to optimize the parameters `Nprev` and `Npost`: `Nprev` and `Npost` may be decreased (relating to the default settings of **set_sky_writing**, see below) to minimize marking times. Alternatively, `Nprev` may be increased to enable a correspondingly large negative `LaserOnShift`.

- With **set_sky_writing** and **set_sky_writing_list**, `Nprev` and `Npost` are predefined:
 - `Nprev` = approx. $0.15 \times \text{Timelag}$
 - `Npost` = approx. $0.1 \times \text{Timelag}$
 This results in the following run-in and run-out durations:

	Mode	Duration [<code>Timelag</code>]
Run-in	1	3
Run-in	2, 3	1.5
Run-out	1	2
Run-out	2, 3	1

- With **set_sky_writing_para** or **set_sky_writing_para_list**, you can also specify other run-in and run-out phases, for example, shorter ones to reduce marking times (but this may be at the expense of accuracy). Shorter run-in and run-out phases are particularly beneficial when lines are marked in only one direction without interruption and jump speed and mark speed are set equally. Note that in **Sky Writing** mode no automatic adjustment of laser and **Scanner Delays** is performed:
 - If the next **Mark command**'s run-in phase begins when the preceding **Mark command**'s **LaserOn Delay** has not yet expired, then the laser does not switch on for the preceding command.
 - If the next **Mark command**'s run-out phase begins when the preceding **Mark command**'s **LaserOff Delay** has not yet expired, then the laser does not switch off for the preceding command.
 - In **Sky Writing Mode 1**, a positive `LaserOnShift` time (in μs) should exceed the smallest occurring marking time by no more than $(20 \times \text{Npost} - \text{Timelag})$ – and in **Sky Writing Mode 2** and **Sky Writing Mode 3** by no more than $(10 \times \text{Npost} - \text{Timelag})$. Otherwise, the laser is not switched on for this marking.

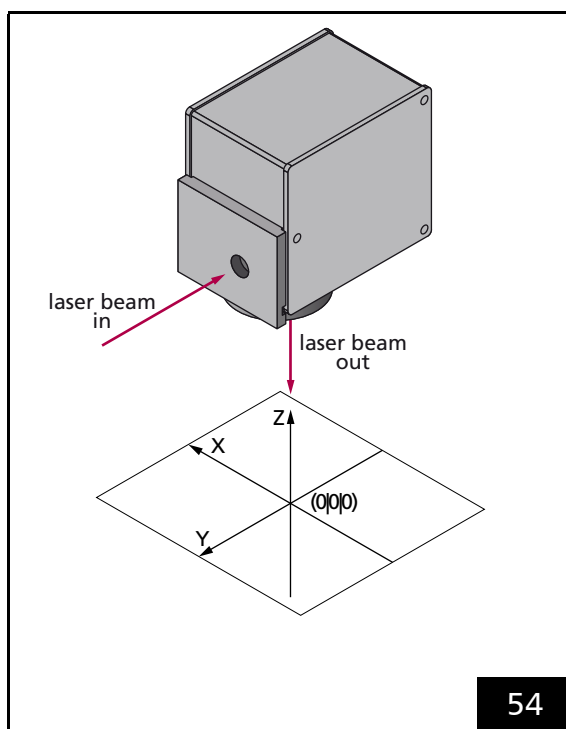
- Pulse lengths and frequencies (in YAG modes additionally the Q-Switch delay and the **FirstPulseKiller signal** parameters) previously defined for the **Signals for "Laser Active" Operation** are kept unchanged in the **Sky Writing** mode. On the other hand, previously defined **LaserOn Delays**, **LaserOff Delays**, **Mark Delays**, **Polygon Delays** or **"Variable Polygon Delays"** are not taken into account in the **Sky Writing** mode. They are fully functional again after deactivation of **Sky Writing** mode. Deactivation of **Sky Writing** mode results in addition of a **Mark Delay** defined prior to activation (see **Figure 53**).
- In **Sky Writing Mode 1**, the run-in and run-out phases take place fully within the respective **Mark command**. As a result, execution of the **Mark command** is extended by a time period (in $10\ \mu\text{s}$) of $(2 \times N_{\text{prev}} + 2 \times N_{\text{post}})$. In **Sky Writing Mode 2** and **Sky Writing Mode 3** execution is extended by a time period (in $10\ \mu\text{s}$) of at least $(N_{\text{prev}} + N_{\text{post}})$.
- In **Sky Writing Mode 1**, **Jump Delays** are performed normally. The **Jump Delay** value (in $10\ \mu\text{s}$, see **set_scanner_delays**) can be reduced by approx. $(2 \times N_{\text{prev}})$ or even to 0. In **Sky Writing Mode 2** and **Sky Writing Mode 3**, the **Jump Delay** is automatically (internally, dynamically) reduced by up to N_{prev} .
- Time-based **[*]mark[*] Commands** and **"Arc" commands** can also be performed in **Sky Writing** mode (see **Chapter 8.9 "Timed Commands"**, **Page 250**). Here, however, a shorter marking period is coupled with a higher mark speed and therefore with longer distances and acceleration phase in the run-in and run-out phases. Herefore, N_{prev} and N_{post} can be separately adjusted with respect to the specified mark speed.
- **Sky Writing** mode is not taken into account (but also not deactivated)
 - During execution of **[*]para[*] Commands**, for example, with activated "vector-controlled laser control", see **Section "Vector-Defined Laser Control"**, **Page 194**
 - During execution of **micro_vector[*] commands**, see **Chapter 8.8 "micro_vector[*] Commands"**, **Page 249**
- With **Sky Writing Mode 2** or **Sky Writing Mode 3** activated, the following functionalities of the 2D **Jump commands** **jump_abs** and **jump_rel** are not executed:
 - Automatic tuning switching in **Jump Mode**, see **Chapter 8.1.5 "Jump Mode"**, **Page 202**
 - Coordinate transformations with $\text{at_once} = 2$, see list item "With $\text{at_once} = 2 \dots$ ", **page 211**.

7.3 Scan Head Control

7.3.1 Reference System

The reference system for the **Image field** which is used by the RTC5 PCI Board is shown in **Figure 54**.

The y axis points in the *reverse* direction of the input laser beam, the z axis points in the *reverse* direction of the output laser beam. x axis, y axis and z axis form a right-handed reference system. The origin of the reference system, that is, the point (0|0|0), is in the center of the **Image field**.



Reference system for the RTC5 coordinates.

7.3.2 Image Field Size and Image Field Calibration

The size of the usable **Image field** is determined by the maximum scan angle and the focal length of the objective or the working distance (that is, the distance between the input laser beam axis and the **Image field**).

The x and y coordinates of a vector must be specified as signed 20-bit values (that is, as numbers between -524,288 and +524,287) whereas z coordinates for a 3D scan system must be specified as signed 16-bit values (that is, as numbers between -32,768 and +32,767).

The calibration factor K defines the ratio of the digital point coordinates in *bits*⁽¹⁾ and the actual position of the point in *millimeters*.

Let a_0 denote the side length of the **Image field** given by the maximum scan angle. The theoretical calibration factor is then $K_0 = 2^{20}/a_0$ [bits per mm⁽¹⁾].

SCANLAB provides a rounded value for the calibration factor K . This value is slightly larger than, but close to, the theoretical value. The actual calibration factor K can be read out from a used correction table by **get_table_para** or **get_head_para**.

Given the calibration factor K , the side length a of the usable **Image field** in *millimeters* can be calculated:

$$a = \frac{2^{20}}{K}$$

Notes

- The calibration factor is the same for the X and y direction.
- With 3D scan systems, the following applies:
 $K_z = K_{xy} / 16$
("z calibration factor")

(1) The expression "bits" is here synonymous with "digital control value" (see footnote ⁽³⁾ on **page 124**).

Typical Image Field

In general, the size of the usable (or “typical”) **Image field** – dependent on the objective and the optical configuration of the scan system – is smaller than the maximum adjustable **Image field**.

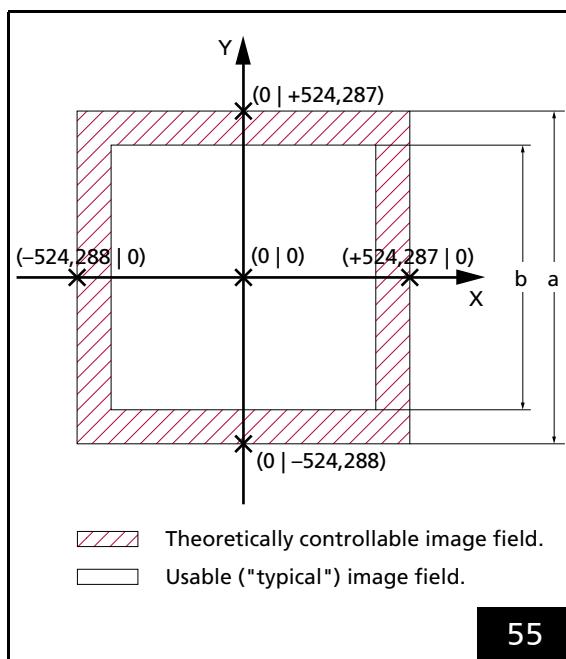


Image field size.

The scan head has a usable **Image field**. If the laser focus moves outside this field, some vignetting of the laser beam can occur. The interior of the scan head can be damaged due to excessive absorption of laser power. Refer to your scan head operating manual's section on objectives.

- Compare the calculated side length a of the maximum adjustable **Image field** with the side length b of the usable **Image field** given in the technical specifications of your scan head manual (see **Figure 55**).
- If the laser focus is to be restricted to points within the usable **Image field**, the absolute values of the x and y coordinates (in bits) must be smaller than the maximum value M , where M is the calibration factor K multiplied by *half* the side length of the usable **Image field**:

$$M = K \times b/2$$

Compatibility Modes

RTC5 Standard Mode

(Default after **load_program_file** and **set_rtc5_mode**)

The **Image field** coordinates for x axis and y axis and all associated parameters are to be specified as signed 20-bit values.

The **Image field** coordinates for the z axis are to be specified as signed 16-bit values. The RTC5 PCI Board automatically multiplies all z coordinates by $16^{(1)}$. As the z calibration factor, a value 16 times smaller must be used.

RTC4 Compatibility Mode

(**set_rtc4_mode**)

The **Image field** coordinates for the x axis, y axis and z axis and all related parameters are to be specified as signed 16-bit values. The RTC5 PCI Board automatically multiplies them by $16^{(1)}$. As xy calibration factor and z calibration factor, a value 16 times smaller must be used. See also **Chapter 2.10.2 "Porting RTC4 Source Code to the RTC5 PCI Board"**, Page 42.

(1) The allowed value range decreases accordingly.

7.3.3 Virtual Image Field

In particular for Processing-on-the-fly applications, a virtual **Image field** of size 24-bit is available.

Therefore, **Vector commands** and **"Arc" commands** can be loaded for objects up to 16 times larger than the real **Image field** (in the Processing-on-the-fly direction). For details on Processing-on-the-fly in the virtual **Image field**, see **Chapter 8.6.6 "Virtual Image Field with Processing-on-the-fly"**, Page 237.

At runtime, the current coordinates are clipped to the real **Image field** [-524,288...+524,287], see **#2...#6** in **Chapter 7.3.6 "Output Values to the Scan System"**, Page 170.

In addition, the extended value range of the virtual **Image field** can also be used for utilizing the complete real 20-bit **Image field** during coordinate transformations (such as rotations, shrinkages or shifts, see **Chapter 8.2 "Coordinate Transformations"**, Page 209). Otherwise, certain edge areas of the real **Image field** may remain inaccessible during such coordinate transformations.

Coordinate Transformations in the Virtual Image Field

For **set_fly_2d** Processing-on-the-fly applications, you can also define coordinate transformations in the virtual **Image field**. As of version DLL 541, OUT 541 this is also possible with **set_fly_x** and/or **set_fly_y**.

"Virtual coordinate transformations" (particularly translations and rotations) are sometimes needed to compensate certain mechanical tolerances of object positioning when performing continuous marking larger than the real **Image field**.

See **3** in **Chapter 7.3.6 "Output Values to the Scan System"**, Page 170.

"Virtual coordinate transformations" are defined by:

- **set_angle**(HeadNo = 4)
- **set_matrix**(HeadNo = 4)
- **set_offset**(HeadNo = 4)
- **set_offset_xyz**(HeadNo = 4)

They let you define matrix coefficients and 2D offsets (with **set_offset_xyz**, **ZOffset** is ignored). With **at_once = 1**, they become effective immediately, if no list is currently being processed. Otherwise, they are only saved as with **at_once = 0**.

Stored transformations are automatically activated when a **Processing-on-the-fly session** is restarted (the changed output position is reached at jump speed). They are deactivated after the end of this **Processing-on-the-fly session**.

As long as a **Processing-on-the-fly session** is active, the parameters for the "virtual coordinate transformation" cannot be changed, they can only be saved.

By **set_matrix**(4, 0.0, 0.0, 0.0, 0.0), the "virtual coordinate transformation" can be deactivated again, see **Section "Clock Overruns"**, Page 171.

The adjustment range for offsets is ± 23 bits. Matrix coefficients must not exceed an absolute value of 1.5. The offset is applied after the matrix operation.

For "virtual coordinate transformations" *cannot* be used:

- **set_scale**
- all list commands for coordinate transformations

7.3.4 3D Image Field

Carrying Out Adjustment

SCANLAB 3D correction tables are calculated such that the plane of the middle focus position is controlled with $z = 0$.

Therefore, the mechanical distance between scan system and working plane must be adjusted accordingly. With the varioSCAN series, a specific distance to the scan system has to be maintained in addition. The values are to be taken from the manual of the respective 3-axis scan system or varioSCAN.

Deviations from these should preferably be set via the user program by **set_defocus**, **set_defocus_offset**, **set_offset_xyz** or the corresponding list commands.

Once the working distance and distance to the scan system are correctly adjusted, then subsequently the laser focus position should be fine-tuned. For this, a test pattern is to be marked in the middle of the $z = 0$ working plane. The optimum laser focus position for processing results can be achieved:

- With the varioSCAN series, by manually turning the focusing ring, see corresponding Manual
- With a varioSCAN FLEX, by adjusting the focusing optic position, see Manual for "RTC5/6 varioSCAN 40 FLEX Extension" extension board (#0128683)
- With a varioSCAN FC and intelliWELD FC by adjusting the coefficient A of the used 3D correction table⁽¹⁾

Checking the z Axis Calibration

The optimum output values for the z axis also depend on various parameters such as beam divergence of the used laser and tolerances of the optical components.

Such information is generally not available to SCANLAB and therefore, are not included in the calculation of 3D correction tables.

Therefore, in some cases the calculated 3D correction table might not fit optimally the individual scan system. To test whether this is the case, run a laser marking test that covers the entire **3D image field**.

Check if the laser focus meets the requirements of your application. If you find that the spot diameter varies considerably, then a recalibration of the z axis correction may help under certain circumstances.

The aim of the z axis calibration procedure is to determine suitable coefficients A , B and C . These can be transferred to the RTC5 PCI Board by **load_z_table**.

A , B and C are coefficients of the parabolic function

$$z_{\text{out}} = A + B| + C|^2$$

They determine the relationship between the z output value z_{out} and the focus length value l . For each point $(x|y|z)$ in the **3D image field**, the focus length value l corresponds to the focus length difference between the specified point $(x|y|z)$ and the point $(0|0|0)$.

The ABC coefficient values of a correction file on the PC can be read-out directly with **read_abc_from_file** and written into with **write_abc_to_file**.

Notes

- ABC values from the *_ReadMe.txt file of correction file are to be interpreted as 16-bit values.

(1) Coefficients A , B and C can be queried by **get_head_para** and newly set by **load_z_table**. Only A should be modified. B and C should be passed over unchanged – as queried – by **load_z_table**.

Procedure

- Refer also to the “[Calibrating a 3-Axis Laser Scan System](#)” Manual.

- (1) Adjust the correct mechanical distance between the scan system and the $z = 0$ working plane and the correct distance between the varioSCAN device and the scan system⁽¹⁾.
- (2) Load the 3D correction file by [load_correction_file](#).
- (3) Assign the 3D correction table by [select_cor_table](#).
- (4) Read out the assigned coefficients A , B and C by [get_head_para](#).
- (5) varioSCAN FC and intelliWELD FC only: proceed with step 10. Steps 6...9 do not need to be performed.
- (6) Move the laser spot to the reference point⁽²⁾ by [goto_xyz](#).
- (7) Set the z axis to the neutral (middle) position by [load_z_table](#)(0, 0, 0).
- (8) Place a test object at the reference point.
- (9) Adjust the laser focus, see [page 159](#).
- (10) Move the focus to an arbitrary point $(x|y|z)$ within the (required) 3D image field by [goto_xyz](#)⁽³⁾.
- (11) Query the focus length value l (in bits) set by the RTC5 PCI Board for this point⁽⁴⁾ by [get_z_distance](#).
- (12) Optimize the laser focus at this point: vary the z output value z_{out} until the quality of the laser focus meets your requirements⁽⁵⁾ by [load_z_table](#)($A=z_{out}$, 0, 0).
A starting value can be calculated in accordance with $A + Bl + Cl^2$ by using the previously read focus length value l and the previously read or used values A , B and C .

- (1) See the corresponding manual.
- (2) The reference point is a point in the $z = 0$ working plane for which a middle focus length value is required for a sharp laser focus. The laser beam should be focused to the reference point when the z axis is set to the neutral position (z output value $z_{out} = 0$). The coordinate values of the reference point (in mm) are provided in the *_ReadMe.txt file that accompanies the 3D correction file or in the 3-axis scan system's or the varioSCAN's user manual. If you are using an F-Theta objective, the reference point is generally the origin (0|0|0).

- (13) Repeat steps 10...12 for as many locations $(x|y|z)$ as possible⁽⁶⁾ and write down the values $(l|z_{out})$ for each new point. If possible, seek to thereby cover the entire 3D image field required by your application.

- (14) Fit the function $z_{out} = A + Bl + Cl^2$ to your value pairs $(l|z_{out})$.

- (15) Use the resulting coefficients A , B and C to adjust the 3D correction table by [load_z_table](#)(A , B , C).

Values loaded by [load_z_table](#) are overwritten by a subsequent [load_correction_file](#) ([load_correction_file](#) sets the three coefficients A , B and C to the values of the loaded correction table). After each [load_correction_file](#) or [select_cor_table](#), you should therefore also call [load_z_table](#)(A , B , C) again.

Alternatively, the ABC values can be written permanently to the header of the correction file by [write_abc_to_file](#), see also parameter 5...7 in [Section “ct5 Correction File Header”, Page 167](#).

- (3) If you are using a scan system with an F-Theta objective, it is sufficient to select various points (0|0| z) on the z coordinate axis. The maximum possible 3D image field is specified in the corresponding user manual.
- (4) The focus length value l can be positive or negative, see notes on [get_z_distance](#).
- (5) The optimal z_{out} output value can be positive or negative.
- (6) Note that larger z control values lead to shorter working distances and that the focus thereby shifts toward the scan system. The test object might have to be tracked accordingly.

Testing 3-Axis Scan Systems with F-Theta Objective

- With the adjusted 3D correction table and 3D vector commands, see [Chapter "3D Commands", Page 223](#), mark two identical large squares. Mark one of the squares with the work piece in z position $z = z_{\min}$, the other one with $z = z_{\max}$.
- These two squares should match exactly. If this is not the case, your correction file is not perfectly suited to your objective. To solve this problem, measure the size (x and y) of both squares and report these values to SCANLAB. You will then receive a new 3D correction file.
- See also [Section "Enhanced 3D Correction", Page 225](#).

7.3.5 Image Field Correction and Correction Tables

Field Distortion

The deflection of a laser beam with a two-mirror system results in three effects:

- (1) The arrangement of the mirrors leads to a certain distortion of the **Image field**⁽¹⁾, see Figure 56.
- (2) The distance in the **Image field** is not proportional to the scan angle itself, but to the tangent of the scan angle. Therefore, the mark speed of the laser focus in the **Image field** is not proportional to the angular velocity of the corresponding galvanometer scanner.
- (3) If an ordinary lens is used for focusing the laser beam, the focus lies on a sphere. In a flat **Image field** plane, a varying spot size results.

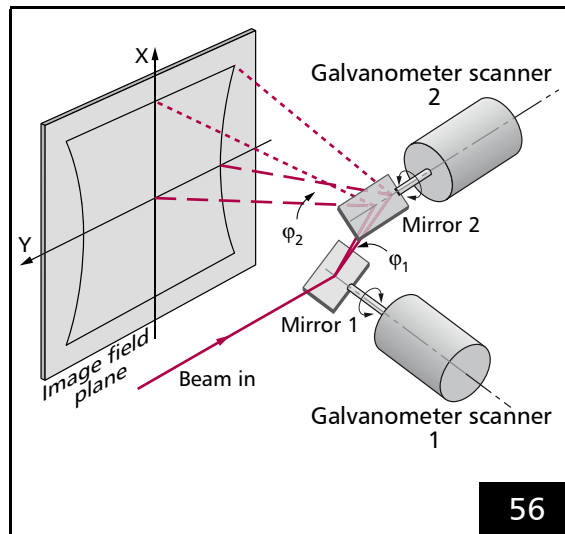
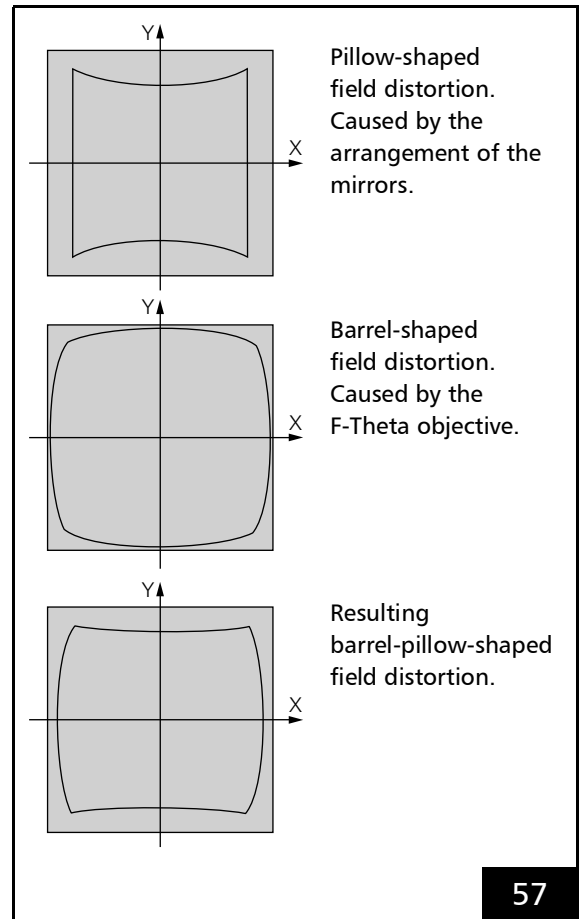


Image field distortion when deflecting a beam in a 2-mirror deflection system.

By focusing the deflected laser beam with an F-Theta objective, effect 2 and effect 3 can be avoided. However, this causes a barrel-shaped distortion of the **Image field**, see Figure 57.



Field distortion caused by the arrangement of the mirrors and by the F-Theta objective.

- (1) Cause: the distance between mirror 1 and the **Image field** depends on the size of the scan angles of mirror 1 and mirror 2. A larger scan angle leads to a longer distance.

Image Field Correction Algorithm

For these field distortions, the RTC5 PCI Board internally uses a algorithm to compensate for it. The algorithm is based on a correction table.

An orthogonal grid of 257×257 points is superimposed on the ideal square **Image field**. The adjusted x and y coordinates for a corrected output of these grid points are stored in a correction table.

To move the focus to any point within the **Image field**, the RTC5 PCI Board calculates the corrected coordinates by interpolating from the grid points in the correction table.

The correction is executed for every single **Microstep**, see also **Chapter 7.1.2 "Microstepping", Page 128**.

SCANLAB creates a correction file for every system type. For this purpose, the correction table is calculated based solely on general system data (such as mirror geometry, calibration and the objective specifications). Individual system characteristics and errors during adjustment are not taken into account.

For those customers requiring more accurate correction tables tailored to the unique properties of their particular scan systems, SCANLAB offers the **correXion** program line. These generate RTC correction files based on data derived from actual test measurements performed by the customer. For further information, refer to the corresponding manual or contact SCANLAB).

Activating Image Field Correction

To activate **Image field** correction, at the beginning of a user program the required correction tables must:

- (1) be loaded from the corresponding correction files into RTC5 PCI Board memory (by **load_correction_file**)
- (2) assigned to the used scan head connectors (by **select_cor_table** or **select_cor_table_list**)

2D correction tables activate an **Image field** correction for x and y coordinates, 3D correction tables additionally for z coordinates.

2D Correction Files and 3D Correction Files

SCANLAB supplies 2D correction files and 3D correction files. They have the extension *.ct5 and the following naming scheme applies:

- D2_xxx.ct5 for 2D correction files
- D3_yyy.ct5 for 3D correction files

Here, xxx and yyy are numbers. Each correction file is calculated for a specific optical configuration. The configuration is specified in the accompanying *_ReadMe.txt file and in parameters of the correction file header, see also **Section "ct5 Correction File Header", Page 167**.

Loading Correction Tables

By **load_correction_file**, up to 4⁽¹⁾ correction tables can be loaded from their corresponding correction file into the RTC5 PCI Board memory, see also **Chapter 8.5 "Controlling 2D Scan Systems and 3D Scan Systems"**, Page 221.

If the **Option "3D"** is *not* enabled, then only 2D correction tables can be loaded.

2D correction tables can also be loaded from 3D correction files (by **load_correction_file** with **Dim** = 2). The 3D part is ignored.

If the **Option "3D"** is enabled, a 3D correction table can even be loaded from a 2D correction file (by **load_correction_file** with **Dim** = 3). Thereby, the 2D correction table is automatically supplemented with a linear z correction.

The actually suitable Z correction can be loaded by **load_z_table** after the assignment by **select_cor_table** (see below), see **Chapter 7.3.4 "3D Image Field"**, Page 159.

Notes

- ct5-correction files differ from those of prior RTC products (file extension *.ctb) in terms of size, structure and content.
- If *.ct5 and *.ctb correction files have the same file name (except for the different extensions), then they were calculated for the same optical configuration.
- For converting older correction files, see **Section "Converting Correction Files"**, Page 169.

Assigning Loaded Correction Tables

Loaded correction tables are assigned to the two scan head connectors by **select_cor_table** or **select_cor_table_list**.

If neither the **Option "Second Scan Head Control"** nor the **Option "3D"** is enabled, then a 2D correction table can be assigned only to the first scan head connector.

If the **Option "Second Scan Head Control"** is enabled, then the two scan head connectors can each be assigned their own 2D correction table.

If the **Option "3D"** is enabled but not the **Option "Second Scan Head Control"**, then a 2D correction table or a 3D correction table can be assigned exclusively to the first scan head connector. The first scan head connector outputs position signals for an xy scan system.

With a 3D correction table, the second scan head connector outputs additionally position signals for the z axis (for example, varioSCAN) via both channels.

If the **Option "Second Scan Head Control"** and the **Option "3D"** are enabled, up to two (even different) 2D correction tables or a single 3D correction table can be assigned.

In order to control a 3D system, the (only) 3D correction table must be assigned exclusively to the connector for the xy scan system. There is no assignment for the z axis. The output for the z axis occurs automatically on both channels of the other port (**select_cor_table**(n, 0) or **select_cor_table**(0, n)).

(1) Up to 8 on request. With additional limitations on data recording and available list memory.

Notes

- If no correction table has been loaded into the RTC5 PCI Board memory, then the board outputs unforeseen values to the scan system. Therefore, at least a 1to1 correction table must be loaded (as 2D or 3D correction table), see [Section "1to1 Correction Tables", Page 166](#), if no 2D correction file or 3D correction file is available that has been calculated for the specific optical configuration.
- Correction files should have been assigned⁽¹⁾:
 - before a list is started for the first time or
 - before the galvanometer scanners of a scan system are moved by a control command (like [goto_xy](#))
- If only one scan head connector has an 2D correction table assigned, then the other scan head connector outputs no *position* signals.
- During initialization, [load_program_file](#) assigns a correction table according to [select_cor_table\(1, 0 \)](#). However, it leaves the galvanometer scanners at position (0,0). [load_program_file](#) cannot determine whether a correction table #1 is loaded at all. Therefore, there is no internal jump to the output position that is specified in the correction file. Its contents could be random (see above).
- [load_program_file](#) does not initialize the memory contents of correction tables #1 and #2. The correction table values are random immediately after switching on the RTC5 PCI Board. It is recommended to explicitly call [select_cor_table](#). Before returning, [select_cor_table](#) itself internally waits for the end of the jump .
 - If [load_correction_file](#) is called after [load_program_file](#), then:
 - [load_correction_file](#) automatically executes a [select_cor_table\(HeadA, HeadB \)](#) with the last used values for [HeadA](#) and [HeadB](#)
 - [load_correction_file](#) positions the galvanometer scanners with an internal jump at the currently set jump speed to the output position specified there
 - If [load_correction_file](#) is called prior to [load_program_file](#), then the correction table is only saved. There can be no internal jump to the intended output position.

(1) Correction files can also be loaded before the [RTC5 files](#) have been loaded by [load_program_file](#).

1to1 Correction Tables

1to1 correction tables

- Are not assigned to a particular scan system
- In general, serve test purposes only
- Alternatively, can be loaded by **load_correction_file** with parameter **Name = NULL**.
 - According to the further **Dim** parameter a 2D 1to1 correction table or 3D 1to1 correction table is being generated internally.

Because some user programs require a file name and do not accept **NULL** as the name, the 1to1 correction file **Cor_1to1.ct5** is supplied within the RTC5 software package. This contains a calibration factor of value 0. If a user program strictly needs an “authentic” calibration factor, a corresponding value can be specified with **CorrectionFileConverter.exe**, see **Section “Folder CorrectionFileConverter”, Page 27** (button **Show File Header** > field ‘Field Calibration [Bit/mm]’).

Inverse Tables

The ct5-correction file format supports “inverse tables”. These are required for back transforming actual scan head positions to the associated **Image field** coordinates.

For further information, see:

- Parameter **11**, see **page 168**
- **Chapter 8.1.3 “Monitoring the Positioning”, Page 200**
- **Section “Converting Correction Files”, Page 169**

ct5 Correction File Header

The ct5-correction file header contains 16 parameters, see following table.

These can be read out and thus directly incorporated into a user program:

- from the currently loaded correction tables by **get_table_para**
- from assigned correction tables by **get_head_para**

Notes

- With ctb-correction files, some parameter values can exclusively be taken from its associated *_ReadMe.txt file. These must be transferred manually to the user program.
- A ct5 file created by converting another ctb file, see [Section "Converting Correction Files", Page 169](#), does not contain all parameter data.
- A 1to1 correction file does not contain all parameter data.
- Parameters 3...7 are only relevant for 3D marking. In 2D correction tables, they are 0.
- For the same 3-axis scan system, the ABC coefficients (parameter 5...7) in ctb-correction files and ct5-correction files are identical.

Parameter ^(a)	Description
0	Type of the correction table. • = 0.0: 2D correction table • = 1.0: 3D correction table
1	Calibration factor K_{xy} [bit/mm]. See Chapter 7.3.2 "Image Field Size and Image Field Calibration", Page 156 .
2	Focal length or working distance [mm]. • For a configuration with a scan objective: the effective focal length of the objective [mm]. • For a configuration without a scan objective: the working distance A [mm]. A = distance from the optical axis of the incident laser beam at the first deflection mirror to the image plane.
3	Stretch factor for the x direction. Compensates the pyramid-shaped Image field change which exists in the z direction of 3D markings.
4	Stretch factor for the y direction. See parameter 3.
5	Coefficient A of the polynomial for z axis control, see page 159 : offset part, ± 26 Bit.
6	Coefficient B of the polynomial for z axis control, see page 159 : linear part, ± 11 Bit.
7	Coefficient C of the polynomial for z axis control, see page 159 : square part, ± 4 Bit.
8	Number of the correction file. With correction files supplied by SCANLAB, the parameter corresponds to the number in the file name (for example, 145 for D2_145.ct5 or D3_145.ct5).

Parameter ^(a) (cont'd.)	Description (cont'd.)
9	Differentiation between correction with or without an F-Theta objective. The following applies: Parameter = $10 \times P_{Obj} + P_{Typ}$ with $P_{Obj} = 0$: Correction without F-Theta objective $P_{Obj} = 1$: Correction with F-Theta objective - If correction with F-Theta objective: $P_{Typ} = 0.0$: without distortion data $P_{Typ} = 1.0$: with F-Theta's F-stop progression condition $P_{Typ} = 2.0$: with image height table
10	Indicator for the source of the correction file. The following applies: Parameter = $1000 \times P_{Orig} + P_{Ver}$ with $P_{Orig} = 10000$: Originally calculated file $P_{Orig} = 20000$: converted from <i>ctb</i> file $P_{Orig} = 30000$: reconstructed from txt file By manipulating a correction file using correction programs available from SCANLAB, P_{Orig} is increased by 1 in each case. P_{Ver} = Version number of the program used to create the correction file
11	Information about the inverse table. The following applies: Parameter = $P_{Exist} + 2 \times P_{Calc}$ with $P_{Exist} = 1.0$: valid inverse table is present $P_{Exist} = 0.0$: no valid inverse table present - If valid inverse table is present: $P_{Calc} = 0$: inverse table calculated ab initio $P_{Calc} = 1$: inverse table numerically calculated
12	Angle calibration of the scan system. Mechanical angle deflection in $[\pm \text{ }^\circ]$ at 96% of the maximum control.
13	Code for the scan head geometry used for the calculation (for internal use only), for example, $= -1.0$: unknown geometry (for example, for a table converted from a <i>ctb</i> file) $= 0.0$: standard geometry
14	Indicator for whether an additional protective window has been taken into account. The following applies: Parameter = $1,000,000 \times P_T + 1,000 \times P_I$ with P_T = Protective window thickness in mm (max. 2 decimal places) P_I = Refraction index (max. 3 decimal places) Example: The value 3,521,450.0 corresponds to a protective window thickness of 3.52 mm and a refraction index of 1.450.
15	Indicator for whether the Image field size has been limited in the correction file. $= 0.0$: without field size limit $= 2.0$: with field size limit

(a) Numbering according **get_table_para** and **get_head_para**.

Converting Correction Files

The RTC5 software package includes the program **CorrectionFileConverter.exe** for converting **ctb** correction files to **ct5** correction files and vice versa. The corresponding manual is supplied in English and German.

When converting a **ctb**-correction file into a **ct5**-correction file, the **ct5**-correction file header receives a corresponding origin indicator (parameter **10**, **page 168**, $P_{\text{Orig}} = 20,000$).

Such conversions do *not* produce fully complete **ct5**-correction files because the file header lacks several parameters. These can be subsequently entered manually by **CorrectionFileConverter.exe** (via button **Show File Header**).

Moreover, it may happen that the inverse correction table to be calculated numerically during the conversion does not cover the entire **Image field** or cannot be calculated. In this case, a 1to1 correction table is inserted instead, see also parameter **11**, **page 168**.

In contrast, converting a **ct5**-correction file into a **ctb**-correction file results in a fully complete **ctb**-correction file.

Parameter information from the **ct5**-correction file header are:

- no longer contained in a **ctb**-correction file for 2D correction table
- contained partially in a **ctb**-correction file for 3D correction tables

7.3.6 Output Values to the Scan System

Calculation

Calculation steps for generating the RTC output values to the scan system from the current x, y, z coordinate values
(1) Decomposing vectors and arcs to Microsteps ^(a)
(2) If applicable: Applying a wobble motion ^(b) – for example, from set_wobble_mode call
(3) If applicable: Applying a global coordinate transformation in virtual Image field ^(c) – for example, from calls of set_matrix , set_angle , set_offset with HeadNo = 4
(4) If applicable: Applying the set Processing-on-the-fly correction ^(d) – for example, from calls of set_fly_x or set_fly_x_pos
(5) If applicable: Applying coordinate transformations to align the 1 (2) scan system(s) relative to the Image field ^(e) (a) Transforming x coordinate values and y coordinate values as per matrix × angle × scale ("2D transformation") • for example, from calls of set_matrix , set_angle , set_scale and corresponding list commands (b) Applying an offset to x coordinate values and y coordinate values as well as z coordinate values ^(f) • for example, from calls of set_offset or set_offset_xyz and corresponding list commands
(6) Clipping x coordinate values and y coordinate values to the boundary values of the real 20-bit Image field as soon as they exceed the real Image field range (virtual Image field) ^(g)
(7) If applicable: Correcting x coordinate values and y coordinate values according to the assigned correction table ("2D Image field correction") ^(h) – From select_cor_table call or select_cor_table_list call
(8) If applicable: Applying the focus shift to z coordinate values – for example, from set_defocus call or set_defocus_list call
(9) If applicable: Correcting z coordinate values according to the assigned correction table ("3D image field correction") ^(h) – From select_cor_table call or select_cor_table_list call
(10) Clipping z coordinate values to the boundary values of the real 20-bit Image field as soon as they exceed the real Image field range (virtual Image field) ⁽ⁱ⁾
(11) If applicable (only some legacy scan systems): Applying an explicitly ^(j) or automatically ^(k) set gain correction and offset correction for the x galvanometer scanner and y galvanometer scanner – From set_hi call (explicit) – From auto_cal call (automatic)

(a) See Chapter 7.1.2 "Microstepping", Page 128.

(b) See Chapter 8.4 "Wobble Mode", Page 217.

(c) See Section "Coordinate Transformations in the Virtual Image Field", Page 158.

(d) See Chapter 8.6 "Processing-on-the-fly", Page 227.

(e) See Chapter 8.2 "Coordinate Transformations", Page 209 and Chapter 8.5.2 "3D Scan Systems", Page 222.

(f) A complete 3D calculation is always performed. If the Option "3D" is not enabled, z values are just not outputted.

(g) See Chapter 7.3.3 "Virtual Image Field", Page 158.

(h) See Chapter 7.3.5 "Image Field Correction and Correction Tables", Page 162.

(i) See Chapter 7.3.3 "Virtual Image Field", Page 158.

(j) See Section "Customer-Specific Calibration", Page 254.

(k) See Chapter 8.10 "Automatic Self-Calibration", Page 251.

Notes

- Overflowing values are always clipped to the boundary values of the maximum possible value range.
- A complete 3D calculation is always performed. If the Option "3D" is not enabled, z values are just not outputted.

Value Ranges

For all scan system axes, signed 20-bit values (–524,288...+524,287) are outputted. This also applies to values which the scan system returns to the RTC5 PCI Board, see also:

- [Section "RTC5 Standard Mode", Page 157](#)
- [Section "RTC4 Compatibility Mode", Page 157](#)
- [Chapter 4.5.2 "XY2-100 Converter \(Accessory\)", Page 56](#)

Precalculation and Diagnosis

With **get_galvo_controls** the output values resulting for the given coordinates and setting parameters in the current configuration (correction table, coordinate transformations) can be determined without the galvanometer scanners moving.

Clock Overruns

The 10 μ s clock cycle might not always suffice for calculating all data required by the computation-intensive **Jump Commands**, **Mark Commands** and **Arc Commands** if several of the available command options are utilized simultaneously – for example, simultaneous control of two scan systems, wobble motion, coordinate transformation in the virtual **Image field**, Processing-on-the-fly for two axes (correction by **McBSP interface**), "Automatic Laser Control", para vectors, data recording, "Variable **Polygon Delay**", short list commands (for example, in a **Polyline**), control commands during list execution.

This overrun situation can be internally detected and counted. You can appropriately test your user program by using **get_overrun** to count the number of overruns. Such overruns result in one or several peripheral ports not being accessible during the current 10 μ s clock cycle, possibly including output to the scan head. The galvanometer scanner motion might also could pause for 10 μ s.

7.3.7 Status Monitoring and Diagnostics

For status monitoring and diagnostic purposes, **get_value** or **get_values** can be used to read out a variety of signals:

- A 20-bit status word which is returned by the scan system, see [Section "Status Information Returned from the Scan System", Page 172](#)
- The current LASERON signal
- The current Cartesian control values (that is, the so-called "sample values" common to both scan head connectors, see #2, #3 and #4.
- The control values specific to each scan head connector which take into account
 - any coordinate transformations defined by **set_matrix**, **set_scale**, **set_angle** or by the corresponding list commands, see #5a.
 - any z coordinate offset defined by **set_offset_xyz** or **set_offset_xyz_list**, see #5b.
 - any focal length offset defined by **set_defocus** or **set_defocus_list** and any loaded correction table, see #8 and #9.

Each of these signals can be recorded on the RTC5 PCI Board for a longer time period by **set_trigger/set_trigger4** – with a selectable sample period. After that, **get_waveform** can be used to transfer them to the PC for analysis.

The current status of a measurement session started with **set_trigger/set_trigger4** can be queried by **measurement_status**.

The values returned by the scan system are always 20-bit values.

Status Information Returned from the Scan System

The scan system transmits the following signals to the RTC5 PCI Board every 10 μ s via the SL2-100 protocol:

- A 20-bit status word.
It can mean the following data types:
 - The actual 20-bit status word (the upper 16 bits are identical to the 16-bit status word of the XY2-100 protocol (**XY2-100 status word**), for a description see **get_head_status**).
 - Only for iDRIVE scan systems⁽¹⁾, see [Chapter 8.1 "iDRIVE Functions", Page 198](#): a different data type selectable with **control_command**.

The 20-bit status word can:

- be read out by **get_value/get_values**
- be recorded by **set_trigger/set_trigger4**
- 6 additional status bits.
With iDRIVE scan systems⁽¹⁾ with SL2-100 protocol, the status bits (PowerOK, TempOK and PosAck; per axis) are transferred in parallel to and independently from the 20-bit status word. Therefore, these status bits can even be used to monitor the scan system (see [Section "Automatic Suppression of Laser Control Signals", Page 176](#)) if, for example, an "Automatic Laser Control" (see [Chapter 7.4.9 "Automatic Laser Control", Page 187](#)) is active that requires a different return data type.
The 6 additional status bits can be read out by **get_head_status**.

Notes

- Scan systems with XY2-100 protocol require the use of the **XY2-100 Converter (Accessory)**.
- For iDRIVE scan systems⁽¹⁾ with XY2-100 protocol, the actual **XY2-100 status word** must be set as the to-be-returned data type (default type after switching on).

(1) See Glossary entry on [page 23](#).

7.4 Laser Control

At its laser signal output ports (LASERON, LASER1 and LASER2), the RTC5 PCI Board outputs signals which can be used for controlling various laser types ("laser control signals").

The laser signal output ports are part of:

- the D-SUB-LASER connector, see [Chapter 4.6.1 "LASER Connector", Page 60](#)
- the EXTENSION 2 socket connector, see [Chapter 4.6.3 "EXTENSION 2 Socket Connector", Page 67](#)

The laser control mode is set by **set_laser_mode**:

- The **CO₂ Mode** (laser mode 0; default after **load_program_file**) is designed for controlling a CO₂ laser.
- The YAG modes (laser mode 1, 2, 3, 5) are designed for controlling lasers from the Nd:YAG family (and related).
- **Laser Mode 4** and **Laser Mode 6** are designed for controlling free-running lasers.

All laser control signals are TTL signals. By **set_laser_control**, it is set whether they are active-HIGH or active-LOW, see also [Chapter 4.6.1 "LASER Connector", Page 60](#). Their current setting can be queried by **get_startstop_info** (Bit #13).

For the maximum current load values, see [Chapter 15 "Technical Specifications – RTC5 PCI Board", Page 734](#).

7.4.1 Enabling, Activating and Switching Laser Control Signals

All laser control signals are suppressed:

- After a **Hardware reset** and
- After initialization with **load_program_file**.

Then, all laser signal output ports (LASERON, LASER1 and LASER2) are in the high impedance mode.

Before laser control signals can be outputted at all, their polarity must first be set by **set_laser_control**. This cancels the tristate state at the same time (= "global unblock"). By default, LASERON and LASER1/LASER2 are set to their respective "Off" levels.

The tristate state is only set again:

- after a **Hardware reset** (restart)
- a call of **load_program_file**

Furthermore, real laser control signals must be enabled:

- either simultaneously by **set_laser_control**(Bit #2 = 0) or
- afterwards with the separate command **enable_laser**

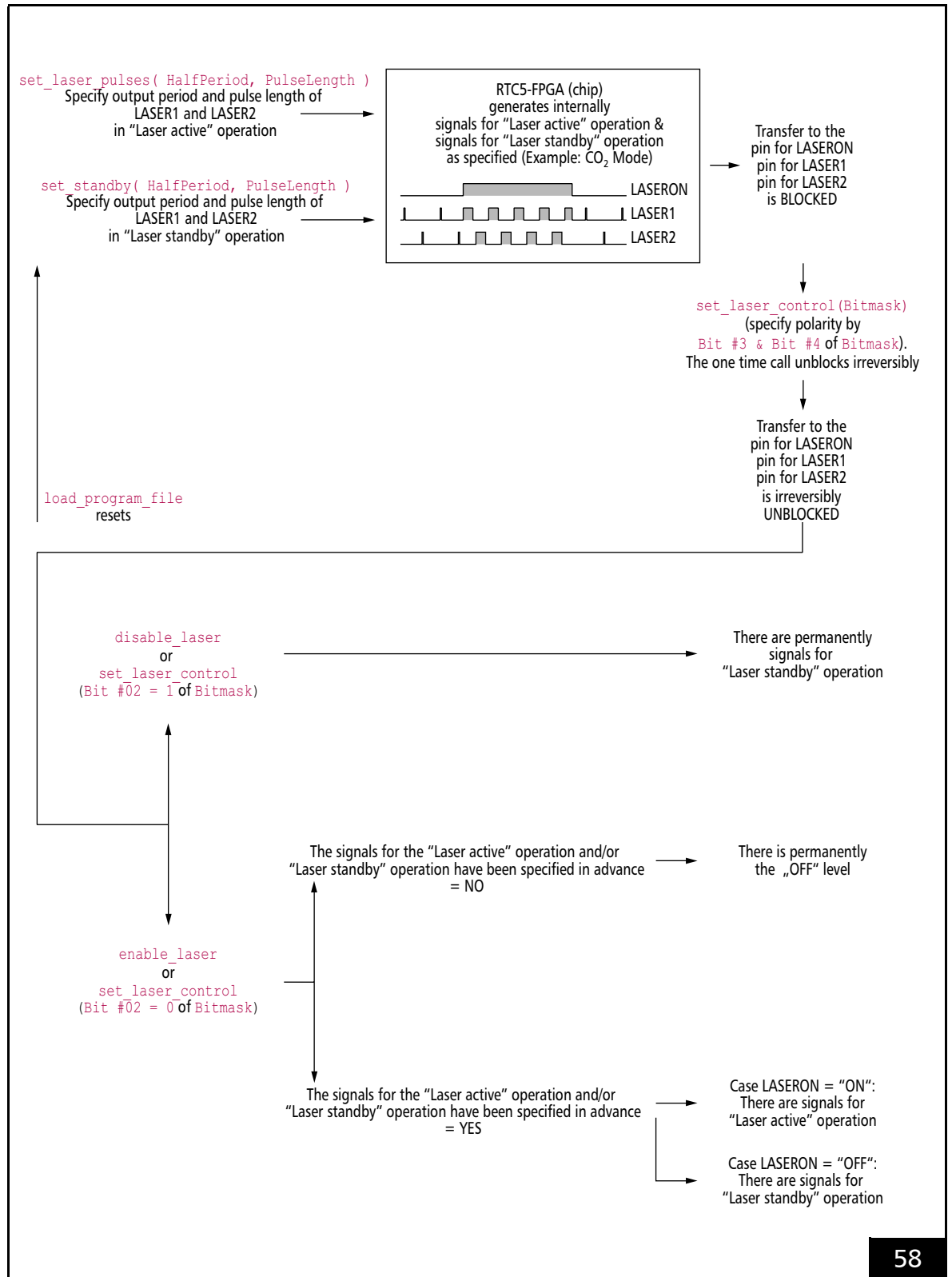
These can be suppressed again by **disable_laser** or **pause_list**. In both cases, all laser control signals are set to their respective "Off" levels.

By **get_startstop_info** (Bit #13, Bit #10 as well as Bit #9) it can be read out:

- The polarities of LASERON and LASER1/LASER2
- The "global unblock" by **set_laser_control**
- With $\geq$ DLL 546, $\geq$ OUT 546: The enabling of the **Signals for "Laser Active" Operation** by **enable_laser**

General notes on the laser control signals can be found in [Section "Signals for "Laser Active" Operation", Page 175](#) and [Section "Signals for "Laser Standby" Operation", Page 175](#). Mode-specific details depend on the set laser mode, see [Chapter 7.4.3 "CO₂ Mode", Page 177](#) to [Chapter 7.4.8 "Pulse Picking Laser Mode", Page 186](#).

The assignment of the laser control signals to the respective output pins at the LASER connector can be configured, see [Chapter 7.4.2 "Configuring the LASER Connector", Page 176](#).



RTC5 boards hardware and laser control-related RTC5 commands.

Signals for “Laser Active” Operation

By **set_laser_pulses**, **set_laser_pulses_ctrl** or **set_laser_timing**, the Signals for “Laser Active” Operation can be

- **activated:** HalfPeriod $\neq 0$ and PulseLength $\neq 0$
- **deactivated:** HalfPeriod = 0 and/or PulseLength = 0

Even if the Signals for “Laser Active” Operation have been enabled and activated, they are *only outputted at the output ports, if they are switched on by further commands*.

They are automatically switched on, when:

- **Mark commands** are called, see Chapter 7.1.1 “Marking with Vector Commands and “Arc” Commands”, Page 124

They are automatically switched off, when:

- a **Mark command** is followed by a normal non-mark command
- a list is terminated by **set_end_of_list** or **stop_execution**
- a list is temporarily suspended by **set_wait**, **pause_list** or **stop_list**

In the latter case, the Signals for “Laser Active” Operation are switched on again if the list is continued by **release_wait** or **restart_list**.

They can also be switched on and off within a list with **laser_signal_on_list** and **laser_signal_off_list** or outside a list with **laser_signal_on** and **laser_signal_off** for an unlimited time.

Pulse Completion

Whether a modulation pulse (Q-Switch pulse) started with the LASERON signal switched on is still completely executed or cut off at LASER1 if it has not yet been fully processed when the LASERON signal is switched off, can be set by **set_laser_control(Bit #0)**.

Signals for “Laser Standby” Operation

By **set_standby** or **set_standby_list**, the Signals for “Laser Standby” Operation can be

- **activated:** HalfPeriod $\neq 0$ and PulseLength $\neq 0$
- **deactivated:** HalfPeriod = 0 and/or PulseLength = 0

The Signals for “Laser Standby” Operation are continuously outputted at the output ports after their activation (without further commands), if no Signals for “Laser Active” Operation are switched on.

If the Signals for “Laser Active” Operation are deactivated (for example, by PulseLength = 0), there is no changeover. Then the Signals for “Laser Standby” Operation are continuously outputted even during the execution of **Mark commands**.

If the Signals for “Laser Standby” Operation are deactivated, all output ports are set to the “Off” level in the “Laser standby” mode.

If activated signals for the “Laser standby” operation are to be deactivated when stopping a list with **pause_list** (here, only the Signals for “Laser Active” Operation are automatically deactivated), users must explicitly initiate this by calling **set_standby(PulseLength = 0)**.

The current “Laser standby” parameters that may have been changed within a list can be read out by the control command **get_standby**.

Pulse Completion

With the Signals for “Laser Standby” Operation, pulse completion, see Section “Pulse Completion”, Page 175, is not supported.

Automatic Suppression of Laser Control Signals

Case: Scan System Status Errors

By **set_laser_control** (Bit #16...Bit #27), it can set that the laser control signals are to be automatically suppressed when the corresponding scan-system status signal (PowerOK, TempOK, PosAck of axis X/Y of head A/B) indicates an error (that is, is 0; "NOK").

As soon as at least one of the specified status signals is 0, then:

- Output of the laser control signals are automatically interrupted. They are only continued, if all selected status signals are simultaneously 1 (laser control signals disabled by **set_laser_control**, **disable_laser** or **pause_list** remain disabled regardless of the status signals' current value)
- Internal error bits are (cumulatively) set (which can be read-out by **get_marking_info**)
- If accordingly set by **set_laser_control**(Bit #28 = 1), a **stop_execution** is automatically executed (the list stops, laser control signals get permanently switched off)

Case: Galvanometer Scanner Position Exceedances

range_checking can be used to define that the laser control signals are to be automatically suppressed as soon as a galvanometer scanner exceeds a predefined range limit.

They are automatically switched on again as soon as the next **Mark command** starts within the permitted range; that is, an interrupted **Polyline** remains suppressed for the rest of the **Polyline**.

7.4.2 Configuring the LASER Connector

LASER connector pin (01), (02) and (09) can be configured by **config_laser_signals** and **config_laser_signals_list**, see **Figure 17**.

If you employ a variety of lasers or laser operational modes, then these commands might eliminate the need to configure at the hardware level (that is, using various cables and switches).

Whereas the default setting (for normal markings) outputs the LASERON signal as a laser start signal on the LASERON channel, you could, for example, configure the LASER2 signal as a gate signal outputted on the LASERON channel in **Pixel Output Mode** with pulse picking.

For other operational modes, you could also configure the **FirstPulseKiller signal** as the laser start signal on the LASERON channel. This way, LASER1 signals can be outputted even before a delayed switch-on of the laser (which is not possible in the default setting).

The following description of the various laser modes applies to the default setting for laser signal output.

7.4.3 CO₂ Mode

set_laser_mode(0) sets the **CO₂ Mode** ("Laser Mode 0").

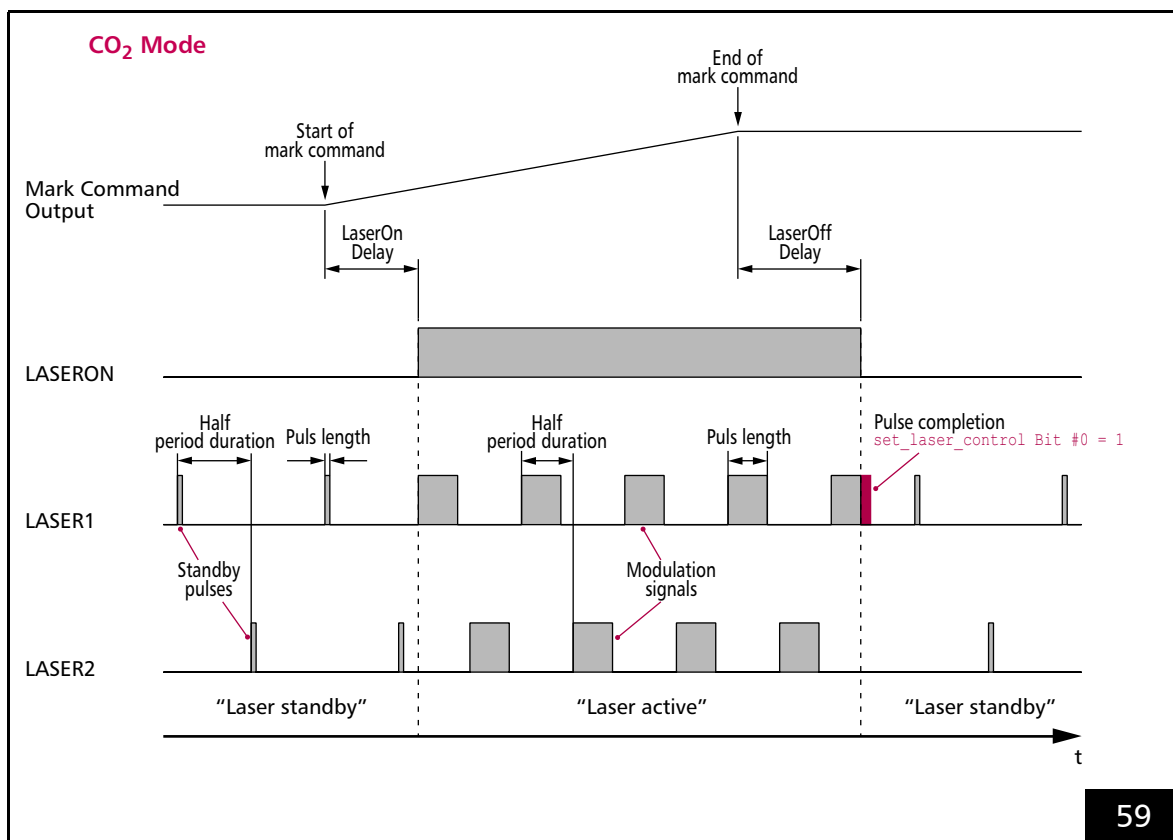
The timing diagram, see **Figure 59**, shows the corresponding signals using the example of an isolated mark command.

For "laser active" operation:

- The LASERON signal is switched on
- 2 alternating modulation signals are outputted at the LASER1 and LASER2 output port. Their pulse length and period duration can be defined by **set_laser_pulses**, **set_laser_pulses_ctrl** or **set_laser_timing**.

For "laser standby" operation:

- The LASERON signal is switched off
- Alternating standby pulses are outputted at the LASER1 and LASER2 output port. Their pulse length and period duration can be defined by **set_standby** or **set_standby_list**.



Timing diagram of the signals in **CO₂ Mode** (with active-HIGH laser control signals).
Example: isolated mark command.

Notes

- LASER1 signals and LASER2 signals are
 - activated by: HalfPeriod $\neq 0$ and PulseLength $\neq 0$
 - deactivated by: HalfPeriod = 0 and/or PulseLength = 0 (default setting after **load_program_file**)
- The LASER2 signal is phase-shifted by half a signal period in relation to the LASER1 signal. It can be used for the control of a second laser tube. By **set_laser_control** (Bit #1 = 1), both signals can be exchanged with each other. To control laser power, the pulse length of the LASER1 signals and LASER2 signals can be varied. Both signals share the same pulse lengths and periods.
- For the LASER1 signals and LASER2 signals, *half* of the output period must be specified.
- **set_laser_pulses** and **set_laser_timing** are delayed short control commands. They can also be used to change the laser parameters between two **Mark commands**.
The time base for the signals is always:
 - 1/64 μs in **RTC5 Standard Mode**, see also **Section “RTC5 Standard Mode”, Page 157**
 - 1/8 μs in **RTC4 Compatibility Mode**, see also **Section “RTC4 Compatibility Mode”, Page 157**
- **Section “Pulse Completion”, Page 175** affects also LASER2 signals.

7.4.4 YAG Mode 1, YAG Mode 2, YAG Mode 3, YAG Mode 5

By `set_laser_mode`, a YAG Mode can be set:

YAG Mode	Call
YAG Mode 1	<code>set_laser_mode(Mode = 1)</code>
YAG Mode 2	<code>set_laser_mode(Mode = 2)</code>
YAG Mode 3	<code>set_laser_mode(Mode = 3)</code>
YAG Mode 5	<code>set_laser_mode(Mode = 5)</code>

The timing diagram, see [Figure 60](#), shows the corresponding signals using the example of an isolated mark command.

In each YAG mode, for “laser active” operation:

- The LASERON signal is switched on
- A Q-Switch signal is outputted at the LASER1 output port, see [Section “Q-Switch Signal”, Page 179](#).
- A adjustable [FirstPulseKiller signal](#) is outputted at the LASER2 output port, see [Section “FirstPulseKiller Signal”, Page 179](#).

In each YAG mode, for “laser standby” operation:

- The LASERON signal is switched off
- Standby pulses are outputted at the LASER1 output port. Pulse length and period duration of the standby pulses can be set by `set_standby` or `set_standby_list`.

Notes

- LASER1 signals are
 - activated by: `HalfPeriod ≠ 0` and `PulseLength ≠ 0`
 - deactivated by: `HalfPeriod = 0` and/or `PulseLength = 0` (default setting after `load_program_file`)

Q-Switch Signal

The Q-Switch signal serves to control the quality switch of the laser. The Q-Switch period duration and pulse length are set by `set_laser_pulses`, `set_laser_pulses_ctrl` or `set_laser_timing`.

Notes

- See also [Section “Pulse Completion”, Page 175](#) (Signals for “Laser Active” Operation).
- See also [Section “Pulse Completion”, Page 175](#) of (Signals for “Laser Standby” Operation).

FirstPulseKiller Signal

The [FirstPulseKiller signal, Page 311](#) serves to signal the first laser pulses of a pulse sequence, if they do not correspond to the usual continuous power.

The [FirstPulseKiller signal, Page 311](#) is only provided. The laser itself must respond appropriately.

The [FirstPulseKiller signal, Page 311](#) is outputted automatically together with the LASERON signal.

The length of the [FirstPulseKiller signal, Page 311](#) is set with `set_firstpulse_killer` or `set_firstpulse_killer_list`.

See also [config_laser_signals, FirstPulseKiller signal, Page 311](#).

Differences Between the YAG Modes

YAG Mode 1, YAG Mode 2, YAG Mode 3 and YAG Mode 5 only differ in the relative start time of the first Q-Switch pulse with reference to the **FirstPulseKiller** signal, see also the timing diagrams in **Figure 60**.

After the Q-Switch delay has expired, the first Q-Switch pulse starts. The Q-Switch delay value can be specified by **set_qswitch_delay** or **set_qswitch_delay_list**; alternatively by selecting the YAG mode:

- YAG Mode 1: 0
- YAG Mode 2: length of the **FirstPulseKiller** signal
- YAG Mode 3: 10 μ s
- YAG Mode 5: value from **set_qswitch_delay** or **set_qswitch_delay_list**

By **set_laser_mode**, the Q-Switch delay is merely preset. Therefore, in YAG Mode 1, YAG Mode 2, YAG Mode 3, too, the Q-Switch delay can be subsequently changed by **set_qswitch_delay** or **set_qswitch_delay_list**.

In YAG Mode 2, the Q-Switch delay is also adjusted accordingly with each **set_firstpulse_killer** or **set_firstpulse_killer_list**.

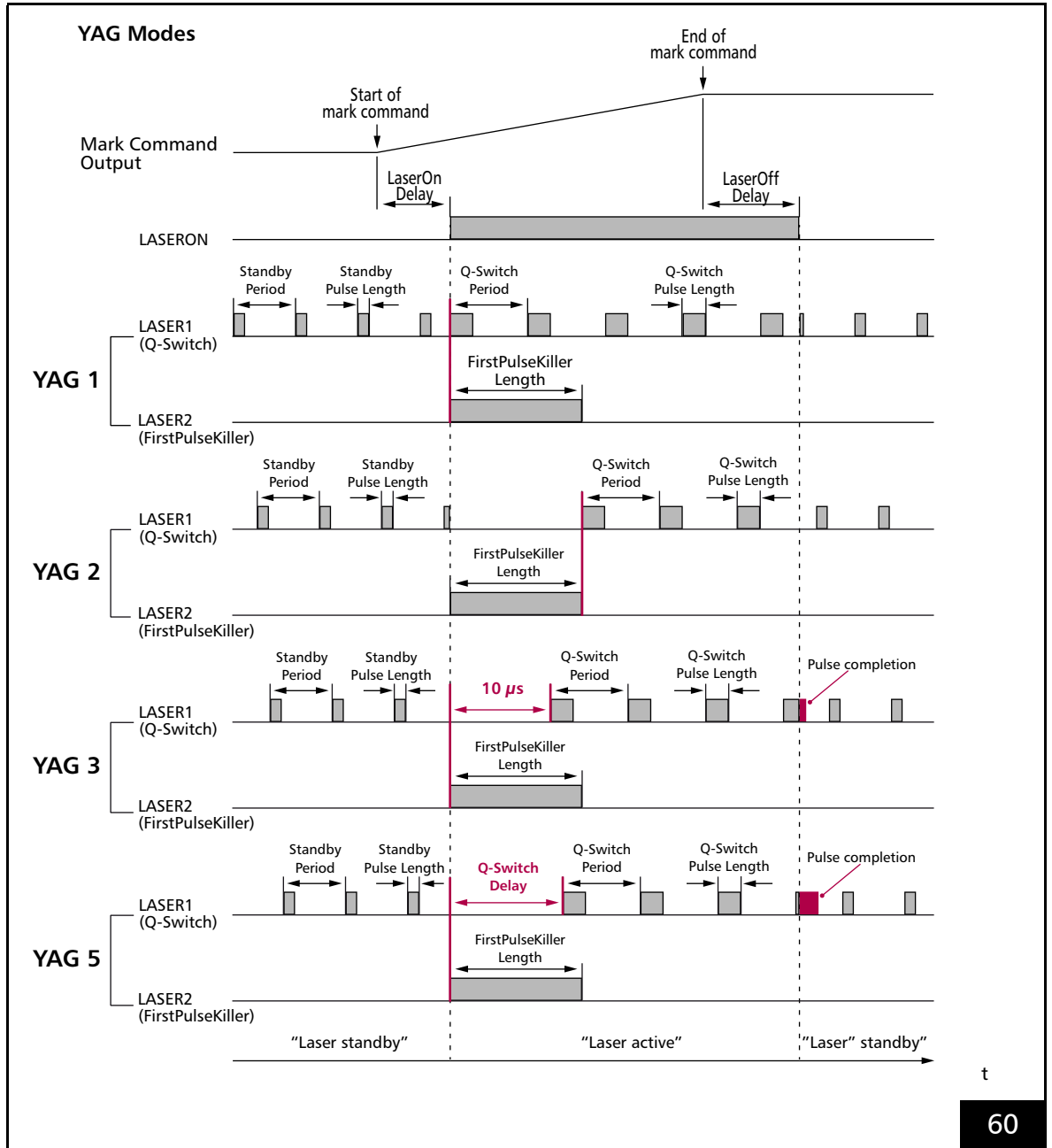
Lamp Current (Laser Power)

To control the lamp current of a YAG laser, the 12-bit analog output ports, **ANALOG OUT1** or **ANALOG OUT2** can be used. They are available at the 15-pin D-SUB LASER connector, see **Figure 17**. **ANALOG OUT2** is also available at the MARKING ON THE FLY socket connector, see **Figure 25**.

To define the analog output signal, the control command **write_da_x** and the delayed short list command **write_da_x_list** are provided.

Alternatively, the lamp current can be controlled digitally via the 8-bit digital output port (at the EXTENSION 2 socket connector, see **Figure 24**), see also **Section "8-Bit Digital Output Port", Page 68**. The commands for setting the 8-bit output port are **write_8bit_port** and **write_8bit_port_list**.

When the lamp current is changed, list execution can be halted by **long_delay** until a constant laser power has been achieved.



Timing diagram of the signals in the YAG modes (with active-HIGH laser control signals). Standby signals are not synchronized with the "laser-active" signals and can have arbitrary phase alignments.
Example: isolated mark command.

7.4.5 Laser Mode 4

`set_laser_mode(4)` sets the **Laser Mode 4**.

The timing diagram, see **Figure 61**, shows the corresponding signals using the example of an isolated mark command.

At LASER1 output port, for “laser active” operation as well as for “laser standby” operation standby pulses are outputted continuously.

Pulse length and period duration of the standby pulses can be set by `set_standby` or `set_standby_list`.

For “laser active” operation:

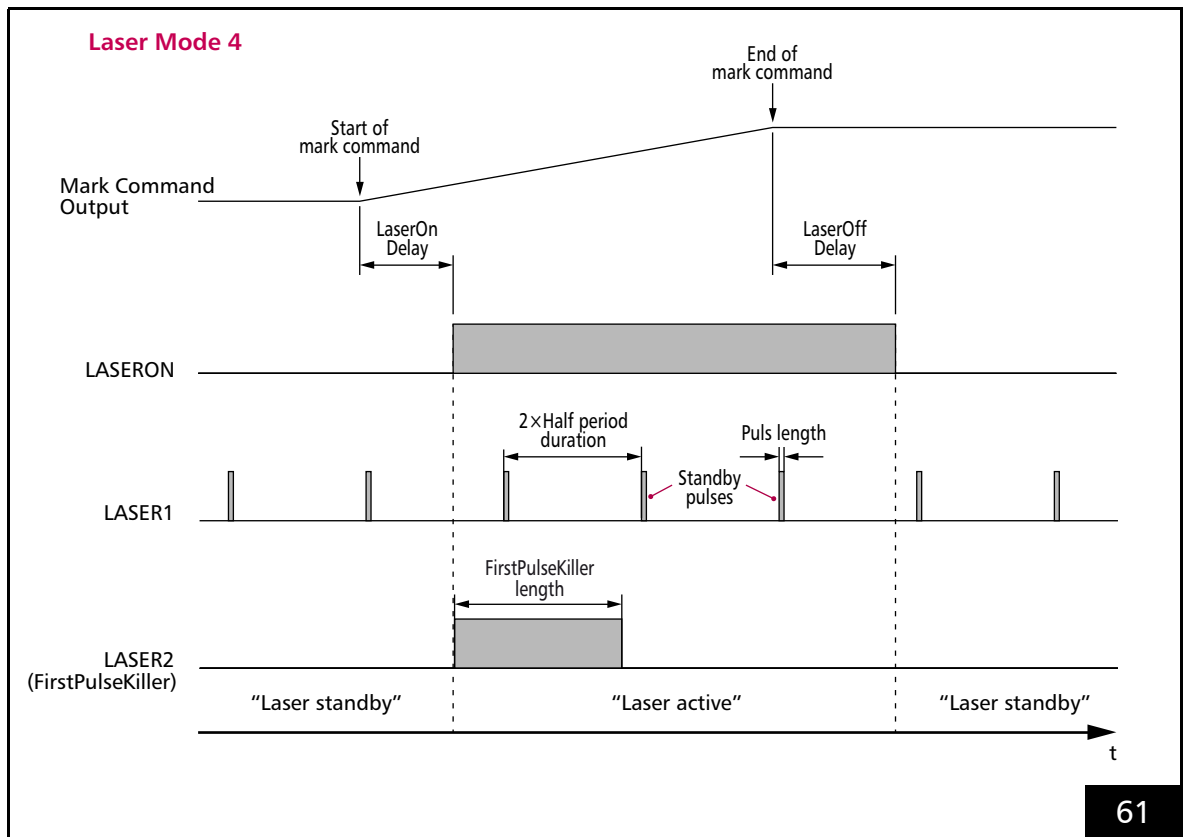
- The LASERON signal is switched on
- A programmable **FirstPulseKiller** signal is outputted at the LASER2 output

For “laser standby” operation:

- the LASERON signal is switched off
- and the LASER2 signal is switched off

Notes

- LASER1 signals are
 - activated by: $\text{HalfPeriod} \neq 0$ and $\text{PulseLength} \neq 0$
 - deactivated by: $\text{HalfPeriod} = 0$ and/or $\text{PulseLength} = 0$ (default setting after `load_program_file`)
- The **FirstPulseKiller** signal is started together with the LASERON signal automatically. The length of the **FirstPulseKiller** signal is set by `set_firstpulse_killer` or `set_firstpulse_killer_list`.
- **Laser Mode 4** is used for some fiber lasers.



Timing diagram of the signals in **Laser Mode 4** (with active-HIGH laser control signals).
Example: isolated mark command.

7.4.6 Laser Mode 6

`set_laser_mode(6)` sets the **Laser Mode 6**.

Laser Mode 6 is like **Laser Mode 4** and is provided for those free-running lasers whose gate signals (LASERON) must *not* be changed during the duration of a pulse.

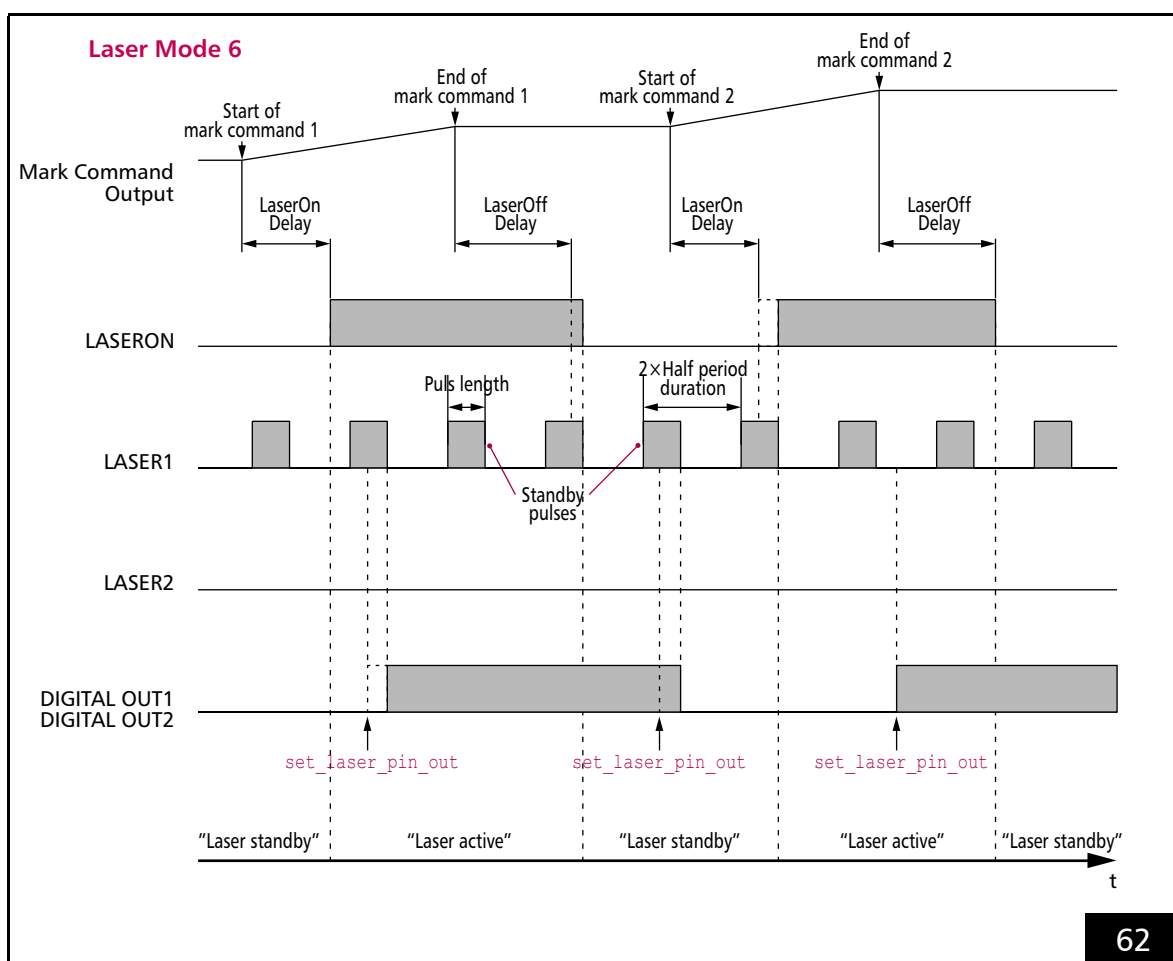
The timing diagram, see **Figure 62**, shows the corresponding signals using the example of an isolated mark command.

Laser Mode 6 signals and **Laser Mode 4** signals are the same (see also Notes there, **page 182**) with the following exception:

- As long as a standby pulse is active, switching of the LASERON signal is delayed accordingly. LASERON switches $5/64 \mu\text{s}$ after the end of the standby pulse.

Notes

- No LASER2 signal is outputted.
- The delay of the switching time when a standby pulse is still active is also valid for switching the output values at the 2-bit digital output port (DIGITAL OUT1 and DIGITAL OUT2) by `set_laser_pin_out` or `set_laser_pin_out_list`, see **Figure 62**.



Timing diagram of the signals in **Laser Mode 6** (with active-HIGH laser control signals).
Example: isolated **Mark Commands**.

7.4.7 Softstart Mode

For some applications it is important to control the laser intensity at the beginning of a marking process. SCANLAB provides a convenient solution in the form of the **Softstart Mode**, which can be used for all laser modes (for “laser active” operation).

In the **Softstart Mode**, various control values ($\text{Level}(0) \dots \text{Level}(\text{Number} - 1)$, $\text{Number} \leq 1024$) for as many as the first 1024 pulses (at the beginning of a marking process) can be defined. Either pulse length values for the laser signal at the LASER1 output (see [Figure 17](#)) or analog voltage levels for variable laser power can be defined. Analog voltage softstart values can be outputted either at the **ANALOG OUT1** or **ANALOG OUT2** output port, see [Figure 17](#).

Initialization of the **Softstart Mode** is performed by **set_softstart_mode** or **set_softstart_mode_list**. Here, the type of softstart value is defined. If analog voltage values are to be outputted, then the analog output port can also be defined. Afterwards the individual softstart control values can be defined by **set_softstart_level** or **set_softstart_level_list** commands.

After the laser control signals are switched on, the defined softstart values (max. 1024) are outputted at the selected output port simultaneously with the laser pulses of the LASER1 signal – always with the leading edge of the laser pulse. The **set_softstart_mode/set_softstart_mode_list** calls allows specification of a time period (Delay) for which the laser must have been switched off before softstart is activated at the next switch-on.

Notes

- With a large enough value for the Delay time, one can avoid the **Softstart Mode** being also activated after very short **Jump commands**.
- As soon as the LASERON signal (the laser) remains switched off longer than Delay , the value $\text{Level}(0)$ is assigned to the output port, provided the **Softstart Mode** has not been meanwhile deactivated.
- When the LASERON signal is switched on, then the values $\text{Level}(0) \dots (\text{Number} - 1)$ are automatically assigned to the output port simultaneously with the first laser pulses.
- Analog softstart values are outputted simultaneously with the laser pulses of the LASER1 signal. It might be beneficial under some circumstances to acquire these analog softstart values triggered by a signal shifted back 180° related to the LASER1 signal. For this purpose, the **CO₂ Mode** provides a LASER2 signal at the LASER2 output. And in the YAG version's, the signal at the LASER1 output can be phase-shifted back 180° by **set_laser_control** (Bit #1 = 1). For the **CO₂ Mode**, this is equivalent to exchanging LASER1 with LASER2.
- The laser pulse frequency should not exceed a value of approx. 308 kHz (this corresponds to a **set_laser_pulses-HalfPeriod** of 104), because the changing values cannot be transmitted faster. However, the laser pulse frequency cannot be automatically checked for this case.
- If Softstart values are to be outputted as analog voltage levels, then also note that digital-to-analog conversion of laser frequencies above around 100 kHz (that is, for a **set_laser_pulses-HalfPeriod** < approx. 320) cannot always be fully completed. For such laser frequencies, users must carefully verify that sufficient capability is available.
- If **Softstart Mode** is inactive, then values for the analog outputs can be set by **write_da_x** or **write_da_x_list**.

- The **Softstart Mode** is also available in **Laser Mode 4** and **Laser Mode 6**. In these modes, the LASER1 output only outputs standby pulses and therefore only analog voltage levels can be variably defined (**set_softstart_mode** _{Mode} = 1, 2, 11 or 12). Here, too, the defined softstart values are outputted simultaneously with the pulses of an internally-generated (but not outputted) LASER1 signal. The standby pulses cannot be phase shifted in **Laser Mode 4** and **Laser Mode 6** and are not synchronized with the internal LASER1 pulses. If the period durations of the standby and LASER1 pulses are set (by **set_laser_pulses** and **set_standby**) to be equal, then the analog voltage softstart values are outputted in parallel with the standby pulses, though they might have a (random) constant phase shift with respect to the standby pulses.
- The **Softstart Mode** cannot be used simultaneously with the “freely definable wobble shapes” wobble mode, see **set_wobble_vector**.

7.4.8 Pulse Picking Laser Mode

By **set_pulse_picking** or **set_pulse_picking_list**, the **Pulse Picking Laser Mode** can be selected. The laser control timing diagram in **Figure 63** shows the corresponding signals.

For "laser active" operation:

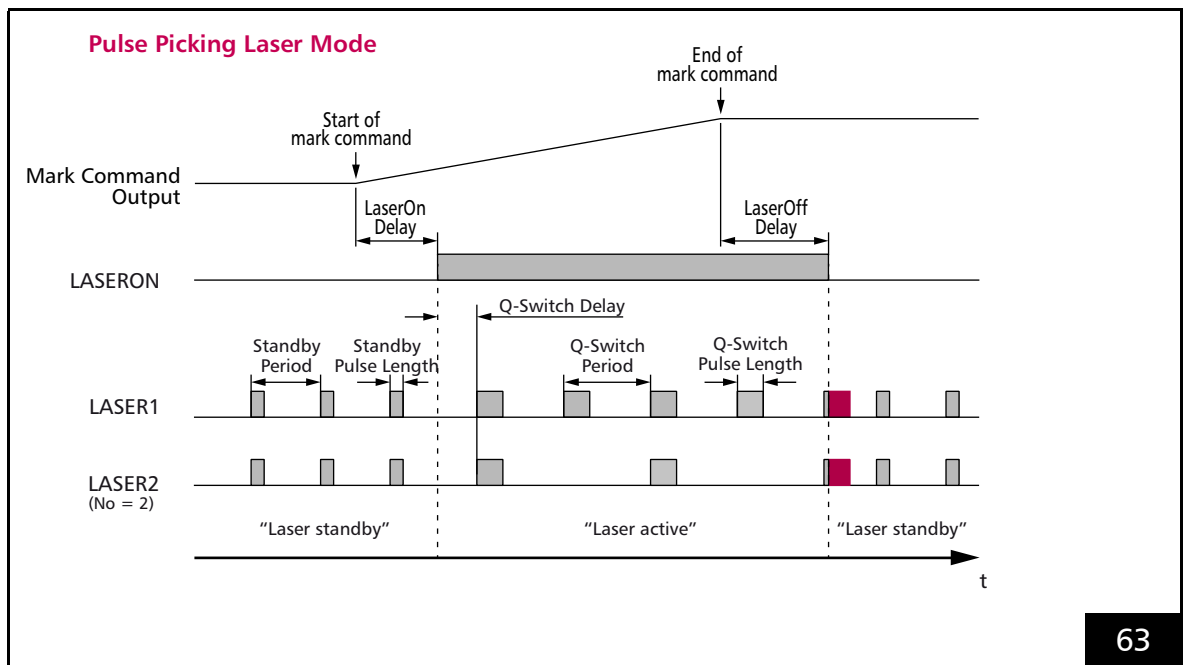
- The LASERON signal is switched on
- A modulation signal is provided at the LASER1 output with pulse length and period duration that can be defined by **set_laser_pulses**, **set_laser_pulses_ctrl** or **set_laser_timing**
- At the LASER2 output every N_o^{th} pulse of the signals is outputted at the LASER1 output (in phase). N_o is set with **set_pulse_picking** or **set_pulse_picking_list**

For "laser standby" operation:

- The LASERON signal is switched off
- Standby pulses are provided in phase at the LASER1 and LASER2 outputs with pulse lengths and periods that can be defined by **set_standby** or **set_standby_list**. Standby pulses cannot be "pulse-picked".

Notes

- With **set_laser_control**(Bit #7 = 1) the Pulse Picking signal at LASER2 can be set to a constant length independent of the LASER1 signal, which is set with **set_pulse_picking_length**.
- **set_pulse_picking** and **set_pulse_picking_list** overwrite a laser mode previously set with **set_laser_mode**. Vice versa, **set_laser_mode** switches off the **Pulse Picking Laser Mode**.
- For the LASER1 signals, half the period duration must be specified.
- A Q-Switch delay is effective, see also **Section "Differences Between the YAG Modes"**, **Page 180**.
- A **FirstPulseKiller** signal is not outputted.
- The setting sequence of the LASER1 signals and the **Pulse Picking Laser Mode** is irrelevant.



Timing diagram of the signals in **Pulse Picking Laser Mode** (with active-HIGH laser control signals and $N_o = 2$).
Example: isolated mark command.

7.4.9 “Automatic Laser Control”

By **set_auto_laser_control**, automatic readjustment of **Signals for “Laser Active” Operation** – even dynamically during execution of **Mark commands** can be achieved.

The **Ctrl** parameter determines the output port and the **Value** parameter the output value for 100% power at the target (set) mark speed at the **Image field** center (for the other parameters, see command description).

This can be used to compensate for variations in laser energy input caused by a changing power density, spot size or processing speed.

For “Automatic Laser Control”, a position-, speed- or vector-controlled correction of the laser control signals can be activated and combined.

- *Position-dependent* laser control, see [Section “Position-Dependent Laser Control”, Page 189](#), allows compensation of radial laser energy input variations during execution of **Mark commands**. Such variations may be caused, for example, by a objective edge diminution or different incidence angles of the laser beam.
- *Speed-dependent* laser control, see [Section “Speed-Dependent Laser Control”, Page 191](#), allows compensation of laser energy input variations during execution of **Mark commands** that resulting from an uneven galvanometer scanner motion (acceleration phase and deceleration phase, time-dependent **Mark commands**).
- *Encoder speed-dependent* laser control, see [Section “Encoder-Speed-Dependent Laser Control”, Page 192](#), allows the laser energy input to be controlled based on the currently present encoder speed.

- Speed-dependent laser control and encoder speed-dependent laser control can also be combined with each another, if galvanometer scanners and workpiece move simultaneously.
- *Vector-defined* laser control, see [Section “Vector-Defined Laser Control”, Page 194](#), allows linear readjustment of a signal parameter along parameterized mark vectors or jump vectors (see [Section “\[*\]Para\[*\] Commands”, Page 127](#)). This signal parameter can be combined with the laser power control if the control parameter **Ctrl** matches **Ctrl** from **set_auto_laser_control** (the current parameter **Para** dynamically replaces the 100% value from **set_auto_laser_control**) or as an independent control (for example, as defocus).

Selectively, “Automatic Laser Control” adjusts one of the following signal parameters:

- 12-bit output value at the **ANALOG OUT1** or **ANALOG OUT2** output port of the LASER connector, see also [Section “12-Bit Analog Output Port 1 and 2”, Page 61](#)
- Output value at the 16-bit digital output port of the EXTENSION 1 socket connector, see also [Section “16-Bit Digital Input Port and 16-Bit Digital Output Port”, Page 65](#)
- Output value at the 8-bit digital output port of the EXTENSION 2 socket connector, see also [Section “8-Bit Digital Output Port”, Page 68](#)
- Pulse length (**PulseLength**) or output period (**HalfPeriod**) of the laser control signals **LASER1** and **LASER2**

The automatically readjusted value can be recorded by **set_trigger/set_trigger4** (Signal $n = 24$).

Notes

- “Automatic Laser Control” does *not* compensate for an explicit change in mark speed between two **Mark commands** caused by a **set_mark_speed** call.
- “Automatic Laser Control” and **Pixel mode** should not affect the same output port at the same time.
- “Automatic Laser Control” cannot be combined with variable laser control via the **McBSP interface** (see **set_multi_mcbbsp_in**).

General Notes

- Each individual contribution as well as the total correction cannot exceed a factor of 4.0 (clipping⁽¹⁾).
- If laser power and/or energy input into the to-be-processed material is not strictly proportional to the output values of the selected signal parameter, then **load_auto_laser_control** can be used to load a nonlinearity curve that defines this (application-specific) relationship, see **Section “Loading and Determining the Nonlinearity Curve”, Page 192**.
- In addition, the selected signal parameter can neither exceed the value range allowed with **set_auto_laser_control** (parameter `MinValue` and `MaxValue`) nor the value range allowed for the respective output port. It is clipped correspondingly.
- For the laser control signal a corresponding default value is outputted (see **set_port_default** or **set_laser_off_default**), if:
 - **Signals for “Laser Active” Operation** are switched off when “Automatic Laser Control” is active
 - “Automatic Laser Control” itself is deactivated. If no default value has been explicitly defined, the permitted maximum value is outputted. As substitute for the control parameters `HalfPeriod` and `PulseLength`, the parameter `Value` (the 100% value) from **set_auto_laser_control** is used.
- Once an “Automatic Laser Control” has been switched on with **set_auto_laser_control**, then the following can be changed subsequently by **set_auto_laser_params** or **set_auto_laser_params_list**:
 - the signal parameter
 - the 100% value
 - limit values assigned to it
 The `Mode` parameter cannot be changed subsequently.

(1) < DLL 546: overflow.

Position-Dependent Laser Control

To activate the position-dependent laser control for the output port specified by **set_auto_laser_control**, a user-defined scaling function must be loaded by **load_position_control**, see Section “Notes on Loading a Scaling Function”, Page 189.

This scaling function represents a scaling factor as a function of the distance to the center of the **Image field**.

After **load_program_file**, it is initialized for all distances with “factor 1.0”. The “position-dependent” laser control is de facto deactivated by that.

When calculating the correction for “position-dependent laser control”, the current Cartesian control values are used as a basis:

- including wobble correction (#2 in Chapter 7.3.6 “Output Values to the Scan System”, Page 170)
- including coordinate transformation in the virtual **Image field** (#3)
- including Processing-on-the-fly correction (#4)
- excluding head-specific coordinate transformation (#5a and #5b)
- excluding **Image field** correction (#7)

Notes on Loading a Scaling Function

- For the *Scale(Position)* scaling function, **load_position_control** loads a table from an ASCII text file.
- The ASCII text file can contain one or several tables.⁽¹⁾
- Each table can contain up to 50 data points (*Position* | *Scale(Position)*).
- The *Scale(Position)* function is linearly interpolated from the data points.

For the scaling function tables, the following rules apply:

- Each table must begin with the line:
`[PositionCtrlTable<No>]`
 <No> represents the table number.
- If the table contains multiple `[PositionCtrlTable<No>]` entries with the same <No>, then only the lines after the first entry are used. Only lines up to the next '[' character (that is not preceded by a semicolon) are used.
- Each data point (*Position* | *Scale(Position)*) is defined as follows:
`Position<n> = <Value>`
`Scale<n> = <Value>`
 where <n> corresponds to the index ($1 \leq \text{<n>} \leq 50$) of the data point. The values <Value> can be specified as (unsigned) floating point numbers. Decimal separator: period (.).
- If the table contains multiple data points with the same Index <n>, then the most recently read one is used.
- If the table contains multiple data points with the same position value *Position*, then the data point with the largest Index <n> is used. Equality is checked to within ± 0.01 .
- The position value is specified radially as the distance between the to-be-marked point and the coordinate midpoint ($= (x^2 + y^2)^{1/2}$) as percent of half the image-field side length.
 Example: ($X_{\max}|0$) corresponds to 100%,
 ($X_{\max}|Y_{\max}$) corresponds to $2^{1/2} \times 100\%$.

(1) Even of another type, see table 1, page 141.

- For `<Value>`, the following ranges apply:
 $0.0 \leq \text{Position} \leq 150.0$ and
 $0.0 \leq \text{Scale}(\text{Position}) \leq 4.0$.
- Each instruction must be in a separate line.
- Spaces and tabs in a line (for example, between '=' and `<Value>`) are ignored.
- Empty lines are ignored.
- Data points with invalid values are ignored.
- The data point of a particular index `<n>` is ignored if the corresponding `Position<n>` and/or `Scale<n>` definition is missing.
- The semicolon ';' can be used for comments. All characters in a line following a semicolon are ignored.
- The instructions for data points in the table can be ordered as desired.
- Indices for data point pairs in the table can be selected as desired within the range [1...50] (the table is then automatically sorted by ascending position values).
- If the table contains *no* valid data point, **load_position_control** has no effect (return value 1 or 13).
- If there is no entry for `Position = 0.0`, then an entry with `Scale = Min(Scale<i>)` is inserted (the smallest allowed value defined in the table is used for lower positions values). Likewise for `Position = 150.0` with `Max(Scale<i>)`.
- If the selected text file only contains a single valid data point with `Scale<n> = S`, then (for the entire position range) the scaling function `Scale(Position) = S` is loaded. The correction has a multiplicative effect on the laser control signal. For `S = 1.0`, position-dependent correction is therefore switched off. Alternatively, this can also be achieved with `Name = NULL` in **load_position_control**.

Speed-Dependent Laser Control

When activating the “speed-dependent laser control” for the output port specified by **set_auto_laser_control**, the **Mode** parameter is used to select which input parameters are to be used for calculating the correction.

As reference value for the 100% speed, always the set (target) mark speed (set by **set_mark_speed** or **set_mark_speed_ctrl**) applies (its change is never compensated by the “Automatic Laser Control”).

- **Mode** = 1 is intended (for consistency reasons) especially for analog scan systems. The current **Microstep** length per 10 μ s is used as input. It may deviate from the target speed due to the 10 μ s clock pulse rounding or with **[*]timed[*]** commands. Variations in the acceleration and deceleration phases cannot be compensated.
- **Mode** = 2 is intended as basic mode for **iDRIVE** scan systems⁽¹⁾. It can be combined with other special corrections (see below).
- Extensions to **Mode** = 2 (any combination possible):
 - +0
Basic mode for **intelliSCAN** systems
 - +4
Combines the speeds from **Mode** = 2 and **Mode** = 5 to a total speed. The 100% reference speed from **Mode** = 5 remains unconsidered. Instead, the encoder speeds are converted into galvanometer scanner bit speeds with the fly scaling factors and then vectorially added to the galvanometer scanner speed. A corresponding **Processing-on-the-fly session** must be active.

Notes

- Usually **set_auto_laser_control**(**Mode** = 2) requires a special **intelliSCAN** firmware, which returns an actual speed corrected by the signal runtimes between the RTC5 PCI Board and the scan head. If you have any questions as to whether your **intelliSCAN** is equipped with it or is upgradeable, contact SCANLAB.
- Usually a combination of “speed-dependent laser control” and a negative **LaserOn Delay** does not make sense.

(1) See Glossary entry on [page 23](#).

Encoder-Speed-Dependent Laser Control

set_auto_laser_control(*Mode* = 5) is intended for a pure encoder speed-dependent correction, if only the workpiece moves and the galvanometer scanners (ideally) are idle.

The 100% reference speed is defined by **set_encoder_speed_ctrl** or **set_encoder_speed**. Processing-on-the-fly should not be active at the same time.

For a combination of galvanometer scanner speeds and encoder speeds in a **Processing-on-the-fly session**, see *Mode* = 6 = 2 + 4.

Loading and Determining the Nonlinearity Curve

- For the *Scale(Percent)* nonlinearity curve, **load_auto_laser_control** loads a table from an ASCII text file.
- The ASCII text file can contain one or several tables.⁽¹⁾
- Each table can contain up to 50 data points (*Percent* | *Scale(Percent)*).
- The *Scale(Percent)* function is linearly interpolated from the data points.

For the tables, the following rules apply:

- Each table must begin with the line:
[AutoLaserCtrlTable<No>]
<No> represents the table number.
- If the table contains several [AutoLaserCtrlTable<No>] entries with the same <No>, then only the lines after the first entry are used. Only lines up to the next '[' character (that is not preceded by a semicolon) are used.
- Each data point (*Percent* | *Scale(Percent)*) is defined as follows:
Percent<n> = <Value>
Scale<n> = <Value>
where <n> corresponds to the index ($1 \leq \text{<n>} \leq 50$) of the data point. The values <Value> can be specified as (unsigned) floating point numbers. Decimal separator: period (.).
- If the table contains multiple data points with the same Index <n>, then the most recently read one is used.
- If the table contains multiple data points with the same percent value *Percent*, then the data point with the largest Index <n> is used. Equality is checked to within $\pm 0.01^\circ$.

(1) Even of another type, see table 1, page 141.

- The percent value is relative to the 100% value from **set_auto_laser_control** (Parameter Value) or dynamically from a ""vector-controlled laser control"", see Section "Vector-Defined Laser Control", Page 194.

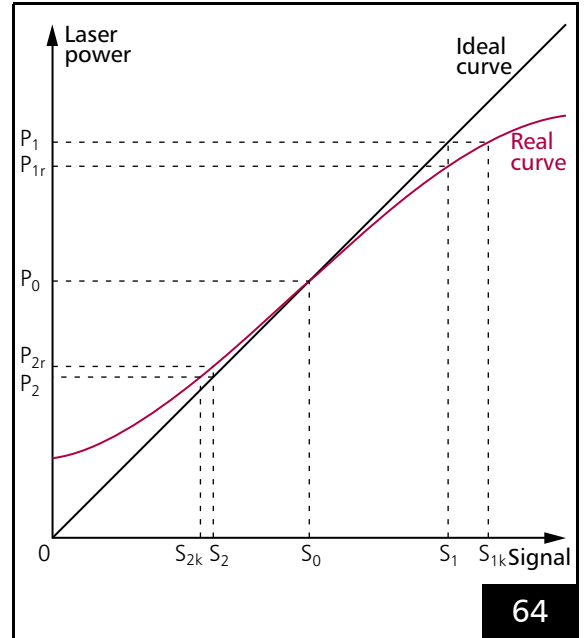
In the following example, a nonlinearity factor of 1.2 is set for a 1.5x multiple of the target value:

```
Percent<n> = 150
Scale<n> = 1.2
```

- For <Value>, the following ranges apply
 $0.0 \leq \text{Percent} \leq 400.0$ and
 $0.0 \leq \text{Scale}(\text{Percent}) \leq 4.0$.
- Each instruction must be in a separate line.
- Spaces and tabs in a line (for example, between '=' and <Value>) are ignored.
- Empty lines are ignored.
- Data points with invalid values are ignored.
- The data point of a particular index <n> is ignored if the corresponding Percent<n> and/or Scale<n> definition is missing.
- The semicolon ';' can be used for comments. All characters in a line following a semicolon are ignored.
- The instructions for data points in the table can be ordered as desired.
- Indices for data point pairs in the table can be selected as desired within the range [1...50] (the table is then automatically sorted by ascending percent values).
- If the table contains no valid data point, then **load_auto_laser_control** has no effect (return value 1 or 13).
- If there is no entry for Percent = 0.0, then an entry with Scale = Min(Scale<i>) is inserted (the smallest allowed value defined in the table is used for lower percent values). Likewise for Percent = 400.0 with Max(Scale<i>).

After **load_program_file** this function is initialized for all percentage values with "Factor 1.0", the nonlinear laser control is "deactivated". Alternatively, this can also be achieved with Name = NULL in **load_auto_laser_control**.

The example diagram in Figure 64 illustrates how the nonlinearity curve can be determined.



Laser power progression – example of determining a nonlinearity curve.

The straight line in the diagram describes an ideal relationship between laser power and the laser control signal parameter (here, the term laser power also represents the pulse frequency = $0.5/\text{LaserHalfPeriod}$), the curved line simulates a realistic relationship.

S_0 is the signal parameter value defined as the target value and P_0 is the associated laser power. At point $(S_0 | P_0)$ (this corresponds to the data point Percent0 = 100, Scale0 = 1.0) the two curves are normalized to each other. The combination of a nonlinearity curve with a ""vector-controlled laser control"" is therefore generally not recommended.

An increase (decrease) of the signal parameter to S_1 (S_2) results in an ideal laser power P_1 (P_2) and a real laser power P_{1r} (P_{2r}). For the actually desired laser power P_1 (P_2), a corrective signal parameter value S_{1k} (S_{2k}) is needed. The following two value pairs are then to be entered as data points for the nonlinearity curve:

$$\begin{aligned} \text{Percent1} &= S_1/S_0 \times 100 = P_1/P_0 \times 100 \\ \text{Scale1} &= S_{1k}/S_1 \end{aligned}$$

$$\begin{aligned} \text{Percent2} &= S_2/S_0 \times 100 = P_2/P_0 \times 100 \\ \text{Scale2} &= S_{2k}/S_2 \end{aligned}$$

Vector-Defined Laser Control

The “vector-controlled laser control” allows a signal parameter to be varied linearly along a mark vector or jump vector.

To initialize the “vector-controlled laser control”, **set_vector_control** must be used to specify which signal parameter is to be varied with which initial value (parameters **Ctrl** and **Value**).

Then the signal parameter is varied linearly along a parameterized mark or jump vector. The end value is automatically the start value for a subsequent **[*]para[*] Command**.

Notes

- List commands for explicitly changing the signal parameter output value are temporarily effective or not at all.
- [*]para[*] Commands** always use the end value of the previous **[*]para[*] Command** as their start value.
Control commands that write to the same output port should be avoided while processing a list of **[*]para[*] Commands**.
- If the same output port is selected via **set_auto_laser_control** for **Ctrl**, the control parameter from the **[*]para[*] Commands** acts as 100% value for the laser control.
Special care should be taken with **set_vector_control** (**Ctrl** = 7) (Defocus): The setting of the signal value is always done immediately. This can lead to **Hard jumps** on the varioSCAN.
- During execution of **[*]para[*] Commands**, the **Sky Writing** mode, see **Chapter 7.2.4 “Sky Writing”, Page 149**, is *not* taken into account (but also not deactivated).

7.4.10 Output Synchronization

The RTC5 provides the so-called (galvanometer scanner) “output synchronization” functionality for synchronizing the scan system’s scanning motions (during all marking commands) to the laser pulses of a free-running laser.

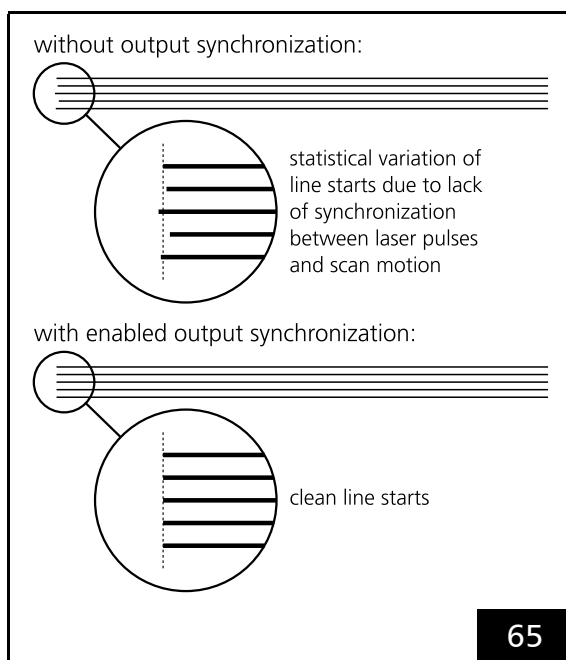
The laser pulse signal (the synchronization master clock) must be supplied at the LASER connector’s DIGITAL IN1 digital input (see [Section “2-Bit Digital Input Port”, Page 61](#)).

Output synchronization is enabled or disabled by Bit #6 of [set_laser_control](#).

When enabled, output synchronization affects all [Mark command](#) marking commands (mark, arc and ellipse). At each marking command’s start, the RTC5 determines the time shift between that marking command’s starting time and the first laser pulse after LASERON⁽¹⁾ and appropriately corrects the scan system output values. The RTC5 thus ensures that the position of the galvanometer scanners upon the first laser pulse output after LASERON does not depend on random phase shifts of the laser pulse. This way, jittery line images can be avoided, see [Figure 65](#).

Notes

- The supplied laser pulse signal must have a frequency between 50 kHz and 6.4 MHz and its pulse length must exceed 0.080 μ s.
- The output period of the laser control signals must be set according to the laser pulse frequency in the user program (before enabling output synchronization) by [set_laser_pulses](#), [set_laser_pulses_ctrl](#) or [set_laser_timing](#). This also applies, if [Laser Mode 4](#) standby signals are used as external input.
- For frequencies between 50 kHz and 100 kHz, the [LaserOn Delay](#) must be set to 50 μ s, otherwise the [LaserOn Delay](#) must exceed 40 μ s.
- If no laser pulse appears at the DIGITAL IN1 digital input within 20 μ s of a marking command’s start, then no output synchronization is performed on that command’s output.
- Output synchronization can also be used in conjunction with [Sky Writing](#).
- Output synchronization cannot be used together with [Pixel mode](#).



Example of line image marked with a free-running laser.

(1) the first laser pulse output following the [LaserOn Delay](#)

7.5 Marking Dates, Times and Serial Numbers

The **RTC5 DLL** contains a series of commands to mark the current time, current date or the serial number of products.

Before times, dates and serial numbers can be marked, the required characters and text strings must be defined as indexed characters and indexed text strings.

Separate text strings can be defined for marking times/dates and serial numbers. See [Section "Defining Indexed Text Strings for Times, Dates and Serial Numbers"](#), Page 108.

7.5.1 Marking the Date and Time

By **time_update** the PC time/date is transferred to the RTC5 PCI Board. This is necessary after every PC restart. After that, the board (as long as it remains energized) internally counts the date/time with the quartz-controlled 10 μ s clock to the second.

By **time_fix**, **time_fix_f** or **time_fix_f_off** the current time/date of the board is hold in a cache.

The time (hours, minutes, seconds) can be marked by **mark_time** or **mark_time_abs** and the date (year, month, day, day-of-the-week) by **mark_date** or **mark_date_abs**.

To mark the date and time, the Gregorian or Julian date can be set as well as the 12- or 24-hour format.

For marking dates of expiry, **time_fix_f_off** is available to fix a forward date based on the current date and current time.

7.5.2 Marking Serial Numbers

By **mark_serial** and **mark_serial_abs**, up to 12-digit serial numbers can be marked. It can be specified how leading zeros are handled.

The RTC5 PCI Board manages up to 4 serial-number-sets (each with its own serial number and increment size). Serial-number-set 0 is selected at initialization with **load_program_file**.

Other serial-number-sets must be selected in advance by **select_serial_set** or **select_serial_set_list** (see notes below).

By **set_serial**, **set_serial_step** or **set_serial_step_list**, a starting serial number (max. 10 digits) and an increment size for each serial-number-set can be specified. At initialization with **load_program_file** all starting serial numbers are set to 0 and all increment sizes are set to 1.

Each call of **mark_serial** or **mark_serial_abs** causes the current serial number to be incremented (yet before execution of the serial number marking) by the specified increment size.

If a serial number is to be omitted a blank marking can be executed (**Digits** = 0), which increments the serial number by 1 (*not* by the specified increment size).

Notes

- If a serial-number-set is to be marked by **mark_serial** or **mark_serial_abs**, then you can only select that set by the list command **select_serial_set_list**. **mark_serial**, **mark_serial_abs** and **set_serial_step_list** are always applied to the serial-number-set most recently selected by **select_serial_set_list**.
- You can use the control command **get_list_serial** to query the number of the serial-number-set most recently selected by **select_serial_set_list** as well as the current serial number of that set. This also lets you determine (among other things) whether the current number has been or has not been incremented after an uncontinued aborted list.

- The control command **select_serial_set** lets you select (independently of selection by **select_serial_set_list**) a serial-number-set for the control commands **set_serial_step** and **set_serial** (Note that the RTC5 PCI Board does not prohibit modifying parameters of the serial-number-set currently being marked). The control commands **set_serial_step** and **set_serial** always apply to the serial-number-set most recently selected by **select_serial_set**.
- **get_serial** returns the current serial number of the serial-number-set selected by **select_serial_set** (if multiple serial-number-sets exist, then the returned serial number is not necessarily the most recently marked serial number).
- **set_max_counts** allows specification of the maximum number of **External Starts** and thus the maximum number of externally started markings. The number of already occurred **External Starts** can be obtained with **get_counts**. When a single serial-number-set is used, these commands let you indirectly set the maximum serial number and query the current serial number. But when multiple serial-number-sets are used, these commands do not differentiate between the various serial-number-sets.

8 Advanced Functions for Scan Head Control and Laser Control

8.1 iDRIVE Functions

SCANLAB iDRIVE scan systems⁽¹⁾ utilize the iDRIVE technology. This servo and control approach exploits the advantages of fully digital servo electronics to deliver significantly expanded functionality. An enhanced transfer protocol between the servo electronics and the controller facilitates support of all the new features (see also the following section).

This allows users to adjust a number of settings of the respective scan system, for example,

- To select which data it has to transmit to the controller
- To choose from different dynamic settings (tunings)
- To set the PosAck limit value
- To set the effective calibration of the scan system
- To set the start behavior of the scan system
- To perform a fault diagnosis
- To perform a functional test of the data transfer

For more information, refer to the manual of the scan system.

iDRIVE functions are executed by **control_command**⁽²⁾.

During the **control_command** execution, the RTC5 PCI Board does not transfer position data to the scan system. Due to this, the motion of the galvanometer scanners is briefly interrupted.

8.1.1 Transfer Protocol

Data transfer between RTC5 PCI Board and scan system is carried out according SL2-100 protocol, which supports the full functionality of iDRIVE technology.

With iDRIVE scan systems⁽¹⁾ this protocol allows, for example, status signals of the x axis and y axis to be separately and simultaneously evaluated. For a 3D scan system, z axis status signals can be simultaneously read back by the channel of the scan head connector to which the z axis is connected.

The **XY2-100 Converter (Accessory)** can be used with iDRIVE scan systems⁽¹⁾ equipped with a XY2-100 interface or XY2-100 Enhanced interface.

(1) See Glossary entry on [page 23](#).

(2) See also [Section "Folder iSCANConfig", Page 28](#).

8.1.2 Configuring the Data Signal Transmission Behavior of the Scan System

Setting Data Types

The digital servo architecture of iDRIVE scan systems⁽¹⁾ allows a wide variety of data signals to be returned from the scan system to the RTC-control board.

Each axis has its own status channel on which data is transmitted to the RTC-control board every 10 μ s:

- STATUS channel
 - Designed for the x axis
(Galvanometer scanner 2)
- STATUS1 channel
 - Designed for the y axis
(Galvanometer scanner 1)

This opens up possibilities such as monitoring the actual values of the galvanometer scanners during an application or carrying out comprehensive troubleshooting in case of operational malfunction.

control_command (Data = 05nn_H) allows to set which data the scan system has to transmit to the RTC5 PCI Board. The available data types are described in detail in the manual of the respective scan system and (in parts) in the command reference of the **control_command**. The set data source is transferred until another data source is set.

After every power-up or reset (after the initialization has been completed), the scan system transmits (on all receive channels) the XY2-100 status word.

Reading Out Data

At any time, data received by the RTC5 PCI Board can be:

- Read out asynchronously by **get_value**, **get_values** or **get_head_status**
- Synchronously recorded by **set_trigger/set_trigger4**

See also Chapter 7.3.7 "Status Monitoring and Diagnostics", Page 172.

Note that switching of the data source is followed by a short (serial transmission-related) delay of typically 50 μ s before the first data is transmitted, see also comment at **control_command**, page 788.

The value ranges of these data and the possible status states are described in Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747.

Notes

- **get_head_status** queries the XY2-100 status word. With SL2-100 protocol-compliant data transfer, the scan system always transfers the XY2-100 status word in parallel with other status information.
Important: If the XY2-100 Converter (Accessory) is used for control, then it must explicitly be set that the scan system has to transfer the XY2-100 status word to the RTC5 PCI Board. Otherwise, **get_head_status** returns unusable values.

(1) See Glossary entry on page 23.

8.1.3 Monitoring the Positioning

For some applications, it is important to monitor and, if necessary, to document the scan system positioning even during operation.

For this purpose, the actual position of the scan axes must be set to be returned from the scan system by **control_command**. The returned actual position values can be queried then by **get_values** or recorded by **set_trigger/set_trigger4** ⁽¹⁾.

If the returned actual positions of the scan axes are to be compared with the Cartesian target coordinate values (x, y, z), then these must be transformed back by the corrections made on the RTC5 PCI Board, **Chapter 7.3.6 "Output Values to the Scan System", Page 170**.

For runtime reasons, the backward transformation needs to subsequently be performed by the PC rather than on the RTC5 PCI Board itself. For this, all correction and transformation settings currently assigned to the scan system can be transferred from the RTC5 PCI Board to the PC by **upload_transform**. Afterwards, an individual xy value pair or an individual z value can be backward transformed by **transform**. With **get_transform** (see also **get_waveform**), an entire series of xy value pairs or z values previously recorded by **set_trigger/set_trigger4** can be backward transformed.

Notes

- For some forward transformations, a backward transformation is not possible:

Forward transformation	Backward transformation possible?
Wobbel motion (#2 on page 170)	no
Global coordinate transformation (#3 on page 170)	no
Processing-on-the-fly correction (#4 on page 170)	no
Coordinate transformations (total matrix and offset) to the x and y coordinates (#5a and #5b on page 170)	yes
Offset to the z coordinate (#5b on page 170)	yes
Clipping to the limits of the controllable Image field (#6 on page 170)	no
2D Image field correction (#7 on page 170)	yes
3D image field correction (#9 on page 170)	yes
Gain and offset correction of automatic self-calibration (#11 on page 170)	yes
Clipping to the maximum possible control values	no

- Furthermore, backward transformation is not possible if a noninvertible transformation matrix has been defined during forward transformation (notice: clipping to ± 50 for each individual matrix coefficient; see **Chapter 8.2 "Coordinate Transformations", Page 209**). The non-invertability is already reported by **upload_transform**.

(1) Due to communication runtimes, the currently returned actual positions are several clock pulses later than the currently outputted control signals.

- If forward transformation included a clipping to the edges of the positionable **Image field** or to the edges of the maximum possible range of control values, then backward transformation (ideally) calculates the Cartesian coordinates of these edge values instead of the original values.
- By **get_values**, 4 arbitrary signals can be queried at the same time. Example:
 - the actual position of **Galvanometer scanner 2** by StatusAX
 - actual position of **Galvanometer scanner 1** by StatusAY
 - the actual z axis position by StatusBX
 - an additional desired signal, for example, LASERON

In contrast, **get_value** (not: **get_values**) is not useful for monitoring xy positioning because it is only meant for querying a single signal and multiple calls unavoidably lead to xyz values across different points of time.
- By **set_trigger**, you can simultaneously record two desired signals by two measurement channels (with **set_trigger4** 4 signals by 4 measurement channels).
- By **transform** and **get_transform**, you can use the parameter `Code` to specify that values queried by **get_values** or **set_trigger** are to be assigned to x, y or z for backward transformation.
- **transform** and **get_transform** also allow specification of which partial transformations are to be performed.
- Values queried by **get_values**, or arbitrary synthetic values can be backward transformed by **transform**. During backward transformation of synthetic values beyond the forward transformation's achievable **Image field**, values sometimes are only calculated by extrapolation, due to possible range exceedances or other errors.
- If the user program binarily stores both the recorded values and the transferred transformation data (see **get_waveform**), then subsequent backward transformation by **transform** (not **get_transform**) can also be executed offline, hence without needing to further access a RTC5 PCI Board.
- If **control_command** is used to specify positioning error rather than actual position as the to-be-returned data type by the scan system, then it is not possible to directly compare the originally defined pattern with the marked pattern. But you can check if the scan system correctly processed the RTC5 PCI Board output values. This is particularly useful if backward transformation of actual values is not (fully) possible or when it cannot be determined if deviations between backward-transformed actual positions and originally defined coordinate values are due to scan system error or clipping during forward transformation.

8.1.4 Configuring Tuning (Dynamics Settings)

SCANLAB can optimize the dynamics setting of scan systems (tuning) to accommodate differing requirements of diverse applications regarding the laser positioning dynamics, for example,

- Vector tuning
to execute vectors or circular arcs at a constant processing speed
- Jump tuning
to execute jumps of minimized duration

iDRIVE scan systems⁽¹⁾ can be optionally equipped with several tunings. For different applications, the suitable tuning can be switched to – separately for each axis – by **control_command**(Data = 11nn_H).

For scan systems equipped with one or several **Jump tunings**, you can also activate **Jump Mode** (and hereby tuning autoswitching) for 2D jumps, see [Chapter 8.1.5 "Jump Mode", Page 202](#).

The default set start behavior is that the scan system starts with tuning number 0 upon power-up or after a reset.

8.1.5 Jump Mode

For applications such as drilling holes with defined spacing (whereby laser processing is actually point-by-point rather than along lines and curves), you can optimize process times by activating the so-called **"Jump Mode"**.

This requires the scan system to be equipped with a **Jump tuning**, see also [Section "Requirements and Activation", Page 203](#).

Functional Principle

In the default setting (after **load_program_file**), both **Jump commands** and **Mark commands** are executed in vector mode:

- The jump length gets subdivided into individually executable **Microsteps** in accordance with the current jump speed. If the scan system is only equipped with a **Jump tuning**, then the **Microsteps** execute using this tuning.
- A **Jump Delay** defined by **set_scanner_delays** is executed before a subsequent list command.

In contrast, when **Jump Mode** is enabled and activated by **set_jump_mode** or **set_jump_mode_list**, every 2D jump (see below) is executed as follows:

- The entire jump length of the 2D jump is controlled as a **"Hard jump"** over a time dimensioned jump of 10 μ duration. The target position is executed without **Microstepping**.
- The jump executes with a **Jump tuning**. **set_jump_mode** can be used to designate which **Jump tuning** to use. If a different tuning has been set before the jump, then the RTC5 PCI Board automatically switches at the beginning of the jump to the tuning specified by **set_jump_mode**.
- At the end of the 2D jump, the RTC5 PCI Board automatically switches to a **Vector tuning** (if the scan system is equipped with one and if a corresponding setting has been made by **set_jump_mode**).

(1) See Glossary entry on [page 23](#).

- At the end of the 2D jump, a jump-length-dependent **Jump Delay** occurs. This **Jump Delay** can be specified for the corresponding jump length by **load_jump_table_offset** or **set_jump_table**, see also Section "Jump-Length-Dependent Jump Delays", Page 204. Here, an external **Jump Delay** specified by **set_scanner_delays** is *not* taken into account.

Notes

- **Jump Mode** works exclusively on
 - **jump_abs**, **jump_rel**, **goto_xy** (not on the corresponding 3D, para or timed commands)
 - home jumps and home returns (see **home_position**)
- If a 2D jump occurs where the jump length limit (**Length** parameter) specified by **set_jump_mode** is *not* reached or exceeded on at least one of the two axes, then the jump executes in vector mode even if **Jump Mode** has been enabled and activated. This allows exploitation of the fact that *short* jumps can in some circumstances execute faster by **Vector tuning** than with **Jump tuning**. But if no **Vector tuning** is installed or none specified, then you should set the **Length** parameter to 0.
- Each switch between different tunings (servos) requires an additional 10 μ s clock cycle. For applications such as pure drilling, this can be avoided by not specifying a **Vector tuning** to switch back to when you call **set_jump_mode**.
- When you deactivate or disable **Jump Mode** (by **set_jump_mode** or **set_jump_mode_list**), then subsequent jumps again execute in vector mode (split-up into **Microsteps** and without further servo autoswitching). Here, the **Vector tuning** is used that has been most recently set at the end of **Jump Mode**, unless deactivation has been followed by selection of a different tuning by **control_command**. Moreover, the most recently set jump speed is again used and jumps are followed by the **Jump Delay** specified by **set_scanner_delays**.

Requirements and Activation

The following are required for enabling and activating **Jump Mode**:

- At least one of the two scan head connectors must have been assigned a correction table.
- At least one of the two scan head connectors must be connected to an intelliSCAN, intellcube, intelliWELD or intelliDRILL scan system.
- As of scan system firmware ≥ 2078 .
- The attached scan system must be equipped with at least one **Jump tuning**. In contrast, a **Vector tuning** is not absolutely required.
- The tuning numbers specified by **set_jump_mode** must match those stored on the board.
- The tunings specified by **set_jump_mode** must be of the proper type – **Vector tuning** or **Jump tuning** – (the **Tuning type** is stored in the scan system firmware) and must be suitable for rapid switching.

Before **Jump Mode** can be activated by **set_jump_mode_list**, it must have been successfully enabled at least once by **set_jump_mode** (see command description).

The **set_jump_mode** control command (but not the **set_jump_mode_list** list command) performs an appropriate check if **Jump Mode** has not been already enabled.

Jump-Length-Dependent **Jump Delays**

When executing a “**Hard jump**”, it takes the scan head some time to reach the specified position. The RTC5 PCI Board takes this delay (also called step response) into account by appending a **Jump Delay** at the end of the jump.

Point-by-point laser processing does not need to take other **Scanner Delays** into account and you can generally set **Laser Delays** to 0.

The specific step response behavior of the respective scan system (step response time vs. jump length) can be stored on the RTC5 PCI Board in a user-specific **Jump Delay** table. With **Jump Mode** enabled, the RTC5 PCI Board uses the specified **Jump Delay** table to determine the appropriate **Jump Delay** value for each 2D jump in accordance with the jump’s longer edge (that is, either the x or y component of the jump).

You can determine the step response behavior experimentally and then load it onto the board as a table of values using **load_jump_table_offset**. Alternatively, the **Jump Delay** table can also be automatically determined by **load_jump_table_offset** (parameter `Name = NULL`). Additionally, the currently loaded **Jump Delay** table can be retrieved as a binary table by **get_jump_table** and reloaded onto the board by **set_jump_table**.

The step response time (at least for longer jumps) typically scales with the squareroot of the jump length, and **load_program_file** accordingly initializes the internal jump table – with an end value of 10.24 ms for a jump length of 2^{20} bits.

Experimental Determination of **Jump Delay** Values

The user manual of the scan system typically specifies the step response times for each **Jump tuning** at selected jump lengths.

To experimentally determine the step response behavior, you need to have the scan system perform jumps of various lengths and query the resulting position values by the status channel for analysis.

After you activate **Jump Mode**, perform the jumps by using **jump_abs** or **jump_rel**. The scan system should have been previously set to return the actual-position data type by **control_command**. You can then record the latter by **set_trigger/set_trigger4** and retrieve it by **get_waveform**.

The determined **Jump Delay** values must be supplied in an ASCII text file. If the step response behaviors of both axes differ, then the higher of the two axes’ **Jump Delay** values should be supplied in the ASCII text file.

Notes on Loading Determined **Jump Delay** Values

- For jump length values and **Jump Delay** values, **load_jump_table_offset** loads a table from an ASCII text file.
- The ASCII text file can contain one or several tables.⁽¹⁾
- Each table can contain up to 50 data points (*Length* | *Delay(Length)*).
- The complete (internal) **Jump Delay** table *Delay(Length)* is linearly interpolated from the data points.

For the tables, the following rules apply:

- Each table must begin with the line:
[JumpTable<No>]
<No> represents the table number.
- If the table contains multiple [JumpTable<No>] entries with the same <No>, then only the lines after the first entry are used. Only lines up to the next '[' character (that is not preceded by a semicolon) are used.
- Each data point (*Length* | *Delay(Length)*) is defined as follows:
Length<n> = <LengthValue>
Delay<n> = <DelayValue>
where <n> is the data point index ($1 \leq \text{<n>} \leq 50$).
The <Value> numbers can be supplied as (unsigned) floating point numbers. Decimal separator: period (.).
- If the table contains multiple data points with the same index <n>, then the most recently read one is used and the previous ones ignored.
- If the table contains multiple data points with the same jump length value *Length*, then the data point with the largest index <n> is used and the others ignored. Equality is checked to within ± 0.01 .
- For <Value> the following ranges apply:
 $0.0 \leq \text{Length} \leq 1048576.0$
 $0.0 \leq \text{Delay(Length)} \leq 65535.0$
Delay values are supplied in units of $10 \mu\text{s}$, jump lengths in bits.
- Each instruction must be in a separate line.
- Space characters and tabs within a line (for example, between '=' and <Value>) are ignored.
- Empty lines are ignored.
- Data points with invalid values are ignored.
- The data point of a particular index <n> is ignored if the corresponding *Length*<n> and/or *Delay*<n> definition is missing.
- The semicolon ';' can be used for comments. All characters in a line following a semicolon are ignored.
- The instructions for data points in the table can be ordered as desired.
- Indices for data point pairs in the table can be selected as desired within the range [1...50] (the table is then automatically sorted by ascending position values).
- If the table contains no valid data points, then **load_jump_table_offset** has no effect (return value 1 or 13).
- If there is no entry of *Length* = 0.0, then one with *Delay* = Min(*Delay*<i>) is inserted (the smallest valid value encountered is filled downward). The same applies to *Length* = 524288.0 with Max(*Delay*<i>).
- If the specified text file contains only one valid data point with *Delay*<n> = D, then the **Jump Delay** table *Delay(Length)* = D (for the whole jump length range) is loaded.

(1) Even of another type, see table 1, page 141.

Automatic Determination of the **Jump Delay** Table

You can initiate automatic determination of the **Jump Delay** table by **load_jump_table_offset** (parameter `Name = NULL`) if you had previously enabled and activated **Jump Mode** successfully by **set_jump_mode** (`Flag = 1`).

For automatic determination, "Automatic Laser Control" is deactivated, the data type to be returned by the scan system is set to target position and the tuning set to the **Jump tuning** that had been defined by **set_jump_mode** (the original settings are restored when **load_jump_table_offset** completes).

For automatic determination, several diagonal jumps of varying lengths are performed. For each jump, the target position returned by the scan system is recorded by **set_trigger** (not: **set_trigger4**) and retrieved by **get_waveform**. The data is analyzed for the point in time at which the specified position tolerance (`PosAck` parameter) has been last exceeded, that is, when the target position persistently remained in the jump target positional range $\pm \text{PosAck}$. This value is then reserved as the **Jump Delay** value associated with the corresponding jump length.

Notes

- Before automatic determination, you must absolutely switch off the laser.
- Data recording requires execution of a list with a total of six commands. The commands are automatically written to list memory and processed. The parameter `ListPos` indicates the position in list memory ("list 1" or "list 2") in which storage is to occur. This position should be such that any previously entered list commands can be harmlessly overwritten.

- For automatic determination, the longest jump is performed first, followed by increasingly shorter jumps (with a maximum of up to 16 different jump lengths). For the first (longest) jump length (jump across the entire image diagonal), the measurement period is specified by the parameter `MaxDelay` [10 μs].

`MaxDelay` should be chosen to be adequate but not significantly larger than the **Jump Delay** for the longest jump. A larger `MaxDelay` increases the total required execution time. With

`MaxDelay = 500`, the total execution time for automatic determination is typically a few seconds.

- For statistical noise reduction, 4 identical jumps are performed for each jump length and the results averaged. Additionally, the values for each individual measurement are low-pass filtered (2-point smoothing). This permits selection of a position tolerance `PosAck` that can also be somewhat (but not substantially) under the expected noise level (but note that only whole multiples of 16 are returned with an **XY2-100 Converter (Accessory)**. Here, a noise level of $\pm 3 \times 16$ bits can be expected).
- If the `PosAck` range is not persistently reached within the measurement period, then `MaxDelay` becomes the determined **Jump Delay**.
- If the determined **Jump Delay** is smaller than `MinDelay`, then `MinDelay` becomes the determined **Jump Delay** and the measurement terminates. Then, shorter jumps are no longer performed and `MinDelay` is also the determined **Jump Delay** for these shorter jump lengths. A longer `MinDelay` reduces the total execution time for automatic determination.

- If an offset is specified for automatic determination (by the according **load_jump_table_offset** parameter), this offset is added to all automatically determined delay values before the overall **Jump Delay** table gets calculated by linear interpolation and loaded onto the board (in addition, the delay values are clipped to the value range 0...65,535). The Offset can be used to compensate for measurement runtime latencies (for example, caused by an **XY2-100 Converter (Accessory)**, by tuning switching or by a runtime latency of the signal returned by the scan system) when calculating the **Jump Delay** table. It can also be used to add a safety margin to the delay values to compensate for noise-induced random deviations. **load_jump_table** and **load_jump_table_offset** (**Offset** = 0) are identical.
- For simultaneous control of two scan systems, you should determine the **Jump Delay** values for both systems and, after comparing, use the values of the slower system.
- The resulting table can be retrieved in binary form by **get_jump_table** and reloaded onto the board by **set_jump_table**.

8.1.6 Configuring the **PosAck** Limit Value

control_command(**Data** = 15nn_H) can be used to set the **PosAck** limit value nn. The default start behavior is for the scan system to set the limit value to 0.28% of the full position range after every power-up.

If other limit values are desired, they must be separately set for each axis.

8.1.7 Configuring the Effective Calibration

The servo electronic can be configured by **control_command**(**Data** = 12nn_H) to down scale the position values received from the RTC5 PCI Board by a specific factor (1, 1/2, 1/4 or 1/8). The position signals (optionally) returned by the scan systems to the RTC5 PCI Board remain unaffected, as do the pre-configured calibrations of SCANLAB scan systems. However, the effective calibration can be thereby reduced to confine the scan area to a smaller angular range – with a higher angular resolution.

The default start behavior is for the scan system to start with a scaling factor of 1 upon power-up.

By **control_command**(**Data** = 053F_H), the currently set scaling factor can be queried.

8.1.8 Configuring the Start Behavior

The default configuration of iDRIVE scan systems⁽¹⁾ is set as follows:

- Tuning number 0, see also [Section "Configuring Tuning \(Dynamics Settings\)", Page 202](#)
- PosAck limit value is B8_H (corresponds to 0.28% of the full position range of 2¹⁶ bit), see also [Section "Configuring the PosAck Limit Value", Page 207](#)
- Scaling factor = 1, see also [Chapter 8.1.7 "Configuring the Effective Calibration", Page 207](#)

These settings can be changed by **control_command**. The changed settings are only temporary, however they can be additionally saved as starting settings for subsequent power-ups by **control_command** (Data = 0A00_H).

The transmission behavior of the scan system can only be temporarily changed, see also [Chapter 8.1.2 "Configuring the Data Signal Transmission Behavior of the Scan System", Page 199](#). After a power-up, the scan system transmits the XY2-100 status word.

8.1.9 Fault Diagnosis and Functional Test

If a problem occurs, the versatile status return functions of the iDRIVE scan system⁽²⁾ can be used for scan system diagnosis, too.

For example, an event code can be queried that indicates which event has been responsible for the change to an error state.

To verify that data transfer capability between the RTC5 PCI Board and a scan system is intact, by **control_command** (Data = 21nn_H) an 8-bit value nn – separately for each axis – can be transmitted to the scan system. Subsequently, a 20-bit value is returned on the corresponding status channel: If data transfer is error-free, then the upper 8 bits of the returned 20-bit value is identical with the originally sent 8-bit value, and the next lower 8 bits are identical with the complement of the sent 8-bit value.

These 20-bit values are returned until **control_command** (Data = 05nn_H) is used to select another return data type, see [Section "Configuring the Data Signal Transmission Behavior of the Scan System", Page 199](#).

Prior to a transfer test, the data type currently selected for transmission can be cached by **control_command** (Data = 17FF_H). After the test, **control_command** (Data = 1700_H) restores the same transmission behavior as before the test.

(1) See Glossary entry on [page 23](#).

(2) See Glossary entry on [page 23](#).

8.2 Coordinate Transformations

For precise set-up of the scan system relative to the **Image field** (or, if the **Option "Second Scan Head Control"** is enabled, *two scan heads* can be adjusted relative to a *common Image field*), a linear coordinate transformation can be defined (separately for the first and second scan head connectors) for all X and Y output coordinates (x|y) defined by **Vector commands** or **"Arc" commands**:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = M \times \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

The (2 x 2) total matrix M is thereby automatically calculated by the RTC5 PCI Board as a product of a scaling matrix M_S , a rotation matrix M_R and a general transformation matrix M_T :

$$M = M_T \times M_R \times M_S$$

The coefficients of the three matrices (M_T , M_R , and M_S) and the offset values ($x_0|y_0$) can be individually defined for the first and second scan head connector.

The offset ($x_0|y_0$) is set by **set_offset** or **set_offset_list**.

For 3-axis scan systems, **set_offset_xyz** or **set_offset_xyz_list** enables setting of an offset z_0 for the z coordinate, too (z_0 has the opposite effect of **set_defocus** or **set_defocus_list**).

The following applies:

$$z' = z + z_0$$

The coefficients of the scaling matrix M_S are set by **set_scale** or **set_scale_list** using a scaling factor k that is common to both axes:

$$M_S = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$$

The coefficients of the rotation matrix M_R are set by **set_angle** or **set_angle_list** by specifying a rotation angle α (in accordance with mathematical convention: positive angles produce counterclockwise rotation):

$$M_R = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix}$$

The coefficients $m_{11} \dots m_{22}$ of the general transformation matrix M_T are set by **set_matrix** or **set_matrix_list**:

$$M_T = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}$$

With the general transformation matrix M_T , the two above matrices (M_S and M_R , as special case) as well as further transformations for scaling, rotating, mirroring or skewing objects can be defined:

- Scaling by the factors k_x and k_y :

$$M_T = \begin{bmatrix} k_x & 0 \\ 0 & k_y \end{bmatrix}$$

- Rotation by the angle α :

$$M_T = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix}$$

Example:

`set_matrix( 1, 0.5, -0.866, 0.866, 0.5, 1 )`
defines a rotation by 60° (counterclockwise)
around the center of the **Image field** for the first
scan head connector.
This can also be achieved by `set_angle(1, 60)`.

- Mirroring around the y axis (flipping in the x direction):

$$M_T = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

- Mirroring around the x axis (flipping in the y direction):

$$M_T = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- Mirroring around the first dimension diagonal (exchanging the X and Y coordinates):

$$M_T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- Skewing in the x direction by the angle α (slanting):

$$M_T = \begin{bmatrix} 1 & -\sin\alpha \\ 0 & 1 \end{bmatrix}$$

Example: `set_matrix( 1, 1, -0.25, 0, 1, 0 )`

A general transformation defined by `set_matrix` or `set_matrix_list` can also represent a combination of various transformations (users can calculate the corresponding matrix M_T by multiplying the corresponding individual matrices in the correct order).

Notes

- The described coordinate transformations are primarily intended for small corrections when setting up the scan system relative to the **Image field**. Separate settings for scaling and rotations thereby provide more handling flexibility in comparison to a single matrix setting.
- Initialization by `load_program_file` results in an offset of (0|0|0) and in matrices M_S , M_R and M_T , each predefined as identity matrices.
- Each matrix or offset definition overwrites prior definitions.
- The RTC5 PCI Board calculates the total matrix M independently of the order in which the individual transformation matrices were defined.
- The value range of scaling factor k for the scaling matrix M_S is $[-16...+16]$. The value range for the coefficients of the general transformation matrix M_T is $[-50...+50]$. Also be sure that the value range $[-50...+50]$ for individual coefficients of the total matrix M are not exceeded; otherwise calculation of the corrected coordinates might result, under some circumstances, in overflows.
- Rotations take place exclusively around the centerpoint of the **Image field**; mirroring is relative to the axes.
- For each definition, the parameter `at_once` can be used to specify whether the new setting should have immediate effect on the current position (`at_once = 1` or `3`) or whether it should only be provisionally accumulated and cached (`at_once = 0` or `2`). The most recently called `at_once` parameter value determines when a (accumulated) transformation takes effect.

- Before the total transformation is applied to the current position, the **Signals for "Laser Active" Operation** are switched off with `at_once = 0...2`. They remain unchanged with `at_once = 3`.
- With `at_once = 1` or `3`, all settings (only) accumulated until then are processed immediately and simultaneously. In the process, the scan system axes are moved from the current position to the corrected position at the defined jump speed. Consequently, this can require some clock cycles.
This needs to be observed especially when RTC5 PCI Boards are master/slave synchronized and are supposed to execute different jump lengths. With this use case, the coordinate transformations should be executed prior the synchronized start with `at_once = 1` and their end should be waited for.
Any **Scanner Delays** are not initialized. The **INTERNAL-BUSY list execution status** is set while the jump to the corrected position is executed.
- With `at_once = 2`, the accumulated settings only become effective upon execution of the next **jump_abs**, **jump_rel**, **goto_xy** or **goto_xyz** (but not other **Jump commands** such as **jump_abs_3d** or **jump_rel_3d**) unless afterwards another call triggers immediate execution. Correction of the current output position then occurs together with the specified coordinate jump. This eliminates unnecessary galvanometer scanner motions (incl. delays).
Example: The following command sequence produces a first jump to (0, 0), followed by – if `at_once = 1` or `at_once = 3` – a second jump from (0, 0) to (1000, 500) and a third from (1000, 500) to (0, 500). But if `at_once = 2`, then only a second jump occurs from (0, 0) to (0, 500).

```
jump_abs( 0, 0 );  
set_offset_list( 1000, 500, at_once );  
jump_abs( -1000, 0 );
```

Notes on data recording:

- The galvanometer scanner motion of **jump_abs** or **jump_rel** at the beginning of the list can be recorded with **set_trigger** (signal type **7, 8, 9**), if no coordinate transformation with `at_once = 2` has been applied before.
- In case of motions *after* a coordinate transformation (for example, by **set_offset_list** with `at_once = 2`) the "sample values" (**set_trigger** signal type **7, 8, 9**) are immediately set to the target coordinates and remain unchanged for the duration of the motion. The motion recorded here does not appear like **Microstepping**⁽¹⁾ has been performed, but rather like a **Hard jump**. The real galvanometer scanner motion can be recorded with **set_trigger**, signal type **25** and **26** (transformed control values).
- In case of motions caused by coordinate transformation in a list (for example, by **set_offset_list** with `at_once = 1`), the "sample values" (**set_trigger** signal type **7, 8, 9**) remain unchanged for the duration of the motion. The real galvanometer scanner motion can be recorded with **set_trigger**, signal type **25** and **26** (transformed control values).
- Settings via control commands by `at_once = 0` are only saved as long as no list is running. When the execution is started or the list is already running, the settings take effect immediately before the next list command. Settings via list commands with `at_once = 0` are only saved until they are retrieved elsewhere (`at_once > 0` or by a control command).
- If no correction table is assigned to the corresponding scan head connector, then the new settings for the coordinate transformations are only stored on the RTC5 PCI Board. They take effect when a correction table is assigned.

(1) See **Chapter 7.1.2 "Microstepping"**, Page 128.

- Coordinate transformations are applied to all to-be-outputted coordinates from all **Vector commands** ([*]jump[*] or [*]mark[*] list commands, but also **goto_xy** or **goto_xyz**) and **"Arc" commands**. See also [Chapter 7.3.6 "Output Values to the Scan System"](#), [Page 170](#).
- With a scaling matrix, the effective jump speed and mark speed changes.
- With 3D vector commands:
 - The transformation matrix only affects the x and y components
 - The offset affects all three components
- In order to utilize the complete real **Image field** with coordinate transformations (such as rotations, shrinkages or shifts), the extended value range of the virtual **Image field** can be used even without Processing-on-the-fly, see [Chapter 7.3.3 "Virtual Image Field"](#), [Page 158](#).
- Coordinate transformations for the virtual **Image field** can be defined, see [Section "Coordinate Transformations in the Virtual Image Field"](#), [Page 158](#).
See also [4](#) in [Chapter 7.3.6 "Output Values to the Scan System"](#), [Page 170](#).

Notes for RTC4 Users

- With RTC5 PCI Boards, coordinate transformations can no longer be set upon loading a correction file by **load_correction_file**. Instead, coordinate transformations are separately defined for the first and second scan head connector and serve the same purpose as those of **load_correction_file** of the RTC4.
- RTC5 PCI Boards do not support the coordinate transformations (collectively for both *scan heads*) before **Microstepping** which is possible with the RTC4.
The global coordinate transformations have a slightly different effect than the coordinate transformations above, see [#3](#), [#5a](#) and [#5b](#) in [Chapter 7.3.6 "Output Values to the Scan System"](#), [Page 170](#).

8.3 Online Positioning

The preceding [Chapter 8.2 “Coordinate Transformations”](#), [Page 209](#) details how to precisely align a scan system relative to the [Image field](#), see also [Section “Coordinate Transformations in the Virtual Image Field”](#), [Page 158](#).

The user program can, for example, determine the required transformation values by automatic position analysis for a workpiece on a conveyor belt and then execute the associated transformations.

However, it is not easy to achieve well-controlled timing (referenced to the RTC5’s 10 μ s clock) while positioning a workpiece and aligning the scan system by the (control) commands described in [Chapter 8.2 “Coordinate Transformations”](#), [Page 209](#). For applications in which such timing is important, commands for so-called [Online Positioning](#) are available. Here, data for an offset and/or rotation coordinate transformation or a general matrix operation can be inputted by the [McBSP interface](#).

The following variants are provided for different use cases:

- The (ever present) [“Local Online Positioning”](#) concerns the scan system-specific coordinate transformations in the real [Image field](#) analogous to [set_offset](#), [set_angle](#) and [set_matrix](#) with parameter `HeadNo = 1` or `2` according to [#5a](#) and [#5b](#) in [Chapter 7.3.6 “Output Values to the Scan System”](#), [Page 170](#). It is described in [Chapter 8.3.1 ““Local Online Positioning””](#), [Page 213](#).
- The [“Global Online Positioning”](#) concerns global coordinate transformations in the virtual [Image field](#) analogous to [set_offset](#), [set_angle](#) and [set_matrix](#) each with `HeadNo = 4` according to [#3](#) in [Chapter 7.3.6 “Output Values to the Scan System”](#), [Page 170](#). It is described in [Chapter 8.3.2 ““Global Online Positioning””](#), [Page 216](#).

[Online Positioning](#) cannot be combined with Processing-on-the-fly applications with position information, but it can be combined with encoder-based Processing-on-the-fly applications.

Notes

- Since coordinate transformations [#5a](#) and [#5b](#) are calculated after the Processing-on-the-fly correction [#4](#), larger [Image field](#) rotations are not well compatible with linear Processing-on-the-fly corrections. In such cases, [“Global Online Positioning”](#) is preferable.

8.3.1 “Local Online Positioning”

Reading in data for [“Local Online Positioning”](#) via the [McBSP interface](#) needs to be activated and configured as desired with the commands [set_mcbasp_x](#), [set_mcbasp_y](#) and/or [set_mcbasp_rot](#) or [set_mcbasp_matrix](#) (or by the equivalent list commands) (see below). With [apply_mcbasp](#) or [apply_mcbasp_list](#), you can acquire the most recent fully transferred values and define the required coordinate transformations (as with [set_offset](#) and/or [set_angle](#) or [set_matrix](#) or the equivalent list commands). Here, as with the commands described in [Chapter 8.2 “Coordinate Transformations”](#), [Page 209](#) an `at_once` parameter can be used to specify when the newly defined (total) transformation should take effect.

For precise timing, execution of the list command that triggers the transformation (depending on the `at_once` parameter, this would be [apply_mcbasp_list](#) or [jump_abs](#) or [jump_rel](#) or any other list command) can be made dependent on the input of an external control signal (for conditional command execution, see [Chapter 9.3.2 “Conditional Command Execution”](#), [Page 271](#)).

The [McBSP interface](#) is described on [Chapter 4.6.6 “SPI / I2C Socket Connector”](#), [Page 71](#).

Configuring “Local Online Positioning”

set_mcbasp_x, **set_mcbasp_y** and/or **set_mcbasp_rot** (or the equivalent list commands) determine both how the values inputted at the **McBSP interface** are interpreted by the RTC5 and which internal memory location is used to read the values:

- Depending on which of the above commands is called, the RTC5 interprets the inputted values as offsets in the x direction and/or y direction and/or as rotation values or as matrix coefficients. The desired scaling factor always needs to be supplied as a command parameter (except with **set_mcbasp_matrix**, see command description). The three options **x**, **y** and **rot** can be used either separately or in any desired combination. By the appropriate command, each option can be enabled or disabled independently of the other two. In contrast, the **matrix** option cannot be used in conjunction with other options.
- As soon as one of the 4 options becomes activated, all values subsequently inputted at the **McBSP interface** are internally stored in memory location 1 or possibly memory location 2 (see below). Transferred values can subsequently be queried by **read_mcbasp** or applied in coordinate transformations by **apply_mcbasp** or **apply_mcbasp_list**.

- **x or y or rot**

If you activate only *one* of the three options, then x offset or y offset correction values can be supplied as signed 32-bit values or rotation correction values as unsigned 32-bit values. The **McBSP** input values are transferred to internal memory location 1.

- **x and y (without rot)**

If you activate x offset and y offset corrections, but no rotation correction, then the two offset correction values must be supplied as a signed 16-bit values, each, combined to a 32-bit value (the x value in the lower 16 bits and the y value in the upper 16 bits).

The **McBSP** input values are transferred to internal memory location 1.

- **x or y and rot**

If you activate an x offset or a y offset correction together with a rotation correction, then the offset and rotation correction values should be alternately supplied as 32-bit values. The **McBSP** input values are then alternately transferred to internal memory locations 1 and 2. The RTC5 identifies the data type by examining the coding Bit #31 (Bit #31 = 0 for offset values, Bit #31 = 1 for rotation correction values). Signed 31 bits are effectively available for transferring offset values. 31 bits *without* sign are available for rotation correction values.

- **x and y and rot**

If you activate all three options together, then the two offset correction values must be combined and supplied as one 32-bit value alternatingly supplied with the rotation correction value as a second 32-bit value. The **McBSP** input values are likewise alternatingly transferred to internal memory locations 1 and 2.

The RTC5 identifies the data type by examining the coding Bit #31 (Bit #31 = 0 for offset values, Bit #31 = 1 for rotation correction values). Signed 15 bits are effectively available for transferring offset values (whereby x values reside in the lower 16 bits and y values in the upper 16 bits). 31 bits *without* sign are available for rotation correction values.

The last two cases designate the data type by coding Bit #31. Though this makes the order of transmission irrelevant, always *both* data types nevertheless must be transmitted (preferably always alternatingly). If a request is made by **apply_mcbbsp** (or **apply_mcbbsp_list**) when two values of the same data type exist in both memory locations, then the most recently transferred value is always used. If two identical Bit #31 codings are present, then the last transfer should have already ended at the time of the request.

- **matrix**

With this option activated, matrix coefficients are then transferable as signed 32-bit values. The indices are encoded in the data word (see command description). **McBSP** input values get transferred to internal memory location 1.

Notes

- You can use “**Local Online Positioning**” in conjunction with an encoder-controlled Processing-on-the-fly application, but *not* in conjunction with a Processing-on-the-fly application controlled by **McBSP/SPI** signals:
 - When you use the commands for configuring “**Local Online Positioning**”, then Processing-on-the-fly correction activated by **set_fly_x_pos**, **set_fly_y_pos**, **set_fly_rot_pos**, **set_mcbbsp_in**, **set_mcbbsp_in_list**, **set_multi_mcbbsp_in** or **set_multi_mcbbsp_in_list** gets automatically deactivated. Subsequently, **McBSP** input values are copied to internal memory locations 1 and possibly 2 (see above) and are then available for “**Local Online Positioning**”, but no longer for a Processing-on-the-fly application.
 - In reverse, **set_fly_x_pos**, **set_fly_y_pos**, **set_fly_rot_pos**, **set_mcbbsp_in**, **set_mcbbsp_in_list**, **set_multi_mcbbsp_in** or **set_multi_mcbbsp_in_list** deactivate a previously activated “**Local Online Positioning**”. Subsequently, **McBSP** input values are then copied possibly instead of or additionally to the internal memory location 0 and/or 3 and are then available for the Processing-on-the-fly application.
 - If you switch off (intentionally or with an invalid scaling factor) “**Local Online Positioning**” by **set_mcbbsp_x**, **set_mcbbsp_y** or **set_mcbbsp_rot**, then the data is continued to be copied to internal memory locations 1 or 1 and 2 (as long as data is transmitted), but it is no longer applied (**apply_mcbbsp** has no effect).

8.3.2 “Global Online Positioning”

“Global Online Positioning” (available as of DLL 545, OUT 545) needs to be activated by one of the following commands:

- **set_mcbasp_global_matrix**
- **set_mcbasp_global_rot**
- **set_mcbasp_global_x**
- **set_mcbasp_global_y**
- **set_mcbasp_global_matrix_list**
- **set_mcbasp_global_rot_list**
- **set_mcbasp_global_x_list**
- **set_mcbasp_global_y_list**

Once activated, all other **McBSP** processings are deactivated (“Processing-on-the-fly” applications controlled by **McBSP** signals, “Local Online Positioning” as described in Chapter 8.3.1 ““Local Online Positioning””, Page 213, processings activated by **set_mcbasp_in** or **set_multi_mcbasp_in** and vice versa.

All subsequent data transferred via **McBSP** are internally handled in the same way as with “Local Online Positioning” (copied to internal memory locations 1 and possibly 2; readable by **read_mcbasp**), but instead of the scan system-specific coordinate transformations (see Chapter 8.3.1 ““Local Online Positioning””, Page 213) automatically used for coordinate transformations in the virtual **Image field** instead.

Calling **apply_mcbasp** or **apply_mcbasp_list** is no longer necessary and even has no effect.

Coordinate transformations in the virtual **Image field** (see also Chapter 8.6.4 “Compensating 2D Motions”, Page 234) are automatically applied, as soon as a Processing-on-the-fly application is activated afterwards.

During a Processing-on-the-fly application, new data can only be sent and stored, but not applied, see also:

- **set_matrix**(HeadNo = 4)
- **set_offset_xyz**(HeadNo = 4)
- **set_angle**(HeadNo = 4)

“Global Online Positioning” is compatible with Processing-on-the-fly applications controlled by encoder signals.

Notice!

- The latest transferred data value is used immediately according to the current “Global Online Positioning” mode. Therefore, make sure to transmit correct values after changing that mode and before applying the data by starting a Processing-on-the-fly session.
- Example: **set_mcbasp_global_x** and **set_mcbasp_global_y** combine the xy offsets in the lower and upper half word of the transmitted data.
set_mcbasp_global_y(Scale = 0.0) disables the y offset and the latest sent data value is used *in total* as x offset. Make sure to transmit the correct x offset again.

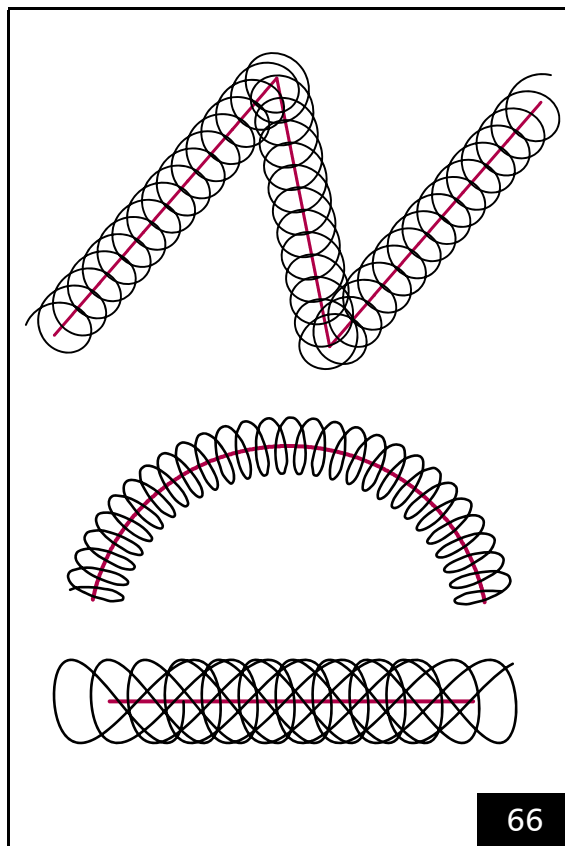
Configuring “Global Online Positioning”

The “Global Online Positioning” is configured the same way as the “Local Online Positioning”, see Chapter 8.3.1 ““Local Online Positioning””, Page 213.

8.4 Wobble Mode

The wobble mode allows varying the line width for laser marking.

For this purpose, an ellipse-shaped motion is added to the regular, linear motion of the output position. This results in a spiral motion of the laser focus in the **Image field**, see **Figure 66**. Alternatively, a combination with a figure-of-8 (horizontal or vertical to the direction of motion) can be activated.



Principle of the Wobble mode. Top: circular wobble. Middle: ellipse-shaped wobble. Bottom: figure-of-8 wobble (horizontal 8).

A broadening of the original line is obtained by choosing suitable values for the transverse and longitudinal amplitudes and the frequency of the wobble motion. With figure-of-8s, broader mid-line processing can be achieved by appropriate parameter values. If the specified transverse and longitudinal amplitudes are identical, then the wobble shape remains stationary in space; otherwise the orientation of the wobble shape follows the current direction of motion.

As of version DLL 543, OUT 543, RBF 524 the present wobble amplitude (from **set_wobble** or **set_wobble_mode**) can be recorded with trigger signal 53 (see **set_trigger** or **set_trigger4**). The format of the data is ((transversal << 16) + longitudinal).

During Processing-on-the-fly correction by the McBSP/SPI or encoder interfaces where the scan head only compensates differences between actual external motions and intended total motion (see **Chapter 8.6.2 "Compensation of Linear Motions", Page 228**, a slight jitter in the direction of galvanometer scanner motion might occur, particularly during exact external path motions. Here, the wobble motion superimposed onto the direction of motion does correspondingly jitter, too. You can avoid such jitter by specifying a fixed (instead of the momentary) direction of motion for the wobble shape by the **set_wobble_direction** list command.

For optimum marking results, the wobble frequency should be appropriate for the specified mark speed. In some cases it can be useful to adjust the mark speed when the wobble mode is used.

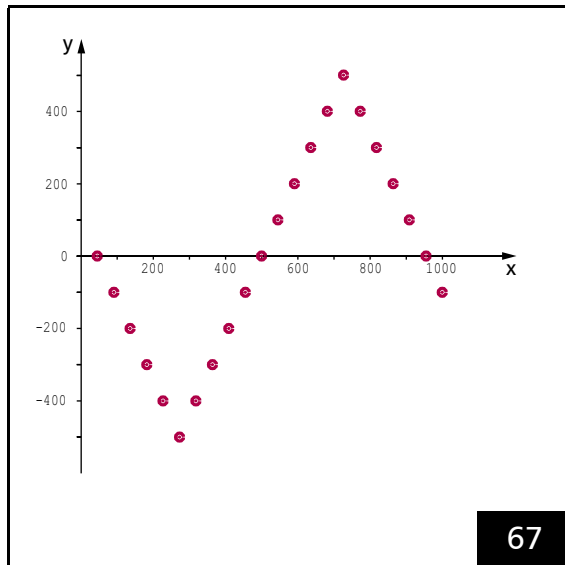
After **set_wobble** or **set_wobble_mode**, the wobble start point is always set for the same value relative to the vector/arc startpoint and direction. The Wobble phase is then continued both within an uninterrupted **Polyline** (including arcs) and after interruptions (for example, by a **Jump command**) until **set_wobble** or **set_wobble_mode** are called again.

The wobble mode cannot be combined with:

- Sky Writing
- Pixel Output Mode
- Jumps
- **laser_on_list**

For further details, see **set_wobble** and **set_wobble_mode**.

For many welding applications, the default set of “classic” wobble shapes (circle, ellipse, sine, figure-8) does not produce optimal (smooth) results in the area of the weld seam. With ellipses, for example, high-speed motion occurs parallel to translation motions and low-speed motion occurs in the opposing direction, resulting in uneven energy deposition. **set_wobble_vector** lets you define a wobble shape customized for your user program, consisting of up to 512 piecewise linear sections, while also specifying variation of laser power along that shape (see also **set_wobble_control**). However, **set_wobble_vector** cannot be used together with the Softstart function nor with automatic or vector-based laser control.



See Section “Example Code (C++)”, Page 218.

Example Code (C++)

The following example C++ source code shows a zigzag pattern, see [Figure 67](#).

The code must be included in a user program, see [Chapter 6.2.5 “Example Code \(C\)”, Page 89](#).

```
// Zigzag pattern

// Transversal micro step
const double dTrans( 100.0 );
// Longitudinal micro step
const double dLong( 0.0 );

// Number of steps
const uint16 Period( 5 );

// Start a new wobble shape
set_wobble_vector( 0, 0, 0, 0 );

// 1st wobble vector
set_wobble_vector( dTrans, dLong,
    Period, 0.0 );
// 2nd wobble vector
set_wobble_vector( -dTrans, dLong,
    Period * 2, 0.0 );
// 3rd wobble vector
set_wobble_vector( dTrans, dLong,
    Period, 0.0 );

// if laser power variation is needed,
include here
// set_wobble_control( Ctrl, Value,
    MinValue, MaxValue );

// Activate freely definable wobble shapes
set_wobble_mode( 100, 0, 100, 2 );

// Mark a vector
timed_mark_rel( 1000, 0, 22*10 );
```

8.4.1 Wobbel Shapes – Important Notes on Choosing Appropriate Parameter Values

“Classic” Wobbel Shapes

“Classic” wobbel shapes are defined by **set_wobbel_mode** or **set_wobbel**. Only assign hardware appropriate values to the Transversal, Longitudinal and Freq parameters.

Notice!

- Too big values define control situations where very high waste heat is produced. The galvanometer scanners and digital control board/amplifier board overheating and permanent damage may occur even in short-term operation (overload!). Take the highest possible dynamics of scan head and laser into account.
- If the frequency values are too high the galvanometer scanners may not be able to follow the nominal curve. This may lead to unexpected marking results.

Rule of thumb to estimate appropriate maximum values:

- (1) Maximum frequency: $F = 1/(10 \times T)$
where $T = \text{Tracking error}^{(1)}$
- (2) Wobbel amplitude⁽²⁾: $A = T \times V$
where $V = \text{typical positioning speed}^{(1)}$

Notes

- Also take the path velocity on the wobbel shape itself into account. For circular wobbel shapes it is calculated as follows:

(3) Path velocity $V_{\text{Path}} = 2 \times \pi \times A \times F$.

- Make sure that the combination of the used values and the **Trajectory** velocity are suitable for a long-term operation without causing damages (process safety!).
- Check the temperature status already during an evaluation period (see **get_head_status**) in order to recognize potential overload situations at an early stage. With iDRIVE systems you can also read-out the present temperatures of galvanometer scanners and/or digital control boards (see **control_command**).

Example of Use

System Specification

- **Tracking error** $T = 0.33 \text{ ms}$.
- Calibration $\pm 0.349 \text{ rad}_{\text{optical}}$
($= 10^\circ_{\text{mechanical}}$) at $\pm 503,316 \text{ bit}$.
– This is an angle control AC of
 $\approx 1.44 \times 10^6 \text{ bit/rad}_{\text{optical}}$
($\approx 50,300 \text{ bit/}^\circ_{\text{mechanical}}$).
- Calibration factor⁽³⁾ $K = 5,000 \text{ bit/mm}$.
- Typical positioning speed
 $V_{\text{rad}} = 100 \text{ rad}_{\text{optical}}/\text{s}$.
– Converted to control values this corresponds to a typical positioning speed of
 $V = V_{\text{rad}} \times AC$
 $\approx 1.44 \times 10^8 \text{ bit/s} = 1,440 \text{ bit/}10 \mu\text{s}$.
– In the **Image field** this corresponds to a typical positioning speed of $V/K = 28.8 \text{ m/s}$.

(1) See technical specifications in the scan head manual.

(2) At the maximum frequency estimated with (1).

(3) See *_ReadMe.txt file of correction file or **get_table_para**.

Wobbel Parameters

- The maximum frequency estimated with rule of thumb (1):
 $F = 1/(10 \times 0.33 \times 10^{-3} \text{ s}) \approx 300 \text{ Hz}$.
- Wobbel amplitude estimated with rule of thumb (2):
 $A = 0.33 \times 10^{-3} \text{ s} \times 1.44 \times 10^8 \text{ bit/s}$
 $\approx 47,500 \text{ bit}$.
 – In the **Image field** this corresponds to a wobbel amplitude of $A/K \approx 9,5 \text{ mm}$.
- Path velocity of circular wobbel shapes as given by rule of thumb (3) is:
 $V_{\text{path}} \approx 895 \text{ bit}/10 \mu\text{s}$.
- The *maximum mark speed* results from V_{path}/K (in this example $89,500,000 \text{ bit/s} / 5,000,000 \text{ bit/m} \approx 17.9 \text{ m/s}$. Note that this value is a rough estimate. Furthermore, it is dependent on a position due to the correction file (which is non-linear).
 The calculated path velocity for wobbel only (!)
 – shall be lower than the specified maximum positioning speed,
 – but there may be certain circumstances where it is much higher than the *typical mark speed*.
 Make sure that your values are suitable for operation (temperature status and so on, same as above).

“Freely Definable Wobbel Shapes”

When defining “freely definable wobbel shapes” (see **set_wobbel_vector**) take the dynamics of the scan head into account.

- The maximum repetition rate for “freely definable wobbel shapes” corresponds to the maximum wobbel frequency estimated by the rule of thumb (1), **page 219**.
- The sum of the path (**Trajectory**) velocity and the velocity on the freely defined wobbel figure should not exceed the maximum positioning velocity.
- The acceleration should not significantly exceed the value of 2.5 V/S (where V = typical positioning speed and S = **Tracking error**).
- Also with “freely definable wobbel shapes”, observe the heating of the scan system for freely definable wobbel figures, see safety notice on **page 219** and rule of thumb (3).

8.5 Controlling 2D Scan Systems and 3D Scan Systems

8.5.1 2D Scan Systems

Up to two 2D scan systems can be controlled by one RTC5 PCI Board.

The second one requires the **Option "Second Scan Head Control"**, Page 34.

When controlling two 2D scan systems, a separate **Image field** correction can be carried out for each of the both scan systems.

By **select_cor_table** or **select_cor_table_list**, the two correction tables are assigned to the respective scan head connector, see also **Section "2. SCANHEAD Socket Connector"**, Page 55).

To use two correction tables in a double scan system configuration

(1) Load each of the desired 2D correction files by

```
load_correction_file(Name, n, 2)
(n = 1...4).
```

See also **Chapter 8.5.3 "Using Several Correction Tables"**, Page 226.

(2) Assign a loaded 2D-correction table to the first and the second scan head each by calling

```
select_cor_table(HeadA, HeadB)
with (HeadA and HeadB = 1...4).
```

(3) By **set_offset**, **set_scale**, **set_angle** or the corresponding list commands, specify offset, scaling factor and rotation to align the two **Image fields** precisely with respect to each other.

Notes

- If you are not using one of the scan head connectors: assign the correction table 0 to it.
- The default setting for **select_cor_table** and **select_cor_table_list** is (1, 0):
 - For the first scan head, correction table #1 is used
 - At the second scan head, there are *no position outputs*
- The RTC5 PCI Board returns to this setting after every **load_program_file**. If a different setting is to be used, **select_cor_table** or **select_cor_table_list** must be called again.
- The scan head connectors *cannot* be simultaneously assigned:
 - two 3D correction tables
 - a 2D correction table *and* a 3D correction table
- **load_program_file** deletes correction tables number 3 and 4.

8.5.2 3D Scan Systems

An RTC5 PCI Board can also be used to control a 3-axis scan system⁽¹⁾. This requires the **Option "3D"**, **Page 34**.

Intended Use

3-axis scan systems can be used for positioning the laser focus within a flat processing field without the need for a flat field objective. Therefore, they are frequently used in applications for which flat field objectives are not available.

3-axis scan systems can also be used as 3D beam deflection systems. Here, the laser focus is guided along the contour of the workpiece being processed, thus enabling workpiece processing in 3 dimensions.

SCANLAB offers dynamic focusing units⁽²⁾ that extend xy scan systems to 3-axis scan systems.

Connection and Initialization

In order to control a 3-axis scan system by an RTC5 PCI Board:

- The **Option "3D"**, **Page 34** of the RTC5 PCI Board must be enabled (this can be checked by **get_rtc_version**)
- The xy scan system must be connected to the first scan head connector
- The z axis must be connected to the second scan head connector
- A 3D correction table (D3_*.CT5) must exclusively be assigned to the first scan head connector (by **select_cor_table** or **select_cor_table_list**).
- If, in addition, the **Option "Second Scan Head Control"**, **Page 34**, is enabled, it is alternatively possible to connect:
 - the xy scan head to the second scan head connector
 - the z axis to the first scan head connector
 - In this constellation the 3D correction table must be assigned to the second scan head connector.

See also **Chapter 4.5 "Interfaces to Scan System"**, **Page 54** and **Section "2D Correction Files and 3D Correction Files"**, **Page 163**.

Other than that, no additional drivers or software files are needed for controlling a 3-axis scan system. Even initialization and program launching remain unchanged, see **Chapter 6.2 "Initialization and Program Start-Up"**, **Page 84**.

Notice!

- The standard **RTC5 DLL (RTC5DLL.dll)** supports all RTC5 commands for controlling 3-axis scan systems. However, the full functionality of the RTC5 commands – the actual *output* of z coordinates – is only available with enabled **Option "3D"**, **Page 34**.

Several 3-axis scan systems can be simultaneously controlled (by multi-board commands) from a single PC. This requires a corresponding number of RTC5 PCI Boards with enabled **Option "3D"**s to be installed in that PC.

(1) Consisting of an xy scan system and a z axis dynamic focusing unit (see footnote 2) as z axis.

(2) See Glossary entry on **page 23**.

3D Commands

These are, for example, the following RTC5 commands:

- `[*]3d[*]`
- `goto_xyz`
- `set_offset_xyz` and `set_offset_xyz_list`

Except for the additional motion in the third dimension, the 3D commands function identically to their corresponding 2D commands:

- Specified vectors and arcs are split-up into **Microsteps**, see [Chapter 7.1.2 "Microstepping"](#), [Page 128](#)
- The jump speed and mark speed are specified by `set_jump_speed` and `set_mark_speed`, see [Chapter 7.1.1 "Marking with Vector Commands and "Arc" Commands"](#), [Page 124](#). As for timed **Vector commands**, the speeds are automatically determined from the specified jump or marking duration
- **3D image field** correction is applied in accordance with the 3D correction table assigned by `select_cor_table` or `select_cor_table_list`, see [Chapter 7.3.5 "Image Field Correction and Correction Tables"](#), [Page 162](#)
- For **Jump commands** and `[*]mark[*]` **Commands**, the laser control signals are switched on and off while taking delay settings into account, see [Chapter 7.2 "Delay Settings – Coordinating Scan Head Control and Laser Control"](#), [Page 133](#)

For the value range of the data, see [Section "Compatibility Modes"](#), [Page 157](#).

The calibration factor K_{xy} can be read out from the correction table used by `get_table_para`.

In **RTC4 Compatibility Mode** and **RTC5 Standard Mode**, the 3 coordinate values are always scaled up to 20-bit values internally, so that the 3 spatial directions are treated equally with regard to the jump speed or mark speed.

The 3 coordinate values are internally always scaled up to 20-bit values, so that the 3 spatial directions are treated equally with regard to the jump speed or mark speed.

With big z jumps, observe the different dynamics of varioSCAN and 2D scan systems.

Notes

- After "vector-controlled laser control" has been activated by `set_vector_control` (`Ctrl = 7`), the para-**Mark commands** and para-**Jump commands** can be used to set a dynamic focus shift linearly along the mark or jump vector, see [Section "Vector-Defined Laser Control"](#), [Page 194](#). See also `set_defocus` or `set_defocus_list`.
- The size of the usable **Image field**⁽¹⁾ and the focus shift in the z direction⁽²⁾ (height of the usable **3D image field**) can be obtained from the `*_ReadMe.txt` file supplied with the 3D correction file as well as from the user manual of the 3-axis scan system/varioSCAN ("Technical Specifications" chapter).
- If a z axis is to be used to hold the laser focus only in a certain plane (especially in 3D systems without a lens), then even 2D vector commands can be used. 2D vector commands leave the z position previously set with a 3D vector command unchanged and only adjust the focus length.
- If the **Option "3D"**, [Page 34](#) is not enabled or no 3D correction table has been assigned:
 - The split-up into **Microsteps** is calculated including the z axis (which influences the effective jump speed and mark speed in the xy plane).
 - There is merely no output for the z axis

(1) Line "Max. Field Size (z=0): nnn.nnn mm".

(2) Line "Max. Z-Range: +/- nn.n mm".

- By **set_offset_xyz** or **set_offset_xyz_list**, an offset can be defined for the z coordinate. In addition, **set_defocus** or **set_defocus_list** can be used for defining an offset to the calculated focal length, which then appropriately affect the z output value (in the direction opposite to the offset).
- During a marking process, the scan system focal length is continuously readjusted. During execution of a **Jump command** (or **goto_xyz**) this readjustment can be switched off. With **DirectMove3D = 1** in **set_delay_mode** or **set_delay_mode_list**, the z output value is directly (linearly) taken to its final value during the jump.

Enhanced 3D Correction

- For more information on the actual procedure, refer to the *"Calibrating a 3-Axis Laser Scan System" Manual, Chapter 5.4 "Step 4: Correcting Stretch"*.

The image size of 3D scan systems without objectives or with non-telecentric F-Theta objectives depends on its distance to the scan head objective (z coordinate).

The **Image field** size typically gets stretched or squeezed linearly with the z value. Therefore, SCANLAB 3D correction files include stretch correction factors to compensate for this effect, see *Section "ct5 Correction File Header", Page 167*.

With some objectives, the typical stretching is combined with a change in the image geometry. Then additional stretch corrections are necessary which compensate the image geometry changes xy position-dependent but linear in z (2D stretch correction table). The stretch correction values are meaning the Z gradient for the location deviation at this point.

Assumption: for any chosen non-zero Z plane, (X, Y) is the desired position and (X', Y') the measured position. The corrections <StretchX> and <StretchY> are then calculated as follows:

$$\langle \text{StretchX} \rangle = (X - X') / Z$$

$$\langle \text{StretchY} \rangle = (Y - Y') / Z$$

You can then use **load_stretch_table** to load the enhanced 3D correction onto the RTC5 PCI Board from an ASCII text file and assign it to an already loaded 3D-correction table. This ASCII text file must contain corrections for a complete rectangular grid. The number of gridlines and their spacings can be freely defined and even differ in X and Y. If the absolute values of the corrections exceed 0.03125, then they are clipped to this limit value. The corrections are bilinearly interpolated for data points within the specified grid and linearly extrapolated for data points outside the grid.

Enhanced 3D correction is not active by default after **load_program_file**. It becomes active upon loading a valid table onto the RTC5 PCI Board. It can be deactivated by calling **load_stretch_table** using a **NULL** pointer instead of the filename.

For the 2D stretch correction tables, the following rules apply:

- The ASCII text file can contain one or several tables.⁽¹⁾
- The 2D stretch correction table must begin with the line:
`[StretchTable<No>]`
 <No> represents the table number to be specified by **load_stretch_table**.
- This is directly followed by a block of data points.
- If several tables with the same number exist, then only data from the first encountered table is read. The others are ignored.
- Reading of the text data terminates upon end of file or upon a line containing a caption.
- All characters to the right of a semicolon are treated as comments and ignored.
- The order of data points is up to you.
- The maximum length of a data line is 255 characters.
- A data line contains two position coordinates (in bits, signed integers) and two correction values (dimensionless, signed floating point numbers, use the period (.) or comma as the decimal separator), each separated by spaces or tabs:
`<Xpos> <Ypos> <StretchX> <StretchY>`
- If a data point reoccurs, then the most recently read value is used.
- Empty lines or incomplete data lines are invalid and are ignored.

(1) Even of another type, see table 1, page 141.

8.5.3 Using Several Correction Tables

The RTC5 PCI Board memory can store 2 different correction tables at the same time. These remain after a **load_program_file** call. 2 more correction tables can be loaded into the data recording memory (then only half of the entries are available there, see **set_trigger**). These correction tables are deleted by **load_program_file**.

It can be useful to work with several different correction tables even if only a single scan system (2D or 3D) is used. Example: a pilot laser and a working laser with different wavelengths are used.

Special applications sometimes also require different correction tables in quick succession. This change can be done, for example, by **select_cor_table**(n, 0) or **select_cor_table_list**(n, 0) (n = 1...**number_of_correction_tables**).

number_of_correction_tables serves

- to protect other commands (for example, **load_correction_file** and **select_cor_table**) from unwanted table numbers and
- to configure the memory organization, if necessary.

There is no need to change existing user programs except when user input is to be rejected in the future (by means of explicit RTC5 error messages).

8.6 Processing-on-the-fly

- “Processing-on-the-fly” means the combination of workpiece motion and scan system motion.
- The use of the *Processing-on-the-fly* functionality requires the **Option Processing-on-the-fly**.
- Whether the **Option Processing-on-the-fly** is enabled can be checked by **get_rtc_version**.

8.6.1 Intended Use and Initialization

With its **Option Processing-on-the-fly** enabled, the RTC5 PCI Board allows processing of parts in motion (for example, parts on a conveyor belt, rotating plate or xy positioning stage), as well as stationary parts with a moving scan system (for example, by a robot arm).

To adjust laser scan processes to the current workpiece position relative to the scan system, the position of the workpiece or scan system can be forwarded to the RTC5 PCI Board:

- indirectly by encoder counters
- directly by the **McBSP interface**

If the Processing-on-the-fly correction is active, the coordinate values of all **Vector commands** and **“Arc” commands** are transformed based on the forwarded position values.

Upon forwarding the motion as encoder pulses, RTC5-internal encoder counters are triggered. The counter values correspond to the current position. They get a scaling factor (from certain RTC5 commands) assigned and are then used as *Processing-on-the-fly* correction.

See also **Chapter 9.3.3 “Synchronization by Encoder Signals”**, Page 274.

Upon forwarding the position values via the **McBSP interface**, the input values get a scaling factor assigned and are then used as *Processing-on-the-fly* correction.

See also **Chapter 9.3.4 “Synchronization and Online Positioning by McBSP/SPI Signals”**, Page 276.

Simultaneous usage of both forwarding methods for Processing-on-the-fly correction of two independent motions is not possible, see **Section “Overview”**, Page 227.

The **McBSP interface** cannot be simultaneously used for a Processing-on-the-fly application and an **Online Positioning**, see Notes, page 215.

Processing-on-the-fly correction is activated and deactivated by list commands. The parameters required for activation may have to be determined beforehand by a calibration process (see below).

If both scan head connectors each have a 2D correction table assigned, then a Processing-on-the-fly correction has the same effect at both scan head connectors.

For the z axis, a Processing-on-the-fly correction can be activated as well, see **Chapter 8.6.11 “Processing-on-the-fly Correction for the z Axis”**, Page 242.

Overview

The following encoder-based Processing-on-the-fly corrections of motions are available:

- **set_fly_x**, **set_fly_y**, even in combination
To compensate linear motions of the workpiece (for example, by conveyor belt or xy positioning stage), see **Chapter 8.6.2 “Compensation of Linear Motions”**, Page 228.
- **set_fly_rot**
To compensate rotary motions of the workpiece (for example, by rotating plate), see **Chapter 8.6.3 “Compensating Rotary Motions”**, Page 232.
- **set_fly_2d**
To compensate 2D motions of the workpiece (for example, xy positioning stage), see **Chapter 8.6.4 “Compensating 2D Motions”**, Page 234.

The following **McBSP**-based Processing-on-the-fly corrections of motions are available:

- **set_fly_x_pos**, **set_fly_y_pos** even in combination
To compensate 1D or 2D motions of the scan system (for example, robot arms), see [Chapter 8.6.2 "Compensation of Linear Motions"](#), Page 228.
- **set_fly_rot_pos**
To compensate rotary motions of the scan system (for example, rotary table), see [Chapter 8.6.3 "Compensating Rotary Motions"](#), Page 232.
- **set_mcbasp_in**, **set_mcbasp_in_list**
To compensate 1D or 2D motions or rotary motions (for example, robot arms, rotary table), see [Chapter 8.6.2 "Compensation of Linear Motions"](#), Page 228 and [Chapter 8.6.3 "Compensating Rotary Motions"](#), Page 232.

The following Processing-on-the-fly corrections can be combined:

- **set_fly_x** with **set_fly_y**.
- **set_fly_x_pos** with **set_fly_y_pos**
- For the special case **set_fly_z**, see [Chapter 8.6.11 "Processing-on-the-fly Correction for the z Axis"](#), Page 242
- For linear 3D Processing-on-the-fly corrections with positional values, see [Section "Correction via McBSP Interface with Enhanced McBSP Input"](#), Page 231.

The following Processing-on-the-fly corrections *cannot* be combined:

- Encoder-based with **McBSP**-based
- linear and rotating

The last command always determines the overall correction unless it can be combined with previous settings.

However, mutual synchronization of any Processing-on-the-fly corrections by **wait_for_encoder_mode**, **wait_for_encoder_in_range** and **wait_for_mcbasp** is possible, see [Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications"](#), Page 237.

8.6.2 Compensation of Linear Motions

Processing-on-the-fly correction for linear motions (translations) can be activated⁽¹⁾ by:

- **set_fly_x** and/or **set_fly_y**
- **set_fly_x_pos** and/or **set_fly_y_pos**
- **set_mcbasp_in** (Mode = 1...3)
- **set_mcbasp_in_list** (Mode = 1...3)

A scaling factor must thereby be specified in each case.

With **set_multi_mcbasp_in** or **set_multi_mcbasp_in_list**, you can activate additional Processing-on-the-fly correction with positional values for linear motion in all three coordinate directions without bit-resolution restrictions. No scaling factor is required.

Processing-on-the-fly correction can be deactivated (simultaneously for both directions) by **fly_return**. See the corresponding notes in [Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections"](#), Page 236.

Correction via Encoder Counters

To be able to pass position values via the board-internal encoder counters⁽²⁾, the Processing-on-the-fly correction must be activated by:

- **set_fly_x** and/or **set_fly_y**

Here, the scaling factor [in bits per count] defines the relation between the shift [in bits] of the current output position in the **Image field** and one counter pulse (count) of the corresponding encoder counter. See also [Section "Determining the Scaling Factors"](#), Page 229.

By **set_fly_x**, the encoder counter "Encoder0" is reset to zero. By **set_fly_y** the encoder counter "Encoder1" is reset to zero.⁽³⁾

(1) Different Processing-on-the-fly corrections cannot be combined arbitrarily, see the [Section "Overview"](#), Page 227.

(2) See [Notice!](#), Page 49.

(3) By **set_control_mode Bit #9** it can be set beforehand whether the counter is reset at the respective Processing-on-the-fly command or at the start trigger.

Thereafter, the output value is calculated from the current output position by adding (for all spatial directions) the product of the scaling factor and the current counter value.

This correction takes place every 10 μs .⁽¹⁾

Notes

- **set_fly_x** and **set_fly_y** can be used in combination for 1D as well as for 2D Processing-on-the-fly applications (for example, an xy positioning stage), particularly for separate or separable marking tasks in the real **Image field**.
- For continuous marking in the virtual **Image field**, SCANLAB recommends to preferably use **set_fly_2d**, see Chapter 8.6.4 "Compensating 2D Motions", Page 234.

Determining the Scaling Factors

The scaling factors can be determined experimentally:

- (1) Read out the counter start value by **get_encoder**.
- (2) Start the motion.
- (3) Stop the motion.
- (4) Read out the counter end value by **get_encoder**⁽²⁾.
- (5) Measure the distance travelled in mm.
- (6) Calculate the encoder increment i as follows:

$$i = \frac{(\text{counter end value} - \text{counter start value})}{\text{distance travelled}}$$

- (7) Calculate the scaling factors Scale_x and Scale_y [in bits per count] as follows:

$$\text{Scale}_x = \frac{K}{i_x}$$

$$\text{Scale}_y = \frac{K}{i_y}$$

Whereby K is the calibration factor [in bits per mm], see Chapter 7.3.2 "Image Field Size and Image Field Calibration", Page 156 and i is the encoder increment from Step 6.

If the workpiece moves at a constant speed v_x or v_y [in mm per second] and an encoder simulation has been activated by **simulate_encoder**, then the scaling factors are calculated as follows:

$$\text{Scale}_x = \frac{K \times v_x}{(1,000,000 \text{ counts / s})}$$

$$\text{Scale}_y = \frac{K \times v_y}{(1,000,000 \text{ counts / s})}$$

- (8) Adjust the signs of the scaling factors according to the motion direction.

(1) The encoder counters are signed 32-bit counters with overflow (= after reaching the maximum (minimum) counter value, counting continues at the minimum (maximum) counter value).

(2) Alternatively, the counter start and end values can be stored in a cache on the RTC5 PCI Board by the list command **store_encoder** and then retrieved from there by the control command **read_encoder**.

Correction via McBSP Interface

To be able to pass position values via the **McBSP interface**, the Processing-on-the-fly correction must be activated by:

- **set_fly_x_pos** and/or **set_fly_y_pos**
- **set_mcbbsp_in** for positional values in 2 spatial directions
- **set_multi_mcbbsp_in** for positional values in all 3 spatial directions (requires no scaling factors)

Here, the scaling factor [in bits per bits] defines the relation between the shift [in bits] of the current output position in the **Image field** and the input value [in bits] at the **McBSP interface**. See also **Section "Determining the Scaling Factors", Page 230**.

Thereafter, the output value is calculated from the current output position by adding (for all spatial directions) the product of the scaling factor and the current input value at the **McBSP interface**.

This correction takes place every 10 μ s.

At the **McBSP interface**, see **Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71**, there are available:

- For 1D corrections – signed 32-bit values
- For 2D corrections – signed 16-bit values per axis (the y value is in the lower 16 bits and the y value in the upper 16 bits of the value at the **McBSP interface**)

Notes

- After **set_fly_x_pos** and **set_fly_y_pos**, the input values received at the **McBSP interface** automatically get copied to internal memory location 0. This still applies even after Processing-on-the-fly correction gets switched off by **fly_return**, **set_fly_x_pos**, **set_fly_y_pos** or **set_fly_rot_pos**, as well as after **load_program_file**. The current data at memory location 0 can be queried by **read_mcbbsp(0)**.

- In contrast, activation of Processing-on-the-fly correction by **set_mcbbsp_in** or **set_mcbbsp_in_list** also result in **McBSP** input values being copied to internal memory locations 1, 2 and/or 3, see **Section "Correction via McBSP Interface with Additional McBSP Input", Page 231**.
- After Processing-on-the-fly correction is activated by **set_multi_mcbbsp_in** or **set_multi_mcbbsp_in_list**, the transmitted data gets consecutively written to memory locations 0...3. From there, they can be read out unsorted by **read_mcbbsp** or sorted by **read_multi_mcbbsp**.

Determining the Scaling Factors

The scaling factors can be determined experimentally:

- (1) Read out the start input value by **read_mcbbsp**.
- (2) Start the motion.
- (3) Stop the motion.
- (4) Read out the end input value by **read_mcbbsp**.
- (5) Measure the distance travelled in mm.
- (6) Calculate the position increment i as follows:

$$i = \frac{(\text{end input value} - \text{start input value})}{\text{distance travelled}}$$

- (7) Calculate the scaling factors Scale_x and Scale_y [in bits per bits] as follows:

$$\text{Scale}_x = \frac{K}{i_x}$$

$$\text{Scale}_y = \frac{K}{i_y}$$

Whereby K is the calibration factor [in bits per mm], see **Chapter 7.3.2 "Image Field Size and Image Field Calibration", Page 156** and i is the encoder increment from Step 6.

Correction via McBSP Interface with Additional McBSP Input

To activate Processing-on-the-fly correction for linear motions using **McBSP** input values (as an alternative to **set_fly_x_pos** or **set_fly_y_pos**), you can also call **set_mcbasp_in** or **set_mcbasp_in_list** (Mode = 1...3).

These commands offer the advantage of using the **McBSP interface** to input additional desired signals that should not be subjected to Processing-on-the-fly correction even when it is activated.

For this, all **McBSP** input values must be coded by Bit #31.

- Bit #31 = 0: The input value gets copied to internal memory location 0 and is applied for Processing-on-the-fly correction.
- Bit #31 = 1: The input value gets copied to internal memory location 3 but is not applied for Processing-on-the-fly correction.

Notes

- All input values always get copied alternately to internal memory locations 1 and 2 and subsequently, in accordance with their Bit #31 coding, to internal memory locations 0 and 3. But after deactivation of correction by **set_mcbasp_in(0)** or **set_mcbasp_in_list(0)**, they only get copied to memory locations 1 and 2.
- You can query the data currently stored at internal memory locations 0 - 3 by **read_mcbasp**.
- A scaling factor is specified at **set_mcbasp_in** or **set_mcbasp_in_list**. For 2D corrections (Mode = 3), the scaling factor applies to both axes simultaneously. Calibration for determining the scaling factor is the same as for **set_fly_x_pos** and **set_fly_y_pos** (see above).
- For transferring 1D correction values, 31 bits with sign are available (Bit #31 is reserved as the coding bit). In contrast, only 15 bits with sign are available per axis for transferring 2D correction values, whereby the x value lies in the lower 16 bits and the y value in the upper 16 bits of the value at the **McBSP interface**. For a description of the interface, see **Chapter 4.6.6 "SPI / I2C Socket Connector"**, Page 71.

Correction via McBSP Interface with Enhanced McBSP Input

As an alternative to the previous chapter's methods, you can also activate Processing-on-the-fly correction of linear motion with **set_multi_mcbasp_in** or **set_multi_mcbasp_in_list**.

These commands have an advantage over **set_mcbasp_in** or **set_mcbasp_in_list** in that they offer Processing-on-the-fly correction not only in the x direction and y direction, but also in the z direction and with laser power variation. Furthermore, up to 4 additional signals can be transmitted for usage as you wish.

Each 10 μ s, data asynchronously transmitted to **McBSP** memory locations 0 through 3 gets sorted and copied to an additional internal memory location. From there it is available for final usage and can be read-out sorted by signal types using **read_multi_mcbasp**.

8.6.3 Compensating Rotary Motions

Before activating Processing-on-the-fly correction for rotary xy motion, you must define the rotation center by **set_rot_center** or **set_rot_center_list**. The rotation center may also be situated outside the **Image field**.

The Processing-on-the-fly correction itself can be activated by:

- **set_fly_rot**
- **set_fly_rot_pos**
- **set_mcbssp_in** (Mode = 4)
- **set_mcbssp_in_list** (Mode = 4)

Processing-on-the-fly correction can be stopped by **fly_return** (see the corresponding notes on Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236).

Notes

- A Processing-on-the-fly rotation correction:
 - Cannot be combined with other Processing-on-the-fly corrections, see Section "Overview", Page 227.
 - Simultaneously for both scan head connectors is only practical, if both attached scan systems are aligned to exactly the same rotation center.

Correction via Encoder Counter

To be able to pass rotation angles via the board-internal encoder counters⁽¹⁾, encoder input port **ENCODER X**⁽²⁾ must be connected, see also Section "Input Ports for External Encoder Signals", Page 275.

Then, Processing-on-the-fly correction must be activated by **set_fly_rot**. The parameter **Resolution** [in counts per revolution] must thereby be specified. See also Section "Determining the Resolution Parameter", Page 232.

set_fly_rot resets the encoder counter "Encoder0" to zero.⁽³⁾

The output value is calculated from the current output position via a rotation matrix around the specified rotation center with rotation angle / $360^\circ = \text{current "Encoder0" counter reading} / \text{Resolution}$.

This correction is performed every $10 \mu\text{s}$.⁽⁴⁾

Determining the Resolution Parameter

The **Resolution** parameter can be determined experimentally:

- (1) Read out the counter start value by **get_encoder**.
- (2) Start the rotation.
- (3) Count the number of revolutions.
- (4) Stop the rotation.
- (5) Read out the counter end value by **get_encoder**.⁽⁵⁾
- (6) Calculate the parameter **Resolution** [in counts per revolution] as follows:

$$\text{Resolution} = \frac{(\text{counter end value} - \text{counter start value})}{\text{number of revolutions}}$$

$$\text{Resolution} = (\text{counter end value} - \text{counter start value}) / \text{number of revolutions}$$

If the workpiece rotates at a constant speed ω [in number of revolutions per second] and an encoder simulation has been activated by **simulate_encoder**, then the **Resolution** parameter can be calculated as follows:

$$\text{Resolution} = \frac{(1.000.000 \text{ counts} / \text{s})}{\omega}$$

- (7) Adjust the sign of **Resolution** to the direction of rotary motion.

(1) See Notice!, Page 49.

(2) **ENCODER X** = Encoder0.

(3) By **set_control_mode Bit #9** it can be set beforehand whether the counter is reset at the respective Processing-on-the-fly command or at the start trigger.

(4) The encoder counters are signed 32-bit counters with overflow (= after reaching the maximum (minimum) counter value, counting continues at the minimum (maximum) counter value).

(5) Alternatively, the counter start and end values can be stored in a cache on the RTC5 PCI Board by the list command **store_encoder** and then retrieved from there by the control command **read_encoder**.

Correction via McBSP Interface

If angle-position values for Processing-on-the-fly correction of rotary motion are forwarded by the **McBSP interface**, then Processing-on-the-fly correction must be activated by **set_fly_rot_pos**.

The required **Resolution** parameter has the same meaning as with **set_fly_rot** and is similarly determined, see [Section "Determining the Resolution Parameter"](#), [Page 232](#).

McBSP input values are read out by **read_mcbbsp**.

Notes

- **McBSP** input values get copied to board-internal memory location 0, see notes in [Section "Correction via McBSP Interface"](#), [Page 230](#).

Correction via McBSP Interface with Additional McBSP Input

A Processing-on-the-fly correction for rotary motions with **McBSP** input values can be activated by:

- **set_mcbbsp_in** (Mode = 4)
- **set_mcbbsp_in_list** (Mode = 4)

These commands offer the advantage of using the **McBSP interface** to input additional desired signals that should not be subjected to Processing-on-the-fly correction even when it is activated.

For this, all **McBSP** input values must be coded by Bit #31.

Notes

- All input values always get copied alternately to internal memory locations 1 and 2 and subsequently, in accordance with their Bit #31 coding, to internal memory locations 0 and 3. But after deactivation of correction by **set_mcbbsp_in(0)** or **set_mcbbsp_in_list(0)**, they only get copied to memory locations 1 and 2.
- You can query the data currently stored at internal memory locations 0 - 3 by **read_mcbbsp**.
- By **set_mcbbsp_in** or **set_mcbbsp_in_list**, a rotation resolution can be specified. Calibration for determining the rotation resolution is the same as for **set_fly_rot_pos**, see [Section "Determining the Resolution Parameter"](#), [Page 232](#).
- For transferring rotary correction values, 31 bits with sign are effectively available.

8.6.4 Compensating 2D Motions

You can use the **set_fly_2d** command to activate encoder-based 2D Processing-on-the-fly correction (for example, for a xy positioning stage), particularly for continuous marking in the virtual **Image field**.

Compared to an activation by **set_fly_x** and **set_fly_y**, this has the following advantages:

- **set_fly_2d** simultaneously resets both encoders (whereas the separate commands **set_fly_x** and **set_fly_y** do so with a slight temporal offset between both channels)
- **set_fly_2d** allows compensation of non-linear relations between encoder values and actual xy positioning stage motions, see [Section "2D Encoder Compensation for xy Positioning Stages", Page 234](#)
- Even during an interruption of a **set_fly_2d** marking by **wait_for_encoder_mode** or **wait_for_encoder_in_range**, the galvanometer scanner positions receive continuous Processing-on-the-fly correction in accordance with the current xy positioning stage encoder values (whereby the laser focus remains stationary relative to the xy positioning stage). Thus, unnecessary jumps are avoided after xy positioning stage forwarding motions, see [Chapter 8.6.6 "Virtual Image Field with Processing-on-the-fly", Page 237](#)

Notes

- You *cannot* combine **set_fly_2d** correction with other Processing-on-the-fly corrections, see [Section "Overview", Page 227](#).
- Coordinate transformations within the virtual **Image field**, see [Section "Coordinate Transformations in the Virtual Image Field", Page 158](#) can be activated when starting a **set_fly_2d-Processing-on-the-fly session** with changed values, if they have been loaded before with the corresponding control commands. As of version DLL 541, OUT 541 this is also possible with **set_fly_x** and/or **set_fly_y**.

2D Encoder Compensation for xy Positioning Stages

For particularly demanding marking requirements, it might be necessary to also compensate mechanical deviations of a xy positioning stage.

For this purpose, you can use the **load_fly_2d_table** command to load a 2D compensation table onto the RTC5 (see below).

Then – during a Processing-on-the-fly application initiated by **set_fly_2d** – encoder values are 2D interpolated and compensated just like with field correction tables.

To avoid unnecessary initialization motions, you can use **init_fly_2d** to define a desired positioning stage start position as the reference value for 2D encoder compensation (and to store it on the RTC5 PCI Board).

Upon every subsequent encoder reset by **set_fly_2d**, the current position is automatically applied as the new reference position, so that the relation between current encoder values and the absolute position of the positioning stage is retained for compensation. This relation remains even upon termination with **fly_return** or upon a new start with **set_fly_2d**.

This relation does not remain, if you meanwhile activate other Processing-on-the-fly corrections or reset the encoders by an external /START (see **set_control_mode** (Bit #9 = 1)).

At any time, you can query the currently valid reference values by **get_fly_2d_offset**.

Encoder compensation is applied exclusively to Processing-on-the-fly correction. The original uncompensated encoder values use:

- **get_encoder**
- **store_encoder**
- **read_encoder**
- **wait_for_encoder**
- **wait_for_encoder_mode**
- **wait_for_encoder_in_range**
- Recording by **set_trigger** (Signal1/Signal2 = 43 or 44)
- Recording by **get_value**

For **load_fly_2d_table**, you need to provide an ASCII text file that contains a compensation table. This table must have encoder compensations for reference points on a rectangular grid. The number of grid lines and their spacing is up to you (both may also differ in X and Y). Missing reference point data is automatically assigned a compensation of 0. The largest occurring encoder reference points form a frame within which encoder compensation values are bilinearly interpolated. Encoder values outside this frame is clipped to this frame prior to interpolation. Therefore, the reference points should cover at least the range of the xy positioning stage required by your application. Reference points with values exceeding ± 524288 can result in some loss of precision.

After **load_program_file**, 2D encoder compensation is inactive by default. It becomes active as soon as a valid table from an ASCII-readable text file gets loaded onto the RTC5 Board. You can subsequently deactivate it by calling **load_fly_2d_table** using a **NULL** pointer instead of the filename.

For the 2D compensation tables, the following rules apply:

The ASCII text file can contain one or several tables.⁽¹⁾

- The 2D compensation table must begin with the caption **[Fly2DTable<No>]**, whereby **<No>** corresponds to the number supplied with **load_fly_2d_table**.
- This is directly followed by a block of data points.
- If several tables with the same number exist, then only data from the first encountered table is read. The others are ignored.
- Reading of the text data terminates upon end of file or upon a line containing a caption.
- All characters to the right of a semicolon are treated as comments and ignored.
- The order of data points is up to you.
- The maximum length of a data line is 255 characters.
- A data line contains the two reference point coordinates for Encoder0 and Encoder1 (signed integers) and two compensation values (signed integers), each separated by spaces or tabs:

```
<Encoder0> <Encoder1> <Encoder0-delta>
<Encoder1-delta>
```
- If a data point reoccurs, then the most recently read value is used; the others are ignored.
- Empty lines or incomplete data lines are invalid and are ignored.

(1) Even of another type, see table 1, page 141.

8.6.5 Deactivating Processing-on-the-fly Corrections

All Processing-on-the-fly corrections can be deactivated (simultaneously for both spatial directions) by **fly_return**.

fly_return requires a new output position to be specified, which then is reached by a normal jump.

Processing-on-the-fly correction that has been activated by **set_multi_mcbasp_in** should be terminated by **fly_return_z**.

In all other cases (see below) a “Hard jump” to a new output position may occur.

Notes

- If Processing-on-the-fly correction is not explicitly deactivated⁽¹⁾, then:
 - it is also effective during execution of subsequent lists
 - it is not effective in the pause between two lists⁽²⁾
- Processing-on-the-fly correction enabled by **set_fly_x**, **set_fly_y**, **set_fly_2d**, **set_fly_rot**, **set_fly_x_pos**, **set_fly_y_pos** or **set_fly_rot_pos** also gets deactivated if the same command is called again but with invalid parameter values. This could lead to a jump to an uncorrected output position (also refer to the command descriptions).
- If Processing-on-the-fly has been activated by **set_fly_x**, then **set_fly_y** does not deactivate it and vice versa. The same applies to **set_fly_x_pos** and **set_fly_y_pos**. Other than that, every Processing-on-the-fly command automatically deactivates any Processing-on-the-fly correction activated by another Processing-on-the-fly command, see also [Section “Overview”, Page 227](#). This could lead to a “Hard jump” to a new output position.
- Processing-on-the-fly correction activated by **set_mcbasp_in** or **set_mcbasp_in_list** also gets deactivated if you call **set_mcbasp_in** with **Mode = 0** or **set_mcbasp_in_list** with **Mode = 0**. This could lead to a “Hard jump” to an uncorrected output position
- Processing-on-the-fly correction activated by **set_fly_x_pos**, **set_fly_y_pos**, **set_fly_rot_pos**, **set_mcbasp_in** or **set_mcbasp_in_list** also gets deactivated if you call the command for configuring [Online Positioning](#), see [Section “Notes”, Page 215](#). This could lead to a “Hard jump” to a new output position.
- A deactivation of a Processing-on-the-fly correction occurs only after the scanner delay.
- If Processing-on-the-fly has been activated by **set_mcbasp_in** or **set_mcbasp_in_list**, then the **fly_return** command deactivates Processing-on-the-fly correction, but it does not deactivate copying to internal memory locations (just like with **set_mcbasp_in(5)**). To afterward also deactivate copying to internal memory locations, you can use **set_mcbasp_in(0)** or **set_mcbasp_in_list(0)**.
- Processing-on-the-fly correction also gets deactivated (for both spacial directions simultaneously) by **stop_execution** or an [External Stop](#).

(1) **set_end_of_list** does not deactivate Processing-on-the-fly correction.

(2) Therefore, the correction does not affect the control commands **goto_xy** and **goto_xyz**.

8.6.6 Virtual Image Field with Processing-on-the-fly

With Processing-on-the-fly applications, the full value range (24-bit) of the virtual **Image field** can be used, see [Chapter 7.3.3 "Virtual Image Field", Page 158](#) to load and process command lists with objects that are up to 16 times larger than the real 20-bit **Image field**.

With 1D Processing-on-the-fly applications (for example, workpieces on a conveyor belt), objects can only exceed the real **Image field** in the very dimension parallel to the Processing-on-the-fly direction.

With 2D Processing-on-the-fly applications (for example, with an xy positioning stage), the to-be-marked objects of a command list may be distributed across the entire virtual **Image field**.

If an entire marking task consists of several partial markings ("tiles") whose extent does not exceed the real **Image field**, the Processing-on-the-fly functionality can also only be used to move the partial marking to be marked into the real **Image field** and then process it there.

Coordinate values which are still outside the real image *after* Processing-on-the-fly correction are clipped to the boundary values of the real **Image field**. Here, the **get_marking_info** error bits get set automatically, see [Chapter 8.6.9 "Monitoring Processing-on-the-fly Corrections", Page 240](#).

If there is a risk of the real 20-bit **Image field** being exceeded during list execution, specifically a forwarding motion of the conveyor belt or xy positioning stage should be executed. The forwarding motion can be waited for with **wait_for_encoder**, **wait_for_encoder_mode**, **wait_for_encoder_in_range** or **wait_for_mcbbsp** by interrupting the list execution until certain encoder values or **McBSP** values are reached, see [Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237](#).

The appropriate break up of the entire marking task and synchronization with the 1D or 2D Processing-on-the-fly motion is the task of the user program.

8.6.7 Synchronizing Processing-on-the-fly Applications

Processing of command lists can be started by either a RTC5 command or an external start signal, see [Chapter 6.4.4 "Starting and Stopping Lists", Page 98](#).

If the command list is not to be started immediately with the start signal, then an appropriate delay may be implemented as follows:

- When transferring positions via the **McBSP interface** – by a **wait_for_mcbbsp** at the beginning of the command list.
- When transferring positions via encoder counters – by a **set_fly_x/set_fly_y**, **set_fly_2d** or **set_fly_rot** (to reset the counter(s)) and a subsequent **wait_for_encoder_mode** or **wait_for_encoder_in_range** at the beginning of the command list.

wait_for_encoder_in_range is useful for 2D encoder-based Processing-on-the-fly applications. **wait_for_encoder_in_range** waits until both encoders are within the specified range at the same time and thus does not depend on the actual **Trajectory** that used to reach this range⁽¹⁾.

When transferring positions via encoder counters, a track delay can be configured (even independently of an active **Processing-on-the-fly session**) to delay the execution of the actual start relative to the triggering start signal or RTC5 command, see [Section "External Start", Page 266](#).

(1) If you would use **wait_for_encoder** or **wait_for_encoder_mode** instead, you would need to call these commands individually and consecutively for each encoder. Here, simultaneous fulfilment of both encoder criteria would depend on the xy positioning stage motion's explicit **Trajectory** and you would need to define and reproduce an appropriate **Trajectory** even before loading the lists.

External Starts triggered by an external start signal (or by **simulate_ext_start** or **simulate_ext_start_ctrl**) that do not execute immediately because of the track delay setting are held in a queue that can accommodate up to 8 starts (each start trigger is automatically generated when the delay has expired, see also [Section “External Start”, Page 266](#)). This helps avoid dead time between the execution of multiple (Processing-on-the-fly) list programs. Moreover, the list command **simulate_ext_start** can be used to execute several lists at defined intervals.

If **wait_for_encoder**, **wait_for_encoder_mode**, **wait_for_encoder_in_range** and **wait_for_mcbasp** are used to interrupt a list for intermediate forwarding motions, see [Chapter 8.6.6 “Virtual Image Field with Processing-on-the-fly”, Page 237](#), then the **Signals for “Laser Active” Operation** are not changed before the forwarding motion.

With encoder-based Processing-on-the-fly applications, the laser focus with **park_position** can be moved to a safe parking position before an interruption and back to the starting position or any other position after an interruption with **park_return**. See command description for more details.

The behavior of the galvanometer scanners during such a forwarding motion depends on the Processing-on-the-fly correction used:

- With **McBSP**-based Processing-on-the-fly correction as well as the encoder-based Processing-on-the-fly corrections **set_fly_x**, **set_fly_y** and **set_fly_rot**, the galvanometer scanners are stationary relative to the real **Image field** during list interruption. As a rule, a jump to the next marking must then be made.
- With the encoder-based **set_fly_2d** Processing-on-the-fly correction, the positions of the galvanometer scanners are continuously Processing-on-the-fly-corrected (the laser focus thus remains stationary relative to the to-be-marked object). The subsequent jump to the next marking or continuation of the same is usually very small.
- With **set_fly_2d**, clipping might occur if the Processing-on-the-fly-corrected coordinate values exceed the real **Image field** during a long forwarding motion. To avoid this, it might make sense to switch off Processing-on-the-fly correction before the forwarding motion (by **fly_return**, see [Chapter 8.6.5 “Deactivating Processing-on-the-fly Corrections”, Page 236](#); the galvanometer scanners then remain stationary). To switch correction back on after the forwarding motion, you can use **activate_fly_2d** instead of **set_fly_2d** (or **activate_fly_xy** instead of **set_fly_x** and **set_fly_y**). These commands do not reset the encoders, but instead convert the last Processing-on-the-fly-uncorrected coordinate values such that the Processing-on-the-fly-corrected output matches the current output. If an error occurs when restarting with these commands (for example, because the recalculated coordinate values would then be outside the 24-bit virtual **Image field**, or because another Processing-on-the-fly mode has been activated in the meantime), then an error bit gets set. This error bit can be read out by **get_marking_info**(Bit #9).

If, during list processing, it is necessary to immediately respond to this error, then the list command **if_not_activated** can be called to possibly jump to an error-handling routine.

- **activate_fly_2d_encoder** or **activate_fly_xy_encoder** can be used to continue at another positioning stage position when switching on again. They reset the encoders and simulate a positioning stage position using the encoder values passed as parameters. This avoids larger forwarding motions for initialization.

8.6.8 Encoder Resets

Many encoder-based Processing-on-the-fly applications (for example, a conveyor belt continuously traveling in the same direction) need to occasionally reset the encoders (for example, before a new marking sequence). For this, you can integrate Processing-on-the-fly reactivations into your lists by **set_fly_x**, **set_fly_y**, **set_fly_rot** or **set_fly_2d**.

In encoder-based Processing-on-the-fly applications with continuous marking (for example, using an xy positioning stage in the virtual **Image field**), however, the relationship between the encoder values and the absolute position of the xy positioning stage should usually be preserved.

For this purpose, **set_fly_2d** should be used for Processing-on-the-fly activation. Before a long interruption, you can deactivate Processing-on-the-fly correction with **fly_return**. And after the interruption you can use **activate_fly_2d** or **activate_fly_2d_encoder** to resume correction. See also Chapter 8.6.4 "Compensating 2D Motions", Page 234 and Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237. Processing-on-the-fly correction can also be resumed with **activate_fly_xy** or **activate_fly_xy_encoder**, but some functions are no longer available, particularly those necessary for the relation between current encoder values and absolute xy positioning stage positions, see the Section "2D Encoder Compensation for xy Positioning Stages", Page 234.

By **set_control_mode** Bit #9 it can be set beforehand whether the counter is reset at the respective Processing-on-the-fly command or at the start trigger.

8.6.9 Monitoring Processing-on-the-fly Corrections

If a Processing-on-the-fly user program is not optimized for the motion of the workpiece or scan system or if considerable unintended change of workpiece or scan system speed occurs during a Processing-on-the-fly operation, then the **Image field**'s boundaries might be reached. Because the RTC5 PCI Board clips coordinate values at the boundaries to prevent unallowed values, this could cause some parts of the to-be-marked pattern to not be scanned.

To allow user programs to monitor Processing-on-the-fly applications, the RTC5 PCI Board sets internal error bits (**Bit #0...Bit #3**) if the **Image field** boundaries are exceeded (and coordinate values get clipped). These can be read out by **get_marking_info**.

To avoid boundary exceedance, you should repeatedly call **get_marking_info** during test runs or normal operation of your Processing-on-the-fly application and modify your user program accordingly.

Notes

- The error bits **Bit #0...Bit #3** can be used to determine *which* edge of the **Image field** has been exceeded. Each boundary exceedance results in setting of the corresponding error bit.
- The error bits **Bit #0...Bit #3** are reset during initialization by **load_program_file** and by **get_marking_info**. Therefore, **get_marking_info** returns information about errors that occurred since the last initialization or the last call of **get_marking_info**. Subsequent transformations of type **#5a** and **#5b...#11**, see [Chapter 7.3.6 "Output Values to the Scan System"](#), [Page 170](#), are not taken into account here.
- During the adjustment phase of a Processing-on-the-fly correction, coordinate points at the edge of the **Image field** should therefore be approached and checked for possible limitations.
- The error bit **Bit #9** indicates if the 24-bit virtual **Image field** range has been exceeded during resumption of Processing-on-the-fly correction by **activate_fly_2d** or **activate_fly_xy**, see [Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications"](#), [Page 237](#).

Customer-Defined Monitoring Area

For Processing-on-the-fly applications, the RTC5 PCI Board also checks for exceedance of a second value range. Its boundaries can be specified by **set_fly_limits**.

Such exceedances likewise result in setting internal error bits (**Bit #4...Bit #7**). These can be read out by **get_marking_info**.

Moreover, the following conditional commands allow execution of any list command to be made dependent upon whether boundary exceedance in a customer-defined monitoring area occurred or not:

- **if_fly_x_overflow**
- **if_fly_y_overflow**
- **if_not_fly_x_overflow**
- **if_not_fly_y_overflow**

If the condition specified in the command parameter to the error bits **Bit #4...Bit #7** is fulfilled, the immediately following list command is executed. Otherwise, it is skipped.

Notes

- Boundary exceedance of a customer-defined monitoring area does not necessarily result in clipping of the output coordinate values. Clipping (and setting error bits **Bit #0...Bit #3**) only occurs if the maximum **Image field** (–524,288...+524,287 bit) is exceeded (the customer-defined monitoring area is typically smaller).
- The error bits **Bit #4...Bit #7** are reset during initialization (by **load_program_file**), but *not* by **get_marking_info**. Individual error bits get implicitly reset by the conditional commands and can also be explicitly reset by **clear_fly_overflow** or **clear_fly_overflow_ctrl** (see command description).
- The error bits **Bit #4...Bit #7** do *not* take into account transformations of type **#5a** and **#5b...#11**, see **Chapter 7.3.6 "Output Values to the Scan System"**, Page 170.
- Also for Rotation-fly applications it is possible to monitor a rectangular area, but not a rotation angle range.

8.6.10 Tracking Error Compensation of Encoder Values for Processing-on-the-fly Applications

Tracking errors of a scan system's galvanometer scanners can result in a certain amount of positioning inaccuracy, particularly for Processing-on-the-fly applications with variable encoder speeds.

Accordingly, you can activate Tracking error compensation by **set_fly_tracking_error** for applications requiring higher precision.

8.6.11 Processing-on-the-fly Correction for the z Axis

The encoder-based Processing-on-the-fly correction for the z axis (referred to as FlyZ correction in the following) can be activated by **set_fly_z**.

You can add FlyZ to Processing-on-the-fly correction for the x axis and y axis that had been activated by **set_fly_x/set_fly_y/set_fly_rot** or can be effective on its own.

This makes dynamic marking possible if the workpiece plane ($z = 0$) and the scan system do not move parallel to each other.

With FlyZ correction activated, the z component workpiece motion gets compensated by re-adjustment of the z axis (siehe **Dynamic focusing unit**) in accordance with the current encoder signals.

Prerequisites

- Both, **Option Processing-on-the-fly** as well as the **Option "3D"** must be enabled.
- The desired marking must function properly when static (that is, without workpiece motion):
 - The user program must model workpiece skew by using suitable 3D vector commands.
 - The varioSCAN must be capable of tracking the xy galvanometer scanner motions of the scan system.
- For moving workpieces, the **Dynamic focusing unit** must also be capable of following workpiece motion. Take care to exceed neither the maximum focus range in z direction nor the depth of field of the objective. Ensure that the precision of the correction table is sufficient in terms of edge sharpness, stretching etc. When used with **set_fly_rot**, ensure that the rotation angle across the **Image field** is small enough and the rotation center is sufficiently far away. Users are responsible for appropriate testing. SCANLAB provides no additional functionality for this. A virtual **Image field** for the z coordinate does *not* exist.
- See also the operational prerequisites for 3-axis scan systems in **Section "Connection and Initialization"**, Page 222.

Notes on Usage

- For general information on Processing-on-the-fly correction and determining scaling factors, see [Chapter 8.6 "Processing-on-the-fly", Page 227](#).
- When activating FlyZ correction by **set_fly_z**, you can specify which of the two encoder counters should be used. Because **set_fly_x** already uses encoder counter "Encoder0" and **set_fly_y** uses encoder counter "Encoder1" (**set_fly_rot** uses "Encoder0"), you might need to use one of the two encoders twice.
- FlyZ correction can be applied together with **set_fly_x/set_fly_y/set_fly_rot**.
- FlyZ correction can also be used alone (only encoder-based z variation).
- Activated FlyZ correction can be deactivated by **fly_return_z** or **fly_return**. As a result, all Processing-on-the-fly corrections for all three spatial directions are simultaneously deactivated. You can supply a command parameter for a new output position (only the x and y coordinates for **fly_return**, in addition the z coordinate for **fly_return_z**).
- Activated FlyZ correction can also be deactivated by **set_fly_z** in conjunction with an invalid parameter (for example, `set_fly_z( ScaleZ = 0 )`).
 - If only z correction has been activated here, then a jump to an uncorrected output position might occur.
 - If Processing-on-the-fly corrections were also activated for other spatial directions (by **set_fly_x/set_fly_y/set_fly_rot**), then these do not get deactivated here (and no jump to an uncorrected output position occurs).
- If correction for all three spatial directions is activated by **set_fly_x**, **set_fly_y** and **set_fly_z**, then a subsequent `set_fly_x( ScaleX=0 )` only deactivates X correction without affecting Y and Z correction (likewise for `set_fly_y(ScaleY=0)`). But if only z correction (by **set_fly_z**) or 2D correction (by **set_fly_z** and **set_fly_x** or **set_fly_y**) is activated, then a subsequent **set_fly_x**, **set_fly_y** or **set_fly_rot** with invalid parameter value (for example, `set_fly_x( ScaleX=0 )`) also deactivates z correction. In the latter case, a jump to a (partially) uncorrected output position might occur.
- **set_fly_z** deactivates a Processing-on-the-fly correction with positional values via **McBSP**.
- Conversely, Processing-on-the-fly correction activated by **set_fly_z** gets deactivated by such a correction. Here, a "Hard jump" to a new output position might occur.
- To avoid "Hard jumps", use only **fly_return_z** to deactivate Processing-on-the-fly correction.
- **get_marking_info** also provides information on possible range exceedances during FlyZ correction (**Bit #22...Bit #25**).
- **set_fly_limits_z**, **if_fly_z_overflow** and **if_not_fly_z_overflow** are available for defining a customer-specific monitoring range for FlyZ correction and for corresponding conditional command execution, see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#). You can use **clear_fly_overflow** and **clear_fly_overflow_ctrl** to reset the error bits (**Bit #24...Bit #25** from **get_marking_info**) for customer-specific monitoring of FlyZ applications.
- A 3D Processing-on-the-fly correction with positional values via **McBSP** can be activated by **set_multi_mcbasp_in** and **set_multi_mcbasp_in_list**.

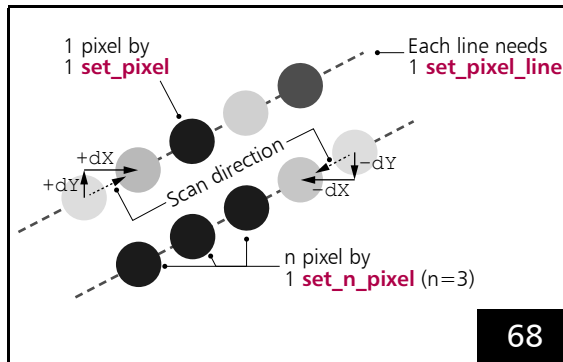
8.7 Pixel Output Mode

The **Vector commands** described in [Chapter 7.1 "Marking Dots, Lines and Arcs", Page 124](#) are intended for scanning vector based images. Furthermore, the RTC5 PCI Board allows marking pixel images (bitmaps). With suitably adjusted lasers, black-and-white images or greyscale images can be obtained. Raster and vector based images can be combined as desired.

8.7.1 Principle of Operation

Pixel images are created image line by image line, see [Figure 68](#). Each line consists of a number of equidistant pixels. A line is generated in a single pass. During this pass, the laser focus moves – as with a normal **[*]mark[*] Command** – at an (approximately) constant velocity along the entire image line (the motion is split-up into **Microsteps**). The individual pixels are marked in passing: each pixel gets a laser pulse assigned at the appropriate location. By varying the laser energy from pixel to pixel, greyscale images (including black & white images) are produced.

For controlling the laser, pulse lengths (digital) and voltage levels (analog) are outputted.



An individual **set_pixel_line** is required for each image line. The individual pixels of this line are then defined by successive **set_pixel/set_n_pixel**.

- By **set_pixel_line**, a single line is configured:
 - The *spatial* distance between the individual pixels with the parameters **dX**, **dY** (with **set_pixel_line_3d** additionally **dZ**)
 - The *temporal* distance between the individual pixels with the parameter **HalfPeriod**
 - Parameter **Channel** = ANALOG output port

Notes

- The **Pixel Output Mode** can be combined with Processing-on-the-fly, see [Chapter 8.6 "Processing-on-the-fly", Page 227](#).
- The **Pixel Output Mode** *should not* be used in conjunction with "Automatic Laser Control" (see [Chapter 7.4.9 "Automatic Laser Control", Page 187](#)) if "Automatic Laser Control" readjusts the 12-bit output values at the **ANALOG OUT1** or **ANALOG OUT2** output port or pulse lengths (**PulseLength**) or output periods (**HalfPeriod**) of the laser signals **LASER1** and **LASER2**.
- The **Pixel Output Mode** *cannot* be combined with Sky Writing, see [Chapter 7.2.4 "Sky Writing", Page 149](#).
- The **Pixel Output Mode** *cannot* be combined with Wobbel, see [Chapter 8.4 "Wobbel Mode", Page 217](#).

8.7.2 RTC5 Commands

Before writing an image line, a jump to the beginning of the line should be executed by a **Jump command**.

At the beginning of each image line, the **Pixel Output Mode** is activated by **set_pixel_line** or **set_pixel_line_3d**. The pixel distance and pixel output period (and, resultingly, the speed at which the image line is traversed) are simultaneously set as well.

The pixel output period is defined by the parameter `HalfPeriod` (half pixel output period). This is also half the laser period. The pixel distance between two adjacent pixels in the line – and thus also the marking direction – is defined by a:

- 2D vector (`dx,dy`) with `set_pixel_line`
- 3D vector (`dx,dy,dz`) with `set_pixel_line_3d`

Directly after `set_pixel_line/set_pixel_line_3d`, `set_pixel` has to be called separately for each pixel of the line. This defines the laser energies to be outputted at the corresponding pixel locations.

The length of a pulse and a 12-bit-valued analog voltage value must be specified, see [Chapter 8.7.3 "Laser Control", Page 246](#). Pixel pulses are always outputted at the LASER1 output port, even for the purpose of synchronizing the laser (gating).

As an alternative to a sequence of `n` identical `set_pixel` calls, `set_n_pixel` can be used.

Then only one command is stored on the RTC5 PCI Board, but during list processing the corresponding `set_pixel` is executed `n` times. Particularly for black & white images, this can drastically reduce the size of lists. Do not confuse `set_n_pixel` with `n_set_pixel` (multi-board version of `set_pixel`).

Prior to the end of an image line, no command other than `set_pixel` or `set_n_pixel` must be inserted into the list. After `set_pixel_line/set_pixel_line_3d`, the first list command that is *not* a `set_pixel` or `set_n_pixel` command stops the **Pixel Output Mode** and thus processing of the image line.

Each `set_pixel/set_n_pixel` command that does not follow another `set_pixel/set_n_pixel` or `set_pixel_line/set_pixel_line_3d` command is ignored during processing and is thus a short list command, see [Section "Normal, Short, Variable and Multiple List Commands", Page 278](#).

Notes

- The number of pixels in an image line is limited only by the capacity of the RTC5 PCI Board list memory, see [Chapter 15 "Technical Specifications – RTC5 PCI Board", Page 734](#). It is suggested – especially for large bitmaps – to set up a new list for each image line to avoid a list change during the execution of one line.
- Each image line must start with `set_pixel_line` or `set_pixel_line_3d`.
- The pixel distance (`dx, dy`) in the x direction and y direction (in bits) (and with `set_pixel_line_3d` also `dz`) can be specified with floating point numbers. This allows scaling and rotating the image without rounding errors.
- Depending on the **Pixel Output Mode**, the half pixel output period can be any integer-multiple of $1/64 \mu\text{s}$. It is independent of the position output period of $10 \mu\text{s}$, which is used to move the galvanometer scanners of the scan system (split-up into **Microsteps**). Very low pixel output frequencies result in multiple galvanometer scanner steps per pixel, higher frequencies result in multiple pixel pulses per galvanometer scanner step.
- You can implement a **Pixel Output Mode** in a $10 \mu\text{s}$ raster with variable speed and/or curvilinear paths with the help of `micro_vector[*]` commands, see [Chapter 8.8 "micro_vector\[*\] Commands", Page 249](#).

8.7.3 Laser Control

Depending on the type of laser employed, the laser energy outputted at each pixel position can be varied by:

- laser pulse length
- laser power per pixel

“Classic Mode”⁽¹⁾

The “Classic Mode” corresponds to the **Pixel Output Mode** of the RTC4. Maximum pixel output frequency: 308 kHz.

- Variation Of Laser Pulse Length: **set_pixel** defines – for each pixel – the length of the pixel pulse (in units of $1/64 \mu s$), which is outputted at the LASER1 port. For synchronization, see [Chapter 8.7.4 “Synchronization”, Page 247](#).
- Variation Of Laser Power: **set_pixel** allows specification of a 12-bit analog voltage level for each pixel. The value is transferred either to the **ANALOG OUT1** or **ANALOG OUT2** output port (see above). For synchronization, see [Chapter 8.7.4 “Synchronization”, Page 247](#). **set_n_pixel** defines the analog voltage level for n directly successive pixels of an image line.

Notes

- It is recommended that some experiments be performed to determine an appropriate gradation curve for producing smooth greyscales. The resulting pixel greyscale values (“colors”) strongly depend on the employed material and the laser.
- The LASERON signal – as with a normal **[*]mark[*] Command** – switches to “On” after the **LaserOn Delay** has expired and remains “On” for the entire pixel line. After the last pixel has been outputted, it automatically goes to “Off”. See [Chapter 7.4 “Laser Control”, Page 173](#).
- The LASER1 signal depends on the respective laser mode and the associated settings: a periodic signal with **HalfPeriod** from **set_pixel_line** and a **PulseLength** from **set_pixel/set_n_pixel** is outputted. The following exceptions apply:
 - In **Laser Mode 4** and **Laser Mode 6**, only a standby signal is outputted that cannot be varied. Power changes are only possible via the output ports **ANALOG OUT1** and **ANALOG OUT2**.
- The LASER2 signal follows the specifications of the respective laser mode.
- No Softstart is performed, see [Chapter 7.4.7 “Softstart Mode”, Page 184](#).
- **Pixel Output Mode** and **Sky Writing** cannot be combined.
- The values for **HalfPeriod** and **PulseLength** set prior to **set_pixel_line** are not restored again. **set_laser_pulses** must be explicitly called again.

(1) RTC6 supports additional modes.

8.7.4 Synchronization

The pixel output timing diagram for one image line with 3 pixels is shown in [Figure 69](#).

To prepare the laser control, [set_pixel_line/set_pixel_line_3d](#) switches off the Signals for "Laser Active" Operation from a previous [Mark command](#) after a [LaserOff Delay](#) (as with a [Jump command](#)) and waits until the laser is actually off.

During this waiting period, the galvanometer scanners do not move.⁽¹⁾ Initial acceleration phases can be hidden by a [LaserOn Delay](#) (as with a normal [\[*\]mark\[*\] Command](#)) or by a corresponding number of "idle pixels" (see below).

After that, depending on the laser mode, pixel output starts immediately or after a Q-Switch delay, see [Figure 69](#). Analog signals at [ANALOG OUT1](#) or [ANALOG OUT2](#) change synchronously with the leading edge of each pixel pulse. The digital-to-analog converter requires about $1.5\ \mu\text{s} \dots 3\ \mu\text{s}$ to produce a stable analog output signal. With pixel output frequencies above around 100 kHz ($\text{HalfPeriod} < \text{approx. } 320$) digital-to-analog conversion cannot always be fully completed. At such pixel output frequencies, it must be carefully checked whether the functionality is sufficient for the intended purpose.

Bit #1 in [set_laser_control \(Ctrl \)](#) can be used to shift the laser pulse and thus the start of the digital-to-analog conversion by half a pixel period.

The pixel line ends with the first list command that is not a [set_pixel](#) or [set_n_pixel](#).

For the laser to be switched off even in the middle of a $10\ \mu\text{s}$ cycle, a default pixel is automatically outputted after the last pixel. This is repeated as often as necessary until a started $10\ \mu\text{s}$ cycle is finished. Then the laser is finally switched off.

The default pixel should be defined appropriately prior to [set_pixel_line/set_pixel_line_3d](#) in order to achieve a non-visible laser marking (= "idle pixels") in the run out, see [set_port_default](#) and [set_default_pixel](#).

The RTC5 board waits – especially at pixel output frequencies $< 100\ \text{kHz}$ – until the default pixel is outputted.

The galvanometer scanners continue to run during this time and stop afterwards.⁽¹⁾

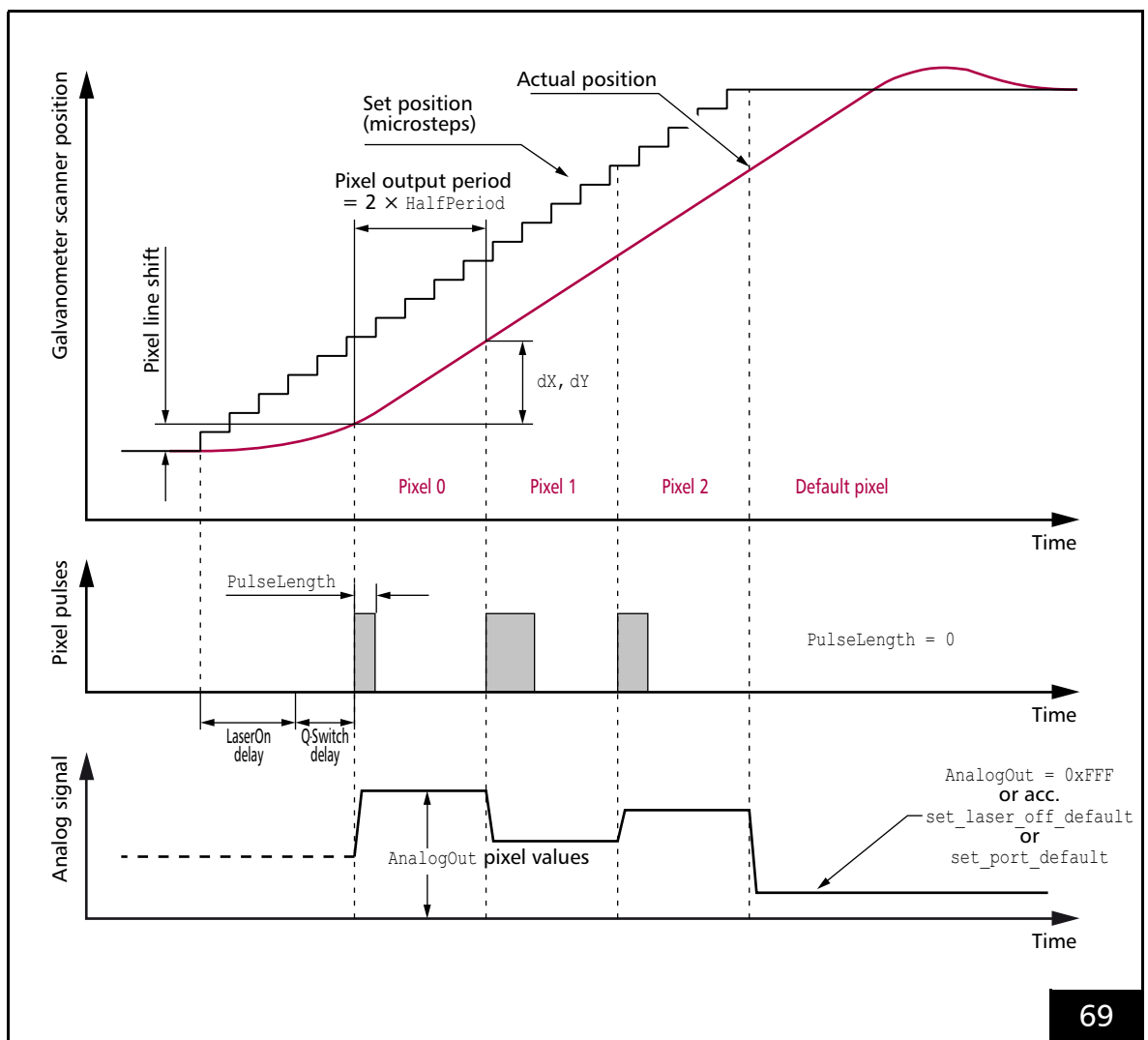
No scanner delay is automatically inserted.

The [Tracking error](#) and the hidden acceleration phase mean a pixel line shift", see [Figure 69](#). Normally, this needs to be compensated for by an adjusted run-in.

(1) The [set_pixel_line](#) parameter [Channel](#) + 256 can be used to suppress the stopping in the run-in phase and run-out phase. Then the galvanometer scanners move on (with the speed defined by [HalfPeriod](#) and pixel distance) in equidistant [Microsteps](#). In this way, jerk-free run-in and run-out motions can be programmed.

Notes

- With $\text{HalfPeriod} < \text{PulseLength} / 2$ the laser does not switch off between the pixels.
- When specifying HalfPeriod and pixel distance, observe the dynamics of the scan system (mark speed) and the properties of the laser system (power modulation).
- Output synchronization cannot be used together with **Pixel mode**. You should first deactivate output synchronization.



Timing of the galvanometer scanner positions and the laser control signals in the **Pixel mode** Mode = 0 and a YAG Mode, see [Chapter 7.4.4 "YAG Mode 1, YAG Mode 2, YAG Mode 3, YAG Mode 5"](#), Page 179. The pixel output period in this example is approx. 4.5 Microsteps.

8.8 **micro_vector[*]** Commands

micro_vector[*] commands move the galvanometer scanners directly to the specified position by a “Hard jump”

- in 2D Image field:
 - **micro_vector_abs**
 - **micro_vector_rel**
- in 3D image field:
 - **micro_vector_abs_3d**
 - **micro_vector_rel_3d**

The Signals for “Laser Active” Operation can be switched on and off individually for each **micro_vector[*]** command by the specified **Laser Delays** (parameters **LasOn** and **LasOff**). The **Laser Delays** defined by **set_laser_delays** are not overwritten by that. These remain valid for normal **Jump commands** and **[*]mark[*]** Commands.

The **micro_vector[*]** commands can be used to mark trajectories (see Glossary entry on page 26) of arbitrary shape and at variable speed, as long as the dynamic limits of the scan system are not exceeded.

Notes

- **micro_vector[*]** commands wait for a preceding scanner delay. They never trigger new **Scanner Delays**.
- Users themselves are responsible for appropriately parameterizing **micro_vector[*]** commands:
 - Complying with the dynamic limits of the scan system
 - Avoiding cross over and take over with **LaserON** and **LaserOff**, see Chapter 7.2.3 “Notes on Optimizing the Delays”, Page 143
- A Wobbel motion specified by **set_wobbel** or **set_wobbel_mode**, see Chapter 8.4 “Wobbel Mode”, Page 217 is not taken into account (but also not deactivated).
- The **Sky Writing** mode is not taken into account (but also not deactivated).
- The output values are calculated according Chapter 7.3.6 “Output Values to the Scan System”, Page 170 without 1 and 2.

8.9 Timed Commands

"Normal" **Vector commands** and "**Arc**" commands are executed in such a way that the laser focus moves with a defined speed⁽¹⁾. This is fine for most laser marking and laser material processing applications.

At the beginning of a jump, the **Signals for "Laser Active" Operation** are switched off. The **Signals for "Laser Standby" Operation** are switched on depending on the settings, see **Chapter 7.4.1 "Enabling, Activating and Switching Laser Control Signals"**, Page 173.

However, some applications require that each **Vector command** or "**Arc**" command consumes exactly the same amount of time, regardless of its spacial length.

In this case, it is necessary to specify a jump *duration* or mark process *duration* (rather than the jump speed or mark speed).

Timed commands allow specification of the duration of the **Vector command** or "**Arc**" command with an accuracy of 10 μs (the output period of the **Microsteps**) and in the range from 10 μs to 167,772,160 μs (≈ 2.8 min).

A vector or arc is split-up into the specified ($\mathbb{T}$ Parameter) number of **Microsteps**.

For $\mathbb{T} \geq 5$, the following applies:

$$\begin{aligned} \text{Length } \Delta s \text{ of a Microstep} &= L / t \\ \text{with } t &= \text{integer}((\mathbb{T} + 5) / 10)^{(2)} \end{aligned}$$

Thus, jump speed and mark speed speed automatically depend on the length of the vectors or arcs.

The following timed commands are available:

- Vectors and arcs
 - **timed_jump_abs**
 - **timed_jump_rel**
 - **timed_mark_abs**
 - **timed_mark_rel**
 - **timed_arc_abs**
 - **timed_arc_rel**
- Parametrized vectors
 - **timed_para_jump_abs**
 - **timed_para_jump_rel**
 - **timed_para_mark_abs**
 - **timed_para_mark_rel**
- 3D vectors
 - **timed_jump_abs_3d**
 - **timed_jump_rel_3d**
 - **timed_mark_abs_3d**
 - **timed_mark_rel_3d**
- Parametrized 3D vectors
 - **timed_para_jump_abs_3d**
 - **timed_para_jump_rel_3d**
 - **timed_para_mark_abs_3d**
 - **timed_para_mark_rel_3d**

Notes

- After a timed command, a "normal" **Jump Delay**, **Mark Delay** or **Polygon Delay** is inserted.
- Ellipses are fundamentally not available as timed commands.
- With $\mathbb{T} < 5$ μs , a timed command is synonym to its "normal" (= without [**timed_***]) command.
- The total execution time of a timed command is the sum of the specified time and the associated delay, see also **Chapter 7.2.2 "Scanner Delays"**, Page 135.

(1) Either jump speed or mark speed.

(2) For $\mathbb{T} < 5$, the command is carried out as non-timed command, see **Chapter 7.1.2 "Microstepping"**, Page 128.

8.10 Automatic Self-Calibration

- This functionality does not apply to current SCANLAB scan systems
- Previously, some SCANLAB scan systems have been equipped with an additional internal sensor system for automatic self-calibration (ASC sensor system, home-In sensors) to meet higher requirements for long-term repeatability

8.10.1 Using for Drift Compensation

Long-term repeatability is very important in many applications, for example, for rapid prototyping in which the processing operation can span several hours. For such laser applications, the galvanometer scanner's long-term drift and temperature drift, which manifest as a shift (offset drift) and increase or decrease in the size (gain drift) of the working **Image field**, can exceed the allowed tolerances.

In such applications, it is helpful to start up the application only after the galvanometer scanners have reached their operating temperature. In addition, the magnitudes of environmental fluctuations (for example, operating temperature changes to which the scan system is exposed) should be kept as small as possible and the scan system preferably operated with a constant load.

For higher long-term repeatability requirements, SCANLAB scan systems may have been equipped with an additional internal sensor system for automatic self-calibration (ASC sensor system, Home-In sensors).

The ASC sensor system (see above) allows the position detectors of the galvanometer scanners to be calibrated and gain drift and offset drifts to be reliably compensated. The positioning accuracy is thus maintained over long periods of time. Remaining long-term drift effects are of the same order of magnitude as short-term repeatability accuracies.

8.10.2 How it Works

By **auto_cal**, a measurement routine can be started for determining the exact control values for reference positions (Home-In positions) defined by the internal sensor system.

For drift *measurement*, the routine should be executed:

- When setting up the equipment – to determine reference values for the Home-In positions, see [Chapter 8.10.3 "Determining Reference Values"](#), Page 253
- During the application at appropriate time intervals – to determine if and how the Home-In positions have changed, see [Chapter 8.10.4 "Calibration During the Application"](#), Page 253

For drift *compensation*, appropriate gain values and offset values can be calculated and set separately for each galvanometer scanner based on the determined deviations between the current Home-In position and the reference value. See also #11 in [Chapter 7.3.6 "Output Values to the Scan System"](#), Page 170.

The setting can also be made manually with **set_hi**, if customer-specific measuring procedures are to be used, see [Section "Customer-Specific Calibration"](#), Page 254.

Notes

- Prior to performing a measurement routine, **auto_cal**(*Command* = 4) can be used to check if:
 - the attached scan system in fact has an internal sensor system for automatic self-calibration (Home-In sensors)
 - the sensor system is functioning properly
 This ASC hardware check also occurs automatically by **auto_cal**(*Command* = 0) and if required, for **auto_cal**(*Command* = 1 and 3), see also **auto_cal** command description and **get_auto_cal**.
- During execution of the measurement routine determining the Home-In positions:
 - the laser should be switched off
 - no other commands should be sent to the scan system
- For Gain/Offset Correction:

See [Chapter 7.3.6 "Output Values to the Scan System", Page 170 #11](#).

After an initialization by **load_program_file** or **auto_cal**(*Command* = 2), the following is set:

 - Gain = 1.0
 - Offset = 0
- For iDRIVE scan systems⁽¹⁾, the following applies in addition:
 - At the beginning of a measurement routine, the **XY2-100 status word** is automatically set as to-be-returned by the scan system. At the end of the measurement routine, the previously set data type is restored.
 - **auto_cal** aborts with an error result value 7 if
 - **auto_cal** is to be executed for the first scan head connector, and
 - an "Automatic Laser Control" in *Mode* = 2 (basic mode) has been activated, see [Section "Speed-Dependent Laser Control", Page 191](#).
The "Automatic Laser Control" must be deactivated before performing automatic self-calibration.
 - **auto_cal** also aborts (result value 1, 10 or 11), if the scaling factor has been set by **control_command**(*Data* = 12xx_H) to a value < 1. This is because the sensor positions then are not reachable, see also **control_command**(*Data* = 053F_H). The scaling factor must have been set to 1 (default setting) by prior to automatic self-calibration
 - **control_command**(*Data* = 1283_H) and
 - **control_command**(*Data* = 1200_H).

(1) See Glossary entry on [page 23](#).

8.10.3 Determining Reference Values

The reference values are determined by **auto_cal** (**Command** = 0). This starts the measurement routine for determining the current Home-In positions. These are then the reference values for later calibrations with **auto_cal** (**Command** = 1 or 3). Immediately after determination, they can be read out by **get_hi_pos** and are available for customer-specific calibration, see [Section "Customer-Specific Calibration", Page 254](#). The reference values are stored in the EEPROM. This guarantees that the reference values are available even after a restart.

Notes

- After **auto_cal**(**Command** = 0), the galvanometer scanners are in the same real **Image field** position as before the command.
- The reference values should be determined when the machine is set up, for example, when the scan system is installed or exchanged. When exchanging the scan system, for which reference values have already been determined earlier and read out by **get_hi_pos**, these reference values can be transferred directly to the RTC5 PCI Board by **write_hi_pos**.
- Reference values should be determined under conditions (ambient temperature, load) typical for the application and after the overall system has fully attained its operating temperature. The reference values should always be determined only after a warm-up time of more than 20 minutes and not before the TempOK signal has been activated.
- Execution of **auto_cal**(**Command** = 0) typically lasts up to 10 seconds.

8.10.4 Calibration During the Application

Automatic Self-Calibration

Automatic self-calibration of the scan system during an application can be executed with **auto_cal**(**Command** = 1).

This starts a measurement routine for determining the current Home-In positions. From the deviations from the stored reference values (see [Chapter 8.10.3 "Determining Reference Values", Page 253](#)) determined in the process, gain values and offset values are calculated and set separately for the x axis and y axis.

This drift compensation is effective immediately. It is valid until:

- **auto_cal**(**Command** = 1) is called again
- it is switched off
 - by **auto_cal**(**Command** = 2)
 - after **load_program_file**

Notes

- After **auto_cal**(**Command** = 1) the galvanometer scanners are in the same position in the real **Image field** as before the command.
- The calibration routine started with **auto_cal**(**Command** = 1) typically lasts 1...2 seconds (depending on the strength of the drift). An ASC hardware check is automatically performed, if **auto_cal**(**Command** = 0 or 4) has not been executed prior to the first time call of **auto_cal**(**Command** = 1). This extends the execution of **auto_cal** by a few seconds.

Customer-Specific Calibration

Automatic self-calibration is midpoint centered, that is, the **Image field** center remains stable.

If an alternative is preferred (for example, if the left edge should remain stable, size is irrelevant), then the calibration can also be externally calculated and set.

For such a customer-specific calibration, the Home-In positions determined by **auto_cal** (**Command** = 0) can be read out by **get_hi_pos**. From this, compensating gain values and offset values can be calculated and set by **set_hi**. The drift compensation set in this way is effective immediately.

Notes

- **get_hi_pos** returns the last measured Home-In positions. These are the stored Home-In reference values immediately after
 - **auto_cal** (**Command** = 0)
 - initialization by **load_program_file**
- After **set_hi**, a correction of the current position occurs at jump speed.
- Gain and offset correction factors can also be set by **set_hi** for systems *without* Home-In sensors.
- After **auto_cal** (**Command** = 3) the galvanometer scanners are in exactly the same position as before the command.

Supplemental Information about Calibration

- The accuracy of fit of the set drift compensation can decrease with increasing time. Therefore, calibration should be repeated after appropriate time intervals. The shorter the time interval between individual calibrations, the higher the attained long-term repeatability. Time intervals are typically in the range of minutes. Events such as workpiece changes or line feeds are ideal opportunities for conducting a new calibration.
- The accuracy of fit of the calibration, and the thereby attained long-term repeatability, are further enhanced by steady environmental and load conditions.
- The measurement routine determines the Home-In positions by several measurements. If deviations between the individual measurements are too large (maximum – minimum > 96 bits), the measurement routine aborts and an error 2 is returned. In this case, no reference values are

stored for the affected axis (**Command** = 0) and the gain and offset factors remain unchanged with (**Command** = 1)⁽¹⁾. Then **get_hi_pos** returns 0 instead of the faulty values.

- An error within an individual measurement cycle can be caused by a brief mechanical or electrical disturbance. If significant spreading occurs within an individual measurement cycle, we recommend the following:
 - either a further measurement cycle is immediately conducted or
 - the error is initially ignored while using the current gain values and offset values until new correction values are determined by the next (successful) measurement cycle.
- Significant spreading across several measurement cycles might indicate a defect in the internal sensor system or another part of the scan system. But because continuous (mechanical or electrical) external disturbances or contamination can also impair automatic self-calibration, the scan system and its environment should in such cases be appropriately inspected to assess overall functionality.
- Certain hardware error states (for example, signal faulty or not found) get permanently stored. After correcting the error, you must call **auto_cal** with **Command** = 0 or 4 to clear the error state. Until this time, correction with **Command** = 1 or 3 is not possible.

(1) (**Command** = 0) always sets gain = 1.0 and offset = 0.

8.11 Camming

camming produces a marking that simulates the classic camshaft action of moving a valve tappet – or more generally a cam disk moving a lever. An example for a **Camming** process is shown in Figure 70.

The galvanometer scanner motion here is a lever motion defined as a 2D curve. It is written in a list as a closed point-by-point sequence of **mark_abs** commands.

The entire curve must fit within a contiguous list region, see **config_list**. Though it is not possible to switch among lists to load further portions of the curve.

“Propulsion” is furnished either by external fed in encoder pulses or by internally simulated ones.

Each outputted point is derived from the xy coordinate of a **mark_abs** at the position $\text{FirstPos} + \text{Index}$, whereby $\text{Index} = \text{Round}((\text{EncoderCurrent} - \text{EncoderStart}) \times \text{Scale})$. **FirstPos** is the first **mark_abs** command’s list position and **Scale** is a freely selectable scaling factor. **EncoderStart** gets automatically determined when **camming** is called. There is no automatic encoder reset. The first outputted point is always $\text{Index} = 0$.

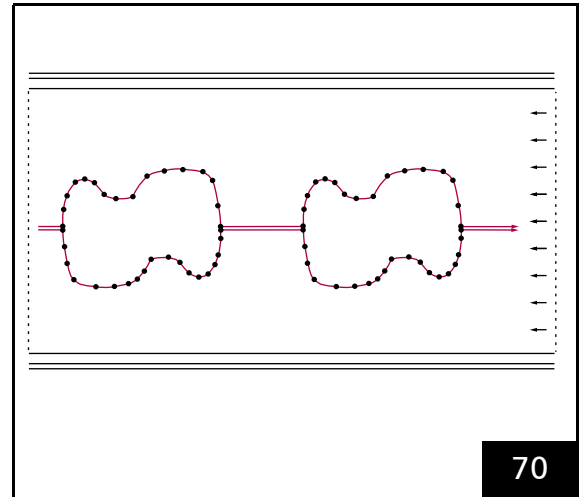
The individual points are executed every $10 \mu\text{s}$ as a “Hard jump” without **Scanner Delays**.

The distance between two points (= “resolution”) is freely selectable.

Scale determines how precisely the curve is sampled. The larger **Scale** is, the coarser is the piecewise linear approximation of the curve.

The number of encoder pulses per $10 \mu\text{s}$ clock cycle and the spacing of the points determine the actual mark speed.

“Resolution”, **Scale** and encoder speed should suit the dynamic characteristics of the connected scan system.



Camming process example. A transport system moves a continuous workpiece. Two *scan heads* team up to mark the contours. Schematic depiction.

The **Camming** process can be controlled in various ways, see **camming** command description:

- **Ctrl** > 0
The laser is controlled externally (with **laser_signal_on_list** before **camming** and **laser_signal_off_list** after **camming**)
- **Ctrl** = 0
The laser is controlled RTC5 board-internally automatically (as with a normal **Polyline** with consideration of **Laser Delays**)

The curve can be executed once and then automatically ended (**Ctrl** = 0 or **Ctrl** = 1). The list then continues by executing the next command that follows the end of the point list (the length of the point list is defined by **NPos**).

In accordance with encoder direction point lists can also run backward. Then, the curve is terminated with **Index** = 0.

The curve can be repeated indefinitely (**Ctrl** = 2 or **Ctrl** = 3). To avoid “Hard jumps” at the start of a repeat, the point list should represent a closed curve. Indefinite repeating must be cancelled by **stop_execution** or an external /STOP.

At any time, the curve can be restarted at the address defined by **set_extstartpos** or **set_extstartpos_list** (typically but not necessarily `FirstPos`) by an external /START (independently of the current index, possibly hard-jumped to the first marking position).

Notes

- An external /START during list execution is:
 - Suppressed during normal operation
 - Not suppressed during a **Camming** process with indefinite repeating

If `Ctrl` = 2, then the **Camming** process waits at the end of the point list for a new start. If `Ctrl` = 3, then processing automatically starts again from the beginning (the point list is processed as a ring buffer).

Furthermore, the **Camming** process can be combined with Processing-on-the-fly (which can apply its own scaling if different from **camming** parameter *Scale*).

The laser control can be combined with the automatic “vector-controlled laser control” (**set_vector_control**). This requires to use **para_mark_abs[*]** commands instead of **mark_abs[*]** commands. The value defined there is outputted immediately, thus allowing systematic variation of laser power along the curve. If automatic vector-defined laser control is not enabled, then no laser power is outputted by the **para_mark_abs[*]** commands.

In addition, a combination with Encoder-speed-dependent laser control is possible, see [Section “Encoder-Speed-Dependent Laser Control”, Page 192](#).

Alternatively, **mark_abs_3d** or **para_mark_abs_3d** as well as **timed_mark_abs_3d** can be used.

However, the z coordinate and `T` parameter is ignored during outputting. Other commands within point lists are likewise ignored.

Point output then remains unchanged for one clock cycle.

Notes for Testing

- If a curve point list is finished by **set_end_of_list** and does not cross the boundary between List 1 and List 2 (see [Chapter 6.4 “List Handling”, Page 94](#)), then the curve shape can be tested using a normal execution start, for example, by `execute_at_pointer( Pos )` (to some extent as a **Polyline**, but with a predefined mark speed and using the currently defined “Variable **Polygon Delay**”).
- The point list should be closed with **list_return**, if it is part of a subroutine and a **sub_call** call is used for testing.
- If the test should include the “Hard jump”, then **timed_mark_abs_3d** (with `Time` = 10 μ s) can be used as well, but not **timed_para_mark_abs_3d**.

8.12 Time Measurements

8.12.1 RTC5 Timer

RTC5 boards are equipped with an integrated timer, which is referred to as the “RTC5 Timer” in the following. The RTC5 Timer only counts clock cycles of control commands. Counting is paused during interruptions by **set_wait** or **pause_list**.

In order to measure the marking time consumed by any particular marking process, **save_and_restart_timer** is called before and then after the marking process. **save_and_restart_timer** saves the present RTC5 Timer value and resets it to 0. The elapsed time can then be read by **get_time**, which returns the RTC5 Timer value saved during the most recent call of **save_and_restart_timer**.

The present RTC5 Timer value can be read out by **get_lap_time**. It returns the elapsed time since the last call of **save_and_restart_timer** but without resetting the RTC5 Timer to zero. In this way the interim execution time of lengthy marking processes can be monitored.

8.12.2 Timestamps

32-bit “Timestamp Counter”

As of DLL 543, OUT 543 a 32-bit “Timestamp Counter” is additionally available. It starts counting at 0 with **load_program_file** and counts uninterruptible all 10 μ s clock periods.

Using the 32-bit “Timestamp Counter”, an absolute reference to the individual time periods can be established, even if the RTC5 Timer has been reset by **save_and_restart_timer**. The 32-bit “Timestamp Counter” can be recorded by the trigger signal 52 (see **set_trigger** or **set_trigger4**) and is read-out by **get_value** ⁽⁵²⁾.

Notes

- **get_time** and **get_lap_time** only take list execution times into account (because the RTC5 Timer only counts list command clock cycles and pauses during interruptions by **set_wait** or **pause_list**).
- To compare RTC5-internal **save_and_restart_timer** time measurements to external time measurements by the BUSY pin, you should insert a **list_nop** between **save_and_restart_timer** and **set_end_of_list**. This ensures that any scanner delay is processed before **set_end_of_list**. Without **list_nop**, **save_and_restart_timer** includes the scanner delay in its measurement even though it completes only after **set_end_of_list** (and therefore the BUSY pin is already LOW).

9 Programming Peripheral Interfaces

Scan systems are often used in equipment that needs to synchronize processing by the laser and scan system with other process steps (for example, workpiece placement, robotic motion, process monitoring etc.).

For this purpose, the RTC5 PCI Board provides a variety of peripheral interfaces, see [Chapter 4.6 "Interfaces for the Laser and Peripheral Equipment"](#), [Page 60](#).

With the commands for programming these interfaces, you can supplementally and/or synchronously control the following in addition to lasers and scan systems:

- Signals transmitted for peripheral control
- Querying and evaluation of peripheral signals
- Control and synchronization of laser scan processes and peripheral control by external control signals

9.1 Signal Output

For peripheral control (for example, controlling a workpiece transport system or a shutter), appropriate signals can be outputted by the interfaces described below.

The output values can be changed at any time by control commands or – during processing of a list – by list commands.

9.1.1 16-Bit Digital Output Port

The EXTENSION 1 socket connector provides a buffered 16-bit digital TTL output (DIGITAL OUT0...15). The level of its output signals must be configured with a jumper (see [Section "16-Bit Digital Input Port and 16-Bit Digital Output Port"](#), [Page 65](#)).

The [write_io_port_list](#), [write_io_port](#), [write_io_port_mask_list](#) and [write_io_port_mask](#) commands specify the digital output values. The output is in high-impedance state until an initial value is assigned to it. In addition, [set_port_default](#) (Port = 3) can be used to define the value to be outputted at the 16-bit digital output port, as soon as processing of a list has ended with [stop_execution](#) or by an external stop signal.

The default value also takes effect:

- With position-dependent and speed-dependent laser control (see [set_port_default](#))
- Upon terminating [Pixel mode](#)

If "Automatic Laser Control" is activated with [Ctrl=6](#) from [set_auto_laser_control](#), then the value at the 16-bit digital output automatically gets adjusted, see [Chapter 7.4.9 "Automatic Laser Control"](#), [Page 187](#). You can log this by [set_trigger/set_trigger4](#) (signal 24).

When the output value is outputted, a LATCH signal is outputted at the EXTENSION 1 socket connector as a trigger signal for synchronization of data transmission.

The [get_io_status](#) command reads the current value of the digital output port.

9.1.2 8-Bit Digital Output Port

The EXTENSION 2 socket connector provides a (jumper-configurable) buffered 8-bit digital output port (DATA0 to DATA7) (see [Section "8-Bit Digital Output Port", Page 68](#)).

Its output values can be set by **write_8bit_port** or **write_8bit_port_list**. The output is in high-impedance state until an initial value is assigned to it. In addition, the commands **set_port_default** (Port = 2) or **set_laser_off_default** can be used to define the value to be outputted at the 8-bit digital output port, as soon as processing of a list has ended with **stop_execution** or by an external stop signal.

The default value also takes effect:

- With position-dependent and speed-dependent laser control (see **set_port_default**)
- Upon terminating **Pixel mode**

If "Automatic Laser Control" is activated with **Ctrl = 3** from **set_auto_laser_control**, then the value at the 8-bit digital output automatically gets adjusted, see [Chapter 7.4.9 "Automatic Laser Control", Page 187](#). This can be recorded by **set_trigger/set_trigger4** (signal 24).

When the output value is outputted, a LATCH signal is outputted at the EXTENSION 2 socket connector as a trigger signal for synchronization of data transmission (provided that pin (17) has been correspondingly configured by the jumper setting, see [Section "8-Bit Digital Output Port", Page 68](#)).

9.1.3 2 Bit Digital Output Port

The LASER connector provides a buffered 2-bit digital output port (DIGITAL OUT1 and DIGITAL OUT2), see [Section "2-Bit Digital Output Port", Page 61](#).

By **set_laser_pin_out** or **set_laser_pin_out_list**, values are assigned to this output port.

The value is 0 (pins are LOW) until an initial value is assigned to it.

In addition, **set_port_default** (Port = 4) can be used to define the value to be outputted at the 2-bit digital output port, as soon as processing of a list is cancelled either by **stop_execution** or an external stop signal.

9.1.4 12-Bit Analog Output Port 1 and 2

The LASER connector provides the two 12-bit analog output ports, **ANALOG OUT1** and **ANALOG OUT2**, see [Section "12-Bit Analog Output Port 1 and 2", Page 61](#).

The **ANALOG OUT2** output port is also available by the MARKING ON THE FLY socket connector, see [Chapter 4.6.4 "MARKING ON THE FLY Socket Connector", Page 69](#).

The output values of the output ports can be separately set by **write_da_x** or **write_da_x_list**. The values are 1 (corresponds to approx. 0 V) until initial values are assigned to it.

In addition, the commands **set_port_default** (Port = 0 or 1) or **set_laser_off_default** can be used to define the values to be outputted at the 12-bit analog output ports, as soon as processing of a list has ended with **stop_execution** or by an external stop signal. The default value also takes effect together with the position-dependent or speed-dependent laser control (see **set_port_default**). If accordingly preset by corresponding commands, the values at the analog output ports are also changed:

- By a softstart, see [Chapter 7.4.7 "Softstart Mode", Page 184](#)
- In **Pixel Output Mode**, see [Chapter 8.7 "Pixel Output Mode", Page 244](#)

If "Automatic Laser Control" is activated with **Ctrl = 1** or **Ctrl = 2** from **set_auto_laser_control**, then the value at one of the 12-bit analog output ports automatically gets adjusted, see [Chapter 7.4.9 "Automatic Laser Control", Page 187](#). This can be recorded by **set_trigger/set_trigger4** (signal 24).

9.1.5 Controlling Stepper Motors

Output Signals

The signals (ENABLE, DIRECTION and CLOCK) for controlling up to two stepper motors are outputted at the "STEPPER MOTOR" socket connector⁽¹⁾:

- You can appropriately change the ENABLE signal (to switch motor current on or off) during initialization by **stepper_init** and afterward by **stepper_enable** or **stepper_enable_list**.
- The RTC5 generates periodic CLOCK signal pulses (during reference motions by **stepper_init** and set-position motions by **stepper_abs** etc.). With each CLOCK pulse, the stepper motor executes a single step. You can adjust the CLOCK signal's pulse period (and thereby the speed of stepper motor motion) during initialization by **stepper_init** and afterward by **stepper_control** or **stepper_control_list** (the period is specified in units of 10 μ s cycles).
- You can explicitly set the DIRECTION signal (and thereby the direction of stepper motor motion) during reference motions by **stepper_init**. In contrast, the DIRECTION signal is internally controlled during set-position motions by **stepper_abs** etc.: the signal gets set (to HIGH) if the next CLOCK pulse (in accordance with the defined set-position value) would increase the internal position variable. The DIRECTION signal also remains set for the cycles between two clock cycles and even when the stepper motor has reached its set position. Only upon an actual change of direction does the DIRECTION signal correspondingly change in place of a CLOCK pulse. Here, output of the next CLOCK pulse is delayed by a full CLOCK pulse period (undefined truncation of CLOCK pulse periods never occurs).

Notes

- For signal specifications, see [Chapter 4.6.7 "STEPPER MOTOR Socket Connector"](#), Page 75.
- For querying signals, see [Section "Querying Signals and Status Values"](#), Page 262.
- Stepper motor signals are outputted independently of any executing lists. A **set_end_of_list**, **pause_list**, **set_wait**, **stop_execution** or external stops do not terminate or pause the stepper motor motion.
- For changes of direction or pulse period, the new values do not become active until an already-begun period is complete. Thus, pulse intervals are never be shorter than the currently defined value. For change of direction, an additional empty period (without CLOCK pulse) gets inserted.

(1) Stepper motor signals are available only for RTC5 Boards with **DSP** version numbers 2 and higher (see **get_rtc_version** Bit #16...Bit #23). For older RTC5 Boards, stepper motor commands do not execute.

Reference Motions and Position Initialization

With **stepper_init**, you can initiate reference motions to limit switches. Here, the desired direction can be specified by the **Dir** parameter. To ensure that, despite mechanical play or long signal transit times, the reached end position still does not lie beyond the limit switch, you can define a tolerance value **Tol** that moves the stepper motor in the opposite direction after reaching the limit switch.

SCANLAB recommends executing a (fast) reference motion with a short CLOCK pulse period **Period** first and then a further (shorter but slower) reference motion with a longer CLOCK pulse period.

Once the reference motion has been successfully completed, the position variable (for the current position) is set to the value defined by parameter **Pos** as the reference for subsequent set-position motions. The reached reference position is offset from the limit switch position by $\pm Tol$ (direction dependent).

During reference motion, the status is "Init" (Bit #5 = 1), see [Section "Querying Signals and Status Values", Page 262](#). The limit switch position is traversed 4 times and a mean limit switch position is calculated from this. Then the stepper motor moves away from the limit switch by **Tol** steps in the opposite direction to **stepper_init Dir**. **Tol** must be large enough to overcome a possible hysteresis. During this set-position motion, see [Section "Set-Position Motions", Page 261](#), the status is no longer "Init" but "Busy" (Bit #4 = 1). If you select **Pos = Tol** with **stepper_init**, the average position of the limit switch is 0. The only resulting inaccuracy is the fluctuation of the limit switch position measurement when the limit switch position is passed over 4 times.

If **Period = 0** and/or **Dir < 0**, then the reference motion does not occur. Instead, the current stepper motor position becomes the new reference position (with the value newly defined in **Pos** as the position variable).

Because **stepper_init** always stops a previously begun stepper motor motion, you could also use **stepper_init** as an emergency stop for the stepper motor.

Parallel execution of reference motions for both stepper motors is also possible, but cannot be simultaneously started through a single command.

Set-Position Motions

Set-position motions can be initiated by **stepper_abs**, **stepper_rel**, **stepper_abs_no** and **stepper_rel_no** or the corresponding list commands.

Specify absolute set-position values for **_abs** commands and relative values for **_rel** commands (always as CLOCK pulse units). **_no** commands only produce set-position motions for *one* stepper motor output, the other set-position commands do so for both stepper motor output ports simultaneously.

With **stepper_wait**, you can interrupt further execution of a list until a started stepper motor motion is completed.

The list commands for set-position motions are short list commands. Therefore, a stepper motor motion can also execute synchronously with a galvanometer scanner motion.

If the limit switch activates during a set-position motion, then the motion immediately aborts and cannot be resumed as a normal set-position motion. You either have to request a reference motion or mechanically or via the software, see below, deactivate the limit switch. If a stepper motor motion aborts once (for example, also by **Period = 0**), then the existing set position gets overwritten by the current position value. Therefore the stepper motor motion to the original set position cannot be resumed by eliminating the cause of interruption (limit switch or CLOCK pulse period = 0). Instead, it needs to be newly retriggered.

To work around this behavior, the consideration of the limit switch can be deactivated by **stepper_disable_switch**. Then, the "release" can occur without carrying-out an initialization motion once again, even if a limit switch is present. The deactivation is especially useful, if a continuously rotating rotary axis is controlled by the stepper motor. During an initialization motion a possible deactivation of a limit switch is not considered.

Querying Signals and Status Values

The current status of stepper motor signals (ENABLE, DIRECTION, CLOCK and SWITCH), the currently defined CLOCK pulse period and the current values of internal position variables for both stepper motors can be queried by **get_stepper_status**.

The **get_stepper_status** command also returns the Busy and Init statuses of both stepper motors. The Init status is set during reference motions and the Busy status during set-position motions. As long as the Init status is set, no set-position commands (**stepper_abs**, **stepper_rel**, etc.) are permitted; control commands (except **stepper_init**) are denied with a **get_error** return code of **RTC5_BUSY** and list commands wait until the Init status gets reset. In some circumstances, the list itself or the motion process must be aborted.

Terminating Infinite Motions

Depending on chosen parameters, very long or even infinite stepper motor motions can be initiated by **stepper_init**, **stepper_abs**, for example, if no limit switch exists in the specified direction or if a very large set-position value is combined with a long pulse period.

- If an infinite motion is started by a control command (for example, **stepper_abs**), then this control command completes at the latest when the positive time (in seconds) supplied for the **WaitTime** parameter has expired. However, the stepper motor's infinite motion itself continues and you need to separately abort it by setting the CLOCK pulse period (for example, by the control command **stepper_control**) to 0 (emergency stop) or by defining an appropriate new set position.
- If you start an infinite motion by a list command (for example, **stepper_abs_list**) and wait for its completion by **stepper_wait**, then further list execution is blocked as long as the infinite motion is not aborted by a control command such as **stepper_control** with **Period = 0** or an appropriate new set position. You could also abort the list by **stop_execution** or an external stop. But here, too, the stepper motor's infinite motion needs to be separately stopped with **stepper_control**(**Period = 0**).

WaitTime Parameter

The **WaitTime** parameter of the control commands can be used to set them to return to the calling program after the specified time (in seconds) has elapsed, regardless of whether the stepper motor motion is complete or not.

With **WaitTime = 0**, the command returns immediately. In this case, users must ensure that no unallowed commands are called. In particular, initialization should not be cancelled before it is complete. Otherwise, this leads to incorrect reference positions.

9.1.6 RS-232 interface

The RS232 socket connector provides an RS-232 interface, see [Chapter 4.6.5 "RS232 Socket Connector", Page 70](#).

For configuring the RS-232 interface, see [Chapter 4.6.5 "RS232 Socket Connector", Page 70](#).

`rs232_write_data` can be used to send single data words (bytes) to the RS-232 interface. Texts can be sent to the interface by `rs232_write_text` or `rs232_write_text_list`.

9.1.7 McBSP Interface

At the [McBSP interface](#), see [Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71](#), a 32-bit data word every 10 μ s at DX0 pin (07) is permanently outputted.

The `set_mcbbsp_out` and `set_mcbbsp_out_ptr` commands allow selection of the signal types (analogously to `set_trigger`) to be outputted there:

- `set_mcbbsp_out` lets you specify two signal types for simultaneous output at the [McBSP interface](#). A 16-bit portion of the first signal type is packed along with a 16-bit portion of the second signal type into a 32-bit data word for output every 10 μ s. For a detailed description, see [set_mcbbsp_out](#).
- `set_mcbbsp_out_ptr` lets you define a list of up to 8 signal types. The signals are outputted sequentially in the specified order. For every 10 μ s clock cycle, the lower 24 bits of the corresponding data signal and the associated signal type number (8 bits) are packed into a 32-bit data word and outputted at the [McBSP interface](#). For a detailed description, see [set_mcbbsp_out_ptr](#).

Notes

- For signals and operating conditions, see [Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71](#).
- In the default setting, the [McBSP interface](#) always outputs Bit #4...Bit #19 of the Cartesian control values for the x axis and y axis (SampleX and SampleY) in a common 32-bit data word. This is equivalent to specifying `set_mcbbsp_out(7, 8)`.
- Signals specified by `set_mcbbsp_out` or are outputted (with `set_mcbbsp_out_ptr` sequentially) until you call one of these two commands again.

9.2 Signal Input

Signals of peripherals (for example, signals of a transport system, workpiece recognition system or process monitoring camera) can be queried by the interfaces described below through control commands at any desired time or – during processing of a list – by list commands.

9.2.1 16-Bit Digital Input Port

The EXTENSION 1 socket connector provides a 16-bit digital TTL input (DIGITAL IN0...15) (see [Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65](#)) for receiving 16-bit digital values. For synchronization of data transmission, the EXTENSION 1 socket connector also provides a SYNC signal.

The `read_io_port` or `read_io_port_buffer` and `read_io_port_list` commands can be used to read the current value of the digital input port.

Further commands are provided for conditional command execution (see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#)).

9.2.2 2-Bit Digital Input Port

For querying 2-bit digital values, the LASER connector provides a 2-bit digital input port (DIGITAL IN1 and DIGITAL IN2, see [Section "2-Bit Digital Input Port", Page 61](#)).

Its input value can be read by `get_laser_pin_in`.

Further commands are provided for conditional command execution (see [Chapter 9.3.2 "Conditional Command Execution", Page 271](#)).

9.2.3 RS-232 Interface

The RS232 socket connector provides an RS-232 interface for reading signals, see [Chapter 4.6.5 "RS232 Socket Connector", Page 70](#).

For configuring the RS-232 interface, see [Chapter 4.6.5 "RS232 Socket Connector", Page 70](#).

Data can be read in by `rs232_read_data`.

9.2.4 McBSP Interface

At the [McBSP interface](#), see [Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71](#), the 32-bit data word most recently fully transmitted to the specified memory location can be queried with `read_mcbbsp`.

The interpretation as one 32 bit data word or two 16 bit data words is the responsibility of the user.

Signals (position values) received by the [McBSP interface](#) can also be integrated directly into Processing-on-the-fly correction of workpiece or scan-system motion (see [Chapter 8.6 "Processing-on-the-fly", Page 227](#) and [Chapter 9.3.4 "Synchronization and Online Positioning by McBSP/SPI Signals", Page 276](#)) or can be used for an Online Positioning, see [Chapter 8.3.1 "Local Online Positioning", Page 213](#) and [Chapter 8.3.2 "Global Online Positioning", Page 216](#).

Notes

- For signals and operating conditions, see [Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71](#).

9.3 Control by External Signals

The previously described input and output of peripheral signals can be synchronized with scan system control and laser control, as follows:

- The related list commands can be inserted in command lists at appropriate locations.
- Execution of related control commands can be made dependent on the current status of list execution. For this, the list status can be requested by **read_status**, see Chapter 6.4.2 "List Status", Page 96 and the list execution status by **get_status**, see Chapter 6.4.3 "List Execution Status", Page 97.
- In addition, the **BUSY list execution status List Execution Status** is provided by the BUSY OUT signal:
 - at the LASER connector, see Section "BUSY List Execution Status", Page 61
 - at the EXTENSION 1 socket connector, see Section "BUSY List Execution Status", Page 66
 - at the MARKING ON THE FLY socket connector, see Section "BUSY list execution status Status", Page 69

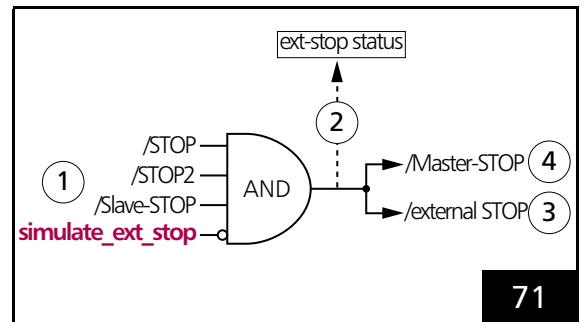
Moreover, the RTC5 PCI Board provides commands and interfaces (described in the following sections) that allow external control signals (for example, from a light-barrier or from an encoder) to directly control and synchronize execution of command lists or individual commands (including the output of peripheral signals).

Moreover, the RTC5 PCI Board provides interfaces to control and synchronize list execution directly with external signals.

9.3.1 Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization

External Stop

By a signal at the inputs /STOP, /STOP2 or /Slave STOP, or by **simulate_ext_stop**, an **External Stop** can be initiated, see (1) and (3) in Figure 71.



External Stop. See text for description.

This:

- immediately cancels the currently executed list
- switches off the **Signals for "Laser Active" Operation** (but does not deactivate them)

Like after calling **stop_execution** (internal stop), the following output ports are then set to the previously defined (by **set_port_default**) stop-case values given these have not been defined as $\neq$ "-1":

- the 16-bit digital output port of the EXTENSION 1 socket connector
- the 8-bit digital output port (DATA0 to DATA7) of the EXTENSION 2 socket connector
- the 2-bit digital output port
- the two 12-bit analog output ports (**ANALOG OUT1** and **ANALOG OUT2**) of the LASER connector

The inputs for external stop signals (1) are always unlocked so that an **External Stop** can occur at any time (emergency stop).

The /STOP input is accessible by the 15-pin D-SUB LASER connector, see [Section "External Control Signals", Page 61](#), the /STOP2 input at the MARKING ON THE FLY socket connector, see [Section "External Control Signals", Page 69](#). Both signal inputs are internally connected to +3.3 V by pull-up resistors (4.7 kΩ). Both inputs are TTL active-LOW and level sensitive. A list abort is triggered as soon as at least one of the two inputs is set to LOW (that is, to 0 V or ground) for at least 10 μs.

If an RTC5 PCI Board is connected to other boards by the Master or Slave connector, see [Figure 7](#), then external stops from Master to Slave. See also [Chapter 6.6.3 "Master/Slave Operation", Page 113](#).

Stops triggered by **stop_execution** are *not* passed on. In contrast, external stops triggered by **simulate_ext_stop** are passed through.

By **get_startstop_info** the current stop status (that is, whether one of the inputs is currently set to LOW) (2) can be queried and whether or not a new **External Stop** has occurred since the last query.

External Start

By a signal at the inputs /START, /START2 or /Slave-START, or by **simulate_ext_start** or **simulate_ext_start_ctrl**, an **External Start** can be initiated (see (1), (5) and (7) in [Figure 72](#)).

This starts execution at the beginning of "List 1". But the commands **set_extstartpos** or **set_extstartpos_list** also allow pre-selection of another absolute start address. A list is only started if neither the **BUSY list execution status** (as during list execution) nor the **INTERNAL-BUSY list execution status** (as for example, with **goto_xy**) nor the **PAUSED list execution status** (after **pause_list**, **stop_list** or **set_wait**) is set at the moment.

Before the /START, /START2 or /Slave-START inputs (1) can be used, they must be enabled by **set_control_mode** (3).

As of version DLL 543, OUT 543, the control command **simulate_ext_start_ctrl** can be deactivated by **set_control_mode** (Bit #4 = 1). The list command **simulate_ext_start** still remains active.

The /START input is accessible by the 15-pin D-SUB LASER connector, see [Section "External Control Signals", Page 61](#), the /START2 input at the MARKING ON THE FLY socket connector, see [Section "External Control Signals", Page 69](#). Both signal inputs are internally connected to +3.3 V by pull-up resistors (4.7 kΩ). Both inputs are TTL active-LOW and edge sensitive (HIGH to LOW level transition). A start is triggered – after activation by **set_control_mode** – as soon as one of the three input signals changes from HIGH to LOW (that is, to 0 V or ground).

If an RTC5 PCI Board is connected to other boards by the Master or Slave connector, see [Figure 7](#), then external starts are passed on from Master to Slave. See also [Chapter 6.6.3 "Master/Slave Operation", Page 113](#).

Internal starts triggered by **execute_list** or **execute_at_pointer** are *not* passed on.

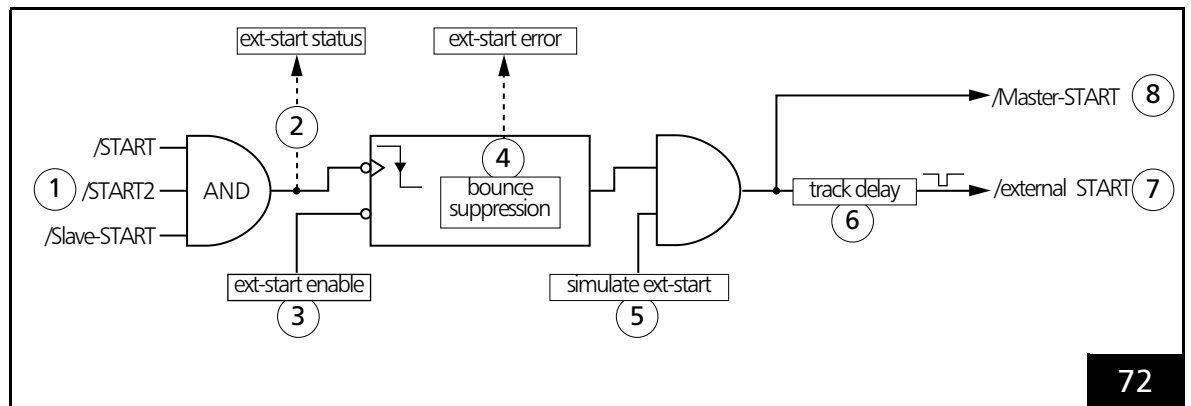
In contrast, **External Starts** triggered by **simulate_ext_start** or **simulate_ext_start_ctrl** are passed on, see also [Chapter 6.6.3 "Master/Slave Operation", Page 113](#).

get_startstop_info queries the current start status (that is, whether one of the inputs is set to LOW) (2) and whether a list has successfully started since the last query.

bounce_supp enables debouncing of start signals received at the /START, /START2 or /Slave-START inputs (4). Start signals occurring within the defined debouncing time after a successful start signal are thereby suppressed. **get_marking_info** queries whether a start signal has been suppressed (4).

simulate_ext_start_ctrl can be used to start by control command a synchronous start of master/slave-synchronized boards.

The list command **simulate_ext_start** can be used, for instance, to trigger further starts at defined intervals after the successful one-time **External Start** (see below).



External Start. See text for description.

External Start with Track Delay

For many applications (for example, if a workpiece must be initially transported from the light barrier to the scan system), the start must be delayed with reference to the triggering start signal.

For this purpose, **set_ext_start_delay**, **set_ext_start_delay_list** or **simulate_ext_start** allow configuring a track delay (see (6) in [Figure 72](#)) that postpones execution of a start relative to the triggering input signal or corresponding command. The track delay is specified in counting units of an internal encoder (encoder-counter) that itself can be triggered by an external or simulated encoder signal, see [Chapter 9.3.3 "Synchronization by Encoder Signals"](#), Page 274.

External Starts triggered by an external start signal or by **simulate_ext_start** or **simulate_ext_start_ctrl** that do not execute immediately because of the track delay setting are held in a queue that can accommodate up to 8 starts (each start trigger is automatically generated when the delay has expired). This can be useful, for instance, when processing multiple workpieces transported to the scan system (even) at irregular intervals: here, up to 8 workpieces can simultaneously reside within the track delay (distance between the light barrier and scan system). If more **External Starts** are triggered than can be simultaneously held in the 8-start wait loop, then an error bit is set, which can be queried by **get_startstop_info** (**Bit #11**). If a track delay is set, then any previous queue is canceled (see **set_ext_start_delay** and **simulate_ext_start**).

By **set_control_mode** (**Bit #1 = 1**) it can be set that the queue with the external start entries get explicitly cancelled upon an **External Stop**. With **set_control_mode** (**Bit #1 = 0**) the queue remains existing after an **External Stop**.

Notes

- The **/START**, **/START2** and **/Slave-START** inputs are *edge* sensitive (HIGH to LOW level *transition*).
- The **/STOP**, **/STOP2** and **/Slave-STOP** inputs are *level* sensitive.
- A **stop_execution** call disables the **/START**, **/START2** and **/Slave-START** inputs. An external stop signal also (at least temporarily) disables these inputs, that is, as long as one of the inputs **/STOP**, **/STOP2** or **/Slave-STOP** is LOW. **set_control_mode** can be used to define whether or not the **/START**, **/START2** and **/Slave-START** inputs also stay disabled when the external stop signal is no longer active.
- **set_control_mode** additionally allows activation or deactivation of the inputs **/START**, **/START2** or **/Slave-START** and deactivation of track delay.
- **External Starts** are also be suppressed after **pause_list**, **stop_list** or **set_wait** (**PAUSED** list execution status is set). **restart_list**, **stop_execution**, **release_wait** or an **External Stop** ends suppression of the start.
- If list inputs are not yet finished, a buffer flush should be initiated before an **External Start**, for example, by
`set_input_pointer( get_input_pointer() )`, so that any still buffered list commands are fully transferred to list memory, see [Chapter 6.4.1 "Loading Lists"](#), Page 94.
- If a master board is started internally (for example, by **execute_list_pos**) and subsequently a slave board by **simulate_ext_start**, then the master and slave boards do not run synchronously if a home jump has been previously activated by **home_position** or **home_position_xyz**: the home return executes on the master board *before* **simulate_ext_start** starts the slave board, but executes on the slave board *afterward*. While the home return executes on the slave board, the master board continues running. This asynchronicity does not occur if all boards are started by an external start signal (or by **simulate_ext_start** or **simulate_ext_start_ctrl**) or if no home jump is activated.

Regular (Periodic) External Starts

By **set_control_mode** and **set_control_mode_list** (Bit #10), equidistant **External Starts** can be created that are independent of the point in time of the start trigger as long as they occur within the specified track delay.

This strongly periodic list processing is – independently of a list's actual duration of execution and the exact point in time of the **External Start** – exactly synchronized to the 10 μ s clock of the RTC5 PCI Board.

If desired, set Bit #10 = 1 (Mode|Bit #10) to configure the internal encoder-counter's processing so that the track delay of an **External Start** is *not* counted only beginning with the point in time of the triggering external start signal or **simulate_ext_start** (Bit #10 = 0) but already beginning with the most recently executed **External Start** (also executed by an external start signal or **simulate_ext_start**), see **Figure 73**. This makes the distance between consecutive **External Starts** (in encoder pulses) constant.

For activation of this mode, an **External Start** must have successfully occurred (only one-time) in mode Bit #10=0 (Mode & ~Bit #10). Each subsequent **External Start** must be requested within the specified track delay.

Example in Pascal of a typical command sequence without use of an external start signal:

```
set_control_mode(Mode & ~Bit #10);
// (one-time) reset (disable) Bit #10
// (initialization)
set_start_list_pos(ListNo, Pos);
// open some list
// afterward: some commands
simulate_ext_start(Delay, EncoderNo);
// first time start in mode Bit #10 = 0,
// otherwise in mode Bit #10 = 1
set_control_mode_list(Mode|Bit #10);
// set Bit #10 = 1
// afterward: further commands
set_end_of_list;
// close the list
execute_list_pos(ListNo, Pos);
// (one-time) start the list
```

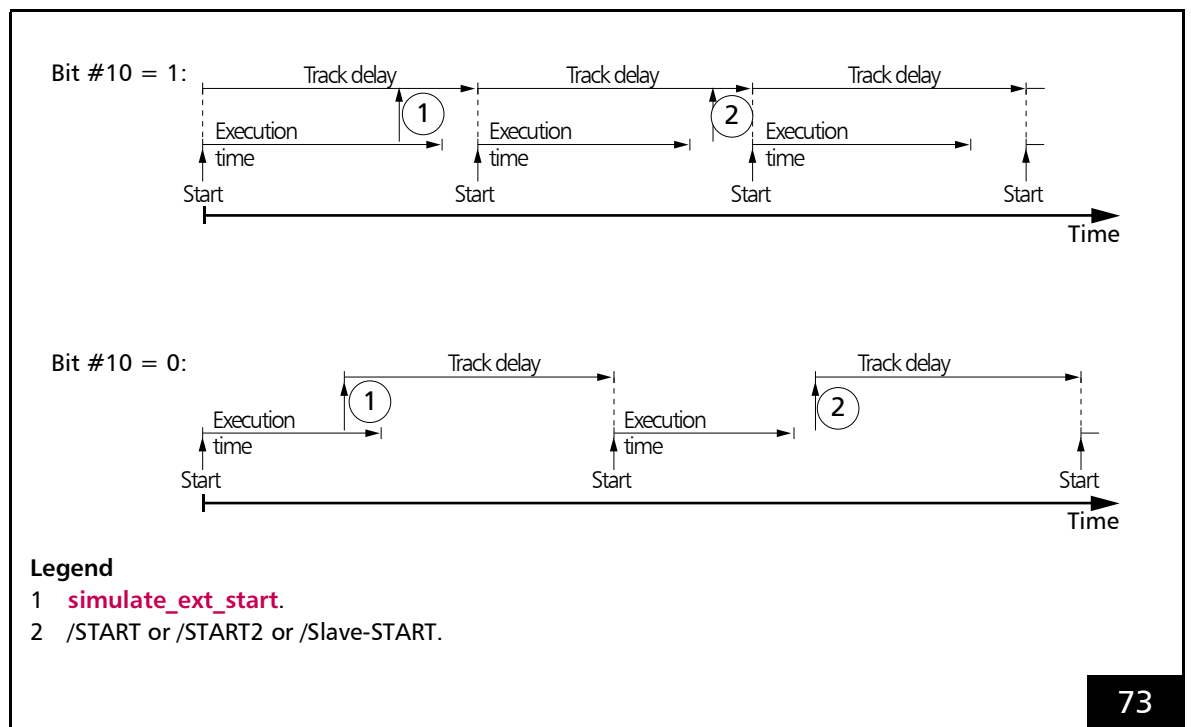
If the first start is to be triggered externally (for example, by /START or by **simulate_ext_start_ctrl**) rather than by an **execute_list_pos** command, but all subsequent starts triggered by **simulate_ext_start**, then **set_control_mode_list** in the above example must be called before **simulate_ext_start**.

After setting **set_control_mode** (Bit #1 = 1), the external start queue entries get explicitly cancelled upon an **External Stop** (thereby, **External Starts** can be permanently stopped by an **External Stop**).

For **set_control_mode** (Bit #1 = 0) (default setting) – after an otherwise infinitely repetitive series has been stopped (for example, by **set_control_mode** (Bit #0 = 0) – you should deactivate the track delay and cancel the queue of not-yet-executed **External Starts** by **set_ext_start_delay** (Delay = 0). Otherwise, the next "equidistant" **External Start** does not have the correct gap. **set_control_mode** (Bit #2 = 1) alone is not sufficient for termination, because the track delay is reactivated by any not-yet-executed **simulate_ext_start** calls.

If a further **External Start** is missed within the track delay, then you should delete the wait loop (otherwise the encoder counter needs to run through a full 31-bit sequence before a start can again be successfully triggered). Deletion can be accomplished by resetting the track delay with **set_ext_start_delay**.

In any case, Bit #10 needs to first be reset (disabled) for initialization and then a (one-time) **External Start** must be triggered (for external start signals Bit #0 must be set) before Bit #10 can again be set (see example). Otherwise, the first track delay in the wait loop is undefined.



Regular and irregular **External Starts** (see text for description).

9.3.2 Conditional Command Execution

The so-called conditional commands allow the execution of individual list commands to be made dependent on external control signals.

The conditional commands read out the current value at the 16-bit digital input port at the EXTENSION 1 socket connector, see [Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65](#), and their execution *depend on the read out value*:

- Conditional Jumps
list_jump_pos_cond (synonymous with **list_jump_cond**) and **list_jump_rel_cond** result in either a jump within a buffer area or no jump. The thereby specified jump addresses must fulfill the same conditions as with **list_jump_pos** and **list_jump_rel**
- Variable-distance jump
switch_ioport executes a relative list jump
- Conditional Calls of Non-Indexed Subroutines
list_call_cond and **list_call_abs_cond** either call or do not call a non-indexed subroutine at a specified memory address
- Conditional Calls of Indexed Subroutines
sub_call_cond and **sub_call_abs_cond** either call or do not call – depending on the queried value – an indexed subroutine with a specified index
- Conditional output of peripheral signals
set_io_cond_list and **clear_io_cond_list** associate the output value of the 16-bit digital output port at the EXTENSION 1 socket connector, see [Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65](#), directly with the signals at the digital input port: individual bits of the output port are set or cleared

- Conditional execution of any desired list commands:

if_cond and **if_not_cond** have no effect if the condition for the queried value is fulfilled or not. Otherwise, they result in skipping the next list command. In this way, all list commands can be executed conditionally.

Example: the command sequence

```
if_cond(...)
```

```
list_call(...)
```

is functionally identical to

```
list_call_cond(...).
```

The execution of any desired list command can also be made dependent on the current value at the 2-bit digital input port of the LASER connector. For this, **if_pin_cond** and **if_not_pin_cond** are available. These are functionally similar to the commands **if_cond** and **if_not_cond**. Reliable functioning of these conditional commands requires that the signals at the 2-bit digital input port remain unchanged for at least 10 μ s.

As of version DLL 543, OUT 543, a condition for the 16-bit digital input port of the EXTENSION 1 socket connector can be defined by the control command **set_pause_list_cond**. If this condition is met, a currently executed list is paused by an automatically set **pause_list**. The list can only be resumed by **restart_list**. The condition is checked once per 10 μ s clock cycle. A conditional **pause_list** (as of DLL 544, OUT 544) takes precedence over a simultaneously present /STOP signal.

set_pause_list_not_cond does the same as **set_pause_list_cond**, if the specified condition is not met.

Example Code (Pascal)

(1) Confirm a signal:

```
set_start_list(1);
...
// set Bit #0 of the 16-bit digital output port
set_io_cond_list(0, 0, 1);
// loop until the signal is confirmed (that is, Bit #0 of the digital input turns HIGH)
list_jump_rel_cond(0, 1, 0);
// clear Bit #0 of the 16-bit output
clear_io_cond_list(0, 0, 1);
// loop until the signal is confirmed
list_jump_rel_cond(1, 0, 0);
...
set_end_of_list;
execute_list(1);
```

(2) If the lower 4 bits of the digital input have the value (0110), set Bit #1 of the 16-bit digital output. Otherwise clear Bit #1:

```
set_start_list(2);
...
// RTC4 style: list_jump_cond($0006, $0009, get_input_pointer + 3);
// this command uses absolute addresses and is not relocatable
// the following RTC5 command uses relative addresses and is relocatable:
// skip the next two commands, if the state
// of the 16-bit input is (xxxx xxxx xxxx 0110)
// list_jump_rel_cond($0006, $0009, 3);

// clear Bit #1 of the 16-bit output and...
clear_io_cond_list(0, 0, 2);
//RTC4 style: set_list_jump(get_input_pointer + 2);
//this command uses absolute addresses and is not relocatable
//the following RTC5 command uses relative addresses and is relocatable
//...skip the next command
list_jump_rel(2);

// set Bit #1 of the 16-bit output
set_io_cond_list(0, 0, 2);

// (continue)
...
set_end_of_list;
execute_list(2);
...
bit1 := (get_io_status AND $0002) // returns the current state of Bit #1
```



(3) Choose between 15 small subroutines at defined memory addresses:

```
...
for i := 1 to 15 do
    // call subroutine at address i*100, if [Bit #3..Bit #0] (binary) = i
    list_call_cond(i, 15-i, i*100);
...
```

(4) Choose between 15 indexed subroutines:

```
...
for i := 1 to 15 do
    // call subroutine with index i, if [Bit #3..Bit #0] (binary) = i
    sub_call_cond(i, 15-i, i);
...
```

9.3.3 Synchronization by Encoder Signals

Intended Use

When processing moving workpieces, the laser scan processes need to be adapted to the current workpiece position.

To incorporate the current workpiece position, the RTC5 PCI Board can evaluate signals of up to two user-supplied incremental encoders. Though incremental encoders do not register the current workpiece position, they register the motion of the transport system (conveyor belt, rotating plate, etc.)⁽¹⁾. For each transport motion, they provide signals (depending on the direction of motion) to the RTC5 PCI Board which can result in incrementing or decrementing of its two internal encoder counters⁽²⁾. The states of the RTC5 encoder counters thereby correspond directly to the position of the workpiece⁽³⁾.

If workpieces are always processed at a constant speed and an encoder is therefore not mandatory, then the encoder signals can also be simulated by **simulate_encoder**, so that the encoder counters are incremented with a constant counting rate of 1 MHz.

The current counts of both encoder counters can be queried by the control command **get_encoder**. Alternatively, they can be stored in a buffer by the list command **store_encoder** and then retrieved from there by the control command **read_encoder**.

In addition, the RTC5 PCI Board automatically evaluates the current counts if execution of the laser scan processes is controlled as follows:

- For Processing-on-the-fly-applications (see [Chapter 8.6 "Processing-on-the-fly", Page 227](#)), the coordinate values of all **Vector commands** and **"Arc" commands** are transformed in accordance with the current encoder counts.
- By the list command **wait_for_encoder_mode**, further execution of a list can be postponed until the selected encoder counter has overstepped or understepped a pre-defined value.
- With 2D motions, **wait_for_encoder_in_range** waits until the encoder values are within a given rectangle.
- For **External Starts**, a track delay can be defined by **simulate_ext_start**, **set_ext_start_delay** or **set_ext_start_delay_list** for postponing execution of a start relative to the triggering input signal or corresponding command, see [Section "External Start", Page 266](#).
- Encoder-speed-dependent "Automatic Laser Control", see [Section "Encoder-Speed-Dependent Laser Control", Page 192](#), uses the current encoder speed (counter pulses in the most recent 10 μ s interval) of an encoder counter to control a laser signal parameter.

(1) The actual workpiece position can also be forwarded by the **McBSP interface** to the RTC5 PCI Board (see [Chapter 9.3.4 "Synchronization and Online Positioning by McBSP/SPI Signals", Page 276](#)).

(2) See [Notice!](#), [Page 49](#).

(3) The encoder counters are signed 32-bit counters. Upon reaching the maximum (minimum) counter value, counting continues with the minimum (maximum) value. A counter reset only occurs if triggered by Processing-on-the-fly-commands (see **set_fly_x**, **set_fly_y**, **set_fly_rot**). **set_control_mode(Bit #9 = 1)** can be used to precisely synchronize the encoder reset with external start signals.

Input Ports for External Encoder Signals

For receiving encoder signals, the MARKING ON THE FLY socket connector provides 2 encoder input ports, see [Section “Encoder Input Ports”, Page 69](#).

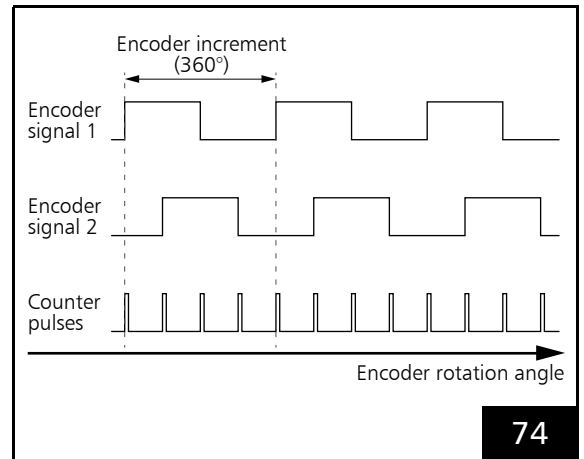
- **ENCODER X**
- **ENCODER Y**

For linear motions of the parts to be processed, up to two user-supplied incremental encoders (that define, independently from each other, the workpiece motion in the x direction and y direction) can be connected to these inputs.

For rotational motions only *one* incremental encoder is necessary. For Processing-on-the-fly applications, it must be connected to the encoder input port **ENCODER X**.

ENCODER X and **ENCODER Y** are each designed for a pair of standardized differential input signals (RS-422; HIGH level ≥ 2.0 V, LOW level ≤ 0.8 V).

The timing diagram of a typical encoder signal pair shows [Figure 74](#). The second encoder signal is usually phase-shifted by 90° relative to the first signal. The corresponding RTC5-internal encoder counter⁽¹⁾ is triggered at each edge of both signals, that is, one encoder increment results in 4 counter pulses (counts). The relative 90° phase-shift of the two signals allows the RTC5 PCI Board to detect the motion direction of the workpiece as well. Depending on the direction of motion, the counter value is increased or decreased.



Timing diagram of a typical encoder signal pair and of the corresponding RTC5 counter pulses.

The signals at encoder input port:

- **ENCODER X** trigger encoder counter “Encoder0”
- **ENCODER Y** trigger encoder counter “Encoder1”

The encoder signals should not exceed the maximum allowed frequency of 4 MHz (16 encoder signal edges / μ s).

Encoder Simulation

The encoder simulation can be activated (deactivated) by **simulate_encoder**. The RTC5 PCI Board provides a 1 MHz clock signal (counter pulses), which replaces the signals of an external incremental encoder.

(1) See [Notice!, Page 49](#).

9.3.4 Synchronization and Online Positioning by McBSP/SPI Signals

Instead of incremental encoders, the motion between the workpiece and the scan system can be synchronized using absolute position data via the **McBSP interface**.

Alternatively, this interface also allows the alignment of the workpiece relative to the (stationary) scan system ("**Online Positioning**").

The input value most recently fully transmitted at the **McBSP interface** can be queried by **read_mcbbsp**. In addition, the RTC5 PCI Board automatically evaluates the current input value if execution of the laser scan processes is controlled as follows:

- For Processing-on-the-fly-applications (see **Chapter 8.6 "Processing-on-the-fly", Page 227**), the coordinate values of all **Vector commands** and **"Arc" commands** are transformed in accordance with the current input value.
- By the list command **wait_for_mcbbsp**, further execution of a list can be postponed until the input value has reached, overstepped or understepped a pre-defined value.
- By **Online Positioning** (see **Chapter 8.3.1 "Local Online Positioning", Page 213** and **Chapter 8.3.2 "Global Online Positioning", Page 216**), the scan system is aligned in accordance with the current input values.

Notes

- To compensate linear motion, Cartesian coordinates can be forwarded by the **McBSP interface**. Compensation of rotational motion, on the other hand, requires transmission of angle positions. During evaluation, full revolutions are suppressed as long as the input values do not exceed the allowed value range (20 full circles, see **set_mcbbsp_rot**).
- The signals at the **McBSP interface** have no impact on the track delay, which can be defined for **External Starts** (by **simulate_ext_start**, **set_ext_start_delay** or **set_ext_start_delay_list**).

9.4 Periodical I/O Signals

By **periodic_toggle** and **periodic_toggle_list**, periodically repeated signals can be outputted at a selectable IO port:

- **ANALOG OUT1** output port, see Section "12-Bit Analog Output Port 1 and 2", Page 61
- **ANALOG OUT2** output port, see Section "12-Bit Analog Output Port 1 and 2", Page 61
- 8-bit digital output port, see Section "8-Bit Digital Output Port", Page 68
- 16-bit digital output port, see Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65
- 2-bit digital output port, see Section "2-Bit Digital Output Port", Page 61

Thereby, for example, external peripheral equipment can be triggered synchronously to list execution.

Notes

- At a **set_end_of_list**, **stop_execution** or external /STOP the periodical signals continue, even if they have been activated by the list command **periodic_toggle_list**.
- As of version DLL 543, OUT 543, **periodic_toggle** and **periodic_toggle_list** toggle endless with $\text{Count} = 2^{32}-1$.

10 RTC5 Commands

10.1 Overview

The following pages describe the complete RTC5 command set (control commands and list commands). The commands are listed according to their intended use. The page numbers refer to [Chapter 10.2 "RTC5 Command Set", Page 290](#), where the commands (alphabetically) are explained in detail.

10.1.1 Designations

Multi-Board Commands

All commands marked (n_) in the following list also exist in a version for multiple RTC5 Boards installed in one computer, see also [Chapter 6.6 "Using Several RTC5 PCI Boards in One PC", Page 112](#).

An associated multi-board command is also provided for almost all commands. Exceptions are explicitly noted in the description list (in [Chapter 10.2 "RTC5 Command Set", Page 290](#)).

Normal, Short, Variable and Multiple List Commands

The list commands of the RTC5 command set vary somewhat in their length of command execution. To differentiate, list commands in the list description (in [Chapter 10.2 "RTC5 Command Set", Page 290](#)) are designated as "normal", "short", "variable" and "multi".

- Normal list commands require a full 10 μ s clock cycle for command execution.
- Short list commands require less time for command execution. Therefore, they can be carried out along with the next list command, one directly after the other within a single 10 μ s clock cycle, during which control commands cannot execute. In contrast, a short list command that immediately follows a normal list command executes in the subsequent 10 μ s clock cycle.

The quicker execution of short list commands reduces total list processing time. In addition, during a **Polyline**, the laser power can, for instance, be varied or the IO ports can be addressed (see [write_da_x_list](#)) between the **Polyline**'s individual mark vectors (see [set_laser_pulses](#)), all without interrupting the **Polyline** (the laser remains on).

In contrast, if a specific time behavior is desired (10 μ s clock cycle), you can insert an additional **list_nop** or **list_continue** command after any short list command to ensure that the next command only executes in the following 10 μ s clock cycle. Insertion of **list_nop** (but not insertion of **list_continue**) results in the interruption of the **Polyline** (**Signals for "Laser Active" Operation** are switched off).

Currently, up to 12 short list commands per 10 μ s clock cycle are possible. However, the maximum number can be lower, depending on the workload of the RTC5 Board and the **DSP** version⁽¹⁾. Short list commands that alter the output pointer (for example, **sub_call**, **list_return** or **list_jump_pos**) count as two commands. If the maximum number is exceeded, a 10 μ s clock cycle is inserted (equivalent to an additionally inserted **list_continue**, during the **Polyline** the laser remains on).

A maximum of two short list commands per 10 μ s clock cycle are allowed before a normal list command. If a normal list command succeeds more than two short list commands, then the short list commands execute immediately and the normal list command execute delayed by a 10 μ s clock cycle.

(1) On older RTC5 Boards where **DSP** version < 2 (**get_rtc_version** Bit #16...Bit #23), only up to 8 short list commands per 10 μ s clock cycle are possible.

The maximum number of currently up to 12 short list commands may change in the future. For fully future-safe applications, only one short list command should precede a normal list command. If necessary, you should explicitly insert a **list_continue** or **list_nop** (**list_nop** interrupts the **Polyline**).

- The command execution lengths of variable list commands are dependent on additional parameters and the user program. Details are provided in the corresponding command description.
- About multiple list commands, see [page 280](#).

Notes

- The total execution time of normal and variable list commands equals the sum of the command execution time and the execution time of the process initiated by the command. A subsequent list command only runs after this total execution time has completed. Example: the total execution time of the normal list command **mark_abs** is $10\ \mu\text{s}$ (command execution time) + the execution time of the marking process. The latter is dependent on the settings of such parameters as marking length, mark speed, and delays etc.

Undelayed and Delayed Short List Commands

Most short list commands (for example, **list_jump_pos**, **list_call**, **sub_call** or **list_return**) (with all software versions) execute before the next list command and prior to a possible scanner delay (**Jump Delay**, **Mark Delay**, **Polygon Delay**). These commands are called “undelayed short list commands” in the corresponding command descriptions (in [Chapter 10.2 “RTC5 Command Set”](#), [Page 290](#)).

However, some short list commands only execute after the respective scanner delay (that is, directly before the next command). Such commands are called “delayed short list commands” in the corresponding command descriptions. These include commands that immediately affect output or laser power (for example, **set_rot_center_list**, **set_wobble_mode**, **write_da_x_list**, **set_laser_pulses**, **set_standby_list**, **set_mark_speed**, **set_encoder_speed**) or commands that affect data acquisition or time measurement (for example, **set_trigger** or **save_and_restart_timer**).

Without this delayed execution, such commands would, for example, result in a laser power change already being effective at the end of a **Mark command** (that is, during a still-active **Mark Delay** or **Polygon Delay**) and not at the beginning of the following **Mark command**.

set_trigger and **save_and_restart_timer** would erroneously take into account the delay of a preceding command instead of the delay of the subsequent command.

If these commands are directly before a **set_end_of_list**, add a **list_nop** so that they are still executed within the list.

For several short list commands in a row, even a “delayed short list command” only executes delayed if no further “delayed short list commands” directly follow.

With several “delayed short list commands” in a sequence of short list commands, only the last “delayed” command can actually be executed delayed. All others before that are executed immediately (yet before a scanner delay).

When you sequence the commands, make sure to place the most important “delayed” command at the end, especially within a **Polyline**.

Alternatively, you can also explicitly initiate processing of the scanner delay (for example, by **list_nop**), so that all subsequent short list commands in fact always execute “delayed”.

If a normal list command succeeds more than two short list commands, then the normal list command is executed delayed by a 10 μ s clock cycle.

Multiple List Commands

A multiple list command has multiple components that accordingly occupy multiple list memory area positions. The initial components are always undelayed short list commands. The final component is always a short, normal or variable list command. All components immediately execute successively. Any still-pending delayed short list commands execute first. The RTC5 command set currently only contains two-component multiple list commands (for example, **wait_for_encoder_in_range** or **set_pixel_line_3d**).

10.1.2 Compatibility

For RTC5 users who previously used the RTC4, the command descriptions (in [Chapter 10.2 “RTC5 Command Set”, Page 290](#)) note in each command’s “RTC4→RTC5” section:

- to what extent a command differs from that of the RTC4,
- whether the command is new to the command set,
- whether and how parameter values are converted in **RTC4 Compatibility Mode**.

Some RTC3/RTC4 commands and a few RTC2 commands emulated by the RTC3/RTC4 are not supported by the RTC5. These commands are listed in [Chapter 10.3 “Unsupported RTC2/RTC3/RTC4 Commands”, Page 723](#).

For general information about migrating from the RTC4 to the RTC5 and about the **RTC4 Compatibility Mode**, see [Chapter 2.10 “Notes for RTC4 Users”, Page 40](#).



10.1.3 Version Information

Descriptions for a number of commands include version information, listed under “**Version info**”:

- For newly added commands: the software or **DSP** version in which they become available
- For some older commands: information on implemented changes.

New commands as well as all changes to existing commands are described in

[RTC5_Software_RevisionHistory_<Date>.pdf](#).

10.1.4 Optional Functions

If an option is not present on the RTC5 PCI Board, see [Chapter 2.6 “Options”, Page 34](#), then the associated commands have only partial or non-existent effect:

- Without **Option Processing-on-the-fly**, commands to activate a Processing-on-the-fly correction have no effect
- Without **Option “3D”**, 3D vector commands are executed, however, no z axis signals are outputted
- Without **Option “Second Scan Head Control”**, commands for which the scan head connector can be explicitly specified have not effect on the second scan head connector (for example, **set_matrix**)

If applicable, the command descriptions contain corresponding notes.

10.1.5 Control Commands

DLL Initialization

free_rtc5_dll	325
get_rtc_mode	352
init_rtc5_dll	386
set_rtc4_mode	612
set_rtc5_mode	613

Using Several RTC5 PCI Boards in One PC

acquire_rtc	291
release_rtc	500
rtc5_count_cards	507
select_rtc	516

List Memory Commands

(n_) config_list	313
(n_) get_config_list	328
(n_) get_list_space	346
(n_) load_disk	420
(n_) save_disk	509

Board Initialization and Image Field Correction

(n_) get_head_para	337
(n_) get_sync_status	362
(n_) get_table_para	363
(n_) load_correction_file	416
(n_) load_program_file	430
(n_) load_stretch_table ⁽¹⁾	433
(n_) load_z_table ⁽¹⁾	439
(n_) number_of_correction_tables	468
read_abc_from_file	489
(n_) select_cor_table	512
(n_) set_dsp_mode	536
write_abc_to_file	715

Laser Mode and Parameters

(n_) config_laser_signals	311
(n_) get_standby	355
(n_) load_auto_laser_control	413
(n_) load_position_control	428
(n_) set_auto_laser_control	522
(n_) set_auto_laser_params	525
(n_) set_encoder_speed_ctrl	539
(n_) set_firstpulse_killer	544
(n_) set_laser_control	566
(n_) set_laser_mode	570
(n_) set_laser_pulses_ctrl	573
(n_) set_pulse_picking	609
(n_) set_pulse_picking_length	609
(n_) set_qswitch_delay	610
(n_) set_softstart_level	624
(n_) set_softstart_mode	626
(n_) set_standby	628

Setting the Scanner Parameters

(n_) load_varpolydelay	437
(n_) set_delay_mode	534
(n_) set_jump_speed_ctrl	564
(n_) set_mark_speed_ctrl	576
(n_) set_sky_writing	617
(n_) set_sky_writing_limit	618
(n_) set_sky_writing_mode	619
(n_) set_sky_writing_para	621

Coordinate Transformations

(n_) set_angle	520
(n_) set_defocus ⁽¹⁾	531
(n_) set_defocus_offset ⁽¹⁾	532
(n_) set_matrix	577
(n_) set_offset	598
(n_) set_offset_xyz ⁽²⁾	599
(n_) set_scale	614

Online Positioning

(n_) apply_mcbbsp	295
(n_) set_mcbbsp_global_matrix	581
(n_) set_mcbbsp_global_rot	582
(n_) set_mcbbsp_global_x	583
(n_) set_mcbbsp_global_y	584
(n_) set_mcbbsp_matrix	587
(n_) set_mcbbsp_rot	591
(n_) set_mcbbsp_x	592
(n_) set_mcbbsp_y	593

(1) Only with Option "3D".

(2) Limited functionality if no Option "3D".

Status Monitoring and Diagnostics

(n_) get_head_status	338
(n_) get_overrun	352
(n_) get_value	368
(n_) get_values	370
(n_) get_waveform	372
(n_) measurement_status	463
(n_) stop_trigger	674

iDRIVE Commands

(n_) control_command	315
(n_) get_transform	365
(n_) read_user_data	499
(n_) send_user_data	519
transform	703
(n_) upload_transform	706

Pixel Output Mode

(n_) set_default_pixel	530
------------------------------	-----

I/O Commands

(n_) get_free_variable	334
(n_) get_io_status	342
(n_) get_mcbasp	351
(n_) mcbasp_init	461
(n_) mcbasp_init_spi	462
(n_) periodic_toggle	484
(n_) read_analog_in ⁽¹⁾	490
(n_) read_io_port	492
(n_) read_io_port_buffer	493
(n_) read_mcbasp	495
(n_) read_multi_mcbasp	496
(n_) rs232_config	503
(n_) rs232_read_data	504
(n_) rs232_write_data	505
(n_) rs232_write_text	505
(n_) set_free_variable	556
(n_) set_laser_off_default	570
(n_) set_mcbasp_freq	580
(n_) set_mcbasp_out_ptr	589
(n_) set_port_default	607
(n_) write_8bit_port	714
(n_) write_da_1	716
(n_) write_da_2	717
(n_) write_da_x	718
(n_) write_io_port	721
(n_) write_io_port_mask	722

Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization

(n_) get_counts	328
(n_) get_master_slave	350
(n_) get_startstop_info	356
(n_) set_control_mode	528
(n_) set_extstartpos	541
(n_) set_max_counts	580
(n_) simulate_ext_start_ctrl	659
(n_) sync_slaves	680

List Handling and Status

(n_) auto_change	304
(n_) auto_change_pos	305
(n_) get_lap_time	343
(n_) get_status	358
(n_) get_wait_status	371
(n_) pause_list	483
(n_) quit_loop	486
(n_) read_status	497
(n_) release_wait	501
(n_) restart_list	503
(n_) set_pause_list_cond	601
(n_) set_pause_list_not_cond	602
(n_) start_loop	661
(n_) stop_execution	673
(n_) stop_list	673

Input Pointer Commands

(n_) get_input_pointer	342
(n_) get_list_pointer	345
(n_) load_list	426
(n_) set_input_pointer	558
(n_) set_start_list	630
(n_) set_start_list_1	630
(n_) set_start_list_2	630
(n_) set_start_list_pos	631

Output Pointer Commands

(n_) execute_at_pointer	320
(n_) execute_list	320
(n_) execute_list_1	321
(n_) execute_list_2	321
(n_) execute_list_pos	321
(n_) get_out_pointer	351

(1) Only with RTC5 PCIe Board or with RTC5 PCI Board and installed **ADC Add-On Board**.

Subroutine Commands

(n_) copy_dst_src	317
(n_) get_char_pointer	327
(n_) get_sub_pointer	361
(n_) get_text_table_pointer	364
(n_) load_char	415
(n_) load_sub	435
(n_) load_text_table	436
(n_) set_char_pointer	526
(n_) set_char_table	527
(n_) set_sub_pointer	632
(n_) set_text_table_pointer	633

Direct Laser and Scan Head Control

(n_) disable_laser	318
(n_) enable_laser	319
(n_) get_laser_pin_in	344
(n_) get_z_distance ⁽¹⁾	373
(n_) goto_xy	374
(n_) goto_xyz ⁽²⁾	375
(n_) laser_signal_off	395
(n_) laser_signal_on	396
(n_) set_laser_pin_out	571

Version Commands

(n_) get_dll_version	329
(n_) get_hex_version	340
(n_) get_rtc_version	353
(n_) get_serial_number	354

Error Commands

(n_) get_error	330
(n_) get_last_error	344
(n_) reset_error	502
(n_) set_verify	645
(n_) verify_checksum	708

Date, Time, Serial Numbers

(n_) get_list_serial	346
(n_) get_serial	354
(n_) select_serial_set	518
(n_) set_serial	615
(n_) set_serial_step	616
(n_) time_update	684

Processing-on-the-fly Commands

(n_) clear_fly_overflow_ctrl	309
(n_) get_encoder	329
(n_) get_fly_2d_offset	334
(n_) get_marking_info	347
(n_) init_fly_2d	385
(n_) load_fly_2d_table	422
(n_) read_encoder	491
(n_) set_ext_start_delay	542
(n_) set_fly_tracking_error	550
(n_) set_mcbasp_in	585
(n_) set_multi_mcbasp_in	594
(n_) set_rot_center	611
(n_) simulate_encoder	657
(n_) simulate_ext_stop	660

Controlling Stepper Motors

(n_) get_stepper_status	360
(n_) stepper_abs	662
(n_) stepper_abs_no	663
(n_) stepper_control	665
(n_) stepper_enable	667
(n_) stepper_disable_switch	666
(n_) stepper_init	668
(n_) stepper_rel	670
(n_) stepper_rel_no	671

Jump Mode

(n_) get_jump_table	343
(n_) load_jump_table	423
(n_) load_jump_table_offset	424
(n_) set_jump_mode	560
(n_) set_jump_table	565

Other Control Commands

(n_) auto_cal	301
(n_) bounce_supp	306
(n_) get_auto_cal	326
(n_) get_galvo_controls	335
(n_) get_hi_data	340
(n_) get_hi_pos	341
(n_) get_time	364
(n_) home_position	376
(n_) home_position_xyz ⁽²⁾	377
(n_) load_zoom_correction_file	440
(n_) move_to	467
(n_) set_hi	557
(n_) set_pause_list_cond	601
(n_) set_zoom	656
(n_) write_hi_pos	720

(1) Only with Option "3D".

(2) Limited functionality if no Option "3D".

10.1.6 List Commands

Meanings:

nor	normal list command
var	variable list command
us	undelayed short list command
ds	delayed short list command
mul	multiple list command

Board Initialization and Image Field Correction

(n_) **select_cor_table_list** ^{var} 515

2D Jump Commands

(n_) **jump_abs** ^{nor} 388
 (n_) **jump_rel** ^{nor} 390
 (n_) **para_jump_abs** ^{nor} 469
 (n_) **para_jump_rel** ^{nor} 471
 (n_) **timed_jump_abs** ^{nor} 687
 (n_) **timed_jump_rel** ^{nor} 689
 (n_) **timed_para_jump_abs** ^{nor} 695
 (n_) **timed_para_jump_rel** ^{nor} 697

3D Jump Commands⁽¹⁾

(n_) **jump_abs_3d** ^{nor} 389
 (n_) **jump_rel_3d** ^{nor} 391
 (n_) **para_jump_abs_3d** ^{nor} 470
 (n_) **para_jump_rel_3d** ^{nor} 472
 (n_) **timed_jump_abs_3d** ^{nor} 688
 (n_) **timed_jump_rel_3d** ^{nor} 690
 (n_) **timed_para_jump_abs_3d** ^{mul} 696
 (n_) **timed_para_jump_rel_3d** ^{mul} 698

micro_vector[*] Commands

(n_) **micro_vector_abs** ^{nor} 464
 (n_) **micro_vector_abs_3d** ^{nor} 465
 (n_) **micro_vector_rel** ^{nor} 466
 (n_) **micro_vector_rel_3d** ^{nor} 467

2D Mark Commands

(n_) **arc_abs** ^{nor} 297
 (n_) **arc_rel** ^{nor} 299
 (n_) **mark_abs** ^{nor} 442
 (n_) **mark_ellipse_abs** ^{nor} 449
 (n_) **mark_ellipse_rel** ^{nor} 450
 (n_) **mark_rel** ^{nor} 451
 (n_) **para_mark_abs** ^{nor} 475
 (n_) **para_mark_rel** ^{nor} 477
 (n_) **set_ellipse** ^{us} 537
 (n_) **timed_arc_abs** ^{nor} 685
 (n_) **timed_arc_rel** ^{nor} 686
 (n_) **timed_mark_abs** ^{nor} 691
 (n_) **timed_mark_rel** ^{nor} 693
 (n_) **timed_para_mark_abs** ^{nor} 699
 (n_) **timed_para_mark_rel** ^{nor} 701

3D Mark Commands⁽¹⁾

(n_) **arc_abs_3d** ^{nor} 298
 (n_) **arc_rel_3d** ^{nor} 300
 (n_) **mark_abs_3d** ^{nor} 443
 (n_) **mark_rel_3d** ^{nor} 452
 (n_) **para_mark_abs_3d** ^{nor} 476
 (n_) **para_mark_rel_3d** ^{nor} 478
 (n_) **timed_mark_abs_3d** ^{nor} 692
 (n_) **timed_mark_rel_3d** ^{nor} 694
 (n_) **timed_para_mark_abs_3d** ^{mul} 700
 (n_) **timed_para_mark_rel_3d** ^{mul} 702

Text Commands

(n_) **mark_char** ^{us} 444
 (n_) **mark_char_abs** ^{us} 445
 (n_) **mark_text** ^{var} 456
 (n_) **mark_text_abs** ^{var} 457
 (n_) **select_char_set** ^{us} 511

Date, Time, Serial Numbers

(n_) **mark_date** ^{nor} 446
 (n_) **mark_date_abs** ^{nor} 448
 (n_) **mark_serial** ^{nor} 453
 (n_) **mark_serial_abs** ^{nor} 455
 (n_) **mark_time** ^{nor} 458
 (n_) **mark_time_abs** ^{nor} 460
 (n_) **select_serial_set_list** ^{us} 518
 (n_) **set_serial_step_list** ^{nor} 616
 (n_) **time_fix** ^{nor} 682
 (n_) **time_fix_f** ^{nor} 682
 (n_) **time_fix_f_off** ^{nor} 683

(1) Limited functionality if no Option "3D".

Status Monitoring and Diagnostics

(n_) set_trigger ^{ds}	634
(n_) set_trigger4 ^{ds}	642

Starting and Stopping Lists by External Control Signals and Master/Slave Synchronization

(n_) set_control_mode_list ^{nor}	530
(n_) set_extstartpos_list ^{us}	541

List Handling and Structured Programming

(n_) list_continue ^{nor}	403
(n_) list_jump_pos ^{us}	405
(n_) list_jump_rel ^{us}	407
(n_) list_next ^{us}	409
(n_) list_nop ^{nor}	409
(n_) list_repeat ^{us}	410
(n_) list_until ^{us}	412
(n_) long_delay ^{nor}	441
(n_) set_end_of_list ^{nor}	540
(n_) set_list_jump ^{us}	575
(n_) set_wait ^{nor}	646

Subroutine Commands

(n_) list_call ^{us}	397
(n_) list_call_abs ^{us}	399
(n_) list_call_abs_repeat ^{us}	400
(n_) list_call_repeat ^{us}	402
(n_) list_return ^{us}	411
(n_) sub_call ^{us}	675
(n_) sub_call_abs ^{us}	676
(n_) sub_call_abs_repeat ^{us}	677
(n_) sub_call_repeat ^{us}	678

Setting the Laser Parameters

(n_) config_laser_signals_list ^{ds}	312
(n_) set_auto_laser_params_list ^{ds}	525
(n_) set_encoder_speed ^{ds}	538
(n_) set_firstpulse_killer_list ^{us}	544
(n_) set_laser_delays ^{us}	569
(n_) set_laser_pin_out_list ^{us}	571
(n_) set_laser_pulses ^{ds}	572
(n_) set_laser_timing ^{ds}	574
(n_) set_pulse_picking_list ^{us}	610
(n_) set_qswitch_delay_list ^{us}	610
(n_) set_softstart_level_list ^{nor}	625
(n_) set_softstart_mode_list ^{var}	627
(n_) set_standby_list ^{ds}	629
(n_) set_vector_control ^{us}	643

Setting the Scanner Parameters

(n_) set_delay_mode_list ^{mul}	535
(n_) set_jump_speed ^{us}	564
(n_) set_mark_speed ^{ds}	576
(n_) set_scanner_delays ^{ds}	615
(n_) set_sky_writing_limit_list ^{us}	618
(n_) set_sky_writing_list ^{nor}	618
(n_) set_sky_writing_mode_list ^{nor}	620
(n_) set_sky_writing_para_list ^{nor}	623

Coordinate Transformations

(n_) set_angle_list ^{var}	521
(n_) set_defocus_list ^{var (1)}	532
(n_) set_defocus_offset_list ^{var (1)}	533
(n_) set_matrix_list ^{var}	579
(n_) set_offset_list ^{var}	598
(n_) set_offset_xyz_list ^{var (2)}	600
(n_) set_scale_list ^{var}	614

Online Positioning

(n_) apply_mcbp_list ^{nor}	296
(n_) set_mcbp_global_matrix_list ^{us}	581
(n_) set_mcbp_global_rot_list ^{us}	582
(n_) set_mcbp_global_x_list ^{us}	583
(n_) set_mcbp_global_y_list ^{us}	584
(n_) set_mcbp_matrix_list ^{us}	588
(n_) set_mcbp_rot_list ^{us}	591
(n_) set_mcbp_x_list ^{us}	592
(n_) set_mcbp_y_list ^{us}	593

Direct Laser and Scan Head Control

(n_) laser_on_list ^{var}	392
(n_) laser_on_pulses_list ^{var}	393
(n_) laser_signal_off_list ^{nor}	395
(n_) laser_signal_on_list ^{nor}	396
(n_) para_laser_on_pulses_list ^{var}	473

Pixel Output Mode

(n_) set_default_pixel_list ^{us}	530
(n_) set_n_pixel ^{var}	597
(n_) set_pixel ^{var}	603
(n_) set_pixel_line ^{nor}	604
(n_) set_pixel_line_3d ^{mul (2)}	606

(1) Only with Option "3D".

(2) Limited functionality if no Option "3D".

I/O Commands

(n_) clear_io_cond_list ^{us}	310
(n_) periodic_toggle_list ^{us}	485
(n_) read_io_port_list ^{us}	494
(n_) rs232_write_text_list ^{var}	506
(n_) set_free_variable_list ^{us}	556
(n_) set_io_cond_list ^{us}	559
(n_) set_mcbssp_out ^{us}	588
(n_) set_mcbssp_out_ptr_list ^{mul}	590
(n_) set_port_default_list ^{us}	608
(n_) write_8bit_port_list ^{ds}	714
(n_) write_da_1_list ^{ds}	716
(n_) write_da_2_list ^{ds}	717
(n_) write_da_x_list ^{ds}	719
(n_) write_io_port_list ^{ds}	721
(n_) write_io_port_mask_list ^{ds}	722

Conditional Commands

(n_) if_cond ^{us}	378
(n_) if_not_cond ^{us}	381
(n_) if_not_pin_cond ^{us}	383
(n_) if_pin_cond ^{us}	384
(n_) list_call_abs_cond ^{us}	400
(n_) list_call_cond ^{us}	401
(n_) list_jump_cond ^{us}	404
(n_) list_jump_pos_cond ^{us}	406
(n_) list_jump_rel_cond ^{us}	408
(n_) sub_call_abs_cond ^{us}	676
(n_) sub_call_cond ^{us}	677
(n_) switch_ioport ^{us}	679

Processing-on-the-fly Commands

(n_) activate_fly_2d ^{var (1)}	292
(n_) activate_fly_2d_encoder ^{mul (1)}	293
(n_) activate_fly_xy ^{var (1)}	294
(n_) activate_fly_xy_encoder ^{mul (1)}	294
(n_) clear_fly_overflow ^{us}	309
(n_) fly_return ^{nor}	323
(n_) fly_return_z ^{nor}	324
(n_) if_fly_x_overflow ^{us}	378
(n_) if_fly_y_overflow ^{us}	379
(n_) if_fly_z_overflow ^{us}	379
(n_) if_not_activated ^{us}	380
(n_) if_not_fly_x_overflow ^{us}	382
(n_) if_not_fly_y_overflow ^{us}	382
(n_) if_not_fly_z_overflow ^{us}	383
(n_) park_position ^{var (1)}	479
(n_) park_return ^{var (1)}	481
(n_) set_ext_start_delay_list ^{nor}	543
(n_) set_fly_2d ^{nor (1)}	545
(n_) set_fly_limits ^{us}	546
(n_) set_fly_limits_z ^{us}	547
(n_) set_fly_rot ^{nor (1)}	548
(n_) set_fly_rot_pos ^{nor (1)}	549
(n_) set_fly_x ^{nor (1)}	551
(n_) set_fly_x_pos ^{nor (1)}	552
(n_) set_fly_y ^{nor (1)}	553
(n_) set_fly_y_pos ^{nor (1)}	554
(n_) set_fly_z ^{nor (1)}	555
(n_) set_mcbssp_in_list ^{us (2)}	586
(n_) set_multi_mcbssp_in_list ^{nor (2)}	596
(n_) set_rot_center_list ^{ds}	611
(n_) simulate_ext_start ^{nor}	658
(n_) store_encoder ^{us}	674
(n_) wait_for_encoder ^{nor}	709
(n_) wait_for_encoder_in_range ^{mul}	710
(n_) wait_for_encoder_mode ^{mul}	711
(n_) wait_for_mcbssp ^{nor}	713

(1) Only with Option Processing-on-the-fly.

(2) Limited functionality if no Option Processing-on-the-fly.

Controlling Stepper Motors

(n_) stepper_abs_list ^{us}	663
(n_) stepper_abs_no_list ^{us}	664
(n_) stepper_control_list ^{us}	665
(n_) stepper_enable_list ^{us}	667
(n_) stepper_rel_list ^{us}	670
(n_) stepper_rel_no_list ^{us}	671
(n_) stepper_wait ^{nor}	672

Jump Mode

(n_) set_jump_mode_list ^{nor}	563
--	-----

Camming

(n_) camming ^{nor}	307
-----------------------------	-----

Wobbel Mode

(n_) set_wobbel ^{ds}	647
(n_) set_wobbel_control ^{us}	649
(n_) set_wobbel_direction ^{us}	650
(n_) set_wobbel_mode ^{ds}	651
(n_) set_wobbel_offset ^{ds}	653
(n_) set_wobbel_vector ^{us}	654

Other List Commands

(n_) jump_abs_drill ^{nor}	389
(n_) jump_abs_drill_2 ^{nor}	389
(n_) jump_rel_drill ^{nor}	391
(n_) jump_rel_drill_2 ^{nor}	391
(n_) range_checking ^{us}	487
(n_) save_and_restart_timer ^{ds}	508
(n_) set_zoom_list ^{us}	656

10.1.7 Data Types

The following table defines the formats and ranges of the different data types used by the RTC5 commands.

Data Format	Range	Pascal	C, C++	C#
unsigned 32-bit value	$[0; (2^{32}-1)]$	longword	unsigned long	uint
signed 32-bit value	$[-2^{31}; +(2^{31}-1)]$	longint	long	int
64-bit IEEE floating point value		double	double	double
pointer to a null-terminated ANSI string (1 byte per char)	4 Byte for Win32-user programs 8 Byte for Win64-user programs	pchar	char*	string

Pointer to Locations in the PC Memory

Some commands (for example, **get_transform**, **get_values**, **get_waveform**, **transform** or **upload_transform**) have pointers to locations in the PC memory as parameters. In C# and Pascal, appropriate pointer data types are herefore used (see import declarations). In C and C++, the data type ULONG_PTR is used for this pointer parameters. The ULONG_PTR data type is defined in the C and C++ import declarations as follows (ULONG_PTR = unsigned 32-bit value for Win32-based applications, ULONG_PTR = unsigned 64-bit value for Win64-based applications):

```
#if !defined(ULONG_PTR)
    #if !defined(_WIN64)
        #define ULONG_PTR UINT
    #else
        #define ULONG_PTR UINT64
    #endif // !defined(_WIN64)
#endif // !defined(ULONG_PTR)
```

Usually, the data type ULONG_PTR is also appropriately defined in the Windows header file <BaseTsd.h>.

10.2 RTC5 Command Set

The commands are in alphabetical order.

The general structure of the command tables is as follows⁽¹⁾:

(1) A program language-neutral form is used. Each real programming language has its own individual naming.

Category of the command	example_command_name_one	
Function	Short description describing the purpose of the command.	
Call	Shows the correct spellings and the sequence of the parameters. Note, there is no semicolon at the end of the line. A '&' (address operator) is only used in this table row and indicates a pointer. Examples: <pre>example_command_name_one(parameter_A, &parameter_B, parameter_C) example_variable = example_command_name_one(parameter_A)</pre>	
Parameters(*)	A	Short text. Data type. (*) in some C/C++ code descriptions labeled with __out.
	C	Short text. Data type.
Returned parameter values(**)	B	Short text. Data type. (**) in some C/C++ code descriptions labeled with __out.
Result	If implemented: mentions the returned result value and data type in generic form (Example: error code #. As an unsigned 32-bit value.). A value range is here only given, if the actual usable one is smaller than the value range of the data type. If not implemented: "None." List Commands generally have no return values.	
Comments	• Additional information on this command. • References to other chapters and publications.	
RTC4→RTC5	States the differences to the command (of the same name) of the RTC4 command set.	
Version info	For example, states the minimum versions of DLL, RBF, OUT which are required to use the command, latest change in version.	
References	Links to related commands: command_name_two , command_name_three	

Ctrl Command	acquire_rtc
Function	Acquires the specified RTC5 board for a user program.
Call	<code>NoOfAcquiredCard = acquire_rtc(CardNo)</code>
Parameters	CardNo RTC5 DLL -internal number (RTC5 board management index) of the desired board. As an unsigned 32-bit value.
Result	The return value is: • CardNo, if the acquisition has been successful • 0, if the board is currently acquired by another user program or the version check detects an error As an unsigned 32-bit value.
Comments	• acquire_rtc is also useful for single-board systems which need to coordinate the use of a board by multiple user programs. • acquire_rtc has no effect if CardNo exceeds the number of RTC5 Boards found during initialization (see rtc5_count_cards) or if CardNo = 0 (real boards begin with 1) (return value 0, get_last_error return code RTC5_PARAM_ERROR). • Access rights to existing boards are assigned exclusively (always to only <i>one</i> user program at a time). acquire_rtc therefore has no effect if the requested board is already reserved by another user program (return value 0, get_last_error return code RTC5_ACCESS_DENIED). Before an already-reserved board can be acquired by acquire_rtc, the user program with current access rights must explicitly release its rights by release_rtc or free_rtc5_dll. On the other hand, if the board has been already freed prior to initialization of a user program, then initialization by init_rtc5_dll result in RTC5 board management assigning board access rights for the user program. In this case, an explicit acquire_rtc call is not needed and has no effect. Nevertheless, the return value is the CardNo. • Assorted versions of the RTC5 DLL and the program files RTC5OUT.out, RTC5RBF.rbf and RTC5DAT.dat cannot be arbitrarily combined with another. acquire_rtc performs a version compatibility check. If no program files are loaded, then a version check cannot be explicitly executed (get_last_error return code RTC5_TIMEOUT), but the check still regarded as successful and acquisition is not hindered. If program files have been loaded and the version check determines an error, then access is denied (return value 0, get_last_error return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH). • A board successfully requested by acquire_rtc does not automatically become the active board. Activation is only achieved by select_rtc or init_rtc5_dll. • Running boards are neither halted nor initialized by acquire_rtc. • acquire_rtc is available even without explicit access rights to a particular RTC5 Board. • acquire_rtc is not available as a multi-board command. • See also Chapter 6.7.1 "Board Acquisition by a User Program", Page 117.

Ctrl Command	acquire_rtc
RTC4→RTC5	New command.
Version info	–
References	init_rtc5_dll, select_rtc, free_rtc5_dll, release_rtc, rtc5_count_cards

Variable List Command	activate_fly_2d
Function	Activates a set_fly_2d Processing-on-the-fly application without encoder resets.
Restriction	If the Option Processing-on-the-fly is not enabled, then activate_fly_2d terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	activate_fly_2d(ScaleX, ScaleY)
Parameters	ScaleX Scaling factor as for activate_fly_2d_encoder . ScaleY Like ScaleX.
Comments	• If no Processing-on-the-fly correction is active, then activate_fly_2d activates set_fly_2d Processing-on-the-fly correction (see Chapter 8.6.4 “Compensating 2D Motions”, Page 234). Unlike set_fly_2d, activate_fly_2d does not thereby reset the encoders, but instead recalculates the last Processing-on-the-fly-uncorrected coordinate values such that the Processing-on-the-fly-corrected output matches the current output. If the then-current recalculated coordinate values would have fallen outside the 24-bit virtual Image field, then Processing-on-the-fly correction is not activated and an error bit is set that is queryable by get_marking_info (Bit #9). • If Processing-on-the-fly correction is active, then activate_fly_2d is a short list command without further effect and merely sets an error bit queryable by get_marking_info (Bit #9). Therefore, activate_fly_2d cannot be used to modify the Processing-on-the-fly mode itself or to modify the scaling factors of the same mode. • You can also query the error bit by the short list command if_not_activated in order to possibly jump to an appropriate error handling routine. • Successful activation by activate_fly_2d does not reset any already-set error bit. It remains set for get_marking_info until get_marking_info is called. • If unallowed parameter values are supplied (for example, for ScaleX = 0), then activate_fly_2d is (already during loading) replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). • activate_fly_2d does not affect the laser signals (Signals for “Laser Active” Operation remain on/off if they were on/off).
RTC4→RTC5	New command. RTC4 Compatibility Mode: see set_fly_2d .
Version info	–
References	set_fly_2d, activate_fly_2d_encoder, activate_fly_xy

Multiple List Command	<code>activate_fly_2d_encoder</code>
Function	Activates a set_fly_2d Processing-on-the-fly application with encoder reset and encoder offsets.
Restriction	If the Option Processing-on-the-fly is not enabled, then <code>activate_fly_2d_encoder</code> terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>activate_fly_2d_encoder(ScaleX, ScaleY, EncX, EncY)</code>
Parameters	<code>ScaleX</code> Scaling factor as for set_fly_2d. <code>ScaleY</code> Like <code>ScaleX</code>. <code>EncX</code> Encoder offset. As a signed 32-bit value. <code>EncY</code> Like <code>EncX</code>.
Comments	• <code>activate_fly_2d_encoder</code> occupies two list memory area positions and also needs two 10 μs clock cycles to execute (the first part of <code>activate_fly_2d_encoder</code> is <i>not</i> a short list command). • <code>activate_fly_2d_encoder</code> is a combination of set_fly_2d (encoder reset) and activate_fly_2d (see also comments there). However, in <code>activate_fly_2d_encoder</code> the current (reset) encoder values are not used to calculate the Processing-on-the-fly-uncorrected virtual Image field coordinates, but the parameter values <code>EncX</code> and <code>EncY</code>. For the error handling, see comments of activate_fly_2d. • Because of this combination, <code>activate_fly_2d_encoder</code> saves the positioning stage motion (which is often long but may be necessary for the Processing-on-the-fly activation without this command) from the initialization position (for example, at the lower left corner) to the center and back again. • Subsequently all encoder values are offset with <code>EncX</code> and <code>EncY</code>, before the Processing-on-the-fly correction is applied. All other encoder related commands refer to the actual encoder values and behave as before. • If the value of <code>EncX</code> or <code>EncY</code> is not allowed (that is, the Processing-on-the-fly-corrected virtual Image field coordinates are outside the virtual Image field limits), then <code>activate_fly_2d_encoder</code> is replaced by a list_nop (get_last_error return code <code>RTC5_PARAM_ERROR</code>). • If the value of <code>ScaleX</code> or <code>ScaleY</code> is not allowed (see set_fly_2d), the first part is transferred to the board (and thus executes the encoder reset). However, the second part is replaced by a list_nop (get_last_error return code <code>RTC5_PARAM_ERROR</code>). • With active Processing-on-the-fly correction, <code>activate_fly_2d_encoder</code> is a short list command without further effect and merely sets an error bit queryable by get_marking_info (<code>Bit #9</code>). Therefore, <code>activate_fly_2d_encoder</code> cannot be used to modify the Processing-on-the-fly mode itself nor the scaling factors or encoder offsets of the same mode.

Multiple List Command	activate_fly_2d_encoder
RTC4→RTC5	New command. RTC4 Compatibility Mode: see set_fly_2d .
Version info	Available as of DLL 544, OUT 544.
References	set_fly_2d , activate_fly_2d , activate_fly_xy_encoder

Variable List Command	activate_fly_xy
Function	Activates a set_fly_x/set_fly_y Processing-on-the-fly application without encoder resets.
Restriction	If the Option Processing-on-the-fly is not enabled, then activate_fly_xy terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>activate_fly_xy(ScaleX, ScaleY)</code>
Parameters	ScaleX Scaling factor as for set_fly_x . ScaleY Scaling factor as for set_fly_y .
Comments	Like activate_fly_2d with the difference that a set_fly_x/set_fly_y Processing-on-the-fly session is activated.
RTC4→RTC5	New command. RTC4 Compatibility Mode: see set_fly_x/set_fly_y .
Version info	–
References	set_fly_x , set_fly_y , activate_fly_xy_encoder

Multiple List Command	activate_fly_xy_encoder
Function	Activates a set_fly_x/set_fly_y Processing-on-the-fly application with encoder reset and encoder offsets.
Restriction	If the Option Processing-on-the-fly is not enabled, then activate_fly_xy_encoder terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>activate_fly_xy_encoder(ScaleX, ScaleY, EncX, EncY)</code>
Parameters	ScaleX Scaling factor as for set_fly_x . ScaleY Scaling factor as for set_fly_y . EncX Encoder offset. As a signed 32-bit value. EncY Encoder offset. As a signed 32-bit value.
Comments	Like activate_fly_2d_encoder with the difference that a set_fly_x/set_fly_y Processing-on-the-fly session is activated.
RTC4→RTC5	New command. RTC4 Compatibility Mode: see set_fly_x/set_fly_y .
Version info	Available as of DLL 544, OUT 544.
References	set_fly_x , set_fly_y , activate_fly_xy , activate_fly_2d_encoder

Ctrl Command	apply_mcbbsp
Function	Queries the most recent values fully transmitted over the McBSP interface for “ Local Online Positioning ” and defines offset and/or rotation matrix M_R or general transformation matrix M_T for subsequent coordinate transformations.
Call	<code>apply_mcbbsp(HeadNo, at_once)</code>
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. = 1: The definition only affects the first scan head connector. = 2: The definition only affects the second scan head connector. = 0, 3: The definition affects <i>both</i> scan head connectors. Only the two least significant bits are evaluated.
	at_once Determines when the defined transformation becomes effective. As an unsigned 32-bit value. = 0: The new total transformation (total matrix and offset) is only calculated and applied to the current position when the next list command is executed. = 1: The new total transformation is calculated immediately (or before the next list command if currently BUSY list execution status or INTERNAL-BUSY list execution status is set) and applied to the current position. = 2: The new total transformation (total matrix and offset) is only calculated and applied to the current position when the next jump_abs, jump_rel, goto_xy or goto_xyz is executed. > 2: Like <code>at_once = 2</code>.
Comments	• Data acquisition by the McBSP interface for “Local Online Positioning” must be activated in advance by set_mcbbsp_x, set_mcbbsp_y and/or set_mcbbsp_rot or set_mcbbsp_matrix (or with the corresponding list commands). Depending on the configuration, apply_mcbbsp only defines (as with set_offset) an x offset and/or y offset or also (as with set_angle) a rotation matrix or (as with set_matrix) a general matrix operation (see Chapter 8.3.1 ““Local Online Positioning””, Page 213). As with the commands described in Chapter 8.2 “Coordinate Transformations”, Page 209, the parameter <code>at_once</code> determines when the newly defined total transformation becomes effective. • Transformations previously defined by set_angle, set_offset or set_matrix get overwritten by apply_mcbbsp. In contrast, transformations and focus shifts previously defined by set_scale or set_defocus and z offsets defined by set_offset_xyz is continued to be taken into account, when the total transformation gets recalculated. • Any new definitions made with set_angle, set_offset or set_matrix overwrite coordinate transformations defined by the McBSP interface. • The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbbsp_init. That is, data provided is not transmitted, see Section “RTC5 as Transmitter”, Page 72.

Ctrl Command	apply_mcbbsp
RTC4→RTC5	New command.
Version info	–
References	apply_mcbbsp_list , set_mcbbsp_x , set_mcbbsp_y , set_mcbbsp_rot , set_mcbbsp_matrix

Normal List Command	apply_mcbbsp_list
Function	Like apply_mcbbsp , but a list command.
Call	<code>apply_mcbbsp_list(HeadNo, at_once)</code>
Parameters	HeadNo Like apply_mcbbsp. at_once Determines when the defined transformation becomes effective. As an unsigned 32-bit value. = 0: The transformation settings are only collected and intermediately stored, but the transformation is not processed as long as this is not activated by another coordinate transformation (for example, by a list command with <code>at_once = 1</code> or a corresponding control command). = 1: The transformation is immediately calculated (including all transformation settings that were collected until then) and processed prior to the next list command. = 2: The transformation settings are only collected and intermediately stored (as with <code>at_once = 0</code>). However, The transformation is immediately calculated (including all transformation settings that were collected and intermediately stored until then) and applied to the current position when the next jump_abs or jump_rel (only if no list is currently being executed: also goto_xy or goto_xyz) is executed. > 2: As <code>at_once = 2</code>.
Comments	• See apply_mcbbsp.
RTC4→RTC5	New command.
Version info	–
References	apply_mcbbsp

Normal List Command	arc_abs
Function	Moves the laser focus from the current position at mark speed along an arc with the specified angle and center point (absolute coordinate values) within a 2D Image field .
Parameters	X Absolute x coordinate of the arc center. In bits. As a signed 32-bit value. Allowed value range: $[-8,388,608 \dots +8,388,607]$. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
	Angle Arc angle. In degrees. As a 64-bit IEEE floating point value. A positive sign means "clockwise". Allowed value range: $[-3,600.0^\circ \dots +3,600.0^\circ]$ (± 10 full circles). Out-of-range values are clipped to the boundary values.
Comments	- If the mark speed has not been previously explicitly set by set_mark_speed or set_mark_speed_ctrl, then the marking is executed at a predefined mark speed of 1000 <i>bits/ms</i>. - The Signals for "Laser Active" Operation are automatically turned on at the beginning of the marking (or remain on after a directly preceding [*]mark[*] Command or "Arc" command). The defined Scanner Delays and Laser Delays are thereby taken into account (see Chapter 7.2 "Delay Settings – Coordinating Scan Head Control and Laser Control", Page 133). Note that other delays are executed in Sky Writing mode . Exception: zero-length "Arc" commands (see notes on page 137).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for X and Y by 16. The allowed value ranges decrease accordingly.
Version info	–
References	set_mark_speed , set_scanner_delays , arc_rel , timed_arc_abs , mark_abs , arc_abs_3d , mark_ellipse_abs

Normal List Command	arc_abs_3d
Function	Moves the laser focus at mark speed from the current position helical around an axis parallel to the z axis. The x and y components thereby characterize an arc with the specified angle around the specified axis, while the z component characterizes a linear motion from the current position to the specified end point. The position of the helical axis and the z end coordinate are specifiable as absolute coordinate values.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then arc_abs_3d has the same effect as arc_abs .
Call	<code>arc_abs_3d(X, Y, Z, Angle)</code>
Parameters	X Position of the helical axis (parallel to the z axis) as absolute x coordinate. In bits. As a signed 32-bit value. Allowed value range: [−8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
	Z Absolute z end coordinate. In bits. As a signed 32-bit value. Allowed value range: [−32,768...+32,767]. Out-of-range values are clipped to the boundary values.
	Angle Arc angle. In degrees. As a 64-bit IEEE floating point value. Positive angle values correspond to clockwise angles. Allowed value range: [−3600.0°...+3600.0°] (±10 full circles). Out-of-range values are clipped to the boundary values.
Comments	- Except for the additional motion in the third dimension, arc_abs_3d functions similarly to arc_abs (see the comments there). - The z motion is not taken into account during calculation of the number of Microsteps.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the values specified for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode .
Version info	–
References	arc_abs , arc_rel_3d

Normal List Command	arc_rel
Function	Moves the laser focus from the current position at mark speed along an arc with the specified angle and center point (relative coordinate values) within a 2D Image field .
Call	<code>arc_rel(dX, dY, Angle)</code>
Parameters	dX Relative x coordinate of the arc center. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values.
	dY Like dX (analogously).
	Angle Arc angle. In degrees. As a 64-bit IEEE floating point value. A positive sign means "clockwise". Allowed value range: [-3,600.0°...+3,600.0°] (±10 full circles). Out-of-range values are clipped to the boundary values.
Comments	The coordinates for the arc center are to be supplied as relative coordinates with respect to the current position. Otherwise, arc_rel is identical to arc_abs (see the comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly.
Version info	–
References	set_mark_speed , set_scanner_delays , arc_abs , timed_arc_rel , mark_rel , arc_rel_3d , mark_ellipse_rel

Normal List Command	arc_rel_3d	
Function	Moves the laser focus at mark speed from the current position spirally around an axis parallel to the z axis. The x and y components thereby characterize an arc with the specified angle around the specified axis, while the z component characterizes a linear motion from the current position to the specified end point. The position of the helical axis and the z end coordinate are specifiable as relative coordinate values.	
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then arc_rel_3d has the same effect as arc_rel .	
Call	arc_rel_3d(dX, dY, dZ, Angle)	
Parameters	dX	Position of the helical axis (parallel to the z axis) as relative x coordinate. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values.
	dY	Like dX (analogously).
	dZ	Relative z end coordinate. In bits. As a signed 32-bit value. Allowed value range: [-32,768...+32,767]. Out-of-range values are clipped to the boundary values.
	Angle	Arc angle. In degrees. As a 64-bit IEEE floating point value. A positive sign means "clockwise". Allowed value range: [-3,600.0°...+3,600.0°] (±10 full circles). Out-of-range values are clipped to the boundary values.
Comments	The position of the helical axis (dX, dY) and the z end coordinate (dZ) are to be supplied as relative coordinates with respect to the current position. Otherwise, arc_rel_3d is identical to arc_abs_3d (see the comments there).	
RTC4→RTC5	New command. RTC4 Compatibility Mode : see arc_abs_3d	
Version info	–	
References	arc_abs_3d , arc_rel	

Ctrl Command	auto_cal
Function	Controls the functions for (automatic self-) calibration of the scan system attached to the specified scan head connector.
Call	ErrorCode = auto_cal(HeadNo, Command)
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. Allowed values: = 1: First scan head connector. = 2: Second scan head connector. Requires Option "Second Scan Head Control".
	Command Control parameter. As an unsigned 32-bit value. Allowed value range: [0...4]. = 0: The RTC5 detects the current Home-In positions, stores them in the DLL and in the EEPROM as Home-In reference values and initializes the gain values and offset values (Gain = 1.0, Offset = 0). = 1: The RTC5 detects the current Home-In positions, calculates and sets the new gain values and offset values and thereby activates drift compensation. = 2: The RTC5 deactivates drift compensation by initializing the gain and offset values (Gain = 1.0, Offset = 0). = 3: The RTC5 detects the current Home-In positions (but – in comparison to Command = 1 – leaves the gain and offset values unchanged and that is, does not activate drift compensation) = 4: The RTC5 checks the ASC hardware (that is, checks whether a scan system attached to the specified scan head connector is equipped with an internal sensor system for automatic self-calibration – Home-In sensors) and returns the type and status of the detected sensor system. The detected type is also stored in the EEPROM.
Result	Error code or type of sensor system. As an unsigned 32-bit value. 3 auto_cal cannot be executed because the BUSY list execution status or INTERNAL-BUSY list execution status is currently set. 6 Parameter error. The following error codes are only returned after Command = 0...3: 0 No error. 1, 10, 11 Home-In sensor not found (this could also mean a Home-In sensor is defective) (1: for x axis (Galvanometer scanner 2) 10: for y axis (Galvanometer scanner 1) 11: for both axes). 2, 20, 22 The spread in measured values during a measurement cycle is too high (2: for x axis / 20: for y axis / 22: for both axes). 4, 40, 44 Reference data not found (only for Command = 1 and 3) (4: for x axis / 40: for y axis / 44: for both axes).

Ctrl Command	auto_cal
Result (cont'd)	5, 50, 55 Calibration error (Error during calibration or error in reference data). 5: for x axis / 50: for y axis / 55: for both axes. 9, 90 Sensor for x axis or y axis is defective. Only returned after <code>Command = 0, 1 or 3</code>. The following error code is only returned after <code>Command = 0 or 4</code>, and even then only if no other errors occurred: 8 EEPROM write error (for this error, the get_last_error return code RTC5_EEPROM_ERROR is always generated). The following values are only returned after <code>Command = 4</code>: 100 For both axes: a sensor system of type1 is included and is functioning. 200 For both axes: a sensor system of type2 is included and is functioning. 19 x axis (Galvanometer scanner 2): a sensor system of type1 is included and is functioning. y axis (Galvanometer scanner 1): sensor system is defective. 29 x axis (Galvanometer scanner 2): a sensor system of type2 is included and is functioning. y axis (Galvanometer scanner 1): sensor system is defective. 91 x axis (Galvanometer scanner 2): sensor system is defective. y axis (Galvanometer scanner 1): a sensor system of type1 is included and is functioning. 92 x axis (Galvanometer scanner 2): sensor system is defective. y axis (Galvanometer scanner 1): a sensor system of type2 is included and is functioning. 99 For both axes: sensor system is defective. 255 For both axes: there is no sensor system included in the scan system.
Comments	• For auto_cal usage, see Chapter 8.10 "Automatic Self-Calibration", Page 251. • At the end of this command's execution, the RTC5 always moves the galvanometer scanners back to the position held prior to the call, possibly with corrections attributable to changed gain values and offset values. • During determination of the current Home-In positions with auto_cal (<code>Command = 0, 1 or 3</code>), the current gain and offset corrections of the galvanometer scanners are not taken into account; neither are any head corrections or the current positions of the scanners. • After first-time or renewed connecting a scan system, a reference value determination should be performed by auto_cal (<code>Command = 0</code>). Otherwise, the RTC5 uses the stored reference values of a previously operated (possibly different) scan system when later performing an automatic self-calibration. • After initialization of the RTC5, or after a reset, drift compensation is turned off (gain = 1.0, offset = 0). However, previously determined reference values are still available.

Ctrl Command	auto_cal
Comments (cont'd)	• If no appropriate Home-In reference values were stored or the scan system is not equipped with Home-In sensors, then the measurement routine for <code>auto_cal</code> (Command = 1) (axis-specific) automatically aborts and restores the prior state. • <code>auto_cal</code> is not executed, if a HeadNo or Command value is invalid (return value 6, <code>get_last_error</code> return code <code>RTC5_PARAM_ERROR</code>). This also applies for HeadNo = 2 if the Option "Second Scan Head Control" is not enabled (return value 6). • <code>auto_cal</code> is not executed (return value 3, <code>get_last_error</code> return code <code>RTC5_BUSY</code>), if: – the <code>BUSY</code> list execution status is set – the <code>INTERNAL-BUSY</code> list execution status is set • <code>auto_cal</code> is even executed, if: – a list has been paused by <code>set_wait</code> (<code>PAUSED</code> list execution status set) • For <code>RTC5_PARAM_ERROR</code>, the <code>BUSY</code> list execution status is not checked; therefore return codes <code>RTC5_BUSY</code> and <code>RTC5_PARAM_ERROR</code> do not occur simultaneously. • For Command = 0, 1 or 2, a valid correction table must be loaded and assigned. Otherwise, unexpected jumps can occur when gain/offset values are updated. • Gain and offset correction can also be directly set by <code>set_hi</code> (even for systems <i>without</i> Home-In sensors). • Reference values determined and stored by <code>auto_cal</code> (Command = 0, 1 or 3) can be queried by <code>get_hi_pos</code>. • ASC hardware checks are performed not just by <code>auto_cal</code> (Command = 4), but also automatically for – <code>auto_cal</code> (Command = 0) and – the first call of <code>auto_cal</code> (Command = 1) and <code>auto_cal</code> (Command = 3) if neither <code>auto_cal</code> (Command = 0) nor <code>auto_cal</code> (Command = 4) were previously executed. In each case, the detected ASC hardware type gets stored in the <code>EEPROM</code> and can be subsequently queried by <code>get_auto_cal</code>. For automatic Command = 4 execution, however, no corresponding return value is generated. The return value is instead simply that of the primary Command call (see above). When an error occurs, a corresponding error code gets saved to the <code>EEPROM</code> (therefore, <code>get_auto_cal</code> does not return 100 or 200). As soon as hardware functionality is restored, you can clear the error from the <code>EEPROM</code> by explicitly calling <code>auto_cal</code> (Command = 0) or <code>auto_cal</code> (Command = 4). The error is <i>not</i> cleared from the <code>EEPROM</code> by calling <code>auto_cal</code> (Command = 1).
RTC4→RTC5	The functions for automatic self-calibration (Command = 0...2) are (largely) unchanged. New: Command = 3 and Command = 4.
Version info	–
References	<code>set_hi</code> , <code>get_hi_pos</code> , <code>get_auto_cal</code> , <code>write_hi_pos</code>

Ctrl Command	auto_change
Function	Activates a one-time automatic list change.
Call	<code>auto_change()</code>
Comments	• auto_change is synonymous with <code>auto_change_pos(0)</code>. This starts the subsequent list at its beginning. • See also comments for auto_change_pos.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	auto_change_pos, get_status, read_status

Ctrl Command	auto_change_pos
Function	Activates a one-time automatic list change and simultaneously defines the list position at which execution continues.
Call	<code>auto_change_pos(Pos)</code>
Parameters	Pos Start position (list memory address) as an offset referenced to the beginning of the list to be started by the automatic list change. As an unsigned 32-bit value.
Comments	• auto_change_pos triggers a subsequent <i>one-time-only</i> list change or a list new start. For further list changes or list new starts, auto_change_pos must be called again. • auto_change_pos can be called at any desired point in time. • If the automatic list change is activated during processing of a list, then upon reaching set_end_of_list execution continues without delay at the supplied start position of the other list. If there is only <i>one</i> list (Mem2 = 0, see config_list), then upon reaching set_end_of_list execution continues at the supplied start position of this list. • During processing of a list, the other list (and also the current list) can be newly loaded (see Chapter 6.4.6 "Automatic List Changing", Page 99). • So that auto_change_pos can function at all, any already active list must absolutely be finalized by set_end_of_list; the new list should already be loaded and the input pointer should be sufficiently ahead of the output pointer (otherwise, "old" commands are executed). If, during list execution, the end of the list is reached without encountering a set_end_of_list, then execution automatically continues at the beginning of the current list. • If an automatic list change is activated when no list is currently being processed, then checking takes place as to whether a list has been already processed and the other list has been started (at the supplied start position). If no list has been previously executed, then "List 1" is regarded as already executed (initialization) and "List 2" is started. • If a list memory address outside the corresponding list area is supplied (depending on which list should be started: $Pos \geq Mem1$ or $Pos \geq Mem2$), then the start position is set to the beginning of the list ($Pos = 0$). • If, during processing of a list, the auto_change_pos($Pos > 0$) and start_loop commands are called, then upon the next set_end_of_list the command auto_change_pos($Pos > 0$) is executed; and at the next one the start_loop command is executed. • The current List Status values can be queried by read_status. • The current List Execution Status values can be queried by get_status. • auto_change_pos triggers a flush of the buffered list input, see Chapter 6.4.1 "Loading Lists", Page 94.

Ctrl Command	auto_change_pos
RTC4→RTC5	Essentially unchanged, however: The list memory address (Pos) is supplied to the RTC5 as a relative memory address referenced to the beginning of the respective list, whereas the RTC4 is supplied an absolute memory address (0...7999). If no memory area is assigned to "List 2" by config_list (Mem2 = 0), then the RTC5 command behaves like the RTC4 command.
Version info	–
References	auto_change , get_status , read_status

Ctrl Command	bounce_supp
Function	Debounces the external start signal.
Call	<code>bounce_supp(Length)</code>
Parameters	Length Debouncing time. In ms. As an unsigned 32-bit value. Allowed value range: [0...1023]. The 22 bits with higher significance are ignored.
Comments	• bounce_supp enables debouncing of start signals received at the /START, /START2 or /Slave-START inputs, see Section "External Start", Page 266. Start signals occurring within the defined debouncing time after a successful start signal are thereby suppressed. • Recommended procedure: Start a list, which operates more than one second. If /START, /START2 or /Slave-START bounces, then an additional trigger error signal is generated, which can be detected by get_marking_info (Bit #8). Increase the debouncing time until this additional signal is no longer detected. • The debouncing time default value is 0 ms.
RTC4→RTC5	New command.
Version info	–
References	get_startstop_info

Normal List Command	camming	
Function	Enables Camming functionality.	
Call	camming (FirstPos, NPos, EncoderNo, Ctrl, Scale, Code)	
Parameters	FirstPos	Starting address of the Camming command list (absolute address in list memory). As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{20}-1)]$.
	NPos	Length of the Camming command list (without terminating set_end_of_list or list_return). As an unsigned 32-bit value. Allowed value range: $[1 \dots (2^{20}-\text{FirstPos})]$.
	EncoderNo	Number of the encoder counter, whose pulses is evaluated for controlling the Camming process. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1". Bits with higher significance are ignored.
	Ctrl	Camming control mode. As an unsigned 32-bit value. Allowed values: = 0: RTC5 controls laser similarly to a [*]mark[*] Command . camming terminates automatically. = 1: User controls laser. camming terminates automatically. = 2: User controls laser. camming does <i>not</i> terminate automatically. The Camming command list is executed until the end point (0 or NPos-1) and then waits for a new external /START. = 3: User controls laser. camming does <i>not</i> terminate automatically. The Camming command list is continuously processed in circulation (output index modulus NPos).
	Scale	Translation (conversion factor) between the encoder-counter value and the command index. As a 64-bit IEEE floating point value. Allowed value range: $2^{-60} < \text{Scale} < 2^{60}$.
	Code	As an unsigned 32-bit value. Is not used.
Comments	- See also Chapter 8.11 "Camming", Page 255. - If there are unallowed parameter values, camming is replaced by a list_nop already during loading (get_last_error return code RTCS_PARAM_ERROR).	

Normal List Command	camming
Comments (cont'd)	- Each time camming is called, the current encoder-counter value is ascertained for use as the new reference value. At a later point in time, the corresponding command index of the Camming command list is calculated for the then-current encoder-counter value (using the reference value and the Scale parameter). For each call of camming, the first command in the Camming command list (index = 0) is always the first command to be processed. camming waits for a scanner delay but sets no delay itself. - If Ctrl = 0, then the laser is (as with a normal mark command) switched on at the beginning of the Camming process and switched off after it terminates (laser delay settings are taken into account). If Ctrl > 0, then the state of the laser is not changed; its control is then the full responsibility of the user. - If Ctrl = 0 or 1, then camming terminates automatically (in the next cycle) as soon as the index first undershoots 0 or overshoots NPos-1 (the final Camming command to be executed is then be the one with an index of 0 or NPos-1). For these two modes (as always, if a list is BUSY), no External Starts are allowed as long as camming has not yet terminated. - In contrast, control modes Ctrl = 2 and 3 are endless. Here, the Camming process can only be terminated by stop_execution or an External Stop. However, in these two modes (as an exception) External Starts are allowed if the list (or camming) is still active. If Ctrl = 2, then the index is executed until the end point (0 or NPos-1) and then waits for a new external /START. If Ctrl = 3, then the index is set to modulus NPos (whereby the encoder speed is supposed not to be so high that a complete rotation is skipped over). Thus Ctrl = 3 works like a ring buffer. camming is a normal list command, but with a variable execution period. camming functions even if the Option Processing-on-the-fly is not activated. - While camming is executing, get_out_pointer always provides the position of camming, not the position of the current index.
RTC4→RTC5	New command.
Version info	–
References	set_encoder_speed , set_auto_laser_control

Undelayed Short List Command	clear_fly_overflow
Function	Resets the specified error bits (from get_marking_info) for customer-defined monitoring of Processing-on-the-fly applications.
Call	<code>clear_fly_overflow(Mode)</code>
Parameters	Mode The error bits to be reset. As an unsigned 32-bit value. Bit #0 = 1: error bit Bit #4 (underflow X). Bit #1 = 1: error bit Bit #5 (overflow X). Bit #2 = 1: error bit Bit #6 (underflow Y). Bit #3 = 1: error bit Bit #7 (overflow Y). Bit #4 = 1: error bit Bit #24 (underflow Z). Bit #5 = 1: error bit Bit #25 (overflow Z). Bit #0...Bit #5 can be combined as desired. Higher-order bits are ignored.
Comments	- For usage of clear_fly_overflow, see Section "Customer-Defined Monitoring Area", Page 241. - All 6 error bits are reset for: - Mode = 0 - Mode = 63
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , clear_fly_overflow_ctrl

Ctrl Command	clear_fly_overflow_ctrl
Function	Like clear_fly_overflow , but a control command.
Call	<code>clear_fly_overflow_ctrl(Mode)</code>
Parameters	Mode Like clear_fly_overflow .
Comments	See clear_fly_overflow.
RTC4→RTC5	New command.
Version info	Available as of DLL 548, OUT 548.
References	clear_fly_overflow

Undelayed Short List Command	<code>clear_io_cond_list</code>
Function	Clears the bits of the 16-bit digital output port on the EXTENSION 1 socket connector that are set in the parameter <code>MaskClear</code>, if the current <code>IOvalue</code> at the 16-bit digital <i>input</i> port on the EXTENSION 1 socket connector meets the following condition: $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits specified in <code>Mask1</code> are 1 and the bits specified in <code>Mask0</code> are 0).
Call	<code>clear_io_cond_list(Mask1, Mask0, MaskClear)</code>
Parameters	<code>Mask1</code> 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	<code>Mask0</code> See <code>Mask1</code> .
	<code>MaskClear</code> See <code>Mask1</code> .
Comments	• <code>clear_io_cond_list</code> clears only those bits of the digital output port that are set in the parameter <code>MaskClear</code> and leaves the other bits unchanged. • See also Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65 and Chapter 9.3.2 "Conditional Command Execution", Page 271.
Examples (Pascal)	• Clear Bit #4 of the output port (DIGITAL OUT4), if Bit #0 of the input port (DIGITAL IN0) is set and Bit #1 to Bit #3 (DIGITAL IN1...3) of the input port are not set: <code>clear_io_cond_list(\$0001, \$000E, \$0010)</code> • Always clear Bit #15 of the output port (and leave the other bits unchanged): <code>clear_io_cond_list(0, 0, \$8000)</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_io_cond_list , write_io_port , write_io_port_mask , get_io_status , read_io_port

Ctrl Command	config_laser_signals
Function	Configures the laser signal types to be outputted on pin 1 (LASER1), pin 2 (LASERON) and pin 9 (LASER2) of the LASER connector.
Call	<code>config_laser_signals(Config)</code>
Parameters	Config Desired signal configuration. As an unsigned 32-bit value. The following bits configure: Bit #0...1: Pin 2 (LASERON channel) Bit #2...3: Pin 1 (LASER1 channel) Bit #4...5: Pin 9 (LASER2 channel) Bit #6: Ignored ... Bit #31: Ignored Meanings: = 0 = 00_b: LASERON signal = 1 = 01_b: LASER1 signal = 2 = 10_b: LASER2 signal = 3 = 11_b: FirstPulseKiller signal (relevant for YAG Mode, Laser Mode 4; see Section "FirstPulseKiller Signal", Page 179 and set_firstpulse_killer) The default setting (after load_program_file) is: Config = 100100_b = 0x24 = 36
Comments	- The specified configuration takes effect the next time the laser is switched on. Therefore, config_laser_signals should not be called during an active marking procedure. - See also set_laser_control, Bit #3 and Bit #4.
Examples	- Config = 000111_b: FirstPulseKiller signal on LASERON channel, LASER1 signal on LASER1 channel, LASERON signal on LASER2 channel. In this configuration, LASER1 signals synchronously generated with the LASERON signal can be immediately outputted, whereas the laser itself switches on only after a delay by the FirstPulseKiller signal. - Config = 100100_b (default setting): LASERON signal on the LASERON channel, LASER1 signal on the LASER1 channel, LASER2 signal on the LASER2 channel. In this configuration, LASER1 signals can only be switched on after the laser, not before it (here the LASERON signal is the laser start signal; LASER1 signals cannot be outputted before the laser start, because the RTC5 only generates LASER1 signals if the LASERON signal is on).
RTC4→RTC5	New command.
Version info	–
References	config_laser_signals_list , set_laser_control

Delayed Short List Command	<code>config_laser_signals_list</code>
Function	Like <code>config_laser_signals</code> , but a list command.
Call	<code>config_laser_signals_list(Config)</code>
Parameters	Config Like <code>config_laser_signals</code> .
Comments	• See <code>config_laser_signals</code>.
RTC4→RTC5	New command.
Version info	–
References	<code>config_laser_signals</code>

Ctrl Command	<code>config_list</code>
Function	Configures the list memory, that is, assigns specific memory locations to the list memory areas.
Call	<code>config_list(Mem1, Mem2)</code>
Parameters	Mem1 Storage positions for list memory area "List 1". As an unsigned 32-bit value.
	Mem2 Storage positions for list memory area "List 2". As an unsigned 32-bit value.
Comments	- The RTC5 list memory contains 1,048,576 ($= 2^{20}$) storage positions in total. They can be divided into three areas ("lists") by <code>config_list</code>. The sizes of "List 1" and "List 2" are specified through parameters <code>Mem1</code> and <code>Mem2</code>. <code>config_list</code> automatically assigns to the protected area ("List 3") all remaining memory not assigned to "List 1" or "List 2". - The following rules apply to the parameters <code>Mem1</code> and <code>Mem2</code> (invalid values are automatically corrected in the specified order): <code>Mem1 > 0</code> ("List 1" cannot be empty) <code>Mem1 = 0</code> is corrected to <code>Mem1 = 1</code>. <code>Mem1 ≤ 2²⁰</code> ("List 1" can contain a maximum of 2^{20} storage positions) <code>Mem1 > 2²⁰</code> is corrected to <code>Mem1 = 2²⁰</code> <code>Mem1 = "-1"</code> is interpreted as <code>Mem1 = (2³²-1)</code> and corrected to <code>Mem1 = 2²⁰</code>. Example: With <code>config_list(-1, x)</code> where <code>x</code> is any value (also <code>x = -1</code>), "List 1" is automatically assigned the entire list memory (<code>Mem1 = 2²⁰</code>, <code>Mem2 = 0</code>, no memory for "List 3"). <code>Mem2 ≤ 2²⁰ - Mem1</code> ("List 2" can maximally receive the "rest" of list memory) <code>Mem2 = 0</code> is allowed. <code>Mem2 > 2²⁰ - Mem1</code> is corrected to <code>Mem2 = 2²⁰ - Mem1</code>. <code>Mem2 = "-1"</code> is interpreted as <code>Mem2 = (2³²-1)</code> and corrected to <code>Mem2 = 2²⁰ - Mem1</code>. Example: With <code>config_list(Mem1, -1)</code>, "List 2" is automatically assigned with the "rest" of list memory (<code>Mem2 = 2²⁰ - Mem1</code>, no memory for "List 3"). - Storage positions for "List 3": $2^{20} - \text{Mem1} - \text{Mem2}$ - By default, the RTC5's list memory area is preconfigured so that "List 1" and "List 2" can each accept 4000 list commands (<code>Mem1 = Mem2 = 4000</code>). The protected "List 3" then owns the remaining 1040576 of the 2^{20} storage positions. <code>config_list</code> is not executed (get_last_error return code <code>RTC5_BUSY</code>), if: - the BUSY list execution status is set - a list has been paused by set_wait (PAUSED list execution status set)

Ctrl Command	config_list
Comments (cont'd)	- Configuration by config_list does not alter the contents of list memory. Repeating the call with differing parameters is therefore nondestructive. However, after a configuration change, previously loaded list commands are processed in accordance with the new configuration. Moreover, a configuration change could in some circumstances affect the input pointer (if, prior to the reconfiguration, it pointed to a memory position which has been assigned to "List 3" by the configuration change, then it is shifted to the beginning of "List 1") or affect a previously started automatic list change. This should be taken into account when further loading or executing command lists. Also observe the notes in Chapter 6.3.2 "Configuring the RTC5 List Memory", Page 92. - If you do not know the current configuration data for list memory (Mem1 and Mem2), you can find out after <code>load_list(ListNo, 0)</code> or <code>set_start_list_pos(ListNo, 0)</code> by using <code>get_list_space</code> (if the board changed "ownership" previously call <code>get_config_list</code>).
RTC4→RTC5	New command.
References	get_config_list

Ctrl Command	control_command	
Function	Sends a control command. Whether and how the addressed scan system reacts depends on its properties (among other things, the scan system firmware). See also Comments, Page 316 .	
Call	control_command(Head, Axis, Data)	
Parameters	Head	Scan head connector number of the RTC control board. As an unsigned 32-bit value. Allowed values: = 1: The scan system at 1st scan head connector. "Scan head A". = 2: The scan system at 2nd scan head connector. "Scan head B".
	Axis	Number of the axis. As an unsigned 32-bit value. Allowed values: = 1: x axis (STATUS channel , Galvanometer scanner 2). = 2: y axis (STATUS1 channel , Galvanometer scanner 1).
	Data	Command code with optional parameter. See Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards" , Page 747 . As an unsigned 32-bit value. The upper part (Bit #16...Bit #31) is <i>not</i> evaluated. The lower part (Bit #0...Bit #15) is evaluated as follows: • Code_{HIGH} The more significant data byte (Bit #8...Bit 15). Represents a command code. Presented as hexadecimal number in Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747. • Code_{LOW} The less significant data byte (Bit #0...Bit #7). Represents an optional parameter. Presented as hexadecimal number in Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747. Example: By Data = 0501 _H , that is, Code _{HIGH} = 05 (SetMode) and Code _{LOW} = 01 the actual position is set as the data type to be transmitted.

Ctrl Command	control_command
Comments	• control_command can only be used in conjunction with iDRIVE scan systems (see Glossary entry on page 23). • control_command is not evaluated by conventional scan systems (without iDRIVE technology). • control_command is not executed with invalid values of Head and/or Axis (get_last_error return code <code>RTC5_PARAM_ERROR</code>). • Command code Data is transmitted to the scan system instead of the usual position data. Therefore, the corresponding galvanometer scanner Microstep is omitted, if control_command is called during execution of a list. • Under some circumstances, control_command might be unavailable at the first scan head connector if speed-dependent laser control has been activated by <code>set_auto_laser_control(Mode = 2)</code>.
RTC4→RTC5	Unchanged functionality. Note that all returned data selected by control_command are always in 20-bit range, even in RTC4 Compatibility Mode (see also comments on get_value).
Version info	–
References	get_value , get_values , get_head_status , set_trigger , set_trigger4 , get_waveform

Ctrl Command	copy_dst_src	
Function	Creates entries in the internal management table for an indexed character, text string or subroutine with the specified index (<i>Dst</i>) by copying the table entries of another index (<i>Src</i>).	
Call	<code>copy_dst_src(Dst, Src, Mode)</code>	
Parameters	<i>Dst</i>	Index of the indexed character, text string or subroutine whose entries should be copied from <i>Src</i> . As an unsigned 32-bit value. Allowed value range: [0...1023] for indexed characters or subroutines, [1024+0...1024+41] for indexed text strings.
	<i>Src</i>	Index of the indexed character, text string or subroutine whose entries should be copied to <i>Dst</i> . As an unsigned 32-bit value. Allowed value range: [0...1023] for indexed characters or subroutines, [1024+0...1024+41] for indexed text strings.
	<i>Mode</i>	Determines which management tables are to be changed. As an unsigned 32-bit value. Bit #0 = 0: <i>Dst</i> is the index of an indexed character or the index of an indexed text string. Bit #0 = 1: <i>Dst</i> is the index of an indexed subroutine. Bit #1 = 0: <i>Src</i> is the index of an indexed character or the index of an indexed text string. Bit #1 = 1: <i>Src</i> is the index of an indexed subroutine. Bit #2...Bit #31: Not evaluated.
Comments	• If an index value (<i>Dst</i> and/or <i>Src</i>) is invalid, then copy_dst_src is ignored (get_last_error return code RTC5_PARAM_ERROR). • copy_dst_src creates an additional reference (index) to an indexed character, text string or subroutine (that can also be called with this new index). copy_dst_src only alters the corresponding entry in the internal management table and does not modify the list memory area's memory contents. This allows copying, renumbering or converting between indexed characters, text strings or subroutines without having to reload each time. • A real copy of an indexed character, text string or subroutine in the protected list memory area "List 3" can be created (after the copy_dst_src command) with save_disk/load_disk. Characters, text strings and/or subroutines with multiple references are thereby written several times to the list memory. Keep this in mind in order to prevent unintended buffer overflow of the protected list memory area "List 3". • See also Section "Managing Indexed Characters and Text Strings", Page 109.	
RTC4→RTC5	New command.	
Version info	–	
References	save_disk, load_disk	

Ctrl Command	disable_laser
Function	Disables the Signals for “Laser Active” Operation .
Call	<code>disable_laser()</code>
Comments	• disable_laser disables the Signals for “Laser Active” Operation at the output ports LASER1, LASER2 and LASERON (the signals are set to their respective “Off” level). Pin (2) of the 15-pin laser connector is then at a fixed level. However, standby signals activated with set_standby or set_standby_list continue to be outputted by the LASER1 and LASER2 output ports. If the standby signals are deactivated, also the pins (1) and (9) of the 15-pin laser connector and pins (19) and (22) of the EXTENSION 2 socket connector are at a fixed level after disable_laser. • The Signals for “Laser Active” Operation can also be disabled by set_laser_control and reenabled by set_laser_control or enable_laser. • After initialization of the RTC5 with load_program_file, the laser control is <i>deactivated</i> and absolutely requires set_laser_control for activation. • If disable_laser results in an RTC5_TIMEOUT error (for example, if no program file has been loaded), the LASER1, LASER2 and LASERON output ports can only be reactivated with set_laser_control after a load_program_file. • By get_startstop_info (Bit #9), the current status of the laser control signals (globally enabled, yes or no) can be queried. • By get_startstop_info (Bit #14), the status “laser enabled” (yes or no) can be queried.
RTC4→RTC5	Unchanged functionality.
Version info	Last change: DLL 546, OUT 546: get_startstop_info (Bit #14).
References	enable_laser , set_laser_control , get_startstop_info

Ctrl Command	enable_laser
Function	Enables the Signals for "Laser Active" Operation .
Call	<code>enable_laser()</code>
Comments	• After a Hardware reset (and after load_program_file), set_laser_control must be called to activate the LASER1, LASER2 and LASERON output ports and define the signal level (see Chapter 7.4 "Laser Control", Page 173). Otherwise, enable_laser has no effect. • After initialization of the RTC5 with load_program_file, the laser control signals are <i>deactivated</i>. For first-time activation, set_laser_control must be called (see above). After a subsequent disabling by disable_laser (or set_laser_control), the enable_laser (or set_laser_control) command can be used for reenabling. • Even if the laser control signals have been enabled with enable_laser or set_laser_control, they are not outputted without further commands (see Chapter 7.4 "Laser Control", Page 173). • By get_startstop_info (Bit #9) the current status of the laser control signals (globally enabled, yes or no) can be queried. • By get_startstop_info (Bit #14), the status "laser enabled" can be queried.
RTC4→RTC5	Basically unchanged functionality. In some circumstances, set_laser_control must be called before enable_laser (see above).
Version info	–
References	disable_laser, set_laser_control, get_startstop_info, set_laser_mode

Ctrl Command	execute_at_pointer
Function	Starts list execution ("List 1" or "List 2") at the specified address in the RTC5 list memory area.
Call	<code>execute_at_pointer(Pos)</code>
Parameters	Pos Absolute address of the first list command to be executed. As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{23}-1)]$.
Comments	execute_at_pointer essentially functions like execute_list_pos (see the comments there). However, execute_at_pointer requires a start address specified as an <i>absolute</i> memory address, whereas execute_list_pos requires specification of the list number and a <i>relative</i> memory address. - For $Pos \geq Mem1 + Mem2$ (see config_list), Pos is set to 0.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	execute_list_pos , get_out_pointer

Ctrl Command	execute_list
Function	Starts execution at the beginning of the specified list ("List 1" or "List 2").
Call	<code>execute_list(ListNo)</code>
Parameters	ListNo Number of the list to be executed. As an unsigned 32-bit value. Allowed values: [uneven: "List 1", even: "List 2"].
Comments	execute_list is synonymous with execute_list_pos with Pos = 0. execute_list_1 and execute_list_2 (with no parameters) can be used alternatively.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_status , execute_at_pointer , execute_list_pos

Ctrl Command	execute_list_1
Function	See execute_list .
Call	<code>execute_list_1</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	execute_list

Ctrl Command	execute_list_2
Function	See execute_list .
Call	<code>execute_list_2</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	execute_list

Ctrl Command	execute_list_pos
Function	Starts list execution ("List 1" or "List 2") at the specified position.
Call	<code>execute_list_pos(ListNo, Pos)</code>
Parameters	ListNo Number of the list to be executed. As an unsigned 32-bit value. Allowed values: [uneven: "List 1", even: "List 2"].
	Pos Address of the first list command to be executed (offset relative to the start of the respective list). As an unsigned 32-bit value. Allowed value range: [0...(2 ²⁰ –1)].
Comments	• execute_list_pos is not executed (get_error return value = RTC5_BUSY), if: – the BUSY list execution status is set – a list has been paused by set_wait (PAUSED list execution status set) • If the INTERNAL-BUSY list execution status is set, execute_list_pos is only executed with a delay (after INTERNAL-BUSY list execution status has been reset again). No checks are performed to determine if a list is currently being loaded. During list processing, the other list (or even the same list) can be simultaneously reloaded, see also Chapter 6.4 "List Handling", Page 94. • Programs in the protected area ("List 3") cannot be directly executed by execute_list_pos. They can only be called from a list ("List 1" or "List 2") as a subroutine. Alternatively, the corresponding area can be assigned by config_list to "List 1" or "List 2". • Uneven ListNo values cause "List 1" to be executed; otherwise "List 2" is executed. This allows automatically generated continuous list changing by an incremented count.

Ctrl Command	execute_list_pos
Comments (cont'd)	• If "List 2" has not been assigned memory ($\text{Mem2} = 0$, see config_list) then "List 1" is opened. • If Pos is specified as being larger than the memory area of the respective list ($\text{Pos} > \text{Mem1}$ or $\text{Pos} > \text{Mem2}$), then Pos is set to 0. • The BUSY list status of the selected list is set and the BUSY list status of the other corresponding list is reset (see read_status). The BUSY list execution status-List Execution Status (see get_status) is set. • Execution stops when a set_end_of_list is encountered. If the end of a list area is reached without encountering a set_end_of_list, then execution continues at the beginning of the same list area instead of with the next list. The output pointer remains in the active list area unless a set_end_of_list has been encountered and an auto_change_pos or start_loop has been previously called. For both lists to be treated as a single list, you must set the configuration appropriately: for example, <code>config_list(Mem1+Mem2, 0)</code>. • If a home jump (defined with home_position or home_position_xyz) has been executed by set_end_of_list, then execute_list_pos leads to a corresponding home return (the INTERNAL-BUSY list execution status is set while the home return is executed). • execute_list_pos triggers a flush of the buffered list input (see Chapter 6.4.1 "Loading Lists", Page 94), even if the start has been unsuccessful. • execute_list_pos also covers the specialized variants execute_list_1, execute_list_2, execute_list and execute_at_pointer.
RTC4→RTC5	New command.
Version info	–
References	execute_list , execute_at_pointer , set_start_list_pos

Normal List Command	fly_return
Function	Deactivates the previously set Processing-on-the-fly correction and subsequently executes a jump to the defined new output position.
Restriction	If the Option Processing-on-the-fly is not enabled, fly_return only executes the jump to the specified new output position.
Call	<code>fly_return(X, Y)</code>
Parameters	X Absolute x coordinate of the new output position. In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
Comments	• The jump to the new output position is executed as with jump_abs (see comments there). • If Processing-on-the-fly-correction has been activated within a subroutine called by an "AbsCall" and subsequently gets deactivated by fly_return, then the coordinate values specified with fly_return receive an offset (based on the current coordinates at the time of the call, see also Section ""AbsCalls"", Page 103). • See also Chapter 8.6 "Processing-on-the-fly", Page 227.
RTC4→RTC5	Unchanged functionality. In addition: increased value range, and "AbsCall", see above. In RTC4 Compatibility Mode , the RTC5 multiplies the values specified for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	set_fly_x, set_fly_y, set_fly_rot, set_fly_x_pos, set_fly_y_pos, set_fly_rot_pos

Normal List Command	fly_return_z
Function	Deactivates the previously set Processing-on-the-fly correction. Subsequently executes a jump to the defined position.
Restriction	If the Option Processing-on-the-fly is not enabled, fly_return_z only executes the jump to the defined new output position.
Call	<code>fly_return_z(X, Y, Z)</code>
Parameters	X Absolute x coordinate of the new output position. In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
	Z Like X (analogously). Allowed value range: [-32,768...+32,767].
Comments	Like fly_return, however, with additional Z output position.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode.
Version info	–
References	set_fly_z

Ctrl Command	<code>free_rtc5_dll</code>
Function	Frees up all resources allocated by the RTC5 DLL for a user program.
Call	<code>free_rtc5_dll</code>
Comments	• RTC5 DLL-allocated resources particularly include memory area in the RTC5 DLL allocated for the RTC5 board management (which is created by <code>init_rtc5_dll</code>). <code>free_rtc5_dll</code> deletes the RTC5 board management of the RTC5 DLL. Afterward, the user program has no access to boards (<code>get_last_error</code> return code <code>RTC5_ACCESS_DENIED</code>). • The calling of <code>free_rtc5_dll</code> is not absolutely necessary, because RTC5 DLL-assigned resources are automatically freed up when the user program terminates and the RTC5 DLL is thereby unloaded by Microsoft Windows. However, some user program development environments (in debug mode) issue “memory leaks detected” warnings even though the RTC5 DLL is unloaded. The calling of <code>free_rtc5_dll</code> eliminates this annoyance. • <code>free_rtc5_dll</code> is not available as a multi-board command.
RTC4→RTC5	New command.
Version info	–
References	<code>init_rtc5_dll</code> , <code>release_rtc</code>

Ctrl Command	get_auto_cal
Function	Returns the attached scan system's type of ASC hardware previously detected by auto_cal .
Call	ASCtype = get_auto_cal(HeadNo)
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. Allowed values: = 1: First scan head connector. = 2: Second scan head connector.
Result	ASC hardware type. As an unsigned 32-bit value.
Comments	• If the ASC hardware type has been previously detected by auto_cal, then get_auto_cal returns the same value as auto_cal (Command = 0), see comments there. • If the ASC hardware type has <i>not</i> been previously detected by auto_cal, then get_auto_cal returns the value 255 (initialized value).
RTC4→RTC5	New command.
Version info	–
References	auto_cal

Ctrl Command	get_char_pointer
Function	Returns the absolute start address of an indexed character.
Call	<code>CharPointer = get_char_pointer(Char)</code>
Parameters	Char Index of the indexed character. As an unsigned 32-bit value. Allowed value range: [0...1023].
Result	Absolute start address. As an unsigned 32-bit value.
Comments	• get_char_pointer reads from the internal management table the start address of the indexed character with the specified index. Whether the read address resides in a protected or the unprotected list memory area depends on whether the character has been loaded into the protected list memory area "List 3" or an unprotected subroutine has been only subsequently referenced. • If <code>Index > 1023</code> or if no character has been referenced with the specified index, then get_char_pointer returns the value "-1" (for example, $2^{32}-1$). • get_char_pointer is useful for checking if a character has already been defined or for calling an indexed character by an absolute memory address as if it were a non-indexed subroutine, for example, for conditional execution with list_call_cond. Be aware, though, that a subsequent save_disk/load_disk might alter the absolute memory address. And you should ensure that get_char_pointer does not return "-1"; otherwise, list_call_cond is ignored.
RTC4→RTC5	New command.
Version info	–
References	get_sub_pointer , get_text_table_pointer

Ctrl Command	get_config_list
Function	Passes the parameters of the current list memory configuration (Mem1 , Mem2) to the RTC5 board management of the RTC5 DLL and initializes it as if the config_list command has been called.
Call	<code>get_config_list()</code>
Result	–
Comments	• get_config_list is useful when a board changes “ownership” and the new RTC5 board management is not aware of the memory configuration (at the start of each user program, the board and board management each independently initialize Mem1 = 4000 and Mem2 = 4000; the board by load_program_file and board management when starting the corresponding user program). See also Chapter 6.7.1 “Board Acquisition by a User Program”, Page 117. • get_config_list does not return a value to the user program. The user program can, however, read the list-memory configuration data after <code>load_list(ListNo, 0)</code> or <code>set_start_list_pos(ListNo, 0)</code> by using get_list_space. • get_config_list is executed regardless of the BUSY list execution status.
RTC4→RTC5	New command.
Version info	–
References	config_list

Ctrl Command	get_counts
Function	Reads the current number of successful External Starts .
Call	<code>Counts = get_counts()</code>
Result	Number of successful External Starts . As an unsigned 32-bit value.
Comments	• The number is read from an internal counter, which is incremented each time a list is started by an external start signal. The counter can be reset to zero by set_control_mode.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_max_counts , set_control_mode , get_startstop_info

Ctrl Command	get_dll_version
Function	Returns the version number of the RTC5 DLL.
Call	<code>DLLVersion = get_dll_version()</code>
Result	Version number. As an unsigned 32-bit value.
Comments	- The RTC5 DLL version numbers are in the range 500-599. get_dll_version is available even without explicit access rights to a specific RTC5 Board. get_dll_version is not available as a multi-board command. - The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by get_dll_version.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_hex_version, get_rtc_version

Ctrl Command	get_encoder
Function	Returns the current counts of the two internal encoder counters.
Call	<code>get_encoder(&Encoder0, &Encoder1)</code>
Returned parameter values	Encoder0 Current count of encoder counter "Encoder0". As a pointer to a signed 32-bit value. Encoder1 Current count of encoder counter "Encoder1". As a pointer to a signed 32-bit value.
Comments	- For get_encoder usage, see Chapter 8.6 "Processing-on-the-fly", Page 227 and Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274. - If the workpiece motion is registered with an incremental encoder: - Encoder counter "Encoder0" is triggered by the signals at encoder input port ENCODER X - Encoder counter "Encoder1" is triggered by the signals at encoder input port ENCODER Y - If an encoder simulation has been started by simulate_encoder(3), both encoder counters are triggered by an internal periodic 1 MHz clock signal.
RTC4→RTC5	Unchanged functionality. In addition: increased value range.
Version info	–
References	store_encoder, read_encoder, set_fly_x, set_fly_y, set_fly_rot, wait_for_encoder

Ctrl Command	get_error																															
Function	Returns the cumulative error code. It corresponds to a list of error types occurring since the last reset or error reset.																															
Call	AccError = get_error()																															
Result	Error code. As an unsigned 32-bit value. If multiple errors occurred, then multiple bits are set. For the specific errors the error constants should be predefined as mentioned below. <table><tr><th>Bit</th><th>Error type</th><th>Error constant</th><th></th></tr><tr><td>–</td><td>No error.</td><td>RTC5_NO_ERROR</td><td>= 0</td></tr><tr><td>Bit #0 (LSB)</td><td>= 1: No board found (this error can only occur by init_rtc5_dll).</td><td>RTC5_NO_CARD</td><td>= 1</td></tr><tr><td>Bit #1</td><td>= 1: Access denied (this error can occur by init_rtc5_dll, select_rtc, acquire_rtc or any multi-board command).</td><td>RTC5_ACCESS_DENIED</td><td>= 2</td></tr><tr><td>Bit #2</td><td>= 1: Command not forwarded (this error implies an internal, PCI or driver error, for example, caused by a hardware defect or an incorrect connection).</td><td>RTC5_SEND_ERROR</td><td>= 4</td></tr><tr><td>Bit #3</td><td>= 1: No response from board (it is likely that no program has been loaded onto the RTC5; this error can especially occur in connection with control commands that expect a response, for example, get_head_para).</td><td>RTC5_TIMEOUT</td><td>= 8</td></tr><tr><td>Bit #4</td><td>= 1: Invalid parameter (this error can occur through all commands for which invalid parameters are not automatically corrected to valid values, for example, parameters with limited choices such as get_head_para; if this error occurs for a list command, it is replaced with list_nop; if this error occurs for a control command, it is not executed).</td><td>RTC5_PARAM_ERROR</td><td>= 16</td></tr></table>				Bit	Error type	Error constant		–	No error.	RTC5_NO_ERROR	= 0	Bit #0 (LSB)	= 1: No board found (this error can only occur by init_rtc5_dll).	RTC5_NO_CARD	= 1	Bit #1	= 1: Access denied (this error can occur by init_rtc5_dll , select_rtc , acquire_rtc or any multi-board command).	RTC5_ACCESS_DENIED	= 2	Bit #2	= 1: Command not forwarded (this error implies an internal, PCI or driver error, for example, caused by a hardware defect or an incorrect connection).	RTC5_SEND_ERROR	= 4	Bit #3	= 1: No response from board (it is likely that no program has been loaded onto the RTC5; this error can especially occur in connection with control commands that expect a response, for example, get_head_para).	RTC5_TIMEOUT	= 8	Bit #4	= 1: Invalid parameter (this error can occur through all commands for which invalid parameters are not automatically corrected to valid values, for example, parameters with limited choices such as get_head_para ; if this error occurs for a list command, it is replaced with list_nop ; if this error occurs for a control command, it is not executed).	RTC5_PARAM_ERROR	= 16
Bit	Error type	Error constant																														
–	No error.	RTC5_NO_ERROR	= 0																													
Bit #0 (LSB)	= 1: No board found (this error can only occur by init_rtc5_dll).	RTC5_NO_CARD	= 1																													
Bit #1	= 1: Access denied (this error can occur by init_rtc5_dll , select_rtc , acquire_rtc or any multi-board command).	RTC5_ACCESS_DENIED	= 2																													
Bit #2	= 1: Command not forwarded (this error implies an internal, PCI or driver error, for example, caused by a hardware defect or an incorrect connection).	RTC5_SEND_ERROR	= 4																													
Bit #3	= 1: No response from board (it is likely that no program has been loaded onto the RTC5; this error can especially occur in connection with control commands that expect a response, for example, get_head_para).	RTC5_TIMEOUT	= 8																													
Bit #4	= 1: Invalid parameter (this error can occur through all commands for which invalid parameters are not automatically corrected to valid values, for example, parameters with limited choices such as get_head_para ; if this error occurs for a list command, it is replaced with list_nop ; if this error occurs for a control command, it is not executed).	RTC5_PARAM_ERROR	= 16																													

Ctrl Command	get_error			
	Bit #5	= 1:	List processing is (not) active (for example, for execute_list , if a list is currently being processed for example, for stop_execution , if no list is currently being processed for example, for restart_list , if pause_list has not been previously called).	RTC5_BUSY = 32
	Bit #6	= 1:	List command rejected, illegal input pointer (for example, for any list command directly after load_char + list_return : the list command is then not loaded).	RTC5_REJECTED = 64
	Bit #7	= 1:	List command has been converted to a list_nop (for example, set_end_of_list in a protected subroutine).	RTC5_IGNORED = 128
	Bit #8	= 1:	Version error: RTC5 DLL version, RTC5RBF.rbf version and RTC5OUT.out version are not compatible with each other. See also load_program_file .	RTC5_VERSION_MISMATCH = 256
	Bit #9	= 1:	Verify error: The download verification (see page 120) has detected an incorrect download.	RTC5_VERIFY_ERROR = 512

Ctrl Command	get_error			
	Bit #10	= 1:	DSP version error: DSP version too old (this error only occurs with older RTC5 Boards, see get_rtc_version Bit #16...Bit #23 – and only through a few commands such as mark_ellipse_abs ; the corresponding command description's "Version info" section contains related comments; Commands that generate the error are neither executed nor replaced by list_nop).	RTC5_TYPE_REJECTED = 1024
	Bit #11	= 1:	A RTC5 DLL-internal Windows memory request failed.	RTC5_OUT_OF_MEMORY = 2048
	Bit #12	= 1:	EEPROM read or write error (can occur during initialization or auto_cal).	RTC5_EEPROM_ERROR = 4096
	Bit #13	Reserved.		–
	Bit #14	Reserved.		–
	Bit #15	Reserved.		–
	Bit #16	= 1:	Error reading PCI configuration register (can only occur during init_rtc5_dll).	RTC5_CONFIG_ERROR = 65536
	Bit #17	Reserved.		–
	...			
	Bit #31			
Comments	- For error handling, see Chapter 6.8 "Error Handling", Page 119. get_error and n_get_error are even available without explicit access rights to a specific RTC5 Board. - The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by get_error.			

Ctrl Command	get_error
Example (C/C++)	Creates an array for specifying which board has no existing access rights and resets the cumulative error code. <pre> UINT NoAccess[MaxCount+1]; // MaxCount is a user-defined constant UINT Error = init_rtc5_dll();// Searches for all installed RTC5 Boards if (Error & RTC5_ACCESS_DENIED) { // at least one board is inaccessible UINT Count = rtc5_count_cards(); // number of boards found for (UINT Num = 1; Num <= Count; Num++) { NoAccess[Num] = n_get_last_error(Num) & RTC5_ACCESS_DENIED; n_reset_error(Num, RTC5_ACCESS_DENIED); } } </pre>
RTC4→RTC5	New command.
Version info	–
References	get_last_error, reset_error, set_verify

Ctrl Command	get_fly_2d_offset
Function	Returns the current reference values (offset values) for 2D encoder compensation.
Call	<code>get_fly_2d_offset(&OffsetX, &OffsetY)</code>
Returned parameter values	OffsetX x reference value. As a pointer to a signed 32-bit value. OffsetY y reference value. As a pointer to a signed 32-bit value.
Comments	For 2D encoder compensation, see Section "2D Encoder Compensation for xy Positioning Stages", Page 234.
RTC4→RTC5	New command.
Version info	–
References	init_fly_2d , set_fly_2d

Ctrl Command	get_free_variable
Function	Returns the current value of a free variable.
Call	<code>VariableValue = get_free_variable(No)</code>
Parameters	No Number of the free variable to be queried. As an unsigned 32-bit value. Allowed value range: [0...7]. Only the 3 least significant bits are evaluated.
Result	The value currently stored in the free variable No. As an unsigned 32-bit value.
Comments	See also Chapter 6.9.1 "Free Variables", Page 123.
RTC4→RTC5	New command.
Version info	Last change DLL 539, OUT 539: value range of No increased to [0...7].
References	set_free_variable , set_free_variable_list

Ctrl Command	get_galvo_controls	
Function	Returns the corresponding control values for given input values.	
Restriction	get_galvo_controls can only be executed, if no list is currently being processed (get_last_error return code RTC5_BUSY).	
Call	get_galvo_controls(InPtr, OutPtr)	
Parameters	InPtr	Pointer (data type ULONG_PTR in C and C++, an unsigned 32-bit or unsigned 64-bit value) to an array of five unsigned 32-bit values, where the desired settings are specified: X, Y, Z, Defocus, Zoom. Out-of-range values are clipped to the boundary values.
	OutPtr	Pointer (data type ULONG_PTR in C and C++, an unsigned 32-bit or unsigned 64-bit value) to an array of 4 unsigned 32-bit values, where the corresponding control values are to be stored: XA, YA, XB, YB. Range of values in each case is [-524,288...+524,287].
Returned parameter values	XA, YA, XB, YB denote the control values for the x axis/y axis of scan head A/B.	
Comments	get_galvo_controls carries-out a virtual jump to the specified coordinates. Though the galvanometer scanners are <i>not</i> moved. - All other settings which are not specified within get_galvo_controls (for example, matrix, offset, angle, etc.; also stretch table) are considered as they are set at the moment. Users are solely responsible to apply these by <code>at_once > 0</code> before get_galvo_controls is called. The settings are not used, if they just only have been saved by <code>at_once = 0</code>. - Control values are calculated only for channels where a correction table has been previously assigned by select_cor_table, see the following examples. <pre>select_cor_table(0,0): XA = YA = XB = YB = 0</pre> 2D correction files: inputs Z, Defocus, Zoom are ignored <pre>select_cor_table(1,0): XA, YA calculated, XB, YB = 0 select_cor_table(0,1): XA, YA = 0, XB, YB calculated select_cor_table(1,1): XA, YA, XB, YB calculated</pre> (Standard-)3D correction file: input Zoom is ignored <pre>select_cor_table(1,0): XA, YA calculated, XB = YB = Zout calculated select_cor_table(0,1): XA= YA = Zout calculated, XB, YB calculated select_cor_table(1,1): same as select_cor_table(0,0)</pre> 3D zoom correction file (only for intelliWELD II with zoom axis): <pre>select_cor_table(1,0): XA, YA calculated, XB = Zout, YB = ZoomOut calculated select_cor_table(0,1): XA= Zout, YA = ZoomOut calculated, XB, YB calculated</pre> - The return values are 0, if a get_last_error return code RTC5_BUSY has been generated.	



Ctrl Command	get_galvo_controls
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for x and y by 16. The allowed value range decreases accordingly. Even in RTC4 Compatibility Mode, all returned values are in the RTC5 20-bit range.
Version info	Available as of DLL 539, OUT 539.
References	select_cor_table

Ctrl Command	get_head_para
Function	Returns the value of the requested parameter in the correction table assigned to the specified scan head.
Call	HeadPara = get_head_para(HeadNo, ParaNo)
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. Allowed values: = 1: First scan head connector. = 2: Second scan head connector.
	ParaNo Number of the parameter. As an unsigned 32-bit value. Allowed values: 0...15. Mapping: see Section "ct5 Correction File Header", Page 167 .
Result	Parameter value, see Section "ct5 Correction File Header", Page 167 . As a 64-bit IEEE floating point value.
Comments	• The parameter values can be read out by get_table_para from a currently loaded correction table and by get_head_para from an assigned correction table and thus directly incorporated into a user program, see Section "ct5 Correction File Header", Page 167. • If the parameters HeadNo and ParaNo are out of range, then the return value is 0 (get_last_error return code RTC5_PARAM_ERROR). The return value is also 0 (no get_last_error return code) if no correction table has been assigned to the specified head (for example, for HeadNo = 2 if the Option "Second Scan Head Control" has not been enabled) and no 3D correction table has been assigned to the other head. • If a 3D correction table has been assigned to a head, then this 3D correction table's parameter is returned regardless of HeadNo (two 3D correction tables cannot be simultaneously assigned). HeadNo must nevertheless be 1 or 2 (see preceding comment).
RTC4→RTC5	New command.
Version info	–
References	get_table_para

Ctrl Command	get_head_status																																															
Function	Returns the XY2-100 status word from the specified scan head connector.																																															
Call	get_head_status(Head)																																															
Parameters	Head	= 1:	Returns the status of the first scan head connector (Byte #1 = Byte #0).																																													
		= 2:	Returns the status of the second scan head connector (Byte #1 = Byte #0).																																													
		Else:	Returns the status of the first scan head connector (Byte #0) and of the second scan head connector (Byte #1).																																													
Result	XY2-100 status word. As an unsigned 32-bit value. <table><tr><td>Byte #0</td><td>Bit #0</td><td>1.</td></tr><tr><td>(LSB)</td><td>(LSB)</td><td></td></tr><tr><td></td><td>Bit #1</td><td>0.</td></tr><tr><td></td><td>Bit #2</td><td>1 (reserved).</td></tr><tr><td></td><td>Bit #3</td><td>Position Acknowledge of x axis, 1 = OK.</td></tr><tr><td></td><td>Bit #4</td><td>Position Acknowledge of y axis, 1 = OK.</td></tr><tr><td></td><td>Bit #5</td><td>1 (reserved).</td></tr><tr><td></td><td>Bit #6</td><td>Temperature Status, 1 = OK. See comment, page 339.</td></tr><tr><td></td><td>Bit #7</td><td>Power Status, 1 = OK. See comment, page 339.</td></tr><tr><td>Byte #1</td><td>Bit #8</td><td>Bit assignments as with Byte #0.</td></tr><tr><td></td><td>...</td><td></td></tr><tr><td></td><td>Bit #15</td><td></td></tr><tr><td>Byte #2</td><td>Bit #16</td><td>0.</td></tr><tr><td>...</td><td>...</td><td></td></tr><tr><td>Byte #3</td><td>Bit #31</td><td></td></tr></table>			Byte #0	Bit #0	1.	(LSB)	(LSB)			Bit #1	0.		Bit #2	1 (reserved).		Bit #3	Position Acknowledge of x axis, 1 = OK.		Bit #4	Position Acknowledge of y axis, 1 = OK.		Bit #5	1 (reserved).		Bit #6	Temperature Status, 1 = OK. See comment, page 339.		Bit #7	Power Status, 1 = OK. See comment, page 339.	Byte #1	Bit #8	Bit assignments as with Byte #0.		...			Bit #15		Byte #2	Bit #16	0.	...	...		Byte #3	Bit #31	
Byte #0	Bit #0	1.																																														
(LSB)	(LSB)																																															
	Bit #1	0.																																														
	Bit #2	1 (reserved).																																														
	Bit #3	Position Acknowledge of x axis, 1 = OK.																																														
	Bit #4	Position Acknowledge of y axis, 1 = OK.																																														
	Bit #5	1 (reserved).																																														
	Bit #6	Temperature Status, 1 = OK. See comment, page 339.																																														
	Bit #7	Power Status, 1 = OK. See comment, page 339.																																														
Byte #1	Bit #8	Bit assignments as with Byte #0.																																														
	...																																															
	Bit #15																																															
Byte #2	Bit #16	0.																																														
...	...																																															
Byte #3	Bit #31																																															
Comments	• get_head_status is available even with the SL2-100 protocol, if the data type is not set to the 20-bit status word itself, see Section "Status Information Returned from the Scan System", Page 172. • The status bits are returned by get_head_status in Bit #3...Bit #7 and/or Bit #11...Bit #15. Independently of the scan system's current state, Bit #0 and Bit #8 are returned by get_head_status as 1, while Bit #1 and Bit #9 are returned as 0. • If no scan system is currently connected or is not switched on, then the status of the most recently connected system is returned. get_startstop_info (Bit #17 and/or Bit #25) can be used for distinguishing. If a scan system is still not connected after a reset (power-on or load_program_file), then get_head_status returns the value 0. • With an XY2-100 Converter (Accessory) the value 0 is returned, if no scan system is connected or not switched on. • The PowerOK signal is an electronically generated signal. It means: The scan system servo control is ready for input data ("PowerOK", actually corresponds to a "ServoOK"). Therefore, it cannot be used to check whether a connected scan head is switched on at all. get_startstop_info (Bit #17 and/or Bit #25) can be used for distinguishing.																																															

Ctrl Command	get_head_status
Comments (cont'd)	• The Power Status and Temperature Status signals deliver combined status information of both axes. • In any case also obey the status signal information described in the manual of your scan system. • With iDRIVE scan systems (see Glossary entry on page 23) – Without the SL2-100 interface, get_head_status only returns meaningful return values, if the XY2-100 status word has been selected for return transmission. – the Position Acknowledge signals of the x- and y axis are logically AND-connected and only returned as a common signal (for example, at Bit #3, Bit #4). – after a reset or power-up of the scan system, it can take around 5 seconds for data to be returned from the scan system. • Status signals can also be queried by get_value, get_values, set_trigger and set_trigger4. • See also Chapter 8.5 "Controlling 2D Scan Systems and 3D Scan Systems", Page 221 for information about using two scan heads.
RTC4→RTC5	Basically unchanged functionality. However: • The RTC5 allows the simultaneous reading of both scan head connectors' status words, and with iDRIVE scan systems with SL2-100 interface even independently of the signal to be returned (set by control_command). • With iDRIVE scan systems, Bit #0 and Bit #1 do not return information about the operational readiness of the scan system (see get_value).
Version info	–
References	get_value , get_values , set_trigger , set_trigger4 , get_waveform

Ctrl Command	get_hex_version
Function	Returns the version number of the DSP program file RTC5OUT.out , which is currently loaded on the RTC5.
Call	HexVersion = get_hex_version()
Result	Version number. As an unsigned 32-bit value.
Comments	- The version numbers of program files are in the range 500...599. get_hex_version returns the following values: - if the Option "3D" is <i>not</i> enabled values in the range 2500...2599 (version number + 2000) - if the Option "3D" is enabled values in the range 3500...3599 (version number + 3000) - The file name extension for RTC5 program files is no longer *.hex (as with the RTC4), but instead *.out (see also load_program_file). - The software version number can also be returned after an RTC5_VERSION_MISMATCH or RTC5_ACCESS_DENIED error. The return value is 0 if no program has yet been loaded. - The board-specific error variables LastError and AccError, see Section "Error Handling", Page 119, are neither generated nor altered by get_hex_version.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_dll_version , get_rtc_version

Ctrl Command	get_hi_data
Function	Returns the Home-In positions, last determined (by auto_cal) of the scan system attached to the first scan head connector.
Call	get_hi_data(&X1, &X2, &Y1, &Y2)
Returned parameter values	X1 x1 coordinate of the currently stored (most recently measured) Home-In position. In bits. As a pointer to a signed 32-bit value.
	X2 Like X1 (analogously).
	Y1 Like X1 (analogously).
	Y2 Like X1 (analogously).
Comments	get_hi_data is synonymous with get_hi_pos with HeadNo = 1 (see comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. The returned values are in the RTC5 20-bit range.
Version info	–
References	get_hi_pos , write_hi_pos

Ctrl Command	get_hi_pos	
Function	Returns the Home-In positions, last determined (by auto_cal) of the scan system attached to the specified scan head connector.	
Call	<code>get_hi_pos(HeadNo, &X1, &X2, &Y1, &Y2)</code>	
Parameters	HeadNo	Number of the scan head connector as an unsigned 32-bit value, allowed values. = 1: First scan head connector. = 2: Second scan head connector.
Returned parameter values	X1	x1 coordinate of the currently stored (most recently measured) Home-In positions. In bits. As a pointer to a signed 32-bit value.
	X2	Like X1 (analogously).
	Y1	Like X1 (analogously).
	Y2	Like X1 (analogously).
Comments	• For get_hi_pos usage, see Section "Customer-Specific Calibration", Page 254. • It is up to users to ensure that the scan system currently attached to the specified scan head connector is the same scan system which has been used to determine the returned Home-In positions. For determination of Home-In position values, this scan system should be equipped with an internal sensor system for automatic self-calibration (Home-In sensors). • The returned values are 0 if no scan system equipped with automatic self-calibration (Home-In sensors) is attached to the specified scan head connector or if an error has occurred during determination of the Home-In values for such a system. • Directly after initialization (init_rtc5_dll), particularly prior to a first call of <code>auto_cal(Command = 0, 1 or 3)</code>, the returned values are the Home-In reference values stored in the EEPROM. If such reference values have not been successfully determined at least once by <code>auto_cal(Command = 0)</code>, get_hi_pos returns 0. • get_hi_pos is available even without explicit access rights to a specific RTC5 Board. • If parameter values are invalid, then all returned coordinates are 0 (get_last_error return code RTC5_PARAM_ERROR).	
RTC4→RTC5	New command.	
Version info	–	
References	get_hi_data , auto_cal , set_hi , write_hi_pos	

Ctrl Command	get_input_pointer
Function	Returns the present <i>absolute</i> position of the input pointer.
Call	<code>InputPointer = get_input_pointer()</code>
Result	position of the input pointer $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	- The position of the input pointer corresponds to the position in RTC5 list memory area (also in the protected "List 3" area), where the next list command is stored. The number of still-available storage positions there can be queried by get_list_space. get_input_pointer returns the absolute list memory address (offset relative to the start of "List 1"). The relative position referenced to the start of the respective list area can be queried by get_list_pointer. - Before loading a non-indexed subroutine or character set, you should use get_input_pointer to obtain the start address if subsequent referencing is to be performed by set_sub_pointer or set_char_pointer. - The absolute position of the output pointer can be queried by get_status or get_out_pointer. - The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by get_input_pointer.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_list_pointer , set_input_pointer , get_list_space , get_status , get_out_pointer

Ctrl Command	get_io_status
Function	Returns the current state of the 16-bit digital output port on the EXTENSION 1 socket connector.
Call	<code>IOStatus = get_io_status()</code>
Result	16-bit value (DIGITAL OUT0...DIGITAL OUT15). As an unsigned 32-bit value.
Comments	get_io_status is intended for use in combination with set_io_cond_list and clear_io_cond_list (see also Section "Example Code (Pascal)", Page 272). - See also Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	write_io_port , write_io_port_mask , set_io_cond_list , clear_io_cond_list

Ctrl Command	get_jump_table
Function	Retrieves the board's currently stored Jump Delay table and copies the 1024 corresponding unsigned 16-bit values to the supplied PC address.
Call	<code>ErrorCode = get_jump_table(Addr)</code>
Parameters	Addr PC address for the 2048 byte memory area.
Result	Error code as unsigned 32-bit value. 0 No error. 11 RTC5 board driver error.
Notes	- Do not call get_jump_table during processing of a list. - The data format is "1024 16-bit values" representing the delay values for a piecewise linear interpolation at the sampling points (=jump lengths) $N \times 1024$ with $0 \leq N < 1024$. The values can thus also be directly generated or modified by users.
RTC4→RTC5	New command.
Version info	–
Reference	set_jump_table , load_jump_table_offset

Ctrl Command	get_lap_time
Function	Returns the <i>current</i> RTC5 Timer value (without resetting it to zero).
Call	<code>TimerValue = get_lap_time()</code>
Result	RTC5 Timer value since the last call of save_and_restart_timer . In seconds. As a 64-bit IEEE floating point value.
Comments	get_lap_time serves to query the elapsed time of a time consuming marking during processing.
RTC4→RTC5	New command.
Version info	Available as of DLL 541, OUT 541.
Version info	–
References	get_time , save_and_restart_timer

Ctrl Command	get_laser_pin_in
Function	Returns the current status of the two digital inputs at the laser connector.
Call	<code>LaserPinIn = get_laser_pin_in()</code>
Result	As an unsigned 32-bit value. Bit #0 DIGITAL IN1. (LSB) Bit #1 DIGITAL IN2. Bit #2 Reserved. Bit 31 Reserved.
Comments	• –
RTC4→RTC5	New command.
Version info	–
References	set_laser_pin_out

Ctrl Command	get_last_error
Function	Returns an error code listing any errors which occurred during execution of the most recent command.
Call	<code>LastError = get_last_error()</code>
Result	Error code. As an unsigned 32-bit value. If multiple errors occurred simultaneously, then multiple bits are set. The meanings of bit numbers, error types and error constants is identical to those for get_error .
Comments	• For error handling see Chapter 6.8 "Error Handling", Page 119. • get_last_error and n_get_last_error are even available without explicit access rights to a specific RTC5 Board. • The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by get_last_error.
RTC4→RTC5	New command.
Version info	–
References	get_error , reset_error , set_verify

Ctrl Command	get_list_pointer
Function	Provides the input pointer's current <i>relative</i> position and the list number.
Call	<code>get_list_pointer(&ListNo, &Pos)</code>
Returned parameter values	ListNo Number of the list in which the input pointer is currently located. As a pointer to an unsigned 32-bit value. [1...3].
	Pos Current position of the input pointer (offset relative to the start of the respective list). As a pointer to an unsigned 32-bit value.
Comments	• The input pointer's absolute list memory area address (offset relative to the start of "List 1") can be queried by get_input_pointer (see also comments there). • The number of list positions until the end of the respective list (from the input pointer) can be queried by get_list_space. • The board-specific error variables <code>LastError</code> and <code>AccError</code> (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by get_list_pointer. • If the input pointer is invalid when get_list_pointer is called (for example, after a list_return), the return values are <code>ListNo = 0</code> and <code>Pos = 0xFFFFFFFF</code>.
RTC4→RTC5	New command.
Version info	–
References	get_input_pointer , get_list_space

Ctrl Command	get_list_serial
Function	Returns the number of the serial-number-set most recently selected by select_serial_set_list (or of serial-number-set 0 after load_program_file) and the current serial number of this serial-number-set.
Call	<code>LastMarkedSerialNo = get_list_serial(&Set)</code>
Result	Serial number. As a 64-bit IEEE floating point value.
Returned parameter value	Set Number of the selected serial-number-set. As a pointer to an unsigned 32-bit value.
Comments	- The serial number queried by get_list_serial is typically the one most recently marked by mark_serial or mark_serial_abs. - For get_list_serial usage, see Chapter 7.5.2 “Marking Serial Numbers”, Page 196.
RTC4→RTC5	New command.
Version info	–
References	select_serial_set_list , get_serial

Ctrl Command	get_list_space
Function	Returns the amount of free list memory, hence the number of list commands that can still be loaded from the input pointer’s current position to the last position in the respective list.
Call	<code>ListSpace = get_list_space()</code>
Result	Number of free list positions. As an unsigned 32-bit value.
Comments	If an indexed subroutine or indexed character set is currently being loaded into the protected list memory area “List 3”, then get_list_space returns the amount of still-available protected memory (otherwise the input pointer is not located in the protected area).
RTC4→RTC5	get_list_space has been made available on the RTC4 to support the RTC4 Circular Queue Mode and returns the distance between the input pointer and output pointer. The RTC5 does not support the RTC4 Circular Queue Mode . The input pointer position can be queried by get_input_pointer or get_list_pointer and the output pointer position can be queried by get_status or get_out_pointer .
Version info	–
References	get_input_pointer , get_status , get_out_pointer , get_list_pointer

Ctrl Command	get_marking_info
Function	Returns information about any boundary exceedances during Processing-on-the-fly correction as well as improper encoder signals. The function also returns the error bits of laser-signal auto-suppression.
Call	MarkingInfo = get_marking_info()
Result	Error code. As an unsigned 32-bit value. Bit #0 = 1: Processing-on-the-fly underflow in x direction ($X < -524,288$). (LSB) Bit #1 = 1: Processing-on-the-fly overflow in x direction ($X > +524,287$). Bit #2 = 1: Processing-on-the-fly underflow in y direction ($Y < -524,288$). Bit #3 = 1: Processing-on-the-fly overflow in y direction ($Y > +524,287$). Bit #4 = 1: Processing-on-the-fly underflow in x direction ($X < X_{min}$). Bit #5 = 1: Processing-on-the-fly overflow in x direction ($X > X_{max}$). Bit #6 = 1: Processing-on-the-fly underflow in y direction ($Y < Y_{min}$). Bit #7 = 1: Processing-on-the-fly overflow in y direction ($Y > Y_{max}$). Bit #8 = 1: TriggerError: an enabled external trigger or simulated trigger occurred during execution of a list. Bit #9 = 1: An error has occurred during activation of Processing-on-the-fly correction by activate_fly_2d, activate_fly_2d_encoder, activate_fly_xy or activate_fly_xy_encoder ("ActivateFlyError"). See also Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237, Chapter 8.6.8 "Encoder Resets", Page 239 and Chapter 8.6.9 "Monitoring Processing-on-the-fly Corrections", Page 240. Bit #10 PosAck error bit of scan system A, x axis. Bit #11 TempOK error bit of scan system A, x axis. Bit #12 PowerOK error bit of scan system A, x axis. Bit #13 PosAck error bit of scan system A, y axis. Bit #14 TempOK error bit of scan system A, y axis. Bit #15 PowerOK error bit of scan system A, y axis. Bit #16 = 1: The distance between the edges in signal 1 of encoder input port ENCODER X is too short. Bit #17 = 1: The distance between the edges in signal 1 of encoder input port ENCODER Y is too short. Bit #18 = 1: The distance between the edges in signal 2 of encoder input port ENCODER X is too short. Bit #19 = 1: The distance between the edges in signal 2 of encoder input port ENCODER Y is too short.

Ctrl Command	get_marking_info
Result (cont'd)	Bit #20 = 1: Improper signal sequence at encoder input port ENCODER X. Bit #21 = 1: Improper signal sequence at encoder input port ENCODER Y. Bit #22 = 1: Processing-on-the-fly underflow in z direction ($Z < -32,768$). Bit #23 = 1: Processing-on-the-fly overflow in z direction ($Z > +32,767$). Bit #24 = 1: Processing-on-the-fly underflow in z direction ($Z < Z_{min}$). Bit #25 = 1: Processing-on-the-fly overflow in z direction ($Z > Z_{max}$). Bit #26 PosAck error bit of scan system B, x axis. Bit #27 TempOK error bit of scan system B, x axis. Bit #28 PowerOK error bit of scan system B, x axis. Bit #29 PosAck error bit of scan system B, y axis. Bit #30 TempOK error bit of scan system B, y axis. Bit #31 PowerOK error bit of scan system B, y axis.
Comments	- For usage of get_marking_info and of the error bits Bit #0...Bit #7, see Chapter 8.6.9 "Monitoring Processing-on-the-fly Corrections", Page 240. - The limits for the customer-defined monitoring range are determined for: - Xmin, Xmax, Ymin, Ymax (Bit #4...Bit #7) by set_fly_limits - Zmin, Zmax (Bit #24...Bit #25) by set_fly_limits_z - The error bits Bit #4...Bit #7 and Bit #24...Bit #25: - Are <i>not</i> reset by get_marking_info - Get implicitly reset by the conditional commands - Can also be explicitly reset by clear_fly_overflow and clear_fly_overflow_ctrl - Encoder-signal spacing could be too short if interfering signals are present, a rapid directional change occurs or the frequency is essentially too high. - An improper encoder signal sequence occurs if both signals 1 and 2 change simultaneously, thus hindering determination of the counting direction. - The error bits Bit #10...Bit #15 and Bit #26...Bit #31 are only set in case of an error if automatic suppression of laser control signals has been activated, see Section "Automatic Suppression of Laser Control Signals", Page 176. Any time an error occurs, all error bits corresponding to an error-indicating status signal are (cumulatively) set. If applicable, even error bits are set, which have not been selected to be used for automatic suppression of laser control signals by set_laser_control. All error bits are reset by get_marking_info.

Ctrl Command	get_marking_info
Comments (cont'd)	• All error bits are reset during initialization (by load_program_file). • All error bits (except Bit #4...Bit #7 and Bit #24...Bit #25): – Are reset when reading out by get_marking_info – However, can be reset at any time as long as the error condition still persists
RTC4→RTC5	Unchanged functionality for info bits that are also used on the RTC4.
Version info	–
References	set_fly_x, set_fly_y, set_fly_z, set_fly_rot, set_fly_x_pos, set_fly_y_pos, set_fly_rot_pos, set_fly_limits, set_fly_limits_z, clear_fly_overflow, clear_fly_overflow_ctrl, if_not_activated

Ctrl Command	get_master_slave
Function	Returns the master/slave status of the addressed RTC5 board.
Call	<code>MasterSlaveStatus = get_master_slave()</code>
Result	Master/slave status. As an unsigned 32-bit value. Information whether a further RTC5 board is connected to the Master or Slave connector of the addressed RTC5 board: Bit #0 = 1: A RTC5 board is connected to the Slave connector. (LSB) Bit #1 = 1: A RTC5 board is connected to the Master connector. Bit #2 = 0. ... Bit #31 = 0. Information, whether the addressed board is operated as a master, slave or single board: = 0 Single board. = 1 Slave without any further downstream slave board. = 2 Master. = 3 Slave together with a downstream slave board.
Comments	• See Chapter 6.6.3 "Master/Slave Operation", Page 113. • get_master_slave only returns the status, if the addressed RTC5 board has been previously initialized by load_program_file. Otherwise, 0 is returned.
RTC4→RTC5	New command.
Version info	Available as of DLL 600, OUT 600, RBF 600.
References	sync_slaves

Ctrl Command	get_mcbasp
Function	Returns the most recent input value that has been fully transferred by the McBSP interface to the memory location for Processing-on-the-fly applications.
Call	<code>mcbasp_value = get_mcbasp()</code>
Result	Input value. As a signed 32-bit value.
Comments	get_mcbasp is equivalent to read_mcbasp(0) (see notes there).
RTC4→RTC5	New command.
Version info	–
References	read_mcbasp

Undelayed Short List Command	get_mcbasp_list
Function	No function.
Call	<code>get_mcbasp_list()</code>
Comments	get_mcbasp_list has no effect on the user program.
RTC4→RTC5	New command.
Version info	–
References	get_mcbasp

Ctrl Command	get_out_pointer
Function	Returns the current (or most recent) position of the output pointer as offset relative to the start of the respective list and the list number.
Call	<code>get_out_pointer(&ListNo, &Pos)</code>
Returned parameter values	ListNo Number of the list ("List 1" or "List 2") in which the output pointer is currently located (or in which it most recently resided). As a pointer to an unsigned 32-bit value.
	Pos Current (or most recent) position of the output pointer (relative memory address). As a pointer to an unsigned 32-bit value.
Comments	get_out_pointer calls get_status (see the comments there, particularly with respect to "List 3") and uses the command's returned absolute output pointer position to determine the list number and the relative position within the list. The relative position and list number returned by get_out_pointer simplify comparing the output pointer position to the current input pointer position (particularly with respect to list consistency) during an alternative list change.
RTC4→RTC5	New command.
Version info	–
References	get_status , get_input_pointer , get_list_pointer

Ctrl Command	get_overnun
Function	Returns the number of overruns of the 10 μ s clock cycle since the last call and resets the overrun counter.
Call	<code>NumberOfOverruns = get_overnun()</code>
Result	Number of overruns of the 10 μ s clock cycle since the last call of get_overnun . As an unsigned 32-bit value.
Comments	See Section "Clock Overruns", Page 171.
RTC4→RTC5	New command.
Version info	–
References	–

Ctrl Command	get_rtc_mode
Function	Returns the currently set operation mode of the RTC5 DLL.
Call	<code>DLLMode = get_rtc_mode()</code>
Result	DLL operation mode as a 32-bit value. = 4: RTC4 Compatibility Mode . = 5: RTC5 Standard Mode (default setting).
Comments	- The RTC5 DLL operation mode can be set by set_rtc4_mode or set_rtc5_mode. The default setting is RTC5 Standard Mode. get_rtc_mode is available even without explicit access rights to a particular RTC5 Board. get_rtc_mode is not available as a multi-board command. - The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by get_rtc_mode.
RTC4→RTC5	New command.
Version info	–
References	set_rtc4_mode , set_rtc5_mode

Ctrl Command	get_rtc_version
Function	Returns the version number of the FPGA firmware (RTC5RBF.rbf) and information about enabled options of the RTC5 Board.
Call	RTCVersion = get_rtc_version()
Result	Version number of the FPGA firmware and additional information. As an unsigned 32-bit value. Bit #0 (LSB) Version number of the FPGA firmware (RTC5RBF.rbf). ... Bit #7 Bit #8 = 1: Option Processing-on-the-fly is enabled. See also Chapter 8.6 "Processing-on-the-fly", Page 227. Bit #9 = 1: Option "Second Scan Head Control" is enabled. See also Section "2. SCANHEAD Socket Connector", Page 55. Bit #10 = 1: Option "3D" is enabled. See also Chapter 8.5.2 "3D Scan Systems", Page 222. Bit #11 Reserved. Bit #12 Reserved. Bit #13 Reserved. Bit #14 Reserved. Bit #15 Reserved. Bit #16 DSP version number. ... Bit #23 Bit #24 Subversion number of the FPGA firmware (RTC5RBF.rbf). ... Bit #31
Comments	• The FPGA firmware version numbers are in the range 500...599. • get_rtc_version(Bit #0...Bit #7) returns values in the range 0...99 (version number – 500). • The current DSP version number is 2 (0 and 1 are older versions). • The current FPGA firmware subversion is 0. • The FPGA firmware version can even be returned after an RTC5_VERSION_MISMATCH or RTC5_ACCESS_DENIED error. The return value is 0 if no program has yet been loaded. • The board-specific error variables LastError and AccError, see Section "Error Handling", Page 119, are neither generated nor altered by get_rtc_version.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_hex_version , get_dll_version

Ctrl Command	get_serial
Function	Returns the current serial number of the serial-number-set selected by select_serial_set (or of serial-number-set 0 after load_program_file).
Call	CurrentSerialNo = get_serial()
Result	Serial number. As a 64-bit IEEE floating point value.
Comments	- For get_serial usage, see Chapter 7.5.2 “Marking Serial Numbers”, Page 196. get_serial should not be confused with get_serial_number, which returns the product serial number of the RTC5 Board.
RTC4→RTC5	New command.
Version info	–
References	select_serial_set

Ctrl Command	get_serial_number
Function	Returns the individual serial number of the active RTC5 Board.
Call	RTCSerialNumber = get_serial_number()
Result	RTC5 serial number. As an unsigned 32-bit value.
Comments	- Serial numbers of installed boards are ascertained by init_rtc5_dll and cached in the DLL, from where you can query them by get_serial_number. get_serial_number is helpful when using several RTC5 Boards in one computer (see Chapter 6.6 “Using Several RTC5 PCI Boards in One PC”, Page 112). The associated multi-board command n_get_serial_number can be used for determining the relationship between the installed boards and the RTC5 DLL-internal numbers assigned to them during initialization. The RTC5 DLL-internal numbers are newly assigned during each initialization of a user program (see init_rtc5_dll) and must be supplied for a variety of commands (in particular, all multi-board commands). The number of boards found during initialization can be queried by rtc5_count_cards. - The get_serial_number and n_get_serial_number commands are even available without explicit access rights to a specific RTC5 Board. - The board-specific error variables LastError and AccError (see Chapter 6.8 “Error Handling”, Page 119) are neither generated nor altered by get_serial_number.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	rtc5_count_cards

Ctrl Command	get_standby
Call	get_standby(&HalfPeriod, &PulseLength)
Returned parameter values	HalfPeriod <i>Half</i> of the currently set standby output period of the standby pulses. As a pointer to an unsigned 32-bit value. 1 bit equals 1/64 μ s.
	PulseLength Currently set pulse length of the standby pulses. As a pointer to an unsigned 32-bit value. 1 bit equals 1/64 μ s.
Comments	For get_standby usage, see Section “Signals for “Laser Standby” Operation”, Page 175.
RTC4→RTC5	New command. n RTC4 Compatibility Mode, the RTC5 divides the values for HalfPeriod and PulseLength by 8.
Version info	–
References	set_standby

Ctrl Command	get_startstop_info
Function	Provides information about internal and External Starts and External Stops since the last call of get_startstop_info . Also provided are the current External Start and External Stop levels, the status and signal level of the laser control signals, and possible transmission errors to and from the attached scan system.
Call	StartStopInfo = get_startstop_info()
Result	Info signal. As an unsigned 32-bit value. Bit #0 = 1: An internal start has been executed (by execute_list or similar) since the last call of get_startstop_info. (LSB) Bit #1 = 1: An External Start has been executed (by /START, /START2, /Slave-START, simulate_ext_start or simulate_ext_start_ctrl) since the last call of get_startstop_info. Bit #2 = 1: An internal stop has been executed (by stop_execution) since the last call of get_startstop_info. Bit #3 = 1: An External Stop has been executed (by /STOP, /STOP2, /Slave-STOP or simulate_ext_stop) since the last call of get_startstop_info. Bit #4 Ext-stop status (= logical AND operation of the signals /STOP, /STOP2, /Slave-STOP and simulate_ext_stop, see Figure 71): = 1: No stop signals are currently present at the inputs or the inputs are not connected. = 0: There is a stop signal at at least one input. Bit #5 Reserved. Bit #6 Reserved. Bit #7 Reserved. Bit #8 Reserved. Bit #9 = 1: The laser control signals are globally enabled, see Chapter 7.4.1 "Enabling, Activating and Switching Laser Control Signals", Page 173. See also set_laser_control. Bit #10 = 1: The TTL laser control signals at the LASER1 and LASER2 output ports are active-LOW (the signal level can be defined by set_laser_control). Bit #11 = 1: Since the last call of get_startstop_info, at least one External Start has failed (more External Starts were triggered than could be simultaneously held in the 8-start wait loop). Bit #12 Ext-Start status (= logical AND operation of the signals /START, /START2 and /Slave-START, see Figure 71): = 1: No start signals are currently present at the inputs or the inputs are not connected. = 0: A start signal is present at at least one input. Bit #13 = 1: The TTL laser control signal at the LASERON output port is active-LOW (the signal level can be defined by set_laser_control).

Ctrl Command	get_startstop_info
Result (cont'd)	Bit #14 = 1: The laser control signals are enabled (enable_laser). = 0: The laser control signals are disabled (disable_laser). Bit #15 Reserved. Bit #16 The error bits. Get set when an error occurs during data transmission from the scan system: ... Bit #16...Bit #23 For the first scan head connector. Bit #24...Bit #31 For the second scan head connector. Bit #16, Bit #24: Incorrect number of frames within a data block. Bit #17, Bit #25: Incorrect pulse length of signal received from scan system, maybe no scan system is connected. Bit #18, Bit #26: Preamble sequence incorrect. Bit #19, Bit #27: Bit count within a subframe incorrect. Bit #20, Bit #28: Parity error when reading data received from scan system. Bit #21, Bit #29: The present data is invalid (old). Bit #22, Bit #30: Reserved. Bit #23, Bit #31: Reserved.
Comments	- After get_startstop_info has been executed, reset are: - The info bits Bit #0...Bit #3 - The error bits Bit #16...Bit #31 - See also Section "External Stop", Page 265 and Section "External Start", Page 266.
RTC4→RTC5	Unchanged functionality for info bits that are also used on the RTC4.
Version info	Last change DLL 546: Bit #14 .
References	get_counts, get_status, set_control_mode

Ctrl Command	get_status	
Function	Returns the current List Execution Status values and the current (or most recent) position of the output pointer.	
Call	get_status(&Status, &Pos)	
Returned parameter values	Status	Status value. As a pointer to an unsigned 32-bit value.. Bit #0 = 1: BUSY list execution status set. (LSB) Bit #1 Reserved. ... Bit #6 Bit #7 = 1: INTERNAL-BUSY list execution status set. Bit #8 Reserved. ... Bit #14 Bit #15 = 1: PAUSED list execution status set. Bit #16 0. ... Bit #31
	Pos	Current (or most recent) position of the output pointer (absolute memory address). As a pointer to an unsigned 32-bit value..
Comments (cont'd)	- For a description of when the BUSY list execution status, INTERNAL-BUSY list execution status or PAUSED list execution status values are set or not set, see Chapter 6.4.3 "List Execution Status", Page 97. (BUSY list execution status and PAUSED list execution status set) requires restart_list for continuation, (BUSY list execution status not set and PAUSED list execution status set) requires release_wait and (both BUSY list execution status and PAUSED list execution status not set) requires execute_list_pos. "Continuation" is not allowed with (BUSY list execution status set and PAUSED list execution status not set) and a currently running list. An improper continuation generates the get_last_error return code RTC5_BUSY. With (INTERNAL-BUSY list execution status set), release_wait and execute_list_pos are only executed with a delay (after INTERNAL-BUSY list execution status has been reset again). - The output pointer points to the command (in "List 1" or "List 2") currently being executed or most recently executed. If, during processing of a subroutine in the protected list memory area "List 3", the output pointer position Pos is queried, then the position is returned of <i>the</i> list command in the list area ("List 1" or "List 2") in which the output pointer most recently resided (typically from where the subroutine has been called (for example, with list_call). pause_list and set_wait leave Pos unchanged.	

Ctrl Command	get_status
Comments (cont'd)	• get_status returns the output pointer position as an absolute memory address (offset relative to the start of "List 1"). The relative position referenced to the start of the respective list area can be queried by get_out_pointer. • The current input pointer position can be queried by get_input_pointer or get_list_pointer. • List Status values for individual lists can be queried by read_status. • get_status, get_input_pointer and read_status can be used during loading of a list to ensure that no list is overwritten that has still not been processed (see also load_list). • As long as no program has been loaded (by load_program_file), get_status returns undefined values. • get_head_status is available for querying the status signals of the scan heads.
RTC4→RTC5	Basically unchanged functionality. However: The Parameter Status returns the BUSY list execution status , INTERNAL-BUSY list execution status and PAUSED list execution status with an RTC5, whereas only the BUSY list execution status with an RTC4.
Version info	–
References	read_status , get_input_pointer , get_list_pointer , get_out_pointer

Ctrl Command	get_stepper_status
Function	Returns the following status information for both stepper motor outputs: the current statuses of the stepper motor signals, the currently defined CLOCK pulse period, the status "Busy" and status "Init", and the current values of the internal position variables.
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), get_stepper_status is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>get_stepper_status(&Status1, &Pos1, &Status2, &Pos2)</code>
Returned parameter values	Status1 Current status of stepper motor output 1. As a pointer to an unsigned 32-bit value. Bit #0 ENABLE signal. (LSB) Bit #1 DIRECTION signal. Bit #2 CLOCK signal. Bit #3 SWITCH signal = limit switch signal. Bit #4 Status "Busy". Bit #5 Status "Init". Bit #6 Reserved. Bit #7 Reserved. Bit #8 CLOCK pulse period (24-bit value). ... Bit #31
	Pos1 Current value of the internal position variable for stepper motor output port 1. As a pointer to a signed 32-bit value.
	Status2 Current status of stepper motor output port 1. Otherwise, like Status1 .
	Pos2 Like Pos1 .
Comments	• For programming the stepper motor signals, see Chapter 9.1.5 "Controlling Stepper Motors", Page 260. • If the SWITCH signal (limit switch signal) is set to (Pin is LOW), no more CLOCK pulses are generated. • Status "Busy" indicates that a previously initiated (by stepper_abs, stepper_rel, etc.) set-position motion has not yet completed. • Status "Init" indicates that a previously initiated (by stepper_init) reference motion has not yet completed.
RTC4→RTC5	New command.
Version info	–
References	–

Ctrl Command	get_sub_pointer
Function	Returns the absolute start address of an indexed subroutine.
Call	<code>SubPointer = get_sub_pointer(Index)</code>
Parameters	Index Index of the indexed subroutine. As an unsigned 32-bit value. Allowed value range: [0...1023].
Result	Absolute start address. As an unsigned 32-bit value.
Comments	• get_sub_pointer reads from the internal management table the start address of the indexed subroutine with the specified index. Whether the read address resides in a protected or the unprotected list memory area depends on whether the subroutine has been loaded into the protected list memory area "List 3" or an unprotected subroutine has been only subsequently referenced. • If <code>Index > 1023</code> or if no subroutine has been referenced with the specified index, then get_sub_pointer returns the value "-1" (for example, $2^{32}-1$). • get_sub_pointer is useful for checking if a subroutine has already been defined or for calling an indexed subroutine by an absolute memory address as if it were a non-indexed subroutine. Be aware, though, that a subsequent save_disk/load_disk might alter the absolute memory address.
RTC4→RTC5	New command.
	—
References	get_char_pointer , get_text_table_pointer

Ctrl Command	get_sync_status
Function	Returns the master/slave synchronization status of the addressed RTC5 Board.
Call	<code>MasterSlaveSyncStatus = get_sync_status()</code>
Result	Master/slave synchronization status [0...640]. As an unsigned 32-bit value. < 11: The board is synchronized to the master board (or to the preceding board in the master/slave chain). = 640: The addressed board is operated as master.
Comments	• For get_sync_status usage, see Chapter 6.6.3 "Master/Slave Operation", Page 113. • If the addressed board has not been initialized previously by load_program_file, the get_sync_status returns 0. • First synchronize the boards of the master/slave chain by the sync_slaves command before using get_sync_status. • To ensure that get_sync_status actually returns the current master/slave synchronization status, you should first have already triggered an External Start for the master board by an external start signal or by simulate_ext_start_ctrl (see Section "External Start", Page 266). This start then initiates measurement of the time differences between each board's /SLAVE start pulse and the corresponding 10 μs clock. This time difference gets stored on each board as the master/slave synchronization status (in units of 15 ns) and remains stored there until the a new External Start is triggered for the master board. This gives you the flexibility to query the synchronization status by get_sync_status even at a later point in time. • In the synchronized state, measured time differences are shorter than the transit time difference for synchronization between the boards themselves (see Chapter 6.6.3 "Master/Slave Operation", Page 113): master/slave synchronization status < 11. The master/slave synchronization status is typically 7. • In a not-explicitly-synchronized state (prior to sync_slaves), the master/slave synchronization status might coincidentally be < 11. If so, then the boards behave as if synchronized.
RTC4→RTC5	New command.
Version info	–
References	sync_slaves , get_master_slave

Ctrl Command	get_table_para
Function	Returns the value of the specified parameter from a currently loaded correction table.
Call	TablePara = get_table_para(TableNo, ParaNo)
Parameters	TableNo Number of the currently loaded correction table. As an unsigned 32-bit value. Allowed values: [1...4]. See also number_of_correction_tables .
	ParaNo Number of the parameter. As an unsigned 32-bit value. Allowed values: 0...15. Assignment see Section "ct5 Correction File Header", Page 167 .
Result	Parameter value (see Section "ct5 Correction File Header", Page 167). As a 64-bit IEEE floating point value.
Comments	• The parameter values can be read out by get_table_para from a currently loaded correction table and by get_head_para from an assigned correction table and thus directly incorporated into a user program (see Section "ct5 Correction File Header", Page 167). • If the parameters TableNo and ParaNo are out of range, then the return value is 0 (get_last_error return code RTC5_PARAM_ERROR). • If no correction table with the specified number has been loaded, then the parameter values are undefined.
RTC4→RTC5	New command.
Version info	–
References	–

Ctrl Command	get_text_table_pointer
Function	Returns the absolute start address of an indexed text string.
Call	<code>TextTablePointer = get_text_table_pointer(Index)</code>
Parameters	Index Index of the indexed text string. As an unsigned 32-bit value. Allowed value range: [0...41].
Result	Absolute start address. As an unsigned 32-bit value.
Comments	• get_text_table_pointer reads from the internal management table the start address of the indexed text string with the specified index. Whether the read address resides in a protected or the unprotected list memory area depends on whether the text string has been loaded into the protected list memory area "List 3" or an unprotected subroutine has been only subsequently referenced. • If <code>Index > 41</code> or if no text string has been referenced with the specified index, then get_text_table_pointer returns the value "-1" (for example, $2^{32}-1$). • get_text_table_pointer is useful for checking if a text string has already been defined or for calling an indexed text string by an absolute memory address as if it were a non-indexed subroutine, for example, for conditional execution with list_call_cond. Be aware, though, that a subsequent save_disk/load_disk might alter the absolute memory address. And you should ensure that get_text_table_pointer does not return "-1"; otherwise, list_call_cond is ignored.
RTC4→RTC5	New command.
Version info	–
References	get_char_pointer , get_sub_pointer

Ctrl Command	get_time
Function	Returns the RTC5 Timer value (without resetting it to zero). It has been stored during the most recent call of save_and_restart_timer .
Call	<code>TimerValue = get_time()</code>
Result	RTC5 Timer value. In seconds. As a 64-bit IEEE floating point value.
Comments	• See save_and_restart_timer. • The number of elapsed list-command clock cycles since the reset to zero can be queried by get_lap_time.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_lap_time , save_and_restart_timer

Ctrl Command	get_transform
Function	Transfers to the PC the position values that were recorded by set_trigger and stored on the RTC5. Applies backward transformation to these values.
Call	<code>get_transform(Number, Ptr1, Ptr2, Ptr, Code)</code>
Parameters	Number Number [1...2²⁰] of to-be-backward-transformed position values. The measured values with index 0 to (Number-1) are backward transformed. As an unsigned 32-bit value.
	Ptr1 Pointers (in C and C++ data type ULONG_PTR, an unsigned 32-bit value or unsigned 64-bit value) to the two areas of PC main memory to which the backward transformed values should be transferred (see also Code).
	Ptr2
	Ptr Pointer (in C and C++ data type ULONG_PTR, an unsigned 32-bit value or unsigned 64-bit value) to the area of PC main memory to which the correction and transformation settings for backward transformation were previously transferred by upload_transform.
	Code Controls aspects of backward transformation, particularly which partial transformations should be performed: If a partial transformation should <i>not</i> be performed, then its corresponding bit (#2...#5) must be set to 1. This parameter has the same meaning as in transform (Ptr1 corresponds to Sig1 and Ptr2 to Sig2). For Bit #0 = 0, the measurement value pairs recorded by set_trigger are backward transformed as XY coordinates: Bit #1 = 0: Values recorded by measurement channel 1 (Signal1) are transferred to the PC using Ptr1 and then backward transformed as X coordinates. Values recorded by measurement channel 2 (Signal2) are transferred to the PC using Ptr2 and then backward transformed as Y coordinates. = 1: Values recorded by measurement channel 1 (Signal1) are transferred to the PC using Ptr1 and then backward transformed as Y coordinates. Values recorded by measurement channel 2 (Signal2) are transferred to the PC using Ptr2 and then backward transformed as X coordinates. Bit #2 = 0: Gain/offset correction of automatic self-calibration is backward transformed. Bit #3 = 0: The Image field correction is backward transformed. Bit #4 = 0: The offset of the defined coordinate transformation is backward transformed. Bit #5 = 0: The total matrix of the defined coordinate transformation is backward transformed. Bit #6 Reserved. ... Bit #31

Ctrl Command	get_transform
Parameters (cont'd)	Code (cont'd) If Bit #0 = 1, then (only) the values recorded with set_trigger by one of the two measurement channels (either channel 1 or 2) are backward transformed as Z coordinates: Bit #1 = 0: Values recorded by measurement channel 1 (Signal1) are transferred to the PC using Ptr1 and then backward transformed as Z coordinates. Values recorded by measurement channel 2 (Signal2) are transferred untransformed to the PC using Ptr2. Bit #1 = 1: Values recorded by measurement channel 2 (Signal2) are transferred to the PC using Ptr1 and then backward transformed as Z coordinates. Values recorded by measurement channel 1 (Signal1) are transferred untransformed to the PC using Ptr2. Bit #2 = 0: The offset to the focal length defined by set_defocus or set_defocus_list is backward transformed. Bit #3 = 0: The ABC correction is backward transformed. Bit #4 = 0: The offset to the z coordinate defined by set_offset_xyz or set_offset_xyz_list is backward transformed. Bit #5 Reserved. ... Bit #31 As an unsigned 32-bit value.
Comments	- For backward transformation of position values see Chapter 8.1.3 "Monitoring the Positioning", Page 200. get_transform(Number, Ptr1, Ptr2, Ptr, Code) transfers to the PC the data pairs recorded by set_trigger by executing get_waveform(1, Number, Ptr1) and get_waveform(2, Number, Ptr2) and overwrites the data pairwise by using transform(Sig1, Sig2, Ptr, Code). - Prior to calling get_transform, you must call upload_transform. Additionally, position values should have been recorded by set_trigger. - Before calling get_transform (as with get_waveform), you can check by measurement_status whether a measurement session is currently running that has been started with set_trigger. We recommend not to read any data when data recording is currently active. The number Pos for the last (or current) data pair of the measurement session can be queried by measurement_status. No more than Pos + 1 data elements should be read. Any further elements are from earlier recordings or the initialization. - The PC main memory areas pointed to by Ptr1 and Ptr2 must have been sufficiently allocated by users (Number × 4 bytes per channel). - If only z position values are to be backward transformed (Code Bit #0 = 1), then you can set the pointer (Ptr1 or Ptr2) for the unused return channel to NULL. This channel does then not transfer and backward transform data to the PC. Ensure here that Code Bit #1 is appropriately set.

Ctrl Command	get_transform
Comments (Forts.)	• If the command call is made with <code>Ptr1 = NULL</code> and <code>Ptr2 = NULL</code>, then neither channel transfers data to the PC and likewise no backward transformation is performed (get_last_error return code <code>RTC5_PARAM_ERROR</code>). The same applies for <code>Number = 0</code> or <code>Number > 2²⁰</code>. • If <code>Ptr = NULL</code>, then no backward transformation is performed. The data recorded by set_trigger is then transferred untransformed to the PC, as with get_waveform(1, <code>Number</code>, <code>Ptr1</code>) and get_waveform(2, <code>Number</code>, <code>Ptr2</code>). The same applies for <code>Ptr ≠ NULL</code> if an error occurred during execution of get_transform (for example, when data referenced by <code>Ptr</code> are invalid or erroneous or when z axis inversion is not possible). Here, however, a get_last_error return code of <code>RTC5_PARAM_ERROR</code> is additionally generated. • If needed, the values recorded with set_trigger and backward-transformed transferred to the PC by get_transform can additionally be untransformed transferred to the PC by get_waveform. • If backward transformation of z position values is requested (<code>Code Bit #0 = 1</code>), but only a 2D correction table has been assigned at the point in time of the prior successful call to upload_transform, then the offsets to the focal length and Z coordinates are initialized with 0 and the values A, B, C with are initialized 0, 1, 0 (1-to-1 backward transformation). • For backward transformation of xy position values (<code>Code Bit #0 = 0</code>), only the <code>z = 0</code> plane are transformed. XY stretching and Z defocus due to z deviations (particularly with non-F-Theta systems) are not taken into account. • PCI transmission errors generate the get_last_error return code <code>RTC5_SEND_ERROR</code>.
RTC4→RTC5	New command. Even in the RTC4 Compatibility Mode, all coordinate values transferred to the PC are in the RTC5 20-bit range. The backward transformed coordinate values must, if necessary, be divided by 16 by users themselves.
Version info	–
References	upload_transform , transform , set_trigger , get_waveform

Ctrl Command	get_value
Function	Returns the current value of the specified signal.
Call	Value = get_value(Signal)
Parameters	Signal Desired signal type. As an unsigned 32-bit value.
Result	Current value of the specified signal. As a signed 32-bit value.
Comments	- The selectable signal types are identical to those of the set_trigger command (refer to the comments there for the allowed value range, signal types and other information). If the value for Signal is unallowed, then get_value does not read out a signal and returns 0 (get_last_error return code RTC5_PARAM_ERROR). - To observe the specified signal over a long time period, use set_trigger to start a corresponding measurement session. - When using an iDRIVE scan system (see Glossary entry on page 23), after a reset or power-up of the scan system, it can take around 5 seconds before valid data starts being returned from the scan system. - For Signal = 0, get_value returns the current laser status (LASERON signal) even when list execution has already been finished. - When you query data returned as status signals from the scan system to the RTC5 (Status<AX...BY>), then be mindful of the returned data type's value range when evaluating it (see control_command): - Data types originally generated in the scan system as unsigned 16-bit values (for example, the XY2-100 status word or the serial number) and returned to the RTC5 as unsigned 20-bit values (whereby bits #0...3 = 0) contain the relevant information in bits #4...19. Here, only bits #4...19 should be evaluated (see code example below). For bits #20...31 of this data type, get_value returns to the PC not only zero, but sometimes (depending on Bit #19 of the underlying 16-bit status value) even the value one. So only evaluate bits #4...19. - In contrast, data types returned to the RTC5 as signed 20-bit values (for example, actual positions or actual speeds) can be safely evaluated as a complete signed 32-bit value returned by get_value (see also code example below). - Although z values must be supplied as a 16 bit values in 3D command parameters, SCANLAB Z axes and 3-axis scan systems (and correspondingly get_value) return z values always as signed 20-bit values.

Ctrl Command	get_value
Example (C/C++)	Querying diverse data types (first scan head connector, x axis): a) XY2-100 status word, PowerOK status <pre> UINT statusword, powerOK; control_command (1, 1, 0x0500); // only applicable for iDRIVE systems statusword = (get_value(1) & 0x000FFFF0) >> 4; powerOK = (statusword & 0x00000080); </pre> b) Serial number (only applicable for iDRIVE systems) <pre> UINT SN_low, SN_high, SN; // the serial number's lower 16 bits are selected for return // and queried by get_value: control_command (1, 1, 0x051E); SN_low = (get_value(1) & 0x000FFFF0)>>4; // the serial number's upper 16 bits are selected for return // and queried by get_value: control_command (1, 1, 0x051F); SN_high = (get_value(1) & 0x000FFFF0)>>4; //Complete serial number: SN = (SN_high << 16) + SN_low; </pre> c) Actual position (only applicable for iDRIVE systems) <pre> long real_position; control_command (1, 1, 0x0501); real_position = get_value(1); </pre>
RTC4→RTC5	Basically unchanged functionality. However: Even in RTC4 Compatibility Mode, all returned values are in the RTC5 20-bit range, but are transferred to the PC as 32-bit values (see above).
Version info	–
References	get_values, set_trigger, get_waveform, get_head_status

Ctrl Command	get_values
Function	Returns the current values of up to 4 specified signals.
Call	<code>get_values(SignalPtr, ResultPtr)</code>
Parameters	SignalPtr Pointer (in C and C++ data type <code>ULONG_PTR</code> , an unsigned 32-bit value or unsigned 64-bit value) to an array of 4 unsigned 32-bit values, where the desired signal types are specified.
	ResultPtr Pointer (in C and C++ data type <code>ULONG_PTR</code> , an unsigned 32-bit value or unsigned 64-bit value) to an array of 4 signed 32-bit values, where the current values of the up to 4 specified signals are to be stored.
Comments	- Up to 4 desired signals can be simultaneously queried. The selectable signal types are identical to those of the set_trigger command (refer to the comments there for the allowed value range, signal types and other information). The desired signal types must be specified by <code>SignalPtr</code>. The corresponding signal values are then stored by <code>ResultPtr</code>. For storage of each queried data set, the user program must make available (at the address specified by <code>ResultPtr</code>) 4×4 bytes of PC memory. get_values functions similarly to get_value (see comments there). get_values returns 0 and performs no query on channels for which an invalid signal type has been specified by <code>SignalPtr</code>. A get_last_error return code <code>RTC5_PARAM_ERROR</code> is only generated if all 4 specified signal types are invalid. - If any of the pointer parameters are <code>NULL</code>, then get_values is not executed (all return values are 0) and a get_last_error return code of <code>RTC5_PARAM_ERROR</code> is generated.
RTC4→RTC5	New command. Even in RTC4 Compatibility Mode, the 4 returned values are in the RTC5 20-bit range, but are transferred to the PC as 32-bit values (see comments for get_value).
Version info	–
References	get_value , set_trigger , transform

Ctrl Command	get_wait_status
Function	Returns the wait state of the RTC5.
Call	<code>WaitStatus = get_wait_status()</code>
Result	Wait state. As an unsigned 32-bit value.
Comments	• If processing has stopped at a wait marker, get_wait_status returns the number of this marker. See set_wait. • If no wait marker has been reached, get_wait_status returns zero. • Processing of the list can be resumed by calling release_wait.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_wait , release_wait

Ctrl Command	get_waveform
Function	Transfers to the PC the data that has been measured and stored onto the RTC5 by set_trigger or set_trigger4 .
Call	<code>get_waveform(Channel, Number, Ptr)</code>
Parameters	Channel Measurement channel [1 or 2; if recordings started by set_trigger4 , then also 3 or 4]; specified. As an unsigned 32-bit value.
	Number Number [1...2 ²⁰ for set_trigger , 1...2 ¹⁹ for set_trigger4] of measured values to transfer. Values of measurement positions 0 to (Number-1) are transferred. As an unsigned 32-bit value.
	Ptr Pointer (data type ULONG_PTR in C and C++, an unsigned 32-bit or unsigned 64-bit value) to a location in the PC memory to where the measured values should be transferred.
Comments	• In the following cases, no data is transferred (get_last_error return code RTC5_PARAM_ERROR): – For Number = 0. – For Number > 2²⁰. – For Number > 2¹⁹ when Channel = 3 or 4 has been selected or if a file has been loaded as correction table No=3 or No=4 with load_correction_file. – For Ptr = NULL. • PCI transmission errors generate the get_last_error return code RTC5_SEND_ERROR. • Before calling get_waveform, you can check by measurement_status whether a measurement session is currently running that has been started with set_trigger or set_trigger4. We recommend not reading any data when data recording is currently active. The number Pos for the last (or current) data pair of the measurement session can be queried by measurement_status. No more than Pos+1 data elements should be read. Any further elements are from earlier recordings or the initialization.
RTC4→RTC5	Basically unchanged functionality. However: Even in RTC4 Compatibility Mode , all values are in the RTC5 20-bit range, but are transferred to the PC as 32-bit values (see comments for get_value).
Version info	–
References	set_trigger , set_trigger4 , get_value , get_values

Ctrl Command	get_z_distance
Function	Returns the focus length value <i>I</i> for the specified point within the 3D image field .
Restriction	If the Option "3D" has not been enabled or if no 3D correction table has been assigned (see select_cor_table), then get_z_distance returns 0 and otherwise has no effect.
Call	<code>ZDistance = get_z_distance(X, Y, Z)</code>
Parameters	X Absolute coordinates of the point (x y z) in the 3D image field . In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
	Z Like X (analogously). Allowed value range: [-32,768...+32,767].
Result	Focus length value. [-32,768...+32,767]. As a signed 32-bit value.
Comments	get_z_distance is only needed for re-calibrating the z axis in a 3-axis scan system, see Section "Checking the z Axis Calibration", Page 159. - The focus length value <i>I</i>: - Has no dimension - Corresponds to the focus length difference between the specified point (x y z) and the point (0 0 0) - Can be positive or negative - With the RTC5, <code>ZDistance</code> is always in 16-bit range [-32,768...+32,767]: - In RTC4 Compatibility Mode - In RTC5 Standard Mode get_z_distance first performs a (virtual) jump to the point (x y z) and then returns the focus length value. - If a list is currently executed, then get_z_distance has no effect and returns 0 (get_last_error return code RTC5_BUSY). get_z_distance is not executed and returns 0 (get_last_error return code RTC5_BUSY), if: - the BUSY list execution status is set - the INTERNAL-BUSY list execution status is set get_z_distance is even executed, if: - a list has been paused by set_wait (PAUSED list execution status set) - For 3D image field calibration, see Chapter "3D Commands", Page 223.
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode.
Version info	–
References	load_z_table

Ctrl Command	goto_xy
Function	Moves the output point (of the laser focus) along a 2D vector at jump speed from the current position to the specified position (absolute coordinate values) within a 2D Image field .
Call	<code>goto_xy(X, Y)</code>
Parameters	X Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
Comments	• If the jump speed has not been previously explicitly set by set_jump_speed or set_jump_speed_ctrl, then the jump is executed at a predefined jump speed of 10000 <i>bits/ms</i>. • goto_xy (unlike the list commands jump_abs and jump_rel) has no effect on the laser control signals and also does not set a Jump Delay. • Previously accumulated coordinate transformations become effective by goto_xy, see list item "With <code>at_once = 2 ...</code>", page 211. • goto_xy is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set – an external stop signal (/STOP, /STOP2 or /Slave STOP) is present It can also be generated by automatic monitoring, see set_laser_control and range_checking. • goto_xy is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • The INTERNAL-BUSY list execution status is set while goto_xy is executed. • goto_xy only returns to the user program when the motion has been completed.
RTC4→RTC5	• Extended range of values. • goto_xy returns only after motion has been executed (see comments above). • In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	set_jump_speed , jump_abs , jump_rel , goto_xyz , get_status

Ctrl Command	goto_xyz	
Function	Moves the output point (of the laser focus) along a 3D vector at jump speed from the current position to the specified position (absolute coordinate values) within a 3D image field.	
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then goto_xyz has the same effect as goto_xy . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.	
Call	goto_xyz(X, Y, Z)	
Parameters	X	Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	Y	Like X (analogously).
	Z	Like X (analogously). Allowed value range: [-32,768...+32,767].
Comments	- Except for the additional motion in the third dimension, goto_xyz functions similarly to goto_xy (see the comments there). - The DirectMove3D parameter of set_delay_mode determines the type of z axis motion (linear or with stepwise correction). - See also Chapter 7.3.6 "Output Values to the Scan System", Page 170.	
RTC4→RTC5	- Extended range of values. goto_xyz returns only after the motion has completed (see comments for goto_xy). RTC4 Compatibility Mode: see goto_xy. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode.	
Version info	–	
References	goto_xy , jump_abs_3d , jump_rel_3d	

Ctrl Command	home_position
Function	Activates the home jump mode (for the x axis and y axis) and defines the home position.
Call	<code>home_position(XHome, YHome)</code>
Parameters	XHome Absolute x coordinate of the home position. In bits. As a signed 32-bit value. Allowed value range: [-524,288... +524,287]. Larger values are clipped.
	YHome Like XHome (analogously).
Comments	• home_position defines the coordinates of a home jump to be executed, for example, upon reaching the end of a list. Accordingly, a home return to the last valid position is then executed again at start. The home jump and home return are executed at jump speed. • home_position is intended for a laser system that does not allow fast switching of the laser. After calling the command, the laser focus moves to the specified home position whenever no list is executing or when a list has been paused by set_wait. A home jump is also executed, if list execution is stopped by stop_execution or by an external stop signal. The home jump itself (in contrary to a home return) cannot be stopped. • While a home jump or a home return is executed, the INTERNAL-BUSY list execution status is set. • A <i>beam dump</i> should be placed in the home position. • The home jump mode is deactivated by <code>home_position(0, 0)</code>, even if it has been activated by home_position_xyz.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for XHome and YHome by 16. The allowed value range decreases accordingly.
Version info	–
References	home_position_xyz

Ctrl Command	home_position_xyz
Function	Activates the home jump mode (for the x axis, y axis and z axis) and defines the home position.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then home_position_xyz has the same effect as home_position . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>home_position_xyz(XHome, YHome, ZHome)</code>
Parameters	XHome Absolute x coordinate of the home position. In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	YHome Like XHome (analogously).
	ZHome Like XHome (analogously). Allowed value range: [-32,768...+32,767].
Comments	- Except for the additional motion in the third dimension, home_position_xyz functions similarly to home_position (see comments there). - The home jump mode is deactivated by: <code>home_position(0, 0)</code> <code>home_position_xyz(0, 0, 0)</code> - The DirectMove3D parameter of the set_delay_mode command determines the type of z axis motion (linear or with stepwise correction).
RTC4→RTC5	New command. RTC4 Compatibility Mode: see home_position . The value range for ZHome is identical in RTC5 Standard Mode and RTC4 Compatibility Mode .
Version info	–
References	home_position

Undelayed Short List Command	if_cond
Function	<i>Conditional command execution:</i> if_cond immediately executes the directly following list command, if the current IOvalue at the EXTENSION 1 socket connector's 16-bit digital input port meets the following condition: $((\text{IOvalue AND Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, this list command is skipped.
Call	<code>if_cond(Mask1, Mask0)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1 .
Comments	See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	if_not_cond , if_pin_cond , if_not_pin_cond

Undelayed Short List Command	if_fly_x_overflow
Function	<i>Conditional command execution for Processing-on-the-fly applications:</i> if_fly_x_overflow immediately executes the directly subsequent list command, if the condition in accordance with Mode for the x axis has been fulfilled. Otherwise, this list command is skipped.
Call	<code>if_fly_x_overflow(Mode)</code>
Parameters	Mode To-be-evaluated condition. As a signed 32-bit value. = 0: Some kind of boundary exceedance occurred (error bit Bit #4 = 1 or error bit Bit #5 = 1). > 0: An overflow occurred (error bit Bit #5 = 1). < 0: An underflow occurred (error bit Bit #4 = 1).
Comments	- For usage of if_fly_x_overflow, see Section "Customer-Defined Monitoring Area", Page 241. - The error bits queried in accordance with Mode (error bit Bit #4 and/or error bit Bit #5) are reset.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , set_fly_limits , clear_fly_overflow , clear_fly_overflow_ctrl , if_not_fly_x_overflow

Undelayed Short List Command	if_fly_y_overflow
Function	Conditional command execution for Processing-on-the-fly applications: if_fly_y_overflow immediately executes the directly subsequent list command, if the condition in accordance with <i>Mode</i> for the y axis has been fulfilled. Otherwise, this list command is skipped.
Call	<code>if_fly_y_overflow(Mode)</code>
Parameters	<i>Mode</i> To-be-evaluated condition. As a signed 32-bit value. = 0: Some kind of boundary exceedance occurred (error bit Bit #6 = 1 or error bit Bit #7 = 1). > 0: An overflow occurred (error bit Bit #7 = 1). < 0: An underflow occurred (error bit Bit #6 = 1).
Comments	- For usage of if_fly_y_overflow, see Section "Customer-Defined Monitoring Area", Page 241. - The error bits queried in accordance with <i>Mode</i> (error bit Bit #6 and/or error bit Bit #7) are reset.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , set_fly_limits , clear_fly_overflow , clear_fly_overflow_ctrl , if_not_fly_y_overflow

Undelayed Short List Command	if_fly_z_overflow
Function	Conditional command execution for Processing-on-the-fly applications: if_fly_z_overflow immediately executes the directly subsequent list command, if the condition in accordance with <i>Mode</i> for the z axis has been fulfilled. Otherwise, this list command is skipped.
Call	<code>if_fly_z_overflow(Mode)</code>
Parameters	<i>Mode</i> To-be-evaluated condition. As a signed 32-bit value. = 0: Some kind of boundary exceedance occurred (error bit Bit #24 = 1 or error bit Bit #25 = 1). > 0: An overflow occurred (error bit Bit #25 = 1). < 0: An underflow occurred (error bit Bit #24 = 1).
Comments	- For usage of if_fly_z_overflow, see also Chapter 8.6.9 "Monitoring Processing-on-the-fly Corrections", Page 240. - The error bits queried in accordance with <i>Mode</i> (error bit Bit #24 and/or error bit Bit #25) are reset.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , set_fly_limits_z , clear_fly_overflow , clear_fly_overflow_ctrl , if_not_fly_z_overflow

Undelayed Short List Command	if_not_activated
Function	Conditional command execution due to an error bit from activate_fly_xy/activate_fly_xy_encoder or activate_fly_2d/activate_fly_2d_encoder : If the error bit is set, then the next list command is executed immediately, otherwise it is skipped.
Call	<code>if_not_activated()</code>
Comments	• See also comments at activate_fly_2d/activate_fly_2d_encoder and activate_fly_xy/activate_fly_xy_encoder. • It is useful to insert a list jump or subroutine call in the list directly after if_not_activated that jumps to an error-handling sequence. Or you could simply insert a set_end_of_list. • if_not_activated resets the error bit from activate_fly_xy/activate_fly_xy_encoder or activate_fly_2d/activate_fly_2d_encoder. If you still need it for subsequent querying by <code>get_marking_info(Bit #9)</code>, then you can include a renewed call of activate_fly_2d/activate_fly_2d_encoder or activate_fly_xy in your error-handling sequence to set the error bit again (provided that the error situation still exists). • Successful activation by activate_fly_2d/activate_fly_2d_encoder or activate_fly_xy/activate_fly_xy_encoder <i>does not</i> reset any already-set error bit. It remains set for get_marking_info until get_marking_info has been called.
RTC4→RTC5	New command.
Version info	–
References	activate_fly_2d , activate_fly_2d_encoder , activate_fly_xy , activate_fly_xy_encoder , get_marking_info

Undelayed Short List Command	if_not_cond
Function	<i>Conditional command execution:</i> if_not_cond immediately executes the directly following list command, if the current IOvalue at the EXTENSION 1 socket connector's 16-bit digital input port <i>does not meet</i> the following condition: $((\text{IOvalue AND Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are <i>not</i> 1 <i>or</i> the bits specified in Mask0 are <i>not</i> 0). Otherwise, this list command is skipped.
Call	<code>if_not_cond(Mask1, Mask0)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1 .
Comments	• See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	if_cond , if_pin_cond , if_not_pin_cond

Undelayed Short List Command	if_not_fly_x_overflow
Function	Conditional command execution for Processing-on-the-fly applications: if_not_fly_x_overflow immediately executes the directly subsequent list command, if the condition in accordance with <i>Mode</i> for the x axis is <i>not</i> fulfilled. Otherwise, this list command is skipped.
Call	<code>if_not_fly_x_overflow(Mode)</code>
Parameters	<i>Mode</i> To-be-evaluated condition. As a signed 32-bit value. = 0: Some kind of boundary exceedance occurred (error bit Bit #4 = 1 or error bit Bit #5 = 1). > 0: An overflow occurred (error bit Bit #5 = 1). < 0: An underflow occurred (error bit Bit #4 = 1).
Comments	• For usage of if_not_fly_x_overflow, see Section “Customer-Defined Monitoring Area”, Page 241. • The error bits queried in accordance with <i>Mode</i> (error bit Bit #4 and/or error bit Bit #5) are reset.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , set_fly_limits , clear_fly_overflow , clear_fly_overflow_ctrl , if_fly_x_overflow

Undelayed Short List Command	if_not_fly_y_overflow
Function	Conditional command execution for Processing-on-the-fly applications: if_not_fly_y_overflow immediately executes the directly subsequent list command, if the condition in accordance with <i>Mode</i> for the y axis is <i>not</i> fulfilled. Otherwise, this list command is skipped.
Call	<code>if_not_fly_y_overflow(Mode)</code>
Parameters	<i>Mode</i> To-be-evaluated condition. As a signed 32-bit value. = 0: Some kind of boundary exceedance occurred (error bit Bit #6 = 1 or error bit Bit #7 = 1). > 0: An overflow occurred (error bit Bit #7 = 1). < 0: An underflow occurred (error bit Bit #6 = 1).
Comments	• For usage of if_not_fly_y_overflow, see Section “Customer-Defined Monitoring Area”, Page 241. • The error bits queried in accordance with <i>Mode</i> (error bit Bit #6 and/or error bit Bit #7) are reset.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , set_fly_limits , clear_fly_overflow , clear_fly_overflow_ctrl , if_fly_y_overflow

Undelayed Short List Command	if_not_fly_z_overflow
Function	<i>Conditional command execution for Processing-on-the-fly applications:</i> if_not_fly_z_overflow immediately executes the directly subsequent list command, if the condition in accordance with <i>Mode</i> for the z axis has not been fulfilled. Otherwise, this list command is skipped.
Call	<code>if_not_fly_z_overflow(Mode)</code>
Parameters	<i>Mode</i> To-be-evaluated condition. As a signed 32-bit value. = 0: Some kind of boundary exceedance occurred (error bit Bit #24 = 1 or error bit Bit #25 = 1). > 0: An overflow occurred (error bit Bit #25 = 1). < 0: An underflow occurred (error bit Bit #24 = 1).
Comments	- For usage of if_not_fly_z_overflow, see also Chapter 8.6.9 "Monitoring Processing-on-the-fly Corrections", Page 240. - The error bits queried in accordance with <i>Mode</i> (error bit Bit #24 and/or error bit Bit #25) are reset.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info , set_fly_limits_z , clear_fly_overflow , clear_fly_overflow_ctrl , if_fly_z_overflow

Undelayed Short List Command	if_not_pin_cond
Function	<i>Conditional command execution:</i> if_not_pin_cond immediately executes the directly following list command, if the current <i>IOvalue</i> at the LASER connector's 2-bit digital input port <i>does not meet</i> the following condition: $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits specified in <i>Mask1</i> are <i>not</i> 1 or the bits specified in <i>Mask0</i> are <i>not</i> 0). Otherwise, this list command is skipped.
Call	<code>if_not_pin_cond(Mask1, Mask0)</code>
Parameters	<i>Mask1</i> 2-bit mask. As an unsigned 32-bit value. Only the lower 2 bits are evaluated. <i>Mask0</i> See <i>Mask1</i>.
Comments	See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	if_pin_cond , if_cond , if_not_cond

Undelayed Short List Command	if_pin_cond
Function	<i>Conditional command execution:</i> if_pin_cond immediately executes the directly following list command, if the current IOvalue at the LASER connector's 2-bit digital input port meets the following condition: $((\text{IOvalue AND Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, this list command is skipped.
Call	<code>if_pin_cond(Mask1, Mask0)</code>
Parameters	Mask1 2-bit mask. As an unsigned 32-bit value. Only the lower 2 bits are evaluated.
	Mask0 See Mask1.
Comments	• See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	if_not_pin_cond , if_cond , if_not_cond

Ctrl Command	init_fly_2d
Function	Initializes the encoder reference values for 2D encoder compensation of a subsequent set_fly_2d Processing-on-the-fly application and resets both encoder values.
Call	<code>init_fly_2d(OffsetX, OffsetY)</code>
Parameters	OffsetX Reference value of the x axis encoder. As a signed 32-bit value. Allowed value range: depends on the compensation table loaded with load_fly_2d_table . Default values (after load_program_file): = 0.
	OffsetY Like OffsetX (analogously).
Comments	The final encoder value for the 2D correction (that is, current encoder value + reference value) must not exceed the allowed range. Otherwise, clipping occurs (see also load_fly_2d_table).
RTC4→RTC5	New command.
Version info	–
References	set_fly_2d , load_fly_2d_table , get_fly_2d_offset

Ctrl Command	init_rtc5_dll
Function	Initializes control of the installed RTC5 Boards for a user program.
Call	<code>InitErrorNo = init_rtc5_dll()</code>
Result	Error code. As an unsigned 32-bit value. If multiple errors occurred simultaneously, then multiple bits are set. The meanings of bit numbers, error types and error constants is identical to those for get_error.
Comments	• init_rtc5_dll must be called before starting each user program. Otherwise, no RTC5s are accessible by the user program. init_rtc5_dll detects all available RTC5 Boards and establishes corresponding board management. If no RTC5 Boards are found, the error code RTC5_NO_CARD is returned. • If an error occurs when reading the PCI configuration register (get_last_error return code RTC5_CONFIG_ERROR), then an RTC5_ACCESS_DENIED error is also generated. The board then remains inaccessible until the error gets cleared by a PC restart. • The init_rtc5_dll command automatically assigns the user program access rights to the found boards (as by an acquire_rtc command) if access rights are not already assigned to another user program (any number of boards and applications may be used, but each particular board cannot be used simultaneously by multiple applications). The first initialization acquires all found boards for itself. Subsequent initializations started by other applications result in return of an RTC5_ACCESS_DENIED error code by init_rtc5_dll. The boards are then only accessible by these applications through RTC5 DLL-internal functions that require no access rights – for example, get_error, get_last_error or select_rtc (most multi-board commands, too, are only callable for boards with existing access rights). If a user program has access rights for a board, then the user program must first explicitly release its access rights with release_rtc or free_rtc5_dll before the board can be used by another user program by init_rtc5_dll or acquire_rtc. An RTC5_ACCESS_DENIED error code is returned if access has been denied by at least one of the found boards. Which board(s) denied access can be determined by <code>n_get_error(CardNo)</code> (CardNo from 1 to the number of found boards) or directly after init_rtc5_dll (before the next command) by <code>n_get_last_error(CardNo)</code>. • If a board is acquired by init_rtc5_dll (as by acquire_rtc, then a version compatibility check is performed. If a version error is detected, then access to the board is denied (return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH). • Only one user program can perform initialization at any one time. Subsequent initializations started by other applications wait until the current initialization is complete.

Ctrl Command	init_rtc5_dll
Comments (cont'd)	• If a user program calls the init_rtc5_dll command multiple times, then the RTC5 board management created by the prior call is deleted for this user program and the originally-assigned access rights canceled. RTC5 board management is then newly created and access rights newly assigned. • Each initialization of a user program by init_rtc5_dll causes the RTC5 DLL-internal numbers for all found RTC5 Boards to be newly reassigned. The relationship between the RTC5 DLL-internal numbers and the installed boards can be determined by get_serial_number. When the accessible board with the smallest RTC5 DLL-internal number is initialized, it then simultaneously becomes the active board and the target of non-multi-board commands. select_rtc can be called at anytime to change the active board. If no access rights exist for any board, then the board with the highest RTC5 DLL-internal number (see rtc5_count_cards) is the active board, in which case only RTC5 DLL-internal commands that require no access rights can be used. • Also observe Chapter 6.7.1 "Board Acquisition by a User Program", Page 117. • init_rtc5_dll is available even without explicit access rights to any particular RTC5 Board. • init_rtc5_dll is not available as a multi-board command. • After initialization of the RTC5 DLL with init_rtc5_dll, the RTC5 Standard Mode is set by default. After that, a different RTC5 DLL operational mode can be set, see set_rtc4_mode and set_rtc5_mode. • init_rtc5_dll does not trigger an initialization of RTC5 boards. Only load_program_file performs an initialization of RTC5 boards.
RTC4→RTC5	New command.
Version info	–
References	–

Normal List Command	jump_abs
Function	Moves the output point (for the laser focus) along a 2D vector at jump speed from the current position to the specified position (absolute coordinate values) within a 2D Image field .
Call	<code>jump_abs(X, Y)</code>
Parameters	X Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
Comments	• If the jump speed has not been previously explicitly set by set_jump_speed or set_jump_speed_ctrl, then the jump is executed at a predefined jump speed of 10000 <i>bits/ms</i>. • The Signals for "Laser Active" Operation are switched off before the jump and remain off during the jump. • After a Jump command, a (variable) Jump Delay is inserted. Exception: a zero-length jump vector's subsequent (variable) Jump Delay is not executed. However, the command itself still requires a 10 μs clock cycle for execution. • Previously accumulated coordinate transformations become effective by jump_abs, see list item "With <code>at_once = 2 ...</code>", page 211.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the values specified for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	set_jump_speed, set_scanner_delays, jump_rel, timed_jump_abs, goto_xy

Normal List Command	jump_abs_3d
Function	Moves the output point (of the laser focus) along a 3D vector at jump speed from the current position to the specified position (absolute coordinate values) within a 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then jump_abs_3d has the same effect as jump_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	jump_abs_3d(X, Y, Z)
Parameters	X Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	Z Like X (analogously), except Allowed value range: [-32,768...+32,767].
Comments	- Except for the additional motion in the third dimension, jump_abs_3d functions similarly to jump_abs (see comments there). - The DirectMove3D parameter of the set_delay_mode command determines the type of z axis motion (linear or with stepwise correction).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode .
Version info	–
References	jump_abs , jump_rel_3d , timed_jump_abs_3d

Normal List Command	jump_abs_drill
Comments	This command is described, for example, in the manual for the intelliDRILL de II 20 Module.

Normal List Command	jump_abs_drill_2
Comments	This command is described, for example, in the manual for the intelliDRILL de II 20 Module.

Normal List Command	jump_rel	
Function	Moves the output point (of the laser focus) along a 2D vector at jump speed from the current position to the specified position (relative coordinate values) within a 2D Image field .	
Call	jump_rel(dX, dY)	
Parameters	dX	Relative x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field , for example, for (enabled) Processing-on-the-fly applications.
	dY	Like dX (analogously).
Comments	The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, jump_rel is identical to jump_abs (see the comments there).	
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly.	
Version info	–	
References	jump_abs, jump_rel_3d, timed_jump_rel	

Normal List Command	jump_rel_3d
Function	Moves the output point (of the laser focus) along a 3D vector at jump speed from the current position to the specified position (relative coordinate values) within a 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then jump_rel_3d has the same effect as jump_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>jump_rel_3d(dX, dY, dZ)</code>
Parameters	dX Relative x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like dX (analogously).
	dZ Like dX (analogously), except Allowed value range: [-32,768...+32,767].
Comments	The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, jump_rel_3d is identical to jump_abs_3d (see the comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for dX and dY coordinates by 16. The allowed value range decreases accordingly. The value range for dZ is identical in RTC5 Standard Mode and RTC4 Compatibility Mode .
Version info	–
References	jump_abs_3d, jump_abs, jump_rel, timed_jump_rel_3d

Normal List Command	jump_rel_drill
Comments	This command is described, for example, in the manual for the intelliDRILL de II 20 Module.

Normal List Command	jump_rel_drill_2
Comments	This command is described, for example, in the manual for the intelliDRILL de II 20 Module.

Variable List Command	laser_on_list
Function	Turns on the Signals for “Laser Active” Operation for a specified time interval. $\geq$
Call	laser_on_list(Period)
Parameters	Period Time interval. In bits. As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $0 \leq \text{Period} \leq (2^{31}-1)$.
Comments	• Bit #31 of Period is ignored (see also Comments at laser_on_pulses_list). • The Signals for “Laser Active” Operation must first be selected with set_laser_mode, defined by further commands, and enabled by set_laser_control or enable_laser before they can be switched on with laser_on_list (see Chapter 7.4 “Laser Control”, Page 173). • While the laser control signals are turned on, the set position of the scanners is not changed. The next list command is executed when the programmed time interval has passed. • The currently set LaserOn Delay is applied at the beginning of the programmed time interval: The laser control signals turn on after a LaserOn Delay. – As with other Mark Commands (for example, mark_abs), the laser control signals do not explicitly turn off at the end of the time interval, but instead only turn off with a subsequent list command (for example, jump_abs or list_nop). The currently set LaserOff Delay is applied. • laser_on_list is useful for marking single dots (see Chapter 7.1.3 “Marking Single Dots”, Page 129). • Wobbel Mode (see Chapter 8.4 “Wobbel Mode”, Page 217) is retained but ignored. • For Period = 0 the laser control signals do not turn on; then laser_on_list is a short list command. • laser_on_list does not trigger a scanner delay. To wait until the laser (after a LaserOff Delay) is actually off before a jump, then explicitly a long_delay should be inserted.
RTC4→RTC5	Unchanged functionality. In addition: increased value range.
Version info	–
References	laser_on_pulses_list , long_delay , para_laser_on_pulses_list

Variable List Command	laser_on_pulses_list
Function	Turns on the LASERON signal for the specified number of external signal pulses, but for no longer than the specified time interval. For Pulses > 65,535 the function of laser_on_pulses_list is identical to laser_on_list .
Call	<code>laser_on_pulses_list(Period, Pulses)</code>
Parameters	Period Time interval. In bits. As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $0 \leq \text{Period} \leq (2^{32}-1)$.
	Pulses Number of external signal pulses. As an unsigned 32-bit value. Allowed value range: $0 \leq \text{Pulses} \leq 65,535$ or larger (see comments below).
Comments	• laser_on_pulses_list is useful for marking single dots in Laser Mode 6, but also effective in the other laser modes. • The external pulses must be supplied as TTL pulses at the LASER connector's DIGITAL IN1 digital input (see Section "2-Bit Digital Input Port", Page 61). By set_laser_control(Bit #5), you can specify whether signal pulses should be counted at rising or falling edges. • If Period = 0, then laser_on_pulses_list has no effect and laser_on_pulses_list becomes a short list command. • If $0 < \text{Period} \leq (2^{31}-1)$, then laser_on_pulses_list duration is always Period clocks (that is, Period $\times$ 10 μs), even if the specified number of external signal pulses expire in a shorter time interval. • If $2^{31} \leq \text{Period} \leq (2^{32}-1)$, laser_on_pulses_list maximum duration is (Period - 2^{31}) clocks (that is, (Period - 2^{31}) $\times$ 10 μs). Here, however, laser_on_pulses_list terminates as soon as the specified number of external signal pulses has been detected. • If Pulses > 65,535, then laser_on_pulses_list's function is identical to laser_on_list (external signal pulses are not taken into account, see comments there). Otherwise (for $0 \leq \text{Pulses} \leq 65,535$), laser_on_pulses_list (in contrast to laser_on_list) <i>do not</i> toggle the laser control signals between "laser active" and "laser standby" operation, but instead only switches the LASERON signal, whereby Laser Delays are not taken into account. Likewise during this command, unexpired Laser Delays activated by prior commands have no effect (though their effect resume when the LASERON signal switches off again after the final pulse or after Period).

Variable List Command	laser_on_pulses_list
Comments (cont'd)	• If $1 \leq \text{Pulses} \leq 65,535$, then the LASERON signal does switch on upon the edge (according the polarity set by set_laser_control(Bit #5)) of the first external pulse (unless it is already on due to an unexpired LaserOff Delay) and remains on for the specified number of pulses, but no longer than until the end of the number of $10 \mu\text{s}$ periods specified by Period. Users should ensure to define Period large enough for processing Pulses number of signal pulses in this time interval. If the DIGITAL IN1 input does not receive any pulses, then laser_on_pulses_list does not alter the LASERON signal. If more signal pulses than specified by Pulses are received during the time interval defined by Period, then the surplus pulses are ignored. • If Pulses = 0, then laser_on_pulses_list has no effect (the LASERON signal is not switched on). Then laser_on_pulses_list becomes a short list command. • The LASERON signal must first be defined and enabled by set_laser_control or enable_laser before it can be switched on with laser_on_pulses_list (see Chapter 7.4 "Laser Control", Page 173). • While laser_on_pulses_list is executed, the set position of the scanners is not changed. The next list command is executed when the programmed time interval Period has passed. • The Wobbel mode (see Chapter 8.4 "Wobbel Mode", Page 217) is retained but ignored. • Any softstarts that were defined are not executed.
RTC4→RTC5	New command.
Version info	–
References	laser_on_list , para_laser_on_pulses_list

Ctrl Command	laser_signal_off
Function	Turns off the Signals for “Laser Active” Operation immediately.
Call	<code>laser_signal_off()</code>
Comments	• laser_signal_off is intended for direct laser control in combination with laser_signal_on. • laser_signal_off is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set • laser_signal_off is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) – the INTERNAL-BUSY list execution status is set
RTC4→RTC5	Unchanged functionality.
Version info	–
References	laser_signal_on

Normal List Command	laser_signal_off_list
Function	Like laser_signal_off , but a list command.
Call	<code>laser_signal_off_list()</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	–

Ctrl Command	laser_signal_on
Function	Turns on the Signals for “Laser Active” Operation immediately.
Call	laser_signal_on()
Comments	• <i>Caution! Check the beam path before turning on the laser!</i> • The Signals for “Laser Active” Operation must first be selected with set_laser_mode, defined by further commands, and enabled by set_laser_control or enable_laser before they can be switched on with laser_signal_on (see Chapter 7.4 “Laser Control”, Page 173). • laser_signal_on is intended for turning on the laser directly, for example, for alignment purposes. • The Signals for “Laser Active” Operation must be turned off with laser_signal_off. • laser_signal_on is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • laser_signal_on is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set)
RTC4→RTC5	Unchanged functionality.
Version info	–
References	laser_signal_off

Normal List Command	laser_signal_on_list
Function	Like laser_signal_on , but a list command and never ignored.
Call	laser_signal_on_list()
RTC4→RTC5	Unchanged functionality.
Version info	–
References	laser_signal_on

Undelayed Short List Command	list_call
Function	Causes an unconditional jump to a subroutine that starts at the specified absolute address (in any desired location within list memory).
Call	<code>list_call(Pos)</code>
Parameters	Pos Absolute jump address $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	- The first command of a subroutine called by list_call is executed (possibly after a list_continue) immediately and without delay. Nested or recursive calls are also possible, up to a depth of 63, see also Chapter 6.5.1 "Subroutines", Page 101. - Each subroutine must be terminated by list_return so that after the subroutine (including the terminating list_return) has been processed, execution continues with the command that follows the subroutine-call command. This, too, executes (after a possible list_continue) immediately and without delay. If set_end_of_list is encountered instead of the expected list_return, then list execution terminates or – if previously activated – an automatic list change takes place (for the latter, the current list status is a decisive factor as described below). Under no circumstances does program flow then return again to the calling location, even in the case of nested subroutine calls. Any not-yet-completed mark_text does not execute to completion. If the end of a list memory area ("List 1" or "List 2") is reached without having encountered a list_return or set_end_of_list, then execution continues at the start of the current list. If such a situation occurs in the protected list memory area "List 3", then a compulsory list_return command is inserted and executed. - The list_call command is replaced by a list_nop if $Pos > (2^{20}-1)$ or if Pos is also the current address (get_last_error return code RTC5_PARAM_ERROR). - If a called subroutine executes a further list_call command to the address of the calling list_call command (recursive call), then the resulting endless loop is terminated as soon as the 63-nested-call upper limit is reached. Further list_call commands are then ignored and the next command is instead executed. - If the subroutine starts directly at the address which follows list_call, then the subroutine is executed once again after list_return (see also comments on a missing corresponding function call in the list_return command description). As of version OUT 540 the next processed command is the one which follows after list_return (see also list_repeat...list_until). This bypasses the possibly unwanted list processing.

Undelayed Short List Command	list_call
Comments (cont'd)	• The BUSY list status readable by read_status is, if necessary, altered by list_call if the called address is in the list area ("List 1" or "List 2"). If this address is instead in protected memory ("List 3"), then the calling location's list status is retained because the protected area does not have its own list status. During execution of a subroutine in protected memory, get_status too returns the calling location's List Execution Status, and get_out_pointer returns the position of the calling list_call command as the output pointer position. • Absolute Vector commands and "Arc" commands execute absolutely after list_call is called. If the subroutine needs to execute at various locations within the Image field, then either the subroutine can only contain relative [*]mark[*] Commands, "Arc" commands or Jump commands or list_call_abs must be used instead. • If list_call is at the last possible memory position in a list ("List 1" or "List 2"), then – even if automatic list changing has been previously enabled – execution continues at the start of the same list after processing of the called subroutine. • <code>list_call(Pos)</code> is synonymous with <code>list_call_repeat(Pos, 1)</code>.
RTC4→RTC5	Essentially unchanged functionality.
Version info	–
References	list_continue , list_repeat , list_return , list_until , list_call_abs , list_call_cond , list_call_repeat , sub_call

Undelayed Short List Command	list_call_abs
Function	Causes an unconditional jump to a subroutine that starts at the specified absolute list memory area address (in any desired area of list memory). In the called subroutine, any absolute Vector commands and "Arc" commands receive an offset (based on the current coordinates at the time of the call).
Call	<code>list_call_abs(Pos)</code>
Parameters	Pos Absolute jump address $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	- The list_call_abs command is basically identical to list_call (see the comments there). However, list_call_abs results in a different execution of absolute Vector commands and "Arc" commands within the called subroutine. When list_call_abs executes, an offset is established for the called subroutine and set according to the current position. Thereby, the current position is automatically considered when absolute Vector commands and "Arc" commands of the called subroutine are subsequently executed. Nested calls are taken into account when the offset is determined (with list_return, the previous offset values are re-established). Subroutines can thus contain absolute Vector commands and "Arc" commands even if they are intended to be repeated in different parts of the Image field. - If the called subroutine contains no absolute commands, then there is no difference between list_call_abs and list_call. <code>list_call_abs(Pos)</code> is synonymous with <code>list_call_abs_repeat(Pos, 1)</code>.
RTC4→RTC5	New command.
Version info	–
References	list_call , list_call_abs_cond , list_call_abs_repeat

Undelayed Short List Command	list_call_abs_cond
Function	<i>Conditional subroutine call (AbsCall):</i> list_call_abs_cond executes list_call_abs(Pos), if the current IOvalue at the EXTENSION 1 socket connector's 16-bit digital input port meets the following condition: $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, the directly following list command is immediately executed.
Call	<code>list_call_abs_cond(Mask1, Mask0, Pos)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the least significant 16 bits are evaluated.
	Mask0 See Mask1.
	Pos Absolute jump address $[0...(2^{20}-1)]$. As an unsigned 32-bit value.
Comments	• See list_call_abs. • See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	list_call_abs

Undelayed Short List Command	list_call_abs_repeat
Function	Causes an unconditional jump to a subroutine that starts at the specified absolute list memory area address (in any desired area of list memory) and executes its body several times.
Call	<code>list_call_abs_repeat(Pos, Number)</code>
Parameters	Pos Absolute jump address $[0...(2^{20}-1)]$. As with list_call_abs: As an unsigned 32-bit value.
	Number Number of repetitions. As an unsigned 32-bit value. Number = 0 is treated as Number = 1.
Comments	• list_call_abs(Pos) is synonymous with <code>list_call_abs_repeat(Pos, 1)</code>. • list_call_abs_repeat avoids an empty cycle at the repetition, which otherwise inevitably occurs with <code>list_call_abs...list_call_abs</code> or <code>list_repeat...list_call_abs...list_until</code> constructions. • See list_call_repeat and list_call_abs.
RTC4→RTC5	New command.
Version info	Available as of DLL 540, OUT 540.
References	list_call , list_call_abs , list_call_abs_cond , list_call_repeat , list_repeat , list_until

Undelayed Short List Command	list_call_cond
Function	<i>Conditional subroutine call:</i> list_call_abs_cond executes list_call(Pos), if the current IOvalue at the EXTENSION 1 socket connector's 16-bit digital input port meets the following condition: $((\text{IOvalue AND Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, the directly following list command is immediately executed.
Call	list_call_cond (Mask1, Mask0, Pos)
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1.
	Pos Absolute jump address $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	• See list_call. • See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	Essentially unchanged functionality (see list_call).
Version info	–
References	list_call

Undelayed Short List Command	list_call_repeat
Function	Causes an unconditional jump to a subroutine that starts at the specified absolute list memory area address (in any desired area of list memory) and executes its body several times.
Call	<code>list_call_repeat(Pos, Number)</code>
Parameters	Pos Absolute jump address $[0...(2^{20}-1)]$ (as with list_call). As an unsigned 32-bit value. Number Number of repetitions. As an unsigned 32-bit value. Number = 0 is treated as Number = 1.
Comments	• <code>list_call(Pos)</code> is synonymous with <code>list_call_repeat(Pos, 1)</code>. • list_call_repeat avoids an empty cycle at the repetition, which otherwise inevitably occurs with <code>list_call...list_call</code> or <code>list_repeat...list_call...list_until</code> constructions. • With list_call_repeat, for example, trajectories (see Glossary entry on page 26) from micro_vector[*] commands for runup curves and coast down curves can be seamlessly joined together with shapes from subroutines. • An empty cycle must be inserted if at list_call_repeat another subroutine call follows immediately (nested calls). This can be avoided, if there is for example, at least one micro_vector[*] command between two subroutine calls. • The subroutine start Pos can be located in any part of the list memory area. This applies in particular directly after list_call_repeat. Thus the complete list memory can continuously be used for list commands without reserving a special area for protected subroutines. • After a list_return the next processed command is the one which follows directly after list_call_repeat. However, if the subroutine start follows directly after list_call_repeat, then the next processed command is the one which follows directly after list_return.
RTC4→RTC5	New command.
Version info	Available as of DLL 540, OUT 540.
References	list_call , list_call_abs , list_call_abs_repeat , list_repeat , list_return , list_until , micro_vector_abs , micro_vector_abs_3d , micro_vector_rel , micro_vector_rel_3d

Normal List Command	list_continue
Function	Inserts a null operation (no operation) into the list memory area.
Call	<code>list_continue()</code>
Comments	• Execution of list_continue (as does list_nop) requires 10 μs. • When list_continue immediately follows a short list command, it ensures (as does list_nop) that the subsequent list command only executes in the next 10 μs clock cycle (the short list command's "effective" execution time is then 10 μs). Unlike list_nop, however, list_continue neither modifies laser signals nor pauses for Scanner Delays. In contrast to list_nop, therefore, list_continue allows postponing short list commands to the next 10 μs clock cycle without interrupting a Polyline by switching off the laser. • With exceedance of the maximum allowed number of directly consecutive short list commands, an empty cycle (which exactly corresponds list_continue) is automatically inserted. • You can also use list_continue to separate from each other several outputs to the same output port. Without list_continue, for example, multiple directly consecutive write_da_x_list commands (short list commands) would occur in a single 10 μs clock cycle, whereby some values to the analog output port would never get outputted. • The RTC5 never uses list_continue as a placeholder for other (rejected) list commands, but instead always uses list_nop.
RTC4→RTC5	New command.
Version info	–
References	list_nop

Undelayed Short List Command	list_jump_cond
Function	<i>Conditional absolute list jump:</i> list_jump_cond executes list_jump_pos(<i>Pos</i>), if the current <i>IOvalue</i> at the EXTENSION 1 socket connector's 16-bit digital input port meets the following condition: $((\text{IOvalue} \text{ AND } \text{Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND } \text{Mask0}) = \text{Mask0})$ (= if the bits specified in <i>Mask1</i> are 1 and the bits specified in <i>Mask0</i> are 0). Otherwise, the directly following list command is immediately executed.
Call	<code>list_jump_cond(Mask1, Mask0, Pos)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1.
	Pos Absolute jump address $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	list_jump_cond is synonymous with list_jump_pos_cond (see comments there).
RTC4→RTC5	Basically unchanged functionality. However: Jumps into or out from the protected list memory area "List 3" are not allowed (illegal Jump commands are ignored during processing, see also list_jump_pos).
Version info	–
References	list_jump_pos , list_jump_pos_cond

Undelayed Short List Command	list_jump_pos
Function	Execution produces an unconditional jump to the specified list-memory address. The next command there is executed immediately without delay.
Call	<code>list_jump_pos(Pos)</code>
Parameters	Pos Absolute jump address $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	• For Pos, an absolute address can be specified within the configured list memory ("List 1" and "List 2"), but not within the protected "List 3" area or completely outside of the list memory area. • Jumps into or out from the protected list memory area "List 3" are not allowed. • Illegal Jump commands are transmitted unaltered to the RTC5, but are ignored during processing. Instead, the next command is executed. Hence, the user program does probably no longer perform as expected. Therefore, Jump commands must be carefully programmed (see also Chapter 6.5.3 "Jumps", Page 109). • Jump commands initiating a jump to themselves (Pos = list position of the Jump command) are also ignored at runtime to prevent an infinite loop that excludes further activities (and that could only be halted by stop_execution or an External Stop). • Decisive are the runtime conditions. When reconfiguring list memory or converting a subroutine, an originally legal jump address might become illegal due to new list boundaries or a relocated subroutine storage position. • After a jump to another list area, the status information might under some circumstances not be correct (see also Chapter 6.4.2 "List Status", Page 96). • The BUSY list status of the two lists is alternatingly set by list_jump_pos, the USED list status of the two lists remains unchanged (see read_status). • The list_jump_pos command is synonymous with set_list_jump.
RTC4→RTC5	New command.
Version info	–
References	set_list_jump , list_jump_rel , list_jump_pos_cond

Undelayed Short List Command	list_jump_pos_cond
Function	<i>Conditional absolute list jump:</i> list_jump_pos_cond executes list_jump_pos(<i>Pos</i>), if the current <i>IOvalue</i> at the EXTENSION 1 socket connector's 16-bit digital input port meets the following condition: $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits specified in <i>Mask1</i> are 1 and the bits specified in <i>Mask0</i> are 0). Otherwise, the directly following list command is immediately executed.
Call	<code>list_jump_pos_cond(Mask1, Mask0, Pos)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1.
	Pos Absolute jump address $[0 \dots (2^{20}-1)]$. As an unsigned 32-bit value.
Comments	• See list_jump_pos. • Unlike the rules for preventing endless loops (see list_jump_pos), jumps by list_jump_pos_cond are allowed even if they are to their own address (<i>Pos</i> = list position of the Jump command), for example, to wait for confirmation of a signal. • list_jump_pos_cond is synonymous with list_jump_cond. • See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
Examples (Pascal)	• wait until Bit #3 of the input port turns HIGH (= loop while the bit is <i>LOW</i>): <code>list_jump_pos_cond(0, \$0008, get_input_pointer);</code> • skip the next two list commands if the state of the input port is xxxx xxxx xxxx 0110: <code>list_jump_pos_cond(6, 9, get_input_pointer + 3);</code> • See also Chapter "Example Code (Pascal)", Page 272.
RTC4→RTC5	New command.
Version info	–
References	list_jump_pos

Undelayed Short List Command	list_jump_rel
Function	Execution produces an unconditional jump to the specified address within the current list. The next command there is executed immediately without delay.
Call	<code>list_jump_rel(Pos)</code>
Parameters	Pos Jump distance $[(-2^{20}+1) \dots (2^{20}-1)]$. As a signed 32-bit value.
Comments	• The list_jump_rel command enables implementation of branching (for example, “if-then-else”) independently of the command’s list position, in particular also coded independently of the list number because of relative addressing. • The current list input pointer can be queried by get_list_pointer. • list_jump_rel is usable in all list areas, including the protected list memory area “List 3”. • When specifying a jump distance within “List 1” or “List 2” or for non-indexed subroutines within “List 3” be sure that the jump does not exceed the boundaries of the corresponding memory area. • If list_jump_rel is used in an indexed subroutine or character set, then also be sure that the jump does not exceed the boundaries of the subroutine or character set. • Illegal Jump commands are transmitted unaltered to the RTC5, but are ignored during processing. Instead, the next command is executed. Hence, the user program does probably no longer perform as expected. Therefore, Jump commands must be carefully programmed (see also Chapter 6.5.3 “Jumps”, Page 109). • Jump commands initiating a jump to themselves ($Pos = 0$) is also ignored at runtime to prevent an infinite loop that excludes further activities (and that could only be halted by stop_execution or an External Stop). • Decisive are the runtime conditions. When reconfiguring list memory or converting a subroutine, an originally legal jump address might become illegal due to new list boundaries or a relocated subroutine storage position.
RTC4→RTC5	New command.
Version info	–
References	list_jump_pos, list_jump_rel_cond

Undelayed Short List Command	list_jump_rel_cond
Function	<i>Conditional relative list jump:</i> list_jump_rel_cond executes list_jump_rel(Pos), if the current IOvalue at the EXTENSION 1 socket connector's 16-bit digital input port meets the following condition: $((\text{IOvalue} \text{ AND } \text{Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND } \text{Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, the directly following list command is immediately executed.
Call	<code>list_jump_rel_cond(Mask1, Mask0, Pos)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1.
	Pos Jump distance $[(-2^{20}+1) \dots (2^{20}-1)]$. As a signed 32-bit value.
Comments	• See list_jump_rel. • Unlike the rules for preventing endless loops (see list_jump_rel), jumps by list_jump_pos_cond are allowed even if they are to their own address (Pos = 0), for example, to wait for confirmation of a signal. • See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
Examples (Pascal)	• Wait until Bit #3 of the input port turns HIGH (= loop while the bit is LOW): <code>list_jump_rel_cond(0, \$0008, 0);</code> • Skip the next two list commands if the state of the input port is xxxx xxxx xxxx 0110: <code>list_jump_rel_cond(6, 9, 3);</code> • See also Section "Example Code (Pascal)", Page 272.
RTC4→RTC5	New command.
Version info	–
References	list_jump_rel

Undelayed Short List Command	list_next
Function	Executes the next list command.
Call	<code>list_next()</code>
Comments	• list_next reads the next list command and executes it immediately, as long as the maximum allowed number of short list commands has not been exceeded. Otherwise, it is executed within the next 10 μs clock cycle. • list_next is a proper place holder for another command in the list, because it prevents an extra 10 μs clock cycle, which is unavoidable with other place holders such as list_nop or list_continue.
RTC4→RTC5	New command.
Version info	Available as of DLL 541, OUT 541.
References	list_nop , list_continue

Normal List Command	list_nop
Function	Inserts a null operation (no operation) into the list memory area.
Call	<code>list_nop()</code>
Comments	• list_nop has the same effect as long_delay(1). It switches off the Signals for "Laser Active" Operation after a LaserOff Delay and waits for a possible scanner delay. Even if no delays need to be waited for, execution of list_nop requires 10 μs. • list_nop serves as a placeholder for rejected list commands (get_last_error return code RTC5_IGNORED). • When list_nop immediately follows a short list command, it ensures that the subsequent list command only executes in the next 10 μs clock cycle (the short list command's "effective" execution time is then 10 μs).
RTC4→RTC5	The RTC4 command is a pure placeholder and is more like list_continue . The RTC5 command switches the Signals for "Laser Active" Operation off and waits for a scanner delay.
Version info	–
References	long_delay , list_continue

Undelayed Short List Command	list_repeat
Function	Initiates repetition of a group of list commands.
Call	<code>list_repeat()</code>
Comments	• See Chapter 6.5.5 "Loops", Page 111. • Any still-pending delayed short list command executes first. • All list commands between list_repeat and the next list_until call is possibly repeated multiple times. • Each executed list_repeat...list_until loop requires a full 10 μs clock cycle. Accordingly, the first list command after a repetition is executed with a 10 μs delay. • Nesting up to 8 list_repeat...list_until loops deep is allowed. Each further list_repeat call beyond that limit is ignored as long as no list_until call has terminated a loop. • If a list terminates (by set_end_of_list or stop_execution or /STOP), then all not-yet-fully-processed list_repeat...list_until loops are deleted. However, this does not occur if an automatic list change (by auto_change, auto_change_pos or start_loop) is active. • Complete list_repeat...list_until loops can reside within a list or subroutine. Changing between lists and subroutines or between different subroutines is not permitted. Changing between lists is permitted as long as the list change occurs without explicit list termination (by an automatic list change or by an explicit list jump to another list with list_jump_pos). • List jumps into a list_repeat...list_until loop body or from inside a loop to a loop-external location should be avoided. Careless use could compromise loop management integrity so severely that loops do not execute as expected. But subroutine calls from inside a loop are always reliably possible.
RTC4→RTC5	New command.
Version info	–
References	list_until

Undelayed Short List Command	list_return
Function	Terminates a previously called subroutine, jumps to the calling location and executes the immediately following list command (possibly after a list_nop) immediately and without delay.
Call	<code>list_return()</code>
Comments	• While loading an indexed subroutine, character or text string in the protected list memory area "List 3", list_return additionally triggers entries into the corresponding internal management table of data such as the starting address. • In contrast, if list_return directly follows a load_sub, load_char or load_text_table command, then the affected subroutine, character or text string are deleted from the corresponding internal management table. • If, during loading into the protected list memory area "List 3", indexed subroutines, characters or text strings are <i>not</i> terminated with list_return before the input pointer is explicitly set to another position, then they are not stored and are thereafter unavailable. • During loading of list_return, a flush of the buffered list input is triggered, see Chapter 6.4.1 "Loading Lists", Page 94. • If list_return follows load_sub, load_char or load_text_table, then the input pointer after list_return is invalid. That is, further commands can not be loaded without a prior request for a new input pointer positioning. • If, during processing, a list_return is encountered without the prior explicit calling of a subroutine, character or text string, then processing continues at the absolute address 0 (starting address of "List 1"). With nested calls the integrity of the subroutine structure is destroyed.
RTC4→RTC5	No changes to the previous functionality (termination of a subroutine). New: impact during loading into the protected list memory area "List 3".
Version info	–
References	list_call , sub_call , load_char , load_text_table , load_sub

Undelayed Short List Command	list_until
Function	Terminates repetition of a group of list commands.
Call	<code>list_until(Number)</code>
Parameters	Number Number of repetitions. As an unsigned 32-bit value. Number = 0 is treated like Number = 1.
Comments	• See Chapter 6.5.5 “Loops”, Page 111. • Any still-pending delayed short list command executes first. • All list commands between this command and the most recent preceding list_repeat command is executed <code>Number</code> number of times. • Nesting up to 8 list_repeat/list_until loops deep is allowed. Each further list_until command for which there has been no preceding list_repeat command is ignored. • An empty loop consisting of list_repeat directly followed by list_until terminates immediately and is not repeated. • If a list terminates (by set_end_of_list or stop_execution or /STOP), then all not-yet-fully-processed list_repeat/list_until loops are deleted. However, this does not occur if an automatic list change (by auto_change, auto_change_pos or start_loop) is active. • Complete list_repeat/list_until loops can reside within a list or subroutine. Changing between lists and subroutines or between different subroutines is not permitted. Changing between lists is permitted as long as the list change occurs without explicit list termination (by an automatic list change or by an explicit list jump to another list with list_jump_pos). • List jumps into a list_repeat/list_until loop’s body or from inside a loop to a loop-external location should be avoided. Careless use could compromise loop management integrity so severely that loops do not execute as expected. But subroutine calls from inside a loop are always reliably possible.
RTC4→RTC5	New command.
Version info	–
References	list_repeat

Ctrl Command	load_auto_laser_control																				
Function	Loads a table with data points from an ASCII text file. Determines – by linear interpolation – the nonlinearity curve (see Section “Loading and Determining the Nonlinearity Curve”, Page 192) for “Automatic Laser Control” (except Vector-Defined Laser Control), see Chapter 7.4.9 “Automatic Laser Control” , Page 187.																				
Call	NoOfDataPoints = load_auto_laser_control(Name, No)																				
Parameters	Name Name of the text file or NULL . The text file may contain one or more tables. As a pointer to a null-terminated ANSI string.																				
	No Determines which table in the text file is to be loaded. The parameter corresponds to the extension <No> of [AutoLaserCtrlTable<No>] at the beginning of the desired table. As an unsigned 32-bit value.																				
Result	A positive error code in case of an error. The negative number of found data points in case of success. As a signed 32-bit value. <table> <thead> <tr> <th>Value</th><th>Description</th></tr> </thead> <tbody> <tr> <td>-1...- 50</td><td>Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored, see also Section “Loading and Determining the Nonlinearity Curve”, Page 192.</td></tr> <tr> <td>-1024</td><td>For Name = NULL (see also comments).</td></tr> <tr> <td>1</td><td>No valid data points found (though Table No found).</td></tr> <tr> <td>3</td><td>File not found.</td></tr> <tr> <td>4</td><td>DSP memory error.</td></tr> <tr> <td>5</td><td>BUSY error, board has been BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).</td></tr> <tr> <td>8</td><td>Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).</td></tr> <tr> <td>11</td><td>PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).</td></tr> <tr> <td>13</td><td>The specified table number could not been found in the file.</td></tr> </tbody> </table>	Value	Description	-1...- 50	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored, see also Section “Loading and Determining the Nonlinearity Curve”, Page 192 .	-1024	For Name = NULL (see also comments).	1	No valid data points found (though Table No found).	3	File not found.	4	DSP memory error.	5	BUSY error, board has been BUSY or INTERNAL-BUSY list execution status , no download (get_last_error return code RTC5_BUSY).	8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).	11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).	13	The specified table number could not been found in the file.
Value	Description																				
-1...- 50	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored, see also Section “Loading and Determining the Nonlinearity Curve”, Page 192 .																				
-1024	For Name = NULL (see also comments).																				
1	No valid data points found (though Table No found).																				
3	File not found.																				
4	DSP memory error.																				
5	BUSY error, board has been BUSY or INTERNAL-BUSY list execution status , no download (get_last_error return code RTC5_BUSY).																				
8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).																				
11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).																				
13	The specified table number could not been found in the file.																				
Comments	- The format requirements for the text file’s table entries with data points for the nonlinearity curve are described in Section “Loading and Determining the Nonlinearity Curve”, Page 192. When loading the table, the RTC5 determines suitable values for the entire range of percent values. load_auto_laser_control overwrites any previously loaded nonlinearity curve. - For Name = NULL (as during initialization by load_program_file), the function Scale(Percent)=1.0 is loaded for the complete percent range (no nonlinearity).																				

Ctrl Command	load_auto_laser_control
Comments (cont'd)	• load_auto_laser_control is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • load_auto_laser_control is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During the runtime of load_auto_laser_control, External Starts are suppressed. • Before loading a table, load_auto_laser_control performs a DSP memory check that produces error code 4 in case of error.
RTC4→RTC5	New command.
Version info	–
References	set_auto_laser_control

Ctrl Command	load_char
Function	Assigns a desired index to a character defined by subsequent list commands, and loads the character into the protected list memory area "List 3".
Call	<code>load_char(Char)</code>
Parameters	Char Index of the indexed character. As an unsigned 32-bit value. Allowed value range: [0...1023].
Comments	- Up to 1024 indexed characters, hence 4 character sets with 256 indexed characters per set, can be stored. For defining character sets, the following applies: $\text{Char} = \text{character set number} \times 256 + \text{ASCII number of the character}$ (character sets are numbered 0 to 3). If Char > 1023 then load_char is ignored (get_last_error return code RTCS_PARAM_ERROR). - The addresses in the protected list memory area "List 3" where the character definitions are to be stored are automatically determined and internally managed. This management is independent of that for indexed subroutines (see load_sub) and text string definitions (see load_text_table). - Indexed character definitions must be terminated with a list_return call. This is a prerequisite for actual storage of the commands, entry of the start address into the internal management table, and initiating a flush of the buffered list input, see Chapter 6.4.1 "Loading Lists", Page 94. Otherwise (the input pointer is altered without a preceding list_return), the character definition with this index is not available. - An indexed character definition is not stored if the protected list memory area "List 3" has not been previously configured for a sufficient size beyond "List 1" and "List 2". - If list_return is the next command after load_char, then the corresponding character definition is deleted from the internal management table. - Observe all notes in Chapter 6.5.2 "Character Sets and Text Strings", Page 107.
RTC4→RTC5	New command.
Version info	–
References	list_return , load_sub , load_text_table

Ctrl Command	load_correction_file																				
Function	Loads the specified correction file into RTC5 memory. Then calls automatically select_cor_table with the most recently used parameter values (or the default parameter values).																				
Call	ErrorNo = load_correction_file(Name, No, Dim)																				
Parameters	Name Name of the correction file. As a pointer to a null-terminated ANSI string.																				
	No Number of the correction table. Allowed values: [1...4]. As an unsigned 32-bit value. See also number_of_correction_tables.																				
	Dim Determines whether the correction file content is stored as a 2D correction table or (if possible) as a 3D correction table (see also comments). As an unsigned 32-bit value. = 2: 2D correction files and 3D correction files are stored as a 2D correction table (downgrade, if necessary). = 3: If the Option "3D" is enabled, 2D correction files and 3D correction files are stored as a 3D correction table (upgrade, if necessary). If the Option "3D" is <i>not</i> enabled, like Dim = 2. Others: Dim has no effect. A 2D correction file is stored as a 2D correction table. A 3D correction file is stored as a 3D correction table, if the Option "3D" is enabled, otherwise as a 2D correction table (downgrade).																				
Result	Error code. As an unsigned 32-bit value. <table> <thead> <tr> <th>Value</th><th>Description</th></tr> </thead> <tbody> <tr> <td>0</td><td>Success.</td></tr> <tr> <td>1</td><td>File error (file corrupt or incomplete).</td></tr> <tr> <td>2</td><td>Memory error (RTC5 DLL-internal, Windows system memory).</td></tr> <tr> <td>3</td><td>File-open error (empty string submitted for Name parameter, file not found, etc.).</td></tr> <tr> <td>4</td><td>DSP memory error.</td></tr> <tr> <td>5</td><td>PCI download error (RTC5 board driver error).</td></tr> <tr> <td>8</td><td>RTC5 board driver not found (get_last_error return code RTC5_ACCESS_DENIED).</td></tr> <tr> <td>10</td><td>Parameter error (incorrect No).</td></tr> <tr> <td>11</td><td>Access error: board reserved for another user program (get_last_error return code RTC5_ACCESS_DENIED) or version check has detected an error (RTC5OUT.out or RTC5RBF.rbf version not compatible to the current RTC5 DLL version, get_last_error return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH).</td></tr> </tbody> </table>	Value	Description	0	Success.	1	File error (file corrupt or incomplete).	2	Memory error (RTC5 DLL -internal, Windows system memory).	3	File-open error (empty string submitted for Name parameter, file not found, etc.).	4	DSP memory error.	5	PCI download error (RTC5 board driver error).	8	RTC5 board driver not found (get_last_error return code RTC5_ACCESS_DENIED).	10	Parameter error (incorrect No).	11	Access error: board reserved for another user program (get_last_error return code RTC5_ACCESS_DENIED) or version check has detected an error (RTC5OUT.out or RTC5RBF.rbf version not compatible to the current RTC5 DLL version, get_last_error return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH).
Value	Description																				
0	Success.																				
1	File error (file corrupt or incomplete).																				
2	Memory error (RTC5 DLL -internal, Windows system memory).																				
3	File-open error (empty string submitted for Name parameter, file not found, etc.).																				
4	DSP memory error.																				
5	PCI download error (RTC5 board driver error).																				
8	RTC5 board driver not found (get_last_error return code RTC5_ACCESS_DENIED).																				
10	Parameter error (incorrect No).																				
11	Access error: board reserved for another user program (get_last_error return code RTC5_ACCESS_DENIED) or version check has detected an error (RTC5OUT.out or RTC5RBF.rbf version not compatible to the current RTC5 DLL version, get_last_error return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH).																				

Ctrl Command	load_correction_file
	12 Warning: 3D correction table or Dim = 3 selected, but the Option "3D" is not enabled. The system subsequently operates as an ordinary 2D system (this warning is only returned, if no other error has occurred). 13 Busy error: no download, board is BUSY or INTERNAL-BUSY list execution status (get_last_error return code RTC5_BUSY). 14 PCI upload error (RTC5 board driver error, only with download verification). 15 Verify error (only with download verification).
Comments	Notes on loading correction tables: • The RTC5 can store 4 (on request: 8) different correction tables at the same time, for example, for use in a multiple scan head configuration. • Correction tables number 3 and 4 must always be loaded only after load_program_file. These tables occupy the upper half of the memory area reserved for data recording by set_trigger or set_trigger4 (see comments for those commands). load_program_file automatically initializes this memory area and any already loaded correction tables number 3 and 4 are lost. • Before storing a correction file, load_correction_file performs a DSP memory check. In case of an error, error code 4 is returned. • If the parameter No is out of range, then no correction table is loaded (return value 10). Users can further restrict the permitted range by number_of_correction_tables. • The name of the to-be-loaded correction file must be passed to load_correction_file. As a pointer to a null-terminated ANSI string. If Name is passed as a NULL pointer, the corresponding table is replaced by a 1-to-1 table (for Dim = 3 and enabled Option "3D" with a 1-to-1 3D correction table, otherwise with a 1-to-1 2D table). An empty string ("") for Name result in an error return code of 3. • If the Option "3D" is <i>not</i> enabled (default), then 2D correction files and 3D correction files are stored as 2D correction tables – regardless of the value specified for Dim (3D correction files also include 2D correction tables). The 3D data sections of 3D correction files are then ignored. • If, on the other hand, the Option "3D" is <i>enabled</i>, then both 2D and 3D correction tables can be loaded. – For Dim = 2, a 2D table is always stored (the 3D data section of 3D correction files are ignored) and, accordingly, only 2D corrections are calculated (the z axis thereby remains unchanged, as if no Option "3D" has been enabled). – For Dim = 3, if the Option "3D" is enabled then both 2D correction files and 3D correction files are stored as 3D correction tables. 2D correction tables are thereby automatically expanded to incorporate a linear Z correction. The actually suitable Z correction can subsequently be loaded by the load_z_table command (see page 159). – All other values for Dim do not change the type of the correction file.

Ctrl Command	load_correction_file
Comments (cont'd)	Notes on assigning correction tables by select_cor_table: • Use the select_cor_table or select_cor_table_list command to assign one (or two) correction table(s) stored on the RTC5 to the scan head (or to both scan head connectors). • load_correction_file automatically calls select_cor_table after saving a correction table. However, if you call load_correction_file <i>before</i> loading the program file (by load_program_file), then the automatic call of select_cor_table has no effect. If you call load_correction_file <i>after</i> load_program_file, then the select_cor_table call uses the parameter values (HeadA = 1, HeadB = 0) or the values most recently used (after load_program_file) when having called select_cor_table. load_correction_file does not return to the user program until the select_cor_table-induced jump to the corrected galvanometer scanner position has been completed. See also Notes, Page 165. Other notes: • RTC5 correction tables contain parameters that formerly (with an RTC4) had to be manually copied into a user program from a supplied *_ReadMe.txt file. To directly integrate these parameters into the user program, the RTC5 can load them by get_table_para from the currently loaded correction table or read them by get_head_para from the assigned correction table. • During the runtime of load_correction_file: – No other control commands are executed – External Starts are suppressed • load_correction_file is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • load_correction_file is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • If an RTC5_VERSION_MISMATCH error occurs (return value 11), a RTC5 DLL version appropriate for the program file must be chosen and the board must be made currentless (to unload the program software) or alternatively program files appropriate for the RTC5 DLL version must be reloaded by the multi-board command n_load_program_file (after RTC5_ACCESS_DENIED, the single-board command load_program_file does not get access rights for the board). Only afterward (and after the board has been accessed by acquire_rtc or select_rtc and possibly specified as default board by select_rtc) load_correction_file can be normally executed again.

Ctrl Command	load_correction_file
RTC4→RTC5	Except for the basic function and the first two parameters (loading of correction files), the functionality has substantially changed: • load_correction_file now uses the new parameter <code>Dim</code>. This allows, for instance, storage of 3D correction files as 2D correction tables and storage of 2D correction files as 3D correction tables. • Now, at least two 3D correction tables can be stored in RTC5 memory (with Option "3D"). However, two 3D correction tables can <i>not</i> be simultaneously assigned to the scan head connectors (see select_cor_table). • The parameters <code>k</code>, <code>Phi</code> and <code>Offset</code> no longer exist. Therefore, table transformations (scaling, rotation, translation) are no longer possible during loading of the correction file. Such coordinate transformations can now be specified by set_scale, set_angle and set_offset in addition to set_matrix. • select_cor_table is automatically called, see comments above and Notes, Page 165. • load_correction_file does not return to the user program until the correction motion has been completed (see comments above).
Version info	–
References	number_of_correction_tables

Ctrl Command	load_disk
Function	Loads into the protected list memory area "List 3" the indexed characters, text strings and subroutines previously stored in a binary file by save_disk and returns the number of actually loaded list commands.
Call	<code>NoOfLoadedCommands = load_disk(Name, Mode)</code>
Parameters	Name File name or NULL . As a pointer to a null-terminated ANSI string.
	Mode Determines how the loading procedure is executed. As an unsigned 32-bit value. = 0: The internal management tables for indexed characters, text strings and subroutines are initialized (all old references are thereby lost) and the input pointer is set to the beginning of "List 3" (the resulting loading process overwrites list commands stored there). > 0: Internal management tables are not initialized (all old references are initially retained, but then replaced or supplemented by other references during the loading process, depending on the file's content) and the input pointer is set to the position after the last stored (by load_char, load_text_table or load_sub) indexed character, text string or subroutine (the resulting loading process does <i>not</i> overwrite the list commands of old indexed characters, text strings and subroutines).
Result	The number of commands loaded by load_disk . As an unsigned 32-bit value.
Comments	- For Name = NULL, only the internal management tables are initialized as with Mode = 0 (no loading occurs, no "empty" binary file must be provided). Otherwise, the load_disk command can only be used for reading files that were stored with save_disk. - For all indexed characters, text strings and subroutines in the specified file, load_disk executes a corresponding load_char, load_text_table or load_sub command. The pertaining list commands are thereby written to the protected list memory area "List 3" and the entries are made in the internal management tables in accordance with the index assignment stored in the file. If there is no character, text string or subroutine for a particular index in the specified file, then no list commands are loaded for this index, and if Mode > 0 then any already existing entries in the internal management table remains unaltered. Thus, supplementary loading of indexed characters, text strings and subroutines from various files is possible.

Ctrl Command	load_disk
Comments (cont'd)	• Together with the save_disk command, load_disk can be used, for example, to defragment the protected list memory area "List 3" and to subsequently protect subroutines (see Section "Subsequent Protection and Conversion of Non-Indexed Subroutines", Page 105 and Section "Index Management and Defragmentation", Page 104). If the characters, text strings and subroutines come from various files, then defragmentation can be achieved if the first file is loaded in <code>Mode = 0</code> and subsequent files are loaded with <code>Mode > 0</code>, provided that no indices are used simultaneously in different files. Memory gaps in the protected area caused by dereferencing of indexed characters, text strings or subroutines are thereby closed. • If, during loading, the end of the protected list memory area "List 3" is reached before the end of the file (EOF), then all further list commands in the file is ignored. Likewise, incomplete characters, text strings and subroutines (as with individual load_char, load_text_table or load_sub commands) are not stored. Before each load_disk command, be sure that the memory configuration provides a sufficiently large protected area above the list areas ("List 1" and "List 2"). save_disk returns the number of list commands stored in the file. If a board changes ownership, it is the user's responsibility to ensure that the memory configuration data is consistent (Mem1 and Mem2 are queried from the DLL, not from the board, see also get_config_list and Chapter 6.7.1 "Board Acquisition by a User Program", Page 117). • If the specified file is corrupt and cannot be read to the end, then only the characters, text strings and subroutines fully readable to that point are stored. • load_disk is not executed, if: – "List 3" has no memory area assigned, that is, $\text{Mem1} + \text{Mem2} = 2^{20}$ (get_last_error return code RTC5_PARAM_ERROR) – the BUSY list execution status is set (get_last_error return code RTC5_BUSY) – the INTERNAL-BUSY list execution status is set (get_last_error-Returncode RTC5_BUSY) • load_disk is even executed, if: – a list has only been paused by set_wait (PAUSED list execution status set). But users are responsible for ensuring that no still-needed commands are overwritten, for example, if set_wait has been called from an indexed subroutine. • During the runtime of load_disk: – No other commands can be executed – External Starts are suppressed
RTC4→RTC5	New command.
Version info	–
References	save_disk

Ctrl Command	load_fly_2d_table																							
Function	Loads a 2D table from an ASCII text file for a set_fly_2d -Processing-on-the-fly application with 2D encoder compensation for xy positioning stages, see Section “2D Encoder Compensation for xy Positioning Stages”, Page 234 .																							
Call	NoOfDataPoints = load_fly_2d_table(Name, No)																							
Parameters	Name	Name of the text file or NULL . The text file may contain one or more tables. As a pointer to a null-terminated ANSI string.																						
	No	Determines which table in the text file is to be loaded. The parameter corresponds to the extension <No> of [Fly2DTable<No>] at the beginning of the desired table. As an unsigned 32-bit value.																						
Result	A positive error code in case of an error. The negative number of found data points in case of success. As a signed 32-bit value. <table><tr><th>Value</th><th>Description</th></tr><tr><td>< 0</td><td>Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see page 235).</td></tr><tr><td>0</td><td>For Name = NULL (see comments).</td></tr><tr><td>1</td><td>No valid data points found (though Table No found).</td></tr><tr><td>2</td><td>Out of Memory (not enough Windows system memory).</td></tr><tr><td>3</td><td>File not found.</td></tr><tr><td>4</td><td>DSP memory error.</td></tr><tr><td>5</td><td>BUSY error, board is BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).</td></tr><tr><td>8</td><td>Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).</td></tr><tr><td>11</td><td>PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).</td></tr><tr><td>13</td><td>The specified table number could not be found in the file.</td></tr></table>		Value	Description	< 0	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see page 235).	0	For Name = NULL (see comments).	1	No valid data points found (though Table No found).	2	Out of Memory (not enough Windows system memory).	3	File not found.	4	DSP memory error.	5	BUSY error, board is BUSY or INTERNAL-BUSY list execution status , no download (get_last_error return code RTC5_BUSY).	8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).	11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).	13	The specified table number could not be found in the file.
Value	Description																							
< 0	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see page 235).																							
0	For Name = NULL (see comments).																							
1	No valid data points found (though Table No found).																							
2	Out of Memory (not enough Windows system memory).																							
3	File not found.																							
4	DSP memory error.																							
5	BUSY error, board is BUSY or INTERNAL-BUSY list execution status , no download (get_last_error return code RTC5_BUSY).																							
8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).																							
11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).																							
13	The specified table number could not be found in the file.																							
Comments	- The text file’s data format requirements for reference points of the 2D encoder compensation table are described in Section “For the 2D compensation tables, the following rules apply:”, Page 235. The largest of these reference points should not exceed the range -524288...+524,287 (otherwise precision may be lost). During runtime, the current encoder values (including reference values) must not exceed the largest values specified in the table. Otherwise, clipping occurs. load_fly_2d_table overwrites any previously loaded table for 2D encoder compensation. - If Name = NULL, then a 0-correction table for 2D encoder compensation is loaded.																							

Ctrl Command	load_fly_2d_table
Comments (cont'd)	• load_fly_2d_table is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • load_fly_2d_table is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During the runtime of load_fly_2d_table, External Starts are suppressed. • Before loading a table, load_fly_2d_table performs a DSP memory check that produces error code 4 in case of error.
RTC4→RTC5	New command.
Version info	–
Version info	Available as of DLL 536, OUT 536.
References	set_fly_2d , init_fly_2d , get_fly_2d_offset

Ctrl Command	load_jump_table
Function	Loads a value table with Jump Delay data points from an ASCII text file (or alternatively performs automatic determination on the attached scan system) and uses linear interpolation to create the internal Jump Delay table for 2D jumps executable in Jump Mode .
Call	NoOfDataPoints = load_jump_table(Name, No, PosAck, MinDelay, MaxDelay, ListPos)
Parameters	Name See load_jump_table_offset .
	No See load_jump_table_offset .
	PosAck See load_jump_table_offset .
	MinDelay See load_jump_table_offset .
	MaxDelay See load_jump_table_offset .
	ListPos See load_jump_table_offset .
Result	See load_jump_table_offset .
Notes	• load_jump_table is identical to load_jump_table_offset with Offset = 0.
Version info	–
RTC4→RTC5	New command.
Reference	load_jump_table_offset

Ctrl Command	load_jump_table_offset	
Function	Loads a value table with Jump Delay data points from an ASCII text file (or alternatively performs automatic determination on the attached scan system) and uses linear interpolation to create the internal Jump Delay table for 2D jumps executable in Jump Mode .	
Call	NoOfDataPoints = load_jump_table_offset(Name, No, PosAck, Offset, MinDelay, MaxDelay, ListPos)	
Parameters	Name	Name of the text file or NULL . The text file may contain one or more tables. As a pointer to a null-terminated ANSI string.
	No	Specifies: - For Name = Filename, which table of the text file should be loaded. The parameter corresponds to the extension <No> of [JumpTable<No>] at the beginning of the desired table. - For Name = NULL, which scan system (or which scan head connector) should be used for automatic determination. Allowed values: [1, 2]. As an unsigned 32-bit value.
	PosAck	Position tolerance value in [bits] (only relevant for automatic determination). As an unsigned 32-bit value.
	Offset	Offset in [10 μ s] which is added to all automatically determined delay values (only relevant for automatic determination). As a signed 32-bit value.
	MinDelay	Minimum Jump Delay in [10 μ s] (only relevant for automatic determination). As an unsigned 32-bit value.
	MaxDelay	Maximum Jump Delay in [10 μ s] (only relevant for automatic determination). As an unsigned 32-bit value.
	ListPos	List position (in the area of list 1 or 2) for six list commands that can be overwritten (only relevant for automatic determination).
Result	Error code. As an unsigned 32-bit value. –1...– 50 Success for Name = filename. The absolute value of the return value equals the number of valid data points found in the table. Invalid entry values are ignored (see also page 205). –1024 Success for Name = NULL: Table has been automatically determined. 1 No valid data points found (but Table No found). 3 File not found. 4 Verify error: DSP memory error. 5 Busy error, board is BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY). 8 Access error: board reserved for another user program (get_last_error return code RTC5_ACCESS_DENIED).	

Ctrl Command	load_jump_table_offset	
Result (cont'd)	10	Only if Name = NULL : Param error: HeadNo or ListPos invalid.
	11	PCI error during download (get_last_error return code RTC5_SEND_ERROR).
	13	The specified table number could not be found in the file.
Comments	• For information on command usage, see Section "Jump-Length-Dependent Jump Delays", Page 204. • See also Chapter 8.1.5 "Jump Mode", Page 202. • Format requirements for placing the table with Jump Delay data points into the text file are described in Section "Notes on Loading Determined Jump Delay Values", Page 205. When loading the table, the RTC5 uses linear interpolation to establish appropriate values for the complete jump length range. • load_jump_table_offset overwrites any previously loaded Jump Delay tables for Jump Mode. • If Name = filename, then the table number No is loaded. The other parameters are not used. • If Name = NULL, then the Jump Delay table for head No is automatically determined and loaded (if Jump Mode has been previously enabled and activated successfully by set_jump_mode (Flag = 1)). Here, the parameters PosAck, Offset, MinDelay, MaxDelay and ListPos are used (see Section "Automatic Determination of the Jump Delay Table", Page 206). • Jump Mode takes effect upon the next 2D jump only if it has been activated and enabled by set_jump_mode or activated by set_jump_mode_list. • load_jump_table_offset is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • load_jump_table_offset is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During the runtime of load_jump_table_offset, External Starts are suppressed. • Before loading a table, load_jump_table_offset performs a DSP memory check that produces error code 4 in case of error.	
RTC4→RTC5	New command. There is no RTC4 Compatibility Mode for this command.	
Version info	–	
Reference	load_jump_table , set_jump_mode , set_jump_mode_list , get_jump_table , set_jump_table	

Ctrl Command	load_list
Function	Opens the list memory area for writing with list commands and sets the input pointer to the specified <i>relative</i> position within the desired list ("List 1" or "List 2"), but only if the list is not currently being processed (BUSY list status is not set) or has already been processed (USED list status is set).
Call	NoOfOpenedList = load_list(ListNo, Pos)
Parameters	ListNo Number of the list in which the input pointer should be set. As an unsigned 32-bit value. Allowed values: [0...3]. Only the two least significant bits are evaluated. = 0: The list is opened for which the BUSY list status is not currently set. "List 1" is opened, if the BUSY list status is not set for either list. = 1: "List 1" is opened, if the BUSY list status for it is not set (BUSY1 not set). = 2: "List 2" is opened if the BUSY list status for it is not set (BUSY2 not set). = 3: The list is opened for which the BUSY list status is not currently set but the USED list status. If for both lists the BUSY list status is not set but the USED list status: that list is opened, which would be executed by a following automatic list change. For Mem2 = 0 (see config_list) "List 1" is opened, if: • the BUSY list status for it is not set (ListNo = 0...2) • the BUSY list status for it is not set but the USED list status (ListNo = 3)
	Pos Position of the input pointer (offset relative to the start of the respective list). As an unsigned 32-bit value. Allowed value range: [0...(2²⁰-1)].
Result	Number of the opened list [1 or 2] if successful, otherwise 0. As an unsigned 32-bit value.
Comments	• If load_list has been successful, the next list command is stored at the specified address and all subsequent list commands at the following addresses in the selected list. • If load_list has not been successful (return code 0), then no list is opened and the input pointer is set to an invalid position. Then no further list commands can be input until the input pointer is correctly set (for example, by repeating load_list with a positive result or by the set_start_list_pos command etc.). It is the responsibility of the user to react to a return code of 0. load_list produces <i>no</i> wait loop and <i>does not</i> block execution of subsequent control commands.

Ctrl Command	load_list
Comments (cont'd)	• load_list (ListNo = 3) is useful in scenarios such as alternating list changes, where you want to wait specifically for a list to be processed (see Section "Alternating List Changes", Page 100) without needing to separately query the list status. Unintentional overwriting of not-yet-executed commands is thereby automatically avoided. To instead perform <i>unconditional</i> loading, you can use commands such as set_start_list_pos. • After a successful check of the list number (BUSY list status not set and, if applicable, USED list status set), load_list behaves like set_start_list_pos (see comments there). • If ListNo = 0...2 when using load_list, then note that the BUSY list status of a list can change between opening of the list with load_list and the actual loading of list commands, for example, if in the meantime an automatic list change has occurred that has been previously defined by auto_change or start_loop. In cases such as this, where loading of a list might occur simultaneously with processing of the same list, users must always ensure that the output pointer does not overtake the input pointer. • In contrast, for ListNo = 3 load_list ensures that a list is actually opened for loading only directly after processing (and after any successful automatic list changes). Particularly, if for no list the BUSY list status is set but for both the USED list status (for example, after initialization, after stop_execution or after an External Stop), the opened list number is synchronized with the following automatic list change.
RTC4→RTC5	New command.
Version info	–
References	set_start_list_pos

Ctrl Command	load_position_control																				
Function	Loads a table with data points from an ASCII text file and determines – by linear interpolation – the scaling function for position-dependent laser control (radial correction, see Section “Position-Dependent Laser Control”, Page 189).																				
Call	NoOfDataPoints = load_position_control(Name, No)																				
Parameters	Name Name of the text file or NULL . The text file may contain one or more tables. As a pointer to a null-terminated ANSI string.																				
	No Determines which table in the text file is to be loaded. The parameter corresponds to the extension <No> of [PositionCtrlTable<No>] at the beginning of the desired table. As an unsigned 32-bit value.																				
Result	A positive error code in case of an error. The negative number of found data points in case of success. As a signed 32-bit value. <table> <thead> <tr> <th>Value</th><th>Description</th></tr> </thead> <tbody> <tr> <td>–1...– 50</td><td>Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see also Section “Notes on Loading a Scaling Function”, Page 189).</td></tr> <tr> <td>–256</td><td>For Name = NULL (see also comments).</td></tr> <tr> <td>1</td><td>No valid data points found (though Table No found).</td></tr> <tr> <td>3</td><td>File not found.</td></tr> <tr> <td>4</td><td>DSP memory error.</td></tr> <tr> <td>5</td><td>BUSY error, board is BUSY list execution status or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).</td></tr> <tr> <td>8</td><td>Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).</td></tr> <tr> <td>11</td><td>PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).</td></tr> <tr> <td>13</td><td>The specified table number could not be found in the file.</td></tr> </tbody> </table>	Value	Description	–1...– 50	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see also Section “Notes on Loading a Scaling Function”, Page 189).	–256	For Name = NULL (see also comments).	1	No valid data points found (though Table No found).	3	File not found.	4	DSP memory error.	5	BUSY error, board is BUSY list execution status or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).	8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).	11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).	13	The specified table number could not be found in the file.
Value	Description																				
–1...– 50	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see also Section “Notes on Loading a Scaling Function”, Page 189).																				
–256	For Name = NULL (see also comments).																				
1	No valid data points found (though Table No found).																				
3	File not found.																				
4	DSP memory error.																				
5	BUSY error, board is BUSY list execution status or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).																				
8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).																				
11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).																				
13	The specified table number could not be found in the file.																				
Comments	- The format requirements for the text file’s table entries with data points for position-dependent laser control are described in Section “Notes on Loading a Scaling Function”, Page 189. When loading the table, the RTC5 determines suitable values for the entire range of control values. load_position_control overwrites any previously loaded scaling function for position-dependent laser control. - For Name = NULL (as during initialization by load_program_file), the scaling function $Scale(Position)=1.0$ is loaded for the complete position range so that no position-dependent correction takes place. - Position-dependent laser control only takes effect during subsequent mark commands or Arc Commands if it has been initialized by set_auto_laser_control. Position-dependent laser control is deactivated by set_auto_laser_control (Ctrl = 0) or by loading $Scale(Position)=1.0$. See also Section “Position-Dependent Laser Control”, Page 189.																				

Ctrl Command	load_position_control
Comments (cont'd)	• load_position_control is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • load_position_control is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During the runtime of load_position_control, External Starts are suppressed. • Before loading a table, load_position_control performs a DSP memory check. In case of an error, error code 4 is returned.
RTC4→RTC5	New command.
Version info	–
References	set_auto_laser_control

Ctrl Command	load_program_file																																				
Function	- Resets the RTC5 (Software reset) - Initializes the list memory - Performs a DSP memory check - Loads RTC5RBF.rbf - Loads RTC5OUT.out - Loads RTC5DAT.dat - Starts the DSP																																				
Call	ErrorNo = load_program_file(Path)																																				
Parameters	Path Path name of the folder, where RTC5OUT.out , RTC5RBF.rbf and RTC5DAT.dat are. As a pointer to a null-terminated string.																																				
Result	Error code. As an unsigned 32-bit value. <table> <thead> <tr> <th>Value</th><th>Description</th></tr> </thead> <tbody> <tr><td>0</td><td>Success.</td></tr> <tr><td>1</td><td>Reset error: the board could not be reset.</td></tr> <tr><td>2</td><td>Unreset error: the board does not restart.</td></tr> <tr><td>3</td><td>File error: file not found or cannot be opened.</td></tr> <tr><td>4</td><td>Format error: RTC5OUT.out has incorrect format.</td></tr> <tr><td>5</td><td>System error: not enough Windows memory.</td></tr> <tr><td>6</td><td>Another user program has already acquired the board.</td></tr> <tr><td>7</td><td>Version error: RTC5 DLL version, RTC5RBF.rbf version and RTC5OUT.out version are not compatible with each other.</td></tr> <tr><td>8</td><td>RTC5 board driver not found (get_last_error return code RTC5_ACCESS_DENIED).</td></tr> <tr><td>9</td><td>DriverCall error: loading of RTC5OUT.out failed.</td></tr> <tr><td>10</td><td>Configuration error: DSP register initialization failed.</td></tr> <tr><td>11</td><td>FPGA firmware error: loading of RTC5RBF.rbf failed.</td></tr> <tr><td>12</td><td>PCI download error: loading of RTC5DAT.dat failed.</td></tr> <tr><td>13</td><td>Busy error: Download locked, board is BUSY or INTERNAL-BUSY list execution status (get_last_error return code RTC5_BUSY).</td></tr> <tr><td>14</td><td>DSP memory error.</td></tr> <tr><td>15</td><td>Verify error (only applicable for download verification).</td></tr> <tr><td>16</td><td>PCI error.</td></tr> </tbody> </table>	Value	Description	0	Success.	1	Reset error: the board could not be reset.	2	Unreset error: the board does not restart.	3	File error: file not found or cannot be opened.	4	Format error: RTC5OUT.out has incorrect format.	5	System error: not enough Windows memory.	6	Another user program has already acquired the board.	7	Version error: RTC5 DLL version, RTC5RBF.rbf version and RTC5OUT.out version are not compatible with each other.	8	RTC5 board driver not found (get_last_error return code RTC5_ACCESS_DENIED).	9	DriverCall error: loading of RTC5OUT.out failed.	10	Configuration error: DSP register initialization failed.	11	FPGA firmware error: loading of RTC5RBF.rbf failed.	12	PCI download error: loading of RTC5DAT.dat failed.	13	Busy error: Download locked, board is BUSY or INTERNAL-BUSY list execution status (get_last_error return code RTC5_BUSY).	14	DSP memory error.	15	Verify error (only applicable for download verification).	16	PCI error.
Value	Description																																				
0	Success.																																				
1	Reset error: the board could not be reset.																																				
2	Unreset error: the board does not restart.																																				
3	File error: file not found or cannot be opened.																																				
4	Format error: RTC5OUT.out has incorrect format.																																				
5	System error: not enough Windows memory.																																				
6	Another user program has already acquired the board.																																				
7	Version error: RTC5 DLL version, RTC5RBF.rbf version and RTC5OUT.out version are not compatible with each other.																																				
8	RTC5 board driver not found (get_last_error return code RTC5_ACCESS_DENIED).																																				
9	DriverCall error: loading of RTC5OUT.out failed.																																				
10	Configuration error: DSP register initialization failed.																																				
11	FPGA firmware error: loading of RTC5RBF.rbf failed.																																				
12	PCI download error: loading of RTC5DAT.dat failed.																																				
13	Busy error: Download locked, board is BUSY or INTERNAL-BUSY list execution status (get_last_error return code RTC5_BUSY).																																				
14	DSP memory error.																																				
15	Verify error (only applicable for download verification).																																				
16	PCI error.																																				

Ctrl Command	load_program_file
Comments	• If <code>Path</code> = NULL, then the path of the user program's current working directory is used. Caution: It is not always the folder from which the user program has been launched. The current working directory can change, for example, when a file from another folder is selected by the Windows Explorer (unless the "NoChangeDir" flag has been set when incorporating the Explorer window into the user program). • After each Hardware reset, the first user program must begin by issuing a load_program_file command during initialization of the RTC5 Board, see Section "Initializing the Board", Page 87. load_program_file should also be executed (for example, if another user program acquires the board, see acquire_rtc) when the board needs to be returned to the default state. • If multiple RTC5 boards are connected as master and slave, then load_program_file must have been called on all boards prior to initializing and operating the individual boards with further commands, see Chapter 6.6.3 "Master/Slave Operation", Page 113. • After execution of load_program_file: – The laser focus is in the center of the Image field (0 0) – The laser control is <i>deactivated</i> – On the state of the various output ports, see Chapter 7.4.1 "Enabling, Activating and Switching Laser Control Signals", Page 173 (LASERON, LASER1, LASER2), Chapter 9.1.1 "16-Bit Digital Output Port", Page 258 (EXTENSION 1 socket connector), Chapter 9.1.2 "8-Bit Digital Output Port", Page 259 (EXTENSION 2 socket connector), Chapter 9.1.3 "2 Bit Digital Output Port", Page 259 (LASER connector), Chapter 9.1.4 "12-Bit Analog Output Port 1 and 2", Page 259 (LASER connector). • <i>Caution! In general, sporadic load_program_file calls in your user program:</i> – <i>Contradict the safe switch-on sequence prescribed in Chapter 5.4 "Installing the RTC5 Software", Page 80</i> – <i>Pose the risk of personal injury and/or property damage (cases have been reported where lasers with poor electricians have emitted)</i> <i>If you absolutely cannot refrain from sporadic load_program_file calls in your user program, you must implement appropriate messages that warn users accordingly and prompt them to take actions that prevent these hazards.</i> • The load_program_file command does <i>not</i> load correction tables. Even 1-to-1 tables therefore need to be explicitly requested, see load_correction_file. Already-loaded correction tables remain loaded after load_program_file (#1, #2 only, not #3, #4). • During a RTC5 reset, list memory contents are erased and all parameters (for example, memory configuration, internal variables, matrices, offsets and table assignments) previously set with config_list or select_cor_table are deleted or reset to their default values. During a reset, a correction table is assigned as by select_cor_table(1,0) but no scanner motion to the corrected output position is executed. • load_program_file only returns to the calling user program, when DSP initialization has been completed.

Ctrl Command	load_program_file
Comments (cont'd)	• load_program_file is <i>not</i> executed (get_last_error return code RTC5_BUSY), if – the BUSY list execution status is currently set – the INTERNAL-BUSY list execution status is currently set • load_program_file is even executed, if: – a list has been previously terminated in an orderly manner – a list has been paused by set_wait (PAUSED list execution status set) – a list has been paused by stop_execution – a list has been paused by an External Stop. • The files RTC5OUT.out, RTC5RBF.rbf and RTC5DAT.dat are included in the RTC5 software package. For easy identifying and archiving of different software versions, the files are also delivered zipped (the zip file names RTC5<...>_<Version>.zip include the version numbers). Copy or unzip the three files (of desired version) to the hard drive of your PC. • Assorted versions of the RTC5 DLL and the files RTC5OUT.out, RTC5RBF.rbf and RTC5DAT.dat cannot be arbitrarily combined with another (each zip file in the RTC5 software includes a text file with corresponding version information). load_program_file performs a version check. If there is a version error, then the loaded programs remain in RTC5 memory, but the board is released by release_rtc directly after the version check and therefore is not available for further commands other than those not requiring access rights (get_last_error return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH). To then load a correct program version, only the multi-board command n_load_program_file can be called. Hereby, temporary access rights are requested and released after the download (if the board has not been acquired by another user program; n_load_program_file does not perform an acquire_rtc command). In this case, the single-board command load_program_file does not get access rights for the board. Alternatively, the board can be made currentless to unload the program software (afterward also load_program_file can be used again).
RTC4→RTC5	• Path specifies a folder name with RTC5 (in contrast a file name with the RTC4). • load_program_file loads three files, with fixed formats and names (RTC5OUT.out, RTC5RBF.rbf, RTC5DAT.dat) (see above). • After execution of load_program_file the laser control is <i>deactivated</i>.
Version info	–
References	–

Ctrl Command	load_stretch_table																						
Function	Loads a table with data pairs from an ASCII text file for enhanced 3D correction, see Section "Enhanced 3D Correction", Page 225 .																						
Call	NoOfDataPairs = load_stretch_table(Name, No)																						
Parameters	Name Name of the text file or NULL . The text file may contain one or more tables. As a pointer to a null-terminated ANSI string.																						
	No As a signed 32-bit value. - For $No \geq 0$, this parameter specifies which table in the text file is to be loaded. The parameter corresponds to the extension <code><No></code> of <code>[StretchTable<No>]</code> at the beginning of the desired table. $No < 0$: Reserved.																						
Result	A positive error code in case of an error. The negative number of found data pairs in case of success. As a signed 32-bit value. <table> <thead> <tr> <th>Value</th><th>Description</th></tr> </thead> <tbody> <tr> <td>< 0</td><td>Success. The absolute value of the return value is equal to the number of valid data pairs found in the table.</td></tr> <tr> <td>0</td><td>Reserved.</td></tr> <tr> <td>2</td><td>Out of Memory (not enough Windows memory).</td></tr> <tr> <td>3</td><td>File not found.</td></tr> <tr> <td>4</td><td>DSP memory error.</td></tr> <tr> <td>5</td><td>BUSY error, board is BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code <code>RTC5_BUSY</code>).</td></tr> <tr> <td>6</td><td>Data error: data pairs missing.</td></tr> <tr> <td>11</td><td>PCI download error (get_last_error return code <code>RTC5_SEND_ERROR</code>).</td></tr> <tr> <td>13</td><td>The specified table number could not be found in the file.</td></tr> <tr> <td>15</td><td>Verify error (get_last_error return code <code>RTC5_VERIFY_ERROR</code>, only possible with active download verification, see set_verify).</td></tr> </tbody> </table>	Value	Description	< 0	Success. The absolute value of the return value is equal to the number of valid data pairs found in the table.	0	Reserved.	2	Out of Memory (not enough Windows memory).	3	File not found.	4	DSP memory error.	5	BUSY error, board is BUSY or INTERNAL-BUSY list execution status , no download (get_last_error return code <code>RTC5_BUSY</code>).	6	Data error: data pairs missing.	11	PCI download error (get_last_error return code <code>RTC5_SEND_ERROR</code>).	13	The specified table number could not be found in the file.	15	Verify error (get_last_error return code <code>RTC5_VERIFY_ERROR</code> , only possible with active download verification, see set_verify).
Value	Description																						
< 0	Success. The absolute value of the return value is equal to the number of valid data pairs found in the table.																						
0	Reserved.																						
2	Out of Memory (not enough Windows memory).																						
3	File not found.																						
4	DSP memory error.																						
5	BUSY error, board is BUSY or INTERNAL-BUSY list execution status , no download (get_last_error return code <code>RTC5_BUSY</code>).																						
6	Data error: data pairs missing.																						
11	PCI download error (get_last_error return code <code>RTC5_SEND_ERROR</code>).																						
13	The specified table number could not be found in the file.																						
15	Verify error (get_last_error return code <code>RTC5_VERIFY_ERROR</code> , only possible with active download verification, see set_verify).																						
Comments	- Details about enhanced 3D correction (for example, format requirements for the text file's table entries) are described on Section "Enhanced 3D Correction", Page 225. - A successfully loaded table activates the new enhanced 3D correction. Here, load_stretch_table overwrites any previously loaded table. - If <code>Name</code> is <i>not</i> NULL, but no table has been successfully read, then load_stretch_table returns an error code (for example, code 13 if a <code>No</code> value is specified for which no <code>[StretchTable<No>]</code> entry is included in the text file), but otherwise has no effect (that is, any previous successfully downloaded table stays enabled).																						

Ctrl Command	load_stretch_table
Comments (cont'd)	• If <code>Name</code> is NULL, then <code>load_stretch_table</code> disables any enhanced 3D correction enabled by a previous <code>load_stretch_table</code>. • <code>load_stretch_table</code> is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • <code>load_stretch_table</code> is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During execution of <code>load_stretch_table</code>, External Starts are suppressed. • Before loading a table, <code>load_stretch_table</code> performs a DSP memory check. In case of an error, error code 4 is returned.
RTC4→RTC5	New command.
Version info	–
References	–

Ctrl Command	load_sub
Function	Assigns the specified index to a subroutine defined by subsequent list commands and loads the subroutine into the protected list memory area "List 3".
Call	<code>load_sub(Index)</code>
Parameters	Index Index of the indexed subroutine. As an unsigned 32-bit value. Allowed value range: [0...1023].
Comments	- Up to 1024 indexed subroutines can be stored. If <code>Index > 1023</code> then load_sub is ignored (get_last_error return code <code>RTC5_PARAM_ERROR</code>). - The address in the protected list memory area "List 3" where the subroutine should be stored is automatically determined and internally managed. - Indexed subroutines must be terminated by a list_return call. This is a prerequisite for actual storage of the commands, entry of the start address into the internal management table, and initiating a flush of the buffered list input, see Chapter 6.4.1 "Loading Lists", Page 94. Otherwise (the input pointer is altered without a preceding list_return), the subroutine with this index is not available. - An indexed subroutine is not stored if the protected list memory area "List 3" has not been previously configured for a sufficient size beyond "List 1" and "List 2". - If list_return is the next command after load_sub, then the corresponding subroutine is deleted from the internal management table. - Indexed subroutines can be called by the sub_call command along with the corresponding index (see Section "General Information on Calling Subroutines", Page 103). - Observe all notes in Section "Indexed Subroutines", Page 102.
RTC4→RTC5	New command.
Version info	–
References	list_return , sub_call , load_char , load_text_table

Ctrl Command	load_text_table
Function	Assigns the specified index to a text string defined by subsequent list commands and loads the text string into the protected list memory area "List 3".
Call	<code>load_text_table(Index)</code>
Parameters	Index Index of the indexed text string. As an unsigned 32-bit value. Allowed value range: [0...41].
Comments	- Up to 42 indexed text strings can be stored (for marking times, dates and serial numbers by other commands). The following ordering applies: - Index = 0...9: digits for marking the time and date [0...9] - Index = 10...21: months [January...December] - Index = 22...28: days-of-the-week [Sunday...Saturday] - Index = 29: blank character for marking serial numbers - Index = 30...39: digits for marking serial numbers [0...9] - Index = 40: text for "a.m." - Index = 41: text for "p.m." - If Index > 41 then load_text_table is ignored (get_last_error return code RTCS_PARAM_ERROR). - Even if digits are defined by mark_text instead of as individual characters with mark_char, the character set can still be subsequently switched for this purpose (see select_char_set). - The addresses in the protected list memory area "List 3" where the text string definitions are stored are automatically determined and internally managed. Management is independent of that for indexed subroutines (see load_sub) and character definitions (see load_char). - Indexed text string definitions must be terminated with a list_return call. This is a prerequisite for actual storage of the commands, entry of the start address into the internal management table, and initiating a flush of the buffered list input, see Chapter 6.4.1 "Loading Lists", Page 94. Otherwise (the input pointer is altered without a preceding list_return), the text string with this index is not available. - An indexed text string definition is not stored if the protected list memory area "List 3" has not been previously configured for a sufficient size beyond "List 1" and "List 2". - If list_return is the next command after load_text_table, then the corresponding text string definition is deleted from the internal management table. - Also observe all notes in the Chapter 6.5.2 "Character Sets and Text Strings", Page 107.
RTC4→RTC5	New command.
Version info	–
References	list_return , load_sub , load_char

Ctrl Command	load_varpolydelay																				
Function	Loads a table with data points from an ASCII text file for the scaling function of the "Variable Polygon Delay", see Section "User-defined "Variable Polygon Delays"", Page 141.																				
Call	NoOfDataPoints = load_varpolydelay(Name, No)																				
Parameters	Name Name of the text file or NULL . The text file may contain one or more tables. As a pointer to a null-terminated ANSI string.																				
	No Table in the text file which is to be loaded. The parameter corresponds to the extension <No> of [VarPolyTable<No>] at the beginning of the desired table. As an unsigned 32-bit value.																				
Result	A positive error code in case of an error. The negative number of found data points in case of success. As a signed 32-bit value. <table> <thead> <tr> <th>Value</th><th>Description</th></tr> </thead> <tbody> <tr> <td>-1...- 50</td><td>Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see page 141).</td></tr> <tr> <td>-1024</td><td>For Name = NULL: the table initialized according to $1-\cos(\varphi)$ has been internally loaded (as with program start; see Figure 44).</td></tr> <tr> <td>1</td><td>No valid data points found (though Table No found).</td></tr> <tr> <td>3</td><td>File not found.</td></tr> <tr> <td>4</td><td>DSP memory error.</td></tr> <tr> <td>5</td><td>BUSY error, board is BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).</td></tr> <tr> <td>8</td><td>Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).</td></tr> <tr> <td>11</td><td>PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).</td></tr> <tr> <td>13</td><td>The specified table number could not been found in the file.</td></tr> </tbody> </table>	Value	Description	-1...- 50	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see page 141).	-1024	For Name = NULL : the table initialized according to $1-\cos(\varphi)$ has been internally loaded (as with program start; see Figure 44).	1	No valid data points found (though Table No found).	3	File not found.	4	DSP memory error.	5	BUSY error, board is BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).	8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).	11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).	13	The specified table number could not been found in the file.
Value	Description																				
-1...- 50	Success. The absolute value of the return value is equal to the number of valid data points found in the table. Invalid entries are ignored (see page 141).																				
-1024	For Name = NULL : the table initialized according to $1-\cos(\varphi)$ has been internally loaded (as with program start; see Figure 44).																				
1	No valid data points found (though Table No found).																				
3	File not found.																				
4	DSP memory error.																				
5	BUSY error, board is BUSY or INTERNAL-BUSY list execution status, no download (get_last_error return code RTC5_BUSY).																				
8	Board is locked by another user program (get_last_error return code RTC5_ACCESS_DENIED).																				
11	PCI error (get_last_error return code RTC5_SEND_ERROR), verify error (get_last_error return code RTC5_VERIFY_ERROR).																				
13	The specified table number could not been found in the file.																				
Comments	- The format requirements for text file's table entries with data points for the user-defined "Variable Polygon Delay" are described in Section "User-defined "Variable Polygon Delays"", Page 141. When loading the table, the RTC5 determines suitable values for the entire range of angles by linear interpolation. load_varpolydelay overwrites any previously loaded table for the "Variable Polygon Delay". - For Name = NULL (as during initialization by load_program_file), the internal (default) table for the "Variable Polygon Delay" ($1-\cos(\varphi)$, see Figure 44) is loaded.																				

Ctrl Command	load_varpolydelay
Comments (cont'd)	• load_varpolydelay is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set (a list is being processed or has been halted by pause_list) – the INTERNAL-BUSY list execution status is set • load_varpolydelay is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During execution of load_varpolydelay, External Starts are suppressed. • Before loading a table, load_varpolydelay performs a DSP memory check. In case of an error, error code 4 is returned.
RTC4→RTC5	Basically unchanged functionality. However: • To return to the internal standard Polygon Delay table, a reset or renewed program loading by load_program_file is <i>no longer</i> necessary (see Name = NULL above). • The ASCII text file can have any filename extension.
Version info	–
References	load_program_file , set_delay_mode

Ctrl Command	load_z_table																											
Function	Loads coefficients A, B and C into the currently assigned 3D correction table.																											
Restriction	If the Option “3D” has not been enabled or a 3D correction table has not been assigned (see select_cor_table), then load_z_table returns the error code 12 or 13 and otherwise has no effect.																											
Call	ErrorNo = load_z_table(A, B, C)																											
Parameters	A	Coefficient A of the parabolic function $z_{out} = A + B I + C I^2$ which is used for calculating the Z output values (I in the RTC4 compatibility range [–32,768...+32,767]). As a 64-bit IEEE floating point value. Allowed value range: [–67108864.0...67108864.0]. Out-of-range values are clipped to the boundary values.																										
	B	Like A (analogously). Allowed value range: [–2048.0...2048.0].																										
	C	Like A (analogously). Allowed value range: [–16.0...16.0].																										
Result	Error code. As an unsigned 32-bit value. Error bits with values 1 to 64 can also occur combined, but not in conjunction with error code 11...14, which only occur separately. Warnings 12 and 13 are only returned if no other errors exist. <table><tr><th>Value</th><th>Description</th></tr><tr><td>0</td><td>No error.</td></tr><tr><td>1</td><td>A exceeded the maximum allowed value.</td></tr><tr><td>2</td><td>A undercut the minimum allowed value.</td></tr><tr><td>4</td><td>B exceeded the maximum allowed value.</td></tr><tr><td>8</td><td>B undercut the minimum allowed value.</td></tr><tr><td>16</td><td>C exceeded the maximum allowed value.</td></tr><tr><td>32</td><td>C undercut the minimum allowed value.</td></tr><tr><td>64</td><td>Execution denied (possibly a BUSY list execution status or INTERNAL-BUSY list execution status error; for exact reason: see get_last_error).</td></tr><tr><td>11</td><td>Access denied.</td></tr><tr><td>12</td><td>Option “3D” not enabled.</td></tr><tr><td>13</td><td>No 3D correction table is currently assigned.</td></tr><tr><td>14</td><td>RTC5 board driver not found.</td></tr></table>		Value	Description	0	No error.	1	A exceeded the maximum allowed value.	2	A undercut the minimum allowed value.	4	B exceeded the maximum allowed value.	8	B undercut the minimum allowed value.	16	C exceeded the maximum allowed value.	32	C undercut the minimum allowed value.	64	Execution denied (possibly a BUSY list execution status or INTERNAL-BUSY list execution status error; for exact reason: see get_last_error).	11	Access denied.	12	Option “3D” not enabled.	13	No 3D correction table is currently assigned.	14	RTC5 board driver not found.
Value	Description																											
0	No error.																											
1	A exceeded the maximum allowed value.																											
2	A undercut the minimum allowed value.																											
4	B exceeded the maximum allowed value.																											
8	B undercut the minimum allowed value.																											
16	C exceeded the maximum allowed value.																											
32	C undercut the minimum allowed value.																											
64	Execution denied (possibly a BUSY list execution status or INTERNAL-BUSY list execution status error; for exact reason: see get_last_error).																											
11	Access denied.																											
12	Option “3D” not enabled.																											
13	No 3D correction table is currently assigned.																											
14	RTC5 board driver not found.																											

Ctrl Command	load_z_table
Comments	• load_z_table is only needed for re-calibrating the z axis in a 3-axis scan system (for adjusting corresponding coefficients see page 159). Both positive and negative coefficients can be specified. The coefficients should preferably be chosen so that all Z output values lie within the range [-32,768...+32,767]. • load_z_table is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • load_z_table is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • load_z_table should always be used <i>after</i> load_correction_file, since load_correction_file sets the three coefficients to the default values of the loaded correction table. • Prior to the next list command that directly follows load_z_table, a smooth transition from the last z position to the changed position is performed at jump_speed. You can also immediately force this by select_cor_table. This way, time delays can be avoided during an External Start. • Coefficients A, B and C can be queried from the loaded 3D correction table by get_table_para (and from the currently assigned 3D correction table by get_head_para).
RTC4→RTC5	Unchanged functionality. In addition: changed value ranges and error codes. If the correct calibration factors are used (see Section "3D Commands", Page 223), then the same ABC coefficients can be used on the same 3-axis scan system with the RTC4 and RTC5 boards.
Version info	–
References	get_z_distance , read_abc_from_file , write_abc_to_file , select_cor_table

Ctrl Command	load_zoom_correction_file
Comments	• This command is described, for example, in the manual for the intelliWELD II FT.

Normal List Command	long_delay
Function	Pauses further processing of the list for the specified time.
Call	<code>long_delay(Delay)</code>
Parameters	Delay Delay time. In bits. As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $0 \leq \text{delay} \leq (2^{32}-1)$.
Comments	• long_delay switches off Signals for "Laser Active" Operation after a LaserOff Delay, waits for a possible scanner delay and pauses further processing of the list for the specified time. • long_delay should always be called after changing the lamp current of a YAG laser to obtain a constant laser power. • list_nop corresponds to <code>long_delay(1)</code>.
RTC4→RTC5	Unchanged functionality (except for the increased range of values).
Version info	–
References	set_defocus_list

Normal List Command	mark_abs
Function	Moves the laser focus at mark speed along a 2D vector from the current position to the specified position (absolute coordinate values) within a 2D Image field .
Call	<code>mark_abs(X, Y)</code>
Parameters	X Absolute x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
Comments	• If the mark speed has not been previously explicitly set by set_mark_speed or set_mark_speed_ctrl, then the marking is executed at a predefined mark speed of 1,000 <i>bits/ms</i>. • The Signals for "Laser Active" Operation are automatically turned on at the beginning of the marking (or remain on after a directly preceding [*]mark[*] Command or "Arc" command). The defined Scanner Delays and Laser Delays are thereby taken into account (see Chapter 7.2 "Delay Settings – Coordinating Scan Head Control and Laser Control", Page 133). Note that other delays are executed in Sky Writing mode. Exception: zero-length [*]mark[*] Commands (see notes on page 137).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the values specified for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	set_mark_speed, set_scanner_delays, mark_rel, arc_abs, timed_mark_abs

Normal List Command	mark_abs_3d
Function	Moves the laser focus at mark speed along a 3D vector from the current position to the specified position (absolute coordinate values) within a 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then mark_abs_3d has the same effect as mark_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	<code>mark_abs_3d(X, Y, Z)</code>
Parameters	X Absolute x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	Z Like X (analogously), except Allowed value range: [-32,768...+32,767].
Comments	Except for the additional motion in the third dimension, mark_abs_3d functions similarly to mark_abs (see comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the values specified for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode .
Version info	–
References	mark_abs , mark_rel_3d , timed_mark_abs_3d

Undelayed Short List Command	mark_char
Function	Marks an indexed character.
Call	<code>mark_char(Char)</code>
Parameters	Char Index of the indexed character to be marked. As an unsigned 32-bit value. Allowed value range: [0...1023]). The following applies: $\text{Char} = \text{character set number} \times 256 + \text{ASCII number of the character}$ (character sets are numbered 0...3).
Comments	• mark_char reads the indexed character's starting address from the internal management table based on the supplied index and then calls list_call (see also the comments there), which then starts the corresponding command list. • mark_char starts indexed characters in protected memory (that were loaded and/or referenced by load_char, load_disk or copy_dst_src) as well as indexed subroutines in the unprotected list area (that were referenced as characters by set_char_pointer or copy_dst_src). • If no character is referenced for the supplied index, then the jump is suppressed and execution continues at the command located after the calling position. In some circumstances, a list_continue might be executed (see Section "Normal, Short, Variable and Multiple List Commands", Page 278). get_char_pointer (Char) can be used to determine whether a character has been referenced for a particular index. If no character has been referenced, get_char_pointer returns the value "-1" (that is, $2^{32}-1$). • If $\text{Char} > 1023$, then mark_char is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). • Absolute Vector commands and "Arc" commands execute absolutely after being called with mark_char. If the character needs to execute at various locations within the Image field, then either the command list can only contain relative [*]mark[*] Commands, "Arc" commands or Jump commands or mark_char_abs must be used instead. • The called character should not contain mark_text commands that also contain this character. Such text is <i>not</i> marked. The called character itself then might not be complete. • See also Chapter 6.5.2 "Character Sets and Text Strings", Page 107.
RTC4→RTC5	New command.
Version info	–
References	mark_char_abs , mark_text , set_char_pointer , get_char_pointer

Undelayed Short List Command	mark_char_abs
Function	Marks an indexed character. In the called command list (see below), any absolute Vector commands and " Arc " commands receive an offset (based on the current coordinates at the time of the call).
Call	<code>mark_char_abs(Char)</code>
Parameters	Char Index of the indexed character to be marked. As an unsigned 32-bit value. Allowed value range: [0...1023]). The following applies: $\text{Char} = \text{character set number} \times 256 + \text{ASCII number of the character}$ (character sets are numbered 0...3).
Comments	• The mark_char_abs command reads the indexed character's starting address from the internal management table based on the supplied index and then calls list_call_abs (see also the comments there), which then starts the corresponding command list. • If the command list of the called character contains no absolute commands, then there is no difference between mark_char_abs and mark_char.
RTC4→RTC5	New command.
Version info	–
References	mark_char

Normal List Command	mark_date
Function	Marks a part of the date previously stored by time_fix , time_fix_f or time_fix_f_off in the selected format at the current position.
Call	<code>mark_date(Part, Mode)</code>
Parameters	Part Specifies which part of the date should be marked. = 0: Year (only the last two digits). = 1: Month (in customer specific notation). = 2: Day (corresponding to the normal Gregorian date). = 3: Day-of-the-week (in customer specific notation). = 4: Julian day (see also time_fix_f_off). = 5: Year (4 digits). = 6: Month as numerals (January = 1,...). = 7: Day-of-the-week as numerals (Sunday = 1,...). As an unsigned 32-bit value. Allowed value range: [0...7].
	Mode Selection of the format. As an unsigned 32-bit value. Allowed value range: [0...3]. a) Only affects Part = 2, 4, 6 or 7: The number of leading zeros in the day number. Bit #0 = 0: Leading zeros are suppressed. Bit #0 = 1: Day of the month (Part = 2), always two digits. Day of the year (Part = 4), always three digits. Month (Part = 6), always two digits. Day-of-the-week (Part = 7), always two digits. b) Only affects Part = 0, 2 or 4...7: Desired character set for marking digits. Bit #1 = 0: The indexed text string for digits [0...9], as defined by load_text_table or set_text_table_pointer , is marked. Bit #1 = 1: The indexed characters for digits [0...9], as defined by load_char or set_char_pointer , are marked. The desired character set can be specified with select_char_set . For Part = 1 or 3, days of the month or days of the week are marked by indexed text strings (Index = 10...28) defined with load_text_table or set_text_table_pointer (here, the parameter Mode is ignored). For marking as numerals, also Part = 6 or 7 can be used (then Mode is considered).

Normal List Command	mark_date
Comments	• Before marking dates (after every boot-up), the RTC5 and PC times should be synchronized (see time_update) and the current (to be marked) date value should be stored with time_fix, time_fix_f or time_fix_f_off (see Chapter 7.5 "Marking Dates, Times and Serial Numbers", Page 196). • The complete date can be marked by multiple calls of mark_date. • mark_date reads (according to the stored date and according to the selected date part) the starting address of the corresponding indexed text string or character from the internal management table and then calls list_call (see also the comments there) an appropriate number of times, which then starts the corresponding command list. The command lists must contain marking instructions for digits 0...9 and for the month and day-of-the-week designations (see page 108). Non-defined text strings or characters are ignored (that is, not marked). The called indexed text strings can also contain calls to indexed characters (mark_char or mark_char_abs) and complete texts (mark_text or mark_text_abs). In the latter case, the character set can be switched when needed (before marking by mark_date) with select_char_set (see also Chapter 6.5.2 "Character Sets and Text Strings", Page 107). • If Part > 7 and/or Mode > 3, then mark_date is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). • Absolute Vector commands and "Arc" commands execute absolutely after being called with mark_date. If date markings need to execute at various locations within the Image field, then the corresponding indexed text strings (or characters) can only contain relative [*]mark[*] Commands, "Arc" commands or Jump commands or mark_date_abs must be used instead. • When marking Gregorian dates or Julian days, the transition to the next day occurs at 00:00 o'clock. Leap years are represented in both date styles.
RTC4→RTC5	New command.
Version info	–
References	time_fix , time_fix_f , time_fix_f_off , load_text_table , set_text_table_pointer , mark_date_abs

Normal List Command	mark_date_abs
Function	Marks a part of the date previously stored by time_fix , time_fix_f or time_fix_f_off in the selected format at the current position. In the called indexed text strings or characters (see below), any absolute Vector commands and " Arc " commands receive an offset (based on the current coordinates at the time of the call).
Call	<code>mark_date_abs(Part, Mode)</code>
Parameters	Part See mark_date .
	Mode See mark_date .
Comments	• The mark_date_abs command works like mark_date. However, internal calling of the indexed text strings (or characters) is by list_call_abs instead of list_call. • If the command lists of the called indexed text strings (or characters) contain no absolute commands, then there is no difference between mark_date_abs and mark_date.
RTC4→RTC5	New command.
Version info	–
References	mark_date

Normal List Command	mark_ellipse_abs
Function	Moves the laser focus at mark speed along an elliptical arc around the specified midpoint (absolute coordinate values) within a 2D Image field .
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), mark_ellipse_abs is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>mark_ellipse_abs(X, Y, Alpha)</code>
Parameters	X Absolute x coordinate of the ellipse midpoint. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
	Alpha Angle between elliptical half-axis a (defined by set_ellipse) and the x axis (the angle is referenced to the positive x direction, positive angle values correspond to counterclockwise angles). In degrees. As a 64-bit IEEE floating point value. Alpha gets normalized to the value range [0...<360°].
Comments	- The parameters for mark_ellipse_abs only determine the position and orientation of the to-be-executed arc. Before execution of mark_ellipse_abs, its shape must have been specified by set_ellipse. For descriptions of the individual parameters, see also Section "Ellipse Commands", Page 126. - If the arc starting point defined by mark_ellipse_abs and set_ellipse does not equal the current position, then a "Hard jump" to the starting point is executed prior to marking, see also notes in Section "Ellipse Commands", Page 126. - See also all comments for arc_abs.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	set_ellipse , mark_ellipse_rel , set_mark_speed , set_scanner_delays , arc_abs

Normal List Command	mark_ellipse_rel
Function	Moves the laser focus at mark speed along an elliptical arc around the specified midpoint (relative coordinate values) within a 2D Image field .
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), mark_ellipse_rel is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>mark_ellipse_rel(dX, dY, Alpha)</code>
Parameters	dX Relative x coordinate of the ellipse midpoint. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values.
	dY Like dX (analogously).
	Alpha Angle between elliptical half-axis a (defined by set_ellipse) and the x axis (see mark_ellipse_abs). In degrees. As a 64-bit IEEE floating point value.
Comments	The coordinates for the ellipse midpoint are to be supplied as relative coordinates with respect to the current position. Otherwise, mark_ellipse_rel is identical to mark_ellipse_abs (see the comments there).
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly.
Version info	–
References	mark_ellipse_abs , set_ellipse

Normal List Command	mark_rel
Function	Moves the laser focus at mark speed along a 2D vector from the current position to the specified position (relative coordinate values) within a 2D Image field .
Call	<code>mark_rel(dX, dY)</code>
Parameters	dX Relative x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like dX (analogously).
Comments	The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, mark_rel is identical to mark_abs (see the comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly.
Version info	–
References	mark_abs , mark_rel_3d , timed_mark_rel

Normal List Command	mark_rel_3d
Function	Moves the laser focus at mark speed along a 3D vector from the current position to the specified position (relative coordinate values) in the 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then mark_rel_3d has the same effect as mark_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	<code>mark_rel_3d(dX, dY, dZ)</code>
Parameters	dX Relative x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like dX (analogously).
	dZ Like dX (analogously), except Allowed value range: [-32,768...+32,767].
Comments	The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, mark_rel_3d is identical to mark_abs_3d (see comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified value for dX and dY by 16. The allowed value range decreases accordingly. The value range for dZ is identical in RTC5 Standard Mode and RTC4 Compatibility Mode .
Version info	–
References	mark_abs_3d, mark_abs, mark_rel, timed_mark_rel_3d

Normal List Command	mark_serial
Function	Marks the current serial number of the serial-number-set most recently selected by select_serial_set_list (or of serial-number-set 0 after load_program_file) in the selected format at the current position. Afterward the serial number is (optionally) automatically incremented.
Call	<code>mark_serial(Mode, Digits)</code>
Parameters	Mode Selection of the serial number format and (de)activation of automatic serial number incrementing. As an unsigned 32-bit value. $\text{Mode} = 20 \times M_1 + 10 \times M_2 + M_3$ Allowed value range: for M_1 [0, 1], for M_2 [0, 1], for M_3 [0...2]. a) Selection of the character set for digit marking. $M_1 = 0$: The indexed text string for digits [30...39], as defined by load_text_table or set_text_table_pointer, is marked. $M_1 = 1$: The indexed characters for digits [0...9], as defined by load_char or set_char_pointer, are marked. The desired character set can be selected (previously) with select_char_set. b) Incrementing of the serial number after marking. $M_2 = 0$: The serial number is incremented after marking. $M_2 = 1$: The serial number is <i>not</i> incremented after marking. c) Marking of leading zeros. $M_3 = 0$: Leading zeros are marked as zeros. Dependent on M_1 the corresponding indexed text string definition (Index = 30) or the indexed character definition '0' is used. $M_3 = 1$: Leading zeros are suppressed (left-justified marking). $M_3 = 2$: Leading zeros are marked as blank characters (right-justified marking). Dependent on M_1 the corresponding indexed text string definition (Index = 29) or the indexed character definition ' ' is used.
	Digits Number [0-12] of to-be-marked digits. As an unsigned 32-bit value. Allowed value range: [0-12]. Larger values are clipped.
Comments	- The first serial number to be marked must have been previously specified by set_serial, set_serial_step or set_serial_step_list; otherwise, the starting serial number is 0. The starting serial number can have a maximum length of 10 digits. - With every call of mark_serial, the serial number is formatted in accordance with M_3 and when $M_2 = 0$ it is automatically incremented before the actual marking. Here, the increment size is 1 unless otherwise specified by set_serial_step or set_serial_step_list. - The current serial number can be queried with get_list_serial, for example, after an aborted list to determine if the current number has been incremented or not. - If the incremented serial number exceeds 10^{Digits}, then marking begins again at 0. The control command set_max_counts allows specification of the maximum number of External Starts and thus the maximum number of markings. Here, all markings of all serial-number-sets contribute jointly to the count.

Normal List Command	mark_serial
Comments (cont'd)	• If <code>Digits = 0</code>, then a “markless” marking is executed. If $M_2 = 0$, then the serial number is incremented by 1 (any increment size defined by <code>set_serial_step</code> or <code>set_serial_step_list</code> are not used in this case!). This can be useful, if a single serial number is to be omitted (repeat n times if necessary; $n = \text{increment}$), but can also be used to indirectly increase the serial number by an <i>additional</i> increment. • For each to-be-marked serial number digit, the <code>mark_serial</code> command reads the starting address of the corresponding indexed text string (or – for $M_1 = 1$ – of the corresponding indexed character) from the internal management table and then calls <code>list_call</code> (see also the comments there) an appropriate number of times, which then starts the corresponding command lists. The command lists must contain marking instructions for digits 0...9 (see page 108). Non-defined text strings or characters are ignored (that is, not marked). The called indexed text strings can also contain calls to indexed characters (<code>mark_char</code> or <code>mark_char_abs</code>) and complete texts (<code>mark_text</code> or <code>mark_text_abs</code>). In the latter case, the character set can be switched if needed (before marking by <code>mark_serial</code>) with <code>select_char_set</code> (see also Chapter 6.5.2 “Character Sets and Text Strings”, Page 107). • For invalid <code>Mode</code> values, the <code>mark_serial</code> is, already during loading, replaced by a <code>list_nop</code> (<code>get_last_error</code> return code <code>RTC5_PARAM_ERROR</code>). • Absolute <code>Vector commands</code> and “Arc” commands execute absolutely after being called with <code>mark_serial</code>. If serial number markings need to execute at various locations within the <code>Image field</code>, then the corresponding indexed text strings (or characters) can only contain relative <code>[*]mark[*] Commands</code>, “Arc” commands or <code>Jump commands</code> or <code>mark_serial_abs</code> must be used instead.
RTC4→RTC5	New command.
Version info	–
References	<code>set_serial</code> , <code>set_serial_step</code> , <code>set_serial_step_list</code> , <code>get_list_serial</code> , <code>set_max_counts</code> , <code>get_counts</code> , <code>load_text_table</code> , <code>set_text_table_pointer</code> , <code>mark_serial_abs</code>

Normal List Command	mark_serial_abs
Function	Marks the current serial number of the serial-number-set most recently selected by select_serial_set_list (or of serial-number-set 0 after load_program_file) in the selected format at the current position. In the called indexed text strings or characters (see below), any absolute Vector commands and "Arc" commands receive an offset (based on the current coordinates at the time of the call). Afterward the serial number is (optionally) automatically incremented.
Call	<code>mark_serial_abs(Mode, Digits)</code>
Parameters	Mode See mark_serial .
	Digits See mark_serial .
Comments	• mark_serial_abs has the same effect as mark_serial; however, internal calling of the indexed text strings (or characters) is by list_call_abs instead of list_call. • If the command lists of the called indexed text strings (or characters) contain no absolute commands, then there is no difference between mark_serial_abs and mark_serial.
RTC4→RTC5	New command.
Version info	–
References	mark_serial

Variable List Command	mark_text
Function	Marks a null-terminated string.
Call	<code>mark_text(Text)</code>
Parameters	Text PC memory address of the first character (byte) of the to-be-marked text string. As a pointer to a null-terminated string.
Comments	• The to-be-marked text (character sequence, byte array, null-terminated string) must be terminated with a \0 character (0 byte, NULL). The 0\ character itself is not marked. • When a mark_text is loaded, the to-be-marked text (if more than 12 characters in length, \0 not included) is split into blocks of 12 characters, with each block receiving its own mark_text in the list memory (keep this in mind to prevent unintended overflow of the corresponding list memory area). During processing of the individual mark_textmark_text, the corresponding mark_char commands (indexed characters) are executed in accordance with the selected character set. • The desired character set can be selected prior to the mark_text by select_char_set. For the default setting, character set 0 is used. If select_char_set is used within a called indexed character, then all subsequently called indexed characters are marked using this character set. • If the end of a list ("List 1" or "List 2") is reached during loading of a mark_text, then loading continues at the start of the corresponding list. In contrast, loading in the protected area (as part of an indexed subroutine) is aborted (get_last_error return code RTC5_REJECTED) and the indexed subroutine is not stored. • Absolute Vector commands and "Arc" commands execute absolutely after being called by mark_text. If the text string needs to execute at various locations within the Image field, then either the indexed character definitions can only contain relative [*]mark[*] Commands, "Arc" commands or Jump commands or mark_text_abs must be used instead. • The mark_text should not be used within an indexed character definition. The corresponding text is <i>not</i> marked and the indexed character is therefore not fully processed.
RTC4→RTC5	New command.
Version info	–
References	mark_text_abs

Variable List Command	mark_text_abs
Function	Marks a null-terminated string. In the called indexed characters (see below), any absolute Vector commands and "Arc" commands receive an offset (based on the current coordinates at the time of the call).
Call	<code>mark_text_abs(Text)</code>
Parameters	Text PC memory address of the first character (byte) of the to-be-marked text string. As a pointer to a null-terminated string.
Comments	• During processing of the individual mark_text_abs commands, the corresponding mark_char_abs commands (indexed characters) are executed in accordance with the selected character set. • If the command list of the called indexed character contains no absolute commands, then there is no difference between mark_text_abs and mark_text.
RTC4→RTC5	New command.
Version info	–
References	mark_text

Normal List Command	mark_time
Function	Marks a part of the time previously stored by time_fix , time_fix_f or time_fix_f_off in the specified format at the current position.
Call	<code>mark_time(Part, Mode)</code>
Parameters	Part Specifies which part of the time to mark. = 0: Hours (24-h time, no a.m./p.m.) = 1: Minutes = 2: Seconds = 3: Hours (12-h time, no a.m./p.m.) = 4: a.m./p.m. (automatically switched in accordance with 24-h time) As an unsigned 32-bit value. Allowed value range: [0...4]. Mode Format. As an unsigned 32-bit value. Allowed value range: [0...3], only affects Part = 0...3). a) Number of leading zeros: Bit #0 = 0: Leading zeros are suppressed. Bit #0 = 1: Always two digits. b) Character set choice for marking of digits: Bit #1 = 0: The indexed text strings for digits [0...9], as defined by load_text_table or set_text_table_pointer, are marked. Bit #1 = 1: The indexed characters for digits [0...9], as defined by load_char or set_char_pointer, are marked. The desired character set can be specified with select_char_set. a.m./p.m. (Part = 4) can only be marked in accordance with the indexed text strings (Index = 40, 41) defined by load_text_table or set_text_table_pointer. Here, the parameter Mode is ignored.
Comments	- Before marking times (after every boot-up), the RTC5 and PC times should be synchronized (see time_update) and the current (to be marked) time should be stored with time_fix, time_fix_f or time_fix_f_off (see Chapter 7.5 "Marking Dates, Times and Serial Numbers", Page 196). - The complete time can be marked by multiple calls of mark_time.

Normal List Command	mark_time
Comments (cont'd)	- The mark_time command reads (according to the stored time and according to the selected time part) the starting address of the corresponding indexed text string (or – for Part = 0... 3 and Mode = 2 or 3 – of the corresponding indexed character) from the internal management table and then calls list_call (see also the comments there) an appropriate number of times, which starts the corresponding command list. The command lists must contain marking instructions for digits 0...9 and for a.m./p.m. (see page 108). Non-defined text strings or characters are ignored (that is, not marked). The called indexed text strings can also contain calls to indexed characters (mark_char or mark_char_abs) and complete texts (mark_text or mark_text_abs). In the latter case, the character set can be switched, if needed (before marking with mark_time), by select_char_set (see also Chapter 6.5.2 "Character Sets and Text Strings", Page 107). - If Part > 4 and/or Mode > 3, then mark_time is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). - Absolute Vector commands and "Arc" commands execute absolutely after being called with mark_time. If time markings need to execute at various locations within the Image field, then the corresponding indexed text strings (or characters) can only contain relative [*]mark[*] Commands, "Arc" commands or Jump commands or mark_time_abs must be used instead.
RTC4→RTC5	New command.
Version info	–
References	time_fix , time_fix_f , time_fix_f_off , load_text_table , set_text_table_pointer , mark_time_abs

Normal List Command	mark_time_abs
Function	Marks a part of the time previously stored by time_fix , time_fix_f or time_fix_f_off in the specified format at the current position. In the called indexed text strings or characters (see below), any absolute Vector commands and " Arc " commands receive an offset (based on the current coordinates at the time of the call).
Call	<code>mark_time_abs(Part, Mode)</code>
Parameters	Part See mark_time .
	Mode See mark_time .
Comments	• mark_time_abs has the same effect as mark_time. However, internal calling of the indexed text strings (or characters) is by list_call_abs instead of list_call. • If the command lists of the called indexed text strings (or characters) contain no absolute commands, then there is no difference between mark_time_abs and mark_time.
RTC4→RTC5	New command.
Version info	–
References	mark_time

Ctrl Command	mcbbsp_init
Function	Defines the data delays for transmitting and receiving data by the McBSP interface (Multi channel Buffered Serial Port, see also page 71).
Call	<code>mcbbsp_init(XDelay, RDelay)</code>
Parameters	XDelay Transmission delay. As an unsigned 32-bit value. Allowed value range: [0...2].
	RDelay Receiving delay. As an unsigned 32-bit value. Allowed value range: [0...2].
Comments	- For invalid parameter values mcbbsp_init is <i>not</i> executed (get_last_error return code RTC5_PARAM_ERROR). - The initialized values (after program start) are <code>XDelay = RDelay = 1</code>. - The signals and operating conditions of the McBSP interface are presented in Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71. - The McBSP interface ignores the first FrameSync signal after a mcbbsp_init. That is, data provided is not transmitted, see page 73.
RTC4→RTC5	New command.
Version info	–
References	read_mcbbsp , set_mcbbsp_out , set_mcbbsp_out_ptr , set_mcbbsp_freq , mcbbsp_init_spi

Ctrl Command	mcbbsp_init_spi
Function	Initializes the SPI functionality at the SPI / I2C Socket Connector (instead of McBSP functionality).
Call	<code>mcbbsp_init_spi(ClockLevel, ClockDelay)</code>
Parameters	ClockLevel = 0: inactive low. > 0: inactive high. As an unsigned 32-bit value. ClockDelay = 0: Clock signal and data bits at the same time. > 0: Clock signal is delayed a half period. As an unsigned 32-bit value.
Comments	• See Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71. • With the SPI protocol a common CLOCK signal, data input and data output occur simultaneously. The RTC5 is the SPI master. • mcbbsp_init_spi can be called at any time. Data transmissions in progress (started but not yet finished) are cancelled. • The clock frequency can be set in advance by set_mcbbsp_freq. • mcbbsp_init can be used to reestablish the McBSP functionality at the SPI / I2C Socket Connector at any time. • Irrespective of the clock frequency the SPI functionality only permits a single transmission every 10 μs (for example, see set_mcbbsp_in). • The McBSP interface ignores the first FrameSync signal after a mcbbsp_init_spi. That is, data provided is not transmitted, see page 73.
RTC4→RTC5	New command.
Version info	Available as of DLL 538, OUT 538.
References	mcbbsp_init , set_mcbbsp_freq

Ctrl Command	measurement_status
Function	Returns the status of a measurement session started by set_trigger or set_trigger4 and the current position of the measurement counter.
Call	measurement_status(&Busy, &Pos)
Returned parameter values	Busy Measurement status. As a pointer to an unsigned 32-bit value.. > 0: A measurement session is currently in progress. = 0: No measurement session is currently in progress.
	Pos Current position of the measurement counter (within the RTC5 measurement storage area) ($0 \leq \text{Pos} \leq 2^{20}-1$ or $0 \leq \text{Pos} \leq (2^{19}-1)$ or $\text{Pos} = 2^{32}-1$, see below). As a pointer to an unsigned 32-bit value.
Comments	• If a measurement session started with set_trigger or set_trigger4 is no longer active, then $\text{Pos}+1$ indicates the number of recorded data pairs up to termination (maximum 2^{20} for set_trigger, maximum 2^{19} for set_trigger4). • $\text{Pos} = 2^{32}-1$ indicates that data recording still has not occurred after load_program_file. • Stored data can be queried with get_waveform. • The status of a measurement session is reset by set_trigger(Period = 0), set_trigger4(Period = 0), stop_trigger or stop_execution (see set_trigger comments).
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_trigger , set_trigger4

Normal List Command	micro_vector_abs	
Function	Moves the output point (of the laser focus) by a “Hard jump” (without split-up into Microsteps) directly from the current position to the specified position (absolute coordinate values) within a 2D Image field.	
Call	micro_vector_abs(X, Y, LasOn, LasOff)	
Parameters	X	Absolute x coordinate of the micro vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y	Like X (analogously).
	LasOn	LaserOn Delay. 1 Bit equals 0.5 μs. As a signed 32-bit value. Allowed value range: [-2³¹...+(2¹⁵-1)]. ≥ 0: Delay is newly set. Values over (2¹⁵-1) are clipped. < 0: The previously set delay continues unaffected.
	LasOff	LaserOff Delay. Like LasOn.
Comments	• See also Chapter 8.8 “micro_vector[*] Commands”, Page 249. • Wobbel is not taken into account, see 2 in Chapter 7.3.6 “Output Values to the Scan System”, Page 170. • The Microvector is always executed as a “Hard jump”. • By LasOn ≥ 0 and LasOff ≥ 0, you can set a new LaserOn Delay or LaserOff Delay for each individual Microvector. Each delay thereby gets set at the end of the clock cycle in which the new position actually gets outputted (this output clock cycle is delayed by a preceding scanner delay). • Negative values (LasOn < 0 and LasOff < 0) do not affect Laser Delays. Hereby, the laser can remain on or off across multiple clock cycles (mark and jump simulation). • Delays set with LasOn and LasOff only apply to the execution of Microvectors. For execution of normal [*]mark[*] Commands and “Arc” commands (such as mark_abs), only the Laser Delays defined by set_laser_delays apply. LasOn and LasOff do not overwrite the laser delay parameter from set_laser_delays.	
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16, those for LasOn and LasOff by 2. The allowed value ranges decrease accordingly.	
Version info	–	
References	micro_vector_rel, micro_vector_abs_3d	

Normal List Command	micro_vector_abs_3d
Function	Moves the output point (of the laser focus) by a “ Hard jump ” (without split-up into Microsteps) directly from the current position to the specified position (absolute coordinate values) within the 3D image field .
Restriction	If the Option “3D” is not enabled or no 3D correction table has been assigned (see select_cor_table), then micro_vector_abs_3d has the same effect as micro_vector_abs .
Call	micro_vector_abs_3d(X, Y, Z, LasOn, LasOff)
Parameters	X Absolute x coordinate of the micro vector end point. In bits. As a signed 32-bit value. Allowed value range: [–8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	Z Like X (analogously), except Allowed value range: [–32,768...+32,767].
	LasOn LaserOn Delay . 1 Bit equals 0.5 μ s. As a signed 32-bit value. Allowed value range: [–2 ³¹ ...+(2 ¹⁵ –1)]. ≥ 0 : Delay is newly set. Values over (2 ¹⁵ –1) are clipped. < 0 : The previously set delay continues unaffected.
	LasOff LaserOff Delay . 1 bit equals 1/64 μ s. Like LasOn.
Comments	Except for the additional motion in the third dimension, micro_vector_abs_3d functions similarly to micro_vector_abs (see comments there).
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16, those for LasOn and LasOff by 2. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode.
Version info	–
References	micro_vector_abs , micro_vector_rel_3d

Normal List Command	micro_vector_rel
Function	Moves the output point (of the laser focus) by a “ Hard jump ” (without split-up into Microsteps) directly from the current position to the specified position (relative coordinate values) within a 2D Image field .
Call	<code>micro_vector_rel(dX, dY, LasOn, LasOff)</code>
Parameters	dX Relative x coordinate of the micro vector end point. In bits. Otherwise, like x from micro_vector_abs .
	dY Relative y coordinate of the micro vector end point. In bits. Otherwise, like y from micro_vector_abs .
	LasOn Like LasOn from micro_vector_abs .
	LasOff Like LasOff from micro_vector_abs .
Comments	• The coordinates for the micro vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, micro_vector_rel is identical to micro_vector_abs (see comments there). • The Microvector is always executed as a “Hard jump”.
RTC4→RTC5	New command. RTC4 Compatibility Mode: see micro_vector_abs .
Version info	–
References	micro_vector_abs , micro_vector_rel_3d

Normal List Command	micro_vector_rel_3d
Function	Moves the output point (of the laser focus) by a “Hard jump” (without split-up into Microsteps) directly from the current position to the specified position (relative coordinate values) within a 3D image field.
Restriction	If the Option “3D” is not enabled or no 3D correction table has been assigned (see select_cor_table), then micro_vector_rel_3d has the same effect as micro_vector_rel.
Call	micro_vector_rel_3d(dX, dY, dZ, LasOn, LasOff)
Parameters	dX Relative x coordinate of the micro vector end point. In bits. Otherwise, like wie X von micro_vector_abs_3d.
	dY Relative y coordinate of the micro vector end point. In bits. Otherwise, like wie Y von micro_vector_abs_3d.
	dZ Relative z coordinate of the micro vector end point. In bits. Otherwise, like wie Z von micro_vector_abs_3d.
	LasOn Like LasOn from micro_vector_abs_3d.
	LasOff Like LasOff from micro_vector_abs_3d.
Comments	- The coordinates for the micro vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, micro_vector_rel_3d is identical to micro_vector_abs_3d (see comments there). - The Microvector is always executed as a “Hard jump”.
RTC4→RTC5	New command. RTC4 Compatibility Mode: see micro_vector_abs_3d.
Version info	–
References	micro_vector_abs_3d, micro_vector_rel

Ctrl Command	move_to
Comments	- move_to is described in the manual “Installation and Operation RTC Step Motor Extension for the RTC4 and RTC5 PC interface boards” (available in English only). - move_to has been introduced for the extension board “RTC4 STEP MOTOR EXTENSION” (#0112097). - move_to is not supported by “RTC5/6 varioSCAN 40 FLEX Extension” extension board (#0128683). See also Chapter 2.8.8 “Extension Board “RTC5/6 varioSCAN FLEX Extension””, Page 39.

Ctrl Command	number_of_correction_tables
Function	Defines the maximum number of allowed correction tables.
Call	<code>number_of_correction_tables(Number)</code>
Parameters	Number Maximum number of allowed correction tables. As an unsigned 32-bit value. Allowed value range: [1...4]. Default after load_program_file: 4. See also Chapter 8.5.3 "Using Several Correction Tables", Page 226.
Comments	• For outside the allowed value range, number_of_correction_tables is ignored (get_last_error return code RTC5_PARAM_ERROR). • number_of_correction_tables serves to protect other commands (for example, such as load_correction_file and select_cor_table) from unwanted table numbers. • Existing user programs do not have to be changed. The exception is, if user input is to be rejected (using explicit RTC5 error messages) in the future. • number_of_correction_tables refers only to subsequent command executions. Existing assignments of correction tables cannot be corrected automatically. This is particularly important, if RTC5 boards that have been initialized by other user programs are subsequently acquired. • On request: 8 allowed correction tables, see Footnote, page 164.
RTC4→RTC5	New command.
Version info	Available as of DLL 540.
References	load_correction_file , select_cor_table , select_cor_table_list

Normal List Command	para_jump_abs
Function	Moves the output point (of the laser focus) along a 2D vector at jump speed from the current position to the specified position (absolute coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>para_jump_abs(X, Y, P)</code>
Parameters	X Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	• If vector-defined laser control has not been previously activated by set_vector_control, then para_jump_abs behaves like jump_abs (see comments there). The parameter P is then ignored. • If vector-defined laser control is activated, then simultaneously with the motion of the output point of the laser focus the signal parameter selected by set_vector_control is linearly varied from the last valid value to P, see Section "Vector-Defined Laser Control", Page 194. • There is no abs mechanism for P. P is clipped to the maximum allowed value. • If para_jump_abs is used along with position-dependent or speed-dependent laser control for the same control parameter, then the current value of P is used as the basis of the 100% value for laser control (see Section "Vector-Defined Laser Control", Page 194).
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for X and Y by 16. The allowed value ranges decrease accordingly. There is no RTC4 Compatibility Mode for the parameter P . The original RTC5 units must be used.
Version info	–
References	jump_abs , set_vector_control , para_jump_abs_3d , para_jump_rel

Normal List Command	para_jump_abs_3d
Function	Moves the output point (of the laser focus) along a 3D vector at jump speed from the current position to the specified position (absolute coordinate values) within a 3D image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then para_jump_abs_3d has the same effect as para_jump_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>para_jump_abs_3d(X, Y, Z, P)</code>
Parameters	X Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	Z Like X (analogously), except Allowed value range: [-32,768...+32,767].
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	- Except for the additional motion in the third dimension, para_jump_abs_3d functions similarly to para_jump_abs (see the comments there). - Further comments see jump_abs_3d.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode. For the parameter P see para_jump_abs.
Version info	–
References	jump_abs_3d , set_vector_control , para_jump_abs , para_jump_rel_3d

Normal List Command	para_jump_rel
Function	Moves the output point (of the laser focus) along a 2D vector at jump speed from the current position to the specified position (relative coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>para_jump_rel(dX, dY, P)</code>
Parameters	dX Relative x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like dX (analogously).
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	• The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, para_jump_rel is identical to para_jump_abs (see comments there). • P is not treated on a relative basis.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly. For the parameter P , see para_jump_abs .
Version info	–
References	jump_rel , set_vector_control , para_jump_abs , para_jump_rel_3d

Normal List Command	para_jump_rel_3d
Function	Moves the output point (of the laser focus) along a 3D vector at jump speed from the current position to the specified position (relative coordinate values) within a 3D image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then para_jump_rel_3d has the same effect as para_jump_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>para_jump_rel_3d(dX, dY, dZ, P)</code>
Parameters	dX Relative x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like dX (analogously).
	dZ Like dX (analogously), except Allowed value range: [-32,768...+32,767].
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	- The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, para_jump_rel_3d is identical to para_jump_abs_3d (see the comments there). P is not treated on a relative basis.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly. The value range for dZ is identical in RTC5 Standard Mode and RTC4 Compatibility Mode. The parameter P does not have a RTC4 Compatibility Mode. The original RTC5 units must be used.
Version info	–
References	jump_rel_3d , set_vector_control , para_jump_abs_3d , para_jump_rel

Variable List Command	<code>para_laser_on_pulses_list</code>
Function	Turns on the LASERON signal for the specified number of external signal pulses (but for no longer than the specified time interval) and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>para_laser_on_pulses_list(Period, Pulses, P)</code>
Parameters	Period Time interval. In bits. As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $0 \leq \text{Period} \leq (2^{32}-1)$.
	Pulses Number of external signal pulses. As an unsigned 32-bit value. Allowed value range: $0 \leq \text{Pulses} \leq 65,535$ or larger (see comments below).
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed value range: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	para_laser_on_pulses_list is useful for marking single dots, see Chapter 7.1.3 "Marking Single Dots", Page 129. - If vector-defined laser control has not been previously activated by set_vector_control, then para_laser_on_pulses_list behaves like laser_on_pulses_list and – if Pulses > 65,535 – like laser_on_list (see comments there). The parameter P is then ignored. - If vector-defined laser control is activated, then the signal parameter selected by set_vector_control is linearly varied from the last valid value to P within the para_laser_on_pulses_list duration (Period $\times$ 10 μs) (see Section "Vector-Defined Laser Control", Page 194). - There is no abs mechanism for P. P is clipped to the maximum allowed value. This maximum value is $(2^{31}-1)$ or a lower value depending on what has been selected with set_vector_control (Ctrl parameter). - If para_laser_on_pulses_list is used along with position-dependent or speed-dependent laser control for the same control parameter, then the current value of P is used as the basis of the 100% value for laser control (see Section "Vector-Defined Laser Control", Page 194). - If Period = 0, para_laser_on_pulses_list has no effect. Then para_laser_on_pulses_list is a short list command. - If $0 < \text{Period} \leq (2^{31}-1)$, then the para_laser_on_pulses_list duration is always Period clocks (that is, Period $\times$ 10 μs), even if the specified number of external signal pulses expires in a shorter time interval. - If $2^{31} \leq \text{Period} \leq (2^{32}-1)$, the para_laser_on_pulses_list maximum duration is (Period – 2^{31}) clocks (that is, (Period – 2^{31}) $\times$ 10 μs). Here, however, para_laser_on_pulses_list terminates as soon as the specified number of external signal pulses has been detected.

Variable List Command	para_laser_on_pulses_list
RTC4→RTC5	New command. For the parameter <i>p</i> , see para_jump_abs .
Version info	–
References	laser_on_pulses_list , laser_on_list

Normal List Command	para_mark_abs
Function	Moves the laser focus at mark speed along a 2D vector from the current position to the specified position (absolute coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	para_mark_abs(X, Y, P)
Parameters	X Absolute x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	• If vector-defined laser control has not been previously activated by set_vector_control, then para_mark_abs behaves like mark_abs (see comments there). The parameter P is then ignored. • If vector-defined laser control is activated, then simultaneously with the laser focus' motion the signal parameter selected by set_vector_control is linearly varied from the last valid value to P (see Section "Vector-Defined Laser Control", Page 194). • There is no abs mechanism for P. P is clipped to the maximum allowed value. • For mark vector lengths = 0, no change of signal parameter P must be programmed: – This change is not outputted – The following [*]para[*] Command (only this one) produces incorrect signal parameter outputs • If para_mark_abs is used along with position-dependent or speed-dependent laser control for the same control parameter, then the current value of P is used as the basis of the 100% value for laser control (see Section "Vector-Defined Laser Control", Page 194). • [*]para_mark[*] commands generally do not take Sky Writing into account.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly. For the parameter P, see para_jump_abs.
Version info	–
References	mark_abs , set_vector_control , para_mark_abs_3d , para_mark_rel

Normal List Command	para_mark_abs_3d
Function	Moves the laser focus at mark speed along a 3D vector from the current position to the specified position (absolute coordinate values) within the 3D image field . Simultaneously varies the signal parameter selected by set_vector_control linearly to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then para_mark_abs_3d has the same effect as para_mark_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	para_mark_abs_3d(X, Y, Z, P)
Parameters	X Absolute x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	Y Like X (analogously).
	Z Like X (analogously), except Allowed value range: [-32,768...+32,767].
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	- Except for the additional motion in the third dimension, para_mark_abs_3d functions similarly to the para_mark_abs command (see the comments there). - Further comments see mark_abs_3d. [*]para_mark[*] commands generally do not take Sky Writing into account.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly. The value range for Z is identical in RTC5 Standard Mode and RTC4 Compatibility Mode. For the parameter P, see para_jump_abs.
Version info	–
References	mark_abs_3d , set_vector_control , para_mark_abs , para_mark_rel_3d

Normal List Command	para_mark_rel
Function	Moves the laser focus at mark speed along a 2D vector from the current position to the specified position (relative coordinate values) within a 2D Image field . Simultaneously varies the signal parameter selected by set_vector_control linearly to the specified value.
Call	<code>para_mark_rel(dX, dY, P)</code>
Parameters	dX Relative x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like dX (analogously).
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	• The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, para_mark_rel is identical to para_mark_abs (see comments there). • P is not treated on a relative basis. • [*]para_mark[*] commands generally do not take Sky Writing into account.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for dX and dY by 16. The allowed value range decreases accordingly. For the parameter P see para_jump_abs .
Version info	–
References	mark_rel , set_vector_control , para_mark_abs , para_mark_rel_3d

Normal List Command	para_mark_rel_3d
Function	Moves the laser focus at mark speed along a 3D vector from the current position to the specified position (relative coordinate values) within the 3D image field . Simultaneously varies the signal parameter selected by set_vector_control linearly to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then para_mark_rel_3d has the same effect as para_mark_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	para_mark_rel_3d(dX, dY, dZ, P)
Parameters	dX Relative x coordinate of the mark vector end point. In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values. The complete value range is only usable as a virtual Image field for example, for (enabled) Processing-on-the-fly applications.
	dY Like X (analogously).
	dZ Like X (analogously), except Allowed value range: [-32,768...+32,767].
	P End value of the signal parameter. As an unsigned 32-bit value. Allowed values: dependent on the selection made by set_vector_control (Ctrl parameter), identical with set_vector_control (Value parameter).
Comments	- The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, para_mark_rel_3d is identical to para_mark_abs_3d (see the comments there). - P is not treated on a relative basis. [*]para_mark[*] commands generally do not take Sky Writing into account.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for dX and dY by 16. The allowed value range decreases accordingly. The value range for dZ is identical in RTC5 Standard Mode and RTC4 Compatibility Mode. For the parameter P, see para_jump_abs.
Version info	–
References	mark_rel_3d , set_vector_control , para_mark_abs_3d , para_mark_rel

Variable List Command	park_position
Function	For temporary parking, this command moves the output point (of the laser focus) along a 2D vector at jump speed from the current position to the specified position (absolute coordinate values) within the 2D Image field .
Restriction	If the Option Processing-on-the-fly is not enabled, then park_position functions like a normal jump_abs .
Call	<code>park_position(Mode, X, Y)</code>
Parameters	Mode As an unsigned 32-bit value. = 0: X and Y are interpreted as park position coordinates in the real Image field. Allowed value range: [-524,288...524,287]. The current Processing-on-the-fly mode is switched off and the laser focus moved at the currently set jump speed to the specified park position in the real Image field. During a subsequent list interruption (by wait_for_encoder etc.), the galvanometer scanners remain stationary (even for a Processing-on-the-fly application initiated by set_fly_2d). > 0: X and Y are interpreted as park position coordinates in the virtual Image field. Allowed value range: [-8388,608...8388,607]. The current Processing-on-the-fly mode remains switched on and the laser focus moved at the currently set jump speed to the specified park position in the virtual Image field (subject to Processing-on-the-fly correction and clipped to the real Image field's boundaries). During a subsequent list interruption (by wait_for_encoder etc.), the positions of the galvanometer scanners are continuously Processing-on-the-fly-corrected in accordance with the current encoder values (even for a Processing-on-the-fly application initiated by set_fly_x/set_fly_y) and clipped to the real Image field's boundaries.
	X Absolute x coordinate of the jump vector end point. In bits. As a signed 32-bit value. Allowed value range: see above. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
Comments	park_position is intended for encoder-based set_fly_2d or set_fly_x/set_fly_y Processing-on-the-fly applications where the laser focus needs to be moved to a safe parking area during an intermediate motion (prior to list interruption by wait_for_encoder etc.) (see Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237). For the return jump (away from the park position), use the park_return command (refer to all comments there). - If another (or no) Processing-on-the-fly application is active, then park_position functions like a normal Jump command, as does the return jump by park_return (but Processing-on-the-fly remains switched off).

Variable List Command	park_position
Comments (cont'd)	• If the laser focus has been already previously moved to a safe park position, then park_position is a short list command with no effect. • park_position switches off the Signals for “Laser Active” Operation after a LaserOff Delay, but does not activate a scanner delay afterward.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	park_return

Variable List Command	park_return
Function	Moves the output point (of the laser focus) away from a park position along a 2D vector at jump speed to the specified position (absolute coordinate values) within the 2D Image field .
Restriction	If the Option Processing-on-the-fly is not enabled, then park_return functions as a normal Jump command . If the laser focus is not currently in a park position, then park_return is a short list command with no effect (see below).
Call	<code>park_return(Mode, X, Y)</code>
Parameters	Mode As an unsigned 32-bit value. = 0: X and Y are ignored (see comments). > 0: X and Y are interpreted as return jump coordinates (see comments).
	X Absolute x coordinate of the jump vector end point in the virtual Image field . In bits. As a signed 32-bit value. Allowed value range: [-8,388,608...+8,388,607]. Out-of-range values are clipped to the boundary values.
	Y Like X (analogously).
Comments	park_return is intended for encoder-based set_fly_2d or set_fly_x/set_fly_y Processing-on-the-fly applications where the laser focus should leave a safe parking area previously reached by park_position after an intermediate motion (following list interruption by wait_for_encoder etc.) (see Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237, see also comments at park_position). - If a set_fly_2d or set_fly_x/set_fly_y Processing-on-the-fly mode has been switched off by a prior park_position(<i>Mode</i> = 0) call, then park_return switches the same Processing-on-the-fly mode back on. Other Processing-on-the-fly modes are not switched back on. - If the park position has been attained by park_position, then park_return(<i>Mode</i> = 0) returns the galvanometer scanners to the most recent valid position before park_position has been called, whereas park_return(<i>Mode</i> = 1) moves them to the position specified with the command. When calculating the new position, the RTC5 takes the current Processing-on-the-fly correction into account. But if clipping to the boundaries of the virtual Image field occurs (for example, due to a too long intermediate motion during a list interruption by wait_for_encoder), then the Processing-on-the-fly mode <i>is not</i> reactivated (see also activate_fly_2d and activate_fly_xy) and the jump executes without Processing-on-the-fly correction.

Variable List Command	park_return
Comments (cont'd)	• If the laser focus <i>has not been</i> previously moved to a safe area by park_position, then park_return is a short list command with no further effect. • If no Processing-on-the-fly mode has been previously active or if any Processing-on-the-fly mode is currently active, then park_return performs a normal jump to the specified location. • If a coordinate transformation in the virtual Image field is active, then the specified or most recent valid position is subsequently appropriately transformed (see page 158). • A scanner Jump Delay is activated after a park_return. • park_return switches off the Signals for "Laser Active" Operation after a LaserOff Delay.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly.
Version info	–
References	park_position

Ctrl Command	pause_list
Function	Pauses execution of the list and disables the Signals for "Laser Active" Operation .
Call	<code>pause_list()</code>
Parameters	–
Comments	• pause_list is synonymous with stop_list. stop_list is often confused with stop_execution. Therefore, it is preferable to use pause_list. • If pause_list is called during execution of a list, then the Signals for "Laser Active" Operation are suppressed (the signals are set to their respective "Off" level) and keeps the scan system in the most recently defined state – even if in the middle of a split-up into Microsteps. The PAUSED list execution status (queryable by get_status) is set, but the BUSY list execution status is left unchanged. Continuation by execute_list_pos or release_wait or by an External Start is not possible. However, stop_execution or an External Stop is possible. restart_list, stop_execution or an External Stop ends suppression of the start. • If processing of a list should be continued, then restart_list <i>must</i> be used. After a subsequent restart_list, the scan system resumes the planned motions (of the current command) and the laser control signals are released again (in general, an interrupted marking cannot be continued without a disruption in the marking result). The PAUSED list execution status is then reset (here too, BUSY list execution status remains unchanged). • If, during calling of pause_list, no list is currently executing (BUSY list execution status not set) or a list has already been halted by pause_list or set_wait (PAUSED list execution status set), then pause_list is ignored (get_last_error return code RTC5_BUSY; the laser is then already off).
RTC4→RTC5	New command.
Version info	–
References	restart_list , set_wait

Ctrl Command	periodic_toggle
Function	Like periodic_toggle_list , but a control command.
Call	<code>periodic_toggle(Port, Mask, P1, P2, Count, Start)</code>
Parameters	Port Like periodic_toggle_list. Mask Like periodic_toggle_list. P1 Like periodic_toggle_list. P2 Like periodic_toggle_list. Count Like periodic_toggle_list. Start Like periodic_toggle_list.
Comments	• See periodic_toggle_list. • If an unallowed parameter value is supplied for <code>Port</code> or 0 for <code>P1</code> or <code>P2</code>, then periodic_toggle is not executed (get_last_error return code <code>RTC5_PARAM_ERROR</code>). • See also Chapter 9.4 "Periodical I/O Signals", Page 277.
RTC4→RTC5	New command.
Version info	Last change DLL 543, OUT 543: endless toggling with <code>Count = 2³²-1</code> .
References	periodic_toggle_list

Undelayed Short List Command	periodic_toggle_list
Function	Generates and periodically toggles a signal at an adjustable output port.
Call	<code>periodic_toggle_list(Port, Mask, P1, P2, Count, Start)</code>
Parameters	Port Output port (0...4, as with set_port_default). As an unsigned 32-bit value. Mask Defines the bits to be toggled, (the maximum value depends on the port). 1 = bit is toggled. 0 = bit remains unchanged. As an unsigned 32-bit value. P1 Duration of the toggled signal in [10 μs] (> 0, 16 bit max.). As an unsigned 32-bit value. P2 Duration of the toggled signal in [10 μs] (> 0, 16 bit max.). As an unsigned 32-bit value. Count Number of P1–P2 period repetitions. As an unsigned 32-bit value. Start Duration until the first toggling starts in [10 μs]. As an unsigned 32-bit value.
Comments	• If an unallowed parameter value is supplied for Port or 0 for P1 or P2, then periodic_toggle_list is replaced by a list_nop and a get_last_error return code RTC5_PARAM_ERROR is generated. • Mask defines the bits to be toggled. Bits which are set to 1 in Mask are toggled and bits which are set to 0 remain unchanged. Mask is limited to the maximum allowed value of the selected port (see also set_port_default). Extra bits are ignored. • The selected bits are toggled. This state is maintained for $P1 \times 10 \mu s$ clock cycles. Then they are toggled once again and kept stable for $P2 \times 10 \mu s$ clock cycles. Users define the toggle bit start values ("off" state; "on" signal is active-HIGH or active-LOW) by other standard commands (write_da_x, write_8bit_port, write_io_port, etc.). These – and (should the situation arise) other queued delayed short list commands – are executed before periodic_toggle_list. • The first toggling occurs after $Start \times 10 \mu s$ clock cycles. Toggling is repeated Count-times. Count = 0 immediately stops an output in progress. The remaining parameters are then not relevant. If the stop happens in the P1 period ("On"), a toggle occurs to restore the initial state ("Off"). • The parameters P1 and P2 are clipped to the value range [0...65,535]. P1 and P2 must not be 0 at the same time. • The selected port should not concurrently be used for other outputs such as "Automatic Laser Control". • At a set_end_of_list, stop_execution or external /STOP the periodical signals continue. • As of version DLL 543, OUT 543, periodic_toggle_list toggles endless with $Count = 2^{32}-1$. • See also Chapter 9.4 "Periodical I/O Signals", Page 277.

Undelayed Short List Command	periodic_toggle_list
RTC4→RTC5	New command.
Version info	Last change DLL 543, OUT 543: endless toggling with $\text{Count} = 2^{32}-1$.
References	periodic_toggle , set_port_default

Ctrl Command	quit_loop
Function	Stops the repeating automatic list change started with start_loop .
Call	<code>quit_loop()</code>
Comments	- Before list execution is stopped, the current list is fully executed until the next set_end_of_list is encountered. start_loop must be called prior to quit_loop. Otherwise, quit_loop has no effect.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	start_loop

Undelayed Short List Command	range_checking	
Function	Defines an emergency action if a galvanometer scanner exceeds its position limit.	
Call	range_checking(HeadNo, Mode, Data)	
Parameters	HeadNo	Scan head to be monitored. As an unsigned 32-bit value. 0: No monitoring (default after load_program_file). 1: First scan head is monitored. 2: Second scan head is monitored. 3: Both scan heads are monitored. Only the 2 least significant bits are evaluated.
	Mode	Action to take. As an unsigned 32-bit value. 0: The Signals for "Laser Active" Operation are suppressed immediately. List execution continues. 1: The Signals for "Laser Active" Operation are switched-off immediately. List execution is stopped immediately (as with stop_execution or /STOP).
	Data	Data type chosen to be monitored. As an unsigned 32-bit value. 0: Sample values. Like set_trigger signal types 7...9. 1: Transformed control values. Like set_trigger signal types 25...30. 2: Corrected control values. Like set_trigger signal types 10...15. 3: Actual output values. Like set_trigger signal types 20...23. 4: Actual position of galvanometer scanners (only with iDRIVE systems, varioSCAN is internally connected to an x axis). Like set_trigger signal types [1, 2 and 4] or [4, 5 and 1]. 5: Actual position of galvanometer scanners (only with iDRIVE systems, varioSCAN is internally connected to a y axis). Like set_trigger signal types [1, 2 and 5] or [4, 5 and 2].
Comments	• No monitoring takes place if the specified scan head does not have a correction table assigned (see select_cor_table).	

Undelayed Short List Command	range_checking
Comments (cont'd)	- The used position limits (that is, if exceeded the emergency action is executed) are the parameters of a customer-specific Processing-on-the-fly application monitoring (see set_fly_limits, set_fly_limits_z). Exceeding these limits only cause error messages in case of Processing-on-the-fly applications. The error messages can be queried with get_marking_info or the respective if_fly_x_overflow/if_fly_y_overflow/if_fly_z_overflow and if_not_fly_x_overflow/if_not_fly_y_overflow/if_not_fly_z_overflow. They do not trigger any activities. - Processing-on-the-fly applications use the data type <code>Data = 0</code> (sample values) for customer-specific range limits. Inconsistent error messages and switch-offs may occur if used simultaneously with range_checking data types <code>Data > 0</code>. - If the laser has been switched-off with <code>Mode = 0</code> and the fault condition is gone then the laser is <i>not</i> switched on until the next Mark command. The laser is <i>not</i> switched-on right in the middle of a currently executing Mark command, not even in the middle of a Polyline. - The range_checking monitoring is active without Processing-on-the-fly application. <code>Data > 5</code> causes a get_last_error <code>RTC5_PARAM_ERROR</code> error code. In this case, range_checking is sent to the RTC5 Board as list_nop. <code>Data=2</code> and <code>Data=3</code> have the identical effect for the z axis. - For <code>Data = 4</code> and <code>Data = 5</code> users must set the set position as the to-be-returned data type by the scan system. A simultaneous use of a speed-dependent laser control with <code>Mode = 2</code> (see set_auto_laser_control) is not possible. <code>Data = 4</code> and <code>Data = 5</code> only differ in regards to the axis onto an varioSCAN is connected. For 2D systems without varioSCAN both selections have the identical effect. <code>Data = 4</code> and <code>Data = 5</code> produce nonsensical switch-offs if an intelliSCAN is connected but is either not switched-on or a actual position has not been set as feedback.
RTC4→RTC5	New command.
Version info	–
References	set_fly_limits , set_fly_limits_z

Ctrl Command	read_abc_from_file	
Function	Reads the ABC values directly from a specified correction file on the PC.	
Call	ErrorNo = read_abc_from_file(Name, &A, &B, &C)	
Result	ErrorNo	Error code. As an unsigned 32-bit value. 0 No error. 1 File error (corrupt or incomplete). 2 Memory error (RTC5 DLL-internal, Windows working memory). 3 File-open error (empty string, file not found etc.).
Parameters	Name	Name of the correction file. As a pointer to a null-terminated ANSI string.
Returned parameter values	A B C	Coefficients of the parabolic function $z_{out} = A + B/I + C/I^2$ which is used for calculating the Z output values (<i>I</i> in the RTC4 compatibility range [-32,768...+32,767]). As 64-bit IEEE floating point values.
Comments	• read_abc_from_file is available even without explicit access rights to a specific RTC5 board. • read_abc_from_file is not available as a multi-board command. • The board-specific error variables LastError and AccError, see Chapter 6.8 "Error Handling", Page 119, are neither generated nor altered by read_abc_from_file.	
RTC4→RTC5	New command.	
Version info	Available as of DLL 538, OUT 538.	
References	write_abc_to_file	

Ctrl Command	read_analog_in
Function	Reads in the analog input values: • ANALOG IN0 and ANALOG IN1 from the ADC Add-On Board of the RTC5 PCI Board • ANALOG IN0 and ANALOG IN1 from the SPI/I²C socket connector of the RTC5 PCIe Board Returns them as 12-bit digital values.
Call	AnalogValue = read_analog_in()
Result	As an unsigned 32-bit value. Bit #0 ANALOG IN0 input value as 12-bit digital value. ... Bit #11 Bit #12 = 0. Channel number. Bit #13 = 0. Bit #14 = 0. Bit #15 Old bit for ANALOG IN0. Bit #16 ANALOG IN1 input value as 12-bit digital value. ... Bit #27 Bit #28 = 1. Channel number. Bit #29 = 0. Bit #30 = 0. Bit #31 Old bit for ANALOG IN1.
Comments	• read_analog_in can only be used with RTC5 PCIe Boards and RTC5 PCI Boards having the ADC Add-On Board installed (see Chapter 4.6.8 "Analog Inputs (Accessory)", Page 75 and Chapter 16.2.4 "SPI/I²C Socket Connector", Page 742). • read_analog_in sets the old bits (Bit #15 and Bit #31) to 1 after reading. As soon new data are available at the corresponding analog input they are automatically set to 0. • As of version DLL 543, OUT 543, RBF 524: Analog input values (ANALOG IN0 and ANALOG IN1) can be recorded with signal 54 (see set_trigger/set_trigger4). However, old bits are not recorded. The format of the data is ((ANALOG IN1 << 16) + ANALOG IN0).
RTC4→RTC5	New command.
Version info	–
References	set_trigger , set_trigger4

Ctrl Command	read_encoder
Function	Returns the counts of the two RTC5 encoder counters that were stored by store_encoder .
Call	<code>read_encoder(&Encoder0_0, &Encoder1_0, &Encoder0_1, &Encoder1_1)</code>
Returned parameter values	Encoder0_0 Count. As a pointer to a signed 32-bit value. For parameter "Encoder $_n$ $_m$ ": n is the number of the encoder counter ("Encoder0", "Encoder1"). m is the number of the storage position (parameter Pos of store_encoder).
	Encoder1_0 Like Encoder0_0 (analogously).
	Encoder0_1 Like Encoder0_0 (analogously).
	Encoder1_1 Like Encoder0_0 (analogously).
Comments	- See also Chapter 8.6 "Processing-on-the-fly", Page 227 and Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274. - If the workpiece motion is registered with an incremental encoder: - Encoder counter "Encoder0" is triggered by the signals at encoder input port ENCODER X - Encoder counter "Encoder1" is triggered by the signals at encoder input port ENCODER Y - If an encoder simulation has been started by simulate_encoder(3), both encoder counters are triggered by an internal periodic 1 MHz clock signal. - For storage positions in which counts were not previously stored by store_encoder, the value 0 is returned (initialized value).
RTC4→RTC5	New command.
Version info	–
References	store_encoder, get_encoder, set_fly_x, set_fly_y, set_fly_rot, wait_for_encoder

Ctrl Command	read_io_port
Function	Returns the current state of the 16-bit digital input port on the EXTENSION 1 socket connector.
Call	<code>IOPort = read_io_port()</code>
Result	16-bit value (DIGITAL IN0...DIGITAL IN15). As an unsigned 32-bit value.
Comments	• See also Chapter 9.2.1 "16-Bit Digital Input Port", Page 264.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	write_io_port , write_io_port_mask , set_io_cond_list , clear_io_cond_list , read_io_port_buffer , read_io_port_list

Ctrl Command	read_io_port_buffer	
Function	Returns the data previously stored in the IOPort buffer by read_io_port_list (input value read at the EXTENSION 1 socket connector's 16-bit digital input; the galvanometer scanner set position and time value that has been current at the time of reading).	
Call	CurrentIndex = read_io_port_buffer(Index, &Value, &XPos, &YPos, &Time)	
Parameters	Index	Number of the to-be-returned entry in the IOPort buffer. As an unsigned 32-bit value. Only the 4 least significant bits are evaluated (the IOPort buffer only has 16 entries). Therefore, the user program can use a continuous counter for the index.
Returned parameter values	Value	16-bit value. As a pointer to an unsigned 32-bit value.
	XPos	Set position of the galvanometer scanner (x value). As a pointer to a signed 32-bit value.
	YPos	Set position of the galvanometer scanner (y value). As a pointer to a signed 32-bit value.
	Time	Time. In seconds. As a pointer to an unsigned 32-bit value.
Result	The index to which data is written upon the next read_io_port_list . As an unsigned 32-bit value.	
Comments	• See also comments for read_io_port_list. • If no read_io_port_list has been called before read_io_port_buffer, then the initialization values (default: 0) are returned.	
Example (C/C++)	Wait until the data under Index gets newly written: <pre>while (Index == read_io_port_buffer(Index, &Value, &XPos, &YPos, &Time));</pre> Read all data from Index to the current (not yet newly written) read position: <pre>while (Index != read_io_port_buffer(Index, &Value, &XPos, &YPos, &Time)) { Index = (Index+1) & 0xf; ...}</pre>	
RTC4→RTC5	New command.	
Version info	–	
References	read_io_port_list	

Undelayed Short List Command	read_io_port_list
Function	Writes into the internal IOPort buffer the current state (DIGITAL IN0...DIGITAL IN15) of the EXTENSION 1 socket connector's 16-bit digital input. At the same time, the current xy set positions and <i>relative</i> time in seconds are stored.
Call	<code>read_io_port_list()</code>
Comments	• read_io_port_list does not return values. However, the values can be subsequently queried from the IOPort buffer by read_io_port_buffer. • The IOPort buffer holds 16 entries and is circularly organized. It always stores the 16 most recent entries and overwrites older entries without warning. The incremental Index starts after a reset (program start) at Index 0, and after passing 15 does rollover to 0. • The stored time is the current value of an internal seconds counter that is set to 0 at program start (load_program_file) or by time_update. • See also Chapter 9.2.1 "16-Bit Digital Input Port", Page 264.
RTC4→RTC5	New command.
Version info	–
References	read_io_port_buffer , read_io_port

Ctrl Command	read_mcbbsp
Function	Returns the most recent input value that has been fully copied to an internal memory location by the McBSP interface .
Call	<code>mcbbsp_value = read_mcbbsp(No)</code>
Parameters	No Number of the internal memory location whose input value should be queried. As an unsigned 32-bit value. 0 Memory location for Processing-on-the-fly applications and for set_mcbbsp_in input with coding Bit #31 = 0. 1 Memory locations for Online Positioning and for set_mcbbsp_in input. 2 Wie 1. 3 Memory location for set_mcbbsp_in input with coding Bit #31 = 1. For set_multi_mcbbsp_in, the following applies: memory locations 0 through 3 are continuously written to as a ring buffer. Only the 2 least significant bits are evaluated.
Result	Input value. As a signed 32-bit value.
Comments	- For information on the McBSP interface and the related memory locations, see also Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71. - The interpretation as - one signed or unsigned 32-bit data word - two signed 16-bit data words - two signed 15-bit data words is the responsibility of the user: - For Processing-on-the-fly applications (No = 0) see Chapter 8.6 "Processing-on-the-fly", Page 227 and Chapter 9.3.4 "Synchronization and Online Positioning by McBSP/SPI Signals", Page 276. - For "Local Online Positioning" (No = 1...2) see Chapter 8.3.1 "Local Online Positioning", Page 213. - For "Global Online Positioning" (No = 1...2) see Chapter 8.3.2 "Global Online Positioning", Page 216. - For set_mcbbsp_in input (No = 0...3) see Section "Correction via McBSP Interface with Additional McBSP Input", Page 231 and Section "Correction via McBSP Interface with Additional McBSP Input", Page 233. - For set_multi_mcbbsp_in, see page 594. - After load_program_file, the input values at the McBSP interface are transferred to location 0 (default) even if no Processing-on-the-fly correction is activated. - The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbbsp_init. That is, data provided is not transmitted, see page 73.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_out , mcbbsp_init , set_fly_x_pos , set_fly_y_pos , set_fly_rot_pos , set_mcbbsp_x , set_mcbbsp_y , set_mcbbsp_rot , set_mcbbsp_matrix , apply_mcbbsp , set_mcbbsp_global_x , set_mcbbsp_global_y , set_mcbbsp_global_rot , set_mcbbsp_global_matrix

Ctrl Command	read_multi_mcbasp	
Function	Queries the McBSP/ SPI multi transmission input value that has been most recently type-sorted and stored.	
Call	<code>mcbasp_value = read_multi_mcbasp(No)</code>	
Parameters	No	Number of the type-sorted internal memory location. (No corresponds to coding bits #0 - 2 of the McBSP/SPI multi input). As an unsigned 32-bit value. 0 x coordinate of the Processing-on-the-fly application. 1 y coordinate of the Processing-on-the-fly application. 2 z coordinate of the Processing-on-the-fly application. 3 Laser power P. 4 Extra parameter E1. 5 Extra parameter E2. 6 Extra parameter E3. 7 Extra parameter E4. Only the 3 least significant bits are evaluated.
Result	Memory value. As a signed 32-bit value.	
Comments	You must have previously activated and used McBSP/SPI multi transmission by set_multi_mcbasp_in or set_multi_mcbasp_in_list. Otherwise, default or old values are returned.	
RTC4→RTC5	New command.	
Version info	–	
References	set_multi_mcbasp_in , set_multi_mcbasp_in_list	

Ctrl Command	read_status																				
Function	Returns the RTC5 list status, see Chapter 6.4.2 "List Status", Page 96.																				
Call	Status = read_status()																				
Result	List status. As an unsigned 32-bit value. <table><tr><th>Bit #</th><th>Name</th><th>Description</th></tr><tr><td>Bit #0 (LSB)</td><td>LOAD1</td><td>= 1: Indicates that the input pointer is currently in "List 1", that is, that all following list commands are stored in "List 1". LOAD1 is set if the input pointer is set into "List 1" (for example, by set_start_list_pos). LOAD1 is reset if a set_end_of_list command is written to "List 1" (READY1 set) or if LOAD2 is set (for example, by set_start_list_pos).</td></tr><tr><td>Bit #1</td><td>LOAD2</td><td>= 1: Indicates that the input pointer is currently in "List 2", that is, that all following list commands are stored in "List 2". LOAD2 is set if the input pointer is set into "List 2" (for example, by set_start_list_pos). LOAD2 is reset if a set_end_of_list command is written to "List 2" (READY2 set) or if LOAD1 is set (for example, by set_start_list_pos).</td></tr><tr><td>Bit #2</td><td>READY1</td><td>= 1: Indicates that during the loading procedure a set_end_of_list command has been written to "List 1". READY1 is reset if LOAD1 is newly set (for example, by set_start_list_pos).</td></tr><tr><td>Bit #3</td><td>READY2</td><td>= 1: Indicates that during the loading procedure a set_end_of_list command has been written to "List 2". READY2 is reset if LOAD2 is newly set (for example, by set_start_list_pos).</td></tr><tr><td>Bit #4</td><td>BUSY1</td><td>= 1: Indicates that "List 1" is executing at the moment (or more precisely: that the output pointer currently resides in "List 1" after execution of "List 1" or "List 2" has been started). BUSY1 is set by starting execution of "List 1" (for example, by execute_list_pos) or by a list change to "List 1" (automatic list change or jump). BUSY1 is reset if set_end_of_list is executed in "List 1", if a jump into "List 2" is executed (for example, list_jump_pos), if "List 2" is started (for example, by execute_list_pos) or if stop_execution is executed.</td></tr></table>			Bit #	Name	Description	Bit #0 (LSB)	LOAD1	= 1: Indicates that the input pointer is currently in "List 1", that is, that all following list commands are stored in "List 1". LOAD1 is set if the input pointer is set into "List 1" (for example, by set_start_list_pos). LOAD1 is reset if a set_end_of_list command is written to "List 1" (READY1 set) or if LOAD2 is set (for example, by set_start_list_pos).	Bit #1	LOAD2	= 1: Indicates that the input pointer is currently in "List 2", that is, that all following list commands are stored in "List 2". LOAD2 is set if the input pointer is set into "List 2" (for example, by set_start_list_pos). LOAD2 is reset if a set_end_of_list command is written to "List 2" (READY2 set) or if LOAD1 is set (for example, by set_start_list_pos).	Bit #2	READY1	= 1: Indicates that during the loading procedure a set_end_of_list command has been written to "List 1". READY1 is reset if LOAD1 is newly set (for example, by set_start_list_pos).	Bit #3	READY2	= 1: Indicates that during the loading procedure a set_end_of_list command has been written to "List 2". READY2 is reset if LOAD2 is newly set (for example, by set_start_list_pos).	Bit #4	BUSY1	= 1: Indicates that "List 1" is executing at the moment (or more precisely: that the output pointer currently resides in "List 1" after execution of "List 1" or "List 2" has been started). BUSY1 is set by starting execution of "List 1" (for example, by execute_list_pos) or by a list change to "List 1" (automatic list change or jump). BUSY1 is reset if set_end_of_list is executed in "List 1", if a jump into "List 2" is executed (for example, list_jump_pos), if "List 2" is started (for example, by execute_list_pos) or if stop_execution is executed.
Bit #	Name	Description																			
Bit #0 (LSB)	LOAD1	= 1: Indicates that the input pointer is currently in "List 1", that is, that all following list commands are stored in "List 1". LOAD1 is set if the input pointer is set into "List 1" (for example, by set_start_list_pos). LOAD1 is reset if a set_end_of_list command is written to "List 1" (READY1 set) or if LOAD2 is set (for example, by set_start_list_pos).																			
Bit #1	LOAD2	= 1: Indicates that the input pointer is currently in "List 2", that is, that all following list commands are stored in "List 2". LOAD2 is set if the input pointer is set into "List 2" (for example, by set_start_list_pos). LOAD2 is reset if a set_end_of_list command is written to "List 2" (READY2 set) or if LOAD1 is set (for example, by set_start_list_pos).																			
Bit #2	READY1	= 1: Indicates that during the loading procedure a set_end_of_list command has been written to "List 1". READY1 is reset if LOAD1 is newly set (for example, by set_start_list_pos).																			
Bit #3	READY2	= 1: Indicates that during the loading procedure a set_end_of_list command has been written to "List 2". READY2 is reset if LOAD2 is newly set (for example, by set_start_list_pos).																			
Bit #4	BUSY1	= 1: Indicates that "List 1" is executing at the moment (or more precisely: that the output pointer currently resides in "List 1" after execution of "List 1" or "List 2" has been started). BUSY1 is set by starting execution of "List 1" (for example, by execute_list_pos) or by a list change to "List 1" (automatic list change or jump). BUSY1 is reset if set_end_of_list is executed in "List 1", if a jump into "List 2" is executed (for example, list_jump_pos), if "List 2" is started (for example, by execute_list_pos) or if stop_execution is executed.																			

Ctrl Command	read_status
Result (cont'd)	Bit #5 BUSY2 = 1: Indicates that "List 2" is executing at the moment (or more precisely: that the output pointer currently resides in "List 2" after execution of "List 1" or "List 2" has been started). BUSY2 is set by starting execution of "List 2" (for example, by execute_list_pos) or by a list change to "List 2" (automatic list change or jump). BUSY2 is reset if set_end_of_list is executed in "List 2", if a jump into "List 1" is executed (for example, list_jump_pos), if "List 1" is started (for example, by execute_list_pos) or if stop_execution is executed. Bit #6 USED1 = 1: Indicates that a set_end_of_list command has been reached during processing of "List 1". USED1 is reset when LOAD1 is set (for example, by set_start_list_pos). Bit #7 USED2 = 1: Indicates that a set_end_of_list command has been reached during processing of "List 2". USED2 is reset when LOAD2 is set (for example, by set_start_list_pos). Bit #8 0 ... Bit #31
Comments	- When interpreting the status values returned by read_status, always take into account the programmed loading or execution processes of the lists. Under some circumstances, the status values can be misleading, as illustrated by the following examples: - Even during a loading process, the LOAD list status can already have been reset and the READY list status set if – after loading of a set_end_of_list into a list – further list commands are loaded into the same list. - The status values remain unchanged if a set_end_of_list command is overwritten with another command (READY list status is not reset). - If – during a loading process – a set_end_of_list command is processed at the same time in the same list, then the list is regarded as already processed (USED list status set), even though it is still newly loaded (USED list status has been then initially reset). - Even if a completely loaded command list (incl. set_end_of_list) has been stored, the READY list status of a list can be reset if the input pointer has been newly set (for example, by set_input_pointer) into the list. Then a list's USED list status can be reset too, even though a completely loaded and already processed command list is stored. - For jumps from one list area to another ("List 1" <-> "List 2", for example, by list_jump_pos) during execution, the USED list status values of both lists (unlike their BUSY list execution status values) remain unchanged and are therefore not meaningful.

Ctrl Command	read_status
Comments (cont'd)	• If the list status is queried during processing of a subroutine in the protected list memory area "List 3", then the status is returned of the list ("List 1" or "List 2") in which the output pointer most recently resided (typically from where the subroutine has been originally called). • If list execution is interrupted (by pause_list, stop_list or set_wait), then the above-mentioned status values remains unchanged. • The List Execution Status values BUSY list execution status and PAUSED list execution status can be queried by get_status. • To read the status signals from the scan heads, use get_head_status.
RTC4→RTC5	Basically unchanged functionality. However: Additional USED list status .
Version info	–
References	get_status , get_head_status

Ctrl Command	read_user_data
Function	No function.
Comments	• To date, read_user_data has no effect.
RTC4→RTC5	New command.
Version info	–
References	send_user_data

Ctrl Command	release_rtc
Function	Releases the specified RTC5 for use by other user programs.
Call	NoOfReleasedCard = release_rtc(CardNo)
Parameters	CardNo RTC5 DLL -internal number of the RTC5 Board. As an unsigned 32-bit value.
Result	The return value is CardNo if the user program has been still in possession of access rights for this board. Otherwise, 0 is returned. As an unsigned 32-bit value.
Comments	• After the user program releases a board with release_rtc, it no longer has access rights for this board. The board can then be subsequently acquired by the same or another user program by acquire_rtc or init_rtc5_dll. • If a board released by release_rtc has been the active board, then (other than multi-board commands) only those non-multi-board commands not requiring explicit access rights are subsequently available. Another board is not automatically selected as the active board. Activation of another board (for which access rights are assigned to the user program) can be achieved by select_rtc. • release_rtc is available even without explicit access rights for a particular RTC5 Board, but release_rtc then has no effect (return value 0). • release_rtc also has no effect (return value 0), if: – CardNo exceeds the number of RTC5 boards found during initialization (see rtc5_count_cards) – CardNo = 0 (real boards begin at 1) • release_rtc is not available as a multi-board command.
RTC4→RTC5	New command.
Version info	–
References	acquire_rtc , init_rtc5_dll

Ctrl Command	release_wait
Function	Resumes execution of a list that has been interrupted by a set_wait .
Call	<code>release_wait()</code>
Comments	• release_wait is only executed if the RTC5 is actually in a wait state (hence if a wait marker has been previously reached and execution has been halted; the PAUSED list execution status is then set but <i>not</i> the BUSY list execution status). Otherwise, release_wait is ignored (get_last_error return code RTC5_BUSY). • release_wait resets the PAUSED list execution status and newly sets the BUSY list execution status (both queryable with get_status, see also Chapter 6.4.3 "List Execution Status", Page 97). • release_wait resets the WaitWord that receives the break point number during an interrupt to zero. • If a home jump (defined with home_position or home_position_xyz) has been executed by set_wait, then release_wait leads to a corresponding home return (the INTERNAL-BUSY list execution status is set while the home return is executed). • The wait state can be queried by get_wait_status.
RTC4→RTC5	Basically unchanged functionality. However: The RTC5 also provides a PAUSED list execution status. It is set with set_wait and reset with release_wait.
Version info	–
References	set_wait , get_wait_status , restart_list

Ctrl Command	reset_error
Function	Resets the cumulative error code.
Call	<code>reset_error(Code)</code>
Parameters	Code OR connection of the error codes = sum by means of $2^{\text{Bitnumber}}$ of all bits to be reset. As an unsigned 32-bit value.
Comments	• For error handling see Chapter 6.8 "Error Handling", Page 119. • The cumulative error code <code>AccError</code> can be reset: – Bitwise (individually for each error type, for example, Bit #5 and Bit #6 by <code>Code = 2⁵ 2⁶</code> or by <code>Code = RTC5_BUSY RTC5_REJECTED</code>) – Completely (by <code>Code = "-1"</code>, therefore by <code>Code = 2³²-1</code>) The meanings of bit numbers, error types and error constants is described at get_error. • <code>reset_error</code> does not delete the error code of another user program currently assigned access rights to the board. • <code>reset_error</code> and <code>n_reset_error</code> are even available without explicit access rights to a specific RTC5 Board. • The board-specific error variable <code>LastError</code> (see Chapter 6.8 "Error Handling", Page 119) is neither generated nor altered by <code>reset_error</code>. In contrast, <code>AccError</code> is altered as specified by <code>Code</code>.
RTC4→RTC5	New command.
Version info	–
References	get_error , get_last_error

Ctrl Command	restart_list
Function	Reenables Signals for “Laser Active” Operation and resumes execution of a list that has been interrupted by pause_list or stop_list .
Call	<code>restart_list()</code>
Comments	restart_list is only executed, if a list has been previously halted by pause_list or stop_list and if the BUSY list execution status and PAUSED list execution status (queryable with get_status) are set. Otherwise (for example, if a list has been halted by set_wait), restart_list is ignored (get_last_error return code RTC5_BUSY). restart_list resets the PAUSED list execution status. The BUSY list execution status is left unchanged. - In general, an interrupted marking cannot be continued without a disruption in the marking result.
RTC4→RTC5	Basically unchanged functionality. However: The RTC5 also provides a PAUSED list execution status that is set with pause_list and stop_list and reset by restart_list .
Version info	–
References	pause_list , stop_list , release_wait

Ctrl Command	rs232_config
Function	Configures the RS-232 interface for the specified baud rate.
Call	<code>rs232_config(BaudRate)</code>
Parameters	BaudRate Baud rate. As an unsigned 32-bit value. Allowed value range: [300...+ 115200].
Comments	- Default value: 9600 baud. - The other RS-232 interface parameters cannot be altered: – Data bits: 8 – Start bits: 1 – Stop bits: 1 – Parity: none - See also Chapter 4.6.5 “RS232 Socket Connector”, Page 70.
RTC4→RTC5	New command.
Version info	–
References	rs232_write_data , rs232_read_data

Ctrl Command	rs232_read_data
Function	Reads a value from the input buffer of the RS-232 interface (see page 264).
Call	RS232Data = rs232_read_data()
Result	As an unsigned 32-bit value. Bit #0 Next (not yet read by the user program) value of the input buffer. (LSB) ... Bit #7 Bit #8 “New” bit: = 1: The value is new (has not been previously read). = 0: The value is old (has been already read once by rs232_read_data). Bit #9 0. ... Bit #15 Bit #16 Number of further (not yet read) characters. ... Bit #23 Bit #24 Number of buffer overruns. ... Bit #31
Comments	• The RS-232 interface is internally read in internally asynchronously (one character at a time). If this character is new, then it is stored in a 256-character ring buffer. From there, it can be transferred to the user program by rs232_read_data (asynchronously to reading). • Byte #0 (Bit #0...Bit #7) returns only one character from the current reading position. • Byte #1 (Bit #8) indicates if the character has already been read by the user program. • Byte #2 (Bit #16...Bit #23) indicates the number of characters in the input buffer that still have not been read by the user program. If no unread characters are present (byte #2 = 0), then the most recently read character is transferred with “new bit” = 0. • Byte #3 (Bit #24...Bit #31) indicates the number of overruns of the input buffer (a corresponding number of characters were overwritten and therefore irretrievably lost). <i>Important: To reset the overflow counter, you must call rs232_read_data correspondingly often!</i> • Example: return value 459098 = 0x0007015A = (0, 7, 1, 90) means: character 90 ('Z') has been read and is new (1), 7 additional characters remain to be read, the buffer has been never overrun (0).
RTC4→RTC5	New command.
Version info	–
References	rs232_write_data

Ctrl Command	rs232_write_data
Function	Sends a data word (byte) to the RS-232 interface.
Call	<code>rs232_write_data(Data)</code>
Parameters	Data Data word. As an unsigned 32-bit value. Only the least significant byte is transferred to the RS-232 interface.
Comments	- The complete transmission of any previous data words is waited for. An overrun at the interface is not possible. - See also Chapter 9.1.6 "RS-232 interface", Page 263.
RTC4→RTC5	New command.
Version info	–
References	rs232_write_text , rs232_read_data

Ctrl Command	rs232_write_text
Function	Sends a text string (character-by-character) to the RS-232 interface.
Call	<code>rs232_write_text(pData)</code>
Parameters	pData PC memory address of the first character (byte) of the to-be-sent text string. As a pointer to a null-terminated string.
Comments	rs232_write_text is split into an appropriate number of rs232_write_data. - During rs232_write_text execution no further control commands can execute, but the board's list execution is not affected. If an executing list itself contains rs232_write_text_list, rs232_write_text should preferably not be used so as to avoid conflicts (race conditions). - See also Chapter 9.1.6 "RS-232 interface", Page 263.
RTC4→RTC5	New command.
Version info	–
References	rs232_write_data , rs232_write_text_list

Variable List Command	rs232_write_text_list
Function	Sends a text string (character-by-character) to the RS-232 interface.
Call	<code>rs232_write_text_list(pData)</code>
Parameters	<code>pData</code> PC memory address of the first character (byte) of the to-be-sent text string. As a pointer to a null-terminated string.
Comments	• When rs232_write_text_list is loaded, the to-be-sent text (if more than 12 characters in length, \0 not included) is split into blocks of 12 characters, with each block receiving its own rs232_write_text_list in the list memory (keep this in mind to prevent unintended overflow of the corresponding list memory area). Processing of the individual rs232_write_text_list is similar to that of rs232_write_text (the 12-character block is split into individual characters and sent sequentially to the RS-232 interface). • rs232_write_text_list takes some time for execution, depending on the baud rate and text length. • See also Chapter 9.1.6 "RS-232 interface", Page 263.
RTC4→RTC5	New command.
Version info	–
References	rs232_write_text

Ctrl Command	rtc5_count_cards
Function	Returns the number of RTC5 Boards detected during initialization.
Call	<code>NoOfCards = rtc5_count_cards()</code>
Result	Number of RTC5 Boards. As an unsigned 32-bit value.
Comments	- Initialization of the installed RTC5 Boards is to be made separately for each user program by calling init_rtc5_dll. rtc5_count_cards is available even without explicit access rights to a specific RTC5 Board. rtc5_count_cards is not available as a multi-board command. - The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by rtc5_count_cards.
RTC4→RTC5	New command. The functionalities of rtc5_count_cards and the RTC4 command rtc4_count_cards are identical.
Version info	–
References	–

Delayed Short List Command	save_and_restart_timer
Function	Stores the current value of the RTC5 Timer and resets it to zero.
Call	<code>save_and_restart_timer()</code>
Comments	• The stored RTC5 Timer value can be read by get_time. • save_and_restart_timer is useful for measuring the marking time of a marking process, see Chapter 8.12 "Time Measurements", Page 257. • The RTC5 Timer only counts list-command clock cycles. Counting is paused during interruptions by set_wait or pause_list. • By get_lap_time the number of elapsed list-command clock cycles since the reset to zero can be queried. • To compare RTC5-internal save_and_restart_timer time measurements to external time measurements by the BUSY pin, you should insert a list_nop between save_and_restart_timer and set_end_of_list. This ensures that any scanner delay completes before set_end_of_list. Without list_nop, save_and_restart_timer includes the scanner delay in its measurement even though it completes only after set_end_of_list (and therefore the BUSY pin is already low). • As of DLL 543, OUT 543, RBF 524 a 32-bit "Timestamp Counter" is available in addition to the RTC5 Timer. It starts counting at 0 with load_program_file and counts uninterruptible all 10 μs clock periods. See also Chapter 8.12 "Time Measurements", Page 257.
RTC4→RTC5	Unchanged functionality.
References	get_lap_time, get_time
Version info	–

Ctrl Command	save_disk
Function	Stores all indexed characters, text strings and/or subroutines to a binary file on a PC storage medium, ordered by index, and returns the number of thereby stored list commands.
Call	<code>NoOfSavedCommands = save_disk(Name, Mode)</code>
Parameters	Name File name. As a pointer to a null-terminated ANSI string.
	Mode Specifies what is to be stored: Bit #0 = 1: All indexed characters and text strings are stored. Bit #1 = 1: All indexed subroutines are stored. Bit #2...Bit #31: Are not evaluated. As an unsigned 32-bit value.
Result	The number of list commands saved by save_disk . As an unsigned 32-bit value.
Comments	- The save_disk command can be used together with load_disk, for example, to perform defragmentation or to apply subsequent protection to subroutines (see page 105 and page 104). save_disk stores complete sets (no individual characters, text strings or subroutines!) in the specified file. Indexed characters, text strings or subroutines that are referenced multiple times with copy_dst_src are also correspondingly stored multiple times by save_disk. save_disk ignores unreferenced non-indexed (not subsequently referenced by set_char_pointer,...) characters, text strings and subroutines. - The number of list commands stored by save_disk can differ from the number of the list commands stored in the protected list memory area "List 3" ($= 2^{20} - \text{Mem1} - \text{Mem2} - \text{get_list_space}$), due to the following: - Indexed characters/text strings/subroutines are referenced several times - Indexed characters/text strings/subroutines reside in the unprotected list memory area "List 1" or "List 2" Prior a subsequent load_disk, be sure to compare the returned number with the size of the protected list memory area "List 3" ($= 2^{20} - \text{Mem1} - \text{Mem2}$). - No-longer-needed characters (or text strings or subroutines) should be dereferenced by load_char (or load_text_table or load_sub) directly followed by list_return previously to save_disk (see page 104).

Ctrl Command	save_disk
Comments (cont'd)	• save_disk always stores all characters, text strings and/or subroutines from the referenced address to the next possible list_return. Jump commands (also branches to various list_return commands) are thereby neither evaluated nor executed. • save_disk automatically replaces unallowed commands (for example, set_end_of_list) with list_nop commands; missing commands (for example, list_return upon reaching the last memory position of "List 3") are added. • save_disk is not executed (get_last_error return code RTC5_PARAM_ERROR), if: – Mode = 0 – Name = NULL • save_disk is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • save_disk is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During the runtime of save_disk: – No further commands are executed – External Starts are suppressed.
RTC4→RTC5	New command.
Version info	–
References	load_disk

Undelayed Short List Command	select_char_set
Function	Selects one of the available character sets (exclusively) for the execution of subsequent mark_text and mark_text_abs .
Call	<code>select_char_set(No)</code>
Parameters	No Number of the desired character set. As an unsigned 32-bit value. Allowed value range: [0...3].
Comments	• The selection remains valid until another is encountered. • The default value (after initialization) is No = 0. • If No > 3, then select_char_set is replaced by a list_nop. The current character set then remains valid (get_last_error return code RTC5_PARAM_ERROR). • A character set change <i>within</i> a mark_text or mark_text_abs command is only possible if a select_char_set is located within a (called) indexed character. The selection encountered there then applies to all subsequent characters. • If the selected character set is not (fully) defined, then missing characters are not marked (with mark_text or mark_text_abs).
RTC4→RTC5	New command.
Version info	–
References	mark_text , mark_text_abs

Ctrl Command	select_cor_table
Function	Assigns the previously loaded correction tables to the scan head connectors and activates Image field correction.
Call	<code>select_cor_table(HeadA, HeadB)</code>
Parameters	HeadA = 0: Turns off the signals for scan head A (first scan head connector). = 1...4: Assigns correction table number HeadA to scan head A. As an unsigned 32-bit value. See also number_of_correction_tables.
	HeadB = 0: Turns off the signals for scan head B (second scan head connector). = 1...4: Assigns correction table number HeadB to scan head B. Requires Option "Second Scan Head Control". As an unsigned 32-bit value. See also number_of_correction_tables.
Comments	• select_cor_table or select_cor_table_list should be called directly after loading the desired correction table(s) by load_correction_file and/or load_z_table (or after a subsequent load_program_file command, see below) in order to assign the correction table(s) to the corresponding connectors (select_cor_table is automatically called by load_correction_file, see Notes, Page 165). select_cor_table and select_cor_table_list issue a jump with <code>jump_speed</code> (no "Hard jump") to the corrected galvanometer scanner position, so that Image field correction is immediately applied to the currently set position of the galvanometer scanners. select_cor_table does this automatically and immediately, whereas select_cor_table_list inserts the jump in front of the next list command. Depending on the table contents and galvanometer scanner position, this can take a few clock cycles (and at least one clock cycle). • Correction tables should be loaded and assigned prior to first-time starting of a list or issuance of a control command (for example, goto_xy) that sets the scan system's galvanometer scanners in motion, and select_cor_table or select_cor_table_list should only be started <i>after</i> loading of the desired correction table(s). Otherwise, the galvanometer scanners can, in some circumstances, be driven to an unexpected position and the laser beam deflected in an unintended direction. • Correction tables, loaded (by load_correction_file) <i>prior</i> to load_program_file, are not fully effective before select_cor_table or select_cor_table_list is called. load_program_file deletes correction tables number 3 and 4. • select_cor_table is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set The list command select_cor_table_list can be used without restrictions. • select_cor_table is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • If the INTERNAL-BUSY list execution status is set, select_cor_table is only executed with a delay (after INTERNAL-BUSY list execution status has been reset again).

Ctrl Command	select_cor_table
Comments (cont'd)	- The INTERNAL-BUSY list execution status is set while the jump to the corrected galvanometer scanner position is executed. - If the values for parameters HeadA or HeadB are invalid (all values > number_of_correction_tables) or 0, then select_cor_table turns off the corresponding scan head signals. The galvanometer scanners then remain in their last position. - If the Option "Second Scan Head Control" has not been enabled, select_cor_table does not assign a correction table to the second scan head connector. - The default setting (after load_program_file) is (1,0), that is, correction table #1 is used for scan head A, whereas the output signals for scan head B are turned off (this corresponds to parameter values HeadA = 1 and HeadB = 0). Initially however, after load_program_file and until select_cor_table is called or the galvanometer scanners are explicitly moved, the galvanometer scanners stay in the position (0 0), even if the correction table content is different. - If the Option "Second Scan Head Control" is disabled, then signals of the second scan head remain permanently turned off; even so, multiple correction tables can still be loaded and used at any time by the first scan head. Even with one scan head, you can thereby quickly switch back and forth between several correction tables, for example, one for a pointer laser and one for the main laser with a different wavelength, see also Chapter 8.5 "Controlling 2D Scan Systems and 3D Scan Systems", Page 221). - In a double scan head system, table #1 is typically used for scan head A, and table #2 is used for scan head B: select_cor_table(1,2). But you can make a different assignment at any time. - For 3D systems, the Option "3D" must be enabled. Provided the RTC5's Option "3D" is enabled, you can load several (2D or 3D) correction tables. If the Option "Second Scan Head Control" is not enabled, then x axis and y axis must be connected to the first scan head connector, the z axis to the x or y channel of the second scan head connector. If a 3D correction table is assigned to the first scan head connector, corrected signals for an xy scan head are then transmitted by the first scan head connector and corrected signals for the z axis are transmitted by both channels of the second scan head connector. If multiple RTC5 Boards with enabled Option "3D" are installed in a PC, then that many 3-axis systems can be simultaneously controlled.

Ctrl Command	select_cor_table
Comments (cont'd)	• Provided the Option "3D" and the Option "Second Scan Head Control" are enabled, users specify which signals (XY or Z) are to be outputted by which connector when assigning the correction table (see also Chapter 4.5.1 "Scan Head Connectors and Transfer Protocol", Page 54 and Section "2D Correction Files and 3D Correction Files", Page 163). 2D correction tables should and 3D correction tables can be thereby assigned only to the xy axes and <i>not</i> to the z axis (for example, by select_cor_table(0,1) for xy at the scan head B connector and z at the scan head A connector). Because unexpected system behavior might otherwise occur and the system would not know where the xy axes and z axis are connected, the signals of both connectors are turned off if both connectors have been assigned a correction table and (at least) one of them is a 3D correction table (for example, for select_cor_table(1,1)). • Parameters from currently loaded correction tables can be read by get_table_para, or from currently assigned correction tables by get_head_para (see also load_correction_file).
RTC4→RTC5	Unchanged functionality. Exception: several 3D correction tables can be stored in RTC5 memory, see Chapter 8.5.3 "Using Several Correction Tables", Page 226. However, <i>two</i> 3D correction tables <i>cannot</i> be simultaneously assigned to the scan head connectors.
Version info	–
References	select_cor_table_list , load_correction_file , load_program_file , number_of_correction_tables

Variable List Command	<code>select_cor_table_list</code>
Function	Like <code>select_cor_table</code> , but a list command.
Call	<code>select_cor_table_list(HeadA, HeadB)</code>
Parameters	HeadA Like <code>select_cor_table</code> .
	HeadB Like <code>select_cor_table</code> .
Comments	• See <code>select_cor_table</code>. • Even though <code>select_cor_table_list</code> is a short list command, execution of the directly following list command is delayed by a few clock cycles due to the intermediate jump to the corrected galvanometer scanner position. The extent of this delay depends on the assigned correction table's content for the current galvanometer scanner position (for example, (0 0) after <code>load_program_file</code>); but it is at least 10 μs, even if the correction table does not specify a correction.
RTC4→RTC5	New command.
Version info	–
References	<code>select_cor_table</code>

Ctrl Command	select_rtc
Function	Defines, in a multi-board system, the active RTC5 Board for a user program. See Chapter 6.6 "Using Several RTC5 PCI Boards in One PC" , Page 112.
Call	<code>NoOfSelectedCard = select_rtc(CardNo)</code>
Parameters	CardNo RTC5 DLL -internal number of the RTC5 Board. As an unsigned 32-bit value.
Result	The return value is: • CardNo if access rights exist for the specified board or if the specified board is not allocated to another user program (in this latter case, access rights are acquired through select_rtc – as by acquire_rtc, see below). • The number of the active board if CardNo exceeds the number of RTC5 Boards found during initialization or if CardNo = 0. • 0 otherwise (particularly when the specified board is reserved by another user program or if a version compatibility error is detected and activation of the board therefore cannot succeed) (get_last_error return code RTC5_ACCESS_DENIED and possibly RTC5_VERSION_MISMATCH). As an unsigned 32-bit value.
Comments	• Activation of a board for a user program already occurs when the RTC5 DLL for this user program is initialized (see init_rtc5_dll). The select_rtc command offers the possibility of changing the active board at any time. All the user program's commands subsequent to select_rtc (except for multi-board commands) are forwarded to the corresponding active board. • If the specified board is reserved for another user program, select_rtc has no effect (return value 0, get_last_error return code RTC5_ACCESS_DENIED). • If no access rights are assigned for the specified board, and it is not in use by another user program, then the board is not only activated but also automatically reserved for this user program and therefore, unavailable to other user programs (return value: CardNo). If a board is acquired (as by acquire_rtc), a version compatibility check is performed. If a version error is detected, then access to the board is denied and select_rtc has no effect (return code 0, get_last_error return code RTC5_ACCESS_DENIED RTC5_VERSION_MISMATCH). • If the specified board is already the active board for this user program, select_rtc has no effect (return value: CardNo). • select_rtc also has no effect if CardNo exceeds the number of RTC5 Boards found during initialization (see rtc5_count_cards) or if CardNo = 0 (real boards begin at 1). The return value is then the number of the active board (see above). Therefore, <code>select_rtc(0)</code> can be used to determine the number of the active board at any time. • select_rtc is available even without explicit access rights to a particular RTC5 Board. • select_rtc is not available as a multi-board command.

Ctrl Command	<code>select_rtc</code>
RTC4→RTC5	With the RTC4, <code>select_rtc</code> only activates the specified board (unconditionally). With the RTC5, <code>select_rtc</code> additionally returns a return value and assigns access rights (as by <code>acquire_rtc</code>) or it has no effect.
Version info	–
References	–

Ctrl Command	select_serial_set
Function	Selects a serial-number-set for serial number control commands.
Call	<code>select_serial_set(No)</code>
Parameters	No Number of the desired serial-number-set. As an unsigned 32-bit value. Allowed value range: [0...3]. Only the two least significant bits are evaluated.
Comments	• After initialization with load_program_file, serial-number-set 0 is selected. • select_serial_set does <i>not</i> assign a serial-number-set for serial number marking. The list command select_serial_set_list is intended for that purpose. • For select_serial_set usage, see Chapter 7.5.2 "Marking Serial Numbers", Page 196.
RTC4→RTC5	New command.
Version info	–
References	select_serial_set_list, set_serial_step, get_serial

Undelayed Short List Command	select_serial_set_list
Function	Selects a serial-number-set for serial number list commands (as well as for serial number marking).
Call	<code>select_serial_set_list(No)</code>
Parameters	No Number of the desired serial-number-set. As an unsigned 32-bit value. Allowed value range: [0...3]; only the two least significant bits are evaluated.
Comments	• After initialization with load_program_file, serial-number-set 0 is selected. • For select_serial_set_list usage, see Chapter 7.5.2 "Marking Serial Numbers", Page 196.
RTC4→RTC5	New command.
Version info	–
References	select_serial_set, set_serial_step_list, get_list_serial, mark_serial, mark_serial_abs

Ctrl Command	send_user_data
Function	No function.
Comments	To date, send_user_data has no effect.
RTC4→RTC5	New command.
Version info	–
References	read_user_data

Ctrl Command	set_angle
Function	Uses a specified rotation angle to define the rotation matrix M_R for all subsequent coordinate transformations (see Chapter 8.2 "Coordinate Transformations", Page 209).
Call	<code>set_angle(HeadNo, Angle, at_once)</code>
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. = 1: The definition only affects the first scan head connector. = 2: The definition only affects the second scan head connector. = 0, 3: The definition affects <i>both</i> scan head connectors. = 4: The definition affects the virtual Image field (see also comments). Only the 3 least significant bits are evaluated.
	Angle Rotation angle. In degrees. As a 64-bit IEEE floating point value. Positive angles result in a counterclockwise rotation around the centerpoint of the Image field .
	at_once Defines when the defined transformation becomes effective. As an unsigned 32-bit value. Like set_matrix .
Comments	• See also Chapter 8.2 "Coordinate Transformations", Page 209. • The coordinate transformation defined by <code>HeadNo = 4</code> is only in effect during a Processing-on-the-fly application initiated by set_fly_2d, see Section "Coordinate Transformations in the Virtual Image Field", Page 158: – set_fly_2d – $\geq$ DLL 541 set_fly_x – $\geq$ DLL 541 set_fly_y Here, the <code>at_once</code> parameter is ignored.
RTC4→RTC5	New command.
Version info	Last change DLL 545, OUT 545: <code>HeadNo = 4</code> .
References	set_angle_list , set_matrix , set_offset , set_scale

Variable List Command	set_angle_list
Function	Like set_angle , but a list command.
Call	set_angle_list(HeadNo, Angle, at_once)
Parameters	HeadNo Like set_angle . However, HeadNo = 4 is not available.
	Angle Like set_angle .
	at_once Determines when the defined transformation becomes effective.
	= 0: The transformation settings are only collected and intermediately stored, but the transformation is not processed as long as this is not activated by another coordinate transformation (for example, by a list command with at_once = 1 or a corresponding control command). = 1: The transformation is immediately calculated (including all transformation settings that were collected until then) and processed prior to the next list command. If necessary, Signals for "Laser Active" Operation are switched off in advance. = 2: The transformation settings are only collected and intermediately stored (as with at_once = 0). However, The transformation is immediately calculated (including all transformation settings that were collected and intermediately stored until then) and applied to the current position when the next jump_abs or jump_rel (only if no list is currently being executed: also goto_xy or goto_xyz) is executed. = 3: Like at_once = 1, but the laser control signals remain unaffected. > 3: Like at_once = 2. As an unsigned 32-bit value.
Comments	HeadNo = 4 is not available.
RTC4→RTC5	New command.
Version info	–
References	set_angle

Ctrl Command	set_auto_laser_control	
Function	Initializes or deactivates "Automatic Laser Control", see Chapter 7.4.9 "Automatic Laser Control" , Page 187 .	
Call	ErrorCode = set_auto_laser_control(Ctrl, Value, Mode, MinValue, MaxValue)	
Parameters	Ctrl	Control parameter. As an unsigned 32-bit value. = 1...6: Defines which signal parameter is corrected by "Automatic Laser Control". = 1: 12-bit output value at the ANALOG OUT1 output port. = 2: 12-bit output value at the ANALOG OUT2 output port. = 3: Output value at the 8-bit digital output port. = 4: Pulse length (PulseLength) of the laser signals LASER1 and LASER2. = 5: Output period (HalfPeriod) of the laser signals LASER1 and LASER2. = 6: Output value at the 16-bit digital output. = 0 or > 6: Deactivates "Automatic Laser Control" (for Ctrl > 6: get_last_error return code RTC5_PARAM_ERROR).
	Value	Defines the 100% value for the parameter selected by Ctrl. As an unsigned 32-bit value. Allowed values: - For Ctrl = 1/2: 12-bit values [0...4095]. Bits with higher significance are ignored. - For Ctrl = 3: 8-bit values [0...255]. Bits with higher significance are ignored. - For Ctrl = 4: [0...(2³²-1)]. 1 bit equals 1/64 µs. - For Ctrl = 5: [0...(2³²-1)]. 1 bit equals 1/64 µs. - For Ctrl = 6: 16-bit values [0...(2¹⁶-1)]. Bits with higher significance are ignored.
	Mode	Speed-dependent laser control (Mode = 1...5) lets you select which data is to be used for calculating speed-dependent correction. As an unsigned 32-bit value. =1: Set speed. =2: Actual speed (as well as extensions, see below). =3: Reserved. =4: Reserved. =5: Encoder speed (counter pulse rate).

Ctrl Command		set_auto_laser_control												
Parameters (cont'd)	Mode (cont'd)	For Mode = 2 above, the following extensions are available in any combination: • M = +4 encoder speed-dependent correction Then the following applies: Mode = 2 + sum of extensions M. For Mode = 0 or none of the above described ones: Disables the speed-dependent or encoder-speed-dependent laser control (get_last_error-Returncode: <code>RTC5_PARAM_ERROR</code>). The position-dependent laser control always remains effective with a valid value for Ctrl. See also Section "Position-Dependent Laser Control", Page 189.												
	MinValue	Defines the <i>lower</i> range limit of the parameter selected by Ctrl for automatic correction and that cannot be undercut. As an unsigned 32-bit value. Allowed values: see Value.												
	MaxValue	Defines the <i>upper</i> range limit of the parameter selected by Ctrl for automatic correction and that cannot be exceeded. As an unsigned 32-bit value. Allowed values: see Value.												
Result	ErrorCode. As an unsigned 32-bit value. <table><tr><td>0</td><td>No error.</td></tr><tr><td>1</td><td>No first scan head active.</td></tr><tr><td>2</td><td>No iDRIVE scan system (see Glossary entry on page 23) active.</td></tr><tr><td>3</td><td>Invalid Ctrl value.</td></tr><tr><td>4</td><td>Invalid Mode value.</td></tr><tr><td>5</td><td>Access denied (board locked, get_last_error return code <code>RTC5_ACCESS_DENIED</code>).</td></tr></table>		0	No error.	1	No first scan head active.	2	No iDRIVE scan system (see Glossary entry on page 23) active.	3	Invalid Ctrl value.	4	Invalid Mode value.	5	Access denied (board locked, get_last_error return code <code>RTC5_ACCESS_DENIED</code>).
0	No error.													
1	No first scan head active.													
2	No iDRIVE scan system (see Glossary entry on page 23) active.													
3	Invalid Ctrl value.													
4	Invalid Mode value.													
5	Access denied (board locked, get_last_error return code <code>RTC5_ACCESS_DENIED</code>).													
Comments	• For set_auto_laser_control usage, see Chapter 7.4.9 "Automatic Laser Control", Page 187. • MinValue and MaxValue are automatically exchanged if MinValue > MaxValue and MinValue is clipped to ≤ Value and MaxValue clipped to ≥ Value. • Control data for speed-dependent laser control is exclusively derived from the scan system data at the first scan head connector and then also applied for the second scan head connector (if that connector has been activated and a scan system is attached). At the time set_auto_laser_control is called, the first scan head connector must already have been assigned a correction table, otherwise error code 1 is returned and Ctrl is set to 0. If only the second scan head connector is being used and/or the scan system at the first scan head connector is shut off, then speed-dependent laser control does not function.													

Ctrl Command	set_auto_laser_control
Comments (cont'd)	- For <i>Mode</i> = 2, a functioning iDRIVE scan system must be attached to the first scan head connector. If this is not the case, then error code 2 is returned and <i>Ctrl</i> set to 0. If the iDRIVE scan system has reacted, then the to-be-transmitted data type for the first scan head connector and both axes are set to "actual velocity" by control_command (<i>Data</i> = 0506_H). As a result, control_command is unavailable for the first scan head connector as long as this <i>Mode</i> selection is in effect. For <i>Mode</i> = 0 or 1, control_command is available without any restriction. - Encoder-speed-dependent laser control (<i>Mode</i> = 5) functions even if no scan heads are attached to the scan-head connectors at runtime. The Option Processing-on-the-fly does not need to be activated here either. The target encoder speed is set by set_encoder_speed. - With the +4 extension, encoder speeds are converted to galvanometer scanner units (<i>bits/ms</i>) by using the scaling factors from set_fly_x and set_fly_y or set_fly_2d. The result is then added to the current galvanometer scanner speeds. This requires an enabled Option Processing-on-the-fly (in contrast to <i>Mode</i> = 5). The current mark speed is used as reference speed. The set_encoder_speed command (which is used for <i>Mode</i> = 5) is not taken into account with the +4 extension. If an axis is not activated for Processing-on-the-fly at runtime, its encoder speed is not taken into account. If both axes are not activated for Processing-on-the-fly, +4 extension is not effective. set_fly_rot, set_fly_rot_pos, set_fly_x_pos and set_fly_y_pos cannot be combined with the galvanometer scanner speed. - If the values for <i>Ctrl</i> or <i>Mode</i> are invalid, then set_auto_laser_control is not executed (return value 3 or 4, get_last_error return code RTC5_PARAM_ERROR). - The signal parameter selected with <i>Ctrl</i> is updated every 10 μs. However, for <i>Ctrl</i> = 5 (output period) it is always only effective after an already started laser signal period has elapsed. "Automatic Laser Control" should not be combined with the variable laser power of set_multi_mcbp_in.
RTC4→RTC5	New command. In RTC4 Compatibility Mode for <i>Ctrl</i> = 1/2, the specified values for <i>Value</i>, <i>MinValue</i> and <i>MaxValue</i> are internally multiplied by 4. For <i>Ctrl</i> = 4/5, the parameter values for <i>Value</i>, <i>MinValue</i> and <i>MaxValue</i> are multiplied by 8. The allowed value ranges decrease accordingly.
Version info	Last change DLL 540, OUT 540: <i>Mode</i> = 6.
References	load_position_control , load_auto_laser_control , set_auto_laser_params , set_auto_laser_params_list , set_fly_x , set_fly_y , set_fly_2d , set_encoder_speed

Ctrl Command	set_auto_laser_params	
Function	Specifies the to-be-controlled signal parameter of the "Automatic Laser Control", the 100% value and its corresponding limit values.	
Call	set_auto_laser_params(Ctrl, Value, MinValue, MaxValue)	
Parameters	Ctrl	The to-be-controlled signal parameter (see set_auto_laser_control). Allowed value range: [1...6].
	Value	100% value. See set_auto_laser_control .
	MinValue	Lower range limit. See set_auto_laser_control .
	MaxValue	Upper range limit. See set_auto_laser_control .
Result	ErrorCode. As an unsigned 32-bit value. 0 No error. 3 Invalid Ctrl value.	
Comments	- For set_auto_laser_params usage, see Chapter 7.4.9 "Automatic Laser Control", Page 187. - The parameter's meaning is identical to that of set_auto_laser_control (see also comments there). However, set_auto_laser_params can neither start nor terminate "Automatic Laser Control". - If the value for Ctrl is invalid (0 or >6), then set_auto_laser_params is not executed (get_last_error return code RTC5_PARAM_ERROR).	
RTC4→RTC5	New command. RTC4 Compatibility Mode : see set_auto_laser_control .	
Version info	–	
References	set_auto_laser_control	

Delayed Short List Command	set_auto_laser_params_list	
Function	Like set_auto_laser_params , but a list command.	
Call	set_auto_laser_params_list(Ctrl, Value, MinValue, MaxValue)	
Parameters	Ctrl	Like set_auto_laser_params .
	Value	Like set_auto_laser_params .
	MinValue	Like set_auto_laser_params .
	MaxValue	Like set_auto_laser_params .
Comments	If the value for Ctrl is invalid (0 or >6), then set_auto_laser_params_list is replaced with a list_nop (get_last_error return code RTC5_PARAM_ERROR).	
RTC4→RTC5	New command. RTC4 Compatibility Mode : see set_auto_laser_control .	
Version info	–	
References	set_auto_laser_params , set_auto_laser_control	

Ctrl Command	set_char_pointer
Function	Stores the absolute start address of a command list in the internal management table for indexed characters.
Call	<code>set_char_pointer(Char, Pos)</code>
Parameters	Char Index of the indexed character whose starting address Pos should be entered in the management table. As an unsigned 32-bit value. Allowed value range: [0...1023]. The same applies as for load_char: $\text{Char} = \text{character set number} \times 256 + \text{ASCII number of the character}$ (character sets are numbered 0 to 3).
	Pos Absolute start address. As an unsigned 32-bit value. Allowed value range: [0...(2²⁰-1)].
Comments	• If Char > 1023 and/or Pos > (2²⁰-1), then set_char_pointer is <i>not</i> executed (get_last_error return code RTC5_PARAM_ERROR). • The set_char_pointer command can be used for referencing an indexed character. It thereby becomes an indexed character that is protectable by save_disk/load_disk and/or callable by the index. • set_char_pointer can also be used to reference anew an indexed subroutine, character or text string so that it can also be called by a second index. Here, it is preferable to use the copy_dst_src command for index management. • The start addresses of command lists that are to be referenced with set_char_pointer can be queried by get_input_pointer <i>before</i> saving the command lists. • set_char_pointer only stores <i>starting addresses</i> in the internal management table. An indexed character only gains protection by a subsequent save_disk/load_disk. • Pos should not be an arbitrary address within a list. Instead, it should be the starting address of an actually existing subroutine that has been finalized by list_return and does not contain set_end_of_list.
RTC4→RTC5	New command.
Version info	–
References	load_char

Ctrl Command	set_char_table	
Function	Stores the absolute start address of a command list in the internal management table for indexed text strings.	
Call	set_char_table(Index, Pos)	
Parameters	Index	Index of the indexed text string whose starting address Pos should be entered in the management table. As an unsigned 32-bit value. Allowed value range: [0...41]). The same ordering applies as for the load_text_table command: = 0...9: Digits for marking the time and date [0...9]. = 10...21: Months [January...December]. = 22...28: Days-of-the-week [Sunday...Saturday]. = 29: Blank character for marking serial numbers. = 30...39: Digits for marking serial numbers [0...9]. = 40: Text for "a.m.". = 41: Text for "p.m.".
	Pos	Absolute start address. As an unsigned 32-bit value. Allowed value range: [0...(2²⁰-1)].
Comments	• set_char_table is synonymous with set_text_table_pointer.	
RTC4→RTC5	New command.	
Version info	–	
References	set_text_table_pointer	

Ctrl Command	set_control_mode		
Function	Enables or disables the external control input for External Starts and resets the counter for External Starts to zero.		
Call	set_control_mode(Mode)		
Parameters	Mode	As an unsigned 32-bit value.	
	Bit #	Value	Description
	Bit #0 (LSB)	= 1:	The external start input (by /START, /START2 or /Slave-START) is enabled.
		= 0:	The external start input is disabled.
	Bit #1	= 1:	With an External Stop (/STOP, /STOP2, /Slave-STOP or simulate_ext_stop) causes explicit cancellation of the external start queue entries (/START, /START2, /Slave-START or simulate_ext_start).
		= 0:	No effect.
	Bit #2	= 1:	The track delay (defined by simulate_ext_start , set_ext_start_delay or set_ext_start_delay_list) that postpones execution of the start relative to the triggering input signal or simulate_ext_start or simulate_ext_start_ctrl (see Section "External Start", Page 266) is deactivated.
		= 0:	No effect. To define and activate the track delay (for example, for Processing-on-the-fly applications), use simulate_ext_start , set_ext_start_delay or set_ext_start_delay_list .
	Bit #3	= 1:	The external start input is <i>not</i> disabled by an external stop signal.
		= 0:	The external start input <i>is</i> disabled by an external stop signal.
	Bit #4		Disables simulate_ext_start_ctrl .
	Bit #5		Reserved.
	Bit #6		Reserved.
	Bit #7		Reserved.
	Bit #8		Reserved.
	Bit #9	= 1:	Encoder resets of the 2 internal encoder counters occur by: • An external start signal • simulate_ext_start • simulate_ext_start_ctrl, postponed by a track delay set by simulate_ext_start, set_ext_start_delay or set_ext_start_delay_list, see also Bit #2
		= 0:	Encoder resets occur immediately with each initiating Processing-on-the-fly command.

Ctrl Command	set_control_mode	
Parameters (cont'd)	Bit #10 = 1: Track delay configured by simulate_ext_start , set_ext_start_delay or set_ext_start_delay_list is counted beginning with the most recent externally (but not with execute_list_pos etc.) triggered or simulated External Start . The interval between subsequent External Starts (in encoder pulses) is thus constant (see also Section "Regular (Periodic) External Starts" , Page 269). For stop_execution or an external stop signal, Bit #10 gets reset to "0". = 0: Track delay configured by simulate_ext_start , set_ext_start_delay or set_ext_start_delay_list is counted beginning with the point in time an External Start has been requested (that is, with the corresponding simulate_ext_start or simulate_ext_start_ctrl or external start signal). The interval between subsequent External Starts (in encoder pulses) can thus vary. Bit #12 Reserved. Bit #31 Reserved.	
Comments	• If execution is aborted by stop_execution, then Bit #0 and Bit #10 gets reset to zero, thus deactivating external start inputs and the counting of track delays with respect to a the triggering event. • If Bit #9 = 0, then there is generally a small (random) time offset (10 μs jitter) between the start signal at /START, /START2 and the actual execution start. If Bit #9 = 1, then this 10 μs jitter is not present because the encoder reset then occurs synchronously with the start signal. • Bit #9 = 0 is useful, when several separate Processing-on-the-fly sessions are to be started within a list. • See also Section "External Stop", Page 265 and Section "External Start", Page 266.	
RTC4→RTC5	Bit #8 cannot not be used to lock or unlock the upper 8 bits of the 16-bit digital output port for I/O commands. The RTC5 automatically reserves these bits for varioSCAN FLEX control by move_to . It is not possible to explicitly release these bits (release automatically occurs by load_program_file). Otherwise: unchanged functionality for the bits that are also used on the RTC4.	
Version info	Last change DLL 543, OUT 543: Bit #4 .	
References	set_extstartpos , get_counts , set_max_counts , get_startstop_info , move_to	

Normal List Command	set_control_mode_list
Function	Like set_control_mode , but a list command.
Call	<code>set_control_mode_list(Mode)</code>
Parameters	Mode Like set_control_mode .
Comments	The counter for External Starts is <i>not</i> reset by set_control_mode_list.
RTC4→RTC5	Unchanged functionality for the bits that are also used on the RTC4.
Version info	Last change DLL 543, OUT 543: Bit #4 .
References	set_control_mode

Ctrl Command	set_default_pixel
Function	Defines the pulse length default value for the default pixel that terminates Pixel Output Mode .
Call	<code>set_default_pixel(PulseLength)</code>
Parameters	PulseLength Default pixel pulse length. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	- In Pixel Output Mode, the pixel pulse length at the end of an image line (at the beginning of the default pixel) gets set to the specified default value (see Chapter 8.7.4 "Synchronization", Page 247). - When initializing (by load_program_file), the <code>PulseLength</code> default value is set to 0.
RTC4→RTC5	New command.
Version info	–
References	set_port_default, set_default_pixel_list

Undelayed Short List Command	set_default_pixel_list
Function	Like set_default_pixel , but a list command.
Call	<code>set_default_pixel_list(PulseLength)</code>
Parameters	PulseLength Like set_default_pixel .
Comments	See set_default_pixel.
RTC4→RTC5	New command.
Version info	–
References	set_default_pixel, set_port_default

Ctrl Command	set_defocus
Function	Defines a focus shift for 3D vector outputs.
Restriction	If the Option "3D" has not been enabled or if no 3D correction table has been assigned (see select_cor_table), then set_defocus has no effect. Nevertheless, the supplied focus shift value is stored internally and takes effect as soon as a 3D correction table is assigned.
Call	set_defocus(Shift)
Parameters	Shift Focus shift. In bits. As a signed 32-bit value. Allowed value range: [-32,768...+32,767]. Out-of-range values are clipped to the boundary values.
Comments	• A focus shift causes a defocusing of the laser focus relative to the working plane. A positive value increases the focal length of the Dynamic focusing unit and shifts the focus position, for example, for the control value (0 0 0) by about $d = \text{Shift} / K$ to the plane $z = -d$ [mm]. For the calibration factor K, see Chapter 7.3.2 "Image Field Size and Image Field Calibration", Page 156; furthermore, #8 in Chapter 7.3.6 "Output Values to the Scan System", Page 170. • If the resulting total Z output value is too large, the z axis moves to the limit stop. Make sure that this is avoided as far as possible, see get_galvo_controls. • If set_defocus is called during output of a vector, then it is only executed directly before the next list command. To avoid "Hard jumps", a jump to the changed z output is performed at jump speed. Length and duration of the jump depend on the changed z control value. The duration can be calculated by get_galvo_controls. If no list is currently BUSY list execution status, then the jump executes immediately, whereby no delay occurs at the next start. • If the INTERNAL-BUSY list execution status is set, set_defocus is only executed with a delay (after INTERNAL-BUSY list execution status has been reset again). • set_defocus sets the INTERNAL-BUSY list execution status while the jump to the changed z output is executed. • After vector-defined laser control is activated by set_vector_control(Ctrl = 7), the focus shift changes with parameterized [*]mark[*] Command or Jump commands, too.
RTC4→RTC5	Basically unchanged functionality. But the RTC5 command avoids " Hard jumps ".
Version info	–
References	set_defocus_list , set_offset_xyz , set_defocus_offset , get_galvo_controls

Variable List Command	set_defocus_list
Function	Like set_defocus , but a list command.
Restriction	Like set_defocus .
Call	<code>set_defocus_list(Shift)</code>
Parameters	Shift Like set_defocus .
Comments	• See set_defocus. • Even though set_defocus_list is a short list command, execution of the directly following list command is delayed by a few clock cycles due to the intermediate jump to the changed z output. The extent of this delay depends on the size of the specified focus shift; but it is at least 10 μs, even if <code>Shift = 0</code>. • After the jump to the changed z output, a Jump Delay previously configured with set_scanner_delays is inserted. Depending on the dynamics of the z axis it may be reasonable to increase the Jump Delay before calling set_defocus_list.
RTC4→RTC5	See set_defocus .
Version info	–
References	set_defocus , long_delay

Ctrl Command	set_defocus_offset
Function	Defines an offset in addition to the focus shift (see set_defocus) for 3D vector outputs.
Restriction	Like set_defocus .
Call	<code>set_defocus_offset(Shift)</code>
Parameters	Shift See set_defocus .
Comments	• set_defocus_offset has the same effect as set_defocus. • set_defocus_offset enables setting a global defocus shift that is not overwritten by parameterized commands such as para_mark_abs (see set_vector_control(Ctrl = 7)). • The set_defocus_offset shift, – works additively to set_defocus, – cannot be used as a control parameter for vector-defined laser control, – cannot be recorded via Signal 32 with set_trigger/set_trigger4, – is taken into account by get_z_distance, get_galvo_controls and the back transformation (transform, get_transform).
RTC4→RTC5	New command.
Version info	Available as of DLL 540, OUT 540.
References	set_defocus , set_defocus_offset_list

Variable List Command	set_defocus_offset_list
Function	Like set_defocus_offset , but a list command.
Restriction	Like set_defocus .
Call	set_defocus_offset_list(Shift)
Parameters	Shift Like set_defocus .
Comments	• See set_defocus_offset. • See set_defocus_list. • Even though set_defocus_offset_list is a short list command, execution of the directly following list command is delayed by a few clock cycles due to the intermediate jump to the changed z output. The extent of this delay depends on the size of the specified focus shift; but it is at least 10 μs, even if Shift = 0.
RTC4→RTC5	New command.
Version info	Available as of DLL 540, OUT 540.
References	set_defocus_offset , set_defocus_list

Ctrl Command	set_delay_mode		
Function	Turns the “Variable Polygon Delays” mode and the “Variable Jump Delays” mode on or off and sets some special scanner-delay related parameters as well as the 3D Z-Move mode.		
Call	set_delay_mode(VarPoly, DirectMove3D, EdgeLevel, MinJumpDelay, JumpLengthLimit)		
Parameters	Name	Allowed Values	Description
	VarPoly	> 0	Enables “Variable Polygon Delays” mode.
		= 0	Disables “Variable Polygon Delays” mode. Default setting.
	DirectMove3D		As an unsigned 32-bit value.
			This parameter effects only 3D-applications.
		> 0	The z output is changed directly (linearly) to its end value during a jump.
		= 0	The z output is changed to its end-value in such a way that the focus is kept in one plane during the entire jump.
	EdgeLevel		As an unsigned 32-bit value.
		0...(2 ³² -1). 1 bit equals 10 μs.	This parameter defines a maximum “laser on” time for the corners of a Polyline. If the Polygon Delay is longer than or equal to this value (because the angle φ is close to 180°, for instance), the laser is switched off (after a LaserOff Delay) and a new Polyline is started. This can be useful for preventing burn-in effects. The EdgeLevel value must be smaller than twice the set value for the Polygon Delay, otherwise it has no effect. See also Figure 44.
	MinJumpDelay		Note: To disable this feature, the EdgeLevel value must be set to (2 ³² -1) (default value).
		0...(2 ³² -1). 1 bit equals 10 μs.	As an unsigned 32-bit value. Minimum Jump Delay that cannot be undercut for jumps shorter than JumpLengthLimit (even for jump vectors of zero length, see Figure 40). For jumps longer than JumpLengthLimit, the MinJumpDelay has no relevance. To avoid anomalies in the range of MinJumpDelay, define a value for MinJumpDelay that is not larger than the Jump Delay.
			As an unsigned 32-bit value.

Ctrl Command	set_delay_mode
	JumpLengthLimit 0...(2³²-1) Jump length limit. In bits. JumpLengthLimit > 0 enables the “Variable Jump Delays” mode. If the jump vector is <i>longer</i> than this value, then the fixed Jump Delay (see set_scanner_delays) is inserted. For all shorter jump lengths, a linearly interpolated “Variable Jump Delay” (page 136) between MinJumpDelay and the Jump Delay is calculated and inserted, see Figure 40. JumpLengthLimit = 0 disables the “Variable Jump Delays” mode. As an unsigned 32-bit value.
RTC4→RTC5	Unchanged functionality. In addition: extended value range. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for JumpLengthLimit by 16. The allowed value range decreases accordingly.
Version info	–
References	set_scanner_delays , load_varpolydelay , set_delay_mode_list

Multiple List Command	set_delay_mode_list
Function	Like set_delay_mode , but a list command.
Call	set_delay_mode_list(VarPoly, DirectMove3D, EdgeLevel, MinJumpDelay, JumpLengthLimit)
Parameters	VarPoly Like set_delay_mode .
	DirectMove3D Like set_delay_mode .
	EdgeLevel Like set_delay_mode .
	MinJumpDelay Like set_delay_mode .
	JumpLengthLimit Like set_delay_mode .
Comments	• See set_delay_mode. • set_delay_mode_list requires two list storage positions. • set_delay_mode_list is executed as two undelayed short list commands. • Any still-pending delayed short list command is executed first.
RTC4→RTC5	New command. RTC4 Compatibility Mode: see set_delay_mode.
Version info	–
References	set_delay_mode

Ctrl Command	set_dsp_mode
Function	Sets a DSP mode for short-command count.
Call	<code>initial_dsp_mode = set_dsp_mode(Mode)</code>
Parameters	Mode Desired DSP mode. As an unsigned 32-bit value.
Result	DSP mode for which the board has been set during initialization. As an unsigned 32-bit value.
Comments	• set_dsp_mode is only needed when multiple RTC5 Boards with differing DSP versions are to be operated synchronously in a master/slave chain (see page 114), and then only if matching of short list counts is not to be performed by sync_slaves. • Even if another mode has been already set by set_dsp_mode, set_dsp_mode always returns the DSP mode that the board has been set to during initialization. This return value (<code>initial_dsp_mode</code>) is identical to Byte #2 of get_rtc_version (DSP version number). • No mode can be set that is larger than <code>initial_dsp_mode</code> (Mode is clipped to <code>initial_dsp_mode</code>). Thus, faster boards can only be matched to slower boards, not the other way around.
RTC4→RTC5	New command.
Version info	–
References	sync_slaves , get_rtc_version

Undelayed Short List Command	set_ellipse
Function	Defines the shape of an elliptical arc that can subsequently be marked by mark_ellipse_abs or mark_ellipse_rel .
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), set_ellipse is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>set_ellipse(a, b, Phi0, Phi)</code>
Parameters	a Length of the elliptical half-axis. In bits. As an unsigned 32-bit value. Allowed value range: [1...+8,388,607]. Out-of-range positive values are clipped to the boundary values.
	b See a.
	Phi0 Beginning phase angle (the arc starting point position relative to the end point of half-axis a). In degrees. As a 64-bit IEEE floating point value. A positive sign means "clockwise". Phi0 gets normalized to the value range [0...<360°].
	Phi Arc angle (to-be-marked ellipse section). In degrees. As a 64-bit IEEE floating point value. A positive sign means "clockwise". Allowed value range: [-2,880.0°...+2,880.0°] (±8 full ellipses). Out-of-range values are clipped to the boundary values.
Comments	- Specify the to-be-marked elliptical arc's position and orientation in the 2D Image field by mark_ellipse_abs or mark_ellipse_rel. For descriptions of the individual parameters, see Section "Ellipse Commands", Page 126. - For $a < 1$ and/or $b < 1$, set_ellipse is replaced with a list_nop (get_last_error return code RTC5_PARAM_ERROR).
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for a and b by 16. The allowed value range decreases accordingly.
Version info	Last change DLL 551, OUT 553: Allowed value range for Phi has been reduced (previously: [-3,600.0°...+3,600.0°] = ±10 full ellipses).
References	mark_ellipse_abs , mark_ellipse_rel

Delayed Short List Command	set_encoder_speed	
Function	Defines the target encoder speed and further parameters for encoder-speed-dependent laser control.	
Call	set_encoder_speed(EncoderNo, Speed, Smooth)	
Parameters	EncoderNo	Number of the encoder counter to be used for speed measurement as an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1". = 2 and 3: vectorial encoder velocity: a vectorial velocity is calculated from both encoder speeds (by the pulse rates of both encoder counters) for use with encoder-speed-dependent laser control in Mode= 5. Bits with higher significance are ignored.
	Speed	Target encoder speed (counter pulse rate) in [counter pulses/ms]. As a 64-bit IEEE floating point value. Allowed value range: [100.0...16000.0]. Out-of-range values are clipped to the boundary values.
	Smooth	Smoothing factor for a 2-stage low-pass filter. As a 64-bit IEEE floating point value. Allowed value range: [0.0...1.0]. Larger values are clipped.
Comments	• If Smooth = 0.0, then only the current encoder speed CurrentSpeed (based on counter pulses received in the most recent 10 μs) is used; if Smooth = 1.0, then the speed from the previous clock cycle PreviousSpeed. In general, the used speed is: $Speed = PreviousSpeed \times Smooth + CurrentSpeed \times (1.0 - Smooth)$. • If Speed ≤ 0.0 or Smooth < 0.0, then set_encoder_speed is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). • One encoder increment results in 4 counter pulses, see Section "Input Ports for External Encoder Signals", Page 275. • The maximum value for Speed (16000.0) corresponds to a counting rate of 16 MHz. The minimum value for Speed (100.0) corresponds to a counting rate of 1/(10 μs), that is, one counter pulse per output period. Beware of the low resolution of encoder-speed-dependent laser control for low speed values! Encoder-speed-dependent correction is only recommended if substantially more than one counter pulse per output period (10 μs) are received. • See also Chapter "Encoder-Speed-Dependent Laser Control", Page 192.	
RTC4→RTC5	New command.	
Version info	–	
References	set_encoder_speed_ctrl, set_auto_laser_control	

Ctrl Command	set_encoder_speed_ctrl
Function	Like set_encoder_speed , but a control command.
Call	set_encoder_speed_ctrl(EncoderNo, Speed, Smooth)
Parameters	EncoderNo Like set_encoder_speed .
	Speed Like set_encoder_speed .
	Smooth Like set_encoder_speed .
Comments	• set_encoder_speed_ctrl is not executed (get_last_error return code RTC5_PARAM_ERROR), if: – Speed ≤ 0.0 – Smooth < 0.0 • set_encoder_speed_ctrl is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set • set_encoder_speed_ctrl is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) – the INTERNAL-BUSY list execution status is set
RTC4→RTC5	New command.
Version info	–
References	set_encoder_speed , set_auto_laser_control

Normal List Command	set_end_of_list
Function	Ends execution of a list.
Call	<code>set_end_of_list()</code>
Comments	• If, during processing of a list, set_end_of_list is encountered and <i>no</i> automatic list change has been previously activated (see Chapter 6.4.6 "Automatic List Changing", Page 99), then list execution ends. The Signals for "Laser Active" Operation are then switched off and a home jump, if defined by home_position or home_position_xyz, is executed (the INTERNAL-BUSY list execution status is set while the home jump is executed). • In contrast, upon reaching a set_end_of_list, execution continues at the other list if an automatic list change has been previously <i>activated</i>. The other list can also be "List 1" if "List 2" has not been configured (Mem2 = 0, see config_list). • Upon processing of a set_end_of_list, the USED list status of the respective list (USED1 or USED2) is always set and the BUSY list status (BUSY1 or BUSY2) of the list is reset, see also Chapter 6.4.3 "List Execution Status", Page 97. The BUSY list execution status, on the other hand, is only reset if <i>no</i> automatic list change has been previously activated. • An automatic list change of the input pointer never occurs during <i>loading</i> of set_end_of_list (in contrast to an automatic list change of the <i>output pointer</i> during execution of set_end_of_list, if previously activated by auto_change_pos or start_loop). • Upon loading set_end_of_list, the READY list status (READY1 or READY2) is set and LOAD list status (LOAD1 or LOAD2) is reset. Additionally, flushing of the buffered list input is triggered, see Chapter 6.4.1 "Loading Lists", Page 94. • set_end_of_list is ignored during loading and execution, that is, replaced with a list_nop if an indexed subroutine is currently being loaded or executed (get_last_error return code RTC5_IGNORED).
RTC4→RTC5	Basically unchanged functionality. However: Additional USED list status .
Version info	–
References	set_start_list

Ctrl Command	set_extstartpos
Function	Defines the start address (within the range of "List 1" or "List 2") where execution should continue upon External Starts .
Call	<code>set_extstartpos(Pos)</code>
Parameters	Pos Absolute address of the first list command to be executed. As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{20} - 1)]$.
Comments	- By default, an External Start results in a continuation or start at the beginning of "List 1" ($Pos = 0$). Pos must be within the range of "List 1" or "List 2". Otherwise, $Pos = 0$ is set. - The specified start address is used for External Starts until a new address is specified by set_extstartpos or set_extstartpos_list. An address range validity check is performed on Pos before each External Start; Pos might therefore become set to 0 at a later point (and remain at this value) if the configuration has been correspondingly changed in the meantime (see config_list). - See also Section "External Start", Page 266.
RTC4→RTC5	New command.
Version info	–
References	set_extstartpos_list , set_control_mode

Undelayed Short List Command	set_extstartpos_list
Function	Like set_extstartpos , but a list command.
Call	<code>set_extstartpos_list(Pos)</code>
Parameters	Pos Like set_extstartpos .
Comments	- This list command can be used within a list, to "link" it to the list that follows. - See set_extstartpos.
RTC4→RTC5	Unchanged functionality. In addition: increased value range.
Version info	–
References	set_extstartpos

Ctrl Command	set_ext_start_delay
Function	Sets a track delay for External Starts , so that the lists are started with a delay relative to the triggering input signal or simulate_ext_start or simulate_ext_start_ctrl .
Call	set_ext_start_delay(Delay, EncoderNo)
Parameters	Delay Track delay (counter steps of the selected encoder counter EncoderNo). As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$. EncoderNo Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".
Comments	• For External Starts, see Section "External Start", Page 266. • The track delay is specified in <i>relative</i> counting units of the selected encoder counter (the RTC5 encoder counter is triggered by an external or simulated encoder signal, see page 274). • Ensure that the sign of the track delay (Delay parameter) is appropriate for the selected encoder's counting direction (for external triggering, this corresponds to the workpiece's direction of motion and is always positive with simulated encoders). • For Delay = 0, the track delay is deactivated. • If a track delay is specified, that causes a start trigger (initiated by simulate_ext_start or an external start signal) to occur when the BUSY list execution status or INTERNAL-BUSY list execution status is set (for example, when outputting a list or during goto_xy), then no starts are get triggered by this start trigger (in this case, Bit #11 of the get_startstop_info return value is set). • Track delays can also be set with simulate_ext_start. Track delays are deactivated by initialization (with load_program_file), by an External Stop and by stop_execution. They can also be deactivated with set_control_mode/set_control_mode_list (Bit #2). • set_ext_start_delay cancels already externally triggered starts that have not yet executed and are still being held in a queue that accommodates up to 8 starts. • If EncoderNo > 1, then set_ext_start_delay is ignored (get_last_error return code RTC5_PARAM_ERROR).

Ctrl Command	set_ext_start_delay
RTC4→RTC5	Unchanged functionality. In addition: increased value range. The RTC5 allows this command to be used not only together with an external encoder, but also with an encoder simulation (see Section "Encoder Simulation", Page 275) started by simulate_encoder. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for Delay by 16. The allowed value range decreases accordingly.
Version info	–
References	simulate_ext_start , set_ext_start_delay_list , set_extstartpos , set_extstartpos_list , set_control_mode , set_control_mode_list

Normal List Command	set_ext_start_delay_list
Function	Like set_ext_start_delay , but a list command.
Call	set_ext_start_delay_list(Delay, EncoderNo)
Parameters	Delay Like set_ext_start_delay. EncoderNo Like set_ext_start_delay.
Comments	If EncoderNo > 1, then set_ext_start_delay_list is replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR).
RTC4→RTC5	See set_ext_start_delay .
Version info	–
References	set_ext_start_delay .

Ctrl Command	set_firstpulse_killer
Function	Sets the length of the FirstPulseKiller signal for a YAG laser.
Call	<code>set_firstpulse_killer(Length)</code>
Parameters	Length Length of the FirstPulseKiller signal. As an unsigned 32-bit value. 1 bit equals 1/64 μs. Allowed value range: $[0 \dots + (2^{26} - 1)]$. Out-of-range values are clipped to the boundary values.
Comments	• The time base for the FirstPulseKiller signal is 64 MHz. 1 bit equals 1/64 μs. • The signal level is defined by set_laser_control. • For YAG Mode 2, the Q-Switch delay is also correspondingly altered, see Section "Differences Between the YAG Modes", Page 180. • In CO₂ Mode, set_firstpulse_killer has no effect. • The laser control mode is set by set_laser_mode, see Chapter 7.4 "Laser Control", Page 173. • Q-Switch pulse length and Q-Switch period can be set by set_laser_pulses_ctrl, set_laser_pulses or set_laser_timing.
RTC4→RTC5	Basically unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified value by 8. The allowed value range decreases accordingly.
Version info	–
References	set_firstpulse_killer_list , set_laser_mode , set_laser_timing

Undelayed Short List Command	set_firstpulse_killer_list
Function	Like set_firstpulse_killer , but a list command.
Call	<code>set_firstpulse_killer_list(Length)</code>
Parameters	Length Like set_firstpulse_killer .
Comments	• See set_firstpulse_killer.
RTC4→RTC5	See set_firstpulse_killer
Version info	–
References	set_firstpulse_killer

Normal List Command	set_fly_2d
Function	Activates Processing-on-the-fly correction for compensation of a linear workpiece-motion in 2 dimensions (based on the encoder values transferred to the RTC5 by encoder counters "Encoder0" and "Encoder1"). Sets the corresponding scaling factors.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_2d terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_2d(ScaleX, ScaleY)</code>
Parameters	ScaleX Scaling factor for the x direction (encoder counter "Encoder0"). In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleX \leq 16000.0$.
	ScaleY Scaling factor for the y direction (encoder counter "Encoder1"). In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleY \leq 16000.0$.
Comments	• ScaleX and ScaleY can be negative depending on the motion direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction (for example, determination of the scaling factor or deactivating Processing-on-the-fly correction), see the Chapter 8.6 "Processing-on-the-fly", Page 227. For set_fly_2d usage, see the Chapter 8.6.4 "Compensating 2D Motions", Page 234. • If unallowed parameter values are supplied (for example, for ScaleX = 0), then set_fly_2d does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_2d (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_2d" Processing-on-the-fly correction). However, Processing-on-the-fly correction successfully activated by set_fly_2d switches off any other Processing-on-the-fly correction and does itself get switched off by any other Processing-on-the-fly command, even if that other command contains unallowed parameters, see Section "Overview", Page 227. • If an encoder compensation has been set by load_fly_2d_table and init_fly_2d, the current encoder values are added to the latest reference values of the 2D encoder compensation and then the sums are saved as new reference values. The encoder counters are then reset to 0. • It should <i>not</i> be set by set_control_mode(Bit #9) that the encoder counters are only reset after the subsequent External Start trigger. Otherwise, the reference values of 2D encoder compensation are lost. See also Section "2D Encoder Compensation for xy Positioning Stages", Page 234. • Do not intermediately call set_fly_x or set_fly_y to switch on the Processing-on-the-fly application, if you intend to use set_fly_2d in conjunction with 2D encoder compensation for an xy positioning stage, because here too the reference values are lost.

Normal List Command	set_fly_2d
Comments (cont'd)	- If no correction table for 2D encoder compensation has yet been loaded onto the board (see load_fly_2d_table), then the encoder values are used without correction. - You can also use activate_fly_2d/activate_fly_2d_encoder to switch on set_fly_2d Processing-on-the-fly correction.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for <i>ScaleX</i> and <i>ScaleY</i> by 16. The allowed value range decreases accordingly.
Version info	–
References	init_fly_2d , load_fly_2d_table , get_fly_2d_offset , activate_fly_2d , activate_fly_xy

Undelayed Short List Command	set_fly_limits
Function	Defines the boundaries of the customer-defined monitoring area for Processing-on-the-fly applications.
Call	<code>set_fly_limits(Xmin, Xmax, Ymin, Ymax)</code>
Parameters	Xmin Range boundary. As a signed 32-bit value. Allowed value range: [–524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	Xmax Like <i>Xmin</i> (analogously).
	Ymin Like <i>Xmin</i> (analogously).
	Ymax Like <i>Xmin</i> (analogously).
Comments	- For set_fly_limits usage, see Section “Customer-Defined Monitoring Area”, Page 241. - During initialization (with load_program_file), the boundary limits get set to the following default values: <i>Xmin</i> = <i>Ymin</i> = –524,288 and <i>Xmax</i> = <i>Ymax</i> = +524,287. - Boundary limits specified using the parameter value 0 also get set to the above-mentioned default values.
RTC4→RTC5	New command.
Version info	–
References	get_marking_info

Undelayed Short List Command	set_fly_limits_z
Function	Defines the boundaries of the customer-defined monitoring range for z axis Processing-on-the-fly applications.
Call	<code>set_fly_limits_z(Zmin, Zmax)</code>
Parameters	Zmin Range boundary. As a signed 32-bit value. Allowed value range: [-32,768...+32,767]. Out-of-range values are clipped to the boundary values.
	Zmax Like Zmin (analogously).
Comments	• For set_fly_limits_z usage, see also Chapter 8.6.9 "Monitoring Processing-on-the-fly Corrections", Page 240. • During initialization (with load_program_file), the boundary limits get set to the following default values: $Z_{min} = -32,768$ and $Z_{max} = +32,767$. • Boundary limits specified using the parameter value 0 also get set to the above-mentioned default values.
RTC4→RTC5	New command.
Version info	–
References	set_fly_limits , get_marking_info

Normal List Command	set_fly_rot
Function	Activates Processing-on-the-fly correction for compensation of workpiece rotary motion (based on angular position values transferred to the RTC5 by encoder counter "Encoder0") and sets the corresponding <code>Resolution</code> parameter.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_rot terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_rot(Resolution)</code>
Parameters	<code>Resolution</code> Number of steps per revolution. As a 64-bit IEEE floating point value. Allowed value range: $ \text{Resolution} > 100.0$.
Comments	• <code>Resolution</code> can be negative depending on the rotation direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction and determining the <code>Resolution</code> parameter, see Chapter 8.6 "Processing-on-the-fly", Page 227. • Before executing set_fly_rot, you should define the rotation center for Processing-on-the-fly correction by set_rot_center or set_rot_center_list. Otherwise, the default value (0 0) is applied. • If unallowed parameter values are supplied (for example, for <code>Resolution = 0</code>), then set_fly_rot does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_rot (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_rot" Processing-on-the-fly correction). • The various Processing-on-the-fly corrections cannot be arbitrarily combined, see Section "Overview", Page 227. • For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. • By set_control_mode(Bit #9), it can be set in advance when the encoder counter "Encoder0" is reset by set_fly_rot.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_rot_center , set_rot_center_list , fly_return , get_encoder , set_fly_rot_pos

Normal List Command	set_fly_rot_pos
Function	Activates Processing-on-the-fly correction for compensation of workpiece or scan system rotary motion (based on angular position values transferred to the RTC5 by the McBSP interface). It thereby sets the corresponding Resolution parameter.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_rot_pos terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_rot_pos(Resolution)</code>
Parameters	Resolution Number of steps per revolution. As a 64-bit IEEE floating point value. Allowed value range: $ \text{Resolution} > 100.0$.
Comments	Resolution can be negative depending on the rotation direction of the workpiece. The restricted value range applies only to the absolute value. - For Processing-on-the-fly correction and determining the Resolution parameter, see Chapter 8.6 "Processing-on-the-fly", Page 227. - Before calling set_fly_rot_pos, you should define the rotation center for Processing-on-the-fly correction by set_rot_center or set_rot_center_list. Otherwise, the default value (0 0) is applied. - If unallowed parameter values are supplied (for example, for Resolution = 0), then set_fly_rot_pos does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_rot_pos (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_rot_pos" Processing-on-the-fly correction). - The various Processing-on-the-fly corrections cannot be arbitrarily combined (see the page 227). - For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. - The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section "Notes", Page 215. - The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbasp_init. That is, data provided is not transmitted, see page 73.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_rot_center , set_rot_center_list , fly_return , read_mcbasp , set_fly_rot

Ctrl Command	set_fly_tracking_error
Function	Activates or deactivates Tracking Error Compensation of Encoder Values for Processing-on-the-fly Applications .
Call	set_fly_tracking_error(TrackingErrorX, TrackingErrorY)
Parameters	TrackingErrorX Tracking error for the x axis. In units of 10 μ s. As a signed 32-bit value. Allowed value range: [0...65,535]. Excessive bits are ignored.
	TrackingErrorY Tracking error for the y axis. Otherwise, like TrackingErrorX.
Comments	• Tracking error compensation is a first-approximation correction: $EC = E + (E - EP) * TrackingError,$ whereby E is the current encoder value, EP is that of the previous cycle and EC is the compensated encoder value (used for Processing-on-the-fly correction). • To avoid step-like galvanometer scanner motions, the used encoders should have a sufficiently high resolution (counts per 10 μs cycle). • If TrackingErrorX,Y = 0 (initialization value), then compensation is switched off. • store_encoder, read_encoder and get_encoder still use the original, uncorrected encoder values. • This compensation is practical only for linear motions (Chapter 8.6.2 "Compensation of Linear Motions", Page 228), not for rotary motions (Chapter 8.6.3 "Compensating Rotary Motions", Page 232). For the latter, TrackingErrorX,Y should be set to 0. • If the encoders get reset by the start trigger (set_control_mode(Bit #9 = 1)), then the output value of the first 10 μs cycle might be incorrect to an unforeseen extent. Resets by the set_fly_x and set_fly_y commands automatically wait internally for an empty cycle. • This compensation only functions for Processing-on-the-fly applications with encoders, but not with position signals by the McBSP interface.
RTC4→RTC5	New command.
Version info	–
References	–

Normal List Command	set_fly_x
Function	Activates Processing-on-the-fly correction for compensation of a linear workpiece-motion in the x direction (based on position values transferred to the RTC5 by encoder counter "Encoder0") and sets the corresponding scaling factor.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_x terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_x(ScaleX)</code>
Parameters	ScaleX Scaling factor for the x direction (encoder counter "Encoder0"). In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleX \leq 16000.0$.
Comments	• ScaleX can be negative depending on the motion direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction and determination of the scaling factor, see Chapter 8.6 "Processing-on-the-fly", Page 227. • If unallowed parameter values are supplied (for example, for ScaleX = 0), then set_fly_x does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_x (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_x" Processing-on-the-fly correction). • The various Processing-on-the-fly corrections cannot be arbitrarily combined (see the page 227). • For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. • By set_control_mode(Bit #9), it can be set in advance when the encoder counter "Encoder0" is reset by set_fly_x. • You can also switch on set_fly_x/set_fly_y Processing-on-the-fly correction by activate_fly_xy/activate_fly_xy_encoder.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for ScaleX by 16. The allowed value range decreases accordingly.
Version info	–
References	fly_return, set_fly_y, get_encoder, set_fly_x_pos, set_fly_y_pos, activate_fly_xy, set_fly_2d

Normal List Command	set_fly_x_pos
Function	Activates Processing-on-the-fly correction for compensation of a linear workpiece or scan system motion in the x direction (based on position values transferred to the RTC5 by the McBSP interface); thereby sets the corresponding scaling factor.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_x_pos terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_x_pos(ScaleX)</code>
Parameters	ScaleX Scaling factor for the x direction in (RTC5)bits/(McBSP)bit. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleX \leq 16000.0$.
Comments	• ScaleX can be negative depending on the motion direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction and determination of the scaling factor, see Chapter 8.6 "Processing-on-the-fly", Page 227. • If unallowed parameter values are supplied (for example, for ScaleX = 0), then set_fly_x_pos does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_x_pos (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_x_pos" Processing-on-the-fly correction). • The various Processing-on-the-fly corrections cannot be arbitrarily combined (see the page 227). • For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. • For 1D correction (when only set_fly_x_pos is used), the McBSP interface provides a (signed) 32-bit value. In contrast, only a (signed) 16-bit value per axis is supplied for 2D correction (set_fly_x_pos and set_fly_y_pos). Here, set_fly_x_pos uses the McBSP interface's lower 16 bits for the x value and set_fly_x_pos uses its upper 16-bits for the y value. • The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section "Notes", Page 215. • The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbbsp_init. That is, data provided is not transmitted, see page 73.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for ScaleX by 16. The allowed value range decreases accordingly.
Version info	–
References	fly_return , set_fly_y_pos , read_mcbbsp , set_fly_x , set_fly_y

Normal List Command	set_fly_y
Function	Activates Processing-on-the-fly correction for compensation of a linear workpiece-motion in the y direction (based on position values transferred to the RTC5 by encoder counter "Encoder1") and sets the corresponding scaling factor.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_y terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_y(ScaleY)</code>
Parameters	ScaleY Scaling factor for the y direction (encoder counter "Encoder1"). In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleY \leq 16000.0$.
Comments	• ScaleY can be negative depending on the motion direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction and determination of the scaling factor, see Chapter 8.6 "Processing-on-the-fly", Page 227. • If unallowed parameter values are supplied (for example, for ScaleY = 0), then set_fly_y does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_y (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_y" Processing-on-the-fly correction). • The various Processing-on-the-fly corrections cannot be arbitrarily combined (see the page 227). • For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. • By set_control_mode(Bit #9), it can be set in advance when the encoder counter "Encoder0" is reset by set_fly_y. • You can also switch on set_fly_x/set_fly_y Processing-on-the-fly correction by activate_fly_xy/activate_fly_xy_encoder.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for ScaleY by 16. The allowed value range decreases accordingly.
Version info	–
References	fly_return, set_fly_x, get_encoder, set_fly_x_pos, set_fly_y_pos, activate_fly_xy, set_fly_2d

Normal List Command	set_fly_y_pos
Function	Activates Processing-on-the-fly correction for compensation of a linear workpiece or scan system motion in the y direction (based on position values transferred to the RTC5 by the McBSP interface); thereby sets the corresponding scaling factor.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_y_pos terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_fly_y_pos(ScaleY)</code>
Parameters	ScaleY Scaling factor for the y direction in (RTC5)bits/(McBSP)bit. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleY \leq 16000.0$.
Comments	• ScaleY can be negative depending on the motion direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction and determination of the scaling factor, see Chapter 8.6 "Processing-on-the-fly", Page 227. • If unallowed parameter values are supplied (for example, for ScaleY = 0), then set_fly_y_pos does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_y_pos (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_y_pos" Processing-on-the-fly correction). • The various Processing-on-the-fly corrections cannot be arbitrarily combined (see page 227). • For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. • For 1D correction (when only set_fly_y_pos is used), the McBSP interface provides a (signed) 32-bit value. In contrast, only a (signed) 16-bit value per axis is supplied for 2D correction (set_fly_x_pos and set_fly_y_pos). Here, set_fly_x_pos uses the McBSP interface's lower 16 bits for the x value and set_fly_y_pos uses its upper 16-bits for the y value. • The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section "Notes", Page 215. • The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbbsp_init. That is, data provided is not transmitted, see page 73.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for ScaleY by 16. The allowed value range decreases accordingly.
Version info	–
References	fly_return , set_fly_x_pos , read_mcbbsp , set_fly_x , set_fly_y

Normal List Command	set_fly_z				
Function	Activates Processing-on-the-fly correction for compensation of a linear workpiece-motion in the z direction (based on position values transferred to the RTC5 by the specified encoder counter) and sets the corresponding scaling factor.				
Restriction	If the Option Processing-on-the-fly is not enabled, then set_fly_z terminates the Processing-on-the-fly process (even though it could not have been activated).				
Call	<code>set_fly_z(ScaleZ, EncoderNo)</code>				
Parameters	<table border="0"> <tr> <td>ScaleZ</td><td>Scaling factor for the z direction. In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleZ \leq 16,000.0$.</td></tr> <tr> <td>EncoderNo</td><td>Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".</td></tr> </table>	ScaleZ	Scaling factor for the z direction. In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleZ \leq 16,000.0$.	EncoderNo	Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".
ScaleZ	Scaling factor for the z direction. In bits/count. As a 64-bit IEEE floating point value. Allowed value range: $1/256 \leq ScaleZ \leq 16,000.0$.				
EncoderNo	Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".				
Comments	• ScaleZ can be negative depending on the motion direction of the workpiece. The restricted value range applies only to the absolute value. • For Processing-on-the-fly correction and determination of the scaling factor, see Chapter 8.6 "Processing-on-the-fly", Page 227. • If unallowed ScaleZ parameter values are supplied (for example, for ScaleZ = 0), then set_fly_z does not activate a Processing-on-the-fly correction or deactivates a Processing-on-the-fly correction previously activated by set_fly_z (but does not deactivate any other Processing-on-the-fly correction). The latter case leads to a jump (at jump speed) to the endpoint of the most recently executed Vector command or "Arc" command (without "set_fly_z" Processing-on-the-fly correction). • For deactivating Processing-on-the-fly correction, see Section "Notes on Usage", Page 243. • By set_control_mode(Bit #9), it can be set in advance when the encoder counter "Encoder0" or "Encoder1" is reset by set_fly_z. • If EncoderNo > 1, set_fly_z is replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR).				
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for ScaleZ by 16. The allowed value range decreases accordingly.				
Version info	–				
References	fly_return_z				

Ctrl Command	set_free_variable	
Function	Sets a free variable to the desired value.	
Call	set_free_variable(No, Value)	
Parameters	No	Number of the free variable to be set. As an unsigned 32-bit value. Allowed value range: [0...7]. Only the 3 least significant bits are evaluated.
	Value	Desired variable value. As an unsigned 32-bit value. Allowed value range: [0...(2 ³² -1)].
Comments	See Chapter 6.9.1 "Free Variables", Page 123.	
RTC4→RTC5	New command.	
Version info	Last change DLL 539, OUT 539: value range of parameter No increased to [0...7].	
References	set_free_variable_list , get_free_variable , set_trigger	

Undelayed Short List Command	set_free_variable_list	
Function	Like set_free_variable , but a list command.	
Call	set_free_variable_list(No, Value)	
Parameters	No	Like set_free_variable .
	Value	Like set_free_variable .
Comments	See set_free_variable.	
RTC4→RTC5	New command.	
Version info	–	
References	set_free_variable	

Ctrl Command	set_hi
Function	Defines gain values and offset values for the galvanometer scanners of the scan system attached to the specified scan head connector.
Call	set_hi(HeadNo, GalvoGainX, GalvoGainY, GalvoOffsetX, GalvoOffsetY)
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. Allowed values: = 1: First scan head connector. = 2: Second scan head connector.
	GalvoGainX x gain value. As a 64-bit IEEE floating point value. Allowed value range: [0.01...100].
	GalvoGainY Like GalvoGainX (analogously).
	GalvoOffsetX x offset value. In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287].
	GalvoOffsetY Like GalvoOffsetX (analogously).
Comments	- For set_hi usage, see Section “Customer-Specific Calibration”, Page 254. - The specified gain values and offset values overwrite the values that were set by auto_cal and can themselves be overwritten by a subsequent call of auto_cal. - With changed gain values and offset values, the transition is automatically performed at the predefined jump speed (see set_jump_speed). set_hi is not executed, if: - A parameter value is invalid (get_last_error return code RTC5_PARAM_ERROR) - the BUSY list execution status is set (get_last_error return code RTC5_BUSY) - the INTERNAL-BUSY list execution status is set (get_last_error return code RTC5_BUSY) set_hi is even executed, if: - a list has been paused by set_wait (PAUSED list execution status set) - For RTC5_PARAM_ERROR, the BUSY list execution status is not checked. Therefore the return codes RTC5_BUSY and RTC5_PARAM_ERROR do not occur simultaneously. - If the Option “Second Scan Head Control” has not been enabled, values specified for the second scan head control have no effect.
RTC4→RTC5	New command.
Version info	–
References	auto_cal , get_hi_pos , write_hi_pos

Ctrl Command	set_input_pointer
Function	Opens the list memory area for writing with list commands and sets the input pointer to the specified <i>absolute</i> address in list memory ("List 1" or "List 2").
Call	<code>set_input_pointer(Pos)</code>
Parameters	Pos Position (absolute memory address) of the input pointer. [0...(2 ²⁰ -1)]. As an unsigned 32-bit value.
Comments	• The next list command is stored at the specified address and all further list commands at the subsequent addresses in the selected list. • set_input_pointer performs basically like set_start_list_pos (see the comments there). But set_input_pointer sets the input pointer based on a specified <i>absolute</i> memory address, whereas set_start_list_pos uses a specified list number and a <i>relative</i> memory address. • For $Pos \geq Mem1 + Mem2$ (see config_list), Pos is set to 0.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_start_list_pos , get_input_pointer

Undelayed Short List Command	set_io_cond_list
Function	Sets the bits of the 16-bit digital output port on the EXTENSION 1 socket connector that are set in the parameter <code>MaskSet</code> , if the current <code>IOvalue</code> at the 16-bit digital <i>input</i> port on the EXTENSION 1 socket connector meets the following condition: $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits specified in <code>Mask1</code> are 1 and the bits specified in <code>Mask0</code> are 0).
Call	<code>set_io_cond_list(Mask1, Mask0, MaskSet)</code>
Parameters	<code>Mask1</code> 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	<code>Mask0</code> Like <code>Mask1</code> .
	<code>MaskSet</code> Like <code>Mask1</code> .
Comments	set_io_cond_list sets only those bits of the digital output port that are set in the parameter <code>MaskSet</code> and leaves the other bits unchanged. - See also Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65 and Chapter 9.3.2 "Conditional Command Execution", Page 271.
Examples (Pascal)	- Set Bit #4 of the output port (DIGITAL OUT4), if Bit #0 of the input port (DIGITAL IN0) is set and Bit #1 to Bit #3 (DIGITAL IN1...3) of the input port are not set: <code>set_io_cond_list(\$0001, \$000E, \$0010)</code> - Always set Bit #15 of the output port (and leave the other bits unchanged): <code>set_io_cond_list(0, 0, \$8000)</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	clear_io_cond_list , write_io_port , write_io_port_mask , get_io_status , read_io_port

Ctrl Command	set_jump_mode
Function	Enables and activates or disables and deactivates Jump Mode for 2D jumps and sets the related parameters.
Prerequisite	Enabling is only possible if a jump-tuning-equipped intelliSCAN (with scan system firmware version ≥ 2078) is attached to at least one of the two scan head connectors. Otherwise, set_jump_mode has no effect.
Call	ErrorCode = set_jump_mode(Flag, Length, VA1, VA2, VB1, VB2, JA1, JA2, JB1, JB2)
Parameters	Flag Switching flag. As a signed 32-bit value. Allowed values: = -1: Jump Mode is disabled. Afterward, switching the Flag by set_jump_mode_list is <i>no longer</i> possible. = 0: Jump Mode is enabled but deactivated. Afterwards it is also possible to switch the flag by set_jump_mode_list. = 1: Jump Mode is enabled and activated. Afterwards it is also possible to switch the flag by set_jump_mode_list.
	Length Limit of the jump length (per axis) under which 2D jumps – even with activated Jump Mode – are performed in vector mode. As an unsigned 32-bit value.
	VA1 Numbers of the tunings that should be used for jump execution in Jump Mode: VA2 V: Vector tuning that is set at the end of a 2D jump. VB1 J: Jump tuning that is set at the beginning of a 2D jump. VB2 A First scan head connector. JA1 B Second scan head connector. JA2 1: x axis (STATUS channel, Galvanometer scanner 2). 2: y axis (STATUS1 channel, Galvanometer scanner 1). JB1 Allowed values: JB2 = - 1: Tuning is neither checked nor used. = 0...3: After passing a check, tuning is used for Jump Mode. The allowed value range also depends on the number of tunings with which the attached scan system is equipped.
	As a signed 32-bit value.

Ctrl Command	set_jump_mode
Result	Error code. As a signed 32-bit value. 0 No error: Flag successfully switched to 0 (Jump Mode deactivated, vector mode activated). 1 No error: Flag successfully switched to 1 (Jump Mode activated, vector mode deactivated). -1 Flag successfully switched to -1 (Jump Mode deactivated and disabled). -2 Busy error: board BUSY list execution status or INTERNAL-BUSY list execution status (get_last_error return code RTC5_BUSY). -3 Board not responding: no program loaded or PCI error (get_last_error return code RTC5_TIMEOUT). -4 Access error: board reserved for another user program (get_last_error return code RTC5_ACCESS_DENIED). > 1 Did not pass the check (see also notes). Flag is set to -1 (Jump Mode deactivated and disabled). The following is returned: Byte #0 = 255. Byte #1 = Error code for first scan head connector. Byte #2 = Error code for second scan head connector. Byte #3 = 0. Whereby error code: =1: x axis (Galvanometer scanner 2) not responding or no intelliSCAN (with scan system firmware version ≥ 2078) attached. =2: y axis (Galvanometer scanner 1) not responding or no intelliSCAN (with scan system firmware version ≥ 2078) attached. =4: no correction table assigned. =8: incorrect tuning number(s): incorrect type or unsuitable for rapid switching.

Ctrl Command	set_jump_mode
Comments	- For usage of set_jump_mode, see Chapter 8.1.5 "Jump Mode", Page 202. - A check (see also Section "Requirements and Activation", Page 203) is only performed if <code>Flag = -1</code> (the initialization state) prior to the set_jump_mode call and/or if the supplied tuning numbers do not match those stored on the board. Otherwise, only the flag is switched. For the check, the board must not be BUSY list execution status or INTERNAL-BUSY list execution status, because meanwhile the data type to-be-returned by the scan system changes and "Automatic Laser Control" is deactivated (both get restored at the end of the command). Depending on results of the check, different error codes are returned (see above). In case of error, <code>Flag</code> gets set to <code>-1</code> (Jump Mode deactivated and disabled). If the check is successful, then you can afterward (even by set_jump_mode_list during processing of a list) switch freely between the states <code>Flag = 1</code> (Jump Mode activated, vector mode deactivated) and <code>Flag = 0</code> (Jump Mode deactivated, vector mode activated) without another check having to be performed. - Use <code>-1</code> as the tuning number if certain tunings should not be checked (for example, because no intelliSCAN scan system is attached or specific tunings are not available – Vector tuning, for example, is not needed in pure drilling applications) or if, after switching to Jump tuning, it is not desirable to return to Vector tuning. As a result, such tunings are neither checked nor switched on. If the Option "Second Scan Head Control" is not enabled, then the tuning numbers for the second scan head connector (B) is automatically set to <code>-1</code> (even if others were supplied). If all tuning numbers are <code>-1</code>, then <code>Flag</code> is set to <code>-1</code> (return value <code>-1</code>). - Even after successful activation of Jump Mode (<code>Flag = 1</code>), the first servo switching only occurs after the first subsequent 2D jump (see Section "Functional Principle", Page 202). If the currently set tuning then does not match the Jump tuning or Vector tuning specified by set_jump_mode, then the first switching can take somewhat longer (approx. 250 ms), depending on the currently set tuning. You can determine ahead of time whether this is so by calling set_jump_mode using the currently set tuning as a parameter. If true (return value <code>> 1</code>, error code = 2), then you can achieve the desired operational sequence by calling control_command before the first 2D jump to set the tuning to one that has been supplied by set_jump_mode.
RTC4→RTC5	New command.
Version info	–
References	set_jump_mode_list , load_jump_table_offset , set_jump_table

Normal List Command	<code>set_jump_mode_list</code>
Function	Activates or deactivates and disables Jump Mode for 2D jumps.
Prerequisite	Like set_jump_mode .
Call	<code>set_jump_mode_list(Flag)</code>
Parameters	Flag See set_jump_mode .
Notes	• For <code>set_jump_mode_list</code> usage, see Chapter 8.1.5 "Jump Mode", Page 202. • <code>set_jump_mode_list</code> functions like the control command set_jump_mode (see notes there) but, as a list command, has the following differences: – No tunings or jump length limit can be specified by <code>set_jump_mode_list</code>. – <code>set_jump_mode_list</code> does not perform a check. Flag must have previously been successfully set to 0 or 1 by set_jump_mode. – Though Jump Mode can be deactivated and disabled by setting Flag to -1 by set_jump_mode or <code>set_jump_mode_list</code>, it can only be reactivated again by set_jump_mode (if the check is successful). If Flag has been -1 prior to calling <code>set_jump_mode_list</code>, then <code>set_jump_mode_list</code> has no effect.
RTC4→RTC5	New command.
Version info	–
Reference	set_jump_mode

Undelayed Short List Command	set_jump_speed
Function	Defines the jump speed for Vector commands .
Call	set_jump_speed(Speed)
Parameters	Speed Jump speed. In <i>bits/ms</i> . As a 64-bit IEEE floating point value. Allowed value range: [1.6...800000.0]. Out-of-range values are clipped to the boundary values.
Comments	• By default a jump speed of 10000 <i>bits/ms</i> is preset. • The specified jump speed is used for all Jump commands until a new value is specified. • The actual jump speed v_{jump} in the image plane in m/s is derived from the specified Speed value [<i>bits/ms</i>] and the calibration factor K [Bits/mm] as follows: $v_{jump} = \text{Speed} / K$ The calibration factor K can be queried from the correction table by get_table_para or get_head_para. • set_jump_speed is also available as control command set_jump_speed_ctrl.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified Speed value by 16. The allowed value range decreases accordingly.
Version info	–
References	jump_abs, jump_rel, set_mark_speed, set_jump_speed_ctrl

Ctrl Command	set_jump_speed_ctrl
Function	Like set_jump_speed , but a control command.
Call	set_jump_speed_ctrl(Speed)
Parameters	Speed Like set_jump_speed .
Comments	• set_jump_speed_ctrl is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set • set_jump_speed_ctrl is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) – the INTERNAL-BUSY list execution status is set
RTC4→RTC5	New command. In RTC4 Compatibility Mode : like set_jump_speed .
Version info	–
References	set_jump_speed, set_mark_speed, set_mark_speed_ctrl

Ctrl Command	set_jump_table
Function	Reads the Jump Delay table with 1024 unsigned 16-bit values stored at the supplied PC address and loads them onto the board as the Jump Delay table (see notes for get_jump_table).
Call	ErrorCode = set_jump_table(Addr)
Parameters	Addr PC Address of a 2048-byte area of PC main memory.
Result	Error code. As an unsigned 32-bit value. 0 No error. 1 Busy error: board BUSY or INTERNAL-BUSY list execution status (get_last_error return code RTC5_BUSY). 4 Verify error: DSP memory error. 11 RTC5 board driver error.
Notes	• set_jump_table is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • set_jump_table is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set)
RTC4→RTC5	New command.
Version info	–
Reference	get_jump_table , load_jump_table_offset

Ctrl Command	set_laser_control	
Function	Defines and enables or disables the laser control signals.	
Call	set_laser_control(Ctrl)	
Parameters	Ctrl	As an unsigned 32-bit value.
	Bit #0 (LSB)	Pulse Switch Setting (does not concern Laser Mode 4 or Laser Mode 6). The setting only affects those LASER1 or LASER2 "laser active" modulation pulses in CO₂ Mode or LASER1 Q-Switch pulses in the YAG modes) that are not yet fully processed at completion of the LASERON signal, see Figure 59 and Figure 60. = 0: The signals are cut off at the end of the LASERON signal. = 1: The final pulse fully executes despite completion of the LASERON signal. See Section "Pulse Completion", Page 175.
	Bit #1	Phase shift of the laser control signals (does not concern Laser Mode 4 or Laser Mode 6). = 0: No phase shift. = 1: CO₂ Mode: The LASER1 signal is exchanged with the LASER2 signal. YAG modes: The LASER1 is shifted back half a signal period.
	Bit #2	Enabling or disabling of Signals for "Laser Active" Operation. = 0: The Signals for "Laser Active" Operation are enabled. = 1: The Signals for "Laser Active" Operation are disabled (the signals are set to their respective "Off" level).
	Bit #3	Level of LASERON signal. = 0: Is set to active-HIGH. = 1: Is set to active-LOW. If there is no change of the pin assignment with config_laser_signals, the LASERON signal corresponds to the signal at the LASERON pin, see Figure 17.
	Bit #4	Levels of LASER1 signal and LASER2 signal. = 0: Are set to active-HIGH. = 1: Are set to active-LOW. If there is no change of the pin assignment with config_laser_signals, the LASER1 signal (LASER2 signal) corresponds to the signal at the LASER1 pin (LASER2 pin), see Figure 17.
	Bit #5	Determines for laser_on_pulses_list whether external signal pulses (at the LASER connector's DIGITAL IN1 digital input) are to be counted at rising or falling edges: = 0: At the falling edge. = 1: At the rising edge.

Ctrl Command	set_laser_control
Parameters (cont'd)	Bit #6 = 0: Synchronization is switched off (default setting). See Chapter 7.4.10 "Output Synchronization", Page 195. = 1: Synchronization is switched on. Bit #7 = 0: The "constant pulse length" mode is switched off (default setting). = 1: The "constant pulse length" mode is switched on. See Chapter 7.4.8 "Pulse Picking Laser Mode", Page 186 and set_pulse_picking_length. Bit #8 Reserved. ... Bit #15 Reserved. Bit #16 For the automatic suppression of laser control signals is used: PowerOK of the head A, x axis. Bit #17 Like Bit #16, but: TempOK of the head A, x axis. Bit #18 Like Bit #16, but: PosAck of the head A, x axis. Bit #19 Like Bit #16, but: PowerOK of the head A, y axis. Bit #20 Like Bit #16, but: TempOK of the head A, y axis. Bit #21 Like Bit #16, but: PosAck of the head A, y axis. Bit #22 Like Bit #16, but: PowerOK of the head B, x axis. Bit #23 Like Bit #16, but: TempOK of the head B, x axis. Bit #24 Like Bit #16, but: PosAck of the head B, x axis. Bit #25 Like Bit #16, but: PowerOK of the head B, y axis. Bit #26 Like Bit #16, but: TempOK of the head B, y axis. Bit #27 Like Bit #16, but: PosAck of the head B, y axis. Bit #28 = 1: In case of error, automatic monitoring (automatic suppression of laser control signals) automatically generates a /STOP signal (list stops, laser control signals get permanently switched off). Bit #29 Reserved. Bit #30 Reserved. Bit #31 Reserved.
Comments	In the default setting (after load_program_file), all bits are set to 0. After a Hardware reset, however, the settings become effective following the first-time call of set_laser_control. Prior to this, all laser signal outputs (LASERON, LASER1 and LASER2) are in the high-impedance mode. TTL states (LOW or HIGH) only become available when set_laser_control is called to define the desired TTL level, see also Chapter 7.4 "Laser Control", Page 173. Even after load_program_file, which deactivates the laser control signals, set_laser_control must be called for first-time activation.

Ctrl Command	set_laser_control
Comments (cont'd)	- For the RTC5 predecessors, the laser signal levels are defined by solder jumpers. The RTC5 lets users control them completely by software. get_startstop_info queries the current status of the laser control signals (Bit #9) and whether the signals are set to active-HIGH or active-LOW (Bit #13). - Enabling and disabling of laser control signals can also be achieved by enable_laser or disable_laser. As of DLL 546, OUT 546: By get_startstop_info (Bit #14) the current state can be queried. - Even if the laser control signals were enabled with set_laser_control or enable_laser, they are not outputted without further commands, see Chapter 7.4 "Laser Control", Page 173. - The phase shift of the laser control signals (Bit #1 = 1) can be set for better synchronization of an analog output, for example, in Softstart Mode, see Chapter 7.4.7 "Softstart Mode", Page 184 or the Pixel Output Mode, see Chapter 8.7 "Pixel Output Mode", Page 244.
RTC4→RTC5	New command.
Version info	Last change: DLL 546, OUT 546: get_startstop_info (Bit #14) support.
References	set_laser_mode , config_laser_signals

Undelayed Short List Command	set_laser_delays
Function	Sets the LaserOn Delay and the LaserOff Delay .
Call	set_laser_delays(LaserOnDelay, LaserOffDelay)
Parameters	LaserOnDelay LaserOn Delay. As a signed 32-bit value. 1 Bit equals 0.5 μ s. Allowed value range: $[-2^{31} \dots + (2^{21}-1)]$. Values over $(2^{21}-1)$ are clipped.
	LaserOffDelay LaserOff Delay. As an unsigned 32-bit value. 1 Bit equals 0.5 μ s. Allowed value range: $[0 \dots + (2^{21}-1)]$. Values over $(2^{21}-1)$ are clipped.
Comments	- The delays can be freely chosen within the allowed ranges. If <code>LaserOffDelay < LaserOnDelay</code>, overlaps of LaserOn and LaserOff are automatically prevented during processing of short vectors (see Section "Automatic Delay Adjustments", Page 143). The <code>LaserOffDelay > LaserOnDelay</code> condition required by the RTC5's predecessor boards is therefore unnecessary. - Observe the notes in Chapter 7.2.1 "Laser Delays", Page 133. - If the LaserOn Delay is negative, the total marking time is extended. - The XY2-100 Converter (Accessory) introduces a 10 μs runtime latency to scan system control. This runtime latency can be compensated by increasing the LaserOn Delay and LaserOff Delay by 10 μs each. - The default setting after load_program_file corresponds to <code>set_laser_delays(20, 20)</code>.
RTC4→RTC5	Basically unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified delay values by 2. The allowed value ranges decrease accordingly.
Version info	–
References	set_scanner_delays

Ctrl Command	set_laser_mode
Function	Sets the laser mode of the RTC5.
Call	<code>set_laser_mode(Mode)</code>
Parameters	Mode As an unsigned 32-bit value. = 0: CO₂ Mode. = 1: YAG Mode 1. = 2: YAG Mode 2. = 3: YAG Mode 3. = 4: Laser Mode 4. = 5: YAG Mode 5. = 6: Laser Mode 6.
Comments	The available laser signals depend on the set laser mode, see also Chapter 7.4 "Laser Control", Page 173.
RTC4→RTC5	Additional laser modes, for example, YAG Mode 5, Laser Mode 6 and Pulse Picking. Otherwise, unchanged functionality.
Version info	–
References	set_laser_control, set_laser_pulses_ctrl, set_laser_pulses, set_laser_timing, set_firstpulse_killer, set_firstpulse_killer_list, set_standby, set_standby_list, set_qswitch_delay, set_qswitch_delay_list

Ctrl Command	set_laser_off_default
Function	Sets the default output value for the ANALOG OUT1 and ANALOG OUT2 output ports, as well as for the 8-bit digital output port of the RTC5.
Call	<code>set_laser_off_default(AnalogOut1, AnalogOut2, DigitalOut)</code>
Parameters	AnalogOut1 12-bit value for analog output port ANALOG OUT1, see also Section "12-Bit Analog Output Port 1 and 2", Page 61. As an unsigned 32-bit value. Higher bits are ignored (exception: AnalogOut1/AnalogOut2 = "–1", see comments for set_port_default).
	AnalogOut2 12-bit value for analog output port ANALOG OUT2. See AnalogOut1.
	DigitalOut 8-bit value for the 8-bit digital output port, see also Section "8-Bit Digital Output Port", Page 68. As an unsigned 32-bit value. Higher bits are ignored (exception: DigitalOut = "–1", see comments for set_port_default).
Comments	The default values can also be defined by set_port_default (see also all comments there).
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for AnalogOut1 and AnalogOut2 by 4.
Version info	–
References	set_port_default

Ctrl Command	set_laser_pin_out
Function	Sends a value to the two digital outputs of the laser connector.
Call	<code>set_laser_pin_out(Pins)</code>
Parameters	Pins Output value (DIGITAL OUT1 and DIGITAL OUT2). As an unsigned 32-bit value. Bit # 0: DIGITAL OUT1. Bit # 1: DIGITAL OUT2. Bit # 2: Reserved. ... Bit #31: Reserved.
Comments	• See also Chapter 9.1.3 "2 Bit Digital Output Port", Page 259.
RTC4→RTC5	New command.
Version info	–
References	set_laser_pin_out_list , get_laser_pin_in

Undelayed Short List Command	set_laser_pin_out_list
Function	Like set_laser_pin_out , but a list command.
Call	<code>set_laser_pin_out_list(Pins)</code>
Parameters	Pins Like set_laser_pin_out .
Comments	• See set_laser_pin_out.
RTC4→RTC5	New command.
Version info	–
References	set_laser_pin_out

Delayed Short List Command	set_laser_pulses
Function	Defines the output period and the pulse lengths for the signals LASER1 and LASER2 for "laser active" operation.
Call	<code>set_laser_pulses(HalfPeriod, PulseLength)</code>
Parameters	HalfPeriod <i>Half</i> of the output period. In bits. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
	PulseLength Pulse length of the laser signals LASER1 and LASER2. In bits. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	• Note that <i>half</i> the period length must be specified for <code>HalfPeriod</code> (see Figure 59 and Figure 60). • If <code>HalfPeriod</code> = 0 and/or <code>PulseLength</code> = 0, no laser signals are outputted. • If the Softstart Mode is used (see Chapter 7.4.7 "Softstart Mode", Page 184), <code>HalfPeriod</code> should not be smaller than 104 (this is not automatically checked). This value corresponds to a laser pulse frequency of approx. 308 kHz. • If <code>PulseLength</code> is larger than the output period ($2 \times \text{HalfPeriod}$), the laser is permanently on. • The signal level is defined by set_laser_control. • The set_laser_pulses command is also available as the control command set_laser_pulses_ctrl. • set_laser_pulses is largely identical to the RTC4 command set_laser_timing, but has less parameters.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for <code>HalfPeriod</code> and <code>PulseLength</code> by 8. The allowed value ranges decrease accordingly.
Version info	–
References	set_laser_pulses_ctrl , set_laser_timing

Ctrl Command	set_laser_pulses_ctrl
Function	Like set_laser_pulses , but a control command.
Call	set_laser_pulses_ctrl(HalfPeriod, PulseLength)
Parameters	HalfPeriod <i>Half</i> of the output period. In bits. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
	PulseLength Pulse length of the laser signals LASER1 and LASER2. In bits. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	See set_laser_pulses
RTC4→RTC5	New command. In RTC4 Compatibility Mode : like set_laser_pulses .
Version info	–
References	set_laser_pulses , set_laser_timing

Delayed Short List Command	set_laser_timing	
Function	Defines the output period and the pulse lengths for the signals LASER1 and LASER2 for "laser active" operation.	
Call	set_laser_timing(HalfPeriod, PulseLength1, (PulseLength2), TimeBase)	
Parameters	HalfPeriod	<i>Half</i> of the output period. In bits. As an unsigned 32-bit value. - In RTC5 Standard Mode: 1 bit equals 1/64 μs. Allowed value range: $[0 \dots (2^{32}-1)]$. - In RTC4 Compatibility Mode: 1 bit equals 1/8 μs or 1 μs, depending on the selected clock frequency. With respect to the specified TimeBase, the value is converted to an integer-multiple of 1/64 μs. The allowed range is correspondingly smaller.
	PulseLength1	Pulse lengths of the laser signals LASER1 and LASER2. In bits. As an unsigned 32-bit value. - In RTC5 Standard Mode: 1 bit equals 1/64 μs. Allowed value range: $[0 \dots (2^{32}-1)]$. - In RTC4 Compatibility Mode: 1 bit equals 1/8 μs or 1 μs, depending on the selected clock frequency. With respect to the specified TimeBase, the value is converted to an integer-multiple of 1/64 μs. The allowed range is correspondingly smaller.
	(PulseLength2)	Value is not used. (As an unsigned 32-bit value.) With the RTC5, the pulse lengths of laser signals LASER1 and LASER2 are always identical and are definable by PulseLength1.
	TimeBase	As an unsigned 32-bit value. - In RTC5 Standard Mode, the value is ignored and the clock frequency is fixed at 64 MHz. 1 bit equals 1/64 μs. - In RTC4 Compatibility Mode, the TimeBase value is handled as follows: =0: sets the time base to 1 MHz. 1 bit equals 1 μs. ≠0: sets the time base to 8 MHz. 1 bit equals 1/8 μs.

Delayed Short List Command	set_laser_timing
Comments	• In RTC5 Standard Mode, set_laser_timing is synonymous with set_laser_pulses. See comments there (on HalfPeriod, PulseLength). • The clock frequency settings apply <i>only</i> for the parameters of set_laser_timing. • For RTC4 Compatibility Mode, SCANLAB generally recommends setting the time base to 8 MHz. In this mode, a time base of 1 MHz should only be chosen if necessary. • Refer to Chapter 7.4 "Laser Control", Page 173, for details. • In RTC4 Compatibility Mode, in Laser Mode 4 and Laser Mode 6, the time base for signals LASER1 and LASER2 is independently of TimeBase always 1/8 μs.
RTC4→RTC5	• With the RTC5, the pulse lengths of laser signals LASER1 and LASER2 are always identical. • Clock frequency: – In RTC5 Standard Mode: fixed clock frequency of 64 MHz. – In RTC4 Compatibility Mode: 1 MHz or 8 MHz.
Version info	–
References	set_laser_pulses_ctrl , set_laser_pulses , set_laser_mode , set_firstpulse_killer , set_firstpulse_killer_list , set_standby , set_standby_list

Undelayed Short List Command	set_list_jump
Function	Execution produces an unconditional jump to the specified address within the list memory area. The next command there is executed immediately without delay.
Call	<code>set_mark_speed_ctrl(Speed)</code>
Parameters	Speed Like set_mark_speed .
Comments	• set_list_jump is synonymous with list_jump_pos, see the comments there.
RTC4→RTC5	Unchanged functionality. In addition: increased value range.
Version info	–
References	list_jump_pos

Delayed Short List Command	set_mark_speed
Function	Sets the mark speed for the Vector commands and "Arc" commands .
Call	<code>set_mark_speed(Speed)</code>
Parameters	Speed Marking speed. In bits/ms. As a 64-bit IEEE floating point value. Allowed value range: [1.6...800000.0]. Out-of-range values are clipped to the boundary values.
Comments	- By default a mark speed of 1,000 <i>bits/ms</i> is preset. - The specified mark speed is used for all [*]mark[*] Commands and "Arc" commands until a new value is specified. - The actual mark speed v_{mark} in the image plane in m/s is derived from the specified Speed value [<i>bits/ms</i>] and the calibration factor K [Bits/mm] as follows: $v_{mark} = \text{Speed} / K$ The calibration factor K can be queried from the correction table by get_table_para or get_head_para. set_mark_speed is also available as control command set_mark_speed_ctrl.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified Speed value by 16. The allowed value range decreases accordingly.
Version info	–
References	mark_abs , mark_rel , set_jump_speed , set_mark_speed_ctrl

Ctrl Command	set_mark_speed_ctrl
Function	Like set_mark_speed , but a control command.
Call	<code>set_mark_speed_ctrl(Speed)</code>
Parameters	Speed mark speed. In bits/ms. As a 64-bit IEEE floating point value. Allowed value range: [1.6...800000.0]
Comments	set_mark_speed_ctrl is not executed (get_last_error return code RTC5_BUSY), if: - the BUSY list execution status is set set_mark_speed_ctrl is even executed, if: - a list has been paused by set_wait (PAUSED list execution status set) - the INTERNAL-BUSY list execution status is set
RTC4→RTC5	New command. In RTC4 Compatibility Mode : like set_mark_speed
Version info	–
References	set_mark_speed , set_jump_speed , set_jump_speed_ctrl

Ctrl Command	set_matrix
Function	Sets the coefficients of the general transformation matrix M_T for all subsequent coordinate transformations, see Chapter 8.2 "Coordinate Transformations", Page 209 .
Call	<code>set_matrix(HeadNo, M11, M12, M21, M22, at_once)</code>
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. = 1: The definition only affects the <i>first</i> scan head connector. = 2: The definition only affects the <i>second</i> scan head connector. Requires Option "Second Scan Head Control". = 0, 3: The definition affects <i>both</i> scan head connectors. = 4: The definition affects the virtual Image field (see also comments). Only the three least significant bits are evaluated.
	M11 Matrix coefficient of the general transformation matrix M_T. As a 64-bit IEEE floating point value. Allowed value range: $[-50...+50]$ in the real Image field and $[-1.5...+1.5]$ in the virtual Image field. With invalid value, <code>set_matrix</code> is ignored.
	M12 Like M11 (analogously).
	M21 Like M11 (analogously).
	M22 Like M11 (analogously).
	at_once Defines when the defined transformation becomes effective. As an unsigned 32-bit value. For <code>HeadNo = 0...3</code>, the following applies: = 0: The new total transformation (total matrix and offset) is only calculated when the next list command is executed and applied to the position which is current at that time. = 1: The new total transformation is calculated immediately (or prior the next list command, if currently the BUSY list execution status or INTERNAL-BUSY list execution status is set) and applied to the current position. Signals for "Laser Active" Operation are switched off in advance. = 2: The new total transformation (total matrix and offset) is only calculated and applied to the current position when the next jump_abs, jump_rel, goto_xy or goto_xyz is executed. = 3: Like <code>at_once = 1</code>, but the laser control signals remain unchanged. > 3: Like <code>at_once = 2</code>.

Ctrl Command	set_matrix
Comments	• See Chapter 8.2 "Coordinate Transformations", Page 209. • The coordinate transformation defined by HeadNo = 4 is only in effect during a Processing-on-the-fly application initiated by set_fly_2d, see Section "Coordinate Transformations in the Virtual Image Field", Page 158: – set_fly_2d – ≥ DLL 541 set_fly_x – ≥ DLL 541 set_fly_y Here, the <code>at_once</code> parameter is ignored.
RTC4→RTC5	Unchanged primary functionality (matrix definition), however: • The parameters <code>HeadNo</code> and <code>at_once</code> are new. • Reduced value range for coefficients. • See also Section "Notes for RTC4 Users", Page 212.
Version info	Last change DLL 541: also available with set_fly_x , set_fly_y .
References	set_matrix_list , set_angle , set_offset , set_scale

Variable List Command	set_matrix_list
Function	Sets <i>one</i> of the 4 coefficients of the general transformation matrix M_T during execution of a list, see Chapter 8.2 "Coordinate Transformations" , Page 209.
Call	<code>set_matrix_list(HeadNo, Ind1, Ind2, Mij, at_once)</code>
Parameters	HeadNo See set_matrix. However, HeadNo = 4 is not available. Ind1 Row index and column index of the matrix coefficient to be changed. As an unsigned 32-bit value. Ind2 Allowed values: [Index uneven: 1, Index even: 2]. Mij Matrix coefficient. As a 64-bit IEEE floating point value. Allowed value range: [-50...+50]. If the parameter is set to an invalid value, set_matrix_list is replaced by a list_nop. at_once Determines when the defined transformation becomes effective: = 0: The transformation settings are only collected and intermediately stored, but the transformation is not processed as long as this is not activated by another coordinate transformation (for example, by a list command with <code>at_once = 1</code> or a corresponding control command). = 1: The transformation is immediately calculated (including all transformation settings that were collected until then) and processed prior to the next list command. Signals for "Laser Active" Operation are switched off in advance. = 2: The transformation settings are only collected and intermediately stored (as with <code>at_once = 0</code>). However, The transformation is immediately calculated (including all transformation settings that were collected and intermediately stored until then) and applied to the current position when the next jump_abs or jump_rel (only if no list is currently being executed: also goto_xy or goto_xyz) is executed. = 3: As with <code>at_once = 1</code>, but the laser control signals remain unaffected. > 3: See <code>at_once = 2</code>. As an unsigned 32-bit value.
Comments	set_matrix_list only allows changing <i>one</i> of the 4 coefficients at a time. To change several coefficients during execution of a list, set_matrix_list has to be called repeatedly. Here, we recommend making the first calls with <code>at_once = 0</code> and only the last call with <code>at_once = 1</code>. - See Chapter 8.2 "Coordinate Transformations", Page 209. - HeadNo = 4 is not available.
RTC4→RTC5	See set_matrix .
Version info	–
References	set_matrix

Ctrl Command	set_max_counts
Function	Defines the maximum number of External Starts .
Call	<code>set_max_counts(Counts)</code>
Parameters	Counts Maximum number of External Starts . As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	- When the specified number of External Starts has been reached, the external start input is disabled (see set_control_mode, Bit #0 = 0). - If Counts = 0, the number of External Starts is unlimited. - When the RTC5 is initialized (by load_program_file), Counts is set to 0. - The current number of successful External Starts can be read from the corresponding internal counter with get_counts. The counter can be reset by set_control_mode.
RTC4→RTC5	Unchanged functionality. In addition: increased value range.
Version info	–
References	get_counts , set_control_mode

Ctrl Command	set_mcbasp_freq
Function	Sets the transmission frequency of the McBSP interface .
Call	<code>mcbasp_freq = set_mcbasp_freq(Freq)</code>
Parameters	Freq Desired transmission frequency of the McBSP interface in Hz. As an unsigned 32-bit value. Allowed value range: $[4000000 \dots 16000000]$ (4...16 MHz). Out-of-range values are clipped to the boundary values. Out-of-range values cause the get_last_error return code RTC5_PARAM_ERROR to be generated.
Result	The actually set frequency in Hz. As an unsigned 32-bit value. With overflowing values 0 is returned.
Comments	- The default transmission frequency (after initialization) is 8 MHz (Freq = 8,000,000). - Because not every arbitrary frequency can be implemented, set_mcbasp_freq returns the actually set frequency. Example: <code>mcbasp_freq = set_mcbasp_freq(7,000,000);</code> returns <code>mcbasp_freq = 7,200,000</code>. - The new transmission frequency only becomes effective when you re-initialize the McBSP interface by mcbasp_init. - The signals and operating conditions of the McBSP interface are presented in the Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71. - Note: The receiving frequency is exclusively determined by the incoming clock pulses and has a maximum limit of 16 MHz.
RTC4→RTC5	New command.
Version info	–
References	mcbasp_init , set_mcbasp_out , set_mcbasp_out_ptr

Ctrl Command	set_mcbasp_global_matrix
Function	Activates matrix correction for "Global Online Positioning" by the McBSP interface .
Call	<code>set_mcbasp_global_matrix()</code>
Comments	• See also Chapter 8.3.2 "Global Online Positioning", Page 216. • A matrix correction cannot be used in conjunction with offset and/or rotation corrections. Any such already-activated options gets deactivated by set_mcbasp_global_matrix. Subsequent activation of other options (by set_mcbasp_global_x, set_mcbasp_global_y or set_mcbasp_global_rot) deactivates the matrix option. • The following restrictions apply to the matrix coefficients transferred over the McBSP interface (as with set_matrix (HeadNo = 4, ...)): – The allowed value range for matrix coefficients is [–1.5...+1.5]. – Transferred coefficients exceeding this range are ignored. • Users must individually supply as input value M_{in} to the McBSP interface each matrix coefficient M_{ij} of the transformation matrix M_T as a normalized integer with associated indices i and j as follows: $M_{in} = (\text{integer}(M_{ij} * 2^{28}) \ll 2) + (i \ll 1) + j$ with $M_T = \{ M00, M01, M10, M11 \} = \{ m_{11}, m_{12}, m_{21}, m_{22} \}$. Conversely, the RTC5 determines a coefficient from the input value as follows: $M_T[M_{in} \& 0x3] = (M_{in} \gg 2) / 2^{28}.$ • The coefficients are transferred to internal memory location 1 and can be checked there by querying with <code>read_mcbasp(1)</code>. • The McBSP interface cannot be simultaneously used for an Online Positioning and Processing-on-the-fly applications. See also Section "Notes", Page 215.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbasp_global_matrix_list , set_mcbasp_matrix

Undelayed Short List Command	set_mcbasp_global_matrix_list
Function	Like set_mcbasp_global_matrix , but a list command.
Call	<code>set_mcbasp_global_matrix_list()</code>
Comments	• Like set_mcbasp_global_matrix.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbasp_global_matrix

Ctrl Command	set_mcbasp_global_rot
Function	Activates or deactivates rotation correction for “Global Online Positioning” by the McBSP interface .
Call	<code>set_mcbasp_global_rot(Resolution)</code>
Parameters	Resolution As a 64-bit IEEE floating point value. $2^{-26} < \text{Resolution} < 2^{26}$: scaling factor (correction is activated), otherwise: correction is deactivated. Resolution = McBSP/SPI bits per full circle.
Comments	• See also Chapter 8.3.2 “Global Online Positioning”, Page 216. • With an McBSP/SPI rotation correction input value of Rot_{in}, set_mcbasp_global_rot functions like <code>set_angle(4, Angle × 360°, ...)</code>, whereby Angle (in full circles) = $\text{Rot}_{\text{in}} / \text{Resolution}$. Only Angle values in the range [0.0...+20.0 full circles] are allowed. • The McBSP interface cannot be simultaneously used for an Online Positioning and Processing-on-the-fly applications. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbasp_global_rot_list , set_mcbasp_rot

Undelayed Short List Command	set_mcbasp_global_rot_list
Function	Like set_mcbasp_global_rot , but a list command.
Call	<code>set_mcbasp_global_rot_list(Resolution)</code>
Parameters	Resolution Like set_mcbasp_global_rot .
Comments	• See set_mcbasp_global_rot.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbasp_global_rot

Ctrl Command	set_mcbbsp_global_x
Function	Activates or deactivates x offset correction for “Global Online Positioning” by the McBSP interface .
Call	<code>set_mcbbsp_global_x(Scale)</code>
Parameters	Scale As a 64-bit IEEE floating point value. $2^{-26} < \text{Scale} < 2^{26}$: scaling factor (correction is activated), otherwise: correction is deactivated.
Comments	• See also Chapter 8.3.2 “Global Online Positioning”, Page 216. • With an McBSP input value of X_{in} for the x offset correction, set_mcbbsp_global_x functions like <code>set_offset_xyz(4, XOffset,...)</code>, whereby $XOffset \text{ (in bits)} = \text{Scale} \times X_{in}$. $XOffset$ values outside of $[-524,288...+524,287]$ are clipped to the boundary value as long as $\text{Scale} \times X_{in}$ does not exceed the value range $\pm 2^{31}$. • The McBSP interface cannot be simultaneously used for an Online Positioning and Processing-on-the-fly applications. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbbsp_global_x_list , set_mcbbsp_x

Undelayed Short List Command	set_mcbbsp_global_x_list
Function	Like set_mcbbsp_global_x , but a list command.
Call	<code>set_mcbbsp_global_x_list(Scale)</code>
Parameters	Scale Like set_mcbbsp_global_x .
Comments	• See set_mcbbsp_global_x.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbbsp_global_x

Ctrl Command	set_mcbbsp_global_y
Function	Activates or deactivates y offset correction for “Global Online Positioning” by the McBSP interface .
Call	<code>set_mcbbsp_global_y(Scale)</code>
Parameters	Scale As a 64-bit IEEE floating point value. $2^{-26} < \text{Scale} < 2^{26}$: scaling factor (correction is activated), otherwise: correction is deactivated.
Comments	• See also Chapter 8.3.2 “Global Online Positioning”, Page 216. • With an McBSP input value of Y_{in} for the y offset correction, set_mcbbsp_global_y functions like <code>set_offset_xyz(4, ..., YOffset,...)</code>, whereby $YOffset \text{ (in bits)} = \text{Scale} \times Y_{in}$. $YOffset$ values outside of $[-524,288...+524,287]$ are clipped to the boundary value as long as $\text{Scale} \times Y_{in}$ does not exceed the value range $\pm 2^{31}$. • The McBSP interface cannot be simultaneously used for an Online Positioning and Processing-on-the-fly applications. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbbsp_global_y_list , set_mcbbsp_y

Undelayed Short List Command	set_mcbbsp_global_y_list
Function	Like set_mcbbsp_global_y , but a list command.
Call	<code>set_mcbbsp_global_y_list(Scale)</code>
Parameters	Scale Like set_mcbbsp_global_y .
Comments	• See set_mcbbsp_global_y.
RTC4→RTC5	New command.
Version info	Available as of DLL 545, OUT 545.
References	set_mcbbsp_global_y

Ctrl Command	set_mcbssp_in
Function	Activates Processing-on-the-fly correction for compensation of a workpiece or scan system motion (based on position values transferred to the RTC5 by the McBSP interface). The McBSP interface can also be used for inputting other desired signals.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_mcbssp_in terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	set_mcbssp_in(Mode, Scale)
Parameters	Mode As an unsigned 32-bit value. Allowed values: = 0: Processing-on-the-fly correction is switched off. = 1: Compensation of linear motion in the x direction. = 2: Compensation of linear motion in the y direction. = 3: Compensation of linear motion in the x direction and y direction. = 4: Compensation of rotary motion. = 5: Processing-on-the-fly correction is switched off. • Mode = 0...5: All McBSP input values are alternately copied to internal memory locations 1 and 2. • Mode = 1...5 (but not for Mode = 0): McBSP input values coded with "Bit #31 = 0" is additionally copied to internal memory location 0 and those with "Bit #31 = 1" to internal memory location 3. • Mode = 1...4: Values copied to internal memory location 0 are applied for Processing-on-the-fly correction.
	Scale Scaling factor or rotation resolution. As a 64-bit IEEE floating point value. • Mode = 1...3: scaling factor in $(RTC5)bits/(McBSP)bit$. Allowed value range: $1/256 \leq Scale \leq 16000.0$ (if Mode = 3, Scale applies to both axes). • Mode = 4: number of steps (counts) per revolution. Allowed value range: $Scale > 100.0$.
Comments	• You can query the internal memory locations at any time by read_mcbssp. • For Processing-on-the-fly correction and determination of the scaling factor, see Chapter 8.6 "Processing-on-the-fly", Page 227. • The various Processing-on-the-fly corrections cannot be arbitrarily combined (see the page 227). • For deactivating Processing-on-the-fly correction, see Chapter 8.6.5 "Deactivating Processing-on-the-fly Corrections", Page 236. • 15 bits per axis (with sign) are effectively available for 2D correction (Mode = 3), whereby the x value is in the lower 16 bits of the "Bit #31 = 0"-coded McBSP input value and the y value in the upper 16 bits. • The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section "Notes", Page 215.

Ctrl Command	set_mcbasp_in
Comments (cont'd)	- The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbasp_init. That is, data provided is not transmitted, see page 73. - If Mode > 5, set_mcbasp_in is <i>not</i> executed (get_last_error return code RTC5_PARAM_ERROR). - If an unallowed Scale parameter value is supplied (for example, Scale = 0), set_mcbasp_in behaves as with Mode = 0.
RTC4→RTC5	New command.
Version info	–
References	set_mcbasp_in_list

Undelayed Short List Command	set_mcbasp_in_list
Function	Like set_mcbasp_in , but a list command.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_mcbasp_in_list terminates the Processing-on-the-fly process (even though it could not have been activated).
Call	<code>set_mcbasp_in_list(Mode, Scale)</code>
Parameters	Mode Like set_mcbasp_in .
	Scale Like set_mcbasp_in .
Comments	- If Mode > 5, then set_mcbasp_in_list is replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). - See set_mcbasp_in.
RTC4→RTC5	New command.
Version info	–
References	set_mcbasp_in

Ctrl Command	set_mcbasp_matrix
Function	Activates matrix correction for “Local Online Positioning” by the McBSP interface .
Call	<code>set_mcbasp_matrix()</code>
Comments	- For “Local Online Positioning”, see Chapter 8.3.1 ““Local Online Positioning””, Page 213. - Matrix corrections cannot be used in conjunction with offset and/or rotation corrections. Any such already-activated options gets deactivated by set_mcbasp_matrix. Subsequent activation of other options (by set_mcbasp_x, set_mcbasp_y or set_mcbasp_rot) deactivates the matrix option. - The following restrictions apply to the matrix coefficients transferred over the McBSP interface (as with set_matrix): The allowed value range for matrix coefficients is [-50...+50]. Transferred coefficients exceeding this range are ignored. - You must individually supply as input value M_{in} to the McBSP interface each matrix coefficient M_{ij} of the transformation matrix M_T as a normalized integer with associated indices i and j as follows: $M_{in} = (\text{integer}(M_{ij} * 2^{24}) << 2) + (i << 1) + j$ with $M_T = \{ M00, M01, M10, M11 \} = \{ m_{11}, m_{12}, m_{21}, m_{22} \}$. Conversely, the RTC5 determines a coefficient from the input value as follows: $M_T[M_{in} \& 0x3] = (M_{in} >> 2) / 2^{24}.$ - You must separately fetch each transferred matrix coefficient by apply_mcbasp or apply_mcbasp_list. We recommend fetching the first coefficient with <code>at_once = 0</code> and only the last one with <code>at_once > 0</code>. - The coefficients get transferred to internal memory location 1 and can be checked there by querying with <code>read_mcbasp(1)</code>. - The McBSP interface cannot be simultaneously used for an Online Positioning and Processing-on-the-fly applications. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	–
References	set_mcbasp_matrix_list

Undelayed Short List Command	set_mcbasp_matrix_list
Function	Like set_mcbasp_matrix , but a list command.
Call	<code>set_mcbasp_matrix_list()</code>
Comments	Like set_mcbasp_matrix.
RTC4→RTC5	New command.
Version info	Available as of DLL 533, OUT 534, RBF 524.
References	set_mcbasp_matrix

Undelayed Short List Command	set_mcbasp_out				
Function	Defines two signal types for output at the McBSP interface , see also Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71 .				
Call	<code>set_mcbasp_out(Signal1, Signal2)</code>				
Parameters	<table border="0"> <tr> <td>Signal1</td><td>To-be-outputted signal type. As an unsigned 32-bit value.</td></tr> <tr> <td>Signal2</td><td>To-be-outputted signal type. As an unsigned 32-bit value.</td></tr> </table>	Signal1	To-be-outputted signal type. As an unsigned 32-bit value.	Signal2	To-be-outputted signal type. As an unsigned 32-bit value.
Signal1	To-be-outputted signal type. As an unsigned 32-bit value.				
Signal2	To-be-outputted signal type. As an unsigned 32-bit value.				
Comments	- The selectable signal types are identical to those of set_trigger (refer to the comments there for the allowed value range, signal types and other information). If the value for <code>Signal1/Signal2</code> is unallowed, then set_mcbasp_out is replaced by list_nop (get_last_error return code <code>RTC5_PARAM_ERROR</code>). - Both selected data signals are continuously transmitted (once per 10 μs cycle). - Only 16-bit portions of the selected data signals are packed into a common 32-bit data word for output. <code>Signal1</code> is the lower half and <code>Signal2</code> the upper half of this 32-bit data word. - Only RTC4-compatible Bit #4...Bit #19 of the sample values and status values returned by the scan system (<code>Signal1, Signal2 = 1...23, 25...30</code>) are outputted. The least significant 4 bits and any exceeding bits (in the virtual Image field) are ignored. - For the other data types (<code>Signal1, Signal2 = 0, 24, 31...54</code>), the least significant 16 bits are outputted and the upper bits are ignored. <code>McBSP_Output = ((Out2 & 0x0000FFFF) << 16) (Out1 & 0x0000FFFF);</code> – - Transmitted values are those of the preceding clock cycle: data is processed at the end of a cycle and transmitted at the beginning of the next clock cycle at the set transmission frequency (see set_mcbasp_freq). - The signals and operating conditions of the Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71 interface are presented in Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71.				
RTC4→RTC5	New command.				
Version info	–				
References	read_mcbasp , mcbasp_init , set_mcbasp_freq , set_mcbasp_out_ptr , set_trigger				

Ctrl Command	set_mcbbsp_out_ptr
Function	Defines a list of up to 8 signal types for output at the McBSP interface (Multi channel Buffered Serial Port, see also page 71).
Call	set_mcbbsp_out_ptr(Number, SignalPtr)
Parameters	Number As an unsigned 32-bit value. Allowed value range: [0...8]. = 1...8: Number of signal types to be outputted. = 0 Output at the McBSP interface occurs in accordance with the pre-defined settings or as specified by a prior set_mcbbsp_out.
	SignalPtr Pointer (in C and C++ data type ULONG_PTR, an unsigned 32-bit value or unsigned 64-bit value) to an array of Number unsigned 32-bit values, where the to-be-outputted Number signal type numbers are specified.
Comments	- The memory area for the SignalPtr array must be provided by the user program. - If Number > 8 and/or SignalPtr = NULL, set_mcbbsp_out_ptr is <i>not</i> executed (get_last_error return code RTC5_PARAM_ERROR). - The selectable signal types are identical to those of set_trigger (refer to the comments there for the allowed value range, signal types and other information). - The up to 8 selected data types are outputted sequentially (one data type per 10 μs clock cycle). Each individual signal belongs to a different clock cycle. Transmitted values are always from the previous clock cycle. - When outputting, each signal value is supplemented by the corresponding signal type number: the signal type number gets inserted into the lowest byte of the 32-bit data word. The actual signal value therefore gets shifted 8 bits to the left, whereby all bits above the 24th get truncated (overflow, no clipping, relevant only for signal types 24, 31 and 37...54): 32-bit output value = (signal value << 8) (signal type number & 0xFF). - The signals and operating conditions of the McBSP interface are presented in the Section "McBSP Interface", Page 71.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_out_ptr_list , set_mcbbsp_out , set_trigger

Multiple List Command	set_mcbssp_out_ptr_list
Function	Like set_mcbssp_out_ptr , but a list command.
Call	<code>set_mcbssp_out_ptr_list(Number, SignalPtr)</code>
Parameters	Number Like set_mcbssp_out_ptr .
	SignalPtr Like set_mcbssp_out_ptr .
Comments	• set_mcbssp_out_ptr_list requires two list memory positions for <code>Number ≥ 1</code>. Sequence of execution: 1. Pending delayed short list commands 2. First command part as an undelayed short list command 3. Second command part as a normal list command • See set_mcbssp_out_ptr.
RTC4→RTC5	New command.
Version info	Available as of DLL 550, OUT 550.
References	set_mcbssp_out_ptr

Ctrl Command	set_mcbbsp_rot
Function	Activates or deactivates rotation correction for “ Local Online Positioning ” by the McBSP interface .
Call	set_mcbbsp_rot(Resolution)
Parameters	Resolution Value. As a 64-bit IEEE floating point value. $2^{-26} < \text{Resolution} < 2^{26}$: scaling factor (correction is activated), otherwise: correction is deactivated. $\text{Resolution} = \text{McBSP/SPI bits per full circle}$.
Comments	- For “Local Online Positioning”, see Chapter 8.3.1 ““Local Online Positioning””, Page 213. - For an McBSP/SPI rotation correction input value of Rot_{in}, apply_mcbbsp functions like set_angle(..., Angle $\times$ 360°, ...), whereby $\text{Angle in full circles} = \text{Rot}_{\text{in}} / \text{Resolution}$ Only Angle values in the range [0.0...+20.0 full circles] are allowed. - The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_rot_list , set_mcbbsp_x , set_mcbbsp_y , apply_mcbbsp

Undelayed Short List Command	set_mcbbsp_rot_list
Function	Like set_mcbbsp_rot , but a list command.
Call	set_mcbbsp_rot_list(Resolution)
Parameters	Resolution Like set_mcbbsp_rot .
Comments	Like set_mcbbsp_rot.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_rot

Ctrl Command	set_mcbbsp_x
Function	Activates or deactivates x offset correction for “Local Online Positioning” by the McBSP interface .
Call	set_mcbbsp_x(Scale)
Parameters	Scale As a 64-bit IEEE floating point value. $2^{-26} < \text{Scale} < 2^{26}$: scaling factor (correction is activated). Otherwise: correction is deactivated.
Comments	- For “Local Online Positioning”, see Chapter 8.3.1 ““Local Online Positioning””, Page 213. - With an McBSP input value of X_{in} for the x offset correction, apply_mcbbsp functions like <code>set_offset(..., XOffset, ...)</code>, whereby $XOffset \text{ (in bits)} = \text{Scale} \times X_{in}$ $XOffset$ values outside of $[-524,288...+524,287]$ are clipped to the boundaries as long as $\text{Scale} \times X_{in}$ does not exceed the value range $\pm 2^{31}$. - The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_x_list , set_mcbbsp_y , set_mcbbsp_rot , apply_mcbbsp

Undelayed Short List Command	set_mcbbsp_x_list
Function	Like set_mcbbsp_x , but a list command.
Call	set_mcbbsp_x_list(Scale)
Parameters	Scale Like set_mcbbsp_x .
Comments	See set_mcbbsp_x.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_x

Ctrl Command	set_mcbbsp_y
Function	Activates or deactivates y offset correction for “ Local Online Positioning ” by the McBSP interface .
Call	<code>set_mcbbsp_y(Scale)</code>
Parameters	Scale As a 64-bit IEEE floating point value. $2^{-26} < \text{Scale} < 2^{26}$: scaling factor (correction is activated), otherwise: correction is deactivated.
Comments	- For “Local Online Positioning”, see Chapter 8.3.1 ““Local Online Positioning””, Page 213. - With an McBSP input value of Y_{in} for the y offset correction, apply_mcbbsp functions like <code>set_offset(..., YOffset, ...)</code>, whereby $YOffset \text{ (in bits)} = \text{Scale} \times Y_{in}$ YOffset values outside of $[-524,288...+524,287]$ are clipped to the boundaries as long as $\text{Scale} \times Y_{in}$ does not exceed the value range $\pm 2^{31}$. - The McBSP interface cannot be simultaneously used for both Processing-on-the-fly applications and Online Positioning. See also Section “Notes”, Page 215.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_y_list , set_mcbbsp_x , set_mcbbsp_rot , apply_mcbbsp

Undelayed Short List Command	set_mcbbsp_y_list
Function	Like set_mcbbsp_y , but a list command.
Call	<code>set_mcbbsp_y_list(Scale)</code>
Parameters	Scale Like set_mcbbsp_y .
Comments	See set_mcbbsp_y.
RTC4→RTC5	New command.
Version info	–
References	set_mcbbsp_y

Ctrl Command	set_multi_mcbbsp_in
Function	Activates a Processing-on-the-fly application and McBSP interface multi transmission with up to 8 different data types.
Restriction	If the Option Processing-on-the-fly is not enabled, then set_multi_mcbbsp_in switches off the Processing-on-the-fly process (even if it has been never switched on).
Call	<code>set_multi_mcbbsp_in(Ctrl, P, Mode)</code>
Parameters	Ctrl Control parameter for initializing or deactivating laser power variation. As an unsigned 32-bit value. = 1...6: Defines which signal parameter to vary. For a description, see set_auto_laser_control. = 0 or > 6: Deactivates laser power variation (for Ctrl > 6: get_last_error return code RTC5_PARAM_ERROR). P Initialization value for laser power. As an unsigned 32-bit value. Allowed value range: see set_auto_laser_control. Mode Mode. As an unsigned 32-bit value. = 0: The transmitted value is used directly. = 1: The transmitted value is multiplied by P/16384 and then put out. = 2: The transferred value is only stored but not used. The Processing-on-the-fly correction is switched off. Ctrl, P and Mode are only relevant for Type 3 (= laser power) of the transmitted data word, see read_multi_mcbbsp.
Comments	- Any other previously activated McBSP transmission for Online Positioning and any other previously activated Processing-on-the-fly application with encoder signals or positional values is terminated first. set_multi_mcbbsp_in enables inputting by the McBSP interface of up to 8 different types for asynchronous transmission every 10 μs. Each 10 μs, the data is copied in accordance with its type code to separate memory, from where it can also be queried by read_multi_mcbbsp. - To avoid faulty sorting, no active McBSP transmission should be occurring when you call the command. - The McBSP interface ignores the first FrameSync signal after a load_program_file or mcbbsp_init. That is, data provided is not transmitted, see page 73. - For the transmission, the type assignment must be coded into the 3 least significant bits of the data word according to: $\text{McBSPValue} = (\text{Value} \ll 3) \mid \text{Type}$ The RTC5 board stores the data word according to: $\text{Memory}[\text{McBSPValue} \& 0x7] = (\text{long}) \text{McBSPValue} \gg 3$ - For more on type assignments, see read_multi_mcbbsp.

Ctrl Command	set_multi_mcbssp_in
Comments (cont'd)	- The remaining (upper) 29 bits are available for the data word itself. There are no further restrictions other than the type-dependent value ranges themselves. The transmitted values are not checked with respect to their ranges. Clipping or data overflow may occur. - The 4 extra parameters are not the same as the free variables (see Chapter 6.9.1 "Free Variables", Page 123, although they can be used for similar purposes. Upon program start, they are initialized with 0 and this command does not further modify them. The transmitted values merely get copied into type-sorted memory. - If more than 4 McBSP/SPI transfers complete within a 10 μs clock cycle, then prior values might get overwritten. - If the Option Processing-on-the-fly is enabled, then set_multi_mcbssp_in activates a Processing-on-the-fly application with positional values for the three coordinate directions x, y and z. The memory values of their respective types are initialized with 0. Even z is used in 20-bit resolution. - Additionally, laser power can be varied by outputting the transmitted type 3 value at the port assigned by the Ctrl parameter. The initialization value P gets put out immediately. - For Mode = 0, subsequent type-3 transfers are outputted directly at the port assigned by Ctrl. For Mode = 1, the transferred value is handled as a multiplication factor in accordance with Normalization 1.0 = 16384 (14 bits). Thus, the following is put out: $(P \times \text{the transmitted value} / 16384)$. This mode is an alternative to laser power variation by "freely definable wobble shapes", but without the Softstart suppression. - These laser power variation method can be combined with the vector-defined laser control (with parameterized commands). It cannot be combined with other "Automatic Laser Control" methods. It overwrites other variations sharing the same Ctrl parameter. Though unidentical Ctrl parameters are allowed, they serve no practical purpose. - If Ctrl = 0, then laser power variation is switched off. The values transmitted by McBSP/SPI continue to be copied into type-sorted memory, but are not put out. - Special case: if a "freely definable wobble shape" is active, then P is always (except for Mode = 2) regarded as relative laser power and multiplicatively coupled with the wobble-shape's laser power (see set_wobble_vector). Because this wobble shape defines its own laser power (see set_wobble_control), the command's Ctrl parameter has no relevance. It should be set to an invalid value (for example, with Ctrl = 0) to avoid doubled or meaningless output.
RTC4→RTC5	New command.
Version info	Last change DLL 550, OUT 550: Mode = 2 switches off the Processing-on-the-fly correction.
Comments	set_multi_mcbssp_in_list , read_multi_mcbssp , set_wobble_control , set_wobble_vector

Normal List Command	set_multi_mcbasp_in_list
Function	Like set_multi_mcbasp_in , but a list command.
Restriction	Like set_multi_mcbasp_in .
Call	set_multi_mcbasp_in_list(Ctrl, P, Mode)
Parameters	Ctrl Like set_multi_mcbasp_in. P Like set_multi_mcbasp_in. Mode Like set_multi_mcbasp_in.
Comments	• See set_multi_mcbasp_in.
RTC4→RTC5	New command.
Version info	–
References	set_multi_mcbasp_in , read_multi_mcbasp , set_wobbel_control , set_wobbel_vector

Variable List Command	set_n_pixel
Function	In Pixel Output Mode , executes PortOutValue1 and PortOutValue2 assigned to the set_pixel command Number times in immediate succession.
Call	set_n_pixel(PortOutValue1, PortOutValue2, Number)
Parameters	PortOutValue 1 Like set_pixel .
	PortOutValue 2 Like set_pixel .
	Number Number of pixels. As an unsigned 32-bit value. Allowed value range: [1...(2 ³² -1)]. 0 is automatically set to 1.
Comments	• For usage of set_n_pixel, see Chapter 8.7 "Pixel Output Mode", Page 244. • Before the first set_n_pixel command of a line, set_pixel_line must have been called. • set_n_pixel defines the parameters for the following Number (identical) set_pixel commands of an image line. For usage, see comments on set_pixel. • If only an individual pixel is to be defined, then set_pixel can be used as an alternative to set_n_pixel. set_pixel is synonymous with set_n_pixel (Number = 1). • Outside Pixel Output Mode (if set_n_pixel is not directly preceded by set_pixel_line, set_pixel or set_n_pixel), set_n_pixel is a short list command and otherwise ignored. Under some circumstances, a list_continue might be inserted, see Section "Normal, Short, Variable and Multiple List Commands", Page 278.
RTC4→RTC5	New command. For RTC4 Compatibility Mode : see set_pixel .
Version info	—
References	set_pixel , set_pixel_line , set_pixel_line_3d

Ctrl Command	set_offset
Function	Defines an offset for all subsequent coordinate transformations (see Chapter 8.2 "Coordinate Transformations", Page 209).
Call	<code>set_offset(HeadNo, XOffset, YOffset, at_once)</code>
Call	<code>set_offset(HeadNo, XOffset, YOffset, at_once)</code>
Parameters	HeadNo See set_offset_xyz .
	XOffset See set_offset_xyz .
	YOffset See set_offset_xyz .
	at_once See set_offset_xyz .
Comments	• See Chapter 8.2 "Coordinate Transformations", Page 209. • <code>set_offset</code> has the same effect as <code>set_offset_xyz</code>, but leaves <code>ZOffset</code> unchanged.
RTC4→RTC5	Unchanged primary functionality (offset definition). However: • The parameters <code>HeadNo</code> and <code>at_once</code> are new. • See set_offset_xyz.
Version info	–
References	set_offset_list , set_offset_xyz , set_angle , set_matrix , set_scale

Variable List Command	set_offset_list
Function	Like set_offset_xyz , but a list command.
Call	<code>set_offset_list(HeadNo, XOffset, YOffset, at_once)</code>
Parameters	HeadNo Like set_offset_xyz . However, <code>HeadNo = 4</code> is not available.
	XOffset Like set_offset_xyz .
	YOffset Like set_offset_xyz .
	at_once Like set_offset_xyz_list .
RTC4→RTC5	See set_offset_xyz .
Version info	–
References	set_offset , set_offset_xyz_list

Ctrl Command	set_offset_xyz
Function	Defines an offset for all subsequent coordinate transformations, see Chapter 8.2 "Coordinate Transformations" , Page 209.
Call	set_offset_xyz(HeadNo, XOffset, YOffset, ZOffset, at_once)
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. = 1: The definition only affects the first scan head connector. = 2: The definition only affects the second scan head connector. = 0, 3: The definition affects <i>both</i> scan head connectors. = 4: The definition affects the virtual Image field (see also comments). Only the three least significant bits are evaluated.
	XOffset Offsets for the x direction and y direction (coordinate translation relative to the origin of the Cartesian coordinate system). In bits. As a signed 32-bit value. Allowed value range: [-524,288...+524,287]. Out-of-range values are clipped to the boundary values.
	ZOffset Offset for the z direction (coordinate translation relative to the origin of the Cartesian coordinate system). In bits. As a signed 32-bit value. Allowed value range: [-32,768...+32,767]. Out-of-range values are clipped to the boundary values.
	at_once Like set_matrix.
Comments	• See Chapter 8.2 "Coordinate Transformations", Page 209. • ZOffset is stored only once, applying jointly for both scan head connectors (HeadNo is therefore ignored). • ZOffset ≠ 0 causes a shift of the working plane (opposite to the direction of the laser beam). A positive value increases the z coordinate. For a hypothetical control value of (0 0 0), this would be by $d = ZOffset / K$ to the plane $z = +d$. On the calibration factor K, see Chapter 7.3.2 "Image Field Size and Image Field Calibration", Page 156; furthermore, also #3 and #5 in Chapter 7.3.6 "Output Values to the Scan System", Page 170. • The coordinate transformation defined by HeadNo = 4 is only in effect during a Processing-on-the-fly application initiated by set_fly_2d, see Section "Coordinate Transformations in the Virtual Image Field", Page 158: – set_fly_2d – ≥ DLL 541 set_fly_x – ≥ DLL 541 set_fly_y Here, the at_once parameter is ignored. • If HeadNo = 4, then ZOffset is ignored.
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified values for XOffset and YOffset by 16. The allowed value ranges decrease accordingly. The value range for ZOffset is identical in RTC5 Standard Mode and RTC4 Compatibility Mode.
Version info	Last change DLL 541, OUT 541: also available with set_fly_x and set_fly_y .
References	set_offset_xyz_list , set_offset , set_angle , set_matrix , set_scale , set_defocus

Variable List Command	set_offset_xyz_list
Function	Like set_offset_xyz , but a list command.
Call	set_offset_xyz_list(HeadNo, XOffset, YOffset, ZOffset, at_once)
Parameters	HeadNo Like set_offset_xyz . However, HeadNo = 4 is not available.
	XOffset Like set_offset_xyz .
	YOffset Like set_offset_xyz .
	ZOffset Like set_offset_xyz .
	at_once Like set_matrix_list .
Comments	HeadNo = 4 is not available.
RTC4→RTC5	See set_offset_xyz .
Version info	–
References	set_offset_xyz , set_offset_list , set_defocus_list

Ctrl Command	set_pause_list_cond
Function	Defines the condition at the 16-bit digital input port of the EXTENSION 1 socket connector under which a pause_list automatically is executed.
Call	set_pause_list_cond(Mask1, Mask0)
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the least 16 bits are evaluated. Mask0 As Mask1.
Comments	• If a list is currently executed and the following condition is met (see also Chapter 9.3.2 "Conditional Command Execution", Page 271): $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits in IOvalue specified in Mask1 are 1 and the bits specified in Mask0 are 0), then automatically a pause_list is executed. The condition is checked once per 10 μs clock cycle. • The paused list can only be resumed by restart_list. • Mask1 = Mask0 = 0 disables the condition. • If the condition is disabled or no list is currently executed, nothing else happens. • With set_pause_list_cond in the case of an error the currently executed list can be paused immediately by a hardware circuit. This avoids using a time critical call of a command via the operating system. • As of DLL 544, OUT 544, the following applies: a conditional pause_list takes precedence over a simultaneously present /STOP signal.
RTC4→RTC5	New command.
Version info	Last change DLL 544, OUT 544: precedence reversed in favor of pause_list .
References	pause_list , restart_list , set_pause_list_not_cond

Ctrl Command	set_pause_list_not_cond
Function	Defines the "NOT" condition at the 16-bit digital input port of the EXTENSION 1 socket connector under which a pause_list automatically is executed.
Call	set_pause_list_not_cond(Mask1, Mask0)
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the least 16 bits are evaluated. Mask0 Like Mask1.
Comments	• If a list is currently executed and the following condition is <i>not</i> met (see also Chapter 9.3.2 "Conditional Command Execution", Page 271): $((IOvalue \text{ AND } Mask1) = Mask1) \text{ AND } (((\text{not } IOvalue) \text{ AND } Mask0) = Mask0)$ (= if the bits in IOvalue specified in Mask1 are 1 and the bits specified in Mask0 are 0), then automatically a pause_list is executed. The condition is checked once per 10 μs clock cycle. • The paused list can only be resumed by restart_list. • Mask1 = Mask0 = 0 disables the condition. • If the condition is disabled or no list is currently executed, nothing else happens. • With set_pause_list_not_cond in the case of an error the currently executed list can be paused immediately by a hardware circuit. This avoids using a time critical call of a command via the operating system. • A conditional pause_list takes precedence over a simultaneously present /STOP signal.
RTC4→RTC5	New command.
Version info	Available as of DLL 544, OUT 544.
References	pause_list , restart_list , set_pause_list_cond

Variable List Command	set_pixel
Function	In the Pixel Output Mode , defines the laser control parameters (pulse length and analog voltage level) for one pixel in an image line.
Call	set_pixel(PortOutValue1, PortOutValue2)
Parameters	PortOutValue 1 Pixel pulse length. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
	PortOutValue 2 12-bit output value (analog voltage level at the analog output port selected with set_pixel_line) for the pixel. As an unsigned 32-bit value. Higher bits are ignored (see also write_da_x).
Comments	set_pixel is synonymous with set_n_pixel with <code>Number = 1</code> (see comments there).
RTC4→RTC5	- The RTC5 does <i>not</i> support querying of analog voltage levels (see the RTC4's <code>ADChannel</code> parameter and read_pixel_ad command). - RTC4-Pixel mode 0 is no longer supported. - The RTC5 controls the pixel pulses (at the LASER1 port) directly by the <code>PulseLength</code> parameter, no longer by the LASERON signal. "Classic Mode" (compatible to RTC4 Pixel Output Mode 1): In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for <code>PulseLength</code> (<code>PortOutValue1</code>) by 8 and the one for <code>AnalogOut</code> (<code>PortOutValue2</code>) by 4. The allowed value ranges decrease accordingly. The other Pixel Output Modes are not RTC4 compatible.
Version info	–
References	set_n_pixel , set_pixel_line , set_pixel_line_3d

Normal List Command	set_pixel_line	
Function	Activates the Pixel Output Mode and defines various pixel output parameters.	
Call	set_pixel_line(Channel, HalfPeriod, dX, dY)	
Parameters	Channel	Number of the analog output port that should output the analog voltage levels defined by subsequent set_pixel/set_n_pixel calls. As an unsigned 32-bit value. Allowed value range: [1, 2] (1: ANALOG OUT1 , 2: ANALOG OUT2).
	HalfPeriod	<i>Half</i> pixel output period. In bits. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: [104...(2 ³² -1)].
	dX dY	Distance in the x direction and y direction between adjacent pixels. In bits. Each: As a 64-bit IEEE floating point value.
Comments	- Each image line must start with set_pixel_line. set_pixel_line should be preceded by a jump or mark command to the start point of the image line. - Directly after set_pixel_line, the required number of set_pixel and set_n_pixel commands must follow. These transmit the PortOutValue1 and PortOutValue2 parameters. - The first list command after set_pixel_line that is <i>not</i> a set_pixel/set_n_pixel turns off the Pixel Output Mode (set_pixel_line, too, ends the Pixel Output Mode before starting it again). In the process, a default pixel is inserted. - If Channel < 1 or Channel > 2, then set_pixel_line is replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR) and the Pixel Output Mode is <i>not</i> activated. - Note that <i>half</i> the period length must be specified for HalfPeriod. 2 × HalfPeriod is thus the chronological distance between individual pixels (see Chapter 8.7 "Pixel Output Mode", Page 244). HalfPeriod must not be smaller than 104 (otherwise it is clipped). This value corresponds to a pixel frequency of approx. 308 kHz. - For pixel output frequencies above around 100 kHz (that is, for a HalfPeriod < approx. 320) digital-to-analog conversion cannot always be fully completed. With such pixel output frequencies users must carefully verify whether the results are as expected. - The Pixel Output Mode is incompatible with the Softstart Mode (see Chapter 7.4.7 "Softstart Mode", Page 184). - The Pixel Output Mode <i>should not</i> be used in conjunction with "Automatic Laser Control" (see Chapter 7.4.9 "Automatic Laser Control", Page 187) if "Automatic Laser Control" readjusts the 12-bit output values at the ANALOG OUT1 or ANALOG OUT2 output port or pulse lengths (PulseLength) or output periods (HalfPeriod) of the laser signals LASER1 and LASER2. - The Pixel Output Mode can be combined with Processing-on-the-fly (see Chapter 8.6 "Processing-on-the-fly", Page 227).	

Normal List Command	set_pixel_line
RTC4→RTC5	• The Pixel Output Mode <i>cannot</i> be combined with Sky Writing (see Chapter 7.2.4 “Sky Writing”, Page 149). • The Pixel Output Mode <i>cannot</i> be combined with Wobbel (see Chapter 8.4 “Wobbel Mode”, Page 217). • See also Chapter 8.7 “Pixel Output Mode”, Page 244. • The RTC5 only provides the RTC4-Pixel mode = 1. • The RTC5 allows selection of the output channel <code>Channel</code> for analog signals. • For the RTC5, the pixel output period is specified as the half output period <code>HalfPeriod</code>. • With the RTC5, set_pixel_line requires only one list entry, as with other normal list commands. • In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for <code>HalfPeriod</code> by 8 and those for <code>dx</code> and <code>dy</code> by 16. The allowed value ranges decrease accordingly.
Version info	–
References	set_pixel , set_n_pixel , set_pixel_line_3d

Multiple List Command	set_pixel_line_3d
Function	Activates the Pixel Output Mode and defines various pixel output parameters.
Call	set_pixel_line_3d(Channel, HalfPeriod, dX, dY, dZ)
Parameters	Channel Like set_pixel_line .
	HalfPeriod Like set_pixel_line .
	dX Like set_pixel_line .
	dY Like set_pixel_line .
	dZ Distance in the z direction between adjacent pixels. In bits. As a 64-bit IEEE floating point value. As with all z coordinates, dZ too is scaled 16x smaller than dX and dY.
Comments	• set_pixel_line_3d lets you define the pixel spacing (between two adjacent image points on a line) by a 3D vector. • set_pixel_line_3d additionally requires two list-memory positions, if parameter dZ is non-zero. The initial component executes as a short list command before the principal part (a normal list command). Any still-pending delayed short list command gets executed first. • In other respects, set_pixel_line_3d behaves similarly to set_pixel_line (see comments there).
RTC4→RTC5	New command. RTC4 Compatibility Mode: see set_pixel_line . In RTC4 Compatibility Mode , the RTC5 <i>does not</i> multiply the specified value for dZ by 16.
Version info	–
References	set_pixel_line , set_pixel , set_n_pixel

Ctrl Command	set_port_default
Function	Defines the default output value for the selected output port.
Call	<code>set_port_default(Port, Value)</code>
Parameters	Port Selection of the output port for which a default Value is to be defined. As an unsigned 32-bit value. Allowed values: = 0: ANALOG OUT1 output port. See also Section "12-Bit Analog Output Port 1 and 2", Page 61. = 1: ANALOG OUT2 output port. See also Section "12-Bit Analog Output Port 1 and 2", Page 61. = 2: 8-bit digital output port. See also Section "8-Bit Digital Output Port", Page 68. = 3: 16-bit digital output port. See also Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65. = 4: 2-bit digital output port. See also Section "2-Bit Digital Output Port", Page 61. Value Default value. As an unsigned 32-bit value. Allowed values: - For Port = 0/1: 12-bit values [0...4095]. Bits with higher significance are ignored. - For Port = 2: 8-bit values [0...255]. Bits with higher significance are ignored. - For Port = 3: 16-bit values [0...(2¹⁶-1)]. Bits with higher significance are ignored. - For Port = 4: 2-bit values [0...3]. Bits with higher significance are ignored. - For Value = "-1" (= 2³²-1 = 0xFFFFFFFF), the corresponding default output functionality is switched off (see comment below).
Comments	• During initialization of the RTC5 (by load_program_file), the default values of all output ports are set to "-1" (= 2³²-1). • If a default value ≠ "-1" has been specified for an output port, the port is set to this default value as soon as processing of a list has ended with stop_execution or by an External Stop. For a default value of "-1", the corresponding output port is <i>not</i> addressed (any existing high-impedance state of the digital output ports is retained). • Default values (≠ "-1") at the output ports 0/1 (ANALOG OUT1 or ANALOG OUT2) are also set at the end of Pixel Output Mode (see Chapter 8.7 "Pixel Output Mode", Page 244). • After initialization of position-dependent or speed-dependent laser control by set_auto_laser_control, the corresponding default value (for output port 0, 1, 2 or 3, depending on the selected laser signal parameter Ctrl), is also outputted each time the laser is switched off after marking or when position-dependent or speed-dependent laser control is set to another Ctrl parameter by set_auto_laser_control or deactivated by set_auto_laser_control (Ctrl = 0) (see Section "General Notes", Page 188). If the default value is then defined as "-1", then the maximum allowed value (4095, 4095, 255 or 65,535) is outputted.

Ctrl Command	set_port_default
Comments (cont'd)	- If the value for <code>Port</code> is invalid, then set_port_default is not executed (get_last_error return code <code>RTC5_PARAM_ERROR</code>). - The default values for the output ports 0...2 can also be defined by set_laser_off_default.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified <code>Value</code> for <code>Port</code> = 0/1 by 4.
Version info	–
References	set_laser_off_default , set_default_pixel , set_port_default_list

Undelayed Short List Command	set_port_default_list
Function	Like set_port_default , but a list command.
Call	<code>set_port_default_list(Port, Value)</code>
Parameters	<code>Port</code> Like set_port_default .
	<code>Value</code> Like set_port_default .
Comments	See set_port_default.
RTC4→RTC5	New command.
Version info	Available as of DLL 544, OUT 544.
References	set_port_default

Ctrl Command	set_pulse_picking
Function	Switches on Pulse Picking Laser Mode .
Call	<code>set_pulse_picking(No)</code>
Parameters	No As an unsigned 32-bit value. Allowed value range: [0...63]. = 0: LASER2 puts out the LASERON signal. = 1...63: LASER2 puts out each Noth LASER1 pulse. No > 63: No is clipped to 63 (get_last_error return code RTC5_PARAM_ERROR).
Comments	• For Pulse Picking Laser Mode, see Chapter 7.4.8 "Pulse Picking Laser Mode", Page 186. • The pulse picking signals are outputted until a different laser mode gets set by set_laser_mode (the Pulse Picking Laser Mode gets switched off if any other laser mode switches on by set_laser_mode). • You can <i>not</i> switch on Pulse Picking Laser Mode by set_laser_mode.
RTC4→RTC5	New command.
Version info	–
References	set_laser_mode , set_pulse_picking_list

Ctrl Command	set_pulse_picking_length
Function	Defines a constant pulse length for the "laser active" LASER2 signal in Pulse Picking Laser Mode .
Call	<code>set_pulse_picking_length(Length)</code>
Parameters	Length Pulse length. As an unsigned 32-bit value. Allowed value range: [0...65,535]. Bits with higher significance are ignored. 1 bit equals 1/64 µs. The default value after load_program_file is 0.
Comments	• For the value to take effect, you must activate Pulse Picking Laser Mode by set_pulse_picking and constant pulse length mode by set_laser_control(Bit #7 = 1). The value then takes immediate effect, even if a marking is carried out. • If Pulse Picking Laser Mode has been activated, but not constant pulse length mode (set_laser_control(Bit #7 = 0)), then the pulse-picking signal uses the pulse length of the LASER1 signal (see Chapter 7.4.8 "Pulse Picking Laser Mode", Page 186). • If neither Pulse Picking Laser Mode nor constant pulse length mode has been activated, then the value <code>Length</code> is irrelevant (set_pulse_picking, set_laser_control and set_pulse_picking_length can be activated in any desired order). • After set_pulse_picking(0), LASER2 is continuously output the LASERON signal, but no constant pulse length signal.
RTC4→RTC5	New command.
Version info	–
References	–

Undelayed Short List Command	set_pulse_picking_list
Function	Like set_pulse_picking , but a list command.
Call	<code>set_pulse_picking_list(No)</code>
Parameters	No Like set_pulse_picking .
RTC4→RTC5	New command.
Version info	–
References	set_pulse_picking

Ctrl Command	set_qswitch_delay
Function	In the YAG modes, defines the delay length of the first Q-Switch pulse with reference to the FirstPulseKiller signal (see also Figure 60).
Call	<code>set_qswitch_delay(Delay)</code>
Parameters	Delay Q-Switch delay. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{26}-1)]$.
Comments	• Values over $(2^{26}-1)$ are clipped. • The YAG modes are selectable by set_laser_mode ([1, 2, 3 or 5]). • Also for YAG modes defined with set_laser_mode([1-3]), the length of the Q-Switch delay can be subsequently changed by this command; and for YAG Mode 2 also with set_firstpulse_killer or set_firstpulse_killer_list.
RTC4→RTC5	New command. In RTC4 Compatibility Mode , the RTC5 multiplies the specified value for Delay by 8. The allowed value range decreases accordingly.
Version info	–
References	set_qswitch_delay_list , set_laser_control , set_laser_pulses_ctrl , set_laser_pulses , set_laser_timing , set_firstpulse_killer , set_firstpulse_killer_list

Undelayed Short List Command	set_qswitch_delay_list
Function	Like set_qswitch_delay , but a list command.
Call	<code>set_qswitch_delay_list(Delay)</code>
Parameters	Delay Like set_qswitch_delay .
RTC4→RTC5	New command.
Version info	–
References	set_qswitch_delay

Ctrl Command	set_rot_center	
Function	Sets the rotation center of a Processing-on-the-fly rotation correction (see set_fly_rot and set_fly_rot_pos).	
Call	set_rot_center(X, Y)	
Parameters	X	Position of the rotation center referenced to the zero point (0 0) of the Image field . As a signed 32-bit value. Allowed value range: $[-2^{24} \dots (2^{24}-1)]$.
	Y	Like X (analogously).
Comments	- For Processing-on-the-fly correction, see Chapter 8.6.3 "Compensating Rotary Motions", Page 232. - The position of the rotation center should be defined by set_rot_center or set_rot_center_list before the Processing-on-the-fly correction is activated by set_fly_rot or set_fly_rot_pos. - The rotation center can also lie outside the Image field (the allowed range equals the 32 fold of the Image field). - Usage of a second scan head is only practical if it is set up for exactly the same rotational center.	
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for X and Y by 16. The allowed value range decreases accordingly.	
Version info	–	
References	set_rot_center_list , set_fly_rot , set_fly_rot_pos	

Delayed Short List Command	set_rot_center_list	
Function	Like set_rot_center , but a list command.	
Call	set_rot_center_list(X, Y)	
Parameters	X	Like set_rot_center .
	Y	Like set_rot_center .
RTC4→RTC5	New command.	
Version info	–	
References	set_rot_center	

Ctrl Command	set_rtc4_mode
Function	Sets RTC4 Compatibility Mode as the current RTC5 DLL operation mode.
Call	<code>set_rtc4_mode()</code>
Comments	• RTC4 Compatibility Mode is an optional operation mode of the RTC5 DLL. It has been made available so that user programs written for the RTC4 can also be processed by the RTC5 (to a large extent) without needing to modify the programming code. However, a prerequisite here is that the program can only contain RTC4 commands that also exist with unchanged functionality as RTC5 commands. This user manual's list of commands, when applicable, notes such changes in the "RTC4→RTC5" row. • RTC5 Standard Mode is pre-defined as the default RTC5 DLL operation mode and can also be specified subsequently by set_rtc5_mode. • The current RTC5 DLL operation mode can be queried by get_rtc_mode. • set_rtc4_mode is available even without explicit access rights to a particular board. • set_rtc4_mode is not available as a multi-board command. • This command's scope is not board-specific, but rather global to the RTC5 DLL and all RTC5 Boards to which the user program has access rights. • The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by set_rtc4_mode.
RTC4→RTC5	New command.
Version info	–
References	get_rtc_mode , set_rtc5_mode

Ctrl Command	set_rtc5_mode
Function	Sets RTC5 Standard Mode as the current RTC5 DLL operation mode (default setting).
Call	set_rtc5_mode()
Comments	• The current RTC5 DLL operation mode can be queried by get_rtc_mode. • set_rtc5_mode is available even without explicit access rights to a particular RTC5 Board. • set_rtc5_mode is not available as a multi-board command. • set_rtc5_mode is not board-specific, but rather global to the RTC5 DLL and all RTC5 Boards to which the user program has access rights. • The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by set_rtc5_mode.
RTC4→RTC5	New command.
Version info	–
References	get_rtc_mode , set_rtc4_mode

Ctrl Command	set_scale
Function	Uses a specified scaling factor common to the x axis and y axis to define the scaling matrix M_S for all subsequent coordinate transformations, see Chapter 8.2 "Coordinate Transformations", Page 209 .
Call	<code>set_scale(HeadNo, Scale, at_once)</code>
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. = 1: The definition only affects the first scan head connector. = 2: The definition only affects the second scan head connector. = 0, 3: The definition affects <i>both</i> scan head connectors. Only the two least significant bits are evaluated.
	Scale Scaling factor. As a 64-bit IEEE floating point value. Allowed value range: $[-16...+16]$. If the parameter is set to an invalid value, it is set to 1. Negative values additionally produce a mirroring around both axes (corresponding to a 180° rotation).
	at_once Determines when the defined transformation becomes effective. Like <code>set_matrix(HeadNo = 0...3)</code> . As an unsigned 32-bit value.
Comments	• See Chapter 8.2 "Coordinate Transformations", Page 209.
RTC4→RTC5	New command.
Version info	–
References	set_scale_list , set_angle , set_matrix , set_offset

Variable List Command	set_scale_list
Function	Like set_scale , but a list command.
Call	<code>set_scale_list(HeadNo, Scale, at_once)</code>
Parameters	HeadNo Like set_scale .
	Scale Like set_scale .
	at_once Determines when the defined transformation becomes effective. Like set_matrix_list . As an unsigned 32-bit value.
RTC4→RTC5	New command.
Version info	–
References	set_scale

Delayed Short List Command	set_scanner_delays	
Function	Sets the Scanner Delays .	
Call	set_scanner_delays(Jump, Mark, Polygon)	
Parameters	Jump	Scanner delay of type: Jump Delay . As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
	Mark	Scanner delay of type: Mark Delay . As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
	Polygon	Scanner delay of type: Polygon Delay . As an unsigned 32-bit value. 1 bit equals 10 μ s. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	• See also Chapter 7.2.2 "Scanner Delays", Page 135. • Variable Delays for jumps and Polylines can be set by set_delay_mode. • The specified delays are automatically adjusted by the RTC5 to avoid laser control errors (see Section "Automatic Delay Adjustments", Page 143). • The default setting after load_program_file corresponds to <code>set_scanner_delays(9, 6, 3)</code>.	
RTC4→RTC5	Unchanged functionality. In addition: increased value range.	
Version info	–	
References	set_delay_mode, set_laser_delays	

Ctrl Command	set_serial	
Function	Sets the starting serial number of the serial-number-set most recently selected by select_serial_set (= 0 after load_program_file) and sets the increment size for this serial-number-set to 1.	
Call	set_serial(No)	
Parameters	No	Serial number. As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	• set_serial is synonymous with set_serial_step with Step = 1 (see comments there).	
RTC4→RTC5	New command.	
Version info	–	
References	set_serial_step, select_serial_set	

Ctrl Command	set_serial_step
Function	Sets the starting serial number and the increment size for the serial-number-set most recently selected by select_serial_set (= 0 after load_program_file).
Call	<code>set_serial_step(No, Step)</code>
Parameters	No Serial number. As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{32}-1)]$.
	Step Increment size. As an unsigned 32-bit value. Allowed value range: $[0 \dots 9999]$; only the last 4 decimal digits are used.
Comments	• load_program_file sets the starting serial number to 0 and the increment size to 1. • If mark_serial or mark_serial_abs has been called with Mode $M_2 = 1$ (that is, automatic serial-number incrementing has been deactivated), then the increment size setting has no effect. • If $\text{Step} = 0$, then incrementing does not occur, except in the case of markless marking (see mark_serial or mark_serial_abs: digits = 0), which always increments serial numbers by 1. • For set_serial_step usage, see Chapter 7.5.2 "Marking Serial Numbers", Page 196.
RTC4→RTC5	New command.
Version info	–
References	mark_serial , mark_serial_abs , set_serial , select_serial_set , select_serial_set_list

Normal List Command	set_serial_step_list
Function	Like set_serial_step , but a list command.
Call	<code>set_serial_step_list(No, Step)</code>
Parameters	No Like set_serial_step .
	Step Like set_serial_step .
Comments	• See set_serial_step.
RTC4→RTC5	New command.
Version info	–
References	set_serial_step , select_serial_set_list , get_list_serial , mark_serial , mark_serial_abs

Ctrl Command	set_sky_writing
Function	Activates Sky Writing Mode 1 and sets the corresponding parameters or switches off Sky Writing .
Call	<code>set_sky_writing(Timelag, LaserOnShift)</code>
Parameters	Timelag Like set_sky_writing_para .
	LaserOnShift Like set_sky_writing_para .
Comments	set_sky_writing is identical to <code>set_sky_writing_para(Timelag, LaserOnShift, Nprev, Npost)</code> with $Nprev = \text{approx. } (0.15 \times \text{Timelag})$ and $Npost = \text{approx. } (0.1 \times \text{Timelag})$. - See also information about set_sky_writing_para and Chapter 7.2.4 "Sky Writing", Page 149.
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing_para , set_sky_writing_list

Ctrl Command	set_sky_writing_limit
Function	Defines the limit for Sky Writing switching in Sky Writing Mode 3 .
Call	<code>set_sky_writing_limit(Limit)</code>
Parameters	Limit Limit value. As a 64-bit IEEE floating point value. Allowed value range: [-1.0...+1.0]. Out-of-range values are clipped to the boundary values.
Comments	• For set_sky_writing_limit usage, see Chapter 7.2.4 "Sky Writing", Page 149. • Limit is the cosine of the angular limit (for angular change between consecutive vectors or arcs within a Polyline) for which a Sky Writing motion should be performed. • The initialized value (after program start) is Limit = 0 (angular limit = 90°).
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing_limit_list

Undelayed Short List Command	set_sky_writing_limit_list
Function	Like set_sky_writing_limit , but a list command.
Call	<code>set_sky_writing_limit_list(Limit)</code>
Parameters	Limit Like set_sky_writing_limit .
Comments	• See set_sky_writing_limit.
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing_limit

Normal List Command	set_sky_writing_list
Function	Like set_sky_writing , but a list command.
Call	<code>set_sky_writing_list(Timelag, LaserOnShift)</code>
Parameters	Timelag Like set_sky_writing_para. LaserOnShift Like set_sky_writing_para.
Comments	• See set_sky_writing, set_sky_writing_para and set_sky_writing_para_list.
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing , set_sky_writing_para , set_sky_writing_para_list .

Ctrl Command	set_sky_writing_mode
Function	Toggles the Sky Writing mode.
Call	set_sky_writing_mode(Mode)
Parameters	Mode Sky Writing mode. As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{32}-1)]$. = 0: Sky Writing is deactivated. = 1: Sky Writing Mode 1 is activated. = 2: Sky Writing Mode 2 is activated. > 2: Sky Writing Mode 3 is activated.
Comments	• Sky Writing Mode 2 and Sky Writing Mode 3 can only be activated (by $\text{Mode} > 1$), if: – Sky Writing Mode 1 has been activated by <code>set_sky_writing_para (Timelag $\geq$ 1/8)</code> before – AND is still active • After <code>set_sky_writing_mode(Mode = 0)</code>, subsequent Sky Writing reactivation: – Is possible by <code>set_sky_writing_mode(Mode > 0)</code> – Not possible after <code>set_sky_writing_para (Timelag < 1/8)</code> • Each mode switch by <code>set_sky_writing_mode</code> becomes effective as of the next list command. • If you <i>reactivate</i> Sky Writing Mode 1 (switching from $\text{Mode} = 0$ to 1) during execution of a list, then an already-begun Mark command still executes to completion <i>without</i> Sky Writing. • If you <i>deactivate</i> Sky Writing Mode 1 (switching from $\text{Mode} = 1$ to 0) during execution of a list, then a Mark command already begun in Sky Writing Mode 1 still executes to completion in Sky Writing Mode 1 and then is appended with a Mark Delay. • Activation or deactivation of Sky Writing Mode 2 or Sky Writing Mode 3 (switching to or from $\text{Mode} = 2$ or 3) is not possible if the BUSY list execution status is set. Then <code>set_sky_writing_mode</code> is ignored (get_last_error return code <code>RTC5_BUSY</code>). In contrast, <code>set_sky_writing_mode</code> is executed when a list has been paused by <code>set_wait</code>.
RTC4→RTC5	New command.
Version info	–
References	<code>set_sky_writing_mode_list</code> , <code>set_sky_writing_para</code>

Normal List Command	set_sky_writing_mode_list
Function	Like set_sky_writing_mode , but a list command.
Call	set_sky_writing_mode_list(Mode)
Parameters	Mode Like set_sky_writing_mode .
Comments	• By set_sky_writing_mode_list, Sky Writing Mode 2 and Sky Writing Mode 3 can be activated or deactivated even within a list (switching to or from Mode = 2 or 3). Here, an already-begun Mark command is finished with Sky Writing Mode 1 and the next is started with it. • Deactivation of Sky Writing Mode 1 by set_sky_writing_mode_list (switching from Mode = 1 to 0) results in the addition of a Mark Delay defined prior to activation of Sky Writing – provided that no other delay is in effect (for example, a Jump Delay).
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing_mode

Ctrl Command	set_sky_writing_para						
Function	Activates Sky Writing Mode 1 and sets the corresponding parameters or switches Sky Writing off.						
Call	set_sky_writing_para(Timelag, LaserOnShift, Nprev, Npost)						
Parameters	Timelag Sky Writing parameter. 1.0 equals 1 μs. The Timelag value is used with an accuracy of 1/64 μs. As a 64-bit IEEE floating point value. ≥ 1/4: Sky Writing Mode 1 is activated. From 1/8 to 1/4, Sky Writing is activated, but Timelag = 0 is internally applied. < 1/8: Sky Writing is deactivated.						
	LaserOnShift Shift (for positive values: delay) of the point of time, where the are switched on (see Figure 53). 1 Bit equals 0.5 μs. As a signed 32-bit value. Negative values smaller than the complete run-in phase are clipped at runtime, therefore the following applies automatically: <table><tr><td>LaserOnShift</td><td>Mode</td></tr><tr><td>≥ ca. (−40) × Nprev</td><td>1</td></tr><tr><td>≥ ca. (−20) × Nprev</td><td>2 or 3</td></tr></table>	LaserOnShift	Mode	≥ ca. (−40) × Nprev	1	≥ ca. (−20) × Nprev	2 or 3
	LaserOnShift	Mode					
	≥ ca. (−40) × Nprev	1					
≥ ca. (−20) × Nprev	2 or 3						
Nprev Defines the duration of the run-in phase (see Figure 53). 1 bit equals 10 μs. As an unsigned 32-bit value. Allowed value range: 0 ≤ Nprev ≤ (2³²−1). <table><tr><td>Run-in duration [μs]</td><td>Mode</td></tr><tr><td>20 × Nprev</td><td>1</td></tr><tr><td>10 × Nprev</td><td>2 or 3</td></tr></table>	Run-in duration [μs]	Mode	20 × Nprev	1	10 × Nprev	2 or 3	
Run-in duration [μs]	Mode						
20 × Nprev	1						
10 × Nprev	2 or 3						
Npost Defines the duration of the run-out phase (see Figure 53). 1 bit equals 10 μs. As an unsigned 32-bit value. Allowed value range: 0 ≤ Npost ≤ (2³²−1). <table><tr><td>Run-out duration [μs]</td><td>Mode</td></tr><tr><td>20 × Npost</td><td>1</td></tr><tr><td>10 × Npost</td><td>2 or 3</td></tr></table>	Run-out duration [μs]	Mode	20 × Npost	1	10 × Npost	2 or 3	
Run-out duration [μs]	Mode						
20 × Npost	1						
10 × Npost	2 or 3						
Comments	- For information on Sky Writing mode and a description of the parameters, see Chapter 7.2.4 "Sky Writing", Page 149. - If Nprev ≥ 65,535, then set_sky_writing_para behaves similarly to set_sky_writing: Nprev is set to a value of approx. (0.15 × Timelag). - If Npost ≥ 65,535, then set_sky_writing_para behaves similarly to set_sky_writing: Npost is set to a value of approx. (0.1 × Timelag). - set_sky_writing_para cannot be used to switch back and forth between Sky Writing Mode 1, Sky Writing Mode 2 and Sky Writing Mode 3 (the proper command for this is set_sky_writing_mode).						

Ctrl Command	set_sky_writing_para
Comments (cont'd)	- When set_sky_writing_para is called and <i>no</i> list is running or a list has been halted by set_wait, the following applies: - With $\text{Timelag} \geq 1/8$, Sky Writing Mode 1 is activated, if Sky Writing has not been active before. Sky Writing Mode 2 or Sky Writing Mode 3 remain, but the next Mark command is always started in Sky Writing Mode 1 - With $\text{Timelag} < 1/8$, Sky Writing is deactivated. - When set_sky_writing_para is called and a list is running or a list has been halted by pause_list, the following applies: - If Sky Writing Mode 2 or Sky Writing Mode 3 is active, then set_sky_writing_para is not executed (get_last_error return code RTC5_BUSY) - If no Sky Writing or Sky Writing Mode 1 is active, set_sky_writing_para parameters are going to be effective upon the next list command. - In Sky Writing Mode 1, an already started Mark command is executed with the previous parameters and ended with Sky Writing Mode 1. - In Sky Writing Mode 1, the next Mark command is started in Sky Writing Mode 1, if $\text{Timelag} \geq 1/8$. Otherwise, it is terminated and a Mark Delay is inserted in Sky Writing Mode 1. - For Sky Writing Mode 2 or Sky Writing Mode 3, $N_{\text{prev}} = 0$ and/or $N_{\text{post}} = 0$ automatically get(s) internally corrected to 1 (this is not treated as an error). If you subsequently activate Sky Writing Mode 1 (by set_sky_writing_mode), then $N_{\text{prev}} = 0$ and/or $N_{\text{post}} = 0$ is/are reused.
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing , set_sky_writing_para_list

Normal List Command	set_sky_writing_para_list
Function	Like set_sky_writing_para , but a list command.
Call	set_sky_writing_para_list(Timelag, LaserOnShift, Nprev, Npost)
Parameters	Timelag Like set_sky_writing_para .
	LaserOnShift Like set_sky_writing_para .
	Nprev Like set_sky_writing_para .
	Npost Like set_sky_writing_para .
Comments	• set_sky_writing_para_list can be executed within a list, even if Sky Writing Mode 2 or Sky Writing Mode 3 is active. Here, an already-begun Mark command is finished with Sky Writing Mode 1 and the next is started with it. • By set_sky_writing_para_list, you cannot switch from Sky Writing Mode 2 or Sky Writing Mode 3 to Sky Writing Mode 1 permanently. For this, use set_sky_writing_mode_list. • Deactivation of Sky Writing Mode 1, Sky Writing Mode 2 or Sky Writing Mode 3 by set_sky_writing_para_list results in the addition of a Mark Delay defined prior to activation of Sky Writing – provided that no other delay is in effect (for example, a Jump Delay).
RTC4→RTC5	New command.
Version info	–
References	set_sky_writing_para

Ctrl Command	set_softstart_level		
Function	Sets the softstart values.		
Call	set_softstart_level(Index, Level)		
Parameters	Parameter	Allowed Values	Description
	Index	0...1023	Index number of the softstart table value. As an unsigned 32-bit value.
	Level	0...2 ¹² -1	For Softstart Mode = 1, 2, 11 and 12 (see set_softstart_mode). Value 0 corresponds to an analog voltage value of 0 V. Value 2 ¹² -1 corresponds to 10 V.
		0...(2 ³² -1)	For Softstart Mode = 3 and 13 (see set_softstart_mode). 1 bit corresponds to a pulse length of 1/64 µs (in RTC5 Standard Mode) or 1/8 µs (in RTC4 Compatibility Mode). As an unsigned 32-bit value.
Comments	• set_softstart_level defines the Level value for the pulse number Index. set_softstart_level must be called separately for each individual Level value. • Index must be in the range [0...1023], otherwise set_softstart_level is not executed (get_last_error return code RTC5_PARAM_ERROR). • If Index ≥ Number (Number is defined by set_softstart_mode) then set_softstart_level is executed but the defined value is not used during softstart. • Be sure to set as many softstart values as you defined by set_softstart_mode (unspecified softstart values have undefined contents). • The Softstart Mode is enabled by set_softstart_mode. • For Softstart Modes 1, 2, 11 or 12, Level is restricted to 12 bits; higher bits are ignored (see also write_da_x or write_da_x_list). • If Softstart Mode = 3 or 13 and the specified softstart pulse length is larger than the laser signal period duration specified by set_laser_pulses or set_laser_timing ($2 \times \text{HalfPeriod}$), then the laser remains continuously on. • Particularly if softstart values are to be outputted as analog voltage values (Softstart Mode = 1, 2, 11 or 12), the set_softstart_mode command must be called <i>before</i> first-time use of the set_softstart_level command; otherwise the specified value is not correctly converted to an analog value. • Also observe the note in Chapter 7.4.7 "Softstart Mode", Page 184. • If the "freely definable wobble shape" wobble mode is activated, then this command has no effect, see Chapter 8.4 "Wobble Mode", Page 217.		

Ctrl Command	set_softstart_level
RTC4→RTC5	Basically unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , analog voltage levels are internally multiplied by 4 and pulse length values by 8. The allowed range of values for Level is correspondingly smaller. Due to this difference in the handling of analog voltage and pulse length values, set_softstart_mode must always be called before set_softstart_level .
Version info	–
References	set_softstart_mode , set_softstart_mode_list , set_softstart_level_list

Normal List Command	set_softstart_level_list
Function	Like set_softstart_level , but a list command.
Call	<code>set_softstart_level_list(Index, Level1, Level2, Level3)</code>
Parameters	Index Like set_softstart_level .
	Level1 Like set_softstart_level .
	Level2 Like set_softstart_level .
	Level3 Like set_softstart_level .
Comments	set_softstart_level_list defines three Level values for the pulses number Index through Index+2. set_softstart_level_list must be called separately for each individual Level1/2/3 value-triple. Index must be in the range [0...1023], otherwise set_softstart_level_list is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). - Softstart does not use level values where the index count exceeds Number (Number is specified by set_softstart_mode). - Otherwise, set_softstart_level_list is identical to the control command set_softstart_level (see also the comments there). - If the “freely defined wobble shape” wobble mode is activated, then this command has no effect, see Chapter 8.4 “Wobble Mode”, Page 217.
RTC4→RTC5	New command. In RTC4 Compatibility Mode : see set_softstart_level .
Version info	–
References	set_softstart_level , set_softstart_mode , set_softstart_mode_list

Ctrl Command	set_softstart_mode	
Function	Switches Softstart Mode on or off.	
Call	set_softstart_mode(Mode, Number, Delay)	
Parameters	Mode = 0 Disables the Softstart Mode, but does not remove previously loaded softstart values. = 1, 11 The softstart values defined by set_softstart_level are outputted as analog voltage values at the ANALOG OUT1 output port. = 2, 12 The softstart values defined by set_softstart_level are outputted as analog voltage values at the ANALOG OUT2 output port. = 3, 13 The softstart values defined by set_softstart_level specify the pulselenghts of the first Number laser signal pulses at the LASER1 port (neither in Laser Mode 4 nor in Laser Mode 6). As an unsigned 32-bit value. Number 1...1024 Number of softstart values to be transmitted. As an unsigned 32-bit value. Delay 0...(2³²-1). 1 bit equals 10 µs. Defines the time period for which the laser must have been switched off before softstart is activated at the next switch-on. As an unsigned 32-bit value.	
Comments	• set_softstart_mode must be called <i>before</i> first-time use of the set_softstart_level command, particularly if softstart values are to be outputted as analog voltage values (default setting for softstart value handling: Mode = 3). • The individual softstart values Level(0) – Level(Number – 1) must be provided individually by separate calls of set_softstart_level. Indices for which no Level value has been supplied are undefined. • If the values for Mode or Number are invalid, then Softstart Mode is switched off. • When Softstart Mode is switched off, the already transmitted values are not deleted and can be further used after a renewed switch-on. Here, you should not unintentionally switch between analog voltage values (Mode = 1, 2, 11 or 12) and pulselenghts (Mode = 3 or 13). In contrast, you can switch at any time between ANALOG OUT1 (Mode = 1 or 11) and ANALOG OUT2 (Mode = 2 or 12). • The Softstart Mode is incompatible with the Pixel mode (see Chapter 8.7 “Pixel Output Mode”, Page 244). set_softstart_mode is executed, but softstart is not executed while Pixel mode is operational. • The Softstart Mode should <i>not</i> be used in conjunction with “Automatic Laser Control” (see page 187) if “Automatic Laser Control” readjusts the 12-bit output values at the ANALOG OUT1 or ANALOG OUT2 output ports or pulse lengths (PulseLength) or output periods (HalfPeriod) of the laser signals LASER1 and LASER2. Softstart Mode cannot be combined with “freely definable wobble shapes”. If one is defined and activated, set_softstart_mode has no effect. • Observe the notes in Chapter 7.4.7 “Softstart Mode”, Page 184.	

Ctrl Command	set_softstart_mode
RTC4→RTC5	Basically unchanged functionality. In addition: increased value range. Instead of acquiring analog softstart values with the trailing edge of the laser signal pulse, a 180° phase-shifted laser signal can be set as a trigger signal for realizing phase-shifted data acquisition with the RTC5 (see notes in Chapter 7.4.7 "Softstart Mode", Page 184). An index shift no longer exists under any circumstances.
Version info	–
References	set_softstart_level , set_softstart_level_list , set_softstart_mode_list

Variable List Command	set_softstart_mode_list
Function	Like set_softstart_mode , but a list command.
Call	<code>set_softstart_mode_list(Mode, Number, Delay)</code>
Parameters	Mode Like set_softstart_mode .
	Number Like set_softstart_mode .
	Delay Like set_softstart_mode .
Comments	• set_softstart_mode_list switches off the Signals for "Laser Active" Operation by executing list_nop and waits until the LaserOff Delay has expired. • Otherwise, set_softstart_mode_list is identical to the control command set_softstart_mode (see also the comments there).
RTC4→RTC5	New command.
Version info	–
References	set_softstart_mode , set_softstart_level , set_softstart_level_list

Ctrl Command	set_standby
Function	Defines the output period and the pulse length of the standby pulses for “laser standby” operation or – in Laser Mode 4 and Laser Mode 6 – the continuously-running laser signals for “laser active” and “laser standby” operation.
Call	<code>set_standby(HalfPeriod, PulseLength)</code>
Parameters	HalfPeriod <i>Half</i> of the standby output period. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots + (2^{32} - 1)]$.
	PulseLength Pulse length of the standby pulses. As an unsigned 32-bit value. 1 bit equals 1/64 μ s. Allowed value range: $[0 \dots (2^{32} - 1)]$.
Comments	• After a Hardware reset, (and after load_program_file), the LASER1, LASER2 and LASERON ports must be first-time-activated with set_laser_control before standby pulses can be activated by set_standby or set_standby_list (see Chapter 7.4 “Laser Control”, Page 173). At the same time, the signal level of the standby pulses can be defined with set_laser_control. • The standby pulses are available in <i>all</i> laser modes (CO₂ Mode, YAG Mode 1, YAG Mode 2, YAG Mode 3, Laser Mode 4, YAG Mode 5, Laser Mode 6). They can be deactivated (turned off) by setting the standby pulse length and/or the standby output period to zero (default). • With HalfPeriod, <i>half</i> the standby output period must be specified, see Figure 59 and Figure 61. • If the Softstart Mode is used (see Chapter 7.4.7 “Softstart Mode”, Page 184), HalfPeriod should not be smaller than 104 (this is not automatically checked). This value corresponds to a laser pulse frequency of approx. 308 kHz. • If PulseLength is larger than the output period ($2 \times \text{HalfPeriod}$), the laser is permanently on. • The laser control mode has to be set with set_laser_mode (see also Chapter 7.4 “Laser Control”, Page 173). To set the active output period and pulse length for the “laser active” laser control signals (beside for Laser Mode 4 and Laser Mode 6), use set_laser_pulses_ctrl, set_laser_pulses or set_laser_timing.
RTC4→RTC5	Basically unchanged functionality. In addition: increased value range. However: In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for HalfPeriod and PulseLength by 8. The allowed value ranges decrease accordingly. The smallest reasonable value for HalfPeriod with Softstart Mode is then 13.
Version info	–
References	set_standby_list , get_standby

Delayed Short List Command	set_standby_list
Function	Like set_standby , but a list command.
Call	<code>set_standby_list(HalfPeriod, PulseLength)</code>
Parameters	HalfPeriod Like set_standby .
	PulseLength Like set_standby .
RTC4→RTC5	Siehe set_standby
Version info	–
References	set_standby

Ctrl Command	set_start_list
Function	Opens the list memory area for writing of list commands and sets the input pointer to the start of the desired list ("List 1" or "List 2").
Call	<code>set_start_list(ListNo)</code>
Parameters	ListNo Number of the list in which the input pointer should be set. As an unsigned 32-bit value. Allowed values: [uneven: "List 1", even: "List 2"].
Comments	• set_start_list is synonymous with set_start_list_pos for Pos = 0. • Alternatively, the commands set_start_list_1 and set_start_list_2 (with no parameter) can be used.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	execute_list , read_status , set_start_list_pos

Ctrl Command	set_start_list_1
Function	See set_start_list .
Call	<code>set_start_list_1()</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_start_list

Ctrl Command	set_start_list_2
Function	See set_start_list .
Call	<code>set_start_list_2()</code>
RTC4→RTC5	Unchanged functionality.
Version info	–
References	set_start_list

Ctrl Command	set_start_list_pos
Function	Opens the list memory area for writing of list commands and sets the input pointer to the specified <i>relative</i> position in the specified list ("List 1" or "List 2").
Call	<code>set_start_list_pos(ListNo, Pos)</code>
Parameters	ListNo Number of the list for which the input pointer should be set. As an unsigned 32-bit value. Allowed values: [uneven: "List 1", even: "List 2"].
	Pos Position of the input pointer (offset relative to the start of the respective list). As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{20}-1)]$.
Comments	• The next list command is stored at the specified address and all further list commands at the subsequent addresses in the selected list. • The specified list is unconditionally opened for loading. There is no checking of whether the list is currently being processed, see also Chapter 6.4.1 "Loading Lists", Page 94 and load_list. • The input pointer <i>cannot</i> be set to the protected area ("List 3"). If the protected area is to be written to (at the full risk of users) with set_start_list_pos, then this area can be temporarily allocated to "List 2" by <code>config_list(Mem1, -1)</code>. After writing, configuration of the area's protection should be restored: <code>config_list(Mem1, Mem2)</code>. • For uneven ListNo values, "List 1" is opened, otherwise "List 2". This facilitates continuous automatic list changing though incrementing counts. • If "List 2" has not been assigned memory (Mem2 = 0, see config_list) then "List 1" is opened. • If Pos is specified as being larger than the memory area of the respective list (Pos > Mem1 or Pos > Mem2), then Pos is set to 0. • The status values of the selected list (see also read_status) are set as follows: LOAD list status set, READY list status reset, USED list status reset. The LOAD list status of the other corresponding list is reset. • set_start_list_pos triggers a flush of the buffered list input, see Chapter 6.4.1 "Loading Lists", Page 94. • set_start_list_pos also covers the specialized variants set_start_list_1, set_start_list_2, set_start_list and set_input_pointer. • <i>Notice! If the end of the respective list memory area is reached, the list input pointer is automatically reset to the start of the same list memory area. Make sure not to overwrite any commands still needed by your user program.</i>
RTC4→RTC5	New command.
Version info	–
References	execute_list_pos , read_status

Ctrl Command	set_sub_pointer
Function	Stores the absolute start address of a command list in the internal management table for indexed subroutines.
Call	<code>set_sub_pointer(Index, Pos)</code>
Parameters	Index Index of the indexed subroutine whose starting address <code>Pos</code> should be entered in the management table. As an unsigned 32-bit value. Allowed value range: [0...1023].
	Pos Absolute start address. As an unsigned 32-bit value. Allowed value range: [0...(2 ²⁰ -1)].
Comments	• If <code>Index > 1023</code> and/or <code>Pos > (2²⁰-1)</code>, then set_sub_pointer is <i>not</i> executed (get_last_error return code <code>RTC5_PARAM_ERROR</code>). • The set_sub_pointer command can be used for referencing a nonindexed subroutine, which thereby becomes an indexed subroutine that is protectable by save_disk/load_disk and/or callable by the index. • set_sub_pointer can also be used to reference anew an indexed subroutine, character or text string so that it can also be called by a second index. Here, it is preferable to use the copy_dst_src command for index management. • The start addresses of command lists that are to be referenced with set_sub_pointer can be queried by get_input_pointer <i>before</i> loading the command lists. • set_sub_pointer only stores <i>starting addresses</i> in the internal management table. An indexed subroutine only gains protection by a subsequent save_disk/load_disk. • <code>Pos</code> should not be an arbitrary address within a list. Instead, it should be the starting address of an actually existing subroutine that has been finalized by list_return and does not contain set_end_of_list.
RTC4→RTC5	New command.
Version info	–
References	load_sub

Ctrl Command	set_text_table_pointer
Function	Stores the absolute start address of a command list in the internal management table for indexed text strings.
Call	<code>set_text_table_pointer(Index, Pos)</code>
Parameters	Index Index of the indexed text string whose starting address Pos should be entered in the management table. As an unsigned 32-bit value. Allowed value range: [0...41]. The same ordering applies as for load_text_table: = 0...9: Digits for marking the time and date [0...9]. = 10...21: Months [January...December]. = 22...28: Days-of-the-week [Sunday...Saturday]. = 29: Blank character for marking serial numbers. = 30...39: Digits for marking serial numbers [0...9]. = 40: Text for "a.m.". = 41: Text for "p.m.".
	Pos Absolute start address. As an unsigned 32-bit value. Allowed value range: [0...(2²⁰-1)].
Comments	• Indexed text strings can be used for marking time, date or serial numbers (see Section "Calling Indexed Text Strings", Page 109). • If Index > 41 and/or Pos > (2²⁰-1), then set_text_table_pointer is <i>not</i> executed (get_last_error return code RTC5_PARAM_ERROR). • set_text_table_pointer can be used for referencing a nonindexed subroutine, which thereby becomes an indexed text string that is protectable by save_disk/load_disk and/or callable by the index. • set_text_table_pointer can also be used to reference anew an indexed subroutine, character or text string so that it can also be called by a second index. Here, it is preferable to use the copy_dst_src command for index management. • The start addresses of command lists that are to be referenced with set_text_table_pointer can be queried by get_input_pointer <i>before</i> loading the command lists. • set_text_table_pointer only stores <i>starting addresses</i> in the internal management table. An indexed text string only gains protection by a subsequent save_disk/load_disk. • Pos should not be an arbitrary address within a list. Instead, it should be the starting address of an actually existing subroutine that has been finalized by list_return and does not contain set_end_of_list.
RTC4→RTC5	New command. set_text_table_pointer is synonymous with set_char_table, which is available for the RTC SCANalone Board (standalone version of the RTC4 board).
Version info	–
References	load_text_table , mark_date , mark_serial , mark_time

Delayed Short List Command	set_trigger
Function	Starts measurement value recording of the specified signals.
Call	<code>set_trigger(Period, Signal1, Signal2)</code>
Parameters	Period Measurement period (time interval between 2 data pair recordings). As an unsigned 32-bit value. 1 bit equals 10 μs. Allowed value range: $[0 \dots (2^{31}-1)]$.
	Signal1 Signal type for measurement channel 1. As an unsigned 32-bit value. Allowed value range: see below. 0 LASERON signal 1 = laser control signal on. 0 = laser control signal off. 1 StatusAX Status signal of x axis, first scan head connector. 2 StatusAY Status signal of y axis, first scan head connector. 3 Reserved. 4 StatusBX Status signal of x axis, second scan head connector. 5 StatusBY Status signal of y axis, second scan head connector. 6 Reserved. 7 SampleX Cartesian control value for the x axis (= up to #4 in 7.3.6, Page 170). 8 SampleY Cartesian control value for the y axis (= up to #4 in 7.3.6, Page 170). 9 SampleZ Cartesian control value for the z axis (= up to #4 in 7.3.6, Page 170).

Delayed Short List Command	set_trigger		
Parameters (cont'd)	Signal1 (cont'd)	10	SampleAX_Corr Korrigierter Ansteuerwert für die x axis, erster scan head-Anschluss. (= bis #7 in 7.3.6, Page 170). 11 SampleAY_Corr Korrigierter Ansteuerwert für die y axis, erster scan head-Anschluss (= bis #7 in 7.3.6, Page 170). 12 SampleAZ_Corr Korrigierter Ansteuerwert für die z axis, wenn xy am ersten scan head-Anschluss angeschlossen sind (= bis #10 in 7.3.6, Page 170). Identisch zum tatsächlichen Ausgabewert für die z axis. 13 SampleBX_Corr Korrigierter Ansteuerwert für die x axis, zweiter scan head-Anschluss (= bis #7 in 7.3.6, Page 170). 14 SampleBY_Corr Korrigierter Ansteuerwert für die y axis, zweiter scan head-Anschluss (= bis #7 in 7.3.6, Page 170). 15 SampleBZ_Corr Korrigierter Ansteuerwert für die z axis, wenn xy am zweiten scan head-Anschluss angeschlossen sind (= bis #10 in 7.3.6, Page 170). Identisch zum tatsächlichen Ausgabewert für die z axis. 16 StatusAX+LASERON signal StatusAX wenn laser control signal an. –524,288 wenn laser control signal aus. 17 StatusAY+LASERON signal StatusAY wenn laser control signal an. –524,288 wenn laser control signal aus. 18 StatusBX+LASERON signal StatusBX wenn laser control signal an. –524,288 wenn laser control signal aus. 19 StatusBY+LASERON signal StatusBY wenn laser control signal an. –524,288 wenn laser control signal aus.

Delayed Short List Command	set_trigger		
Parameters (cont'd)	Signal1 (cont'd)	20	SampleAX_Out Tatsächlicher Ausgabewert für die x axis, erster scan head-Anschluss (= bis #11 in 7.3.6, Page 170). Nicht für z axisn-Ausgabewerte verwendbar.
		21	SampleAY_Out Tatsächlicher Ausgabewert für die y axis, erster scan head-Anschluss (= bis #11 in 7.3.6, Page 170). Nicht für z axisn-Ausgabewerte verwendbar.
		22	SampleBX_Out Tatsächlicher Ausgabewert für die x axis, zweiter scan head-Anschluss (= bis #11 in 7.3.6, Page 170). Nicht für z axisn-Ausgabewerte verwendbar.
		23	SampleBY_Out Tatsächlicher Ausgabewert für die y axis, zweiter scan head-Anschluss (= bis #11 in 7.3.6, Page 170). Nicht für z axisn-Ausgabewerte verwendbar.
		24	Lasersteuerparameter der "Automatischen Laser-Steuerung" Siehe set_auto_laser_control.
		25	SampleAX_Trans Transformierter Ansteuerwert für die x axis, erster scan head-Anschluss (= bis #6 in 7.3.6, Page 170).
		26	SampleAY_Trans Transformierter Ansteuerwert für die y axis, erster scan head-Anschluss (= bis #6 in 7.3.6, Page 170).
		27	SampleAZ_Trans Transformierter Ansteuerwert für die z axis, wenn xy am ersten scan head-Anschluss angeschlossen sind (= bis #6 in 7.3.6, Page 170).
		28	SampleBX_Trans Transformierter Ansteuerwert für die x axis, zweiter scan head-Anschluss (= bis #6 in 7.3.6, Page 170).
		29	SampleBY_Trans Transformierter Ansteuerwert für die y axis, zweiter scan head-Anschluss (= bis #6 in 7.3.6, Page 170).
		30	SampleBZ_Trans Transformierter Ansteuerwert für die z axis, wenn xy am zweiten scan head-Anschluss angeschlossen sind (= bis #6 in 7.3.6, Page 170).

Delayed Short List Command	set_trigger		
Parameters (cont'd)	Signal1 (cont'd)	31	Laser control parameter of "vector-controlled laser control" See set_vector_control .
		32	Focus shift See set_vector_control , set_defocus , set_defocus_list .
		33	12-bit output value at the ANALOG OUT1 output port See set_vector_control and Chapter 9.1.4 "12-Bit Analog Output Port 1 and 2" , Page 259 .
		34	12-bit output value at the ANALOG OUT2 output port See set_vector_control and Chapter 9.1.4 "12-Bit Analog Output Port 1 and 2" , Page 259 .
		35	Output value at the 16-bit digital output port See set_vector_control and Chapter 9.1.1 "16-Bit Digital Output Port" , Page 258 .
		36	Output value at the 8-bit digital output port See set_vector_control and Chapter 9.1.2 "8-Bit Digital Output Port" , Page 259 .
		37	Pulse length (PulseLength) of the Laser Control Signals LASER1 and LASER2 See set_vector_control .
		38	Output period (HalfPeriod) of the Laser Control Signals LASER1 and LASER2 See set_vector_control .
		39	FreeVariable0
		40	FreeVariable1
		41	FreeVariable2
		42	FreeVariable3

Delayed Short List Command	set_trigger		
Parameters (cont'd)	Signal1 (cont'd)	43	Counter value of encoder counter "Encoder0"
		44	Counter value of encoder counter "Encoder1"
		45	Marking speed From set_mark_speed , set_mark_speed_ctrl .
		46	16-bit digital input port (EXTENSION 1)
		47	Only for intelliWELD II: Zoom value
		48	FreeVariable4
		49	FreeVariable5
		50	FreeVariable6
		51	FreeVariable7
		52	32-bit "Timestamp Counter" See Chapter 8.12 "Time Measurements" , Page 257.
		53	Wobbel amplitude See set_wobbel , set_wobbel_mode and Chapter 8.4 "Wobbel Mode" , Page 217.
		54	ReadAnalogIn See read_analog_in .
	Signal2	Like (analogously) Signal1.	

Delayed Short List Command	set_trigger
Comments	<i>General Comments</i> • If Signal1 and Signal2 are not from the above list, then set_trigger (except for Period = 0) is replaced with a list_nop (get_last_error return code RTC5_PARAM_ERROR). • After set_trigger with Period > 0, a data pair is stored immediately and subsequently at intervals defined by the specified measurement period. • The measurement value recording ends automatically with 2^{20} data pairs. It ends earlier, if one of the following occurs: - set_trigger (Period = 0) is called - set_trigger4(Period = 0) is called • If set_trigger has been preceded by a load_correction_file which loaded a file as correction table No=3 or No=4, then the memory area's upper half reserved for data recording by set_trigger is already occupied. Then, only 2^{19} data pairs are recordable with set_trigger and measurement value recording automatically terminates at this value. If loading of correction table No=3 or No=4 occurs after more than 2^{19} data pairs had been recorded by set_trigger, then the values beyond 2^{19} are no longer available. load_program_file reestablishes the original state of up to 2^{20} possible measurement pairs. See also number_of_correction_tables(2). • A measurement value recording started by set_trigger occurs only during the execution of a list. • Given the measurement value recording has not been terminated by set_trigger (Period = 0) or set_trigger4(Period = 0), it is automatically continued with the start of a new list (the status of the measurement value recording is not reset by set_end_of_list). During the execution of a list, the status is reset by set_trigger(Period = 0), set_trigger4(Period = 0) or stop_execution. By stop_trigger the status can be reset if no list is executed. • Sample values and status values returned by the scan system and stored on the RTC5 by set_trigger (Signal1, Signal2 = 1-23, 25-30) are always in the RTC5 20-bit range, even if the RTC5 DLL is set to RTC4 Compatibility Mode. The measured and stored values can be subsequently transferred to the PC by the get_waveform command for evaluation. It is generally recommended to end the measurement explicitly before reading out the data. The measured values are transferred to the PC by get_waveform as 32-bit values (see comments for get_value). • The current status of the measurement value recording can be queried by measurement_status. • For aborting a measurement session by set_trigger(Period = 0) or set_trigger4(Period = 0), the values of Signal1 and Signal2 are irrelevant.

Delayed Short List Command	set_trigger
Comments (cont'd)	- If you abort a measurement session with set_trigger(Period = 0) or set_trigger4(Period = 0), then previously recorded measurement values are <i>not</i> lost and the measurement counter halts at its most recent value. This allows subsequent querying by measurement_status of the number of successful data entries. In contrast, if a measurement session is newly started with set_trigger and Period > 0 or set_trigger4(Period > 0), the measurement counter is reset and the measurements obtained thus far are overwritten. It is not possible to resume an explicitly or automatically halted measurement session. <i>Comments on the Data</i> - The type of scan system being used determines which status signals are generated and returned by the status channels. Specific information can be found in your scan system's operating manual. control_command can be used with iDRIVE scan systems (see Glossary entry on page 23) to specify which information is returned on the status channels. - Only X measurement signals contain meaningful data with single-status channel scan systems. - With 3D scan systems, the output and status values of the z axis are transmitted over the scan head connector's channel to which the z axis is attached (if correspondingly configured by select_cor_table). Example: If the z axis is attached to the second scan head connector's x channel (select_cor_table(HeadA, 0)), then its status signal can be queried by StatusBX (Signal1/2 = 4). - The signals 12 and 15 as well as 27 and 30 are identical, each: SampleAZ_Corr = SampleBZ_Corr and SampleAZ_Trans = SampleBZ_Trans. - Status signals Status<...> are a few 10 µs clock cycles later than control signals Sample<...>. See also Section "Reading Out Data", Page 199. - Signal1, Signal2 = 0, 24, 31...42 and 47...51 enables logging of values that were outputted during the previous clock cycle. Data recording of different signals is not synchronous because of the internal processing chain. Depending on how the signals are combined, you must later correct the data by a fixed time offset. - Signal1, Signal2 = 24 enables logging of the signal parameter that is outputted by "Automatic Laser Control" (but not with vector-defined laser control; see set_auto_laser_control). Logging only occurs when "Automatic Laser Control" has been activated and the laser is on. When the laser is switched off, 0 is recorded. - Signal1, Signal2 = 31 enables logging of the signal parameter specified for vector-defined laser control (see set_vector_control). Logging is also possible without executing [*]para[*] Commands, but then the signal values remain unchanged.

Delayed Short List Command	set_trigger
Comments (cont'd)	- Pulse length and output period (Signal1, Signal2 = 37, 38) are only logged for Signals for "Laser Active" Operation (not for standby signals). In Softstart Mode and in Pixel Output Mode, the pulse length and output period might possibly change faster than the 10 μs clock cycle (time resolution of logging by set_trigger). These values cannot be recorded. - For Signal1, Signal2 = 39...42 and 48...51, see Chapter 6.9.1 "Free Variables", Page 123.
RTC4→RTC5	Basically unchanged functionality. However: - The measurement session can be aborted by <code>Period = 0</code>. - The signals <code>StatusAZ</code> and <code>StatusBZ</code> (Signal1/Signal2 = 3, 6) are no longer available. - All coordinate values are referred to (0 0 0) (value range $[-2^{19}...(2^{19}-1)]$) and are no longer zero point shifted.
Version info	Last change DLL 543, OUT 543, RBF 524: Signal1, Signal2 = 52...54.
References	get_waveform, measurement_status, control_command, get_value, get_values, set_mcbasp_out, set_mcbasp_out_ptr, set_trigger4

Delayed Short List Command	set_trigger4
Function	Starts the measurement value recording of the specified measurement signals (has the same effect as set_trigger , but with up to 4 simultaneous measurement signals).
Call	set_trigger4(Period, Signal1, Signal2, Signal3, Signal4)
Parameters	Period Like set_trigger .
	Signal1 Like set_trigger .
	Signal2 Like set_trigger .
	Signal3 Like set_trigger .
	Signal4 Like set_trigger .
Comments	• See comments for set_trigger. • Unlike set_trigger, only 2^{19} entries per measurement channel are available with set_trigger4 (automatic termination upon the maximum 2^{19} data quadruplet). • set_trigger(Period = 0) and set_trigger4(Period = 0) both terminate active data recording regardless of which command started the data recording. If set_trigger(Period > 0) or set_trigger4(Period > 0) is used to start a new measurement, then the measurement value counters are reset and existing data overwritten with new measurement values. See also related comments for set_trigger. • Data channels 3 and 4 occupy the measurement memory's upper half, as do correction files No=3 and No=4. Therefore, both options cannot be used simultaneously. If set_trigger4 has been preceded by loading correction file No=3 or No=4 with load_correction_file, then measurement values overwrite this area and the correction file is no longer available. If loading of correction file No=3 or No=4 succeeded after any data quadruplets were recorded with set_trigger4 (or more than 2^{19} data pairs with set_trigger), then the values for channels 3 and 4 (or beyond 2^{19} data pairs) are no longer available. • load_program_file reestablishes the initial state of measurement memory. See also number_of_correction_tables(2). • As with set_trigger, get_waveform can be used to transfer recorded values to the PC.
RTC4→RTC5	New command.
Version info	Last change DLL 543, OUT 543, RBF 524: Signal1, Signal2 = 52...54.
References	set_trigger , stop_trigger , get_waveform

Undelayed Short List Command	set_vector_control	
Function	Initializes or deactivates vector-defined laser control (see page 194).	
Call	set_vector_control(Ctrl, Value)	
Parameters	Ctrl	Control parameter for initializing or deactivating vector-defined laser control (for Ctrl = 1...6: identical with set_auto_laser_control). As an unsigned 32-bit value. = 1...8: Defines which signal parameter is to be varied by vector-defined laser control. = 1: 12-bit output value at the ANALOG OUT1 output port. = 2: 12-bit output value at the ANALOG OUT2 output port. = 3: Output value at the 8-bit digital output port. = 4: Pulse length (PulseLength) of the laser signals LASER1 and LASER2. = 5: Output period (HalfPeriod) of the laser signals LASER1 and LASER2. = 6: Output value at the 16-bit digital output. = 7: Focus shift ("Defocus"). = 8: Reserved for intelliWELD. = 0 or > 8: deactivates vector-defined laser control.
	Value	Defines the starting value for the parameter selected by Ctrl. As an unsigned 32-bit value. Allowed values (out-of-range values are clipped to the boundary values): - For Ctrl = 1/2: 12-bit values [0...4095]. - For Ctrl = 3: 8-bit values [0...255]. - For Ctrl = 4: [0...(2³²-1)]. 1 bit equals 1/64 μs. - For Ctrl = 5: [0...(2³²-1)]. 1 bit equals 1/64 μs. - For Ctrl = 6: 16-bit values [0...(2¹⁶-1)]. - For Ctrl = 7: [0...65,535], Value = focus shift + 32,768.
Comments	- If Ctrl is an invalid value, then "vector-controlled laser control" is deactivated (Ctrl = 0, initialization state). Otherwise, Value is applied as the starting value of the next [*]para[*] Command. If Value is invalid, then it is clipped to the maximum allowed value. - set_vector_control only affects [*]para[*] Commands. - If an "Automatic Laser Control" has been activated by set_auto_laser_control with the same control parameter (Ctrl = 1...6), then set_vector_control sets the 100% value of the "Automatic Laser Control" to value Value.	

Undelayed Short List Command	set_vector_control
Comments (cont'd)	- For <code>Ctrl = 7</code>, the focus shift is linearly varied as with set_defocus and set_defocus_list, see comments there) with [*]para[*] Commands (in order for the z outputs to be outputted, the Option "3D" must be enabled and a 3D correction file must be loaded and assigned as well) up to the specified end value (this requires enabling the Option "3D" as well as loading and assigning a 3D correction file). If, for the first [*]para[*] Command, the currently set focus shift does not match the initial value (Value) specified by set_vector_control, then the command begins with a "Hard jump" (in the z output) to Value. The setting defined by set_defocus or set_defocus_list is lost. - Unlike signed values in the range $[-32,768...+32,767]$ for set_defocus and set_defocus_list, the focus shift (Value) specified for set_vector_control must be an unsigned number shifted upward by 32,768: $\text{Value} = \text{focus shift} + 32,768$. "Automatic Laser Control" should not be combined with the variable laser power of set_multi_mcbp_in or the "freely definable wobble shape".
RTC4→RTC5	New command. set_vector_control has no RTC4 Compatibility Mode for Value.
Version info	Last change DLL 538: <code>Ctrl = 8</code> .
References	set_auto_laser_control

Ctrl Command	set_verify
Function	Activates or deactivates a download verification, see Chapter 6.8.1 "Download Verification", Page 120 .
Call	OldVerify = set_verify(Verify)
Parameters	Verify Setting parameter. As an unsigned 32-bit value. = 0: Verification is deactivated. > 0: Verification is activated.
Result	The Verify parameter that has been active before calling set_verify . As an unsigned 32-bit value.
Comments	• If verification is activated, the download times are extended. • Complete download verification as described in Chapter 6.8.1 "Download Verification", Page 120 requires RTC5 board driver V1.0.4.0 or higher. > DLL 511 used in conjunction with an older driver version, then it returns error code 9 (DriverCall error) during load_program_file. • Verification of correction file downloads only works if the correction file contains a checksum (otherwise the error code RTC5_VERIFY_ERROR is generated). To ensure that a checksum is present (which is not the case for older ct5 correction files), you should test your correction file prior to loading by using the control command verify_checksum. If the correction file does not contain a checksum, then verify_checksum creates and inserts a checksum into the file. • set_verify is available even without explicit access rights to a specific RTC5 board. set_verify only change settings in the RTC5 DLL of the specified (or default) board. It has no effect on the board itself. • The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by set_verify. • Activation of the download verification by set_verify can generate the get_last_error return code RTC5_OUT_OF_MEMORY, if a RTC5 DLL-internal Windows memory request fails. In this case, the download verification remains deactivated.
RTC4→RTC5	New command.
Version info	–
References	get_error , get_last_error , verify_checksum

Normal List Command	set_wait
Function	Sets a numbered wait marker (break point) in the list.
Call	<code>set_wait(WaitWord)</code>
Parameters	WaitWord Number of the break point. As an unsigned 32-bit value. Allowed value range: $[1 \dots (2^{32}-1)]$. 0 is corrected to 1.
Comments	• The list processing is interrupted at each break point and remains halted until continued by release_wait. This provides a way to implement synchronizations. • The Signals for "Laser Active" Operation are switched off by set_wait and a home jump defined by home_position or home_position_xyz might be executed (the INTERNAL-BUSY list execution status is set while the home jump is executed). Continuation by execute_list_pos, restart_list or an External Stop is consequently suppressed. However, stop_execution or an External Stop is possible. release_wait, stop_execution or an External Stop removes suppression of the start. • set_wait sets the PAUSED list execution status and resets the BUSY list execution status (both queriable with get_status). The opposite occurs with a subsequent release_wait (see also Chapter 6.4.3 "List Execution Status", Page 97). • If processing has been stopped at a wait marker, get_wait_status returns the number of this marker.
RTC4→RTC5	Basically unchanged functionality. However: Additional PAUSED list execution status which is set by set_wait .
Version info	–
References	get_wait_status , release_wait , pause_list , stop_list

Delayed Short List Command	set_wobbel	
Function	Defines the parameters for an ellipse-shaped wobble motion. "Wobble" is a motion of the output position which is added to the regular marking motion. See Chapter 8.4 "Wobble Mode", Page 217 .	
Call	set_wobbel(Transversal, Longitudinal, Freq)	
Parameters	Transversal	Amplitude of the elliptical motion <i>perpendicular</i> to the momentary direction of motion or to the one defined by set_wobbel_direction . In bits. As an unsigned 32-bit value. Allowed value range: [0...131.071 ($=2^{17}-1$)]. Larger values are clipped.
	Longitudinal	Amplitude of the elliptical or figure-8 motion <i>parallel</i> to the momentary direction of motion or to the one defined by set_wobbel_direction . In bits. As an unsigned 32-bit value. Allowed value range: [0...131.071 ($=2^{17}-1$)]. Larger values are clipped.
	Freq	Frequency of the wobble motion. In Hz. (number of ellipses per second). As a 64-bit IEEE floating point value. Allowed value range: [-6000...6000]. Larger values are clipped.
Comments	- The Wobble mode can be used for marking lines with various line widths. An ellipse-shaped motion is added to the linear marking motion, resulting in a spiral motion of the laser focus in the Image field. Alternatively, a figure-of-8 wobble shape (horizontal or vertical to the direction of motion) can be activated by set_wobbel_mode. The line width can be set by appropriate values for the amplitude and frequency (frequency and mark speed should be coordinated, see Chapter 8.4.1 "Wobble Shapes – Important Notes on Choosing Appropriate Parameter Values", Page 219). - For arcs, too, the wobble motion follows the current marking direction (exception: see below). Therefore, independently of the Cartesian angle, the effective line width is always the same. - The Wobble mode is terminated by setting both amplitudes and/or the frequency to zero (more accurate for $-n \leq \text{Freq} \leq n$ with $n = 50000/65536 = 0.7629\dots$). - The frequency is signed. The wobble vector rotates clockwise for positive values and counterclockwise for negative values. Thus, the inner and outer point densities of arcs can be individually set independently of their actual direction of rotation. - At the beginning of a marking, (after set_wobbel or a jump), the wobble start point is generally set for the same value relative to the vector/arc startpoint; it is repeatedly continued, however, for Polylines (including arcs).	

Delayed Short List Command	set_wobbel
Comments (cont'd)	• For identical amplitudes ($\text{Longitudinal} = \text{Transversal}$), the wobbel startpoint is permanently referenced to the coordinate system, that is, independent of the current direction of motion. By set_wobbel_direction, a fixed reference direction can be set which differs from it. • For an angle, ellipse-shaped wobbel motions can result in more or less small jumps after reaching the corner, depending on the current wobbel phase. This does not occur (= "rounded corners") if the wobbel motion is exactly circular ($\text{Longitudinal} = \text{Transversal}$). • $\text{Longitudinal} = 0$ produces a sine-shaped wobbel motion across the direction of motion. • When defining the wobbel shape and its frequency take the dynamics of the scan head and laser into account. Otherwise, an overheating and even a permanent damage of the system may occur, see Chapter 8.4.1 "Wobbel Shapes – Important Notes on Choosing Appropriate Parameter Values", Page 219. • As of version DLL 543, OUT 543, RBF 524 the present wobbel amplitude can be recorded with trigger signal 53 (see set_trigger or set_trigger4). The format of the data is $((\text{transversal} < 16) + \text{longitudinal})$.
RTC4→RTC5	Basically unchanged functionality. More wobbel shape: elliptical, direction dependent adjustable, see also set_wobbel_mode. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for Longitudinal and Transversal by 16. The allowed value ranges decrease accordingly.
Version info	–
References	set_mark_speed , set_wobbel_mode

Undelayed Short List Command	set_wobbel_control
Function	Specifies laser control parameters for laser power variation with “freely definable wobbel shapes”.
Call	<code>set_wobbel_control(Ctrl, Value, MinValue, MaxValue)</code>
Parameters	Ctrl Control parameter for initializing or deactivating laser power variation. As an unsigned 32-bit value. = 1...6: Defines which signal parameter to vary. On the meaning, see set_auto_laser_control. = 0 or > 6: Deactivates laser power variation (for Ctrl > 6: get_last_error return code <code>RTC5_PARAM_ERROR</code>). Value Nominal laser power P0 (100%). As an unsigned 32-bit value. Allowed value range: see set_auto_laser_control; the maximum is [0...65,535] or 0xFFFFFFFF. Excessive values are clipped. MinValue Limit that cannot be exceeded, see set_auto_laser_control. As an unsigned 32-bit value. The allowed value range depends on the selected Ctrl parameter and on Value. MaxValue See MinValue.
Comments	- Any still-pending delayed short list command executes first. - For Ctrl = 0 and Ctrl > 6, laser power variation is switched off (initialization state after load_program_file). The values for McBSP/SPI multi transfers are then not used. - The conditions for Value, MinValue, MaxValue are the same as for “Automatic Laser Control” (see set_auto_laser_control), but with a maximum of [0...65,535]. Initialized values are MinValue = 0 and MaxValue = 0xFFFFFFFF, that is, no restrictions. - The laser power can be combined with the vector-defined laser control, if the corresponding Ctrl parameter has been chosen identically. - If you want variable laser power along a wobbel shape, then set_wobbel_control must execute before you activate the “freely definable wobbel shape”. Otherwise, laser power is not varied, even if you make a change in the data sets. - Special case: if Value = 0xFFFFFFFF, then the nominal laser power is derived from the port assigned by the signal parameter Ctrl, instead of from the command. This is then the current content at this point in time set by other normal commands, for example, write_da_1_list for Ctrl = 1. These are executed prior to set_wobbel_control. But beware: with execution of a “freely definable wobbel shape”, the value at the port can change at any time. Subsequent calls of set_wobbel_control then return other values for the nominal laser power.

Undelayed Short List Command	set_wobbel_control
RTC4→RTC5	New command.
Version info	–
References	set_wobbel_vector , set_multi_mcbasp_in , set_multi_mcbasp_in_list

Undelayed Short List Command	set_wobbel_direction
Function	Defines a direction vector for wobbel motions.
Call	<code>set_wobbel_direction(dX, dY)</code>
Parameters	dX x component of the direction vector. In bits. As a signed 32-bit value.
	dY y component of the direction vector. In bits. As a signed 32-bit value.
Comments	• For non-zero direction vectors (dX and/or dY non-zero), wobbel motion (longitudinal and transversal) is relative to <i>this</i> direction vector instead of to the momentary direction vector. The direction vector's length is inconsequential. The RTC5 normalizes the direction vector. • If dX = dY = 0, then the function is deactivated. • The direction vector setting remains in effect even after the wobbel mode is switched off by set_wobbel or set_wobbel_mode, and continues being used if you switch the wobbel mode back on.
RTC4→RTC5	New command.
Version info	–
References	set_wobbel , set_wobbel_direction , set_wobbel_mode

Delayed Short List Command	set_wobbel_mode
Function	Switches the Wobbel mode on and off and defines the parameters for an ellipse-shaped or figure-of-8 wobbel motion, see Chapter 8.4 “Wobbel Mode”, Page 217 .
Call	<code>set_wobbel_mode(Transversal, Longitudinal, Freq, Mode)</code>
Parameters	Transversal Amplitude of the elliptical or figure-8 motion <i>perpendicular</i> to the momentary direction of motion or to the one defined by set_wobbel_direction . In bits. As an unsigned 32-bit value. Allowed value range: $[0 \dots 131.071 (=2^{17}-1)]$. Excessive values are clipped.
	Longitudinal Amplitude of the elliptical or figure-8 motion <i>parallel</i> to the momentary direction of motion or to the one defined by set_wobbel_direction . In bits. As an unsigned 32-bit value. Allowed value range: $[0 \dots 131.071 (=2^{17}-1)]$. Excessive values are clipped.
	Freq Frequency of the wobbel motion in <i>Hz</i> (number of ellipses or figure-of-8s per second). As a 64-bit IEEE floating point value. Allowed value range: $[-6000 \dots 6000]$. Larger values are clipped.
	Mode Defines the wobbel shape. As a signed 32-bit value. = 0: Ellipse-shaped wobbel motion. < 0: Figure-of-8 wobbel motion perpendicular to the motion direction (vertical 8). = 1: Figure-of-8 wobbel motion parallel to the motion direction (horizontal 8). > 1: “Freely definable wobbel shape”, see set_wobbel_vector .
Comments	• If <i>Mode</i> = 0, set_wobbel_mode functions identically to set_wobbel (see comments there). • If <i>Mode</i> $\neq$ 0, then a figure-of-8 motion is activated. With figure-of-8s, broader mid-line processing can be achieved by appropriate parameter values. If the specified transverse and longitudinal amplitudes are identical, then the wobbel shape remains stationary in space; otherwise the orientation of the wobbel shape follows the current direction of motion. If set_wobbel_mode executes while a list contains a “freely definable wobbel shape” (see set_wobbel_vector and Chapter 8.4 “Wobbel Mode”, Page 217), then <i>Mode</i> > 1 selects this shape, otherwise the horizontal figure-8 shape is selected similarly to <i>Mode</i> = 1. If you <i>subsequently</i> define a wobbel shape and the Wobbel mode has been activated with the parameter <i>Mode</i> > 1, then a switch from the horizontal figure-8 to the wobbel shape automatically occurs. But beware: it is then not possible to vary the power along the wobbel figure, see set_wobbel_control.

Delayed Short List Command	set_wobbel_mode
Comments (cont'd)	- The wobbel mode is terminated by setting both amplitudes and/or the frequency to zero (more accurate for $-n \leq \text{Freq} \leq n$ with $n = 50000/65536 = 0.7629\dots$). - The frequency is signed. During the figure-of-8's first loop, the wobbel vector rotates clockwise for positive values and counterclockwise for negative values. Thus, (especially for ellipse-shaped wobbel motions), the inner and outer point densities of arcs can be individually set independently of their actual direction of rotation. - At the beginning of a marking, (after set_wobbel or a jump), the wobbel start point is generally set for the same value relative to the vector/arc startpoint; it is repeatedly continued, however, for Polyline (including arcs). For identical amplitudes ($\text{Longitudinal} = \text{Transversal}$), the wobbel startpoint is permanently referenced to the coordinate system, that is, independent of the current direction of motion. - For an angle, elliptical or figure-8 wobbel motions can result in more or less small jumps after reaching the corner, depending on the current wobbel phase. This does not occur (= "rounded corners") if the wobbel motion is exactly circular ($\text{Longitudinal} = \text{Transversal}$). $\text{Longitudinal} = 0$ produces a sine-shaped wobbel motion across the direction of motion. - For "freely definable wobbel shapes", the parameter Freq has no meaning. It must nevertheless lie within the valid range, because otherwise the Wobbel mode gets deactivated. This also applies to the parameters Transversal and Longitudinal. - If the Wobbel mode gets deactivated, then any already-defined wobbel shape remains active and is used upon the next switch-on of the Wobbel mode with $\text{Mode} > 1$. - When defining the wobbel shape and its frequency take the dynamics of the scan head and laser into account. Otherwise, an overheating and even a permanent damage of the system may occur, see Chapter 8.4.1 "Wobbel Shapes – Important Notes on Choosing Appropriate Parameter Values", Page 219. - As of version DLL 543, OUT 543, RBF 524 the present wobbel amplitude can be recorded with trigger signal 53 (see set_trigger or set_trigger4). The format of the data is $((\text{transversal} << 16) + \text{longitudinal})$. - The wobbel mode cannot be combined with: Sky Writing Pixel Output Mode Jumps laser_on_list
RTC4→RTC5	New command. In RTC4 Compatibility Mode, the RTC5 multiplies the specified value for Longitudinal and Transversal by 16. The allowed value ranges decrease accordingly.
Version info	–
References	set_wobbel , set_wobbel_control , set_wobbel_vector

Delayed Short List Command	set_wobbel_offset
Function	Defines a wobbel shape shift in the direction of motion or perpendicular to the direction of motion.
Call	<code>set_wobbel_offset(OffsetTrans, OffsetLong)</code>
Parameters	OffsetTrans Transversal offset. As a signed 32-bit value. Allowed value range: $\pm 32,767$. Larger values are clipped. Initialization values after load_program_file : (0,0).
	OffsetLong Longitudinal offset. Otherwise, like OffsetTrans.
Comments	• Offsets can be defined for "classic" wobbel shapes (circle, ellipse, sine, figure-8) as well as for "freely definable wobbel shapes", see Chapter 8.4 "Wobbel Mode", Page 217. • The summed up wobbel amplitude including offsets may never exceed $\pm(2^{17}-1)$. • At the beginning of the wobbel marking offsets are set as "Hard jumps". This applies also in case of switching-off or non-using the Wobbel mode, for example, when using a Jump command (analogously to "classical" wobbel shapes longitudinal amplitudes).
RTC4→RTC5	New command. In RTC4 Compatibility Mode the RTC5 multiplies the specified values for OffsetTrans and OffsetLong by 16. The allowed value ranges decrease accordingly.
Version info	–
References	set_wobbel_mode , set_wobbel_vector , set_wobbel_direction

Undelayed Short List Command	set_wobbel_vector
Function	Defines a linear section of a wobbel shape.
Call	<code>set_wobbel_vector(dTrans, dLong, Period, dPower)</code>
Parameters	dTrans Microstep of a linear wobbel shape section. In bits. dLong As a 64-bit IEEE floating point value. Allowed value range: [-256.0...+255.0]. dTrans is the wobbel excursion perpendicular to the direction of motion (which is either the laser Trajectory or the direction of motion defined by set_wobbel_direction). dLong is correspondingly longitudinal to it. Period As an unsigned 32-bit value. Allowed value range: [0...65,535]. = 1...65,535: number of Microsteps. = 0: The wobbel shape is switched off. dPower Microstep of the relative laser power. As a 64-bit IEEE floating point value. Allowed value range: [-1.0...+1.0]. Out-of-range values are clipped to the boundary values.
Comments	set_wobbel_vector cannot be used together with the Softstart function because both require a shared memory area on the RTC5. To avoid (unintentional) mutual overwriting, you absolutely must explicitly deactivate Softstart. Otherwise, set_wobbel_vector has no effect. set_wobbel_vector itself blocks subsequent memory changes by set_softstart_level or set_softstart_level_list. These commands have no effect as long as the wobbel shape has not been explicitly switched off. Period = 0 is not allowed as a shape section. Period = 0 means explicitly switch off the “freely definable wobbel shape” (not the wobbel mode itself, see set_wobbel_mode). Subsequent calls of set_wobbel_vector begin a new wobbel shape. - Each call of set_wobbel_vector adds a new section at the end of the previous section independently of when the call is performed. - Up to 512 wobbel shape sections can be defined. After 512 sections, storage automatically wraps around to the first section and overwrites it. Each further call does the same to the next section. - The wobbel shape automatically begins with wobbel vector (0,0), that is, directly on the marking itself (the “Hard jump” of “classic” wobbel shapes is eliminated if the longitudinal amplitude $\neq 0$) and also ends there without requiring explicit declaration.

Undelayed Short List Command	<code>set_wobbel_vector</code>
Comments (cont'd)	- Each wobbel shape section consists of a vector defined by Microsteps and their number. The parameters <code>dTrans</code>, <code>dLong</code> get internally rounded to 7 bit decimal places. At runtime each Microsteps is executed within 10 μs. For large <code>Period</code> values, you should therefore take rounding error into account. Positive <code>dTrans</code> values cause that in respect to the direction of motion the start is to the right (which applies to circular wobbel shapes as well). Positive <code>dLong</code> values cause that in respect to the direction of motion the start is forward. The last section's endpoint is also the first section's start point. Independently of its respective position, it always ends at (0,0) and might get executed as a "Hard jump". With the last step the laser power is reset to the initial nominal laser power. - The summed up wobbel amplitude may never exceed $\pm(2^{17}-1)$. - If activated by set_wobbel_control, the RTC5 varies the current laser power <code>P</code> within the wobbel shape section as per $P = P_0 \times (1 + dPower \times n),$ whereby $0 \leq n \leq Period$. <code>n</code> represents the current number of each Microstep. You can define the nominal laser power <code>P0</code> with the list command set_wobbel_control. - If activated by set_multi_mcbbsp_in or set_multi_mcbbsp_in_list, the nominal laser power <code>P0</code> is multiplicatively varied by the laser power parameter <code>P</code> across multiple McBSP/SPI transfers: $P_{McBSP} = P_0 \times P / 16384$. - Beware here that for each both corrections above a maximum laser power of only $P_0 \times 4.0$ is allowed, whereby the maximum range of the laser control parameter likewise must not be exceeded (see set_wobbel_control). - For laser power variation with <code>Ctrl = 5</code> (half period), <code>dPower</code> needs to have the opposite sign. Unlike with "Automatic Laser Control", the multiplication factor cannot be inverted. - By using an "empty" wobbel shape <code>set_wobbel_vector(0.0, 0.0, 1, 0.0)</code>, you can also multiplicatively vary the nominal laser power <code>P0</code> without needing to explicitly wobbel (for another alternative, see set_multi_mcbbsp_in). - When you switch off the wobbel shape (not by deactivation by set_wobbel_mode), the nominal laser power <code>P0</code> is emitted at the port assigned by <code>Ctrl</code> if set_wobbel_control has been last called with <code>Value = 0xFFFFFFFF</code>. Otherwise, the laser power <code>P0</code> multiplied by the external factor from set_multi_mcbbsp_in is emitted. - If the wobbel shape gets switched off while the wobbel mode remains active, then the shape automatically switches to <code>Mode = 1</code> (horizontal figure-8). - The wobbel mode with "freely definable wobbel shapes" also needs to be switched on by set_wobbel_mode.

Undelayed Short List Command	set_wobbel_vector
Comments (cont'd)	• Variable laser power for wobbel shapes cannot be combined with variable laser power for automatic or vector-based laser control (parameterized mark commands). Wobbel variation overwrites other variations if the respective Ctrl parameters are identical. Though unidentical Ctrl parameters are allowed, they serve no practical purpose. • When defining “freely definable wobbel shapes” make sure that the Microsteps of each individual wobbel vector does not exceed the maximum positioning speed significantly! Otherwise, a galvanometer scanner overheating may occur, see also Chapter 8.4.1 “Wobbel Shapes – Important Notes on Choosing Appropriate Parameter Values”, Page 219
RTC4→RTC5	New command.
Version info	–
References	set_multi_mcbasp_in , set_multi_mcbasp_in_list , set_wobbel_control

Ctrl Command	set_zoom
Comments	• This command is described, for example, in the manual for the intelliWELD II FT.

Undelayed Short List Command	set_zoom_list
Comments	• This command is described, for example, in the manual for the intelliWELD II FT.

Ctrl Command	simulate_encoder
Function	Activates or deactivates encoder simulation for the specified encoder.
Call	<code>simulate_encoder(EncoderNo)</code>
Parameters	EncoderNo Encoder number as an unsigned 32-bit value. Allowed values: = 1: ENCODER X pulses are simulated and encoder counter "Encoder0" thereby incremented. = 2: ENCODER Y pulses are simulated and encoder counter "Encoder1" thereby incremented. = 3: Pulses for ENCODER X and ENCODER Y are simulated. = 0: The encoder simulation is deactivated.
Comments	• The encoder simulation is driven by an internal 1 MHz clock (see also Section "Encoder Simulation", Page 275). • simulate_encoder does not trigger a reset of the encoder counter. • If EncoderNo > 3, then simulate_encoder is ignored (get_last_error return code RTC5_PARAM_ERROR).
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_encoder , store_encoder , read_encoder , wait_for_encoder , set_fly_x , set_fly_y , set_fly_rot

Normal List Command	simulate_ext_start
Function	After the specified track delay, causes a simulated External Start .
Call	<code>simulate_ext_start(Delay, EncoderNo)</code>
Parameters	Delay Track delay (in counter steps of the selected <code>EncoderNo</code> encoder counter). As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$. EncoderNo Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".
Comments	• For External Starts, see Section "External Start", Page 266. • The track delay is specified in <i>relative</i> counting units of the selected encoder counter (the RTC5 encoder counters are triggered by an external or simulated encoder signal, see Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274). The start trigger for the start occurs only after the internal encoder counter has reached the specified track delay. External Starts initiated by simulate_ext_start or by an external start signal, but whose execution has been postponed according to the specified track delay, are placed in a queue that accommodates up to 8 starts. simulate_ext_start cancels a previous queue and starts a new one. • A start trigger initiated by simulate_ext_start or an external start signal only triggers a start if it does not occur when the BUSY list execution status is set (for example, when outputting a list), when the INTERNAL-BUSY list execution status is set (for example, during goto_xy) and/or when the PAUSED list execution status is set (after pause_list, stop_list or set_wait). Otherwise, Bit #11 of the get_startstop_info return value is set. Therefore, if an unsuitable track delay is specified (for example, <code>Delay = 0</code>), no start is triggered. If simulate_ext_start is the first command in a list, then get_startstop_info can be used for checking whether processing of this list can finish within the defined track delay. • Ensure that the sign of the track delay (<code>Delay</code> parameter) is appropriate for the selected encoder's counting direction (for external triggering, this corresponds to the workpiece's direction of motion). • Track delays can also be set with set_ext_start_delay or set_ext_start_delay_list. Track delays are deactivated by initialization (with load_program_file), by external stops and by stop_execution. They can also be deactivated by set_control_mode(Bit #2). • The simulate_ext_start command alone <i>does not</i> cause an encoder reset. But if accordingly set with set_control_mode(Bit #9), a start trigger initiated by simulate_ext_start, simulate_ext_start_ctrl or by an external start signal causes an encoder reset if the start trigger is preceded by one of the Processing-on-the-fly commands set_fly_x, set_fly_y, set_fly_2d, set_fly_2d or set_fly_rot. • If <code>EncoderNo > 1</code>, then simulate_ext_start is replaced with a list_nop (get_last_error return code RTC5_PARAM_ERROR).

Normal List Command	<code>simulate_ext_start</code>
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified value for <code>Delay</code> by 16. The allowed value range decreases accordingly.
Version info	–
References	<code>simulate_ext_start_ctrl</code> , <code>set_ext_start_delay</code> , <code>set_ext_start_delay_list</code> , <code>set_extstartpos</code> , <code>set_extstartpos_list</code> , <code>set_control_mode</code> , <code>simulate_ext_stop</code>

Ctrl Command	<code>simulate_ext_start_ctrl</code>
Function	Causes a simulated External Start .
Call	<code>simulate_ext_start_ctrl()</code>
Comments	- For External Starts, see Section “External Start”, Page 266. - The start trigger for the start occurs only after a track delay previously set by <code>set_ext_start_delay</code>, <code>set_ext_start_delay_list</code> or <code>simulate_ext_start</code>. Track delays are deactivated by initialization (with <code>load_program_file</code>), by external stops and by stop_execution. They can also be deactivated with <code>set_control_mode</code> (Bit #2). External Starts initiated by <code>simulate_ext_start_ctrl</code> or by an external start signal, but whose execution has been postponed according to the specified track delay, are placed in a queue that accommodates up to 8 starts. In contrast to <code>simulate_ext_start</code>, <code>simulate_ext_start_ctrl</code> does <i>not</i> cancel the previous queue. - A start trigger initiated by <code>simulate_ext_start_ctrl</code> or by an external start signal does only trigger a start if it does not coincide with the output of a list (otherwise, Bit #11 of the <code>get_startstop_info</code> return value gets set). - The <code>simulate_ext_start_ctrl</code> command alone <i>does not</i> cause an encoder reset. But if accordingly set with <code>set_control_mode</code> (Bit #9), a start trigger initiated by <code>simulate_ext_start_ctrl</code>, <code>simulate_ext_start</code> or by an external start signal causes an encoder reset if the start trigger is preceded by one of the Processing-on-the-fly commands <code>set_fly_x</code>, <code>set_fly_y</code>, <code>set_fly_2d</code> or <code>set_fly_rot</code>. - As of version DLL 544, OUT 544, the following applies: <code>simulate_ext_start_ctrl</code> can be disabled by <code>set_control_mode</code> (Bit #4 = 1) or <code>set_control_mode_list</code> (Bit #4 = 1). - As of version DLL 544, OUT 544, the following applies: provided that an execution start is allowed, <code>simulate_ext_start_ctrl</code> waits 30 μs before it returns. This closes a potential timing gap (with <code>get_status</code>) between command call and actual start.
RTC4→RTC5	New command.
Version info	Last change DLL 544, OUT 544: see comment.
References	<code>simulate_ext_start</code>

Ctrl Command	simulate_ext_stop
Function	Causes a simulated External Stop .
Call	<code>simulate_ext_stop()</code>
Comments	• For external stops, see Section “External Stop”, Page 265. • simulate_ext_stop simultaneously halts the master board and all <i>downstream</i> slave boards. See also Chapter 6.6.3 “Master/Slave Operation”, Page 113. • In contrast, stop_execution only halts the master board.
RTC4→RTC5	New command.
Version info	–
References	simulate_ext_start, simulate_ext_start_ctrl, stop_execution, sync_slaves

Ctrl Command	start_loop
Function	Starts a repeating automatic list change.
Call	<code>start_loop()</code>
Comments	• A list change or execution restart activated with start_loop repeats until execution is ended by calling quit_loop. • start_loop can be called at any desired point in time, but only has effect upon the next set_end_of_list. • If the automatic list change is activated during processing of a list, then upon reaching set_end_of_list execution continues without delay at the other list. If there is only one list ($Mem2 = 0$, see config_list), then upon reaching set_end_of_list execution continues at $Pos = 0$ (that is, at the beginning of the list). • During processing of a list, the other list (and also the current list) can be newly loaded, see Chapter 6.4.6 "Automatic List Changing", Page 99. • So that the start_loop command can function at all, the already active list must absolutely be finalized by set_end_of_list; the new list should already be loaded and the input pointer should be sufficiently ahead of the output pointer (otherwise, "old" commands are executed). If, during list execution, the end of the list is reached without encountering a set_end_of_list, then execution automatically continues at the beginning of the current list, not at the beginning of the other list. • If, during processing of a list, the start_loop and auto_change_pos ($Pos > 0$) commands are called, then upon the next set_end_of_list the command auto_change_pos ($Pos > 0$) is executed; and at the next one the start_loop command is executed. • The current List Status can be queried by read_status. • The current List Execution Status can be queried by get_status. • start_loop triggers a flush of the buffered list input, see Chapter 6.4.1 "Loading Lists", Page 94.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	quit_loop

Ctrl Command	stepper_abs	
Function	Triggers set-position motions to the specified absolute set positions by both stepper motor outputs.	
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_abs is not executed (get_last_error return code RTC5_TYPE_REJECTED).	
Call	stepper_abs(Pos1, Pos2, WaitTime)	
Parameters	Pos1	Absolute set position in CLOCK pulse units for stepper motor output ports 1. As signed 32-bit values. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$.
	Pos2	Like Pos1 (analogously).
	WaitTime	Determines when stepper_abs returns at the latest. As an unsigned 32-bit value. 1 bit equals 1 s. Allowed value range: $[0 \dots + (2^{32}-1)]$.
Comments	- For programming the stepper motor signals, see Chapter 9.1.5 "Controlling Stepper Motors", Page 260. stepper_abs sets the new set-position values even if a previously started set-position motion is still in progress: - If Pos1/Pos2 in the current direction of motion lies in front of the internal position variable's value, then the motion continues and the Busy status remains set. - If Pos1/Pos2 equals the current value of the internal position variable, then the motion stops. The Busy status gets reset. - If Pos1/Pos2 in the current direction of motion already lies past the internal position variable's value, then the corresponding stepper motor's direction of motion reverses (see note on page 260). The Busy status remains set. - If no set-position motion is in progress, then one starts and the Busy status gets set. - During performance of a reference motion (Init status set, see stepper_init), stepper_abs does not execute (get_last_error return code RTC5_PARAM_ERROR). - If the CLOCK pulse period has been set to 0 by stepper_init, stepper_control or stepper_control_list, then no stepper motor motion occurs at the corresponding stepper motor output. - If WaitTime = 0, then stepper_abs returns immediately so that system control is restored to the user program.	
RTC4→RTC5	New command.	
Version info	–	
References	stepper_abs_no , stepper_rel , stepper_rel_no , stepper_abs_list	

Undelayed Short List Command	stepper_abs_list
Function	Like stepper_abs , but a list command and without <code>WaitTime</code> parameter.
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_abs_list is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>stepper_abs_list(Pos1, Pos2)</code>
Parameters	Pos1 Like stepper_abs .
	Pos2 Like stepper_abs .
Comments	• See stepper_abs. • During performance of a reference motion (see stepper_init), execution of stepper_abs_list is delayed until the reference motion completes.
RTC4→RTC5	New command.
Version info	–
References	stepper_abs

Ctrl Command	stepper_abs_no
Function	Triggers a set-position motion to the specified absolute set position at <i>one</i> stepper motor output.
Call	<code>stepper_abs_no(No, Pos, WaitTime)</code>
Parameters	No Number of the stepper motor output port. As an unsigned 32-bit value. Allowed values: = 1: Stepper motor output port 1. = 2: Stepper motor output port 2. If the value is invalid, then stepper_abs_no is not executed (get_last_error return code RTC5_PARAM_ERROR).
	Pos Absolute set position in CLOCK pulse units. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$.
	WaitTime Determines when stepper_abs_no returns at the latest. <i>1 bit equals 1 s.</i> As an unsigned 32-bit value. Allowed value range: $[0 \dots + (2^{32}-1)]$.
Comments	• A set-position motion is only performed at the stepper motor output port specified by No. Otherwise, stepper_abs_no is identical to stepper_abs (see comments there).
RTC4→RTC5	New command.
Version info	–
References	stepper_abs , stepper_abs_no_list

Undelayed Short List Command	stepper_abs_no_list	
Function	Like stepper_abs_no , but a list command and without <code>WaitTime</code> parameter.	
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_abs_no_list is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).	
Call	stepper_abs_no_list(No, Pos)	
Parameters	No	Number of the stepper motor output. As an unsigned 32-bit value. Allowed values: = 1: Stepper motor output 1. = 2: Stepper motor output 2. If the value is invalid, then stepper_abs_no_list is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR).
	Pos	Like stepper_abs_no .
Comments	• See stepper_abs_no. • During performance of a reference motion (see stepper_init), execution of stepper_abs_no_list is delayed until the reference motion completes.	
RTC4→RTC5	New command.	
Version info	–	
References	stepper_abs_no	

Ctrl Command	stepper_control
Function	Sets the CLOCK signal pulse periods for stepper motor control.
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_control is not executed (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>stepper_control(Period1, Period2)</code>
Parameters	Period1 Pulse period of the CLOCK signals for stepper motor output ports 1. 1 bit equals 10 μ s. As a signed 32-bit value. Allowed values: $[0...+(2^{24}-1)]$ or < 0. Larger values are clipped. > 0: The new period waits for an already-running CLOCK pulse period at the corresponding stepper motor output before starting. = 0: An already-running CLOCK pulse period aborts at each stepper motor output (the corresponding stepper motor motion gets stopped, the Init- and/or Busy statuses for the respective stepper motor outputs get reset). < 0: the corresponding stepper motor control remains unchanged.
	Period2 Pulse period of the CLOCK signals for stepper motor output ports 2. Otherwise, like Period1 .
Comments	• For programming the stepper motor signals, see Chapter 9.1.5 "Controlling Stepper Motors", Page 260. • Period1 or Period2= 0 can be used as an emergency stop (see also Section "Terminating Infinite Motions", Page 262).
RTC4→RTC5	New command.
Version info	–
References	stepper_control_list

Undelayed Short List Command	stepper_control_list
Function	Like stepper_control , but a list command.
Restriction	Like stepper_control .
Call	<code>stepper_control_list(Period1, Period2)</code>
Parameters	Period1 Like stepper_control .
	Period2 Like stepper_control .
Comments	• See stepper_control.
RTC4→RTC5	New command.
Version info	–
References	stepper_control

Ctrl Command	stepper_disable_switch	
Function	Controls the usage of the stepper motor control SWITCH signals.	
Call	stepper_disable_switch(Disable1, Disable2)	
Parameters	Disable1	Instruction how the SWITCH signals from stepper motor input 1 are to be used. As a signed 32-bit value. Allowed value range: $[-2^{32} \dots + (2^{32}-1)]$. > 0: The SWITCH signal at the stepper motor input is not used. = 0: The SWITCH signal at the stepper motor input is used. < 0: The use of the stepper motor input signal remains unchanged.
	Disable2	Like Disable1 (analogously).
Comments	• For programming the stepper motor signals, see Chapter 9.1.5 "Controlling Stepper Motors", Page 260. • The limit switch can be ignored during normal motions. For example, this may make sense for continuously rotating axes. • The SWITCH signals are always used with motions initiated by stepper_init.	
RTC4→RTC5	New command.	
Version info	Available as of DLL 542, OUT 542.	
References	stepper_init	

Ctrl Command	stepper_enable
Function	Changes the stepper motor control's ENABLE signals.
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_enable is not executed (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>stepper_enable(Enable1, Enable2)</code>
Parameters	Enable1 ENABLE signal for stepper motor output 1. As a signed 32-bit value. Allowed value range: $[-2^{32} \dots + (2^{32}-1)]$. > 0: The ENABLE signal at the stepper motor output gets set. = 0: The ENABLE signal at the stepper motor output gets reset. < 0: The stepper motor output signal remains unchanged.
	Enable2 Like Enable1 (analogously).
Comments	For programming the stepper motor signals, see Chapter 9.1.5 "Controlling Stepper Motors", Page 260.
RTC4→RTC5	New command.
Version info	–
References	stepper_enable_list

Undelayed Short List Command	stepper_enable_list
Function	Like stepper_enable , but a list command.
Restriction	Like stepper_enable .
Call	<code>stepper_enable_list(Enable1, Enable2)</code>
Parameters	Enable1 Like stepper_enable .
	Enable2 Like stepper_enable .
Comments	See stepper_enable.
RTC4→RTC5	New command.
Version info	–
References	stepper_enable

Ctrl Command	stepper_init	
Function	Initializes a stepper motor.	
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_init is not executed (get_last_error return code RTC5_TYPE_REJECTED).	
Call	stepper_init(No, Period, Dir, Pos, Tol, Enable, WaitTime)	
Parameters	No	Number of the stepper motor output. As an unsigned 32-bit value. Allowed values: = 1: Stepper motor output 1. = 2: Stepper motor output 2. If the value is invalid, then stepper_init is not executed (get_last_error return code RTC5_PARAM_ERROR).
	Period	Pulse period of the CLOCK signal. As an unsigned 32-bit value. 1 bit equals 10 μs. Allowed value range: $[0 \dots + (2^{24}-1)]$. Larger values are clipped. If Period = 0, then no reference motion is performed and the function immediately returns (see comments).
	Dir	Direction of stepper motor motion during the reference motion. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$. > 0: The DIRECTION signal gets set; during the reference motion the internal position variable increments. = 0: The DIRECTION signal gets reset; during the reference motion the internal position variable decrements. < 0: The DIRECTION signal remains unchanged, no reference motion is performed and the function immediately returns.
	Pos	New position variable value. In CLOCK pulse units (see comments). As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$.
	Tol	Tolerance for the reference motion (see comments). As an unsigned 32-bit value. Allowed value range: $[0 \dots + (2^{32}-1)]$. If Dir < 0 and/or Period = 0, then the value Tol = 0 is irrelevant, otherwise Tol = 0 causes stepper_init not to be executed (get_last_error return code RTC5_PARAM_ERROR).
	Enable	ENABLE signal. As an unsigned 32-bit value. Allowed value range: $[0 \dots + (2^{32}-1)]$. = 0: The ENABLE signal is reset. > 0: The ENABLE signal is set.
	WaitTime	Defines when stepper_init, at the latest, returns (see comments). 1 bit equals 1 s. As an unsigned 32-bit value. Allowed value range: $[0 \dots + (2^{32}-1)]$.

Ctrl Command	stepper_init
Comments	- For programming the stepper motor signals, see Chapter 9.1.5 “Controlling Stepper Motors”, Page 260. stepper_init immediately stops all previously started motions of the stepper motor specified by the <code>No</code> parameter. - The ENABLE signal <code>Enable</code> is merely forwarded and always correspondingly set, but has no effect on internal operations. - If <code>Period > 0</code> and <code>Dir ≥ 0</code>, then stepper motor <code>No</code> starts a reference motion with the supplied CLOCK pulse period in the defined direction and the <code>Init</code> status gets set. The first Clock pulse is only generated after a full CLOCK pulse period. - If a limit switch is activated right from the beginning, then the controller attempts to seek a position within the $\pm Tol$ range of the current position, initially opposite to the defined direction, with the limit switch deactivated. If this does not bring success, then the attempt terminates. In this case, the SWITCH status bit remains set (see get_stepper_status). - If a limit switch gets activated during a motion, then the reference motion stops there. Afterward, the limit switch position is crossed 4× to arrive at an averaged value for this position. Finally, the stepper motor is driven in the opposite direction by a normal set-position motion (<code>Init</code> status reset, <code>Busy</code> status set) within the tolerance value <code>Tol</code>. Here, the DIRECTION status signal changes, whereas it remains constant during seeking motions with multiple direction changes. The internal position variable (for the current position) gets set to the value defined by the <code>Pos</code> parameter. Thus, this value represents a positional offset by <code>Tol</code> with respect to the defined position. With <code>Pos = Tol</code>, the middle limit switch position corresponds to position 0. - If no limit switch is found (for example, because no limit switch exists in the defined direction), then the stepper motor performs an infinite motion. stepper_init then returns after <code>WaitTime</code> seconds to restore system control to the user program. But the stepper motor’s infinite motion continues until it is aborted by a new stepper_init command or stepper_control(<code>Period1/Period2 = 0</code>). For more on this, see the Section “Terminating Infinite Motions”, Page 262. - If <code>Period = 0</code>, <code>Dir < 0</code> and/or <code>WaitTime = 0</code>, then stepper_init returns straight away to immediately restore system control to the user program. You can then call get_stepper_status to check whether the reference motion has completed. During the seeking motion the status “Init” (see page 360) is set and during the terminating set-position motion to (limit switch + <code>Tol</code>) the status “Busy” (see page 360) is set. <code>Period = 0</code> and/or <code>Dir < 0</code> can be used as an emergency stop. Then a previously started stepper motor motion gets aborted, but no reference motion is performed. Here, too, the position variable gets set to the value <code>Pos</code>. The DIRECTION signal remains unchanged. - If <code>Period = 0</code>, then status “Init” (see page 360) and status “Busy” (see page 360) get reset. No further clock pulses are outputted until <code>Period</code> is again set to a positive value.

Ctrl Command	stepper_init
RTC4→RTC5	New command.
Version info	–
References	–

Ctrl Command	stepper_rel
Function	Triggers set-position motions to the specified relative set positions by both stepper motor outputs.
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_rel is not executed (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>stepper_rel(dPos1, dPos2, WaitTime)</code>
Parameters	dPos1 Relative set positions in CLOCK pulse units for stepper motor output port 1. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$.
	dPos2 Relative set positions in CLOCK pulse units for stepper motor output port 2. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$.
	WaitTime Like stepper_abs .
Comments	The desired set positions should be specified relative to the current internal set-position values. Otherwise, stepper_rel is identical to stepper_abs (see comments there).
RTC4→RTC5	New command.
Version info	–
References	stepper_abs , stepper_rel_list

Undelayed Short List Command	stepper_rel_list
Function	Like stepper_rel , but a list command and without WaitTime parameter.
Restriction	Like stepper_rel .
Call	<code>stepper_rel_list(dPos1, dPos2)</code>
Parameters	dPos1, Like stepper_rel . dPos2
Comments	- See stepper_rel. - During performance of a reference motion (see stepper_init), execution of stepper_rel_list is delayed until the reference motion completes.
RTC4→RTC5	New command.
Version info	–
References	stepper_rel

Ctrl Command	stepper_rel_no
Function	Triggers a set-position motion to the specified relative position at <i>one</i> stepper motor output port.
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_rel_no is not executed (get_last_error return code RTC5_TYPE_REJECTED).
Call	<code>stepper_rel_no(No, dPos, WaitTime)</code>
Parameters	No Like stepper_abs_no .
	dPos Relative position. In CLOCK pulse units. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$.
	WaitTime Like stepper_abs_no .
Comments	The set positions should be specified relative to the current position values (the - position value is correspondingly get newly set) and a set-position motion is only performed at the stepper motor output specified by No. Otherwise, stepper_rel_no is identical to stepper_abs (see comments there).
RTC4→RTC5	New command.
Version info	–
References	stepper_abs_no , stepper_abs , stepper_rel_no_list

Undelayed Short List Command	stepper_rel_no_list
Function	Like stepper_rel_no , but a list command and without <code>WaitTime</code> parameter.
Restriction	Like stepper_rel_no .
Call	<code>stepper_rel_no_list(No, dPos)</code>
Parameters	No Number of the stepper motor output. As an unsigned 32-bit value. Allowed values: = 1: Stepper motor output 1. = 2: Stepper motor output 2. If the value is invalid, then stepper_rel_no_list is, already during loading, replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR).
	dPos Like stepper_rel_no .
Comments	- See stepper_rel_no. - During performance of a reference motion (see stepper_init), execution of stepper_rel_no_list is delayed until the reference motion completes.
RTC4→RTC5	New command.
Version info	–
References	stepper_rel_no

Normal List Command	stepper_wait	
Function	Interrupts further execution of a list until a previously started (at the specified stepper motor output) stepper motor motion completes.	
Restriction	For older RTC5 Boards with DSP version numbers < 2 (get_rtc_version Bit #16...Bit #23), stepper_wait is neither executed nor replaced by list_nop (get_last_error return code RTC5_TYPE_REJECTED).	
Call	stepper_wait(No)	
Parameters	No	Number of the stepper motor output. As an unsigned 32-bit value. Allowed values: = 1: Stepper motor output 1. = 2: Stepper motor output 2. = 0, 3: Both stepper motor outputs. Only the two least-significant bits are evaluated.
Comments	• For programming the stepper motor signals, see Chapter 9.1.5 "Controlling Stepper Motors", Page 260. • If <i>no</i> stepper motor motion had been previously started at the specified stepper motor output, then stepper_wait still needs 10 μs to execute, even though it otherwise has no effect. • stepper_wait does <i>not</i> influence: – the Signals for "Laser Active" Operation – the List Status – the List Execution Status	
RTC4→RTC5	New command.	
Version info	–	
References	stepper_rel_no	

Ctrl Command	stop_execution
Function	Stops execution of the list and deactivates the “laser active” laser control signals immediately.
Call	<code>stop_execution()</code>
Comments	• stop_execution deactivates the Signals for “Laser Active” Operation even if no list is active (here stop_execution has no other effects; here too: get_last_error return code RTC5_BUSY). • With stop_execution, the galvanometer scanners stay in the current position, unless a home jump has been previously defined by home_position or home_position_xyz (a home jump is executed). Therefore, before a new list is loaded, the galvanometer scanners should be set to a defined position using goto_xy. • The external start inputs are disabled, see Section “External Start”, Page 266. • The Processing-on-the-fly correction is switched off. • The BUSY list status values (see read_status) and the BUSY list execution status-List Execution Status value (see get_status) are reset. • A list that has been interrupted by stop_execution <i>cannot</i> be resumed. It must instead be newly started (for example, by execute_list_pos). To only temporarily halt a list and later resume it, you can use pause_list. • stop_execution only affects the addressed RTC5 board. In a master/slave chain, stop_execution is not passed on to the <i>downstream</i> slave boards. If all RTC5 boards of a master/slave chain shall be synchronously stopped, then simulate_ext_stop or an external stop signal must be applied to the master board, see also Chapter 6.6.3 “Master/Slave Operation”, Page 113.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	get_startstop_info

Ctrl Command	stop_list
Function	Pauses execution of the list and deactivates the Signals for “Laser Active” Operation .
Call	<code>stop_list()</code>
Comments	• stop_list is synonymous with pause_list (see comments there).
RTC4→RTC5	Basically unchanged functionality. However: Additional PAUSED list execution status which is set by stop_list .
Version info	–
References	pause_list

Ctrl Command	stop_trigger
Function	Resets (to <code>Busy = 0</code>) a measurement session's status that can be queried by measurement_status .
Call	<code>stop_trigger()</code>
Comments	• stop_trigger is only needed if a measurement session has been started by set_trigger or set_trigger4, but not subsequently terminated by set_trigger(<code>Period = 0</code>) or set_trigger4(<code>Period = 0</code>). Here, you can reset the measurement session status even when no list is active (see set_trigger comments). • stop_trigger is not executed (get_last_error return code <code>RTC5_BUSY</code>), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • stop_trigger is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set)
RTC4→RTC5	New command.
Version info	–
References	measurement_status , set_trigger , set_trigger4

Undelayed Short List Command	store_encoder
Function	Stores the current counts of the two RTC5 encoder counters in a buffer on the RTC5.
Call	<code>store_encoder(Pos)</code>
Parameters	<code>Pos</code> Storage position. As an unsigned 32-bit value. • Bit #0 = 0: The counter values are saved to storage position 0. • Bit #0 = 1: The counter values are saved to storage position 1.
Comments	• store_encoder can be used, for instance, to determine the exact number of encoder increments occurring within a specific list-processing period. Here, you could begin the command list by storing the counts in storage position 0 by <code>store_encoder(0)</code>. At the end of the command list, you could store the counts to storage position 1 by <code>store_encoder(1)</code> and subsequently retrieve the stored values by read_encoder. • See also Chapter 8.6 "Processing-on-the-fly", Page 227 and Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274.
RTC4→RTC5	New command.
Version info	–
References	read_encoder , get_encoder , set_fly_x , set_fly_y , set_fly_rot , wait_for_encoder

Undelayed Short List Command	sub_call
Function	Causes an unconditional jump to an indexed subroutine.
Call	<code>sub_call(Index)</code>
Parameters	Index Index of the called indexed subroutine. As an unsigned 32-bit value. Allowed value range: [0...1023].
Comments	• sub_call reads the indexed subroutine's starting address from the internal management table based on the supplied index and then calls list_call (see also the comments there). list_call then triggers the jump to the subroutine. • sub_call starts indexed subroutines in protected memory (that were loaded and/or referenced by load_sub, load_disk or copy_dst_src) as well as indexed subroutines in the unprotected list area (that were referenced by set_sub_pointer or copy_dst_src). • If no subroutine is referenced for the supplied index, then the jump is suppressed and execution continues at the command located after the calling position. If applicable, a list_continue is executed. <code>get_sub_pointer(Index)</code> can be used to determine whether a subroutine has been referenced for a particular index. If <i>no</i> subroutine has been referenced, this command returns the value "-1" (= $2^{32}-1$). • If <code>Index > 1023</code>, then sub_call is, already during loading, replaced by a list_nop (get_last_error return code <code>RTC5_PARAM_ERROR</code>). • Absolute Vector commands and "Arc" commands execute absolutely after being called with sub_call. If the subroutine needs to execute at various locations within the Image field, then either the subroutine can only contain relative [*]mark[*] Commands, arc commands or Jump commands or sub_call_abs must be used instead.
RTC4→RTC5	New command.
Version info	–
References	list_call , sub_call_abs , sub_call_cond

Undelayed Short List Command	sub_call_abs
Function	Causes an unconditional jump to an indexed subroutine. In the called subroutine, any absolute Vector commands and " Arc " commands receive an offset (corresponding to the current coordinates at the time of the call).
Call	sub_call_abs(Index)
Parameters	Index Index of the called indexed subroutine. As an unsigned 32-bit value. Allowed value range: [0...1023].
Comments	• sub_call_abs reads the indexed subroutine's starting address from the internal management table based on the supplied index and then calls list_call_abs (see also the comments there). list_call_abs then triggers the jump to the subroutine. • If the called subroutine contains no absolute commands, then there is no difference between sub_call_abs and sub_call.
RTC4→RTC5	New command.
Version info	–
References	sub_call , sub_call_abs_cond

Undelayed Short List Command	sub_call_abs_cond
Function	<i>Conditional call (AbsCall) of an indexed subroutine:</i> sub_call_abs_cond executes sub_call_abs(Index), if the current IOvalue at the 16-bit digital input port of the EXTENSION 1 socket connector meets the following condition: $((\text{IOvalue AND Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, the directly following list command is immediately executed.
Call	sub_call_abs_cond(Mask1, Mask0, Index)
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1.
	Index Index of the called indexed subroutine. As an unsigned 32-bit value. Allowed value range: [0...1023].
Comments	• See sub_call_abs. • See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	sub_call_abs

Undelayed Short List Command	sub_call_abs_repeat
Function	Causes an unconditional jump to an indexed subroutine and executes its body several times.
Call	<code>sub_call_abs_repeat(Index, Number)</code>
Parameters	Index Index of the to be called indexed subroutine (as with sub_call_abs). Number Number of repetitions. 0 is treated as 1. As an unsigned 32-bit value.
Comments	• sub_call_abs(Index) is synonymous with <code>sub_call_abs_repeat(Index, 1)</code>. • See sub_call_repeat and sub_call_abs.
RTC4→RTC5	New command.
Version info	Available as of DLL 538, OUT 538.
References	sub_call_repeat , sub_call_abs , sub_call

Undelayed Short List Command	sub_call_cond
Function	<i>Conditional call of an indexed subroutine:</i> sub_call_cond executes sub_call(Index), if the current IOvalue at the 16-bit digital input port of the EXTENSION 1 socket connector meets the following condition: $((\text{IOvalue AND Mask1}) = \text{Mask1}) \text{ AND } (((\text{not IOvalue}) \text{ AND Mask0}) = \text{Mask0})$ (= if the bits specified in Mask1 are 1 and the bits specified in Mask0 are 0). Otherwise, the directly following list command is immediately executed.
Call	<code>sub_call_cond(Mask1, Mask0, Index)</code>
Parameters	Mask1 16-bit mask. As an unsigned 32-bit value. Only the lower 16 bits are evaluated.
	Mask0 See Mask1.
	Index Index of the to-be-called indexed subroutine. As an unsigned 32-bit value. Allowed value range: [0...1023].
Comments	• See sub_call. • See also Chapter 9.3.2 "Conditional Command Execution", Page 271.
RTC4→RTC5	New command.
Version info	–
References	sub_call

Undelayed Short List Command	sub_call_repeat
Function	Causes an unconditional jump to an indexed subroutine and executes its body several times.
Call	<code>sub_call_repeat(Index, Number)</code>
Parameters	Index Index of the to be called indexed subroutine (as with sub_call). Number Number of repetitions. As an unsigned 32-bit value. Number = 0 is treated like Number = 1.
Comments	• sub_call(Index) is synonymous with <code>sub_call_repeat(Index, 1)</code>. • sub_call_repeat avoids an empty cycle at the repetition, which otherwise inevitably occurs with <code>sub_call...sub_call</code> or <code>list_repeat...sub_call...list_until</code> constructions. • By sub_call_repeat, for example, trajectories (see Glossary entry on page 26) from micro vector commands for runup curves and coast down curves can be seamlessly joined together with shapes from subroutines.
RTC4→RTC5	New command.
Version info	Available as of DLL 538, OUT 538.
References	sub_call_abs_repeat, sub_call, sub_call_abs, micro_vector_abs, micro_vector_abs_3d, micro_vector_rel, micro_vector_rel_3d

Undelayed Short List Command	switch_ioport
Function	Executes a relative list jump list_jump_rel (Pos) whose jump distance Pos (>1) is determined at runtime by the current value (IOvalue) at the 16-bit digital input port of the EXTENSION 1 socket connector. You can specify which of the 16-bit digital input port's bits should be evaluated for this purpose.
Call	<code>switch_ioport(MaskBits, ShiftBits)</code>
Parameters	MaskBits Number of contiguous bits (as an unsigned 32-bit value) of the 16-bit digital input port to be evaluated for determining the jump distance. Allowed value range: [1...16].
	ShiftBits Position of the least significant to-be-evaluated bit of the 16-bit digital input port. As an unsigned 32-bit value. Allowed value range: [0...15].
Comments	• With invalid values of MaskBits or ShiftBits and with (MaskBits + ShiftBits) > 16, switch_ioport is replaced by a list_nop (get_last_error return code RTC5_PARAM_ERROR). • The following applies: Mask = ((1 << MaskBits) – 1) << ShiftBits and SwitchNo = (Mask & IOvalue) >> ShiftBits. Here, a list_jump_rel(Pos) with Pos = (SwitchNo + 1) list positions are then executed. • The jump distance is at least 1. This prevents infinite loops when no signal is present. Jumps to the same address (Pos = 0) are not possible with switch_ioport, but can be simulated by list_jump_rel (–1) as the directly subsequent command. • The maximum jump distance is 2¹⁶ list positions. • See also list_jump_rel. • See also Section "16-Bit Digital Input Port and 16-Bit Digital Output Port", Page 65 and Chapter 9.3.2 "Conditional Command Execution", Page 271.
Example (Pascal)	• It is assumed that the current value at the 16-bit digital input port is \$F152 at runtime: then switch_ioport(\$0008, \$0004) executes list_jump_rel(\$0016), that is, a relative list jump of length 22 list positions.
RTC4→RTC5	New command.
Version info	–
References	list_jump_rel , list_jump_rel_cond

Ctrl Command	sync_slaves
Function	Stably synchronizes (with the addressed RTC5 Board's 10 μ s clock) all slave boards connected in a master/slave chain to the addressed RTC5 Board's SLAVE connector.
Call	<code>sync_slaves()</code>
Comments	- For sync_slaves usage, see Chapter 6.6.3 "Master/Slave Operation", Page 113. - SCANLAB recommends executing synchronization immediately after all boards have been initialized by load_program_file and load_correction_file. Otherwise, all involved boards (that is, the master board and the downstream slave boards allocated to the user program) should already have been halted prior to the call of sync_slaves. To avoid irregularities during execution of this command, you should neither apply external stop signals to the boards nor trigger External Starts (these do not get automatically suppressed). - During the course of sync_slaves, a simulate_ext_stop is passed to the addressed board. This halts the addressed board and all downstream slave boards in the master/slave chain (including boards not allocated to the user program). It is the responsibility of users to ensure that a running process is not disrupted by sync_slaves. - After execution of sync_slaves, the scan system axes of all involved boards are in either the coordinate center position (0, 0 [,0]) or the HomeJump position (possibly shifted by an offset set by set_offset, set_defocus or set_hi). sync_slaves (relating to synchronization) only affects RTC5 Boards connected in a master/slave chain to the addressed RTC5 Board's MASTER connector. It does not affect the addressed board itself or any boards connected to the addressed board's SLAVE connector. Therefore, if all slave boards of a master/slave chain shall be synchronized with the master board, then sync_slaves must address the master board of the master/slave chain. sync_slaves has no effect, if no board is connected to the Master connector of the addressed board. - Synchronization of downstream slave boards by sync_slaves occurs even when the BUSY list execution status of the board or INTERNAL-BUSY list execution status is set (they are automatically halted). Nevertheless, the only slave boards to get synchronized are those allocated to the user program (allocation is not requested automatically). If the user program possesses access rights for the addressed board but no further boards, then sync_slaves has no effect. - During execution of sync_slaves, all get_startstop_info error bits are cleared on all boards (including upstream boards) allocated to the user program. - For each downstream slave board, the 10 μs clock is interrupted for a short term and newly synchronized. Therefore, data transmission between the RTC5 and scan system may be disrupted for up to 20 μs per downstream board. Such disturbances could induce hard galvanometer scanner jumps if the actual output values do not correspond to (0,0,[0]).

Ctrl Command	sync_slaves
Comments (cont'd)	• sync_slaves is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status of the addressed board is currently set – the INTERNAL-BUSY list execution status of the addressed board is set • sync_slaves is even executed, if: – a list has been paused by set_wait (PAUSED list execution status is set)
RTC4→RTC5	New command.
Version info	–
References	get_master_slave , get_sync_status

Normal List Command	time_fix
Function	Stores the current time and date of the RTC clock/calender in a buffer for use with mark_date and mark_time .
Call	time_fix()
Comments	• time_fix is synonymous with time_fix_f_off with FirstDay = 0 and Offset = 0 (see comments there).
RTC4→RTC5	New command.
Version info	–
References	time_fix_f, time_fix_f_off

Normal List Command	time_fix_f
Function	Stores the current time and date of the RTC clock/calender in a buffer for use with mark_date and mark_time .
Call	time_fix_f(FirstDay)
Parameters	FirstDay See time_fix_f_off .
Comments	• time_fix_f is synonymous with time_fix_f_off with Offset = 0 (see comments there).
RTC4→RTC5	New command.
Version info	–
References	time_fix_f, time_fix_f_off, time_update, mark_time, mark_date

Normal List Command	time_fix_f_off
Function	Stores in a buffer the current time and date of the RTC clock/calender or a forward-dated date and time for use with mark_date and mark_time .
Call	<code>time_fix_f_off(FirstDay, Offset)</code>
Parameters	FirstDay Defines the starting number for determining the Julian calendar day from the current date of the RTC calendar: counting proceeds from FirstDay to FirstDay + 364 (+1 for leap years) . As an unsigned 32-bit value.
	Offset Forward dating. In seconds. As an unsigned 32-bit value. Allowed value range: $[0 \dots (2^{32}-1)]$.
Comments	• Before calling time_fix_f_off, time_fix_f or time_fix, synchronization of the RTC5 and PC time should be performed (at least once after each load_program_file) by time_update. • The complete time can be marked through multiple calls of mark_time and the complete date through multiple calls of mark_date. time_fix_f_off, time_fix_f or time_fix must therefore be called <i>before</i> these marking commands so that the to-be-marked time or date do not change during marking. • If time_fix_f_off, time_fix_f or time_fix are not called again before a time or date marking, then the last marked time is marked again. If time_fix_f_off, time_fix_f or time_fix are not called at all after a load_program_file, then a time of 00:00 or a date of January 1, 2000 is marked. • If Offset = 0, then the current date and current time are fixed. One practical use of forward dating (Offset > 0) is for setting a date of expiry based on the current date. Backdating (Offset < 0) is not possible.
RTC4→RTC5	New command.
Version info	–
References	time_fix , time_fix_f , time_update , mark_time , mark_date

Ctrl Command	time_update
Function	Sets the RTC5's 24-hour clock and calendar to the current PC time.
Call	<code>time_update()</code>
Comments	• time_update must be called after each load_program_file, if the 24-hour clock and calendar of the RTC5 are to be calibrated. • The base value for internal time counting is set to January 1, 2000, 00:00 by load_program_file whereas to the PC time by time_update. An internal seconds counter is set to 0 by load_program_file or by time_update, but is always driven by the quartz-controlled 10 μs clock. • Before marking with mark_date or mark_time, you must call time_fix, time_fix_f or time_fix_f_off so that the current time can be captured (as sum of the base value and the current value of the internal seconds counter) and formatted.
RTC4→RTC5	New command.
Version info	–
References	time_fix , time_fix_f , time_fix_f_off , mark_date , mark_time

Normal List Command	timed_arc_abs
Function	Moves the laser focus for the specified marking duration from the current position along an arc with the specified angle and center point (absolute coordinate values) within a 2D Image field .
Call	<code>timed_arc_abs(X, Y, Angle, T)</code>
Parameters	X Like arc_abs .
	Y Like arc_abs .
	Angle Like arc_abs .
	T Duration of the complete arc marking process. In μ s. As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_arc_abs behaves like arc_abs .
Comments	- Unlike arc_abs, timed_arc_abs does not execute the marking process with the specified (by set_mark_speed or set_mark_speed_ctrl) mark speed. Instead, the speed (that is, the number of Microsteps) is adjusted so that the arc lasts as long as specified (see Chapter 8.9 "Timed Commands", Page 250). The total marking time is (for $T \geq 5$) the sum of the specified (rounded) time and the set delays. - See also comments on arc_abs.
RTC4→RTC5	New command. See arc_abs .
Version info	–
References	arc_abs , timed_arc_rel

Normal List Command	timed_arc_rel
Function	Moves the laser focus for the specified marking duration from the current position along an arc with the specified angle and center point (relative coordinate values) within a 2D Image field .
Call	<code>timed_arc_rel(dX, dY, Angle, T)</code>
Parameters	dX Like arc_rel .
	dY Like arc_rel .
	Angle Like arc_rel .
	T Duration of the complete arc marking process. In μ s. As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_arc_rel behaves like arc_rel .
Comments	The coordinates for the arc center are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_arc_rel is analogous to timed_arc_abs (see the comments there).
RTC4→RTC5	New command.
Version info	–
References	timed_arc_abs , arc_rel

Normal List Command	timed_jump_abs
Function	Moves the output point (of the laser focus) for the specified jump duration along a 2D vector from the current position to the specified position (absolute coordinate values) within a 2D Image field .
Call	<code>timed_jump_abs(X, Y, T)</code>
Parameters	X Like jump_abs .
	Y Like jump_abs .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_jump_abs behaves like jump_abs .
Comments	- Unlike jump_abs, timed_jump_abs does not execute the jump with the specified (by set_jump_speed or set_jump_speed_ctrl) jump speed. Instead, the speed (that is, the number of Microsteps) is adjusted so that the vector lasts as long as specified, see Chapter 8.9 "Timed Commands", Page 250. The total jump time is (for $T \geq 5$) the sum of the specified (rounded) time and the set delays. - See also comments on jump_abs.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. In RTC4 Compatibility Mode , the RTC5 multiplies the specified values for X and Y by 16. The allowed value ranges decrease accordingly.
Version info	–
References	jump_abs , timed_jump_rel , timed_jump_abs_3d

Normal List Command	timed_jump_abs_3d
Function	Moves the output point (of the laser focus) for the specified jump duration along a 3D vector from the current position to the specified position (absolute coordinate values) within a 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_jump_abs_3d has the same effect as timed_jump_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>timed_jump_abs_3d(X, Y, Z, T)</code>
Parameters	X Like jump_abs_3d .
	Y Like jump_abs_3d .
	Z Like jump_abs_3d .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_jump_abs_3d behaves like jump_abs_3d .
Comments	- Except for the additional motion in the third dimension, timed_jump_abs_3d functions similarly to the timed_jump_abs command (see the comments there). - See also comments on jump_abs_3d.
RTC4→RTC5	New command. See jump_abs_3d .
Version info	–
References	timed_jump_abs , jump_abs_3d , timed_jump_rel_3d

Normal List Command	timed_jump_rel
Function	Moves the output point (of the laser focus) for the specified jump duration along a 2D vector from the current position to the specified position (relative coordinate values) within a 2D Image field .
Call	<code>timed_jump_rel(dX, dY, T)</code>
Parameters	dX Like jump_rel .
	dY Like jump_rel .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_jump_rel behaves like jump_rel .
Comments	The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_jump_rel is identical to timed_jump_abs (see the comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. See jump_rel .
Version info	–
References	timed_jump_abs , jump_rel , timed_jump_rel_3d

Normal List Command	timed_jump_rel_3d
Function	Moves the output point (of the laser focus) for the specified jump duration along a 3D vector from the current position to the specified position (relative coordinate values) within a 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_jump_rel_3d has the same effect as timed_jump_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>timed_jump_rel_3d(dX, dY, dZ, T)</code>
Parameters	dX Like jump_rel_3d .
	dY Like jump_rel_3d .
	dZ Like jump_rel_3d .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_jump_rel_3d behaves like jump_rel_3d .
Comments	The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_jump_rel_3d is identical to timed_jump_abs_3d (see the comments there).
RTC4→RTC5	New command. See jump_rel_3d .
Version info	–
References	timed_jump_abs_3d , jump_rel_3d , timed_jump_rel

Normal List Command	timed_mark_abs
Function	Moves the laser focus for the specified marking duration along a 2D vector from the current position to the specified position (absolute coordinate values) within a 2D Image field .
Call	<code>timed_mark_abs(X, Y, T)</code>
Parameters	X Like mark_abs .
	Y Like mark_abs .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_mark_abs behaves like mark_abs .
Comments	- Unlike mark_abs, the timed_mark_abs command does not execute the marking process with the specified (by set_mark_speed or set_mark_speed_ctrl) mark speed. Instead, the speed (that is, the number of Microsteps) is adjusted so that the vector lasts as long as specified (see Chapter 8.9 "Timed Commands", Page 250). The total marking time is (for $T \geq 5$) the sum of the specified (rounded) time and the set delays. - See also comments on mark_abs.
RTC4→RTC5	Unchanged functionality. In addition: increased value range. See mark_abs .
Version info	–
References	mark_abs , timed_mark_rel , timed_mark_abs_3d

Normal List Command	timed_mark_abs_3d
Function	Moves the laser focus for the specified marking duration along a 3D vector from the current position to the specified position (absolute coordinate values) within a 3D image field.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_mark_abs_3d has the same effect as timed_mark_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	<code>timed_mark_abs_3d(X, Y, Z, T)</code>
Parameters	X Like mark_abs_3d .
	Y Like mark_abs_3d .
	Z Like mark_abs_3d .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_mark_abs_3d behaves like mark_abs_3d .
Comments	- Except for the additional motion in the third dimension, timed_mark_abs_3d functions similarly to timed_mark_abs (see the comments there). - See also comments on mark_abs_3d.
RTC4→RTC5	New command. See mark_abs_3d .
Version info	–
References	timed_mark_abs , mark_abs_3d , timed_mark_rel_3d

Normal List Command	timed_mark_rel
Function	Moves the laser focus for the specified marking duration along a 2D vector from the current position to the specified position (relative coordinate values) within a 2D Image field .
Call	<code>timed_mark_rel(dX, dY, T)</code>
Parameters	dX Like mark_rel .
	dY Like mark_rel .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_mark_rel behaves like mark_rel).
Comments	The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_mark_rel is analogous to timed_mark_abs (see the comments there).
RTC4→RTC5	Unchanged functionality. In addition: increased value range. See mark_rel .
Version info	–
References	timed_mark_abs , mark_rel , timed_mark_rel_3d

Normal List Command	timed_mark_rel_3d
Function	Moves the laser focus for the specified marking duration along a 3D vector from the current position to the specified position (relative coordinate values) within a 3D image field .
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_mark_rel_3d has the same effect as timed_mark_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	<code>timed_mark_rel_3d(dX, dY, dZ, T)</code>
Parameters	dX Like mark_rel_3d .
	dY Like mark_rel_3d .
	dZ Like mark_rel_3d .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_mark_rel_3d behaves like mark_rel_3d .
Comments	The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_mark_rel_3d is identical to timed_mark_abs_3d (see the comments there).
RTC4→RTC5	New command.. See mark_rel_3d .
Version info	–
References	timed_mark_abs_3d , timed_mark_abs , mark_rel_3d , timed_mark_rel

Normal List Command	timed_para_jump_abs
Function	Moves the output point (of the laser focus) for the specified jump duration along a 2D vector from the current position to the specified position (absolute coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>timed_para_jump_abs(X, Y, P, T)</code>
Parameters	X Like para_jump_abs .
	Y Like para_jump_abs .
	P Like para_jump_abs .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_jump_abs behaves like para_jump_abs .
Comments	- Unlike para_jump_abs, the timed_para_jump_abs command does not execute the jump with the specified (by set_jump_speed or set_jump_speed_ctrl) jump speed. Instead, the speed (that is, the number of Microsteps) is adjusted so that the vector lasts as long as specified (see Chapter 8.9 "Timed Commands", Page 250). The total jump time is (for $T \geq 5$) the sum of the specified (rounded) time and the set delays. timed_para_jump_abs requires two list entries. During runtime, the command's part on the first list entry is executed as a short list command and afterward the second part is executed as a normal list command. Thereby, both command parts are executed within the same $10 \mu\text{s}$ clock, unless further (previous) short list commands induce a list_continue between the two parts. - See also comments on para_jump_abs.
RTC4→RTC5	New command. See para_jump_abs .
Version info	–
References	para_jump_abs , timed_jump_abs , jump_abs , timed_para_jump_rel , timed_para_jump_abs_3d

Multiple List Command	timed_para_jump_abs_3d
Function	Moves the output point (of the laser focus) for the specified jump duration along a 3D vector from the current position to the specified position (absolute coordinate values) within a 3D image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_para_jump_abs_3d has the same effect as timed_para_jump_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	timed_para_jump_abs_3d(X, Y, Z, P, T)
Parameters	X Like para_jump_abs_3d .
	Y Like para_jump_abs_3d .
	Z Like para_jump_abs_3d .
	P Like para_jump_abs_3d .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_jump_abs_3d behaves like para_jump_abs_3d .
Comments	- Except for the additional motion in the third dimension, timed_para_jump_abs_3d functions similarly to timed_para_jump_abs (see the comments there). - For $T \geq 5$, timed_para_jump_abs_3d occupies two list storage positions. The initial component executes as an undelayed short list command before the principal part (a normal list command). - See also comments on para_jump_abs_3d.
RTC4→RTC5	New command. See para_jump_abs_3d .
Version info	–
References	timed_para_jump_abs , para_jump_abs_3d , jump_abs_3d , timed_para_jump_rel_3d

Normal List Command	timed_para_jump_rel
Function	Moves the output point (of the laser focus) for the specified jump duration along a 2D vector from the current position to the specified position (relative coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>timed_para_jump_rel(dX, dY, P, T)</code>
Parameters	dX Like para_jump_rel .
	dY Like para_jump_rel .
	P Like para_jump_rel .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_jump_rel behaves like para_jump_rel .
Comments	The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_para_jump_rel is analogous to timed_para_jump_abs (see the comments there).
RTC4→RTC5	New command. See para_jump_rel .
Version info	–
References	timed_para_jump_abs , para_jump_rel , timed_jump_rel , timed_para_jump_rel_3d

Multiple List Command	timed_para_jump_rel_3d
Function	Moves the output point (of the laser focus) for the specified jump duration along a 3D vector from the current position to the specified position (relative coordinate values) within a 3D image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_para_jump_rel_3d has the same effect as timed_para_jump_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective jump speed in the xy plane.
Call	<code>timed_para_jump_rel_3d(dX, dY, dZ, P, T)</code>
Parameters	dX Like para_jump_rel_3d .
	dY Like para_jump_rel_3d .
	dZ Like para_jump_rel_3d .
	P Like para_jump_rel_3d .
	T Duration of the complete jump vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_jump_rel_3d behaves like para_jump_rel_3d .
Comments	- The coordinates for the jump vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_para_jump_rel_3d is analogous to timed_para_jump_abs_3d (see comments there). - For $T \geq 5$, timed_para_jump_rel_3d occupies two list storage positions. The initial component executes as an undelayed short list command before the principal part (a normal list command).
RTC4→RTC5	New command. See para_jump_rel_3d .
Version info	–
References	timed_para_jump_abs_3d , para_jump_rel_3d , timed_jump_rel_3d , timed_para_jump_rel

Normal List Command	timed_para_mark_abs
Function	Moves the laser focus for the specified marking duration along a 2D vector from the current position to the specified position (absolute coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>timed_para_mark_abs(X, Y, P, T)</code>
Parameters	X Like para_mark_abs .
	Y Like para_mark_abs .
	P Like para_mark_abs .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_mark_abs behaves like para_mark_abs .
Comments	- Unlike para_mark_abs, the timed_para_mark_abs command does not execute the marking process with the specified (by set_mark_speed or set_mark_speed_ctrl) mark speed. Instead, the speed (that is, the number of Microsteps) is adjusted so that the vector lasts as long as specified (see Chapter 8.9 "Timed Commands", Page 250). The total marking time is (for $T \geq 5$) the sum of the specified (rounded) time and the set delays. timed_para_mark_abs requires two list entries for $T \geq 5$. During runtime, the command's part on the first list entry is executed as a short list command and afterward the second part is executed as a normal list command. Thereby, both command parts are executed within the same 10 μs clock, unless further (previous) short list commands induce a list_continue between the two parts. - See also comments on para_mark_abs.
RTC4→RTC5	New command. See para_mark_abs .
Version info	–
References	para_mark_abs , timed_mark_abs , mark_abs , timed_para_mark_rel , timed_para_mark_abs_3d

Multiple List Command	timed_para_mark_abs_3d
Function	Moves the laser focus for the specified marking duration along a 3D vector from the current position to the specified position (absolute coordinate values) within a 3D image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_para_mark_abs_3d has the same effect as timed_para_mark_abs . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	<code>timed_para_mark_abs_3d(X, Y, Z, P, T)</code>
Parameters	X Like para_mark_abs_3d .
	Y Like para_mark_abs_3d .
	Z Like para_mark_abs_3d .
	P Like para_mark_abs_3d .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_mark_abs_3d behaves like para_mark_abs_3d .
Comments	- Except for the additional motion in the third dimension, timed_para_mark_abs_3d functions similarly to timed_para_mark_abs (see the comments there). timed_para_mark_abs_3d occupies two list storage positions for $T \geq 5$. The initial component executes as an undelayed short list command before the principal part (a normal list command). - See also comments on para_mark_abs_3d.
RTC4→RTC5	New command. See para_mark_abs_3d .
Version info	–
References	timed_para_mark_abs , para_mark_abs_3d , mark_abs_3d , timed_para_mark_rel_3d

Normal List Command	timed_para_mark_rel
Function	Moves the laser focus for the specified marking duration along a 2D vector from the current position to the specified position (relative coordinate values) within a 2D Image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Call	<code>timed_para_mark_rel(dX, dY, P, T)</code>
Parameters	dX Like para_mark_rel .
	dY Like para_mark_rel .
	P Like para_mark_rel .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_mark_rel behaves like para_mark_rel .
Comments	- The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_para_mark_rel is analogous to timed_para_mark_abs (see the comments there). - See also comments on para_mark_rel.
RTC4→RTC5	New command. See para_mark_rel .
Version info	–
References	timed_para_mark_abs , para_mark_rel , timed_mark_rel , timed_para_mark_rel_3d

Multiple List Command	timed_para_mark_rel_3d
Function	Moves the laser focus for the specified marking duration along a 3D vector from the current position to the specified position (relative coordinate values) within a 3D image field and, simultaneously as well as linearly, changes the signal parameter selected by set_vector_control to the specified value.
Restriction	If the Option "3D" is not enabled or no 3D correction table has been assigned (see select_cor_table), then timed_para_mark_rel_3d has the same effect as timed_para_mark_rel . However, split-up into Microsteps is calculated like a 3D command and hence influences the effective mark speed in the xy plane.
Call	<code>timed_para_mark_rel_3d(dX, dY, dZ, P, T)</code>
Parameters	dX Like para_mark_rel_3d .
	dY Like para_mark_rel_3d .
	dZ Like para_mark_rel_3d .
	P Like para_mark_rel_3d .
	T Duration of the complete mark vector. In μs . As a 64-bit IEEE floating point value. Allowed value range: [0...167,772,160]. The parameter is rounded to an integer-multiple of 10. Out-of-range values are clipped. If $T < 5$, then timed_para_mark_rel_3d behaves like para_mark_rel_3d .
Comments	- The coordinates for the mark vector end point are to be supplied as relative coordinates with respect to the current position. Otherwise, timed_para_mark_rel_3d is analogous to timed_para_mark_abs_3d (see the comments there). timed_para_mark_rel_3d occupies two list storage positions for $T \geq 5$. The initial component executes as an undelayed short list command before the principal part (a normal list command). - See also comments on para_mark_rel_3d.
RTC4→RTC5	New command. See para_mark_rel_3d .
Version info	–
References	timed_para_mark_abs_3d , para_mark_rel_3d , timed_mark_rel_3d , timed_para_mark_rel

Ctrl Command	transform																											
Function	Performs a backward transformation of individual position values.																											
Call	TransformErrorCode = transform(&Sig1, &Sig2, Ptr, Code)																											
Parameters and Returned parameter values	Sig1	Parameter: to-be-transformed position value. As a pointer to a signed 32-bit value. Returned parameter value: transformed position value. As a signed 32-bit value. The input values are overwritten.																										
	Sig2	Like Sig1 (analogously).																										
Parameters	Ptr	Pointer (in C and C++ data type ULONG_PTR, an unsigned 32-bit value or unsigned 64-bit value) to the area of PC main memory to which the correction and transformation settings for backward transformation were previously transferred by upload_transform.																										
	Code	Controls aspects of the backward transformation, particularly which partial transformations to perform: If a partial transformation is <i>not</i> to be performed, then its corresponding bit (#2...#5) should be set to 1. As an unsigned 32-bit value. The parameter's meaning is similar to that of get_transform (Sig1 corresponds to Ptr1 and Sig2 to Ptr2). If Bit #0 = 0, then both supplied position values (Sig1 and Sig2) are backward transformed as xy coordinates: <table border="0"> <tr> <td>Bit #1</td><td>= 0:</td><td>The value supplied by Sig1 is backward transformed as the x coordinate and the value supplied by Sig2 as the y coordinate.</td></tr> <tr> <td></td><td>= 1:</td><td>The value supplied by Sig1 is backward transformed as the y coordinate and the value supplied by Sig2 as the x coordinate.</td></tr> <tr> <td>Bit #2</td><td>= 0:</td><td>The gain/offset correction of automatic self-calibration is backward transformed.</td></tr> <tr> <td>Bit #3</td><td>= 0:</td><td>The Image field correction is backward transformed.</td></tr> <tr> <td>Bit #4</td><td>= 0:</td><td>The offset of the defined coordinate transformation is backward transformed.</td></tr> <tr> <td>Bit #5</td><td>= 0:</td><td>The total matrix of the defined coordinate transformation is backward transformed.</td></tr> <tr> <td>Bit #6</td><td colspan="2">Reserved.</td></tr> <tr> <td>...</td><td colspan="2"></td></tr> <tr> <td>Bit #31</td><td colspan="2"></td></tr> </table>	Bit #1	= 0:	The value supplied by Sig1 is backward transformed as the x coordinate and the value supplied by Sig2 as the y coordinate.		= 1:	The value supplied by Sig1 is backward transformed as the y coordinate and the value supplied by Sig2 as the x coordinate.	Bit #2	= 0:	The gain/offset correction of automatic self-calibration is backward transformed.	Bit #3	= 0:	The Image field correction is backward transformed.	Bit #4	= 0:	The offset of the defined coordinate transformation is backward transformed.	Bit #5	= 0:	The total matrix of the defined coordinate transformation is backward transformed.	Bit #6	Reserved.		...			Bit #31	
Bit #1	= 0:	The value supplied by Sig1 is backward transformed as the x coordinate and the value supplied by Sig2 as the y coordinate.																										
	= 1:	The value supplied by Sig1 is backward transformed as the y coordinate and the value supplied by Sig2 as the x coordinate.																										
Bit #2	= 0:	The gain/offset correction of automatic self-calibration is backward transformed.																										
Bit #3	= 0:	The Image field correction is backward transformed.																										
Bit #4	= 0:	The offset of the defined coordinate transformation is backward transformed.																										
Bit #5	= 0:	The total matrix of the defined coordinate transformation is backward transformed.																										
Bit #6	Reserved.																											
...																												
Bit #31																												

Ctrl Command	transform												
Parameters (cont'd)	Code (cont'd) If Bit #0 = 1, then one of the two supplied position values (specifiable as Sig1 or Sig2) is backward transformed as the z coordinate: Bit #1 = 0: The value supplied by Sig1 is backward transformed as the z coordinate (Sig2 remains unchanged). = 1: The value supplied by Sig2 is backward transformed as the z coordinate (Sig1 remains unchanged). Bit #2 = 0: The offset to the focal length defined by set_defocus or set_defocus_list is backward transformed. Bit #3 = 0: The ABC correction is backward transformed. Bit #4 = 0: The offset to the z coordinate defined by set_offset_xyz or set_offset_xyz_list is backward transformed. Bit #5 Reserved. ... Bit #31												
Result	Error code. As an unsigned 32-bit value. <table> <tr> <th>Value</th><th>Description</th></tr> <tr> <td>0</td><td>Success.</td></tr> <tr> <td>1</td><td>Ptr = NULL (no memory area specified).</td></tr> <tr> <td>2</td><td>No valid data at Ptr (upload_transform did not execute).</td></tr> <tr> <td>3</td><td>Erroneous data at Ptr (a corresponding error indication has been stored by upload_transform).</td></tr> <tr> <td>4</td><td>z axis inversion not possible.</td></tr> </table>	Value	Description	0	Success.	1	Ptr = NULL (no memory area specified).	2	No valid data at Ptr (upload_transform did not execute).	3	Erroneous data at Ptr (a corresponding error indication has been stored by upload_transform).	4	z axis inversion not possible.
Value	Description												
0	Success.												
1	Ptr = NULL (no memory area specified).												
2	No valid data at Ptr (upload_transform did not execute).												
3	Erroneous data at Ptr (a corresponding error indication has been stored by upload_transform).												
4	z axis inversion not possible.												
Comments	- For backward transformation of position values see Chapter 8.1.3 "Monitoring the Positioning", Page 200. - The execution of transform must be preceded by a call to upload_transform. Additionally, position values should have been requested by get_values. - If execution of transform results in an error (returned error code > 0), then no transformation occurs (Sig1 and Sig2 then remain unchanged). Errors also include Ptr = NULL (error code = 1) or errors resulting from prior, erroneous execution of upload_transform (error code = 3). - If backward transformation of z values is requested (Code Bit #0 = 1), but only a 2D correction table has been assigned at the point in time of the prior successful call to upload_transform, then the offsets to the focal length and z coordinates are initialized with 0 and the values A, B and C are initialized with 0, 1, 0 (1-to-1 backward transformation). - For backward transformation of xy position values (Code Bit #0 = 0), only the z = 0 plane is transformed. xy stretching and Z defocus resulting from z deviations (particularly with non-F-Theta systems) are not taken into account.												

Ctrl Command	transform
Comments (cont'd)	- Because transform does not access any RTC5 Boards, calling it does not require explicit access rights to a specific board. If both the upload_transform data and the queried data recorded by get_values or get_waveform have been binarily stored on the PC, then offline operation of transform is also possible (then transform does not require the presence of an RTC5 Board on the PCI bus). transform is not available as a multi-board command. - The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by transform.
RTC4→RTC5	New command. In the RTC4 Compatibility Mode, all back transformed values (including z values) are in the RTC5 20-bit range.
Version info	–
References	upload_transform , get_transform , get_values

Ctrl Command	upload_transform
Function	Transfers from the RTC5 Board to the PC all correction and transformation settings currently assigned to the scan system.
Call	<code>UploadErrorCode = upload_transform(HeadNo, Ptr)</code>
Parameters	HeadNo Number of the scan head connector whose settings should be queried,. As an unsigned 32-bit value. Allowed values: = 1: First scan head connector. = 2: Second scan head connector.
	Ptr Pointer (in C and C++ data type <code>ULONG_PTR</code> , an unsigned 32-bit value or unsigned 64-bit value) to the PC's area of memory that should receive the queried settings.
Result	Error code. As an unsigned 32-bit value. Bit #0 =1: X gain = 0 (gain of automatic self-calibration for Galvanometer scanner 2). Bit #1 =1: Y gain = 0 (gain of automatic self-calibration for Galvanometer scanner 1). Bit #2 =1: The total matrix of the defined coordinate transformation is noninvertable. Bit #3 =1: No correction table assigned. Bit #4 =1: The ABC values (z axis) are noninvertable. Bit #5 =1: Error querying correction table. Bit #6 =1: Parameter error: invalid <code>HeadNo</code> or <code>Ptr</code> = 0. Bit #7 =1: Busy error, board has BUSY list execution status or INTERNAL-BUSY list execution status (get_last_error return code <code>RTC5_BUSY</code>). Bit #8 Reserved. ... Bit #31
Comments	- The queried and transferred data can be used for backward transforming actual position values by transform or get_transform (see also Chapter 8.1.3 "Monitoring the Positioning", Page 200). - For storage of each queried data set, the user program must provide (at an address specified by <code>Ptr</code>) an area of PC main memory equal to 528,520 bytes. - In case of error (except for Bit #6 = 1), an error indication is stored at <code>Ptr</code> to indicate that the data are erroneous. transform and get_transform recognize this error information and ensure that the backward transformation is not executed (transform then generates a corresponding error code, get_transform generates a get_last_error return code <code>RTC5_PARAM_ERROR</code>).

Ctrl Command	upload_transform
Comments (cont'd)	• upload_transform is not executed (get_last_error return code RTC5_BUSY), if: – the BUSY list execution status is set – the INTERNAL-BUSY list execution status is set • upload_transform is even executed, if: – a list has been paused by set_wait (PAUSED list execution status set) • During the runtime of upload_transform, External Starts are suppressed.
RTC4→RTC5	New command.
Version info	–
References	transform , get_transform

Ctrl Command	verify_checksum
Function	Checks or creates the checksum of a correction file.
Call	<code>verify_checksum(Name)</code>
Parameters	Name Name of the correction file. As a pointer to a null-terminated ANSI string.
Result	Result of the test. As an unsigned 32-bit value. = 0: No error (the tested checksum is OK.). = 1: A checksum has been newly created (no info about file integrity). = 2: The tested checksum is incorrect. = 3: A checksum could not be determined (file error, etc.).
Comments	• Verification of correction file downloads only works for files that contain a checksum (see Loading of correction files, Page 120 and set_verify). • verify_checksum is available even without explicit access rights to a particular RTC5 Board. • verify_checksum is not available as a multi-board command. • The programs <code>CorrectionFileConverter.exe</code> (version 1.04) and <code>correXion5.exe</code> (version 1.01) together with <code>RTC5Base.dll</code> (version 1.0.0.4) already automatically create checksums for the output files. • The board-specific error variables <code>LastError</code> and <code>AccError</code> (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by verify_checksum.
RTC4→RTC5	New command.
Version info	–
References	set_verify

Normal List Command	wait_for_encoder
Function	Waits until the selected encoder counter has overstepped or understepped the specified count for the first time.
Call	<code>wait_for_encoder(Value, EncoderNo)</code>
Parameters	Value Count. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$
	EncoderNo Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".
Comments	• wait_for_encoder is synonymous with wait_for_encoder_mode with parameter Mode = 0 (see comments there).
RTC4→RTC5	New command.
Version info	–
References	wait_for_encoder_mode

Multiple List Command	wait_for_encoder_in_range
Function	Waits until both encoder counters simultaneously lie within the specified range (including limits).
Call	<code>wait_for_encoder_in_range(EncXmin, EncXmax, EncYmin, EncYmax)</code>
Parameters	EncXmin Limit value. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31} - 1)]$.
	EncXmax Like EncXmin (analogously).
	EncYmin Like EncXmin (analogously).
	EncYmax Like EncXmin (analogously).
Comments	- For usage of wait_for_encoder_in_range, see Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274 and Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237. wait_for_encoder_in_range requires two list memory positions. The first part is executed as an undelayed short list command prior to the second part, which executes as a normal list command. Any pending delayed short list commands execute first. - If $\text{EncXmin} > \text{EncXmax}$, then both values are interchanged. - If $\text{EncYmin} > \text{EncYmax}$, then both values are interchanged. - If no encoder-based Processing-on-the-fly correction is active, then wait_for_encoder_in_range merely creates a waiting period (without galvanometer scanner motion). - If $\text{EncXmin} = \text{EncXmax}$ (or $\text{EncYmin} = \text{EncYmax}$), then waiting until a specific encoder value is possible. wait_for_encoder_in_range is available even if the Option Processing-on-the-fly is not enabled. wait_for_encoder_in_range does not alter the laser control signals. If you want the laser off during the wait, then this command must be preceded by some other command that switches off the Signals for "Laser Active" Operation (for example, a list_nop). - The active Processing-on-the-fly mode determines whether the galvanometer scanners remain stationary during the wait or move in accordance with encoder changes, see Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237: They move with set_fly_2d, but otherwise remain stationary.
RTC4→RTC5	New command.
Version info	–
References	wait_for_encoder_mode , park_position , park_return

Normal List Command	wait_for_encoder_mode
Function	Waits until the selected encoder counter has overstepped or understepped the specified count for the first time.
Call	<code>wait_for_encoder_mode(Value, EncoderNo, Mode)</code>
Parameters	Value Count. As a signed 32-bit value. Allowed value range: $[-2^{31} \dots + (2^{31}-1)]$
	EncoderNo Number of the to-be-used encoder counter. As an unsigned 32-bit value. Allowed values: = 0: Encoder counter "Encoder0". = 1: Encoder counter "Encoder1".
	Mode Mode. As a signed 32-bit value. = 0: Waits for understepping/overstepping (position dependent). > 0: Waits for overstepping (position independent). < 0: Waits for understepping (position independent).
Comments	- For usage of wait_for_encoder_mode, see Chapter 9.3.3 "Synchronization by Encoder Signals", Page 274. - If Mode = 0, ensure that the size and sign of the parameter Value is appropriate for the counting direction of the selected encoder (for external triggering, this corresponds to the workpiece's direction of motion). If Value > 0, wait_for_encoder_mode for overstepping, otherwise for understepping. If Value is positive and already less than the current encoder count, then wait_for_encoder_mode waits for a complete traversal of the counter (likewise if Value is negative and larger than the current encoder count). At a 1 MHz counter rate, this can take up to approx. 36 minutes! - If Mode ≠ 0, then wait_for_encoder_mode waits for overstepping/understepping of the Value parameter independently of the current position and direction of motion. - If EncoderNo > 1, then wait_for_encoder_mode is replaced with a list_nop (get_last_error return code RTC5_PARAM_ERROR). - If no encoder-based Processing-on-the-fly correction is active, then wait_for_encoder_mode merely creates a waiting period (without galvanometer scanner motion) that allows implementation of an externally triggered synchronization (as an alternative to External Starts, set_wait or if_cond, etc.). wait_for_encoder_mode is available even if the Option Processing-on-the-fly is not enabled. - For Mode = 0, wait_for_encoder_mode is synonymous with wait_for_encoder. wait_for_encoder_mode does not alter the laser control signals. If you want the laser off during the wait, then wait_for_encoder_mode must be preceded by some other command that switches off the Signals for "Laser Active" Operation (for example, a list_nop).

Normal List Command	wait_for_encoder_mode
Comments (cont'd)	The active Processing-on-the-fly mode determines whether the galvanometer scanners remain stationary during the wait or move in accordance with encoder changes (see Chapter 8.6.7 "Synchronizing Processing-on-the-fly Applications", Page 237): They move with set_fly_2d, but otherwise remain stationary.
Version info	–
RTC4→RTC5	New command.
References	get_encoder , store_encoder , read_encoder , simulate_encoder , wait_for_encoder , wait_for_encoder_in_range , park_position , park_return

Normal List Command	wait_for_mcbbsp
Function	Waits until the input value at the McBSP interface has reached, overstepped or understepped the specified value for the first time.
Call	<code>wait_for_mcbbsp(Axis, Value, Mode)</code>
Parameters	Axis Selects which half-word of the input value is used for evaluation (see below). As an unsigned 32-bit value. Allowed values: = 0: lower half-word (x axis, Galvanometer scanner 2) = 1: upper half-word (y axis, Galvanometer scanner 1)
	Value Limit value. As a signed 32-bit value.
	Mode Mode. As a signed 32-bit value. = 0: Waits for equality. > 0: Waits for overstepping. < 0: Waits for understepping.
Comments	- For usage of wait_for_mcbbsp, see Chapter 9.3.4 "Synchronization and Online Positioning by McBSP/SPI Signals", Page 276. wait_for_mcbbsp is comparable to wait_for_encoder_mode, but the McBSP interface is queried. - If set_fly_x_pos and set_fly_y_pos are <i>simultaneously</i> activated, then Value consists of two 16-bit half-words, see Section "Correction via McBSP Interface", Page 230. Only in this case does Axis control which half-word is used for evaluation. In all other cases, Axis is irrelevant and Value is interpreted as a signed 32-bit value. Axis must be either 0 or 1 (even if it is irrelevant). Otherwise, wait_for_mcbbsp is replaced by list_nop (get_last_error return code RTC5_PARAM_ERROR). - If neither set_fly_x_pos, set_fly_y_pos nor set_fly_rot_pos are activated, then wait_for_mcbbsp merely creates a waiting period (without galvanometer scanner motion) that allows implementation of an externally triggered synchronization (as an alternative to External Starts, set_wait or if_cond, etc.). wait_for_mcbbsp is available even if the Option Processing-on-the-fly is not enabled. wait_for_mcbbsp does not alter the laser control signals. If you want the laser off during the wait, then wait_for_mcbbsp must be preceded by some other command that switches off the Signals for "Laser Active" Operation (for example, a list_nop).
RTC4→RTC5	New command.
Version info	–
References	wait_for_encoder_mode

Ctrl Command	write_8bit_port
Function	Writes a value to the 8-bit digital output port on the EXTENSION 2 socket connector.
Call	<code>write_8bit_port(Value)</code>
Parameters	Value 8-bit output value (DATA0...DATA7). As an unsigned 32-bit value. Only the least significant 8 bits are evaluated.
Comments	• See also Chapter 9.1.2 "8-Bit Digital Output Port", Page 259.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	write_8bit_port_list

Delayed Short List Command	write_8bit_port_list
Function	Like write_8bit_port , but a list command.
Call	<code>write_8bit_port_list(Value)</code>
Parameters	Value Like write_8bit_port .
Comments	• Like write_8bit_port.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	write_8bit_port

Ctrl Command	write_abc_to_file	
Function	Writes the ABC values directly into a specified correction file on the PC.	
Call	ErrorNo = write_abc_to_file(Name, A, B, C)	
Parameters	Name	Name of the correction file. As a pointer to a null-terminated ANSI string.
	A	Coefficient A of the parabolic function $z_{out} = A + Bx + Cx^2$ which is used for calculating the Z output values. As a 64-bit IEEE floating point value. Allowed value range:: see load_z_table .
	B	Like A (analogously).
	C	Like A (analogously).
Result	ErrorNo	Error code. As an unsigned 32-bit value.
	0	No error.
	1	A exceeded the maximum allowed value.
	2	A undercut the minimum allowed value.
	4	B exceeded the maximum allowed value.
	8	B undercut the minimum allowed value.
	16	C exceeded the maximum allowed value.
	32	C undercut the minimum allowed value.
	3	File-open error (empty string, file not found etc.).
	12	File error (checksum could not be determined, file corrupt).
Comments	• write_abc_to_file is available even without explicit access rights to a specific RTC5 board. • write_abc_to_file is not available as a multi-board command. • The board-specific error variables LastError and AccError (see Chapter 6.8 "Error Handling", Page 119) are neither generated nor altered by write_abc_to_file. • If the error code is not 0 or 12, the ABC values are not outputted.	
RTC4→RTC5	New command.	
Version info	Available as of DLL 538, OUT 538.	
References	read_abc_from_file	

Ctrl Command	write_da_1
Function	See write_da_x .
Call	write_da_1(Value)
Parameters	Value 12-bit output value for the ANALOG OUT1 analog output port. As an unsigned 32-bit value. Bits with higher significance are ignored. Value = 0 corresponds to an output value of 0 V. Value = $2^{12}-1$ corresponds to an output value of 10 V.
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode : like write_da_x
Version info	–
References	write_da_x

Delayed Short List Command	write_da_1_list
Function	See write_da_x_list .
Call	write_da_1_list(Value)
Parameters	Value Like write_da_1 .
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode : like write_da_x
Version info	–
References	write_da_x_list

Ctrl Command	write_da_2
Function	See write_da_x .
Call	write_da_2(Value)
Parameters	Value 12-bit output value for the ANALOG OUT2 analog output port. As an unsigned 32-bit value. Bits with higher significance are ignored. Value = 0 corresponds to an output value of 0 V. Value = $2^{12}-1$ corresponds to an output value of 10 V.
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode : like write_da_x
Version info	–
References	write_da_x

Delayed Short List Command	write_da_2_list
Function	See write_da_x_list .
Call	write_da_2_list(Value)
Parameters	Value Like write_da_2 .
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode : like write_da_x .
Version info	–
References	write_da_x_list

Ctrl Command	write_da_x
Function	Writes an output value to one of the analog output ports of the RTC5.
Call	<code>write_da_x(x, Value)</code>
Parameters	x Number of the analog output port. As an unsigned 32-bit value. Allowed value range: [1, 2] (1: ANALOG OUT1 , 2: ANALOG OUT2)
	Value 12-bit output value for the selected analog output port. As an unsigned 32-bit value. Bits with higher significance are ignored. Value = 0 corresponds to an output value of 0 V. Value = $2^{12}-1$ corresponds to an output value of 10 V.
Comments	• See also Chapter 9.1.4 "12-Bit Analog Output Port 1 and 2", Page 259. • The output range of the analog output ports is 0 V...10 V. • For $x < 1$ or $x > 2$, write_da_x is ignored (get_last_error return code RTC5_PARAM_ERROR). • write_da_1 / write_da_2 can be used alternatively to write_da_x (without parameter x).
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode , the RTC5 multiplies the specified Value by 4.
Version info	–
References	write_da_x_list

Delayed Short List Command	write_da_x_list
Function	Like write_da_x , but a list command.
Call	<code>write_da_x_list(x, Value)</code>
Parameters	x Number of the analog output port. As an unsigned 32-bit value. Allowed value range: [1, 2] (1: ANALOG OUT1 , 2: ANALOG OUT2)
	Value 12-bit output value for the selected analog output port. As an unsigned 32-bit value. Bits with higher significance are ignored. Value = 0 corresponds to an output value of 0 V. Value = $2^{12}-1$ corresponds to an output value of 10 V.
Comments	For $x < 1$ or $x > 2$, write_da_x_list is replaced with a list_nop (get_last_error return code RTC5_PARAM_ERROR).
RTC4→RTC5	Unchanged functionality. In RTC4 Compatibility Mode : like write_da_x .
Version info	–
References	write_da_x

Ctrl Command	write_hi_pos
Function	Writes the specified values to the EEPROM to be used as ASC reference values (= Home-In reference positions, not to confuse with Home-In positions).
Call	Result = write_hi_pos (HeadNo, X1, X2, Y1, Y2)
Parameters	HeadNo Number of the scan head connector. As an unsigned 32-bit value. Allowed values: = 1: First scan head connector. = 2: Second scan head connector.
	X1 X1 reference position. In bits. As a signed 32-bit value.
	X2 Like X1 (analogously).
	Y1 Like X1 (analogously).
	Y2 Like X1 (analogously).
Result	Error code. As an unsigned 32-bit value. 0 Kein Fehler. Bit #0 =1: Wrong HeadNo. Bit #1 =1: Wrong sensor position for X1. Bit #2 =1: Wrong sensor position for X2. Bit #3 =1: Wrong sensor position for Y1. Bit #4 =1: Wrong sensor position for Y2. Bit #5 =1: Invalid ASC version.
Comments	• write_hi_pos is used in cases when a scan head is moved from RTC5 Board "A" to RTC5 Board "B" in order to transfer its Home-In reference positions from RTC5 Board "A" to RTC5 Board "B". • Use <code>get_hi_pos(HeadNo)</code> to read out the Home-In reference positions of RTC5 Board "A" (only after init_rtc5_dll, see get_hi_pos). • write_hi_pos is also executed if the scan head is switched off or even a scan head is not attached. • With an ASC type-1 equipped scan head attached, <code>auto_cal(Command = 4)</code> (or <code>auto_cal(Command = 0)</code>) because this command implicitly executes <code>auto_cal(Command = 4)</code> must have been successfully executed at last on the RTC5 Board "B". A cross-check with <code>get_auto_cal(HeadNo)</code> needs to return 100. Otherwise, write_hi_pos would return the error Bit #5 = 1. If required, connect the scan head to RTC5 Board "B" and execute <code>auto_cal(HeadNo, 4)</code> before executing write_hi_pos.
RTC4→RTC5	New command.
Version info	Available as of DLL 538, OUT 538.
References	get_hi_pos , get_auto_cal , auto_cal

Ctrl Command	write_io_port
Function	Writes a value to the 16-bit digital output port on the EXTENSION 1 socket connector.
Call	<code>write_io_port(Value)</code>
Parameters	Value 16-bit output value (DIGITAL OUT0...DIGITAL OUT15). As an unsigned 32-bit value. Only the least significant 16 bits are evaluated.
Comments	• Use set_io_cond_list and clear_io_cond_list to set or clear individual bits of the 16-bit digital output port, depending on the state of the digital <i>input</i> port. • write_io_port_mask and write_io_port_mask_list also allow changing selectable bits of the 16-bit digital output port. • See also Chapter 9.1.1 "16-Bit Digital Output Port", Page 258.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	write_io_port_list , write_io_port_mask , write_io_port_mask_list , set_io_cond_list , clear_io_cond_list , get_io_status

Delayed Short List Command	write_io_port_list
Function	Like write_io_port , but a list command.
Call	<code>write_io_port_list(Value)</code>
Parameters	Value Like write_io_port .
Comments	• See write_io_port.
RTC4→RTC5	Unchanged functionality.
Version info	–
References	write_io_port

Ctrl Command	write_io_port_mask	
Function	Writes those bits of Value specified by the Mask parameter to the 16-bit digital output port on the EXTENSION 1 socket connector.	
Call	write_io_port_mask(Value , Mask)	
Parameters	Value	16-bit output value (DIGITAL OUT0...DIGITAL OUT15). As an unsigned 32-bit value. Only the least significant 16 bits are evaluated.
	Mask	16-bit mask (for DIGITAL OUT0...DIGITAL OUT15). As an unsigned 32-bit value. Only the least significant 16 bits are evaluated.
Comments	- The Mask parameter determines <i>which</i> bits of the 16-bit digital output port are to be altered, the Value parameter determines <i>how</i> they are altered. All bits of the 16-bit digital output port that were not set in Mask remain unaltered, that is, they are outputted again as previously. - For Mask = 0xFFFF, write_io_port_mask behaves like write_io_port. - See also Chapter 9.1.1 "16-Bit Digital Output Port", Page 258.	
RTC4→RTC5	New command.	
Version info	–	
References	write_io_port , write_io_port_mask_list	

Delayed Short List Command	write_io_port_mask_list	
Function	Like write_io_port_mask , but a list command.	
Call	write_io_port_mask_list(Value , Mask)	
Parameters	Value	Like write_io_port_mask .
	Mask	Like write_io_port_mask .
Comments	See write_io_port_mask.	
RTC4→RTC5	New command.	
Version info	–	
References	write_io_port_mask , write_io_port_list	

10.3 Unsupported RTC2/RTC3/RTC4 Commands

Some RTC3/RTC4 commands and a few RTC2 commands emulated by the RTC3/RTC4 are not supported by the RTC5. But most of these can be replaced by RTC5 commands. The following table lists all unsupported commands and appropriate RTC5 replacements.

Ctrl Command	aut_change
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	auto_change

Ctrl Command	dsp_start
Support status	This RTC2/RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	This command is not needed for normal operation. The DSP starts automatically after the program file is loaded by load_program_file .

List Command	field_jump
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	home_position , the “field jump” is automatically executed.

Ctrl Command	get_rtc2_mode
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	The RTC5 has no need to query the mode, which is now be set by the software command set_laser_mode instead of hardware.

Ctrl Command	get_rtc2_version
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	get_rtc_version

Ctrl Command	get_version
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	get_hex_version

Ctrl Command	get_xy_pos
Support status	This RTC2/RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	Replaced by get_value

Ctrl Command	get_xyz_pos
Support status	This RTC2/RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	Replaced by get_value

List Command	home_jump
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	home_position , the “home jump” is automatically executed.

List Command	laser_on
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	laser_on_list

Ctrl Command	load_cor
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	load_correction_file

Ctrl Command	load_pro
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	load_program_file

List Command	pola_abs, polb_abs, polc_abs
Support status	These RTC2 commands are not supported by the RTC5.
Replaced by	mark_abs

Ctrl Command	read_pixel_ad
Support status	This RTC2/RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	There is no equivalent command for the RTC5.

Ctrl Command	rtc3_count_cards
Support status	This RTC3 command is not supported by the RTC5.
Replaced by	rtc5_count_cards

Ctrl Command	rtc4_count_cards
Support status	This RTC4 command is not supported by the RTC5.
Replaced by	rtc5_count_cards

Ctrl Command	select_list
Support status	This RTC2/RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	select_list(0) can be replaced by set_extstartpos (0), select_list (1) by set_extstartpos (Mem1). Thereby, Mem1 is the memory size of "List 1" (and the absolute start address of "List 2"). After initialization (with load_program_file), Mem1 is 4000, otherwise as set by config_list . Mem1 can be determined (for example, after a board changed "ownership") by set_start_list_2 and get_input_pointer .

Ctrl Command	set_base
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	There is no equivalent command for the RTC5 (because it is a plug-and-play board whose base address is automatically set).

Ctrl Command	set_co2_standby
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	set_standby

List Command	set_co2_standby_list
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	set_standby_list

List Command	set_delays
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	set_laser_delays, set_scanner_delays

List Command	set_encoder_values
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	set_fly_x, set_fly_y

Ctrl Command	set_gain
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	set_matrix, set_offset

Ctrl Command	set_list_mode
Support status	This RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	See Chapter 6.5.4 "RTC4 Circular Queue Mode", Page 110.

Ctrl Command	set_mode
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	There is no equivalent command for the RTC5, because its Image field correction algorithm is always enabled. Instead, a 1to1 correction file can be loaded.

Ctrl Command	set_piso_control
Support status	This RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	set_piso_control is not needed for operation of the RTC5. For data transfer in accordance with the SL2-100 protocol, the bi-directional communication between the scan system and the RTC5 does not depend on the data cable length. For data transfer in accordance with the XY2-100 protocol, the timing of the communication must be further on adjusted to reflect the length of the data cable; but the adjustment is now realized by a jumper at the XY2-100 Converter (Accessory).

Ctrl Command	set_speed
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	set_jump_speed, set_mark_speed

Ctrl Command	set_wobble_xy
Support status	This RTC4 command is not supported by the RTC5.
Replaced by	set_wobble, set_wobble_mode

Ctrl Command	set_yag_parameter
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	There is no equivalent command for the RTC5.

Ctrl Command	write_da
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	write_da_1

List Command	write_da_list
Support status	This RTC2 command is not supported by the RTC5.
Replaced by	write_da_1_list

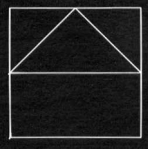
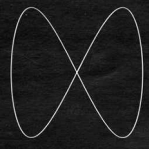
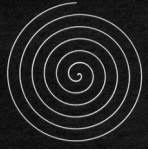
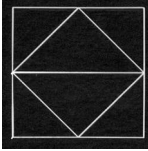
Ctrl Command	z_out
Support status	This RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	For setting the z value in a 3-axis scan system, set_defocus can be used (only together with an RTC5 with enabled Option "3D"). After load_z_table (0.0, 1.0, 0.0), set_defocus (z) has the same effect as z_out (z) (see also set_offset_xyz).

List Command	z_out_list
Support status	This RTC3/RTC4 command is not supported by the RTC5.
RTC4→RTC5	After load_z_table (0.0, 1.0, 0.0), set_defocus_list (z) has the same effect as z_out_list (z) (see also set_offset_xyz_list).

11 Demo Programs

The RTC5 software package contains various program code samples in the **Folder DemoFiles**. They demonstrate **RTC5 DLL** and RTC5 initialization, error handling and usage of the diverse control commands and list commands.

They contain the required calling sequences of RTC5 commands, which you can easily translate into your preferred programming language. The following table summarizes the characteristics of the demo programs.

File name	Demo1	Demo2	Demo3	Demo4	Demo5	Demo6	Demo7
Marking Task	Square and triangle 	Lissajous figures 	Archimedean spirals 	Raster image reproduction 	Squares and triangles 	Raster image reproduction, Processing-on-the-fly	Text output, intelliSCAN
Laser type	CO ₂	YAG	YAG	CO ₂	YAG	CO ₂	YAG
Marking method	vector	vector	vector	Pixel mode	vector	Pixel mode, fly mode	vector
Additional features			home jump		home jump	data recording, time measurement, 16-bit IO port	data recording, time measurement, iDRIVE functions
DLL linking	implicit	explicit	explicit	explicit	explicit	implicit	implicit
List handling	use of a single list	single list, continuous transfer (circular queue)	two alternating lists, continuous transfer	two alternating lists, continuous transfer	use of two lists	two alternating lists, continuous transfer	two lists (default), protected area
External control inputs					/START, /STOP	allowed	
Exception handling	load_list	get_status, set_wait, release_wait, pause_list, restart_list, stop_execution	get_status, load_list, pause_list, restart_list, stop_execution	load_list	get_status, load_list, stop_execution	get_status, load_list, stop_execution	get_status
Other commands	config_list, set_end_of_list, execute_list_pos	config_list, set_start_list_pos, execute_list_pos, set_end_of_list	config_list, set_start_list_pos, set_end_of_list, execute_list_pos, auto_change	config_list, set_end_of_list, execute_list_pos, auto_change, set_pixel_line, set_n_pixel	config_list, set_end_of_list, execute_list_pos, set_extstartpos, set_control_mode, get_startstop_info, bounce_supp	config_list, set_end_of_list, execute_list_pos, auto_change, set_control_mode, simulate_encoder, set_pixel_line, set_n_pixel, set_fly_x, fly_return, set_trigger, save_and_restart_timer, write_io_port_list, get_io_status	control_command, load_char, mark_text, mark_text_abs, set_start_list_pos, set_trigger, measurement_status, get_waveform, set_start_list_pos, save_and_restart_timer, set_end_of_list, execute_list_pos

12 Troubleshooting

Problem	Remedy
PC does not boot	Switch off the PC and check the following: • Check if the RTC5 Board is correctly seated in the PCI slot. Refer to the instructions in your PC manual. • Check for metal parts that may have fallen into the PC housing during installation of the RTC5. • Check for loose cables or connectors
RTC5 does not respond	• Check the RTC5 board driver installation. See Chapter 5.4 "Installing the RTC5 Software", Page 80. • Use the included HPGL converter program to check if the board can be properly accessed. If not: Check the cable type and the cable length. If the scan system is controlled by an XY2-100 Converter (Accessory), then check, whether the solder jumpers of the XY2-100 Converter (Accessory) are set appropriate for the cable length. See Figure 14. If yes: Check the RTC5 DLL import declarations in your user program. See Chapter 6.2.2 "Importing Commands", Page 84.
User program fails	• Check the driver installation. See Chapter 5.4 "Installing the RTC5 Software", Page 80. • Check the RTC5 initialization in the user program. • Check the RTC5 DLL import declarations in your user program. See Chapter 6.2.2 "Importing Commands", Page 84.
Scan head control fails	• Check if the scan head is properly connected to the RTC5 by the data cable. Make sure to follow the specifications for the data cable. See Chapter 4.5.3 "Data Cables (Accessories)", Page 58. • Check the power supply of the scan head. Refer to your scan head operating manual. • Check your user program.
Laser control fails	• Check the interface between the RTC5 and the laser.
Irregular marking results	• Check the laser and Scanner Delays. See Chapter 7.2.3 "Notes on Optimizing the Delays", Page 143.
Laser does not switch off during Jump commands	• Check the laser and Scanner Delays. See Chapter 7.2 "Delay Settings – Coordinating Scan Head Control and Laser Control", Page 133.

Additionally, the following commands can be helpful for troubleshooting:

- With **get_error** and **get_last_error**, nearly any command can be checked for proper execution (see [Chapter 6.8 "Error Handling", Page 119](#)).
- With **set_verify**, you can verify that all downloads (commands, tables) were performed error-free (see [Chapter 6.8.1 "Download Verification", Page 120](#)).
- With **get_value** or **get_values**, you can query specific values returned from the scan-system. With **set_trigger/set_trigger4** and **get_waveform**, you can record an entire series of returned values (see [Chapter 7.3.7 "Status Monitoring and Diagnostics", Page 172](#); for iDRIVE scan systems see also [Chapter 8.1 "iDRIVE Functions", Page 198](#)).
- With **set_wait** and **get_wait_status**, you can check if program branches (conditional jumps) executed as intended.
- With **get_overnrun**, you can check if overruns of the 10 μ s clock cycle occurred (see [Section "Clock Overruns", Page 171](#)).
- With **get_status** or **get_out_pointer**, you can determine which command number the program is currently executing (for example, for "infinite loops" due to a circular argument in the program flow).
- **get_startstop_info** provides information about the laser signals and possible transmission errors to and from the attached scan system.

If specific outputs from a port have no effect, then check if the user program is performing directly consecutive accesses of that same port. Because so-called short list commands (see [page 278](#)) are typically used for this, one command might overwrite the other's output value within the same 10 μ s clock cycle. In this situation, separate both short commands with a (normal) list command, for example, **list_nop** or **list_continue**.

If the execution time (measured by **save_and_restart_timer** and **get_time**) does not correspond with your calculation, check if your user program contains so-called short list commands, which generally do not require their own clock cycle for execution. Another possibility is that additional **Scanner Delays** were automatically inserted to prevent (improper) overlap of LaserOn and LaserOff (see [Section "Automatic Delay Adjustments", Page 143](#)).

If the problems persist, contact SCANLAB.

13 Customer Service

13.1 Servicing and Repairs

All servicing and repairs should be performed only at SCANLAB. The warranty expires if the board has been altered.

13.2 Warranty

SCANLAB guarantees this product to be free of defects in manufacturing and material. The warranty is valid for 12 months after delivery. Repairs covered under the warranty are performed at SCANLAB.

The scope of the warranty is limited to repair or replacement of the SCANLAB product.

SCANLAB is responsible for the return delivery of products repaired under warranty; the customer is responsible for delivery to SCANLAB.

SCANLAB is not held responsible:

- when the product has been damaged through misuse or improper operation
- for repairs not performed by SCANLAB
- if the RTC5 Board has been altered
- for damage resulting from improper packaging of a product returned to SCANLAB
- for consequential damages

13.3 Contacting SCANLAB

For service, repairs, advice or information, simply contact SCANLAB using one of the contact possibilities listed below:

SCANLAB GmbH
Siemensstr. 2a
82178 Puchheim
Germany

Tel. +49 (89) 800 746-0
Fax: +49 (89) 800 746-199

info@scanlab.de
www.scanlab.de

13.4 Product Disposal

The RTC5 can be returned to SCANLAB for a fee to be properly disposed of.

14 Legal

14.1 EU Declaration of Conformity – RTC5 PCI Board

Konformitätserklärung – Declaration of Conformity

EU-Konformitätserklärung

für das Produkt:

RTC4 (PCI); RTC4 PCI-Express; RTC5 (PCI); RTC5 PCI-Express; RTC6 PCI-Express

Der Hersteller

SCANLAB GmbH, Siemensstr. 2a, 82178 Puchheim, Deutschland

erklärt, dass das genannte Produkt die einschlägigen Harmonisierungsrechtsvorschriften der Union erfüllt:

- **2014/30/EU** - Richtlinie des EUROPÄISCHEN PARLAMENTS UND DES RATES zur Harmonisierung der Rechtsvorschriften der Mitgliedstaaten über die elektromagnetische Verträglichkeit (EMV-Richtlinie).
- **2011/65/EU** - Richtlinie des EUROPÄISCHEN PARLAMENTS UND DES RATES zur Beschränkung der Verwendung bestimmter gefährlicher Stoffe in Elektro- und Elektronikgeräten (RoHS-Richtlinie).

Folgende harmonisierte Normen wurden angewandt:

- **EN IEC 61000-6-2:2005** – Elektromagnetische Verträglichkeit (EMV) - Teil 6-2: Fachgrundnormen - Störfestigkeit für Industriebereiche
- **DIN EN 55011:2018-05** - Industrielle, wissenschaftliche und medizinische Geräte - Funkstörungen - Grenzwerte und Messverfahren
- **DIN EN IEC 63000:2019-05** - Technische Dokumentation zur Beurteilung von Elektro- und Elektronikgeräten hinsichtlich der Beschränkung gefährlicher Stoffe

Die alleinige Verantwortung für die Ausstellung dieser Konformitätserklärung trägt der Hersteller.

Unterzeichnet für und im Namen der SCANLAB GmbH.

EU Declaration of Conformity

for the product:

RTC4 (PCI); RTC4 PCI-Express; RTC5 (PCI); RTC5 PCI-Express; RTC6 PCI-Express

The manufacturer

SCANLAB GmbH, Siemensstr. 2a, 82178 Puchheim, Germany

declares that the product mentioned above complies with the relevant Union harmonization legislation:

- **2014/30/EU** - Directive of the EUROPEAN PARLIAMENT AND OF THE COUNCIL on the harmonisation of the laws of the Member States relating to electromagnetic compatibility (EMC Directive)
- **2011/65/EU** - Directive of the EUROPEAN PARLIAMENT AND OF THE COUNCIL on the restriction of the use of certain hazardous substances in electrical and electronic equipment (RoHS Directive).

The following harmonized standards have been applied:

- **EN IEC 61000-6-2:2005** - Electromagnetic compatibility (EMC) - Part 6-2: Generic standards - Immunity for industrial environments
- **DIN EN 55011:2018-05** - Industrial, scientific and medical equipment – Radio-frequency disturbance characteristics - Limits and methods of measurement
- **DIN EN IEC 63000:2019-05** - Technical documentation for the assessment of electrical and electronic products with respect to the restriction of hazardous substances

This declaration of conformity is issued under the sole responsibility of the manufacturer.

Signed for and on behalf of SCANLAB GmbH.

Puchheim, 26.02.2025



Christian Sonner, CTO

02/2025



14.2 Compliance with FCC Rules

The RTC5 PCI Board has been tested and found to comply with the limits for a Class A digital device, pursuant to part 15 of the FCC rules.

These limits are designed to provide reasonable protection against harmful interference when the RTC5 Board is operated in a commercial environment. The RTC5 Board generates, uses, and can radiate radio frequency energy and, if not installed and used in accordance with this instruction manual, may cause harmful interference to radio communications. Operation of the RTC5 Board in a residential area is likely to cause harmful interference in which case the user is required to correct the interference at his own expense.

15 Technical Specifications – RTC5 PCI Board

System Requirements

Windows PC with PCI bus interface

Operating system Microsoft Windows 10, 8, 7 as 32-bit or 64-bit version

Interfaces

SCANHEAD Connector

Connector 9-pin D-SUB connector (female)

Dimensions

Length 160 mm

Height 100 mm

2. SCANHEAD Socket Connector

Connector 10-pin socket connector.

- xy output values only with enabled Option "Second Scan Head Control"

Scan System Control

Number of list memory areas Up to 3 (configurable)

Capacity of list memory area 1000000 commands

Output interval of Microsteps 10 μ s

Maximum range for the Image field coordinates –524,288...
+524,287
(20-bit signed = 21 bit)

Virtual Image field (for example, for Processing-on-the-fly) –8,388,608 to
+8,388,607
(24-bit signed)

Signal transmission SL2-100 protocol.
Via XY2-100 Converter (Accessory):
XY2-100 protocol.

LASER Connector

Connector	15-pin D-SUB connector, female	Inputs for external start and stop signals	TTL active-LOW, internally connected to +3.3 V by pull-up resistors (4.7 kΩ)
Laser control signals	LASER1, LASER2, LASERON	• /START	edge sensitive HIGH level > 2.3 V LOW level < 0.6 V
• TTL level	5 V, active-HIGH or active-LOW (programmable)	• /STOP	level sensitive HIGH level > 2.3 V LOW level < 0.6 V
• Max. current load	20 mA	• Reference	GND ^(a)
• Reference	GND ^(a) (GND2 with Option "DC/DC Converter")	Other signals	
Analog outputs	ANALOG OUT1, ANALOG OUT2	• BUSY OUT • +5 V • Reference	5 V, TTL active-HIGH, max. 10 mA max. 100 mA GND ^(a)
• Output voltage range	0 V...10 V		
• Resolution	12 Bit		
• Max. current load	5 mA		
• Reference	GND ^(a)		
Digital input	2 Bits		
• LOW level	< 0.6 V		
• HIGH level	> 2.3 V		
• Max. input voltage range	-0.5 V...+5.5 V		
• Input resistance (pull-up)	> 4.7 kΩ		
• Reference	GND ^(a)		
Digital output	2 bits, buffered		
• LOW level	< 0.55 V		
• HIGH level	> 3.8 V		
• Max. current load	20 mA		
• Reference	GND ^(a)		

(a) See footnote on [page 60](#).

EXTENSION 1 Socket Connector

Connector	40-pin socket connector, output signal level configurable (by JP1)
Digital output	16 bits, buffered
• LOW level	< 0.4 V
• HIGH level	> 2.0 V (3.3 V or 5 V)
• Max. current load	±8 mA
• Reference	GND ^(a)
• LATCH signal	3.3 V or 5 V, TTL active-HIGH, 5 μs pulse, max. 10 mA
Digital input	16 bits
• LOW level	< 0.5 V
• HIGH level	> 2.6 V...24 V
• Max. input voltage range	−0.5 V...+26 V
• Input resistance	> 10 kΩ
• Reference	GND ^(a)
• SYNC signal	3.3 V or 5 V, TTL active-HIGH, square wave signal (5 μs pulse, 10 μs period) max. 10 mA
Other signals	
• BUSY OUT	3.3 V or 5 V, TTL active-HIGH, max. 10 mA
• VCC	3.3 V or 5 V, max. 100 mA
• +5 V	max. 100 mA
• Reference	GND ^(a)

EXTENSION 2 Socket Connector

Connector	26-pin socket connector, configurable (by JP2...JP8)
Digital output	8 bits, buffered
• LOW level	< 0.4 V
• HIGH level	> 2.0 V
• Max. current load	±8 mA
• Reference	GND ^(a)
• LATCH signal	5 V, TTL active-HIGH, 5 μs pulse, max. 10 mA
Laser signals	LASERON, LASER1, LASER2 (see LASER connector)
Other signals	
• +5 V	max. 100 mA
• Reference	GND ^(a)

MARKING ON THE FLY Socket Connector

Connector	16-pin socket connector
2 encoder input ports for incremental encoders	ENCODER X (1±,2±) and ENCODER Y (1±,2±), designed for a pair of standardized differential input signals (RS-422) each. HIGH level ≥ 2.0 V LOW level ≤ 0.8 V f ≤ 4 MHz
Analog outputs	ANALOG OUT2 (see LASER connector)
Inputs for external start and stop signals	/START2, /STOP2 (see /START, /STOP of the LASER connector)
Other signals	
• BUSY OUT	5 V, TTL active-HIGH, max. 10 mA
• +5 V	max. 100 mA
• Reference	GND ^(a)

RS232 Socket Connector

Connector	10-pin socket connector
Input	RxD
• max. voltage range	–25 V...+25 V
Output	TxD
• max. voltage range	–13 V...+13 V
Reference	GND ^(a)
Baud rate	300...115200

SPI / I2C Socket Connector

Connector	10-pin socket connector
-----------	-------------------------

In McBSP configuration, see [Section “McBSP Interface”, Page 71](#):

- Transmitter signal level 3.3 V TTL
- Receiver signal level 3.3 V or 5 V TTL
- **McBSP** mode
 - Single Phase Frame
 - Single Element per Frame
 - 32 bits per Element
 - Data delay *XDelay* bits
 - Data delay *RDelay* bits
- Reference GND^(a)

In SPI configuration, see [Section “SPI Interface”, Page 73](#):

- Transmitter signal level 3.3 V TTL
- Receiver signal level 3.3 V or 5 V TTL
- **SPI** mode
 - Single Phase Frame
 - Single Element per Frame
 - 32 bits per Element
 - Data delay 1 bit
- Reference GND^(a)

STEPPER MOTOR Socket Connector

Connector	10-pin socket connector
-----------	-------------------------

Signals for controlling two stepper motors:

- ENABLE and DIRECTION outputs 5 V, TTL
- CLOCK output 5 V, TTL active-HIGH, 5 μ s pulse
- SWITCH input TTL active-LOW, internally connected to +3.3 V by pull-up resistors (10 k Ω)
- Max. current load 25 mA
- Reference GND^(a)

16 Appendix A: RTC5 PCIe Board

16.1 Product Overview

16.1.1 Intended Use – Comparison to the RTC5 PCI Board

Just like the RTC5 PCI Board, the SCANLAB RTC5 PCIe Board is intended for synchronous real-time control of scan systems, lasers and peripheral equipment. It differs from the RTC5 PCI Board in the following hardware-related details:

- The RTC5 PCIe Board provides a PCI-Express interface and must be connected to a PCI-Express slot of the PC.
- The SPI/I²C socket connector provides two 10 V analog inputs instead of an I²C interface (see [Chapter 16.2.4 "SPI/I2C Socket Connector", Page 742](#)).

Otherwise, the RTC5 PCIe Board provides the same functionality for controlling scan systems, lasers and peripheral equipment as the RTC5 PCI Board. Neither the number, type and positioning of connectors for controlling scan systems, lasers and peripheral equipment nor the jumpers for customized configuration differ from that of the RTC5 PCI Board. Specifications for signal inputs and outputs are the same as with the RTC5 PCI Board (with above-mentioned exception), as are software installation and developing user programs. Both the RTC5 PCIe Board and the RTC5 PCI Board use the same RTC5 software package (driver, DLL, import declarations, etc.) and same command set.

16.1.2 System Requirements

Hardware

The RTC5 PCIe Board requires a Windows PC with a PCI-Express bus interface and at least one free PCI-Express slot. The dimensions of the RTC5 Board are shown in [Figure 75](#).

RTC5 PCIe Boards intended for synchronized master/slave operation must be installed in adjacent PCI-Express slots.

Software

The RTC5 PCIe Board uses the same driver and **RTC5 DLL** file as the RTC5 PCI Board (about the supported operating systems see [Chapter 2.5.2 "Software", Page 32](#)).

Notice!

- The RTC5 PCIe Board does not support power-saving modes that switch off power to the PCIe bus. Accordingly, you must disable standby or sleep modes of the operating system. See also [Section "Notes", Page 33](#).

16.1.3 Optional Functionality

The RTC5 PCIe Board can be configured for the same functionality as the RTC5 PCI Board (see [Chapter 2.6 "Options", Page 34](#)). Optional functionality must be enabled by SCANLAB or installed.

16.1.4 Labeling

The serial number of the RTC5 PCIe Board is printed on a label attached to the board (format: "RTC SN...").

The article number and configuration are described in the packaging list (see also [Chapter 2.8 "Accessories for the RTC5 PCI Board", Page 37](#) and [Chapter 16.1.5 "Type Identification", Page 739](#)).

16.1.5 Type Identification

The ID number of the RTC5 PCIe Board reveals the board's factory-equipped jumper configuration (JP1...JP8). The jumper configuration is additionally encoded in a three-digit type code scheme. The type code scheme is identical to that of the RTC5 PCI Board (see [Chapter 2.7 "Jumper Settings and Type Identification", Page 35](#)).

16.1.6 Unpacking Instructions and Typical Scope of Delivery

- (1) Carefully remove the RTC5 PCIe Board from the package.
- (2) Keep the packaging, including the antistatic bag the RTC5 PCIe Board is delivered in, so that in case of repair the board can be properly repackaged and returned to SCANLAB.
- (3) Also remove all other articles from the package. Check that all parts have been delivered. Refer to the corresponding packaging list.
The package typically includes an RTC5 PCIe Board and a data CD (containing the corresponding software (see below) and this manual). Some product versions may also supply additional components, see [Chapter 16.1.7 "Accessories", Page 739](#).

Delivered Software

The delivered software package is identical to the RTC5 software package (see [Chapter 2.3 "Delivered RTC5 Software Package", Page 27](#)). It contains drivers for Microsoft's Windows 10, 8, 7 (32-bit or 64-bit) operating systems, correction, **RTC5 DLL** and program files, as well as utility files for C, C++ and C#.

16.1.7 Accessories

In addition to the RTC5 PCIe Board and its RTC5 software package, the following accessories – as with the RTC5 PCI Board – can be obtained from SCANLAB (only hardware extensions from SCANLAB should be used in combination with the RTC5 PCIe Board):

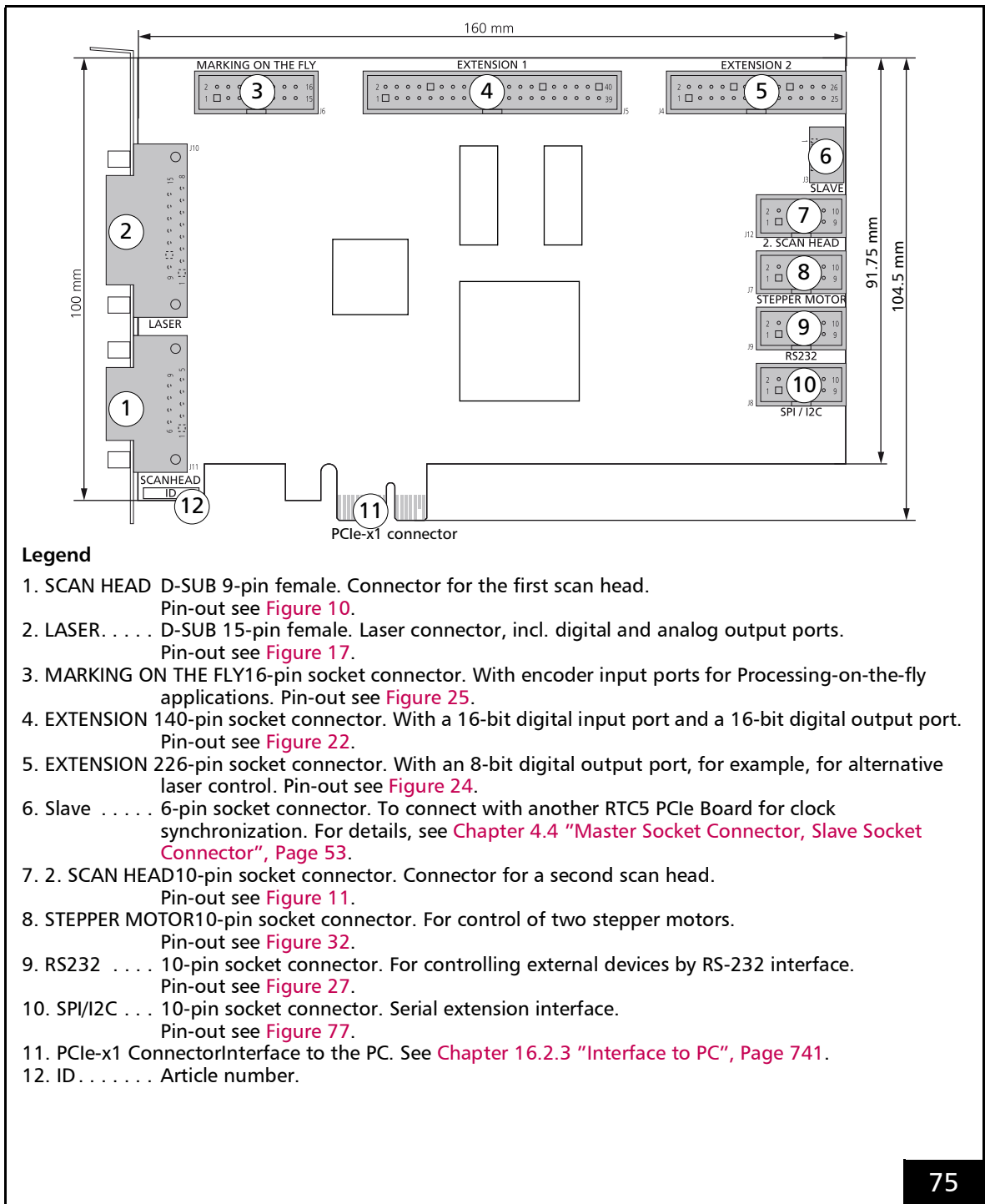
- **XY2-100 Converter (Accessory)**
- Data cables, see [Chapter 2.8.3 "Data Cables", Page 37](#)
- Laser adapter, see [Chapter 2.8.2 "Laser Adapter", Page 37](#)
- Slot cover with a 9-pin D-SUB connector for using the 2. SCAN HEAD socket connector, see [Section "Second Scan Head Slot Cover \(Accessory\)", Page 55](#)
- Slot cover with a 15-pin D-SUB connector for using the MARKING ON THE FLY socket connector, see [Section "MARKING ON THE FLY Slot Cover \(Accessory\)", Page 70](#)

16.1.8 Supplementary Software

To facilitate customizing RTC correction files basing on your own test measurements, SCANLAB offers its correXion line of software and related documentation, see also [Section "Image Field Correction Algorithm", Page 163](#).

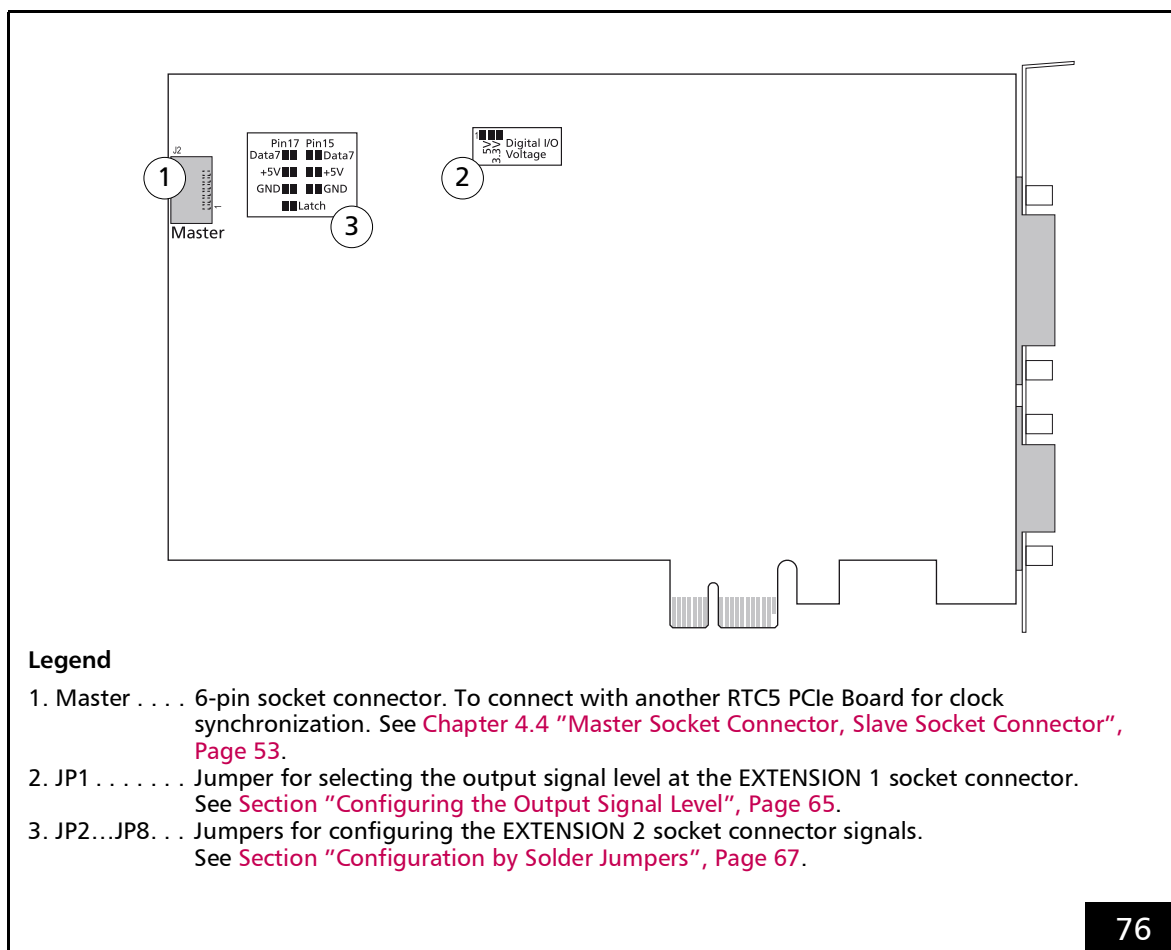
16.2 RTC5 PCIe Board – Layout and Interfaces

16.2.1 Layout – Upper Side



RTC5 PCIe Board: upper side.

16.2.2 Layout – Lower Side



RTC5 PCIe Board: lower side.

16.2.3 Interface to PC

Interface to the PC is the PCIe-x1 connector of the RTC5 PCIe Board.

The RTC5 PCIe Board can be installed into any Windows PC with PCI-Express bus and at least one free PCI-Express slot.

Notice!

- The RTC5 PCIe Board does not support power-saving modes that switch off power to the PCIe bus. Accordingly, you must disable standby or sleep modes of the operating system. See also [Section "Notes", Page 33](#).

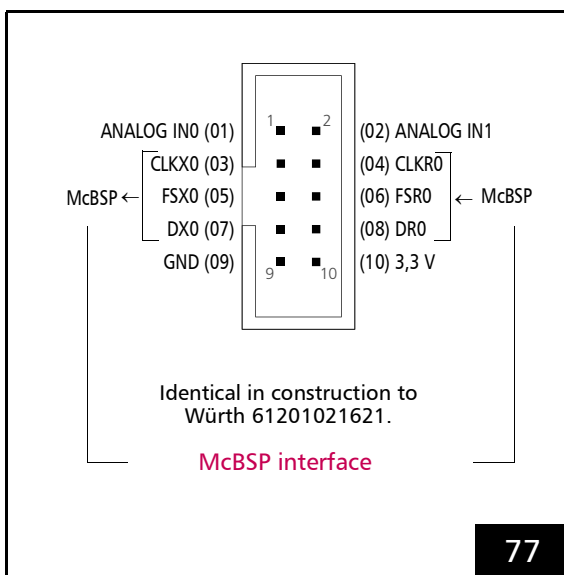
16.2.4 SPI/I2C Socket Connector

The SPI/I2C socket connector has 10 pins. It is located on the upper side of the RTC5 PCIe Board, see [Figure 75](#).

The SPI/I2C socket connector is a hardware interface that

- is designed for bidirectional data exchange (McBSP), see [Section "McBSP Interface", Page 742](#) and
- furthermore, provides two 10 V analog inputs: **ANALOG IN0** at pin (01) and **ANALOG IN1** at pin (02), see [Section "Analog Input Ports", Page 742](#).

The pin-out is shown in [Figure 77](#).



SPI/I2C socket connector: pin-out. The pitch of the pins is 2.54 mm.

McBSP Interface

The **McBSP interface** and the corresponding commands are identical to that of the RTC5 PCI Board (see [Section "McBSP Interface", Page 71](#)).

Analog Input Ports

The RTC5 PCIe Board provides two 10 V analog inputs at the SPI/I2C socket connector (instead of the I²C interface which is provided by the RTC5 PCI Board; similar to the RTC5 PCI Board's optional **ADC Add-On Board**):

- ANALOG IN0
- ANALOG IN1

For using these analog input ports, see [Chapter 4.6.8 "Analog Inputs \(Accessory\)", Page 75](#).

After **read_analog_in** has been called the first time, voltages that are applied there are:

- automatically converted endlessly (duration approx. 0.3 ms)
- transferred to the **DSP** as 12-bit digital values

For technical specifications, see [Analog input ports, Page 746](#).



16.3 Installation and Start-Up

For installing and starting-up an RTC5 PCIe Board and its software proceed as with an RTC5 PCI Board, see [Chapter 5 "Installation and Start-Up", Page 77](#).

Notice!

- The RTC5 PCIe Board does not support power-saving modes that switch off power to the PCIe bus. Accordingly, you must disable standby or sleep modes of the operating system. Particularly disable the Windows sleep mode option (some Windows operating systems enable this option by default). See also [Section "Notes", Page 33](#).

16.4 Legal

16.4.1 EU Declaration of Conformity – RTC5 PCIe Board

Konformitätserklärung – Declaration of Conformity

EU-Konformitätserklärung

für das Produkt:

RTC4 (PCI); RTC4 PCI-Express; RTC5 (PCI); RTC5 PCI-Express; RTC6 PCI-Express

Der Hersteller

SCANLAB GmbH, Siemensstr. 2a, 82178 Puchheim, Deutschland

erklärt, dass das genannte Produkt die einschlägigen Harmonisierungsrechtsvorschriften der Union erfüllt:

- **2014/30/EU** - Richtlinie des EUROPÄISCHEN PARLAMENTS UND DES RATES zur Harmonisierung der Rechtsvorschriften der Mitgliedstaaten über die elektromagnetische Verträglichkeit (EMV-Richtlinie).
- **2011/65/EU** - Richtlinie des EUROPÄISCHEN PARLAMENTS UND DES RATES zur Beschränkung der Verwendung bestimmter gefährlicher Stoffe in Elektro- und Elektronikgeräten (RoHS-Richtlinie).

Folgende harmonisierte Normen wurden angewandt:

- **EN IEC 61000-6-2:2005** – Elektromagnetische Verträglichkeit (EMV) - Teil 6-2: Fachgrundnormen - Störfestigkeit für Industriebereiche
- **DIN EN 55011:2018-05** - Industrielle, wissenschaftliche und medizinische Geräte - Funkstörungen - Grenzwerte und Messverfahren
- **DIN EN IEC 63000:2019-05** - Technische Dokumentation zur Beurteilung von Elektro- und Elektronikgeräten hinsichtlich der Beschränkung gefährlicher Stoffe

Die alleinige Verantwortung für die Ausstellung dieser Konformitätserklärung trägt der Hersteller.

Unterzeichnet für und im Namen der SCANLAB GmbH.

EU Declaration of Conformity

for the product:

RTC4 (PCI); RTC4 PCI-Express; RTC5 (PCI); RTC5 PCI-Express; RTC6 PCI-Express

The manufacturer

SCANLAB GmbH, Siemensstr. 2a, 82178 Puchheim, Germany

declares that the product mentioned above complies with the relevant Union harmonization legislation:

- **2014/30/EU** - Directive of the EUROPEAN PARLIAMENT AND OF THE COUNCIL on the harmonisation of the laws of the Member States relating to electromagnetic compatibility (EMC Directive)
- **2011/65/EU** - Directive of the EUROPEAN PARLIAMENT AND OF THE COUNCIL on the restriction of the use of certain hazardous substances in electrical and electronic equipment (RoHS Directive).

The following harmonized standards have been applied:

- **EN IEC 61000-6-2:2005** - Electromagnetic compatibility (EMC) - Part 6-2: Generic standards - Immunity for industrial environments
- **DIN EN 55011:2018-05** - Industrial, scientific and medical equipment – Radio-frequency disturbance characteristics - Limits and methods of measurement
- **DIN EN IEC 63000:2019-05** - Technical documentation for the assessment of electrical and electronic products with respect to the restriction of hazardous substances

This declaration of conformity is issued under the sole responsibility of the manufacturer.

Signed for and on behalf of SCANLAB GmbH.

Puchheim, 26.02.2025



Christian Sonner, CTO

02/2025



16.4.2 Compliance with FCC Rules

The RTC5 PCIe Board has been tested and found to comply with the limits for a Class A digital device, pursuant to part 15 of the FCC rules.

These limits are designed to provide reasonable protection against harmful interference when the RTC5 PCIe Board is operated in a commercial environment. The RTC5 PCIe Board generates, uses, and can radiate radio frequency energy and, if not installed and used in accordance with this instruction manual, may cause harmful interference to radio communications. Operation of the RTC5 PCIe Board in a residential area is likely to cause harmful interference in which case the user is required to correct the interference at his own expense.

16.5 Technical Specifications

– RTC5 PCIe Board

System Requirements

Windows PC with PCI-Express bus interface

Operating system	Microsoft Windows 10, 8, 7 as 32-bit or 64-bit version
------------------	--

Dimensions

Length	160 mm
Height	104.5 mm

Interface to PC

PCI-Express x1, version 1.0

SPI/I2C Socket Connector

Connector	10-pin socket connector
-----------	-------------------------

McBSP interface	Like RTC5 PCI Board (see page 71)
-----------------	-----------------------------------

Analog input ports	ANALOG IN0, ANALOG IN1
--------------------	---------------------------

- Input voltage range 0 V...10 V
4 k Ω
- Input resistance 12 bit
- ADC resolution GND
- Reference

Other Connectors and Specifications

Identical to RTC5 PCI Board

17 Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards

- For usage, see **control_command**.
- Example:

```
control_command( Data = 0501H )
```

 that is, **Code_{HIGH}** = **05 (SetMode)** and **Code_{LOW}** = **01** requests that the iDRIVE scan system should change the data type to be transmitted to “actual position”. The iDRIVE scan system evaluates the request and – if successful – subsequently transmits the “actual position” to the RTC-control board (notation in this manual is “Signal 0501_H” in this case).
- The following reference is used in several RTC manuals and is thus generic.

Code_{HIGH} (hex)	
05	SetMode SetMode selects the data signal to be returned from the scan system for the specified axis. See also Chapter 8.1.2 “Configuring the Data Signal Transmission Behavior of the Scan System”, Page 199. Each Code_{LOW} parameter value corresponds to a particular data type. Default setting: Code_{LOW} = 00_H (XY2-100 status word). See also “On SetMode”, Page 788. See also “On SetMode, SetControlDefinitionMode, SetEchoMode and Store/RestoreTransmissionMode”, Page 788. iDRIVE scan systems with XY2-100 interface transmit a signed 16-bit-value. iDRIVE scan systems with SL2-100 interface transmit an signed 20-bit value. For example, excelliSCAN-scan heads. iDRIVE scan systems with XY2-100 Enhanced interface transmit a signed 16-bit-value. With these, the XY2-100 Converter (Accessory) converts (by multiplying by 16) the value to a signed 20-bit value. In this document: 0500_H; 0501_H; 0502_H; 0503_H; 0504_H; 0505_H; 0506_H; 0507_H; 0508_H; 0509_H; 0510_H; 0511_H; 0512_H; 0513_H; 0514_H; 0515_H; 0516_H; 0517_H; 0518_H; 0519_H; 051A_H; 051B_H; 051C_H; 051D_H; 051E_H; 051F_H; 0520_H; 0521_H; 0522_H; 0523_H; 0524_H; 0525_H; 0526_H; 0527_H; 0528_H; 0529_H; 052A_H; 052B_H; 052C_H; 052D_H; 052E_H; 052F_H; 0530_H; 0531_H; 0532_H; 0533_H; 0534_H; 0535_H; 0536_H; 0537_H; 0538_H; 0539_H; 053A_H; 053B_H; 053C_H; 053D_H; 053E_H; 053F_H; 0549_H; 055E_H; 0564_H; 0565_H; 0574_H;

Notice! Generic information. May be incomplete or even incorrect for your scan system.
 Always consult the dedicated scan system manual!
 Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	00	As of scan system firmware ≥ 2001 . XY2-100 status word		
		16-bit protocol	20-bit protocol	
		Bit #15 (MSB)	Bit #19 (MSB)	Like [Bit #7 Bit #11]. = 1: The scan system servo control is ready for input data ("PowerOK", actually corresponds to a "ServoOK"). For scan systems with sensors as well as scan systems with interlock: there is also no interlock error.
		Bit #14	Bit #18	Like [Bit #6 Bit #10]. = 1: The system temperature within optimal range ("TempOK"). For scan systems with sensors: there is also no warning.
		Bit #13	Bit #17	= 1.
		Bit #12	Bit #16	Like [Bit #4 Bit #8]. = 1: The y axis position errors are within allowed range (PosAck signal).
		Bit #11	Bit #15	Like [Bit #3 Bit #7]. = 1: The x axis position errors are within allowed range (PosAck signal).
		Bit #10	Bit #14	Like [Bit #2 Bit #6]. = 1. Only intelliWELD: Reserved. For scan systems with sensors for automatic self calibration: Reserved.
		Bit #9	Bit #13	= 0.
		Bit #8	Bit #12	= 1.
		Bit #7	Bit #11	Like [Bit #15 Bit #19].
		Bit #6	Bit #10	Like [Bit #14 Bit #18].
		Bit #5	Bit #9	= 1.
		Bit #4	Bit #8	Like [Bit #12 Bit #16].
		Bit #3	Bit #7	Like [Bit #11 Bit #15].
		Bit #2	Bit #6	Like [Bit #10 Bit #14].
		Bit #1	Bit #5	= 0.
		Bit #0	Bit #4	= 1.
		–	Bit #3	= 0.
		–	Bit #2	= 0.
		–	Bit #1	= 0.
		–	Bit #0	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	01	As of scan system firmware $\geq$ 2001.		
		Actual position (angular position of galvanometer scanners)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In bits. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In bits. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	02	As of scan system firmware $\geq$ 2001.		
		Set position (angular position of galvanometer scanners)		
		As of scan system firmware $\geq$ 5050.		
		With excelliSCAN scan heads: Precalculated set position (angular position of galvanometer scanners). Refer to "excelliSCAN Scan Heads – Functional Principle of SCANahead Servo Control and Operation by RTC6 Boards" Manual, Chapter 2.1.6 "SCANahead System Timing", Page 23 and Figure 8.		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In bits. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In bits. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	03	As of scan system firmware ≥ 2001 . Position error (= set position 0502 _H – actual position 0501 _H) As of scan system firmware ≥ 5050 . With excelliSCAN scan heads: Trajectory error. See Glossary entry.		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In bits. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In bits. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	04	As of scan system firmware ≥ 2001 . Actual current (output stage current)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In mA. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In 16^{-1} mA. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	05	As of scan system firmware ≥ 2001 . Relative galvanometer scanner control Not intellicube.		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In per mille. Value range with 16-bit protocol: $[-1,000...+1,000]$. Unit with 20-bit protocol: In 16^{-1} per mille. Value range with 20-bit protocol: $[-16,000...+16,000]$. The return value 16 entspricht 1‰.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	06	As of scan system firmware ≥ 2001 . Actual velocity (angular velocity)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In Bit/ms. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In Bit/ms. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	07	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	08	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	09	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US



Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	10	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0 ...	Bit #0 ...	–
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	11	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0 ...	Bit #0 ...	–
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	12	As of scan system firmware $\geq$ 2001. Only intelliWELD with varioSCAN de FCi. Not: intelliWELD with varioSCAN FC: Actual z axis position Can only be read out from the channel of the scan head connector to which the z axis is connected.		
		16-bit protocol	20-bit protocol	
		Bit #0	Bit #0	Unit with 16-bit protocol: In bits.
		...	...	Value range with 16-bit protocol: [-32.768...+32.767].
		Bit #15	Bit #19	Unit with 20-bit protocol: In bits. Value range with 20-bit protocol: -524.288...+524.287.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	13	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	14	As of scan system firmware $\geq$ 2001. Temperature of galvanometer scanner		
		16-bit protocol	20-bit protocol	
		Bit #0	Bit #0	Unit with 16-bit protocol: In 10^{-1} °C.
		...	...	Value range with 16-bit protocol: [-32.768...+32.767].
		Bit #15	Bit #19	Unit with 20-bit protocol: In 10^{-1} °C. Value range with 20-bit protocol: -524.288...+524.287.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	15	As of scan system firmware ≥ 2001 . Servo board temperature As of scan system firmware ≥ 5050 . Axis 1 (X-axis): Temperature of the servo board Axis 2 (Y-axis): Temperature of the servo add-on board for evaluating the encoder signals		
		16-bit protocol	20-bit protocol	
		Bit #0	Bit #0	Unit with 16-bit protocol: In 10^{-1} °C.
		...	...	Value range with 16-bit protocol: $[-32.768...+32.767]$.
		Bit #15	Bit #19	Unit with 20-bit protocol: In 160^{-1} °C. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	16	As of scan system firmware ≥ 2001 . AGC voltage (position detector supply voltage)		
		16-bit protocol	20-bit protocol	
		Bit #0	Bit #0	Unit with 16-bit protocol: In 10^{-2} V.
		...	...	Value range with 16-bit protocol: $[-32.768...+32.767]$.
		Bit #15	Bit #19	Unit with 20-bit protocol: In 1600^{-1} V. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	17	As of scan system firmware $\geq$ 2001.		
		DSP core supply voltage		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In 10^{-2} V. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In 1600^{-1} V. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	18	As of scan system firmware $\geq$ 2001.		
		DSP IO voltage		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In 10^{-2} V. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In 1600^{-1} V. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	19	As of scan system firmware ≥ 2001 .		
		Analog section voltage		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In 10^{-2} V. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In 1600^{-1} V. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	1A	As of scan system firmware ≥ 2001 .		
		Analog-digital converter supply voltage		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In 10^{-2} V. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In 1600^{-1} V. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	1B	As of scan system firmware $\geq$ 2001. AGC current (PD supply current)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	Unit with 16-bit protocol: In mA. Value range with 16-bit protocol: $[-32.768...+32.767]$. Unit with 20-bit protocol: In 16^{-1} mA. Value range with 20-bit protocol: $-524.288...+524.287$.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	1C	As of scan system firmware $\geq$ 2066. Hardware version of servo board		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	16-bit protocol: 2nnn = DSCB 5021 = ISB1 5031 = ISB2 6011 = microISB 20-bit protocol: $2nnn \times 16$ = DSCB 5021×16 = ISB1 5031×16 = ISB2 6011×16 = microISB

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	1D	As of scan system firmware $\geq$ 2001.		
		Relevant for scan systems with galvanometer scanner heating.		
		Relative heating output of the corresponding galvanometer scanner heater		
		Not relevant for intellicube.		
		Not relevant for dynAXIS XS.		
		Not relevant for scan systems with water-cooled galvanometer scanners.		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In per mille. Value range with 16-bit protocol: [0...1,000]. Unit with 20-bit protocol: In per mille. Value range with 20-bit protocol: [0...1,000].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	1E	As of scan system firmware $\geq$ 2001.		
		Serial number, lower 16 bits		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: None. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: None. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	1F	As of scan system firmware $\geq$ 2001.		
		Serial number, upper 16 bits		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: None. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: None. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	20	As of scan system firmware $\geq$ 2001.		
		SCANLAB article number, lower 16 bits		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: None. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: None. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	21	As of scan system firmware $\geq$ 2001.		
		SCANLAB article number, upper 16 bits		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: None. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: None. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	22	As of scan system firmware $\geq$ 2001.		
		Version number of scan system firmware		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: None. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: None. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	23	As of scan system firmware ≥ 2001 .		
		Calibration angle		
		Mechanical scan angle ($\pm$) for 96% of the maximum or minimum control value. With 16-bit protocol ± 31457 bit. With 20-bit protocol ± 503316 bit.		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In 10^{-3} degrees. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: In 10^{-3} degrees. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	24	As of scan system firmware ≥ 2001 .		
		Aperture		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In mm. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: In mm. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	25	As of scan system firmware $\geq$ 2001.		
		Wavelength		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In nm. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: In nm. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	26	As of scan system firmware $\geq$ 2078.		
		Tuning number		
		16-bit protocol	20-bit protocol	
		Bit #8 ... Bit #15	Bit #12 ... Bit #19	Start setting.
		Bit #0 ... Bit #7	Bit #4 ... Bit #11	Current setting.
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	27	As of scan system firmware ≥ 2078 .		
		Only after <code>control_command(Data = 17FF_H)</code> : number of the temporarily stored data type		
		16-bit protocol	20-bit protocol	
		Bit #8	Bit #12	= 0.
		...	...	
		Bit #15	Bit #19	
		Bit #0	Bit #4	Cached setting.
		...	...	
		Bit #7	Bit #11	
		–	Bit #0	= 0.
			...	
			Bit #3	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
 Always consult the dedicated scan system manual!
 Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	28	As of scan-system firmware $\geq$ 2060.		
		Diagnostic flags, lower 16 bits		
		16-bit protocol	20-bit protocol	–
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	–

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	29	As of scan-system firmware $\geq$ 2060.		
		Diagnostic flags, upper 16 bits		
		16-bit protocol	20-bit protocol	–
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	–

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	2A	As of scan system firmware ≥ 2001 .		
		Stop event code		
		Value range with 16-bit protocol: [0...65.535].		
		Value range with 20-bit protocol: [0...65.535].		
		16-bit protocol	20-bit protocol	
		(hex)	(hex)	
		0001	00010	The galvanometer scanner has reached a critical edge position.
		0002	00020	AD converter error.
		0003	00030	Temperature in scan system above max. allowed value.
		0004	00040	External power supply voltages have dropped below the allowed value.
		0005	00050	Flags are not valid.
		0006	00060	Reserved.
		...	...	...
		000C	000C0	Reserved.
		000D	000D0	Reserved.
		000E	000E0	Position Acknowledge time out (set position not reached for long time).
		000F	000F0	Reserved.
		0014	00140	Reserved.
		0015	00150	Reserved.
		0016	00160	Reserved.
		0017	00170	Reserved.
		0018	00180	Reserved.
		0019	00190	Reserved.
		001A	001A0	Reserved.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	2B	As of scan-system firmware $\geq$ 2060.		
		Flags from last stop event, lower 16 bits		
		16-bit protocol	20-bit protocol	–
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	–

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	2C	As of scan-system firmware $\geq$ 2060.		
		Flags from last stop event, upper 16 bits		
		16-bit protocol	20-bit protocol	–
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	–

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US



Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	2D	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	2E	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	2F	As of scan system firmware $\geq$ 2061.		
		Running time (seconds)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In s. Value range with 16-bit protocol: [0...59]. Unit with 20-bit protocol: In s. Value range with 20-bit protocol: [0...59].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	30	As of scan system firmware $\geq$ 2061.		
		Running time (minutes)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In min. Value range with 16-bit protocol: [0...59]. Unit with 20-bit protocol: In min. Value range with 20-bit protocol: [0...59].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	31	As of scan system firmware ≥ 2061 .		
		Running time (hours)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In h. Value range with 16-bit protocol: [0...23]. Unit with 20-bit protocol: In h. Value range with 20-bit protocol: [0...23].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	32	As of scan system firmware ≥ 2061 .		
		Running time (days)		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In d. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: In d. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	33	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: 3.3 V sensor board operating voltage		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: $\ln 10^{-3}$ V. Value range with 16-bit protocol: [0...7,000]. Unit with 20-bit protocol: $\ln 10^{-3}$ V. Value range with 20-bit protocol: [0...7,000].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	34	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: Sensor board operating temperature		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: $\ln 0.1$ °C. Value range with 16-bit protocol: [0...65.535]. Unit with 20-bit protocol: $\ln 0.1$ °C. Value range with 20-bit protocol: [0...65.535].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	35	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: Purge gas flow rate		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: [(purge gas flow rate – 4 l/min) * 109,89 min/l]. Value range with 16-bit protocol: [0...2,250]. Unit with 20-bit protocol: [(purge gas flow rate – 4 l/min) * 109,89 min/l]. Value range with 20-bit protocol: [0...2,250].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	36	As of scan system firmware $\geq 206x$.		
		[Temperature of mirror 2 + 26.6 °C]		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: ln 0.05 °C. Value range with 16-bit protocol: [0...1,992]. Unit with 20-bit protocol: ln 0.05 °C. Value range with 20-bit protocol: [0...1,992].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	37	As of scan system firmware $\geq 206x$. [Temperature of mirror 1 + 26.6 °C]		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In 0.05 °C. Value range with 16-bit protocol: [0...1,992]. Unit with 20-bit protocol: In 0.05 °C. Value range with 20-bit protocol: [0...1,992].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	38	As of scan system firmware $\geq 206x$. Only intelliWELD: Protective-window temperature		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In 0,044 °C. Value range with 16-bit protocol: [0...2,250]. Unit with 20-bit protocol: In 0,044 °C. Value range with 20-bit protocol: [0...2,250].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	39	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: Collimator temperature		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In 0.05 °C. Value range with 16-bit protocol: [0...2,250]. Unit with 20-bit protocol: In 0.05 °C. Value range with 20-bit protocol: [0...2,250].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	3A	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: Galvanometer scanner mount temperature		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: In 0.05 °C. Value range with 16-bit protocol: [0...2,250]. Unit with 20-bit protocol: In 0.05 °C. Value range with 20-bit protocol: [0...2,250].
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	3B	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: [Coolant-flow rate + $0.93 \text{ l} \times \text{min}^{-1}$]		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: $\ln 0,00525 \text{ l} \times \text{min}^{-1}$. Value range with 16-bit protocol: [0...2,250]. Unit with 20-bit protocol: [0...2,250]. Value range with 20-bit protocol: $\ln 0,00525 \text{ l} \times \text{min}^{-1}$.
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	3C	As of scan system firmware $\geq 206x$.		
		Only intelliWELD: Protective-window scattered light value		
		16-bit protocol	20-bit protocol	
		Bit #0 ... Bit #15	Bit #4 ... Bit #19	Unit with 16-bit protocol: $\ln 0.00444 \text{ V}$. Value range with 16-bit protocol: [0...2,250]. Unit with 20-bit protocol: [0...2,250]. Value range with 20-bit protocol: $\ln 0.00444 \text{ V}$.
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system			
05	3D	As of scan system firmware ≥ 206x. Only intelliWELD: Flags sensor board Value range with 16-bit protocol: [0...65,535]. Value range with 20-bit protocol: [0...65,535].			
		16-bit protocol	20-bit protocol		
		Bit #15 (MSB)	Bit #19 (MSB)	= 1:	Protective-window temperature value not in [0...1,882].
		Bit #14	Bit #18	= 1:	Temperature of mirror 1 value not in [0...2,250].
		Bit #13	Bit #17	= 1:	Temperature of mirror 2 value not in [0...2,250].
		Bit #10	Bit #14	= 1:	Emerging-beam-opening temperature value not in [0...1,200]. Additionally, an INTERLOCK error is initiated.
		Bit #3	Bit #7	= 1:	Protective-window scattered light value not in [0...2,250].
		Bit #2	Bit #6	= 1:	Coolant-flow rate value not in [750...2,250].
		Bit #1	Bit #5	= 1:	Galvanometer-mount temperature value not in [0...1,600]. Additionally, an INTERLOCK error is initiated.
		Bit #0	Bit #4	= 1:	Collimator temperature value not in [0...2,000]. Additionally, an INTERLOCK error is initiated.
		–	Bit #0 ... Bit #3	= 0. ... = 0.	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	3E	Reserved.		
		16-bit protocol	20-bit protocol	–
		Bit #0 ...	Bit #0 ...	–
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	3F	As of scan system firmware ≥ 2072 . Set scaling factor for position values		
		16-bit protocol	20-bit protocol	
		Bit #2 ...	Bit #6 ...	Reserved.
		Bit #15	Bit #19	
		Bit #0 ...	Bit #4 ...	Scaling factor = $1/2^n$. 16-bit protocol $n = 2 \times \text{Bit \#1} + \text{Bit \#0}$. 20-bit protocol $n = 2 \times \text{Bit \#5} + \text{Bit \#4}$.
		Bit #1	Bit #5	
		–	Bit #0 ... Bit #3	= 0.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	49	As of scan system firmware $\geq$ 5000.		
		Supply voltage (usually 30 V or 48 V)		
		16-bit protocol	20-bit protocol	–
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	–

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	5E	As of scan system firmware $\geq$ 5000.		
		Number of transitions “regular operation → fault condition”		
		16-bit protocol	20-bit protocol	–
		Bit #0 ... Bit #15	Bit #0 ... Bit #19	–

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system
05	64	As of scan system firmware ≥ 5100 . Refer to RTC6 Manual, Chapter 8.13.3 "Advanced Settings for "Spot Distance Control"" , Page 317: TimeShift relative to the SCANLAB default offset.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system
05	65	As of scan system firmware ≥ 5100 . Refer to RTC6 Manual, Chapter 8.13.3 "Advanced Settings for "Spot Distance Control"" , Page 317: TimeShift in total.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	Code _{LOW} (hex)	Data type transmitted by the scan system		
05	74	As of scan system firmware $\geq 51*7 + < 5090$. For Return Channel Multiplexing . For use in conjunction with Code _{HIGH} (hex) = 65 _H and Code _{HIGH} (hex) = 66 _H . See corresponding Chapter incl. example code.		
		16-bit protocol	20-bit protocol	–
		Bit #0	Bit #0	–
		...	...	
		Bit #15	Bit #19	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev.1.0.6 en-US

Code _{HIGH} (hex)					
0A	As of scan system firmware ≥ 2078. UpdatePermanentMemory UpdatePermanentMemory causes the scan system to set the current servo behavior as the startup behavior for subsequent restarts or resets. See also Chapter 8.1.8 "Configuring the Start Behavior", Page 208. As of scan system firmware ≥ 5050. Only excelliSCAN: excelliSCAN scan heads do not support UpdatePermanentMemory.				
	<table> <tr> <th>Code_{LOW} (hex)</th><th></th></tr> <tr> <td>00</td><td>Only allowed value.</td></tr> </table>	Code _{LOW} (hex)		00	Only allowed value.
Code _{LOW} (hex)					
00	Only allowed value.				

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)			
0E	As of scan system firmware $\geq 206x$. SetControlDefinitionMode SetControlDefinitionMode causes the scan system to transmit information about a specific tuning (or the corresponding control algorithm) over the selected status channel. See also “On SetMode, SetControlDefinitionMode, SetEchoMode and Store/RestoreTransmissionMode”, Page 788.		
	Code _{LOW} (hex)	Data type transmitted by the scan system	
	00	As of scan system firmware $\geq 206x$.	
	01	Tuning number	
	02	16-bit protocol	20-bit protocol
	03	Bit #15 (MSB)	Bit #19 (MSB)
			= 0: Tuning available. = 1: Tuning not available.
		Bit #4 ...	Bit #8 ...
		Bit #14	Bit #18
		Bit #0 ...	Bit #4 ...
		Bit #3	Bit #7
			Tuning type = 0: Vector tuning. = 1: Jump tuning. = 2: Reserved. = 3: Reserved. = 4: As of scan system firmware ≥ 5050 . SCANahead servo control.
		–	Bit #0 ...
			Bit #3

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)											
11	As of scan system firmware ≥ 2078. SelectControlDefinition Only scan systems equipped with several tunings (or corresponding servo algorithms). SelectControlDefinition causes the scan systems to switch to a certain tuning. See also Chapter 8.1.4 "Configuring Tuning (Dynamics Settings)", Page 202. Code_{LOW} = 00...03 specifies the tuning number. As of scan system firmware ≥ 5050. Only excelliSCAN: excelliSCAN scan heads do not support SelectControlDefinition. With excelliSCAN scan heads, there is only one tuning. <table> <tr> <th>Code_{LOW} (hex)</th><th></th></tr> <tr> <td>00</td><td>Tuning 0 (= default setting)</td></tr> <tr> <td>01</td><td>Tuning 1</td></tr> <tr> <td>02</td><td>Tuning 2</td></tr> <tr> <td>03</td><td>Tuning 3</td></tr> </table>	Code _{LOW} (hex)		00	Tuning 0 (= default setting)	01	Tuning 1	02	Tuning 2	03	Tuning 3
Code _{LOW} (hex)											
00	Tuning 0 (= default setting)										
01	Tuning 1										
02	Tuning 2										
03	Tuning 3										

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)													
12	As of scan system firmware ≥ 2072. SetPositionScale SetPositionScale causes the scan system to switch to a certain scaling factor. The scan system servo electronics multiplies the actual position values received from the RTC-control board by this scaling factor. The actual position values sent back to the RTC-control board by the scan system are automatically divided by the same scaling factor. In this way, set position and actual position are always scaled to match each other. Important: You cannot detect a scaling factor $\neq 1$ by comparing set position and actual position. See also Chapter 8.1.7 "Configuring the Effective Calibration", Page 207. Important: The scaling factor needs to be changed in 2 steps: 1. Execute SetPositionScale with Code_{LOW} = 83_H. 2. Execute SetPositionScale with Code_{LOW} = [00_H...03_H] (= the desired scaling factor). See also "On SetPositionScale", Page 789. As of scan system firmware ≥ 5050. Only SCANAhead systems: SCANAhead systems do not support SetPositionScale. With SCANAhead systems, the effective calibration cannot be changed. <table> <tr> <th>Code_{LOW} (hex)</th><th></th></tr> <tr> <td>00</td><td>The scaling factor is set to the value 1/1 (= no scaling; default setting)</td></tr> <tr> <td>01</td><td>The scaling factor is set to the value 1/2</td></tr> <tr> <td>02</td><td>The scaling factor is set to the value 1/4</td></tr> <tr> <td>03</td><td>The scaling factor is set to the value 1/8</td></tr> <tr> <td>83</td><td>Make ready for scaling factor change.</td></tr> </table>	Code _{LOW} (hex)		00	The scaling factor is set to the value 1/1 (= no scaling; default setting)	01	The scaling factor is set to the value 1/2	02	The scaling factor is set to the value 1/4	03	The scaling factor is set to the value 1/8	83	Make ready for scaling factor change.
Code _{LOW} (hex)													
00	The scaling factor is set to the value 1/1 (= no scaling; default setting)												
01	The scaling factor is set to the value 1/2												
02	The scaling factor is set to the value 1/4												
03	The scaling factor is set to the value 1/8												
83	Make ready for scaling factor change.												

Notice! Generic information. May be incomplete or even incorrect for your scan system.
 Always consult the dedicated scan system manual!
 Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)	
15	As of scan system firmware ≥ 2066. SetPosAcknowledgeLevel SetPosAcknowledgeLevel causes the scan system to switch to a certain PosAck limit value. See also Chapter 8.1.6 "Configuring the PosAck Limit Value", Page 207. See also "On SetPosAcknowledgeLevel", Page 789.
Code _{LOW} (hex)	
00	Desired PosAck limit value in LSB16 , 16-bit protocol. In bits.
...	
FF	

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	
17	As of scan system firmware ≥ 2078. Store/RestoreTransmissionMode Store/RestoreTransmissionMode causes with Code_{LOW} = FF_H the scan system to cache the data type currently selected for transmission. It can be reinstated at a later time then with Code_{LOW} = 00_H. See also Data = 0527_H. See also "On Store/RestoreTransmissionMode", Page 789.
Code _{LOW} (hex)	
FF	Store temporarily
00	Reinstate

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	
21	As of scan system firmware ≥ 2078. SetEchoMode SetEchoMode verifies whether data transfer is intact. See also Chapter 8.1.9 "Fault Diagnosis and Functional Test", Page 208. SetEchoMode passes an 8-bit value to the scan system with Code_{LOW}. As a consequence, the scan system transmits a 16-bit value (16-bit protocol) or 20-bit value (20-bit protocol) on the corresponding status channel to the RTC-control board. If the data transmission has been error-free, with this: • The upper 8 bits and Code_{LOW} are identical • With the 16-bit value, the lower 8 bits / with the 20-bit value, the next lower 8 bits and the complementary value of Code_{LOW} (NOT Code_{LOW}) are identical Example with 16-bit protocol: For <code>control_command(1, 1, 0x210A)</code> and if data transfer is error-free, <code>(get_value(1) AND 0xFFFF)</code> returns 0x0AF5. Example with 20-bit protocol: For <code>control_command(1, 1, 0x210A)</code> and if data transfer is error-free, <code>(get_value(1) AND 0xFFFF0)</code> returns 0x0AF50. See also "On SetMode, SetControlDefinitionMode, SetEchoMode and Store/RestoreTransmissionMode", Page 788.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)					
40	As of scan system firmware $\geq 5102 + \geq 5112$. Only excelliSCAN: ResetTrAck ResetTrAck causes the scan system to reset the TrAck signal bits. Refer to "excelliSCAN Scan Heads – Functional Principle of SCANahead Servo Control and Operation by RTC6 Boards" Manual, Chapter 2.1.3 "TrAck Signal", Page 19. For more information, for example, about configuring the TrAck limit value, see the corresponding scan system manual. Note: The TrAck limit value cannot be saved for subsequent new starts or resets. excelliSCAN scan heads do not support UpdatePermanentMemory.				
	<table> <tr> <th>Code_{LOW} (hex)</th><th></th></tr> <tr> <td>3A</td><td> Only excelliSCAN: Reset TrAck signal bits. </td></tr> </table>	Code _{LOW} (hex)		3A	Only excelliSCAN: Reset TrAck signal bits.
Code _{LOW} (hex)					
3A	Only excelliSCAN: Reset TrAck signal bits.				

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)	
50	As of scan system firmware ≥ 5100. Only SCANahead systems: Refer to RTC6 Manual, Chapter 8.13.3 "Advanced Settings for "Spot Distance Control"", Page 317: dT_{high}.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)	
51	As of scan system firmware ≥ 5100. Only SCANahead systems: Refer to RTC6 Manual, Chapter 8.13.3 "Advanced Settings for "Spot Distance Control"", Page 317: dT_{low}.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!

Module Rev. 1.0.6 en-US

Code _{HIGH} (hex)	
65	As of scan system firmware $\geq 51 \times 7 + < 5090$. For Return Channel Multiplexing. For use in conjunction with Code_{HIGH} (hex) = 66_H and 0574_H. See corresponding Chapter incl. example code.
Code _{LOW} (hex)	
0N	Sets Return Channel Multiplexing block length to N+1 that is, highest Return Channel Multiplexing index = N. In other words: number of Return Channel Multiplexing values the iDRIVE scan system has to transmit. Allowed values: 1; 2; 3; 4; 6; 8; 12; 16 (= integer divisor of $192 \geq 16$).
8N	Sets the next to-be-configured Return Channel Multiplexing index to N. In other words: Defines the position where the signal is transmitted.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Code _{HIGH} (hex)	
66	As of scan system firmware $\geq 51 \times 7 + < 5090$. For Return Channel Multiplexing. For use in conjunction with Code_{HIGH} (hex) = 65_H and 0574_H. See corresponding Chapter incl. example code.
Code _{LOW} (hex)	
NN	Sets the return value for the current index to NN = writes the signal value to this index. For NN, you can insert the same Code_{LOW} values as for SetMode, see Code_{LOW} = 00_H (XY2-100 status word), Code_{LOW} = 01_H (Actual position (angular position of galvanometer scanners)) etc.

Notice! Generic information. May be incomplete or even incorrect for your scan system.
Always consult the dedicated scan system manual!
Module Rev.1.0.6 en-US

Change Index		
Module Rev.1.0.5 en-US	0536 _H	Not only with intelliWELD.
Module Rev.1.0.5 en-US	0537 _H	Not only with intelliWELD.
Module Rev.1.0.6 en-US	0500 _H	[Bit #15 Bit #19]. Editorial enhancement.
Module Rev.1.0.6 en-US	0500 _H	[Bit #14 Bit #18]. Editorial enhancement.

Comments (RTC5)

- The scan system firmware a.bb.c is read as abbc, for example, 5100 corresponds to version 5.10.0, and 5054 to version 5.05.4.
- On **SetMode**, **SetControlDefinitionMode**, **SetEchoMode** and **Store/RestoreTransmissionMode**
 - The data type selected by the `control_command(CodeHIGH = 05H, 0EH or 21H or Data = 1700H)` is transmitted until another data type is selected.
 - Data returned from the scan system to the RTC5 can be queried by **get_value**, **get_values**, **set_trigger/set_trigger4** and **get_waveform**. The first data after switching the data source are transmitted only after a short time delay due to the serial transmission times. Therefore, a delay time of up to 60 s may occur between the switchover time and the first output of the new data type. **control_command** always automatically inserts a waiting time of 60 μ s after the data source is switched (so that the previously mentioned commands now always return correct values).
 - All data returned from the scan system to the RTC5 are passed over as signed 20-bit values. This applies even if the **RTC5 DLL** is set to **RTC4 Compatibility Mode** and even for scan systems without SL2-100 interface, controlled by an **XY2-100 Converter (Accessory)**. Queried data returned by **get_value**, **get_values** or **get_waveform** are nevertheless generally transferred to the PC as signed 32-bit values (for data evaluation, see comments for **get_value**).
 - For scan systems with integrated SL2-100 interface, **get_head_status** queries the **XY2-100 status word** regardless of settings made by `control_command(CodeHIGH = 05H, 0EH or 21H or Data = 1700H)`. It is returned in addition to the selected data. In contrast, if **iDRIVE** scan systems (*without* an integrated SL2-100 interface) are controlled by an **XY2-100 Converter (Accessory)**, **get_head_status** returns the **XY2-100 status word** only if this signal has been previously selected to be returned from the scan system by **control_command** (see also [Section “Reading Out Data”, Page 199](#)).
- After a reset or power-up of the scan system, it can take around 5 seconds for data to be returned from the scan system (see also **get_value**). After a reset or power-up of the scan system, always the **XY2-100 status word** is returned.
- On **SetMode**
 - Set position (angular position) values returned by the scan system correspond to the effective output values `Sample<AX..BY>_Out` with **set_trigger/set_trigger4**. Additionally, the 20-bit set position values returned from a z axis (and in **RTC4 Compatibility Mode** from the x axis and y axis, too) are scaled by a factor 16 relating to the therefore specified 16-bit coordinate values.
 - The angle bit-values (actual position, set position, position error) or angle bit/ms-values (actual speed) returned to the RTC5 can be converted into °-values or °/ms-values by the scan system's calibration angle. The calibration angle can be read out by `Data = 0523H`.
 - Exact values for the internal voltages referred to in the table can vary for different versions of the scan system.
 - **intelliWELD** scan systems can be supplied with various number of sensors. Therefore, observe the manual of the respective scan system. This manual lists the command codes for currently available sensors.

- On **SetPositionScale**
 - If the scaling factor for scaling the position values is changed, then the user program calibration factor for calculating RTC5 control values (in bits) from the desired **Image field** position values (in mm) (see **Chapter 7.3.2 “Image Field Size and Image Field Calibration”, Page 156**) needs to be subjected to an inverse proportional scaling. In addition, the jump speed and mark speed should be adjusted.
 - The currently set scaling factor can be queried with `Data = 053FH`.
- On **SetPosAcknowledgeLevel**
 - The limit value must be specified related to a 16-bit position range.
 - The default start behavior is for the scan system to set the limit value to 0.28% of the *full* position range (16 bit, `CodeLOW = 183 = B7H`) after every power-up or reset.
If other limit values are desired, they must be separately set for each axis (`CodeHIGH = 15H`). SCANLAB recommends setting only limit values (`CodeLOW`) > 14_H (that is, 0.03% of the full position range). Lower values can lead to frequent system safety shutdowns due to Position Acknowledge time outs (set position not reached for an excessive time).
- On **Store/RestoreTransmissionMode**
 - `control_command( Data = 1700H )` has no effect if a power-up or reset was executed after the most recent execution of `control_command( Data = 17FFH )`.



18 Appendix C: Change Index

The following are changes in this manual due to the technical evolution of the product as well as significant editorial changes.

In this Chapter:

- Changes to Document Revision 1.9 en-US from Document Revision 1.8 en-US, page 791
- Changes to Document Revision 1.10 en-US from Document Revision 1.9 en-US, page 793
- Changes to Document Revision 1.11 en-US from Document Revision 1.10 en-US, page 795
- Changes to Document Revision 1.12 en-US from Document Revision 1.11 en-US, page 796
- Changes to Document Revision 1.13 en-US from Document Revision 1.12 en-US, page 797
- Changes to Document Revision 1.14.2 en-US from Document Revision 1.13 en-US, page 798
- Changes to Document Revision 1.14.3 en-US from Document Revision 1.14.2 en-US, page 800
- Changes to Document Revision 1.14.4 en-US from Document Revision 1.14.3 en-US, page 800
- Changes to Document Revision 1.14.5 en-US from Document Revision 1.14.4 en-US, page 801
- Changes to Document Revision 1.14.6 en-US from Document Revision 1.14.5 en-US, page 801
- Changes to Document Revision 1.14.7 en-US from Document Revision 1.14.6 en-US, page 802
- Changes to Document Revision 1.14.8 en-US from Document Revision 1.14.7 en-US, page 802
- Changes to Document Revision 1.14.9 en-US from Document Revision 1.14.8 en-US, page 803
- Changes to Document Revision 1.15.0 en-US from Document Revision 1.14.9 en-US, page 804
- Changes to Document Revision 1.15.1 en-US from Document Revision 1.15.0 en-US, page 805
- Changes to Document Revision 1.15.2 en-US from Document Revision 1.15.1 en-US, page 806

Changes to Document Revision 1.9 en-US from Document Revision 1.8 en-US

Where	Why & What
Chapter 7.5.2 "Marking Serial Numbers", Page 196	Expanded (as of DLL 537, OUT 537): Serial number marking now with selectable serial-number-sets.
Chapter 8.4 "Wobbel Mode", Page 217	Expanded: specify direction of motion and "freely definable wobbel shapes".
Chapter 8.4.1 "Wobbel Shapes – Important Notes on Choosing Appropriate Parameter Values", Page 219	Added.
Section "Correction via McBSP Interface with Enhanced McBSP Input", Page 231	Processing-on-the-fly correction in z direction and laser power variation is now possible.
<code>clear_fly_overflow</code> , Page 309	Changed with version DLL 531, OUT 532: bits #4, 5.
<code>config_laser_signals_list</code> , Page 312	Added (as of DLL 537, OUT 537).
<code>fly_return_z</code> , Page 324	Added (as of DLL 531, OUT 532).
<code>get_list_serial</code> , Page 346	Added (as of DLL 537, OUT 537).
<code>if_fly_z_overflow</code> , Page 379	Added (as of DLL 531, OUT 532).
<code>if_not_fly_z_overflow</code> , Page 383	Added (as of DLL 531, OUT 532).
<code>load_program_file</code> , Page 430	Expanded (as of DLL 537, OUT 537): PCI error now has its own error code 16.
<code>range_checking</code> , Page 487	Added (as of DLL 537, OUT 537).
<code>read_multi_mcbasp</code> , Page 496	Added (as of DLL 537, OUT 537).
<code>select_serial_set</code> , Page 518	Added (as of DLL 537, OUT 537).
<code>select_serial_set_list</code> , Page 518	Added (as of DLL 537, OUT 537).
<code>set_serial_step_list</code> , Page 616	Added (as of DLL 537, OUT 537).
<code>set_fly_limits_z</code> , Page 547	Added (as of DLL 531, OUT 532).
<code>set_multi_mcbasp_in</code> , Page 594	Added (as of DLL 537, OUT 537).
<code>set_multi_mcbasp_in_list</code> , Page 596	Added (as of DLL 537, OUT 537).
<code>set_sky_writing_para</code> , Page 621	Changed parameter <code>Timelag</code> (as of DLL 537, OUT 537).

Where	Why & What
set_wobbel_control , Page 649	Added (as of DLL 537, OUT 537).
set_wobbel_direction , Page 650	Added (as of DLL 537, OUT 537).
set_wobbel_mode , Page 651	Changed with version DLL 537, OUT 537: <i>Modes</i> > 0.
set_wobbel_offset , Page 653	Added (as of DLL 537, OUT 537).
set_wobbel_vector , Page 654	Added (as of DLL 537, OUT 537).
Appendix C: Change Index , Page 790	

Changes to Document Revision 1.10 en-US from Document Revision 1.9 en-US

Where	Why & What
Chapter 2.8.8 "Extension Board "RTC5/6 varioSCAN FLEX Extension"", Page 39	Added.
Chapter 2.9 "Supplementary Software", Page 40	Expanded.
Chapter 4.6.6 "SPI / I2C Socket Connector", Page 71	Expanded (as of DLL 538, OUT 538): The SPI/I ² C socket connector now can also be initialized with SPI functionality: "SPI interface". "McBSP interface" is an established term since long times in this manual is extended with "/SPI" and used as such.
Section "Repeatedly Executed Calls", Page 103	Added.
Chapter 6.9.1 "Free Variables", Page 123	Changed: 8 free variables are possible as of DLL 539, OUT 539.
Section "Sky Writing Mode 3", Page 151	Expanded.
Figure 52, page 150	Corrected.
Section "Precalculation and Diagnosis", Page 171	Added.
Chapter 8.11 "Camming", Page 255	Added.
Chapter 9.4 "Periodical I/O Signals", Page 277	Added.
auto_cal, Page 301	Text added: error code 9, 90.
camming, Page 307	Added (as of DLL 512, OUT 511).
get_free_variable, Page 334	Changed (as of DLL 539, OUT 539): value range of parameter No increased.
get_galvo_controls, Page 335	Added (as of DLL 539, OUT 539).
get_head_status, Page 338	Expanded.
mcbsp_init_spi, Page 462	Added (as of DLL 538, OUT 538).
periodic_toggle, Page 484	Added (as of DLL 538, OUT 538).
periodic_toggle_list, Page 485	Added (as of DLL 538, OUT 538).
read_abc_from_file, Page 489	Added (as of DLL 538, OUT 538).
set_free_variable, Page 556	Changed (as of DLL 539, OUT 539): value range of parameter No increased.

Where	Why & What
set_trigger, Page 634	Changed (as of DLL 538, OUT 538): Signal1, Signal2 = 45 and 46.
set_trigger, Page 634	Changed (as of DLL 539, OUT 539): Signal1, Signal2 = 47...51.
set_wobbel_vector, Page 654	Corrected: formula for variation of the current laser power P within the wobbel shape section.
sub_call_abs_repeat, Page 677	Added (as of DLL 538, OUT 538).
sub_call_repeat, Page 678	Added (as of DLL 538, OUT 538).
write_abc_to_file, Page 715	Added (as of DLL 538, OUT 538).
write_hi_pos, Page 720	Added (as of DLL 538, OUT 538).
Chapter 15 "Technical Specifications – RTC5 PCI Board", Page 734	Section SPI/I ² C Connector: SPI configuration values added.
"Compliance with EU Directive for Electromagnetic Compatibility (EMC)"	Added.
Appendix C: Change Index, Page 790	

Changes to Document Revision 1.11 en-US from Document Revision 1.10 en-US

Where	Why & What
Chapter 2.3 "Delivered RTC5 Software Package", Page 27 and others	Expanded (Windows 10 is supported).
Section "Non-Indexed Subroutines", Page 101	Expanded.
Section "Example Code (Delphi)", Page 116	Added (identification of master/slave boards).
Section "Speed-Dependent Laser Control", Page 191	Expanded (note on set_auto_laser_control Mode = 6).
control_command , Page 315	Changed: formula for coolant-flow rate at 053B_H .
control_command , Page 315	Changed: formula for protective-window scattered light value at 053C .
list_call_abs_repeat , Page 400	Added (as of DLL 540, OUT 540).
list_call_repeat , Page 402	Added (as of DLL 540, OUT 540).
move_to , Page 467	Command description has been moved to the manual "Installation and Operation RTC Step Motor Extension for the RTC4 and RTC5 PC interface boards" (available in English only).
set_auto_laser_control , Page 522	Changed (as of DLL 540, OUT 540): Mode = 6.
stepper_enable , Page 667	Corrected: stepper_enable does not have a result.
Chapter 17.4 "Technical Specifications – RTC5 PCIe/104 Board"	Expanded (note on ANALOG OUT1 and ANALOG OUT2).
Appendix C: Change Index, Page 790	

Changes to Document Revision 1.12 en-US from Document Revision 1.11 en-US

Where	Why & What
Chapter 2.3 "Delivered RTC5 Software Package", Page 27	Expanded (the RTC5 software package now also contains files for CMake).
Chapter 5.3.2 "Exchange of RTC Boards and Update of RTC Board Driver", Page 79	Added. For further information, refer to the (updated as of version DLL 542) file ReadMe_ScanlabClassChecker.pdf .
Section "Coordinate Transformations in the Virtual Image Field", Page 158	Expanded (coordinate transformations in the virtual Image field are now also available with set_fly_x and/or set_fly_y ; as of version DLL 541, OUT 541).
Chapter 8.6.11 "Processing-on-the-fly Correction for the z Axis", Page 242	Added.
Chapter 8.12 "Time Measurements", Page 257	Added.
control_command , Page 315	Changed: value range decreased (from 2250 to 1992) for temperature of mirror 2 at 0536_H .
control_command , Page 315	Changed: value range decreased (from 2250 to 1992) for temperature of mirror 1 at 0537_H .
get_lap_time , Page 343	Added (as of DLL 541, OUT 541).
list_next , Page 409	Added (as of DLL 541, OUT 541).
set_fly_z , Page 555	Added (as of DLL 530, OUT 531).
set_pulse_picking_list , Page 610	Corrected: this command is an undelayed short list command.
set_trigger4 , Page 642	Corrected: this command is a delayed short list command.
stepper_disable_switch , Page 666	Added (as of DLL 542, OUT 542).
Appendix C: Change Index, Page 790	

Changes to Document Revision 1.13 en-US from Document Revision 1.12 en-US

Where	Why & What
Chapter 4.6.3 "EXTENSION 2 Socket Connector", Page 67	Corrected: signal at pin (26) is LASERON (not: NOT CONNECTED). Figure 24 .
Chapter 4.6.8 "Analog Inputs (Accessory)", Page 75	Expanded: The analog input values (ANALOG IN0 and ANALOG IN1 , see read_analog_in) can be recorded with signal 54 (see set_trigger or set_trigger4 ; as of version DLL 543, OUT 543, RBF 524).
Chapter 8.4 "Wobbel Mode", Page 217	Expanded: The present wobbel amplitude (from set_wobbel or set_wobbel_mode) can be recorded with signal 53 (see set_trigger or set_trigger4 ; as of version DLL 543, OUT 543, RBF 524).
Chapter 8.12 "Time Measurements", Page 257	Expanded: 32-bit "Timestamp Counter". Can be recorded with trigger signal 52 (see set_trigger or set_trigger4 ; as of version DLL 543, OUT 543, RBF 524).
Chapter 9.3.2 "Conditional Command Execution", Page 271	Expanded: control command set_pause_list_cond (as of DLL 543, OUT 543).
periodic_toggle , Page 484	Changed: endless toggling with $\text{Count} = 2^{32}-1$ (as of DLL 543, OUT 543).
periodic_toggle_list , Page 485	Changed: endless toggling with $\text{Count} = 2^{32}-1$ (as of DLL 543, OUT 543).
set_control_mode , Page 528	Expanded: Bit #4 (as of DLL 543, OUT 543).
set_control_mode_list , Page 530	Expanded: Bit #4 (as of DLL 543, OUT 543).
set_pause_list_cond , Page 601	Added (as of DLL 543, OUT 543).
set_trigger , Page 634	Changed: $\text{Signal1}, \text{Signal2} = 52 \dots 54$ (as of DLL 543, OUT 543, RBF 524).
set_trigger4 , Page 642	Changed: $\text{Signal1}, \text{Signal2} = 52 \dots 54$ (as of DLL 543, OUT 543, RBF 524).
Section "EXTENSION 2 Socket Connector", Page 736	Corrected: missing signal LASERON added.
Appendix C: Change Index, Page 790	

Changes to Document Revision 1.14.2 en-US from Document Revision 1.13 en-US

Where	Why & What
Global	Document Revision 1.14.2 en-US applies to RTC5 software package RTC5_Software_2020_11_13.zip ^(a)
Global	Editorial change. The previously used term "Online Positioning" (due to the introduction of "Global Online Positioning") now reads - where applicable - "Local Online Positioning", for example, in Chapter 8.3.1 "Local Online Positioning", Page 213.
Global	All information explicitly referring to DLL 537, OUT 537 and earlier has been removed from this document. However, this information can still be taken from RTC5_Software_RevisionHistory_<Date>.pdf.
Chapter 1 "About this Manual", Page 21	Software change.
Chapter 4.5.2 "XY2-100 Converter (Accessory)", Page 56	Corrected: SCANLAB recommends that cables for data transmission via XY2-100 should be less than 20 m long.
Chapter 4.5.3 "Data Cables (Accessories)", Page 58	Corrected: SCANLAB recommends that cables for data transmission via XY2-100 should be less than 20 m long.
Chapter 4.6.1 "LASER Connector", Page 60	Note for laserDESK users on external documents, page 60.
Chapter 8.2 "Coordinate Transformations", Page 209	Editorial enhancement. Notes on data recording, page 211.
Chapter 8.3.2 "Global Online Positioning", Page 216	Added.
activate_fly_2d_encoder, Page 293	Added (as of DLL 544, OUT 544).
activate_fly_xy_encoder, Page 294	Added (as of DLL 544, OUT 544).
control_command, Page 315	Editorial change. Description of "PowerOK" and "TempOK" has been changed. Furthermore, the description of 0528 _H , 0529 _H , 052B _H and 052C _H has been removed (these data signal types are not intended for users).
control_command, Page 315	Changed: in the formula for 053B _H , the value is now "0.00525 l × min ⁻¹ " (additional decimal place, before: "0.0052 l × min ⁻¹ ").
control_command, Page 315	Changed significance of 0535 _H for intelliWELD-systems delivered as of October 2018. For these, the previous meaning "emerging-beam-opening temperature" no longer applies. The new meaning is now "purge air flow rate".
get_startstop_info, Page 356	Changed: Bit #14, page 357 (as of DLL 546, OUT 546).
load_program_file, Page 430	Editorial enhancement. Links to the state of the various output ports inserted, page 431.

(a) See also RTC5_Software_RevisionHistory_<Date>.pdf.

Where	Why & What
number_of_correction_tables , Page 468	Editorial enhancement. Command has not been documented earlier due to the limited number of users.
set_defocus_offset , Page 532	Added (as of DLL 540, OUT 540).
set_defocus_list , Page 532	Editorial enhancement. New comment.
set_defocus_offset_list , Page 533	Added (as of DLL 540, OUT 540).
set_mcbasp_global_matrix , Page 581	Added (as of DLL 545, OUT 545).
set_mcbasp_global_matrix_list , Page 581	Added (as of DLL 545, OUT 545).
set_mcbasp_global_rot , Page 582	Added (as of DLL 545, OUT 545).
set_mcbasp_global_rot_list , Page 582	Added (as of DLL 545, OUT 545).
set_mcbasp_global_x , Page 583	Added (as of DLL 545, OUT 545).
set_mcbasp_global_x_list , Page 583	Added (as of DLL 545, OUT 545).
set_mcbasp_global_y , Page 584	Added (as of DLL 545, OUT 545).
set_mcbasp_global_y_list , Page 584	Added (as of DLL 545, OUT 545).
set_pause_list_cond , Page 601	A conditional pause_list now takes precedence over a simultaneously present /STOP signal (as of DLL 544, OUT 544).
set_pause_list_not_cond , Page 602	Added (as of DLL 544, OUT 544).
set_port_default_list , Page 608	Added (as of DLL 544, OUT 544).
wait_for_encoder_mode , Page 711	Corrected: This command is not a normal list command but a multiple list command.
Chapter 15 "Technical Specifications – RTC5 PCI Board", Page 734	Added: HIGH level and LOW level of /START and /STOP.
"Compliance with EU Directive for Electromagnetic Compatibility (EMC)"	Editorial change. EU Directive 2014/30/EU.
"Compliance with EU Directive for Electromagnetic Compatibility (EMC)"	Editorial change. EU Directive 2014/30/EU.
Chapter 1.3 "Glossary and Abbreviations", Page 23	Added.
Appendix C: Change Index, Page 790	

Changes to Document Revision **1.14.3 en-US** from Document Revision **1.14.2 en-US**

Where	Why & What
Global	Document Revision 1.14.3 en-US applies to RTC5 software package - RTC5_Software_2020_11_13.zip^(a)
Chapter 14.1 "EU Declaration of Conformity – RTC5 PCI Board", Page 732	Editorial change. Replaces Chapter "Compliance with EC Directive for Electromagnetic Compatibility (EMC)".
Chapter 16.4.1 "EU Declaration of Conformity – RTC5 PCIe Board", Page 744	Editorial change. Replaces Chapter "Compliance with EC Directive for Electromagnetic Compatibility (EMC)".
Appendix C: Change Index, Page 790	

(a) See also [RTC5_Software_RevisionHistory_<Date>.pdf](#).

Changes to Document Revision **1.14.4 en-US** from Document Revision **1.14.3 en-US**

Where	Why & What
Global	Document Revision 1.14.4 en-US applies to RTC5 software package - RTC5_Software_2020_11_13.zip^(a)
Chapter 2.10 "Notes for RTC4 Users", Page 40	Editorial enhancement. Encoder counter direction of RTC4 boards, see Notice! , Page 49.
list_repeat , Page 410	Editorial enhancement. New comment, page 410 .
clear_fly_overflow , Page 309	Editorial change. Mode = 63 (not: = 15).
control_command , Page 315	Editorial change. Content moved to Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747.
Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747	Editorial change. 052A_H : Stop Event Code 000D0 is not intended for users.
Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747	Editorial enhancement.
Appendix C: Change Index, Page 790	

(a) See also [RTC5_Software_RevisionHistory_<Date>.pdf](#).

Changes to Document Revision **1.14.5 en-US** from Document Revision **1.14.4 en-US**

Where	Why & What
Global	Document Revision • 1.14.5 en-US applies to RTC5 software package • RTC5_Software_2021_10_22.zip^(a)
Chapter 1 "About this Manual", Page 21	Software change.
clear_fly_overflow_ctrl, Page 309	Added (as of DLL 548, OUT 548).
Appendix C: Change Index, Page 790	

(a) See also [RTC5_Software_RevisionHistory_<Date>.pdf](#).

Changes to Document Revision **1.14.6 en-US** from Document Revision **1.14.5 en-US**

Where	Why & What
Global	Document Revision • 1.14.6 en-US applies to RTC5 software package • RTC5_Software_2021_10_22.zip^(a)
Chapter 2.8.6 "Slot Cover with 15-pin D-SUB Connector and 9-pin D-SUB Connector", Page 39	Editorial enhancement. #0130209.
Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747	Editorial change. Module Rev.1.0.6 en-US.
Appendix C: Change Index, Page 790	

(a) See also [RTC5_Software_RevisionHistory_<Date>.pdf](#).

Changes to Document Revision **1.14.7 en-US** from Document Revision **1.14.6 en-US**

Where	Why & What
Global	Document Revision • 1.14.7 en-US applies to RTC5 software package • RTC5_Software_2021_12_30.zip^(a)
Chapter 1 "About this Manual", Page 21	Software change.
set_multi_mcbasp_in, Page 594	Software change. Mode = 2.
Appendix C: Change Index, Page 790	

(a) See also RTC5_Software_RevisionHistory_<Date>.pdf.

Changes to Document Revision **1.14.8 en-US** from Document Revision **1.14.7 en-US**

Where	Why & What
Global	Document Revision • 1.14.8 en-US applies to RTC5 software package • RTC5_Software_2022_01_28.zip^(a)
Chapter 1 "About this Manual", Page 21	Software change.
Chapter 7.3.4 "3D Image Field", page 159	Editorial enhancement. ABC values from the *_ReadMe.txt file of correction file are to be interpreted as 16-bit values.
set_mcbasp_out_ptr_list, Page 590	Software change. ^(a) New command.
set_multi_mcbasp_in, Page 594	Software change. ^(a) Mode = 2 changed.
Appendix C: Change Index, Page 790	

(a) See also RTC5_Software_RevisionHistory_<Date>.pdf.



Changes to Document Revision **1.14.9 en-US** from Document Revision **1.14.8 en-US**

Where	Why & What
Global	Document Revision • 1.14.9 en-US applies to RTC5 software package • RTC5_Software_2022_03_09.zip^(a)
Chapter 1 "About this Manual", Page 21	Software change.
Appendix C: Change Index, Page 790	

(a) See also [RTC5_Software_RevisionHistory_<Date>.pdf](#).

Changes to Document Revision 1.15.0 en-US from Document Revision 1.14.9 en-US

Where	Why & What
Global	Document Revision 1.15.0 en-US applies to RTC5 software package - RTC5_Software_2022-11-11^(a)
Global	Software change. ^(a) Microsoft Vista, XP SP2, XP SP3 and earlier are no longer supported.
Global	RTC5 PC/104-Plus-Boards: Last support was 2023-12-31. Information removed.
Global	RTC5 PCIe/104-Boards: Last support was 2023-12-31. Information removed.
Chapter 1 "About this Manual", Page 21	Software change. ^(a)
Chapter 4.4 "Master Socket Connector, Slave Socket Connector", Page 53	Hardware change. #0149963 (previously: #0117241).
get_list_pointer, Page 345	Editorial enhancement. New comment, page 345.
set_auto_laser_params, Page 525	Editorial enhancement. Result.
set_laser_control, Page 566	Editorial enhancement. Further clarification with Bit #3 and Bit #4.
set_vector_control, Page 643	Software change. ^(a) Ctrl = 8.
wait_for_encoder_mode, Page 711	Editorial change. wait_for_encoder_mode is category "Normal List Command" (not: "Multiple List Command").
Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747	Editorial change. Module Rev.1.0.6 en-US.
Appendix C: Change Index, Page 790	

(a) See also RTC5_Software_RevisionHistory_<Date>.pdf.



Changes to Document Revision **1.15.1 en-US** from Document Revision **1.15.0 en-US**

Where	Why & What
Global	Document Revision • 1.15.1 en-US applies to RTC5 software package • RTC5_Software_2024-09-27^(a)
Chapter 1 "About this Manual", Page 21	Software change. ^(a)
get_marking_info , Page 347	Editorial change. Text of Bit #17 and Bit #18 was mixed up.
set_ellipse , Page 537	Software change. ^(a) See Version info , Page 537.
Chapter 17 "Appendix B: iDRIVE Scan Systems – Control Commands and Signals Transmitted to RTC Control Boards", Page 747	Editorial change. Module Rev.1.0.6 en-US.
Appendix C: Change Index, Page 790	

(a) See also [RTC5_Software_RevisionHistory_<Date>.pdf](#).

Changes to Document Revision **1.15.2 en-US** from Document Revision **1.15.1 en-US**

Where	Why & What
Global	Document Revision • 1.15.2 en-US applies to RTC5 software package • RTC5_Software_2024-09-27^(a)
Chapter 14.1 "EU Declaration of Conformity – RTC5 PCI Board", Page 732	Editorial change. New version. Date: 2025-02-26.
Chapter 16.4.1 "EU Declaration of Conformity – RTC5 PCIe Board", Page 744	Editorial change. New version. Date: 2025-02-26.
Appendix C: Change Index, Page 790	

(a) See also `RTC5_Software_RevisionHistory_<Date>.pdf`.